

RoboMaster

视觉从入门到入土

Misaka21

Turn drawings of imagination into racetrack realization.

June 25, 2025

Thank you for using this book ❤️,
I hope you like it 😊

目录

1. 基础篇	7
1.1. 初识 Linux	7
1.1.1. Linux 的前世今生	7
1.1.2. 为什么 RoboMaster 开发选择 Linux	9
1.1.3. Linux 与 Windows 的差异	12
1.1.4. Linux 发行版选择	17
1.1.5. 安装 Linux 的方式	17
1.2. 环境准备	17
1.3. Linux & Ubuntu 操作	17
1.3.1. 图形界面初探	17
1.3.2. 终端与 Shell 基础	17
1.3.3. 文件系统与目录结构	25
1.3.4. 文件与目录操作	32
1.3.5. 文本编辑与处理	44
1.3.6. 用户与权限管理	55
1.3.7. 软件包管理	64
1.3.8. 进程与系统监控	73
1.3.9. 网络与远程访问	84
1.3.10. Shell 脚本入门	92
1.3.11. 开发环境配置	106
1.4. 计算机系统基础	118
1.4.1. 为什么需要了解计算机系统	118
1.4.2. 程序的执行：从源码到运行	118
1.4.3. 进程：程序的运行实例	118
1.4.4. 线程：轻量级执行单元	118
1.4.5. 内存层次结构	118
1.4.6. 虚拟内存	118
1.4.7. 进程的内存布局	118
1.4.8. 文件系统	118
1.4.9. 网络通信基础	118
1.4.10. 并发与同步	118
1.4.11. 操作系统的角色	118
1.4.12. 性能分析与调试工具	118
1.5. C++ 语言基础	118
1.5.1. C++ 基本程序结构	118
1.5.2. 编译一个 C++ 程序	120
1.5.3. 变量与基本数据类型	121
1.5.4. 运算符与表达式	126
1.5.5. 控制语句	132
1.5.6. 函数基础	139
1.5.7. 数组与字符串	146
1.5.8. 指针基础	153

1.5.9. 引用	160
1.5.10. 结构体	167
1.5.11. 类与对象	172
1.5.12. 构造函数与析构函数	182
1.5.13. 拷贝控制	195
1.5.14. 类的继承	204
1.5.15. 多态与虚函数	216
1.5.16. 运算符重载	227
1.5.17. STL 容器	241
1.5.18. STL 算法与迭代器	252
1.5.19. 模板基础	264
1.5.20. 智能指针	275
1.5.21. 右值引用与移动语义	290
1.5.22. Lambda 表达式	301
1.5.23. 多线程基础	313
1.5.24. 文件操作	326
1.5.25. Eigen 矩阵库	343
1.5.26. Ceres 非线性优化库	355
1.5.27. OpenCV 基础	370
1.5.28. 串口通信	370
1.6. CMake 与构建系统	370
1.6.1. 为什么需要构建系统	370
1.6.2. 编译原理回顾	373
1.6.3. CMake 基础语法	380
1.6.4. 构建流程实践	397
1.6.5. 目标与属性	405
1.6.6. 多文件项目组织	416
1.6.7. 查找与链接外部库	434
1.6.8. 安装与导出	446
1.6.9. ROS 2 包的构建	456
1.6.10. 常见问题与调试	472
1.7. 软件工程基础	481
1.7.1. 为什么需要软件工程	481
1.7.2. 代码规范与风格	484
1.7.3. 设计模式的起源	495
1.7.4. 设计模式的基本原则	495
1.7.5. 常用设计模式详解	496
1.7.6. 单元测试	496
1.7.7. 调试技巧	509
1.7.8. 性能分析与优化	509
1.7.9. 文档编写	509
1.7.10. 版本控制与协作（可选/扩展）	522
1.8. Git 版本控制	526

1.8.1.	Git 基本概念	526
1.8.2.	本地仓库操作	528
1.8.3.	GitHub/GitLab 远程协作	529
1.8.4.	分支管理	531
1.8.5.	团队工作流 (PR、Code Review)	532
1.8.6.	.gitignore 与 submodule	533
1.8.7.	小结: 把 Git 当作工程系统而不是命令集合	534
1.9.	ROS/ROS2 入门	534
1.9.1.	ROS/ROS2 架构与概念	534
1.9.2.	话题、服务、动作	535
1.9.3.	创建节点与发布订阅	536
1.9.4.	launch 文件	539
1.9.5.	ROS 工具 (rqt、rviz2)	540
1.9.6.	实战: 简单的视觉/决策节点	541
1.9.7.	小结: 先把系统边界建立清楚, 再追求算法上限	542
2.	数学理论篇	543
2.1.	计算机视觉基础	543
2.2.	传统视觉算法	543
2.3.	深度学习和神经网络	543
2.4.	相机模型	543
2.5.	坐标变换	543
2.6.	卡尔曼滤波	543
2.6.1.	什么是卡尔曼滤波?	543
2.6.2.	如何准确地测量体重	543
2.6.3.	从体重秤到赛车: 静态估计的局限性	545
2.6.4.	最优增益的推导	548
2.6.5.	多维卡尔曼滤波	552
2.6.6.	扩展卡尔曼滤波	558
2.6.7.	无迹卡尔曼滤波	563
3.	实战技术篇	570
3.1.	通信协议设定	570
3.2.	相机标定与手眼标定	570
3.3.	时间戳对齐	570
3.4.	弹道解算	570
4.	RoboMaster 应用篇	571
4.1.	工业相机	571
4.2.	装甲板识别	571
4.3.	装甲板的跟踪	571
4.4.	能量机关识别	571
4.5.	自瞄算法设计	571
4.6.	串口模块	571
5.	进阶篇	572
5.1.	雷达站视觉方案	572

5.2. 哨兵决策视觉	572
5.3. 性能优化技巧	572
5.4. 实战经验与坑点总结	572
6. 项目分析	573
6.1. rm_vision	573
6.2. rm.cv.fans	573
6.3. 同济自瞄	573
Index of Tables	574

1. 基础篇

本书主要针对基础篇的内容。

1.1. 初识 Linux

1.1.1. Linux 的前世今生

在开始学习 Linux 的具体操作之前，了解它的历史背景会帮助你更好地理解这个系统的设计理念。Linux 不是凭空出现的，它继承了 Unix 数十年的智慧，融合了自由软件运动的理想，最终在全球开发者的协作下成长为今天这个无处不在的操作系统。这段历史不仅仅是技术的演进，更是一群人对“软件应该如何被创造和分享”这个问题的回答。

1.1.1.1. Unix：一切的起源

故事要从 1969 年说起。那一年，人类首次登上月球，而在美国新泽西州的贝尔实验室，两位计算机科学家 Ken Thompson 和 Dennis Ritchie 正在做一件同样具有开创性的事情——他们在一台被闲置的 PDP-7 小型机上，开发一个全新的操作系统。这个系统后来被命名为 Unix。

Unix 的诞生带有几分偶然。在此之前，贝尔实验室参与了一个名为 Multics 的大型操作系统项目，但这个项目因为过于复杂而进展缓慢，贝尔实验室最终选择退出。Thompson 和 Ritchie 对 Multics 的一些理念仍然着迷，于是决定自己动手，用更简洁的方式实现类似的功能。Unix 这个名字本身就是对 Multics 的一种调侃——Multics 意为“多路复用”，而 Unix 则暗示“单一”，讽刺 Multics 的过度复杂。

Unix 的设计体现了一种独特的哲学，这种哲学至今仍深刻影响着软件开发的方式。其核心理念可以概括为几个原则：每个程序只做一件事，并把它做好；程序之间通过文本流协作，一个程序的输出可以作为另一个程序的输入；尽早编写原型，快速迭代改进。这种“小而美”的设计思想与当时流行的“大而全”的系统形成鲜明对比。当你在 Linux 终端中用管道符 `|` 将多个命令串联起来时，你正在使用的就是 Unix 哲学的直接体现。

1972 年，Dennis Ritchie 完成了另一项影响深远的工作——他发明了 C 语言，并用 C 语言重写了 Unix。这使得 Unix 成为第一个主要用高级语言编写的操作系统，大大提高了它的可移植性。在此之前，操作系统通常用汇编语言编写，与特定硬件紧密绑定。C 语言的出现让 Unix 可以相对容易地移植到不同的计算机上，这为它后来的广泛传播奠定了基础。

贝尔实验室隶属于 AT&T 公司，当时 AT&T 受到反垄断法规的限制，不能进入计算机业务，因此以极低的成本将 Unix 授权给大学和研究机构。这使得 Unix 在学术界迅速流行起来。加州大学伯克利分校的学生和研究人员在 Unix 的基础上进行了大量改进和扩展，形成了著名的 BSD (Berkeley Software Distribution) 版本。许多我们今天仍在使用的网络功能，包括 TCP/IP 协议栈的实现，都源自 BSD。

然而，好景不长。1984 年，AT&T 被拆分后获准进入计算机市场，Unix 开始商业化，授权费用大幅上涨。与此同时，各个公司和机构开发的 Unix 变体开始分化，互不兼容。这个时期出现了许多商业 Unix 系统：Sun 的 Solaris、IBM 的 AIX、HP 的 HP-UX 等。Unix 世界陷入了碎片化和专利诉讼的泥潭，曾经开放共享的精神逐渐消失。

1.1.1.2. GNU 与自由软件运动

在 Unix 走向封闭的同时，一位 MIT 人工智能实验室的程序员决定反抗这一趋势。他的名字是 Richard Stallman，人们通常称他为 RMS。

Stallman 是一位极具原则性的人。在他看来，软件应该是自由的——用户应该有权运行、研究、修改和分发软件。商业公司将软件封闭起来、禁止用户查看和修改源代码的做法，在他眼中是对用户自由的侵犯。他常常讲述一个故事：实验室的打印机经常卡纸，如果能看到驱动程序的源代码，他可以轻松添加一个卡纸通知功能；但新打印机的驱动程序是闭源的，他无能为力。这种“被软件控制而无法控制软件”的无力感，促使他下定决心做出改变。

1983 年，Stallman 发起了 GNU 计划。GNU 是“GNU’s Not Unix”的递归缩写——一个典型的黑客式幽默。这个计划的目标是创建一个完全自由的、与 Unix 兼容的操作系统。所谓“自由”，Stallman 定义了四项基本自由：自由运行程序、自由研究程序如何工作（需要源代码）、自由分发程序的副本、自由改进程序并发布改进版本。注意，这里的“自由”（free as in freedom）不是“免费”（free as in free beer）的意思，尽管自由软件通常也是免费的。

1985 年，Stallman 创立了自由软件基金会（Free Software Foundation, FSF），并发明了 GPL（GNU General Public License）许可证。GPL 的核心条款是“copyleft”——如果你分发基于 GPL 软件的衍生作品，你必须以同样的 GPL 条款发布，包括提供源代码。这确保了自由软件的自由能够传递下去，不会被人拿去变成专有软件。

在接下来的几年里，GNU 计划开发了大量高质量的软件工具：GCC 编译器、GDB 调试器、Emacs 编辑器、GNU Make、Bash Shell，以及众多的命令行工具（ls、cat、grep 等）。这些工具至今仍是 Linux 系统的核心组成部分，也是每个 RoboMaster 开发者每天都在使用的工具。你用 `g++` 编译代码、用 `make` 构建项目、用 `bash` 编写脚本，使用的都是 GNU 软件。

然而，到了 1990 年代初，GNU 计划仍然缺少一个关键组件——操作系统内核。Stallman 团队正在开发一个名为 GNU Hurd 的内核，但由于采用了过于前卫的微内核架构，开发进度非常缓慢（事实上，Hurd 至今仍未完成）。GNU 拥有了构建完整操作系统所需的几乎所有工具，却唯独缺少让这些工具运行起来的内核。这就像拥有了所有的汽车零件，却没有发动机。

1.1.1.3. Linux 内核的诞生

1991 年 8 月 25 日，芬兰赫尔辛基大学的一名 21 岁学生在 Usenet 新闻组发布了一条消息：

```
Hello everybody out there using minix -  
I' m doing a (free) operating system (just a hobby, won' t be big and  
professional like gnu) for 386(486) AT clones.
```

这个学生的名字是 Linus Torvalds，他正在开发的这个“业余爱好项目”，就是后来的 Linux 内核。

Torvalds 开发 Linux 的初衷很简单：他想在自己新买的 386 电脑上运行 Unix，但商业 Unix 太贵，适合教学的 Minix 系统功能又太弱。于是他决定自己写一个。与 Stallman 的理想主义不同，Torvalds 的动机更加实用——他只是想要一个好用的系统。

Linux 的开发速度令人惊讶。Torvalds 将源代码发布在网上，邀请其他人参与改进。来自世界各地的程序员通过互联网协作，报告 bug、提交补丁、添加新功能。这种分布式协作开发的模式在当时是前所未有的。Torvalds 后来将 Linux 内核以 GPL 许可证发布，正式加入了自由软件的阵营。

Linux 内核与 GNU 工具的结合，终于形成了一个完整的自由操作系统。严格来说，我们日常使用的“Linux”应该称为“GNU/Linux”——它使用 Linux 内核，但用户空间的大部分工具来自 GNU 计划。Stallman 一直坚持这个称呼，以强调 GNU 项目的贡献，但在日常使用中，人们通常简称为“Linux”。

Linux 的成功有多重因素。技术上，它采用了成熟的单内核（monolithic kernel）架构，性能优异且稳定。许可证上，GPL 保证了代码的开放性，任何人都可以自由使用和修改。时机上，互联网的兴起让全球协作成为可能，也创造了对可靠服务器操作系统的巨大需求。社区上，Torvalds 的务实态度和技术领导力吸引了大量优秀的贡献者。

1.1.1.4. 从爱好项目到主导世界

如果 1991 年的 Torvalds 能够预见 Linux 的未来，他一定会惊讶不已。三十多年后的今天，Linux 已经成为世界上最成功的操作系统之一：

全球大多数的服务器运行着 Linux，包括支撑互联网运转的网站、云服务和数据中心。当你使用搜索引擎、观看在线视频或者发送消息时，你的请求几乎必然是由某个 Linux 服务器处理的。

世界上最快的 500 台超级计算机，100% 运行 Linux。科学研究、气象预测、人工智能训练，这些需要巨大计算能力的任务，都依赖于 Linux。

Android 系统基于 Linux 内核，这意味着全球数十亿部智能手机都在运行 Linux 的变体。

嵌入式领域同样是 Linux 的天下。从路由器、智能电视到汽车信息娱乐系统，从树莓派到 NVIDIA Jetson，Linux 无处不在。RoboMaster 机器人上常用的 Jetson 系列计算平台，运行的正是基于 Ubuntu 的 Linux 系统。

如今，Linux 内核的开发已经成为人类历史上最大规模的协作项目之一。数以千计的开发来自数百家公司，包括 Google、Microsoft、Intel、Red Hat、华为等，共同贡献代码。曾经对 Linux 嗤之以鼻的微软，现在不仅是内核的主要贡献者之一，还在 Windows 中集成了 WSL（Windows Subsystem for Linux），让用户可以在 Windows 上直接运行 Linux 程序。

Linux 的成功证明了开源协作模式的力量。一个芬兰大学生的业余项目，在全球开发者的共同努力下，成长为支撑现代数字世界的基础设施。当你开始学习 Linux，你不仅是在学习一个操作系统的使用方法，更是在加入一个拥有数十年历史、影响深远的技术传统。Unix 的设计哲学、GNU 的自由精神、全球社区的协作文化——这些都将在你的学习和实践中逐渐体现。

对于 RoboMaster 开发者来说，理解这段历史还有另一层意义。机器人开发所依赖的几乎所有核心工具——ROS、OpenCV、GCC、CMake、Git——都是开源软件，都植根于 Linux 和自由软件的传统。当你使用这些工具时，你是这个庞大开源生态的受益者；当你有朝一日贡献自己的代码时，你也将成为这个传统的一部分。

1.1.2. 为什么 RoboMaster 开发选择 Linux

如果你观察 RoboMaster 各战队的技术栈，会发现一个有趣的现象：几乎所有队伍的视觉系统、导航算法、仿真环境都运行在 Linux 上。这不是偶然的选择，也不是因为开发者们故意追求“极客感”。Linux 之所以成为机器人开发的主流平台，是多种因素共同作用的结果。理解这些原因，可以帮助你更好地认识 Linux 在机器人领域的地位，也能让你在学习过程中保持动力——毕竟，你学习的是一项在行业中真正被广泛使用的技能。

1.1.2.1. ROS 生态的天然选择

机器人操作系统 ROS（Robot Operating System）是机器人软件开发的事实标准。无论是学术研究还是工业应用，ROS 都占据着主导地位。RoboMaster 的许多队伍使用 ROS 2 来构建他们的软件系统，实现模块化的节点通信、坐标变换、传感器数据处理等功能。

ROS 从诞生之日起就是为 Linux 设计的。虽然 ROS 2 在技术上支持 Windows 和 macOS，但这种支持更多是“能用”而非“好用”的级别。许多 ROS 包只在 Linux 上经过充分测试，一

些底层功能（如实时调度、共享内存通信）在 Linux 上的实现也更加成熟。当你在 Windows 上遇到 ROS 相关的问题时，往往很难找到解决方案，因为社区里几乎所有人都在使用 Linux。选择 Linux，意味着你站在了最大的用户群体中，遇到问题时更容易获得帮助。

更重要的是，ROS 的整个工具链——catkin、colcon、rosdep、roslaunch——都是围绕 Linux 的命令行环境设计的。在 Linux 上，你可以用简单的几条命令完成工作空间的创建、依赖的安装、包的构建。这种流畅的开发体验是其他平台难以复制的。

1.1.2.2. 开源库的最佳支持

机器人开发离不开大量的开源库。OpenCV 处理图像，Eigen 进行矩阵运算，PCL 处理点云，Ceres 解决优化问题，g2o 构建图优化——这些库构成了机器人算法的基础设施。虽然这些库大多是跨平台的，但它们在 Linux 上的支持通常是最好的。

以 OpenCV 为例，在 Linux 上你可以通过包管理器一键安装，也可以从源码编译并启用各种优化选项（如 CUDA、OpenCL、TBB）。编译过程通常很顺利，因为库的开发者自己就在 Linux 上工作。而在 Windows 上，你可能需要折腾 Visual Studio 的版本兼容性、处理各种路径问题、手动配置环境变量。这些额外的麻烦会消耗大量时间，而这些时间本可以用来专注于算法本身。

Linux 的包管理系统也让依赖管理变得简单。`apt install libopencv-dev` 一条命令就能安装 OpenCV 及其所有依赖。在 ROS 2 中，`rosdep install` 可以自动解析并安装包的所有依赖。这种自动化的依赖管理在 Windows 上是很难实现的，你往往需要手动下载、配置每一个库。

1.1.2.3. 实时性与系统控制

机器人是一个实时系统，传感器数据需要及时处理，控制指令需要准时发送。虽然 RoboMaster 比赛对实时性的要求不如工业机器人那么严格，但低延迟和确定性的响应时间仍然很重要。一个视觉系统如果偶尔出现几十毫秒的卡顿，可能会错过移动目标。

Linux 在实时性方面有显著优势。通过 PREEMPT_RT 补丁，Linux 可以被配置为硬实时操作系统。即使不使用实时补丁，Linux 的调度器也提供了多种调度策略（如 SCHED_FIFO、SCHED_RR），允许关键线程获得更高的优先级。开发者可以精细控制内存锁定、CPU 亲和性、中断处理，这些在 Windows 上要么不可能，要么需要特殊的驱动和权限。

Linux 还提供了对系统底层的完全访问权限。你可以直接操作串口、GPIO、I2C、SPI 等硬件接口，不需要安装额外的驱动程序。当你需要调试性能问题时，perf、strace、ltrace 等工具可以帮助你深入理解程序的行为。这种透明度和可控性是机器人开发所需要的。

1.1.2.4. 嵌入式平台的统一

RoboMaster 机器人上运行代码的硬件往往不是普通的 PC。树莓派、NVIDIA Jetson、各种工控机——这些嵌入式平台几乎都运行 Linux。Jetson 系列使用基于 Ubuntu 的 JetPack 系统，树莓派官方系统 Raspberry Pi OS 基于 Debian，大多数工控机预装的也是 Linux 发行版。

在同一个操作系统家族中开发意味着代码的可移植性大大提高。你在自己的 Ubuntu 笔记本上开发和测试的代码，可以几乎不做修改地部署到 Jetson 上运行。环境配置、依赖安装、构建流程都是相似的。如果你在 Windows 上开发，然后部署到 Linux 嵌入式平台，就需要处理跨平台的各种差异，这增加了不必要的复杂性。

使用 Linux 进行开发还意味着你可以在本地模拟目标环境。通过 Docker 容器或虚拟机，你可以在开发机上创建一个与目标平台相同的环境，确保代码在部署前就经过充分测试。这种开发和部署环境的一致性是高效工作流程的基础。

1.1.2.5. 仿真与持续集成

现代机器人开发离不开仿真环境。Gazebo、Isaac Sim、Webots 等仿真器让你可以在虚拟环境中测试算法，不需要真实的机器人硬件。这对于 RoboMaster 开发尤其重要——你不可能 24 小时都能接触到机器人，但你可以随时在仿真环境中运行和调试代码。

这些仿真器在 Linux 上的支持是最好的。Gazebo 就是为 Linux 开发的，虽然也支持其他平台，但许多高级功能（如 ROS 集成、GPU 加速渲染）在 Linux 上最稳定。NVIDIA 的 Isaac Sim 虽然也支持 Windows，但官方文档和教程主要针对 Linux。

持续集成（CI）是软件工程的重要实践，机器人开发也不例外。GitHub Actions、GitLab CI 等 CI 平台的默认环境都是 Linux。当你的代码推送到仓库时，CI 系统会自动在 Linux 容器中构建和测试。如果你的项目只在 Windows 上能构建，就无法利用这些便利的自动化工具。

1.1.2.6. 开发工具链的成熟

C++ 是机器人开发的主要语言，而 Linux 提供了最成熟的 C++ 开发工具链。GCC 和 Clang 是两个主流的开源编译器，它们在 Linux 上的支持最为完善，对最新 C++ 标准的实现也最快。CMake 虽然是跨平台的，但它的设计哲学和 Linux 的命令行文化更加契合。

调试工具同样如此。GDB 是最强大的命令行调试器，在 Linux 上可以与各种 IDE 无缝集成。Valgrind 用于检测内存泄漏，AddressSanitizer 和 ThreadSanitizer 用于发现内存和线程错误，这些工具在 Linux 上的支持都是最好的。当你的机器人程序出现难以复现的崩溃或内存问题时，这些工具可能是你唯一的救星。

性能分析工具也是 Linux 的强项。perf 可以进行 CPU 性能分析，gperftools 可以进行堆内存分析，FlameGraph 可以生成直观的火焰图。这些工具帮助你找到程序的性能瓶颈，在资源受限的嵌入式平台上尤为重要。

1.1.2.7. 免费、开源与可定制

对于学生团队来说，成本是一个现实的考虑因素。Linux 是免费的，你可以在任意数量的机器上安装和使用它，不需要担心许可证问题。相比之下，Windows 的正版授权对于多台开发机和机器人平台来说是一笔不小的开支。

开源意味着透明。当你遇到系统层面的问题时，可以查看源代码，理解系统的行为，甚至修改它。这在调试底层问题时非常宝贵。你也可以根据需要定制系统，移除不需要的组件，优化启动时间和资源占用。在资源受限的嵌入式平台上，这种定制能力尤其重要。

开源社区的力量也不可忽视。Linux 有着庞大而活跃的社区，遇到问题时你可以在论坛、Stack Overflow、GitHub Issues 上找到帮助。许多问题已经有人遇到过并分享了解决方案。这种集体智慧是闭源系统无法比拟的。

1.1.2.8. 职业发展的投资

学习 Linux 不仅仅是为了 RoboMaster 比赛。从更长远的角度看，Linux 技能是软件工程师的必备素养。无论将来从事机器人开发、后端开发、嵌入式系统还是云计算，都会用到 Linux。服务器世界几乎完全由 Linux 主导，Android 系统基于 Linux 内核，大多数 IoT 设备运行 Linux。

在 RoboMaster 的开发过程中学习 Linux，是一种自然而高效的学习方式。你不是为了学习而学习，而是为了解决实际问题而学习。这种目标驱动的学习往往更加深入和持久。当你毕业时，你不仅有比赛经历，还有扎实的 Linux 技能，这在求职时是一个显著的优势。

综合来看，选择 Linux 进行 RoboMaster 开发不是一种偏好，而是一种务实的选择。它让你能够使用最好的工具、获得最广泛的支持、与目标平台保持一致，同时为未来的职业发展打下基础。接下来的章节将带你从零开始，逐步掌握 Linux 的使用和开发技能。不要担心学习曲线——每一个现在熟练使用 Linux 的开发者，都曾经是从完全不懂开始的。重要的是迈出第一步，然后持续学习和实践。

1.1.3. Linux 与 Windows 的差异

从 Windows 转向 Linux 的开发者往往会经历一段适应期。两个系统看起来都是操作系统，都能运行程序、管理文件，但它们的设计哲学和使用方式有着根本的不同。理解这些差异不仅能帮助你更快地适应 Linux，还能让你理解为什么 Linux 会以某种方式工作。本节将深入对比两个系统的核心差异，帮助你建立正确的心智模型。

1.1.3.1. 开源与闭源：根本的哲学分歧

Linux 和 Windows 最根本的区别在于它们的开发模式。Windows 是微软的商业产品，其源代码是保密的。你只能使用微软提供的功能，无法查看系统内部的工作原理，遇到问题时只能等待微软发布补丁。这种模式有其优点——统一的质量控制、商业支持、向后兼容性保证——但也意味着用户对系统的控制是有限的。

Linux 则完全不同。Linux 内核和大多数 Linux 软件都是开源的，任何人都可以查看、修改、分发源代码。这种开放性带来了几个重要的后果。首先，透明度意味着安全性。安全研究者可以审计代码，发现并修复漏洞，没有“安全靠隐藏”的问题。其次，你可以真正理解系统的工作原理。当程序出现问题时，你可以查看源代码，追踪问题的根源。第三，社区驱动的开发意味着软件会朝着用户需要的方向发展，而不是商业利益驱动的方向。

对于开发者来说，开源意味着学习的机会。你使用的每一个工具——编译器、调试器、库——都可以查看它的实现。这是一个巨大的学习资源，也是开源精神对技术社区的馈赠。

1.1.3.2. 文件系统：目录树与盘符

打开 Windows 的文件资源管理器，你会看到 C:、D:、E: 这样的盘符。每个物理磁盘或分区都有自己的盘符，文件路径以盘符开头，如 `C:\Users\Username\Documents\file.txt`。这种设计直观地反映了物理存储结构，但也带来了一些问题：程序需要知道文件在哪个盘符上，移动硬盘会改变盘符，不同机器上的路径可能不同。

Linux 采用了完全不同的设计：单一的目录树。整个文件系统从根目录 `/` 开始，所有文件和目录都是这棵树的一部分。没有盘符的概念，物理磁盘和分区通过“挂载”的方式接入到目录树的某个位置。例如，一个 USB 驱动器可能被挂载到 `/media/usb`，之后你就可以通过这个路径访问它的内容。文件路径看起来像 `/home/username/documents/file.txt`。

这种设计的优雅之处在于统一性。无论是本地硬盘、网络驱动器、USB 设备还是虚拟文件系统，它们都以相同的方式呈现在目录树中。程序不需要关心文件实际存储在哪个物理设备上，只需要知道路径。这种抽象让很多事情变得简单——你可以把一个慢速的本地目录替换成高速的网络存储，而程序完全不需要修改。

Linux 的目录结构遵循文件系统层次标准（FHS），主要目录各有用途：`/home` 存放用户文件，`/etc` 存放配置文件，`/usr` 存放系统程序和库，`/var` 存放可变数据如日志，`/tmp` 存

放临时文件。刚开始可能觉得复杂，但熟悉之后你会发现这种组织方式非常合理——你总是知道去哪里找什么。

1.1.3.3. 路径分隔符与大小写敏感

一个看似微小但经常引起问题的差异是路径分隔符。Windows 使用反斜杠 `\`，Linux 使用正斜杠 `/`。这个差异源于历史原因——早期 DOS 使用 `/` 作为命令行选项的前缀（类似 Linux 的 `-`），所以选择了 `\` 作为路径分隔符。

对于开发者来说，这个差异意味着你需要注意代码中的路径处理。硬编码 `\` 的路径在 Linux 上无法工作，反之亦然。好的做法是使用编程语言提供的路径处理函数（如 Python 的 `os.path.join` 或 C++ 的 `std::filesystem::path`），让程序自动处理路径分隔符的差异。在 CMake 中，无论在什么平台上，都应该使用正斜杠 `/`，CMake 会自动转换。

另一个重要差异是大小写敏感性。在 Windows 上，`File.txt`、`file.txt` 和 `FILE.TXT` 指向同一个文件。在 Linux 上，它们是三个完全不同的文件。这个差异经常让从 Windows 转来的开发者踩坑：代码中 `#include "MyHeader.h"` 在 Windows 上能工作，但如果实际文件名是 `myheader.h`，在 Linux 上就会编译失败。

大小写敏感性也影响命令和程序名。在 Linux 上，`ls` 和 `LS` 是不同的命令（后者通常不存在）。`make` 和 `Make` 是不同的程序。这种设计更加精确——你想要什么就得明确说什么——但也需要更加仔细。养成统一使用小写文件名的习惯可以避免很多问题。

1.1.3.4. 可执行文件与权限

在 Windows 上，可执行文件通过扩展名来识别：`.exe`、`.bat`、`.cmd`、`.msi` 等。双击一个 `.exe` 文件就会运行它。这种方式简单直观，但也带来了安全隐患——恶意软件可以伪装成其他类型的文件（如 `photo.jpg.exe`），利用 Windows 默认隐藏扩展名的设置欺骗用户。

Linux 采用了完全不同的方式：文件是否可执行由其权限决定，而不是扩展名。一个文件要能够执行，必须具有“可执行”权限位。你可以用 `chmod +x script.sh` 给文件添加可执行权限，用 `chmod -x program` 移除可执行权限。扩展名只是给人看的提示，对系统没有任何意义。一个没有扩展名的文件完全可以是可执行程序，一个叫 `data.txt` 的文件如果有可执行权限也可以运行（虽然内容必须是有效的程序或脚本）。

这种设计更加安全。文件不会因为有个扩展名就自动变得可执行。用户必须显式地赋予执行权限，这提供了一层额外的保护。同时，这种设计也更加灵活——可执行程序不需要遵循特定的命名约定。

Linux 的权限系统远比 Windows 复杂和强大。每个文件有三组权限：所有者（owner）、所属组（group）、其他人（others）。每组权限包括读（r）、写（w）、执行（x）三种。你会看到类似 `-rwxr-xr-x` 这样的权限字符串，表示所有者可以读写执行，组成员和其他人可以读和执行但不能写。理解和管理权限是 Linux 系统管理的基础技能，我们会在后面的章节详细介绍。

1.1.3.5. 软件安装：包管理器的革命

在 Windows 上安装软件，你需要找到软件的官网，下载安装程序，运行安装向导，一路点击“下一步”，可能还要处理各种捆绑软件的选项。卸载软件时，你要去控制面板找到程序，运行卸载程序，祈祷它能完全清理干净。更新软件？每个程序有自己的更新机制，有的会弹窗提示，有的需要手动检查。

Linux 的包管理器彻底改变了这一切。Ubuntu 使用 `apt`，Fedora 使用 `dnf`，Arch 使用 `pacman`——虽然命令不同，但理念相同：软件从中央仓库安装，依赖自动解决，更新一键完成。

```
1 # 安装软件 (Ubuntu/Debian)
2 sudo apt install vlc
3
4 # 安装多个软件
5 sudo apt install git cmake build-essential
6
7 # 更新所有软件
8 sudo apt update && sudo apt upgrade
9
10 # 卸载软件
11 sudo apt remove vlc
12
13 # 搜索软件
14 apt search video player
```

包管理器的优势是全方位的。首先，安装过程简单且一致。不需要去各个网站下载，不需要点击安装向导，一条命令就完成了。其次，依赖管理是自动的。如果你要安装的软件需要其他库，包管理器会自动安装所有依赖。第三，更新是集成的。一条命令就能更新系统上的所有软件，包括操作系统本身。第四，卸载是干净的。包管理器知道软件安装了哪些文件，可以完全移除它们。

这种模式也带来了安全性的提升。软件来自官方维护的仓库，经过了审核和签名验证。你不会不小心从钓鱼网站下载到带病毒的安装程序。当软件有安全更新时，只需要运行系统更新，所有软件都会被更新到安全版本。

当然，包管理器也有局限性。仓库中的软件可能不是最新版本（为了稳定性），某些专有软件可能不在仓库中。这时你可能需要添加第三方仓库、使用 Flatpak/Snap 等通用包格式，或者从源码编译。但对于大多数日常使用和开发需求，官方仓库中的软件已经足够。

1.1.3.6. 命令行文化

Windows 的设计哲学是“图形界面优先”。大多数操作都可以通过点击鼠标完成，命令行（cmd 或 PowerShell）虽然存在，但更多是给高级用户使用的。普通用户可能从来不需要打开命令行窗口。

Linux 的传统正好相反：命令行是核心，图形界面是可选的附加层。许多 Linux 服务器根本没有安装图形界面，管理员通过 SSH 远程登录，用命令行完成所有操作。即使在桌面 Linux 上，许多任务用命令行完成也比图形界面更高效。

这种差异不是偶然的。命令行有一些图形界面难以替代的优势。首先是自动化：你可以把一系列命令写成脚本，一键执行复杂的操作序列。其次是可重复性：命令可以被记录、分享、版本控制，别人可以精确复现你的操作。第三是远程操作：通过 SSH，你可以在家里管理机房里的服务器，只需要一个终端窗口和网络连接。第四是精确控制：命令行提供了图形界面不一定暴露的选项，让你可以精确控制程序的行为。

对于 RoboMaster 开发来说，命令行技能尤为重要。ROS 2 的工作空间管理、包构建、节点运行都主要通过命令行进行。Git 版本控制虽然有图形界面工具，但命令行提供了更完整的功能。调试程序时，GDB 命令行调试器是最强大的工具。在机器人上排查问题时，你通常只能通过 SSH 连接，只有命令行可用。

不要把命令行看作障碍，而要把它看作一种强大的工具。一旦熟练掌握，你会发现命令行让很多任务变得更快、更简单。而且，命令行技能是可迁移的——无论你使用什么 Linux 发行版，基本的命令都是相同的。

1.1.3.7. 系统配置：文本文件与注册表

在 Windows 上，系统和程序的配置大多存储在注册表（Registry）中。注册表是一个层次化的数据库，存储了从系统设置到应用程序偏好的各种配置。要查看或修改注册表，你需要使用专门的编辑器（regedit），而且必须非常小心——错误的修改可能导致系统无法启动。注册表的结构复杂，路径冗长，普通用户几乎不会直接与之交互。

Linux 采用了截然不同的方式：配置存储在普通的文本文件中。系统配置在 `/etc` 目录下，用户配置在用户主目录的隐藏文件（以 `.` 开头的文件）中。例如，网络配置在 `/etc/network/interfaces` 或 `/etc/netplan/` 下，SSH 服务器配置在 `/etc/ssh/sshd_config`，用户的 shell 配置在 `~/.bashrc`。

这种设计的优势是显而易见的。文本文件可以用任何文本编辑器查看和修改，不需要专门的工具。文件有清晰的注释说明每个选项的含义，学习配置语法比学习注册表结构容易得多。你可以轻松备份配置文件，用版本控制跟踪配置的变化，把配置从一台机器复制到另一台机器。当配置出错时，文件格式通常比注册表更容易恢复。

对于开发者来说，文本配置文件意味着你可以用脚本自动化配置过程。需要在多台机器上进行相同的配置？写一个脚本，把配置文件复制过去即可。需要根据环境动态生成配置？用模板引擎生成文本文件。这种灵活性在 Windows 的注册表模型下是很难实现的。

当然，文本配置文件也有缺点。格式不统一——有的是 INI 格式，有的是 YAML，有的是自定义格式。语法错误可能导致服务无法启动。配置分散在多个文件中，需要知道去哪里找。但总的来说，文本配置的优点远大于缺点，特别是对于需要精细控制系统的开发者和管理员。

1.1.3.8. “一切皆文件”的哲学

Unix/Linux 有一个著名的设计原则：“一切皆文件”（Everything is a file）。这不仅仅是一句口号，而是深刻影响系统设计核心理念。在 Linux 中，几乎所有东西都可以用文件的方式来访问和操作。

硬件设备是文件。磁盘在 `/dev/sda`，分区在 `/dev/sda1`，串口在 `/dev/ttyUSB0`，摄像头在 `/dev/video0`。你可以像读写普通文件一样与这些设备交互。想要从串口读取数据？`cat /dev/ttyUSB0`。想要向设备发送数据？`echo "command" > /dev/ttyUSB0`。这种统一的接口让设备操作变得简单和一致。

进程信息是文件。`/proc` 目录包含了系统中所有进程的信息。`/proc/1234/status` 显示进程 1234 的状态，`/proc/1234/cmdline` 显示它的命令行，`/proc/1234/fd/` 目录列出了它打开的所有文件描述符。系统信息也在这里：`/proc/cpuinfo` 显示 CPU 信息，`/proc/meminfo` 显示内存使用情况。监控工具如 `top` 和 `htop` 就是通过读取 `/proc` 中的文件来获取信息的。

系统参数是文件。`/sys` 目录提供了对内核参数的访问。你可以读取文件来查询系统状态，写入文件来修改参数。例如，`/sys/class/backlight/intel_backlight/brightness` 控制屏幕亮度，向它写入数值就可以调整亮度。

这种设计的美妙之处在于统一性。无论你要做什么——读取传感器数据、查询系统状态、配置内核参数——都使用相同的操作：打开文件、读取或写入、关闭文件。不需要学习各种专门的 API，标准的文件操作就足够了。这大大简化了系统编程，也让 shell 脚本能够完成复杂的系统管理任务。

对于 RoboMaster 开发来说，这个哲学有实际的应用。与串口通信？打开 `/dev/ttyUSB0` 文件，用标准的读写操作进行通信。读取相机帧？打开 `/dev/video0`，用 V4L2 API（但底层

还是文件操作)。监控系统资源？读取 `/proc` 和 `/sys` 中的文件。这种统一的模型让开发变得更加简单和一致。

1.1.3.9. 适应期的常见困惑

从 Windows 转向 Linux 时，有一些常见的困惑和问题。了解这些可以帮助你更快地适应。

“软件去哪里安装了？”这是新手常问的问题。在 Windows 上，程序通常在 `C:\Program Files` 或 `C:\Program Files (x86)`。在 Linux 上，通过包管理器安装的程序分散在多个目录中：可执行文件在 `/usr/bin`，库在 `/usr/lib`，配置在 `/etc`，文档在 `/usr/share/doc`。这种分散看起来混乱，但实际上是有组织的——相同类型的文件放在一起，而不是相同程序的文件放在一起。你不需要关心程序装在哪里，只需要知道运行它的命令。

“怎么运行程序？”在 Windows 上双击图标，在 Linux 上…取决于情况。如果程序在系统路径中，直接在终端输入命令即可。如果是当前目录下的程序，需要加 `./` 前缀，如 `./my_program`，这是一个安全特性，防止当前目录下的程序覆盖系统命令。如果程序没有执行权限，先用 `chmod +x` 添加权限。

“驱动程序在哪里下载？”大多数情况下，你不需要。Linux 内核包含了大量硬件的驱动程序，即所谓的“开箱即用”。插入 USB 设备、连接打印机，系统通常会自动识别。少数需要专门驱动的硬件（如 NVIDIA 显卡、某些 Wi-Fi 芯片），可以通过包管理器安装，或者使用厂商提供的安装脚本。总的来说，Linux 的硬件支持比很多人想象的要好。

“为什么没有 C 盘？”如前所述，Linux 不使用盘符。所有存储都挂载到目录树的某个位置。你的主目录 `/home/username` 就相当于 Windows 的 `C:\Users\Username`。外置驱动器通常挂载到 `/media` 或 `/mnt` 下。

“权限被拒绝是怎么回事？”Linux 的权限系统比 Windows 更严格。如果你试图修改系统文件或执行某些操作，可能需要管理员权限。使用 `sudo` 命令可以临时获得管理员权限，如 `sudo apt install package`。但不要养成什么都用 `sudo` 的习惯——权限限制是保护你的系统的。

“命令记不住怎么办？”几乎每个命令都有手册页（man page），用 `man command` 查看。例如 `man ls` 显示 `ls` 命令的详细文档。更快捷的方式是 `command --help`，显示简要的帮助信息。还有 `tldr` 这样的工具提供更友好的速查手册。记住最常用的十几个命令就足够日常使用了，其他的可以随时查阅。

这些困惑都是暂时的。随着使用经验的积累，Linux 的工作方式会变得越来越自然。许多人在适应之后甚至会觉得 Linux 的方式更加合理和高效。重要的是保持耐心和好奇心，不要因为一时的不习惯就放弃。

1.1.4. Linux 发行版选择

1.1.5. 安装 Linux 的方式

1.2. 环境准备

1.3. Linux & Ubuntu 操作

1.3.1. 图形界面初探

1.3.2. 终端与 Shell 基础

如果说 Linux 是一个王国，那么终端就是通往这个王国的大门。虽然现代 Linux 发行版都提供了精美的图形界面，但终端——那个黑色（或其他颜色）的窗口，闪烁着等待输入的光标——仍然是 Linux 的灵魂所在。通过终端，你可以用文字与系统对话，执行命令，管理文件，运行程序，完成几乎任何任务。对于 RoboMaster 开发者来说，终端更是日常工作的核心工具。本节将带你从零开始，理解终端的工作原理，掌握基本的使用技巧。

1.3.2.1. 终端、Shell 与 Bash

在深入使用之前，让我们先理清几个容易混淆的概念：终端 (Terminal)、Shell 和 Bash。

终端，准确地说是终端模拟器 (Terminal Emulator)，是一个图形程序，提供了一个窗口让你输入文字命令。在早期的计算机系统中，终端是一个物理设备——一个带键盘和显示器的硬件，通过电缆连接到主机。如今，这个物理设备被软件模拟了，但“终端”这个名字保留了下来。Ubuntu 默认的终端程序叫 GNOME Terminal，你也可以使用其他终端模拟器如 Konsole、Terminator、Alacritty 等，它们的核心功能是相同的。

Shell 是终端中实际运行的程序，负责解释你输入的命令并执行。当你在终端中输入 `ls` 并按回车，是 Shell 解析这个命令，找到 `ls` 程序，执行它，并把结果显示在终端上。Shell 是你和操作系统内核之间的翻译官，把你的意图转化为系统调用。

Bash (Bourne Again Shell) 是最常用的 Shell 程序。它是早期 Bourne Shell (sh) 的增强版，几乎所有 Linux 发行版都默认使用 Bash。除了 Bash，还有 Zsh、Fish、Dash 等其他 Shell，它们有各自的特点和语法差异，但基本概念是相通的。本教程主要以 Bash 为例，因为它是最通用的选择。

简单总结：终端是窗口，Shell 是在窗口中运行的程序，Bash 是最常用的 Shell 程序。当人们说“打开终端输入命令”时，实际上是打开终端模拟器，在其中运行的 Bash Shell 里输入命令。

1.3.2.2. 打开终端

在 Ubuntu 桌面环境中，最快捷的方式是按下 `Ctrl+Alt+T`，这会立即打开一个终端窗口。你也可以在应用程序菜单中搜索“Terminal”或“终端”来打开它。

打开终端后，你会看到一个提示符 (prompt)，类似于：

```
1 username@hostname:~$
```

这个提示符包含了几个信息：`username` 是当前登录的用户名，`hostname` 是计算机的名称，`~` 表示当前所在的目录（`~` 是用户主目录的简写），`$` 表示这是一个普通用户（如果是 root 用户，会显示 `#`）。提示符的格式是可以自定义的，不同的系统或配置可能看起来不同。

现在你可以输入命令了。试着输入 `echo "Hello, Linux!"` 并按回车：

```
1 echo "Hello, Linux!"
2 Hello, Linux!
```

恭喜，你刚刚执行了第一个 Linux 命令！`echo` 命令的作用是把后面的文字打印出来，这是最简单的命令之一，但它验证了你的终端正在正常工作。

1.3.2.3. 命令的格式

Linux 命令遵循一个通用的格式：

```
1 命令 [选项] [参数]
```

命令是你执行的程序名，选项 (options) 修改命令的行为，参数 (arguments) 是命令操作的对象。选项和参数都是可选的，具体取决于命令。

让我们以 `ls` 命令为例。`ls` 用于列出目录内容，是最常用的命令之一：

```
1 # 最简单的形式，列出当前目录的内容
2 ls
3 Desktop Documents Downloads Music Pictures Videos
4
5 # 带选项：-l 表示详细列表格式
6 ls -l
7 total 24
8 drwxr-xr-x 2 user user 4096 Jan 15 10:00 Desktop
9 drwxr-xr-x 2 user user 4096 Jan 15 10:00 Documents
10 ...
11
12 # 带参数：指定要列出的目录
13 ls /usr/bin
14 [大量程序名]
15
16 # 同时带选项和参数
17 ls -l /usr/bin
18 [详细列表]
```

选项通常有两种形式：短选项以单个连字符开头，后跟一个字母（如 `-l`）；长选项以两个连字符开头，后跟完整的单词（如 `--long`）。许多选项同时有短形式和长形式，例如 `-a` 和 `--all` 是等价的。短选项可以组合在一起，`ls -la` 等价于 `ls -l -a`。

不同的命令有不同的选项和参数规则。记住所有命令的所有选项是不现实的，关键是知道如何查找帮助信息——这正是下一节要讲的内容。

1.3.2.4. 获取帮助

Linux 命令众多，选项繁杂，没有人能记住所有的细节。好消息是，Linux 提供了完善的帮助系统，让你可以随时查阅命令的用法。

最全面的帮助来自手册页 (man pages)。几乎每个命令都有对应的手册页，使用 `man` 命令查看：

```
1 man ls
```

这会打开 `ls` 命令的手册页，包含了命令的详细说明、所有选项的解释、使用示例等。手册页使用 `less` 程序显示，你可以用方向键或 `j/k` 上下滚动，用 `/` 搜索，按 `q` 退出。

手册页非常详尽，有时候你只需要快速查看常用选项。这时可以使用 `--help` 选项，大多数命令都支持它：

```
1 ls --help
2 Usage: ls [OPTION]... [FILE]...
3 List information about the FILES (the current directory by default).
4 ...
```

输出会比手册页简短，直接在终端中显示，适合快速参考。

对于完全不熟悉的命令，`tldr`（Too Long; Didn't Read）是一个非常友好的工具。它提供简洁实用的命令示例，比手册页更容易理解：

```
1 # 首先安装 tldr
2 sudo apt install tldr
3
4 # 查看命令的简明帮助
5 tldr tar
6 tar
7
8 Archiving utility.
9 ...
10
11 - Create an archive from files:
12     tar cvf target.tar file1 file2 file3
13
14 - Extract an archive in the current directory:
15     tar xvf source.tar
16 ...
```

`tldr` 直接给出常见用法的例子，非常适合学习新命令。

还有一个有用的命令是 `type`，它告诉你某个命令是什么：

```
1 type ls
2 ls is aliased to `ls --color=auto'
3
4 type cd
5 cd is a shell builtin
6
7 type python3
8 python3 is /usr/bin/python3
```

这可以帮助你了解命令是内置命令、外部程序还是别名。

1.3.2.5. 命令历史与自动补全

高效使用终端的关键是少打字。Shell 提供了多种机制帮助你减少输入。

Tab 键是你最好的朋友。当你输入命令或路径的一部分时，按 Tab 键，Shell 会尝试自动补全。如果只有一个匹配项，会直接补全；如果有多个匹配项，按两次 Tab 会列出所有可能的选项：

```
1 # 输入 "cd Doc" 然后按 Tab
2 cd Doc<Tab>
3 cd Documents/ # 自动补全
4
5 # 输入 "ls /usr/l" 然后按 Tab Tab
6 ls /usr/l<Tab><Tab>
7 lib/    lib64/    local/ # 显示所有可能的选项
```


自动补全不仅适用于命令和路径，还适用于选项（需要安装 `bash-completion` 包）、Git 分支名、SSH 主机名等。养成频繁按 Tab 的习惯，可以大大提高输入效率并减少拼写错误。

命令历史让你可以重用之前输入过的命令。按上箭头 `↑` 可以回溯到上一条命令，继续按可以查看更早的命令；下箭头 `↓` 可以向前浏览。找到想要的命令后，可以直接按回车执行，也可以编辑后再执行。

更强大的是反向搜索功能。按 `Ctrl+R`，然后输入关键词，Shell 会搜索历史中包含该关键词的命令：

```
1 # 按 Ctrl+R, 然后输入 "git"
2 (reverse-i-search)`git': git push origin main
```

继续按 `Ctrl+R` 可以查看更早的匹配项。找到想要的命令后，按回车执行，或按右箭头将其放到命令行上编辑。这个功能在你想重复执行某个复杂命令但记不清完整内容时非常有用。

历史命令保存在 `~/.bash_history` 文件中，默认保存最近的 1000 条（可配置）。你可以用 `history` 命令查看历史列表，用 `!n` 执行第 `n` 条历史命令，用 `!!` 执行上一条命令。

1.3.2.6. 常用快捷键

掌握终端快捷键可以让你的操作更加流畅。以下是最重要的几个：

`Ctrl+C` 是最常用的快捷键，用于中断当前正在运行的命令。如果一个程序运行时间太长，或者你不小心启动了错误的命令，按 `Ctrl+C` 可以强制终止它。这会向程序发送 `SIGINT` 信号，大多数程序会响应这个信号并退出。

```
1 # 运行一个长时间的命令
2 sleep 1000
3 ^C # 按 Ctrl+C 中断
4   # 回到提示符
```

`Ctrl+D` 表示输入结束（EOF）。在提示符下按 `Ctrl+D` 会退出当前 Shell（相当于输入 `exit`）。在某些交互式程序中，`Ctrl+D` 表示输入完成。

`Ctrl+Z` 暂停当前程序并放到后台。程序不会终止，只是暂停执行。你可以用 `fg` 命令把它恢复到前台，或用 `bg` 让它在后台继续运行。这在你需要暂时做其他事情但不想终止当前程序时很有用。

`Ctrl+L` 清屏，相当于 `clear` 命令。当终端内容太多变得混乱时，这个快捷键可以让你获得一个干净的屏幕，但不会影响命令历史或当前正在输入的内容。

编辑命令行时，以下快捷键也很有用：

- `Ctrl+A`：移动光标到行首
- `Ctrl+E`：移动光标到行尾
- `Ctrl+U`：删除光标前的所有内容
- `Ctrl+K`：删除光标后的所有内容
- `Ctrl+W`：删除光标前的一个单词
- `Alt+B`：向后移动一个单词
- `Alt+F`：向前移动一个单词

这些快捷键源自 Emacs 编辑器的键绑定，在许多命令行程序中都通用。一开始可能记不住这么多，但随着使用它们会变成肌肉记忆。

1.3.2.7. 通配符

通配符 (wildcards) 让你可以用模式匹配多个文件，而不需要一个个地输入文件名。Shell 会在执行命令前展开通配符，将其替换为匹配的文件列表。

星号 `*` 匹配任意数量的任意字符（包括零个字符）。这是最常用的通配符：

```
1 # 列出所有 .cpp 文件
2 ls *.cpp
3 main.cpp detector.cpp tracker.cpp
4
5 # 列出所有以 test 开头的文件
6 ls test*
7 test_detector.cpp test_tracker.cpp test_results.txt
8
9 # 删除所有 .o 文件（小心使用！）
10 rm *.o
```

问号 `?` 匹配恰好一个任意字符：

```
1 # 匹配 file1.txt, file2.txt, ... 但不匹配 file10.txt
2 ls file?.txt
3 file1.txt file2.txt file3.txt
```

方括号 `[]` 匹配其中的任意一个字符：

```
1 # 匹配 file1.txt 或 file2.txt
2 ls file[12].txt
3 file1.txt file2.txt
4
5 # 匹配任意数字
6 ls file[0-9].txt
7 file0.txt file1.txt ...
8
9 # 匹配任意字母
10 ls file[a-z].txt
11 filea.txt fileb.txt ...
```

使用通配符时要小心，特别是与 `rm` 等危险命令一起使用时。一个好习惯是先用 `ls` 加通配符查看会匹配哪些文件，确认无误后再用 `rm`：

```
1 # 先查看会删除哪些文件
2 ls *.tmp
3 cache1.tmp cache2.tmp temp.tmp
4
5 # 确认后再删除
6 rm *.tmp
```

另一个需要注意的是，如果通配符没有匹配到任何文件，默认行为是保持原样。这可能导致意外的结果：

```
1 # 如果没有 .xyz 文件
2 ls *.xyz
3 ls: cannot access '*.xyz': No such file or directory
```

1.3.2.8. 输入输出重定向

每个程序都有三个标准的数据流：标准输入（stdin）、标准输出（stdout）和标准错误（stderr）。默认情况下，标准输入来自键盘，标准输出和标准错误都显示在终端上。重定向让你可以改变这些数据流的来源和去向。

> 将标准输出重定向到文件。如果文件存在，会被覆盖；如果不存在，会被创建：

```
1 # 将 ls 的输出保存到文件
2 ls -l > filelist.txt
3
4 # 查看文件内容
5 cat filelist.txt
6 total 24
7 drwxr-xr-x 2 user user 4096 Jan 15 10:00 Desktop
8 ...
```

>> 将标准输出追加到文件末尾，不会覆盖原有内容：

```
1 # 追加内容到文件
2 echo "New line" >> filelist.txt
```

< 将文件内容作为标准输入：

```
1 # 从文件读取输入
2 sort < unsorted.txt
```

标准错误（错误信息）默认不会被 > 重定向。要重定向标准错误，使用 2>：

```
1 # 将错误信息重定向到文件
2 ls nonexistent 2> error.log
3
4 # 将标准输出和标准错误分别重定向
5 command > output.log 2> error.log
6
7 # 将标准错误重定向到标准输出（常用于日志记录）
8 command > all.log 2>&1
9
10 # 更简洁的写法（Bash 4+）
11 command &> all.log
```

2>&1 的含义是“将文件描述符 2（标准错误）重定向到文件描述符 1（标准输出）当前指向的地方”。顺序很重要：> file 2>&1 是正确的（先重定向标准输出到文件，再让标准错误跟随），而 2>&1 > file 是错误的（标准错误会去到原来的标准输出，即终端）。

如果你想丢弃某个输出，可以重定向到 /dev/null，这是一个特殊的“黑洞”设备：

```
1 # 丢弃标准输出
2 command > /dev/null
3
4 # 丢弃所有输出
5 command > /dev/null 2>&1
```

1.3.2.9. 管道

管道（pipe）是 Unix 哲学的核心体现之一：每个程序做好一件事，复杂的任务通过组合简单程序来完成。管道用 | 符号表示，它把一个命令的标准输出连接到另一个命令的标准输入。

```

1 # 列出文件并统计行数
2 ls -l | wc -l
3 15
4
5 # 查找包含特定文本的行
6 cat log.txt | grep "error"
7 [所有包含 error 的行]
8
9 # 排序并去重
10 cat names.txt | sort | uniq

```

管道可以连接多个命令，形成处理流水线：

```

1 # 复杂的例子：找出当前目录下最大的 5 个文件
2 ls -lS | head -6
3 total 1024
4 -rw-r--r-- 1 user user 512000 Jan 15 10:00 bigfile.dat
5 -rw-r--r-- 1 user user 256000 Jan 15 10:00 medium.dat
6 ...

```

这个命令的工作流程是：`ls -lS` 列出文件并按大小排序，输出通过管道传给 `head -6`，后者只保留前 6 行（包括 total 行）。

管道让你可以构建强大的数据处理流程。一些常用的管道组件包括：

- `grep`：过滤包含特定模式的行
- `sort`：排序
- `uniq`：去除相邻的重复行（通常与 `sort` 配合使用）
- `wc`：统计行数、单词数、字符数
- `head` / `tail`：只保留开头/结尾的几行
- `cut`：提取列
- `awk` / `sed`：更强大的文本处理

学习这些工具的基本用法，你就能通过管道组合解决各种文本处理问题。这是命令行的威力所在——不需要编写程序，只需要把现有工具连接起来。

1.3.2.10. 后台运行

默认情况下，当你运行一个命令时，Shell 会等待它完成才显示下一个提示符。对于长时间运行的任务，你可能希望让它在后台运行，这样可以继续做其他事情。

在命令末尾加上 `&`，命令会在后台启动：

```

1 long_running_command &
2 [1] 12345
3 # 立即返回提示符

```

`[1]` 是作业编号，`12345` 是进程 ID。命令在后台运行，你可以继续输入其他命令。

`jobs` 命令列出当前 Shell 的后台作业：

```

1 jobs
2 [1]+  Running                  long_running_command &

```

`fg` 把后台作业带回前台，`bg` 让暂停的作业在后台继续运行：

```

1 # 把作业 1 带到前台

```

```
2 fg %1
3
4 # 让作业 1 在后台继续
5 bg %1
```

前面提到的 `Ctrl+Z` 可以暂停当前前台程序，然后用 `bg` 让它在后台继续：

```
1 long_command
2 ^Z # 按 Ctrl+Z 暂停
3 [1]+ Stopped long_command
4 bg
5 [1]+ long_command &
6 # 命令在后台继续运行
```

但是，后台作业仍然与当前终端关联。如果你关闭终端，后台作业也会被终止。要让命令在终端关闭后继续运行，使用 `nohup`：

```
1 nohup long_command &
2 nohup: ignoring input and appending output to 'nohup.out'
3 [1] 12345
```

`nohup` 让命令忽略挂起信号（`SIGHUP`），即使终端关闭也会继续运行。输出会被保存到 `nohup.out` 文件。

对于更复杂的需求，可以使用 `screen` 或 `tmux` 这样的终端复用器。它们允许你创建多个虚拟终端会话，断开连接后会话保持运行，之后可以重新连接。这对于远程服务器管理特别有用。

1.3.2.11. 环境变量与 PATH

环境变量是 Shell 中存储配置信息的机制。它们是键值对，影响 Shell 和程序的行为。使用 `echo $变量名` 可以查看环境变量的值：

```
1 echo $HOME
2 /home/username
3
4 echo $USER
5 username
6
7 echo $SHELL
8 /bin/bash
```

`env` 命令列出所有环境变量，`export` 命令设置环境变量：

```
1 # 设置环境变量
2 export MY_VAR="hello"
3
4 # 验证
5 echo $MY_VAR
6 hello
```

`export` 设置的变量会传递给子进程。如果只是 `MY_VAR="hello"`（不带 `export`），变量只在当前 Shell 中有效，不会传给运行的程序。

最重要的环境变量之一是 `PATH`。它决定了 Shell 在哪些目录中搜索可执行程序。当你输入一个命令时，Shell 会按 `PATH` 中列出的目录顺序查找同名的可执行文件：

```
1 echo $PATH
```

```
2 /usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
```

PATH 中的目录用冒号分隔。这就解释了为什么你可以直接输入 `ls` 而不是 `/bin/ls` —— 因为 `/bin` 在 PATH 中。

如果你安装了程序到非标准位置，需要把它的目录加入 PATH：

```
1 # 临时添加到 PATH (当前 Shell 有效)
2 export PATH="$PATH:/opt/myprogram/bin"
3
4 # 永久添加 (编辑 ~/.bashrc)
5 echo 'export PATH="$PATH:/opt/myprogram/bin"' >> ~/.bashrc
6 source ~/.bashrc # 立即生效
```

`~/.bashrc` 是 Bash 的配置文件，每次启动交互式 Shell 时会执行。在这里设置的环境变量会对所有新的终端窗口生效。

另一个常见的需求是设置库路径。当程序运行时需要加载共享库，系统会在 `LD_LIBRARY_PATH` 指定的目录中查找：

```
1 export LD_LIBRARY_PATH="/opt/cuda/lib64:$LD_LIBRARY_PATH"
```

对于 RoboMaster 开发，你会经常遇到与 ROS 2 相关的环境变量。ROS 2 使用环境变量来定位包、设置域 ID 等。这就是为什么每次打开终端都需要 `source /opt/ros/humble/setup.bash` —— 这个脚本设置了 ROS 2 需要的所有环境变量。

终端和 Shell 是进入 Linux 世界的钥匙。这里介绍的只是基础，但掌握这些基础已经足够应对日常开发中的大多数情况。随着使用经验的积累，你会逐渐发现更多技巧，命令行会从一个陌生的界面变成一个高效的工具。记住，每个熟练的 Linux 用户都是从第一个命令开始的。保持好奇心，多尝试，多查阅帮助文档，你会越来越得心应手。

1.3.3. 文件系统与目录结构

理解 Linux 的文件系统是使用这个操作系统的基础。与 Windows 的盘符系统不同，Linux 采用了一种更加统一和优雅的方式来组织文件——一切都从根目录 `/` 开始，形成一棵倒置的树。这种设计不仅仅是审美上的选择，它反映了“一切皆文件”的 Unix 哲学，让系统的管理和编程都变得更加一致。本节将带你深入了解 Linux 的目录结构，理解每个重要目录的用途，掌握路径的表示方法，为后续的文件操作打下基础。

1.3.3.1. 根目录与目录树

Linux 文件系统的起点是根目录，用单个斜杠 `/` 表示。这是整个文件系统的顶层，所有其他目录和文件都是根目录的子孙。无论有多少块硬盘、多少个分区，它们最终都会被“挂载”到这棵目录树的某个位置，成为树的一部分。

让我们从根目录开始探索。打开终端，输入 `ls /` 可以看到根目录下的内容：

```
1 ls /
2 bin  dev  home  lib64      media  opt   root  sbin  srv  tmp  var
3 boot etc  lib   lost+found mnt    proc  run   snap sys  usr
```

每个目录都有特定的用途，遵循文件系统层次标准（Filesystem Hierarchy Standard, FHS）。这个标准定义了 Linux 系统中目录的组织方式，让不同的发行版保持一致，也让用户和开发者知道应该在哪里找什么。

这种结构的一个重要特点是“内容按类型组织，而不是按来源组织”。在 Windows 中，一个程序的所有文件（可执行文件、库、配置、文档）通常都放在 `Program Files` 下的一个文件夹里。在 Linux 中，可执行文件放在 `bin` 目录，库放在 `lib` 目录，配置放在 `etc` 目录，文档放在 `share` 目录。同一个程序的文件分散在多个地方，但同一类型的文件集中在一起。这种组织方式让系统管理更加系统化——要查看所有配置文件，去 `/etc`；要查看所有日志，去 `/var/log`。

1.3.3.2. 重要目录详解

让我们逐一了解最重要的目录。理解它们的用途，你就能在需要时知道去哪里找东西，也知道应该把东西放在哪里。

`/home` 是普通用户的家目录所在地。每个用户都有一个以用户名命名的子目录，如 `/home/alice`、`/home/bob`。用户的个人文件、配置、下载、文档都存放在这里。当你在终端中看到 `~` 符号，它就代表当前用户的家目录。对于用户 `alice` 来说，`~` 等价于 `/home/alice`。

家目录是你的私人空间。你对这个目录有完全的控制权，可以创建、修改、删除任何文件。系统级的目录（如 `/usr`、`/etc`）通常需要管理员权限才能修改，但家目录是你自己的领地。RoboMaster 项目的代码、编译产物、个人配置通常都放在家目录的某个子目录中，如 `~/ros2_ws`（ROS 2 工作空间）或 `~/projects`。

```
1 ls ~
2 Desktop Documents Downloads Music Pictures Public Templates Videos
3 ros2_ws projects .bashrc .config .local
```

注意那些以点开头的文件和目录（如 `.bashrc`、`.config`），它们是隐藏文件，普通的 `ls` 不会显示它们，需要加 `-a` 选项。这些隐藏文件通常存储配置信息，我们稍后会详细讨论。

`/etc` 存放系统级的配置文件。“etc”最初代表“et cetera”（其他），但现在普遍理解为“Editable Text Configuration”（可编辑的文本配置）。几乎所有系统服务和许多应用程序的配置文件都在这里。

```
1 ls /etc
2 apt          hostname      passwd       ssh
3 bash.bashrc  hosts        profile      sudoers
4 default      network      resolv.conf  systemd
5 fstab        nginx        shadow       ...
```

一些重要的配置文件包括：`/etc/passwd` 存储用户账户信息，`/etc/hosts` 是本地的域名解析表，`/etc/fstab` 定义了磁盘分区的挂载方式，`/etc/ssh/sshd_config` 配置 SSH 服务器。当你需要修改系统行为时，通常会编辑 `/etc` 下的某个文件。这些修改需要管理员权限（使用 `sudo`），因为它们影响整个系统。

`/usr` 是“Unix System Resources”的缩写，存放用户级的程序和数据。这是文件系统中最大的目录之一，包含了大部分用户命令、库和文档。它的结构镜像了根目录：

```
1 ls /usr
2 bin  games  include  lib  lib64  local  sbin  share  src
```

`/usr/bin` 存放大多数用户命令（如 `gcc`、`python`、`git`），`/usr/lib` 存放程序库，`/usr/include` 存放 C/C++ 头文件，`/usr/share` 存放与架构无关的数据（如文档、图标、翻译）。当你用包管理器安装软件时，大部分文件会被安装到 `/usr` 的相应子目录中。

`/usr/local` 值得特别注意。它用于存放本地安装的软件——不是通过包管理器安装的，而是手动编译安装或从第三方获取的。其结构与 `/usr` 相同（`bin`、`lib`、`include` 等）。当你

从源码编译安装软件并使用默认的安装路径时，通常会安装到这里。这种分离让你可以区分系统软件和本地安装的软件，也避免了与包管理器的冲突。

```
1 # 查看本地安装的程序
2 ls /usr/local/bin
3
4 # 查看本地安装的库
5 ls /usr/local/lib
```

`/opt` 是“optional”的缩写，用于安装第三方软件包。与 `/usr/local` 不同，`/opt` 下的软件通常保持自包含的结构——一个软件的所有文件都在一个子目录中，如 `/opt/google/chrome`、`/opt/ros/humble`。ROS 2 就默认安装在 `/opt/ros/` 下，每个发行版一个子目录。

```
1 ls /opt
2 google  ros   cuda
3
4 ls /opt/ros
5 humble  iron
```

这种组织方式的好处是软件彼此隔离，卸载时只需删除整个目录。但也意味着这些程序的可执行文件不在系统 `PATH` 中，需要手动添加或 `source` 相应的设置脚本（如 `source /opt/ros/humble/setup.bash`）。

`/var` 存放可变的数据文件，即运行时会改变的内容。“var”代表“variable”。最重要的子目录包括：

- `/var/log`：系统和程序的日志文件。当程序出问题时，这里通常是第一个查看的地方。
- `/var/cache`：应用程序的缓存数据。包管理器的下载缓存通常在这里。
- `/var/lib`：程序的状态信息。数据库文件、包管理器的数据库等。
- `/var/tmp`：比 `/tmp` 更持久的临时文件，重启后可能保留。

```
1 # 查看系统日志
2 ls /var/log
3 syslog  auth.log  kern.log  apt  nginx  ...
4
5 # 查看最近的系统日志
6 tail /var/log/syslog
```

`/tmp` 是临时文件目录。任何用户都可以在这里创建文件，但这些文件是临时的——系统重启后通常会被清空。程序运行时的临时文件、下载中的文件、解压的临时内容都可以放在这里。

```
1 ls /tmp
2 systemd-private-xxx  ssh-xxx  ...
```

由于 `/tmp` 对所有用户开放，存在安全隐患。敏感的临时数据应该放在更安全的位置，如用户自己的目录。现代系统通常会为每个用户创建私有的临时目录，可以通过 `mktemp` 命令创建安全的临时文件。

`/dev` 是“devices”的缩写，包含设备文件。在 Linux 的“一切皆文件”哲学中，硬件设备也表现为文件。硬盘是 `/dev/sda`，分区是 `/dev/sda1`，终端是 `/dev/tty`，随机数生成器是 `/dev/random`。

```
1 ls /dev
```

```
2 sda sda1 sda2 tty null zero random urandom ...
```

一些有趣的设备文件：

- `/dev/null`：黑洞设备，写入的任何内容都会消失，读取会立即返回 EOF。常用于丢弃不需要的输出。
- `/dev/zero`：读取会返回无限的零字节，常用于创建空文件或清零磁盘。
- `/dev/random` 和 `/dev/urandom`：随机数生成器，用于加密等需要随机性的场合。
- `/dev/ttyUSB0`：USB 串口设备，RoboMaster 开发中经常用到。

```
1 # 丢弃命令的输出
2 command > /dev/null
3
4 # 创建一个 1MB 的空文件
5 dd if=/dev/zero of=emptyfile bs=1M count=1
```

`/proc` 是一个虚拟文件系统，不占用磁盘空间，而是提供了进程和内核信息的接口。每个运行中的进程在 `/proc` 下都有一个以进程 ID 命名的目录，包含该进程的各种信息。

```
1 ls /proc
2 1 2 3 ... 12345 ... cpuinfo meminfo version ...
3
4 # 查看 CPU 信息
5 cat /proc/cpuinfo
6 processor : 0
7 vendor_id : GenuineIntel
8 model name : Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz
9 ...
10
11 # 查看内存信息
12 cat /proc/meminfo
13 MemTotal:      16384000 kB
14 MemFree:       8192000 kB
15 ...
16
17 # 查看进程 12345 的命令行
18 cat /proc/12345/cmdline
19 /usr/bin/python3 my_script.py
```

`/proc` 是系统监控工具（如 `top`、`htop`、`ps`）获取信息的来源。了解它的存在可以帮助你在需要时直接获取底层信息。

类似地，`/sys` 也是一个虚拟文件系统，提供了对内核子系统、设备驱动和硬件设备的访问。它比 `/proc` 更结构化，主要用于设备管理和内核参数调整。

```
1 # 查看 CPU 的在线状态
2 cat /sys/devices/system/cpu/online
3 0-7
4
5 # 调整屏幕亮度（如果支持）
6 cat /sys/class/backlight/intel_backlight/brightness
7 500
```

1.3.3.3. 绝对路径与相对路径

在 Linux 中定位文件有两种方式：绝对路径和相对路径。理解它们的区别对于正确使用命令行和编写脚本至关重要。

绝对路径从根目录 `/` 开始, 完整地描述了文件在目录树中的位置。无论你当前在哪个目录, 绝对路径都指向同一个文件。

```
1 /home/alice/projects/rm_vision/src/main.cpp
2 /etc/ssh/sshd_config
3 /opt/ros/humble/setup.bash
```

绝对路径的优点是明确无歧义, 缺点是通常比较长。

相对路径从当前工作目录出发, 描述如何到达目标文件。它不以 `/` 开头。

```
1 # 假设当前目录是 /home/alice/projects/rm_vision
2 ls src/main.cpp          # 相对路径
3 ls ./src/main.cpp        # 同上, ./ 表示当前目录
4 ls ../rm_control/        # 上级目录的 rm_control
```

相对路径更简短, 但它的含义取决于当前目录。同样是 `src/main.cpp`, 在不同的目录下指向不同的文件。

在脚本和配置文件中, 通常推荐使用绝对路径, 因为脚本可能在任何目录下执行。但在交互式使用中, 相对路径更方便。

1.3.3.4. 特殊路径符号

Shell 提供了几个特殊的路径符号, 让路径表示更加简洁。

`.` (单个点) 表示当前目录。在大多数情况下, `./file` 和 `file` 是等价的。但在执行当前目录下的程序时, 必须使用 `./`:

```
1 # 执行当前目录下的程序
2 ./my_program
3
4 # 这是安全特性, 防止当前目录下的恶意程序覆盖系统命令
5 my_program # 这会在 PATH 中查找, 而不是当前目录
```

`..` (两个点) 表示父目录 (上级目录)。可以链式使用:

```
1 pwd
2 /home/alice/projects/rm_vision/src
3
4 cd ..
5 pwd
6 /home/alice/projects/rm_vision
7
8 cd ../../
9 pwd
10 /home/alice/projects
```

`~` (波浪号) 表示当前用户的家目录。这是 Shell 提供的便利功能:

```
1 echo ~
2 /home/alice
3
4 cd ~/projects
5 pwd
6 /home/alice/projects
7
8 # 也可以指定其他用户的家目录
9 echo ~bob
10 /home/bob
```

-（减号）在 `cd` 命令中表示上一个工作目录，让你可以在两个目录之间快速切换：

```
1 cd /var/log
2 cd ~/projects
3 cd -      # 回到 /var/log
4 cd -      # 回到 ~/projects
```

这些符号可以组合使用：

```
1 ls ~/projects/../Documents # 等价于 ls /home/alice/Documents
2 cd ~/.config                # 进入配置目录
```

1.3.3.5. 文件类型

Linux 文件系统中不只有普通文件和目录。`ls -l` 的输出第一个字符就表示文件类型：

```
1 ls -l /
2 drwxr-xr-x  2 root root  4096 Jan 15 10:00 bin
3 -rw-r--r--  1 root root   220 Jan 15 10:00 .bashrc
4 lrwxrwxrwx  1 root root    7 Jan 15 10:00 lib -> usr/lib
5 crw-rw-rw-  1 root root  1, 3 Jan 15 10:00 /dev/null
6 brw-rw----  1 root disk 8,  0 Jan 15 10:00 /dev/sda
7 srwxrwxrwx  1 root root    0 Jan 15 10:00 /run/docker.sock
8 prw-r--r--  1 root root    0 Jan 15 10:00 /tmp/mypipe
```

让我们逐一了解每种类型。

普通文件（`-`）是最常见的类型，包括文本文件、二进制文件、图片、可执行程序等。Linux 不像 Windows 那样严重依赖文件扩展名来识别文件类型——扩展名只是命名约定，方便人类识别，系统本身通过文件内容（“魔数”）来判断类型。

```
1 # file 命令可以检测文件的实际类型
2 file main.cpp
3 main.cpp: C++ source, ASCII text
4
5 file /bin/ls
6 /bin/ls: ELF 64-bit LSB shared object, x86-64, ...
7
8 file photo.jpg
9 photo.jpg: JPEG image data, JFIF standard 1.01, ...
```

目录（`d`）是一种特殊的文件，包含其他文件和目录的列表。从用户角度看，目录就是文件夹；从系统角度看，目录文件存储了名称到 inode 的映射。

符号链接（`l`），也叫软链接，类似于 Windows 的快捷方式。它存储了另一个文件的路径，访问链接时会被重定向到目标文件。链接可以指向文件或目录，可以跨文件系统。

```
1 # 创建符号链接
2 ln -s /opt/ros/humble ~/ros
3
4 # 现在 ~/ros 指向 /opt/ros/humble
5 ls -l ~/ros
6 lrwxrwxrwx 1 alice alice 15 Jan 15 10:00 /home/alice/ros -> /opt/ros/humble
7
8 # 访问链接就像访问原始目录
9 ls ~/ros/setup.bash
10 /home/alice/ros/setup.bash
```

符号链接在 RoboMaster 开发中很有用。你可以为常用的长路径创建短链接，或者让多个名称指向同一个配置文件。如果链接的目标被删除，链接会成为“悬空链接”，访问时会报错。

硬链接是另一种链接类型，它直接指向文件的 inode（文件系统中的数据结构），而不是路径。硬链接和原文件是完全平等的，删除其中一个，另一个仍然有效。硬链接不能指向目录，不能跨文件系统。

```
1 # 创建硬链接（不带 -s 选项）
2 ln original.txt hardlink.txt
3
4 # 两个名称指向同一个 inode
5 ls -li original.txt hardlink.txt
6 12345678 -rw-r--r-- 2 alice alice 100 Jan 15 10:00 original.txt
7 12345678 -rw-r--r-- 2 alice alice 100 Jan 15 10:00 hardlink.txt
```

注意 inode 号（12345678）相同，链接计数（第三列的 2）表示有两个名称指向这个文件。

字符设备（c）和块设备（b）是设备文件。字符设备按字符流处理数据（如终端、串口），块设备按块处理数据（如硬盘）。这些文件通常在 /dev 目录下，由系统管理。

套接字（s）和命名管道（p）是用于进程间通信的特殊文件。套接字支持双向通信（如 /var/run/docker.sock 用于与 Docker 守护进程通信），命名管道支持单向的先进先出数据流。这些是高级主题，日常使用中较少直接接触。

1.3.3.6. 隐藏文件

在 Linux 中，文件名以点（.）开头的文件被视为隐藏文件。这不是一个特殊的文件属性，仅仅是命名约定——ls 默认不显示这些文件，但加上 -a 选项就会显示。

```
1 ls
2 Documents Downloads Pictures
3
4 ls -a
5 . .. .bashrc .config .local .ssh Documents Downloads Pictures
```

隐藏文件主要用于存储配置和状态信息，避免在日常浏览时造成视觉干扰。用户家目录下通常有大量隐藏文件和目录，每个应用程序都可能创建自己的配置目录。

一些重要的隐藏文件和目录包括：

~/.bashrc 是 Bash 的用户配置文件。每次打开新的交互式 Shell 时，这个文件会被执行。你可以在这里设置环境变量、定义别名、配置提示符等。

```
1 # 查看 .bashrc 的内容
2 cat ~/.bashrc
3
4 # 常见的自定义内容
5 export PATH="$PATH:/opt/myprogram/bin"
6 alias ll='ls -aLF'
7 alias gs='git status'
```

修改 .bashrc 后，需要重新打开终端或执行 source ~/.bashrc 使改动生效。

~/.bash_history 存储命令历史。你之前输入的命令都记录在这里，这就是为什么按上箭头可以找到历史命令。

~/.ssh/ 目录存储 SSH 相关的文件：id_rsa 和 id_rsa.pub 是你的 SSH 密钥对，known_hosts 记录了你连接过的服务器，config 可以配置 SSH 连接的各种选项。

```
1 ls -la ~/.ssh
2 -rw----- 1 alice alice 1679 Jan 15 10:00 id_rsa      # 私钥, 必须保密
3 -rw-r--r-- 1 alice alice 400 Jan 15 10:00 id_rsa.pub   # 公钥, 可以分享
4 -rw-r--r-- 1 alice alice 1234 Jan 15 10:00 known_hosts
5 -rw-r--r-- 1 alice alice 200 Jan 15 10:00 config
```

`~/.config/` 是现代应用程序存放配置的标准位置, 遵循 XDG Base Directory 规范。很多程序会在这里创建自己的子目录, 如 `~/.config/Code/` (VS Code)、`~/.config/nvim/` (Neovim)。

`~/.local/` 存储用户级的数据和程序。`~/.local/bin/` 可以放置用户自己的可执行文件, 很多用户会把这个目录加入 PATH。`~/.local/share/` 存储应用程序的数据。

```
1 # 将 ~/.local/bin 加入 PATH (在 .bashrc 中添加)
2 export PATH="$PATH:$HOME/.local/bin"
```

`~/.cache/` 存储缓存数据, 可以安全删除 (程序会重新生成需要的缓存)。当磁盘空间紧张时, 这是一个可以清理的地方。

理解文件系统结构是 Linux 使用的基础。当你知道配置文件在 `/etc`、用户文件在 `/home`、日志在 `/var/log`、设备在 `/dev` 时, 你就能更有效地管理系统和排查问题。这种知识也会帮助你理解 ROS 2 的安装位置 (`/opt/ros`)、为什么需要 `source setup` 脚本、以及如何组织自己的项目文件。随着使用经验的积累, 这些目录会变得像你自己房间里的抽屉一样熟悉——你总是知道去哪里找什么。

1.3.4. 文件与目录操作

掌握了文件系统的结构之后, 接下来要学习如何在这个结构中导航和操作。文件与目录操作是 Linux 命令行最基础也是最常用的技能——创建文件、复制目录、移动文件、查看内容、搜索文件——这些操作构成了日常工作的基础。本节将详细介绍这些命令, 从最简单的目录切换到复杂的文件查找, 从基本的文件复制到灵活的压缩解压。这些命令你会每天使用, 熟练掌握它们会显著提升你的工作效率。

1.3.4.1. 目录导航

在命令行中工作, 首先要知道自己在哪里, 然后才能去想去的方向。

`pwd` (print working directory) 显示当前工作目录的绝对路径。当你在目录之间跳转后不确定自己的位置时, `pwd` 可以告诉你:

```
1 pwd
2 /home/alice/projects/rm_vision
```

这个命令非常简单, 没有常用的选项, 但它是定位自己的基本工具。

`cd` (change directory) 用于切换当前工作目录。它是最常用的命令之一:

```
1 # 切换到指定目录 (绝对路径)
2 cd /usr/local/bin
3
4 # 切换到指定目录 (相对路径)
5 cd src/detector
6
7 # 回到家目录
8 cd ~
```

```

9  cd          # 不带参数也会回到家目录
10
11 # 回到上级目录
12 cd ..
13
14 # 回到上两级目录
15 cd ../../
16
17 # 回到上一个工作目录
18 cd -
19 /home/alice/projects # 显示切换到的目录

```

一个常见的工作流程是：用 `cd` 进入项目目录，用 `pwd` 确认位置，然后开始工作。RoboMaster 开发中，你可能会频繁在 ROS 2 工作空间、源码目录、构建目录之间切换：

```

1  cd ~/ros2_ws          # 进入工作空间
2  cd src/rm_vision      # 进入包的源码
3  cd ../../build        # 去构建目录查看编译产物
4  cd -                  # 回到刚才的 src/rm_vision

```

养成使用 Tab 补全的习惯可以大大加快目录切换的速度。输入 `cd ~/pro<Tab>` 会自动补全为 `cd ~/projects/`，然后继续输入和补全。

1.3.4.2. 查看目录内容

`ls` (list) 列出目录的内容，是使用频率最高的命令之一。它的基本用法很简单，但选项丰富，可以满足各种需求。

```

1 # 列出当前目录的内容
2 ls
3 CMakeLists.txt include src test README.md
4
5 # 列出指定目录的内容
6 ls /usr/bin
7
8 # 列出多个目录的内容
9 ls src include

```

最常用的选项组合是 `-la`，它显示详细信息 (`-l`) 并包含隐藏文件 (`-a`)：

```

1  ls -la
2  total 32
3  drwxr-xr-x  6 alice alice 4096 Jan 15 10:00 .
4  drwxr-xr-x 12 alice alice 4096 Jan 15 09:00 ..
5  -rw-r--r--  1 alice alice 1234 Jan 15 10:00 CMakeLists.txt
6  drwxr-xr-x  2 alice alice 4096 Jan 15 10:00 .git
7  -rw-r--r--  1 alice alice  567 Jan 15 10:00 .gitignore
8  drwxr-xr-x  3 alice alice 4096 Jan 15 10:00 include
9  -rw-r--r--  1 alice alice  890 Jan 15 10:00 README.md
10 drwxr-xr-x  4 alice alice 4096 Jan 15 10:00 src
11 drwxr-xr-x  2 alice alice 4096 Jan 15 10:00 test

```

让我们解读这个输出。每行代表一个文件或目录，列的含义是：

1. 文件类型和权限 (`drwxr-xr-x`)：第一个字符是类型 (`d` 目录, `-` 普通文件, `l` 链接)，后面九个字符是权限
2. 硬链接数 (`6`)
3. 所有者 (`alice`)

4. 所属组 (alice)
5. 大小 (字节) (4096)
6. 最后修改时间 (Jan 15 10:00)
7. 文件名 (src)

其他有用的 `ls` 选项:

```
1 # -h 人类可读的文件大小 (KB、MB、GB)
2 ls -lh
3 -rw-r--r-- 1 alice alice 1.2K Jan 15 10:00 CMakeLists.txt
4 -rw-r--r-- 1 alice alice 156M Jan 15 10:00 model.onnx
5
6 # -t 按修改时间排序 (最新的在前)
7 ls -lt
8
9 # -S 按文件大小排序 (最大的在前)
10 ls -ls
11
12 # -r 反向排序
13 ls -ltr # 按时间排序, 最旧的在前
14
15 # -R 递归列出子目录
16 ls -R
17
18 # --color 彩色显示 (现代系统通常默认启用)
19 ls --color=auto
```

很多用户会在 `.bashrc` 中定义别名来简化常用的 `ls` 命令:

```
1 alias ll='ls -aLF'
2 alias la='ls -A'
3 alias l='ls -CF'
```

`tree` 命令以树状结构显示目录内容, 对于理解项目结构非常有帮助。它不是系统自带的, 需要安装:

```
1 sudo apt install tree
2
3 tree
4 .
5 |__ CMakeLists.txt
6 |__ include
7 |   |__ rm_vision
8 |   |   |__ detector.hpp
9 |   |   |__ tracker.hpp
10 |__ README.md
11 |__ src
12 |   |__ detector.cpp
13 |   |__ main.cpp
14 |   |__ tracker.cpp
15 |__ test
16 |   |__ test_detector.cpp
17
18 4 directories, 8 files
```

`tree` 的一些有用选项:

```
1 # -L 限制显示深度
2 tree -L 2
3
```

```

4 # -d 只显示目录
5 tree -d
6
7 # -I 排除匹配模式的文件
8 tree -I 'build|__pycache__'
9
10 # --dirsfirst 目录排在前面
11 tree --dirsfirst

```

1.3.4.3. 创建文件与目录

touch 命令创建空文件，或更新现有文件的时间戳：

```

1 # 创建空文件
2 touch newfile.txt
3 ls -l newfile.txt
4 -rw-r--r-- 1 alice alice 0 Jan 15 10:00 newfile.txt
5
6 # 创建多个文件
7 touch file1.txt file2.txt file3.txt
8
9 # 如果文件已存在，更新其修改时间
10 touch existingfile.txt

```

touch 主要用于快速创建空文件。如果你需要创建有内容的文件，通常会用重定向或编辑器：

```

1 # 用重定向创建有内容的文件
2 echo "Hello, World!" > hello.txt
3
4 # 用 cat 和 heredoc 创建多行文件
5 cat > config.txt << EOF
6 setting1=value1
7 setting2=value2
8 EOF

```

mkdir (make directory) 创建目录：

```

1 # 创建单个目录
2 mkdir myproject
3
4 # 创建多个目录
5 mkdir dir1 dir2 dir3
6
7 # -p 创建多级目录（父目录不存在时自动创建）
8 mkdir -p projects/rm_vision/src/detector
9
10 # 不加 -p 时，如果父目录不存在会报错
11 mkdir projects/newproject/src
12 mkdir: cannot create directory 'projects/newproject/src': No such file or
   directory

```

-p 选项非常实用，它让你可以一次性创建完整的目录结构，而不用一级一级地创建。在脚本中使用 **mkdir -p** 也更安全——如果目录已存在，它不会报错，而是静默成功。

创建 RoboMaster 项目的典型目录结构：

```

1 mkdir -p rm_vision/{include/rm_vision,src,test,config,launch,models}
2 tree rm_vision

```

```

3 rm_vision
4 |— config
5 |— include
6 |   |— rm_vision
7 |— launch
8 |— models
9 |— src
10 |— test

```

这里使用了 Bash 的花括号展开功能, `{a,b,c}` 会展开为 `a b c`。

1.3.4.4. 复制、移动与删除

cp (copy) 复制文件或目录:

```

1 # 复制文件
2 cp source.txt destination.txt
3
4 # 复制文件到目录
5 cp file.txt /path/to/directory/
6
7 # 复制多个文件到目录
8 cp file1.txt file2.txt file3.txt /path/to/directory/
9
10 # -r 或 -R 递归复制目录 (复制目录必须加此选项)
11 cp -r src_dir dest_dir
12
13 # -i 覆盖前询问确认
14 cp -i source.txt existing.txt
15 cp: overwrite 'existing.txt'?
16
17 # -v 显示复制过程 (verbose)
18 cp -rv project/ backup/
19 'project/src/main.cpp' -> 'backup/src/main.cpp'
20 'project/include/header.h' -> 'backup/include/header.h'
21 ...
22
23 # -p 保留文件属性 (时间戳、权限等)
24 cp -p important.txt backup.txt
25
26 # -a 归档模式, 等同于 -dR --preserve=all, 完整复制
27 cp -a project/ project_backup/

```

mv (move) 移动文件或目录, 也用于重命名:

```

1 # 移动文件
2 mv file.txt /path/to/directory/
3
4 # 移动并重命名
5 mv oldname.txt /path/to/newname.txt
6
7 # 重命名 (同一目录内移动)
8 mv oldname.txt newname.txt
9
10 # 移动目录 (不需要 -r)
11 mv mydir /path/to/destination/
12
13 # -i 覆盖前询问
14 mv -i source.txt existing.txt
15
16 # -v 显示移动过程
17 mv -v *.cpp src/

```



```
18 'main.cpp' -> 'src/main.cpp'
19 'detector.cpp' -> 'src/detector.cpp'
```

`mv` 在同一文件系统内移动文件时非常快，因为它只是修改目录条目，不实际复制数据。跨文件系统移动时，`mv` 会先复制再删除原文件。

`rm` (remove) 删除文件或目录。这是一个危险的命令，因为 Linux 命令行删除的文件不会进入回收站，而是直接消失：

```
1 # 删除文件
2 rm file.txt
3
4 # 删除多个文件
5 rm file1.txt file2.txt file3.txt
6
7 # -i 删除前询问确认
8 rm -i important.txt
9 rm: remove regular file 'important.txt'?
10
11 # -r 或 -R 递归删除目录
12 rm -r mydir
13
14 # -f 强制删除，不询问确认，忽略不存在的文件
15 rm -f maybe_exists.txt
16
17 # -v 显示删除过程
18 rm -rv old_project/
19 removed 'old_project/src/main.cpp'
20 removed 'old_project/src/'
21 removed 'old_project/'
```

`rm -rf` 是最危险的命令组合——递归删除，强制执行，不询问确认。它可以在瞬间删除大量文件，包括你不想删除的。有一些臭名昭著的事故：

```
1 # 极度危险！不要执行！
2 rm -rf /           # 删除整个系统
3 rm -rf ~           # 删除整个家目录
4 rm -rf *           # 删除当前目录所有内容
5 rm -rf $UNDEFINED_VAR/* # 如果变量未定义，可能变成 rm -rf /*
```

安全使用 `rm` 的建议：

1. 删除前先用 `ls` 确认要删除的文件
2. 使用 `-i` 选项，特别是配合通配符时
3. 对于重要操作，先移动到临时目录，确认无误后再删除
4. 在脚本中删除前检查变量是否为空
5. 考虑使用 `trash-cli` 这样的工具，删除到回收站而不是直接删除

```
1 # 安全的做法
2 ls *.log           # 先看看会删除什么
3 temp.log debug.log error.log
4 rm -i *.log        # 逐个确认
5
6 # 或者移动到临时目录
7 mv *.log /tmp/to_delete/
8 # 确认后再 rm -rf /tmp/to_delete/
```

`rmdir` 只能删除空目录，比 `rm -r` 更安全：

```
1 rmdir empty_dir          # 成功删除空目录
2 rmdir non_empty_dir
3 rmdir: failed to remove 'non_empty_dir': Directory not empty
```

1.3.4.5. 查看文件内容

有多种命令可以查看文件内容，适用于不同的场景。

cat (concatenate) 将文件内容输出到终端，适合查看小文件：

```
1 # 查看文件内容
2 cat config.yaml
3 setting1: value1
4 setting2: value2
5
6 # 查看多个文件（连接输出）
7 cat file1.txt file2.txt
8
9 # -n 显示行号
10 cat -n script.sh
11     1  #!/bin/bash
12     2  echo "Hello"
13     3  exit 0
14
15 # 将多个文件合并为一个
16 cat part1.txt part2.txt part3.txt > combined.txt
```

对于大文件，**cat** 会一次性输出所有内容，刷屏而过，不实用。这时应该使用分页器。

less 是一个分页查看器，可以前后滚动浏览大文件：

```
1 less largefile.log
```

在 **less** 中的操作：

- 空格或 **f**：向下翻页
- **b**：向上翻页
- **j** 或 **↓**：向下一行
- **k** 或 **↑**：向上一行
- **g**：跳到文件开头
- **G**：跳到文件末尾
- **/pattern**：向下搜索 pattern
- **?pattern**：向上搜索 pattern
- **n**：跳到下一个搜索结果
- **N**：跳到上一个搜索结果
- **q**：退出

less 的一个重要特性是它不会一次性加载整个文件到内存，所以可以查看非常大的文件（如 GB 级的日志文件）。

还有一个类似的命令 **more**，功能比 **less** 少（只能向前翻页），但在某些最小化系统中可能只有 **more**。俗话说“less is more”——**less** 命令的名字就是对 **more** 的调侃。

head 显示文件的开头部分，默认前 10 行：

```
1 # 显示前 10 行
2 head file.txt
```

```

3
4 # 显示前 n 行
5 head -n 20 file.txt
6 head -20 file.txt      # 简写形式
7
8 # 显示前 n 个字节
9 head -c 100 file.txt

```

tail 显示文件的末尾部分，默认最后 10 行：

```

1 # 显示最后 10 行
2 tail file.txt
3
4 # 显示最后 n 行
5 tail -n 20 file.txt
6 tail -20 file.txt
7
8 # -f 实时追踪文件更新 (follow)
9 tail -f /var/log/syslog

```

tail -f 是查看日志的利器。它会持续监控文件，有新内容追加时立即显示。这对于实时监控程序日志非常有用：

```

1 # 在一个终端中运行程序
2 ./my_program
3
4 # 在另一个终端中监控日志
5 tail -f ~/my_program.log

```

按 **Ctrl+C** 停止 **tail -f**。

组合使用这些命令可以完成更复杂的任务：

```

1 # 查看文件的第 11-20 行
2 head -20 file.txt | tail -10
3
4 # 实时监控日志，只显示包含 "error" 的行
5 tail -f app.log | grep error
6
7 # 查看最近修改的文件的前几行
8 ls -t | head -1 | xargs head

```

1.3.4.6. 查找文件

当你不知道文件在哪里时，查找命令就派上用场了。

find 是最强大的文件查找命令，可以按名称、类型、大小、时间等各种条件搜索：

```

1 # 按名称查找（在当前目录及子目录中）
2 find . -name "*.cpp"
3 ./src/main.cpp
4 ./src/detector.cpp
5 ./test/test_detector.cpp
6
7 # 在指定目录中查找
8 find /home/alice/projects -name "CMakeLists.txt"
9
10 # -iname 忽略大小写
11 find . -iname "readme*"
12

```

```

13 # 按类型查找: f 文件, d 目录, l 链接
14 find . -type f -name "*.hpp" # 只找文件
15 find . -type d -name "build" # 只找目录
16
17 # 按大小查找: +大于, -小于
18 find . -size +100M # 大于 100MB 的文件
19 find . -size -1k # 小于 1KB 的文件
20
21 # 按时间查找: -mtime 修改时间, -atime 访问时间
22 find . -mtime -7 # 7 天内修改过的文件
23 find . -mtime +30 # 30 天前修改的文件
24
25 # 组合条件
26 find . -name "*.log" -size +10M -mtime +7
27
28 # -exec 对找到的文件执行命令
29 find . -name "*.o" -exec rm {} \; # 删除所有 .o 文件
30 find . -name "*.cpp" -exec wc -l {} \; # 统计每个 .cpp 文件的行数
31
32 # -delete 直接删除 (比 -exec rm 快, 但更危险)
33 find . -name "*.tmp" -delete

```

`find` 的 `-exec` 语法可能看起来奇怪: `{}` 是找到的文件名的占位符, `\;` 表示命令结束 (分号需要转义)。更高效的变体是用 `+` 代替 `\;`, 它会将多个文件名一起传给命令, 减少命令调用次数:

```

1 find . -name "*.cpp" -exec wc -l {} +

```

`locate` 使用预建的数据库来查找文件, 比 `find` 快得多, 但可能不是最新的:

```

1 # 安装 locate (如果没有)
2 sudo apt install mlocate
3
4 # 更新数据库 (需要 sudo)
5 sudo updatedb
6
7 # 查找文件
8 locate opencv.hpp
9 /usr/include/opencv4/opencv2/opencv.hpp
10
11 # -i 忽略大小写
12 locate -i readme
13
14 # 只计数, 不显示结果
15 locate -c "*.py"

```

`locate` 适合快速查找系统文件, 但对于刚创建的文件可能找不到 (数据库通常每天自动更新一次)。你可以手动运行 `sudo updatedb` 来更新数据库。

`which` 查找命令的可执行文件位置 (在 `PATH` 中搜索):

```

1 which python3
2 /usr/bin/python3
3
4 which gcc
5 /usr/bin/gcc
6
7 which ros2
8 /opt/ros/humble/bin/ros2

```

`which` 只搜索 PATH 中的目录，告诉你当你输入某个命令时实际执行的是哪个文件。类似的命令还有 `whereis`（还会显示手册页和源码位置）和 `type`（还能识别别名和内置命令）。

1.3.4.7. 文件信息

有时你需要了解文件的更多信息，而不仅仅是内容。

`file` 命令识别文件的类型，它通过检查文件内容（而不是扩展名）来判断：

```
1 file main.cpp
2 main.cpp: C++ source, UTF-8 Unicode text
3
4 file /bin/ls
5 /bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
6 dynamically linked, ...
7
8 file image.png
9 image.png: PNG image data, 1920 x 1080, 8-bit/color RGBA, non-interlaced
10
11 file model.onnx
12 model.onnx: data
13
14 file archive.tar.gz
15 archive.tar.gz: gzip compressed data, ...
```

这在处理没有扩展名或扩展名错误的文件时特别有用。

`stat` 显示文件的详细元数据：

```
1 stat README.md
2 File: README.md
3 Size: 1234          Blocks: 8          IO Block: 4096   regular file
4 Device: 801h/2049d Inode: 1234567       Links: 1
5 Access: (0644/-rw-r--r--)  Uid: ( 1000/   alice)   Gid: ( 1000/   alice)
6 Access: 2024-01-15 10:00:00.000000000 +0800
7 Modify: 2024-01-15 09:30:00.000000000 +0800
8 Change: 2024-01-15 09:30:00.000000000 +0800
9 Birth: 2024-01-10 08:00:00.000000000 +0800
```

这显示了文件大小、inode 号、权限、所有者、三种时间戳（访问时间、修改时间、状态改变时间）和创建时间（如果文件系统支持）。

`du`（disk usage）显示文件或目录占用的磁盘空间：

```
1 # 显示当前目录的大小
2 du -sh .
3 256M  .
4
5 # 显示各子目录的大小
6 du -sh */
7 12M   build/
8 4.0K  config/
9 8.0K  include/
10 200M  models/
11 32M   src/
12
13 # 显示所有文件和目录的大小
14 du -ah
15
16 # 找出最大的目录
17 du -sh */ | sort -rh | head -5
18
```

```
19 # -d 限制深度
20 du -h -d 1
```

`-s` 表示汇总 (summary)，只显示总大小；`-h` 表示人类可读格式。

`df` (disk free) 显示文件系统的磁盘空间使用情况：

```
1 df -h
2 Filesystem      Size  Used Avail Use% Mounted on
3 /dev/sda1       100G   45G   50G   48% /
4 /dev/sda2       500G  200G  275G   43% /home
5 tmpfs           7.8G    0    7.8G    0% /dev/shm
```

`df` 显示的是整个分区的使用情况，而 `du` 显示的是特定目录的大小。当磁盘空间不足时，先用 `df` 看哪个分区满了，再用 `du` 找出哪个目录占用最多空间。

1.3.4.8. 创建链接

链接让一个文件可以有多个名称或路径。我们在上一节介绍了链接的概念，这里讲解如何创建它们。

`ln` (link) 命令创建链接，默认创建硬链接，加 `-s` 选项创建符号链接（软链接）：

```
1 # 创建符号链接（最常用）
2 ln -s /opt/ros/humble ~/ros
3
4 # 验证链接
5 ls -l ~/ros
6 lrwxrwxrwx 1 alice alice 15 Jan 15 10:00 /home/alice/ros -> /opt/ros/humble
7
8 # 现在可以用 ~/ros 代替 /opt/ros/humble
9 source ~/ros/setup.bash
```

符号链接的常见用途：

```
1 # 为常用的长路径创建短名称
2 ln -s ~/projects/robomaster_vision_2024 ~/rmv
3
4 # 切换不同版本的配置
5 ln -s config_competition.yaml config.yaml
6 # 需要切换时
7 ln -sf config_debug.yaml config.yaml # -f 强制覆盖现有链接
8
9 # 让程序找到库
10 sudo ln -s /usr/local/cuda-11.8 /usr/local/cuda
```

创建硬链接（不常用，但有时有用）：

```
1 # 创建硬链接
2 ln original.txt hardlink.txt
3
4 # 两个文件共享相同的内容
5 echo "new content" >> original.txt
6 cat hardlink.txt
7 ... 新内容也出现 ...
```

记住硬链接的限制：不能链接目录，不能跨文件系统。符号链接没有这些限制，所以更常用。

1.3.4.9. 压缩与解压

处理压缩文件是日常工作的一部分——下载的软件包、备份的文件、分享的资料经常是压缩格式的。

tar (tape archive) 是最常用的归档工具。它本身只是打包，不压缩，但通常与压缩工具一起使用：

```
1 # 创建归档（不压缩）
2 tar cvf archive.tar dir/
3 # c = create 创建
4 # v = verbose 显示过程
5 # f = file 指定文件名
6
7 # 创建 gzip 压缩的归档（最常用）
8 tar cvzf archive.tar.gz dir/
9 # z = gzip 压缩
10
11 # 创建 bzip2 压缩的归档（压缩率更高但更慢）
12 tar cvjf archive.tar.bz2 dir/
13 # j = bzip2 压缩
14
15 # 创建 xz 压缩的归档（压缩率最高）
16 tar cvJf archive.tar.xz dir/
17 # J = xz 压缩
```

解压归档：

```
1 # 解压 tar.gz
2 tar xvzf archive.tar.gz
3 # x = extract 解压
4
5 # 解压 tar.bz2
6 tar xvjf archive.tar.bz2
7
8 # 解压 tar.xz
9 tar xvJf archive.tar.xz
10
11 # 解压到指定目录
12 tar xvzf archive.tar.gz -C /path/to/destination/
13
14 # 查看归档内容（不解压）
15 tar tvf archive.tar.gz
16 # t = list 列出内容
```

现代版本的 tar 可以自动检测压缩格式，所以解压时可以省略 z/j/J 选项：

```
1 tar xvf archive.tar.gz      # 自动检测 gzip
2 tar xvf archive.tar.xz     # 自动检测 xz
```

gzip 和 **gunzip** 用于压缩和解压单个文件：

```
1 # 压缩文件（原文件会被替换为 .gz 文件）
2 gzip largefile.log
3 ls
4 largefile.log.gz
5
6 # 解压
7 gunzip largefile.log.gz
8 # 或
9 gzip -d largefile.log.gz
```

```

10
11 # 保留原文件
12 gzip -k largefile.log
13
14 # 查看压缩文件内容（不解压）
15 zcat largefile.log.gz
16 zless largefile.log.gz      # 分页查看
17 zgrep "error" largefile.log.gz # 在压缩文件中搜索

```

`zip` 和 `unzip` 处理 ZIP 格式，这是 Windows 世界的标准格式：

```

1 # 创建 zip 归档
2 zip -r archive.zip dir/
3 # -r = recursive 递归
4
5 # 解压 zip
6 unzip archive.zip
7
8 # 解压到指定目录
9 unzip archive.zip -d /path/to/destination/
10
11 # 查看 zip 内容
12 unzip -l archive.zip

```

各种压缩格式的比较：

- `.tar.gz` 或 `.tgz`：gzip 压缩，最常用，平衡了压缩率和速度
- `.tar.bz2` 或 `.tbz2`：bzip2 压缩，压缩率更高，但压缩/解压更慢
- `.tar.xz`：xz 压缩，压缩率最高，但最慢
- `.zip`：跨平台兼容性最好，Windows 原生支持

在 RoboMaster 开发中，你会经常下载 `.tar.gz` 格式的源码包，备份项目时也通常用这个格式：

```

1 # 下载并解压 OpenCV 源码
2 wget https://github.com/opencv/opencv/archive/4.8.0.tar.gz
3 tar xzf 4.8.0.tar.gz
4
5 # 备份项目
6 tar czf rm_vision_backup_$(date +%Y%m%d).tar.gz rm_vision/

```

最后一个命令使用了命令替换 `$(date +%Y%m%d)` 来生成带日期的文件名，如 `rm_vision_backup_20240115.tar.gz`。

文件与目录操作是 Linux 使用的基本功。这些命令你会每天使用，熟练程度直接影响工作效率。不需要一次记住所有选项——记往常用的几个，其他的需要时查帮助文档。随着实践的积累，这些命令会变成条件反射，让你在命令行中如鱼得水。

1.3.5. 文本编辑与处理

在 Linux 的世界里，文本无处不在。配置文件是文本，源代码是文本，日志是文本，脚本是文本。掌握文本编辑和处理的技能，就掌握了 Linux 系统管理和开发的核心能力。本节将介绍两种命令行文本编辑器——适合新手的 nano 和功能强大的 vim，以及一系列文本处理工具——grep 搜索、sed 替换、awk 分析、sort 排序等。这些工具组合起来，可以完成几乎任何文本处理任务。

1.3.5.1. nano：新手友好的编辑器

如果你刚接触 Linux，需要在命令行中编辑文件，nano 是最友好的选择。它简单直观，屏幕底部显示常用快捷键，不需要记忆复杂的命令。

启动 nano 编辑文件：

```
1 nano filename.txt
```

如果文件不存在，nano 会创建一个新文件。打开后，你会看到一个简洁的界面：文件内容在中央，底部是快捷键提示。`^` 符号表示 Ctrl 键，所以 `^X` 表示 `Ctrl+X`。

在 nano 中，你可以直接开始输入文本，就像在普通的文本编辑器中一样。方向键移动光标，Backspace 删除字符，一切都很直观。

最常用的快捷键包括：

- `Ctrl+O`：保存文件（Write Out）。按下后会提示确认文件名，按 `Enter` 确认。
- `Ctrl+X`：退出 nano。如果文件有未保存的修改，会提示是否保存。
- `Ctrl+K`：剪切当前行。
- `Ctrl+U`：粘贴剪切的内容。
- `Ctrl+W`：搜索文本。输入要搜索的内容，按 `Enter` 跳转到第一个匹配处。`Ctrl+W` 再次按下可继续搜索下一个。
- `Ctrl+\：搜索并替换。`
- `Ctrl+G`：显示帮助信息。

一个典型的编辑流程是：打开文件，进行修改，`Ctrl+O` 保存，`Ctrl+X` 退出。如果修改后想放弃更改，直接 `Ctrl+X`，在提示保存时选择 `N`。

nano 的优点是上手零门槛，缺点是功能相对简单，对于大量编辑工作效率不高。它非常适合快速编辑配置文件、修改几行代码这样的小任务。当你 SSH 到服务器上需要改个配置时，nano 是最省心的选择。

```
1 # 编辑 SSH 配置
2 sudo nano /etc/ssh/sshd_config
3
4 # 编辑 bashrc
5 nano ~/.bashrc
6
7 # 快速创建并编辑新文件
8 nano notes.txt
```

1.3.5.2. vim：高效但需要学习的编辑器

vim (Vi IMproved) 是 Linux 世界最著名的编辑器，以其强大的功能和陡峭的学习曲线闻名。几乎每个 Linux 系统都预装了 vim 或其前身 vi，所以掌握基本操作是很有价值的。而一旦熟练，vim 的编辑效率是其他编辑器难以企及的。

vim 与其他编辑器最大的不同是它的“模式”设计。vim 有多种模式，每种模式下按键的含义不同：

- **普通模式 (Normal mode)**：默认模式，用于导航和执行命令。按键是命令而不是输入字符。
- **插入模式 (Insert mode)**：用于输入文本，此时 vim 表现得像普通编辑器。
- **命令模式 (Command mode)**：用于执行保存、退出等命令，以 `:` 开始。
- **可视模式 (Visual mode)**：用于选择文本。

启动 vim:

```
1 vim filename.txt
```

打开文件后，你处于普通模式。此时按字母键不会输入字符，而是执行命令。这是新手最困惑的地方——“我怎么什么都输入不了？”

要输入文本，需要先进入插入模式。最常用的方式是按 **i**（在光标前插入）。进入插入模式后，左下角会显示 **-- INSERT --**，此时可以正常输入文本。编辑完成后，按 **Esc** 返回普通模式。

保存和退出需要进入命令模式。在普通模式下按 **:**，左下角会出现冒号等待输入命令：

- **:w**：保存（write）
- **:q**：退出（quit）
- **:wq** 或 **:x**：保存并退出
- **:q!**：不保存强制退出（放弃所有修改）
- **:wq!**：强制保存并退出（用于只读文件，需要权限）

一个最基本的 vim 使用流程：

```
1 vim file.txt    → 打开文件，处于普通模式
2 i              → 进入插入模式
3 (输入文本)
4 Esc            → 返回普通模式
5 :wq            → 保存并退出
```

这几个操作足以应付基本的编辑需求。但 vim 的威力在于普通模式下的丰富命令。

普通模式下的移动命令：

- **h**、**j**、**k**、**l**：左、下、上、右（可以用方向键代替，但原生玩家用这四个键）
- **w**：移动到下一个单词开头
- **b**：移动到上一个单词开头
- **0**：移动到行首
- **\$**：移动到行尾
- **gg**：移动到文件开头
- **G**：移动到文件末尾
- **数字G**：移动到指定行，如 **10G** 跳到第 10 行

普通模式下的编辑命令：

- **x**：删除光标处的字符
- **dd**：删除（剪切）整行
- **yy**：复制整行
- **p**：粘贴到光标后
- **u**：撤销
- **Ctrl+R**：重做
- **数字+命令**：重复命令，如 **5dd** 删除 5 行

进入插入模式的多种方式：

- **i**：在光标前插入
- **a**：在光标后插入（append）
- **I**：在行首插入

- **A**：在行尾插入
- **o**：在下方新建一行并插入
- **O**：在上方新建一行并插入

搜索：

- **/pattern**：向下搜索 pattern
- **?pattern**：向上搜索
- **n**：跳到下一个匹配
- **N**：跳到上一个匹配

替换（在命令模式下）：

```
1 :s/old/new/      " 替换当前行第一个匹配
2 :s/old/new/g     " 替换当前行所有匹配
3 :%s/old/new/g    " 替换文件中所有匹配
4 :%s/old/new/gc   " 替换所有，每次询问确认
```

vim 的配置文件是 `~/.vimrc`，可以在这里设置各种选项：

```
1 " 显示行号
2 set number
3
4 " 语法高亮
5 syntax on
6
7 " 搜索高亮
8 set hlsearch
9
10 " 自动缩进
11 set autoindent
12
13 " Tab 宽度
14 set tabstop=4
15 set shiftwidth=4
16 set expandtab
17
18 " 显示光标位置
19 set ruler
```

vim 的学习曲线确实陡峭，但它的设计哲学——通过组合简单命令完成复杂操作——一旦掌握会非常高效。比如 `d2w` 表示“删除 2 个单词”，`y$` 表示“复制到行尾”，`ci"` 表示“修改引号内的内容”。这种命令组合的方式让 vim 用户可以在不离开键盘主区的情况下快速编辑。

对于 RoboMaster 开发者，建议至少掌握 vim 的基本操作。当你 SSH 到机器人的计算机上、或者在没有图形界面的环境中工作时，vim 可能是唯一的选择。即使你主要使用 VS Code 或其他 IDE，了解 vim 也是有价值的——很多工具（如 Git）默认使用 vim 编辑提交信息。

如果你想深入学习 vim，可以运行 `vimtutor` 命令，它提供了一个交互式的教程：

```
1 vimtutor
```

1.3.5.3. grep：文本搜索利器

grep（Global Regular Expression Print）是最常用的文本搜索工具。它在文件中搜索匹配特定模式的行，并将这些行输出。

基本用法：

```

1 # 在文件中搜索字符串
2 grep "error" logfile.txt
3 [所有包含 error 的行]
4
5 # 在多个文件中搜索
6 grep "TODO" *.cpp
7 main.cpp:42: // TODO: implement this
8 detector.cpp:100: // TODO: optimize performance
9
10 # -i 忽略大小写
11 grep -i "error" logfile.txt
12 Error: connection failed
13 ERROR: timeout
14 error: invalid input
15
16 # -n 显示行号
17 grep -n "error" logfile.txt
18 42:error: connection failed
19 157:error: timeout

```

递归搜索是开发中最常用的功能。 `-r` 选项让 `grep` 搜索目录下的所有文件：

```

1 # 在当前目录及子目录中搜索
2 grep -r "ArmorDetector" .
3 ./src/detector.cpp:class ArmorDetector {
4 ./include/detector.hpp:class ArmorDetector;
5 ./test/test_detector.cpp:TEST(ArmorDetector, Basic) {
6
7 # -r 配合其他选项
8 grep -rn "ArmorDetector" src/
9 src/detector.cpp:15:class ArmorDetector {
10 src/main.cpp:42: ArmorDetector detector;
11
12 # --include 只搜索特定类型的文件
13 grep -rn "TODO" --include="*.cpp" --include="*.hpp" .
14
15 # --exclude 排除特定文件
16 grep -rn "password" --exclude="*.log" .
17
18 # --exclude-dir 排除目录
19 grep -rn "config" --exclude-dir=build --exclude-dir=.git .

```

上下文显示让你能看到匹配行的周围内容：

```

1 # -B 显示匹配行之前的 n 行 (Before)
2 grep -B 2 "error" logfile.txt
3
4 # -A 显示匹配行之后的 n 行 (After)
5 grep -A 3 "error" logfile.txt
6
7 # -C 显示前后各 n 行 (Context)
8 grep -C 2 "error" logfile.txt

```

反向匹配显示不包含模式的行：

```

1 # -v 反向匹配
2 grep -v "debug" logfile.txt # 显示不含 debug 的行
3
4 # 排除注释行
5 grep -v "^#" config.txt # ^ 表示行首
6 grep -v "//" code.cpp

```

grep 支持正则表达式, `-E` 选项启用扩展正则表达式 (或使用 `egrep`):

```
1 # 基本正则表达式
2 grep "error.*failed" logfile.txt # . 匹配任意字符, * 匹配零个或多个
3
4 # -E 扩展正则表达式
5 grep -E "error|warning|fatal" logfile.txt # | 表示或
6 grep -E "[0-9]{3}\.[0-9]{3}" data.txt # 匹配 xxx.xxx 格式的数字
7
8 # 常用正则模式
9 grep "^#" config.txt # 以 # 开头的行
10 grep "\.cpp$" filelist # 以 .cpp 结尾的行
11 grep "^$" file.txt # 空行
12 grep -v "^$" file.txt # 非空行
```

grep 经常与其他命令通过管道组合使用:

```
1 # 在命令输出中搜索
2 ps aux | grep python
3 alice 12345 0.5 1.2 123456 78900 ? S 10:00 0:30 python3 app.py
4
5 # 在历史命令中搜索
6 history | grep "git push"
7
8 # 统计匹配的行数
9 grep -c "error" logfile.txt
10 42
11
12 # 只显示匹配的文件名
13 grep -l "main" *.cpp
14 main.cpp
15 app.cpp
16
17 # 实时监控日志中的错误
18 tail -f app.log | grep --line-buffered "error"
```

`--line-buffered` 在管道中使用 grep 时很重要, 它让 grep 逐行输出而不是等待缓冲区满。

1.3.5.4. sed: 流式文本编辑

`sed` (Stream Editor) 是一个强大的文本转换工具。它逐行读取输入, 应用指定的编辑命令, 然后输出结果。`sed` 最常用的功能是文本替换。

基本替换语法:

```
1 # 替换每行第一个匹配
2 sed 's/old/new/' file.txt
3 # s = substitute 替换命令
4 # old = 要查找的模式
5 # new = 替换成的内容
6
7 # 替换所有匹配 (g = global)
8 sed 's/old/new/g' file.txt
9
10 # 替换并忽略大小写 (i = case insensitive)
11 sed 's/error/ERROR/gi' file.txt
```

`sed` 默认将结果输出到标准输出, 原文件不变。要直接修改文件, 使用 `-i` 选项:

```
1 # 直接修改文件 (危险, 建议先备份)
```

```

2 sed -i 's/old/new/g' file.txt
3
4 # 创建备份后修改
5 sed -i.bak 's/old/new/g' file.txt
6 # 原文件被修改, 备份保存为 file.txt.bak

```

sed 使用正则表达式, 可以进行复杂的模式匹配:

```

1 # 删除行首空白
2 sed 's/^[ \t]*//' file.txt
3
4 # 删除行尾空白
5 sed 's/[ \t]*$//' file.txt
6
7 # 删除空行
8 sed '/^$/d' file.txt
9 # d = delete 删除命令
10
11 # 删除包含特定模式的行
12 sed '/pattern/d' file.txt
13
14 # 删除注释行
15 sed '/^#/d' config.txt
16 sed '/^\s*\/d' code.cpp

```

限定替换范围:

```

1 # 只替换第 3 行
2 sed '3s/old/new/' file.txt
3
4 # 替换第 3 到第 5 行
5 sed '3,5s/old/new/g' file.txt
6
7 # 替换第 10 行到最后一行
8 sed '10,$s/old/new/g' file.txt
9
10 # 替换匹配某模式的行
11 sed '/error/s/old/new/g' file.txt

```

sed 的实际应用场景:

```

1 # 批量修改代码中的函数名
2 sed -i 's/oldFunction/newFunction/g' *.cpp
3
4 # 修改配置文件中的参数
5 sed -i 's/port=8080/port=9090/' config.ini
6
7 # 给每行添加前缀
8 sed 's/^/PREFIX: /' file.txt
9
10 # 给每行添加后缀
11 sed 's$/ SUFFIX/' file.txt
12
13 # 在第 10 行后插入新行
14 sed '10a\This is a new line' file.txt
15 # a = append 追加
16
17 # 在第 10 行前插入新行
18 sed '10i\This is a new line' file.txt
19 # i = insert 插入
20
21 # 替换配置文件中的值 (处理特殊字符)

```

```
22 sed -i "s|old/path|new/path|g" config.txt
23 # 使用 | 作为分隔符, 避免与路径中的 / 冲突
```

sed 脚本可以包含多个命令:

```
1 # 多个命令用 -e 分隔
2 sed -e 's/foo/bar/g' -e 's/baz/qux/g' file.txt
3
4 # 或用分号分隔
5 sed 's/foo/bar/g; s/baz/qux/g' file.txt
6
7 # 从文件读取 sed 命令
8 sed -f script.sed file.txt
```

1.3.5.5. awk: 强大的文本分析工具

awk 是一个完整的文本处理语言, 特别擅长处理结构化的文本数据 (如 CSV、日志文件)。它逐行读取输入, 将每行分割成字段, 然后对字段进行处理。

awk 的基本语法是 `awk 'pattern { action }' file`。如果某行匹配 pattern, 就执行 action。

```
1 # 打印每行 (相当于 cat)
2 awk '{ print }' file.txt
3
4 # 打印第一个字段 (默认以空白分隔)
5 awk '{ print $1 }' file.txt
6
7 # 打印第一和第三个字段
8 awk '{ print $1, $3 }' file.txt
9
10 # $0 表示整行, $1, $2, ... 表示各个字段, $NF 表示最后一个字段
11 awk '{ print $NF }' file.txt # 打印最后一个字段
12 awk '{ print $(NF-1) }' file.txt # 打印倒数第二个字段
```

处理 CSV 或其他分隔符的文件:

```
1 # -F 指定字段分隔符
2 awk -F',' '{ print $1, $3 }' data.csv
3
4 # 冒号分隔 (如 /etc/passwd)
5 awk -F':' '{ print $1, $7 }' /etc/passwd
6 # 输出用户名和 shell
7
8 # 多字符分隔符
9 awk -F '::' '{ print $1 }' file.txt
```

条件过滤:

```
1 # 只打印匹配模式的行
2 awk '/error/ { print }' logfile.txt
3
4 # 条件判断
5 awk '$3 > 100 { print $1, $3 }' data.txt # 第三字段大于 100 的行
6
7 # 组合条件
8 awk '$3 > 100 && $2 == "active" { print }' data.txt
9
10 # 打印行号
11 awk '{ print NR, $0 }' file.txt # NR = Number of Records (行号)
```

awk 内置变量:

- `$0`: 整行内容
- `$1, $2, ...`: 各字段
- `NR`: 当前行号
- `NF`: 当前行的字段数
- `FS`: 字段分隔符
- `OFS`: 输出字段分隔符

格式化输出:

```
1 # printf 格式化
2 awk '{ printf "%-10s %5d\n", $1, $2 }' data.txt
3
4 # 添加表头
5 awk 'BEGIN { print "Name\tScore" } { print $1 "\t" $2 }' data.txt
6 # BEGIN 在处理任何行之前执行
7
8 # 添加表尾/汇总
9 awk '{ sum += $2 } END { print "Total:", sum }' data.txt
10 # END 在处理完所有行之后执行
```

实际应用示例:

```
1 # 统计日志中各类错误的数量
2 awk '/error/ { count[$4]++ } END { for (type in count) print type,
   count[type] }' app.log
3
4 # 计算文件中数字的平均值
5 awk '{ sum += $1; count++ } END { print sum/count }' numbers.txt
6
7 # 提取 CSV 特定列并计算
8 awk -F',' '{ sum += $3 } END { print "Average:", sum/NR }' data.csv
9
10 # 过滤并格式化 ps 输出
11 ps aux | awk '$3 > 1.0 { printf "%-10s %5.1f%\n", $11, $3 }'
12 # 显示 CPU 使用率超过 1% 的进程
13
14 # 分析 ROS 2 日志
15 awk '/\[ERROR\]/ { print $1, $2, $0 }' ros2.log
```

awk 是一个完整的编程语言, 支持变量、数组、循环、条件、函数等。对于复杂的文本分析任务, awk 比管道组合 grep、sed、cut 更加高效和清晰。

1.3.5.6. 排序、去重与统计

`sort` 对文本行进行排序:

```
1 # 默认按字母顺序排序
2 sort names.txt
3
4 # -n 按数字排序
5 sort -n numbers.txt
6
7 # -r 反向排序
8 sort -r names.txt
9 sort -rn numbers.txt # 数字降序
10
11 # -k 按指定字段排序
12 sort -k 2 data.txt # 按第二字段排序
```



```

13 sort -k 2 -n data.txt # 按第二字段数字排序
14 sort -k 2,2 -k 1,1 data.txt # 先按第二字段，相同则按第一字段
15
16 # -t 指定字段分隔符
17 sort -t',' -k 3 -n data.csv # CSV 按第三列数字排序
18
19 # -u 排序并去重
20 sort -u names.txt
21
22 # 查找最大的几个文件
23 du -sh * | sort -rh | head -5

```

uniq 去除相邻的重复行（通常与 sort 配合使用）：

```

1 # 去重（必须先排序）
2 sort names.txt | uniq
3
4 # -c 统计每行出现的次数
5 sort names.txt | uniq -c
6     3 Alice
7     1 Bob
8     2 Charlie
9
10 # -d 只显示重复的行
11 sort names.txt | uniq -d
12
13 # -u 只显示不重复的行
14 sort names.txt | uniq -u
15
16 # 统计最常出现的项
17 sort items.txt | uniq -c | sort -rn | head -10

```

wc（word count）统计行数、单词数、字符数：

```

1 # 默认显示行数、单词数、字节数
2 wc file.txt
3    100    500   3000 file.txt
4
5 # -l 只显示行数
6 wc -l file.txt
7    100 file.txt
8
9 # -w 只显示单词数
10 wc -w file.txt
11
12 # -c 只显示字节数
13 wc -c file.txt
14
15 # -m 显示字符数（处理多字节字符）
16 wc -m file.txt
17
18 # 统计多个文件
19 wc -l *.cpp
20    100 main.cpp
21    200 detector.cpp
22    150 tracker.cpp
23    450 total
24
25 # 统计目录中的文件数
26 ls | wc -l
27
28 # 统计代码行数

```

```
29 find . -name "*.cpp" -exec cat {} + | wc -l
```

组合使用这些工具可以完成复杂的分析任务：

```
1 # 统计日志中各 IP 的访问次数
2 awk '{ print $1 }' access.log | sort | uniq -c | sort -rn | head -10
3
4 # 找出代码中使用最多的函数
5 grep -oh '\b[a-zA-Z_][a-zA-Z0-9_]*(' *.cpp | sort | uniq -c | sort -rn |
  head -20
6
7 # 统计各类型文件的数量
8 find . -type f | sed 's/.*\./ /' | sort | uniq -c | sort -rn
```

1.3.5.7. diff: 文件比较

`diff` 比较两个文件的差异，这对于代码审查、配置比较、版本对比非常有用。

```
1 # 基本比较
2 diff file1.txt file2.txt
3 2c2
4 < old line
5 ---
6 > new line
7 5a6
8 > added line
```

输出格式说明：< 表示第一个文件的内容，> 表示第二个文件的内容。2c2 表示第 2 行有变化 (change)，5a6 表示在第 5 行后添加了内容 (add)，3d2 表示第 3 行被删除 (delete)。

更友好的输出格式：

```
1 # -u 统一格式 (unified diff), 更易读, 也是 Git 使用的格式
2 diff -u file1.txt file2.txt
3 --- file1.txt    2024-01-15 10:00:00
4 +++ file2.txt    2024-01-15 11:00:00
5 @@ -1,5 +1,6 @@
6   line 1
7   -old line
8   +new line
9   line 3
10  line 4
11  line 5
12  +added line
13
14 # -y 并排显示
15 diff -y file1.txt file2.txt
16 line 1                line 1
17 old line              | new line
18 line 3                line 3
19
20 # --color 彩色显示 (需要较新版本)
21 diff --color -u file1.txt file2.txt
```

比较目录：

```
1 # -r 递归比较目录
2 diff -r dir1/ dir2/
3
4 # -q 只显示哪些文件不同, 不显示具体差异
```

```
5 diff -rq dir1/ dir2/
6 Files dir1/file.txt and dir2/file.txt differ
7 Only in dir1/: extra.txt
```

忽略某些差异：

```
1 # -w 忽略所有空白差异
2 diff -w file1.txt file2.txt
3
4 # -b 忽略空白数量变化
5 diff -b file1.txt file2.txt
6
7 # -B 忽略空行差异
8 diff -B file1.txt file2.txt
9
10 # -i 忽略大小写
11 diff -i file1.txt file2.txt
```

实际应用：

```
1 # 比较配置文件的变化
2 diff -u config.old config.new > changes.patch
3
4 # 应用补丁
5 patch < changes.patch
6
7 # 比较两个版本的代码
8 diff -ru old_version/ new_version/ > upgrade.patch
9
10 # 在代码审查中查看修改
11 git diff HEAD~1 # Git 使用 diff 格式显示变更
```

`colordiff` 是 `diff` 的彩色版本，更易于阅读：

```
1 sudo apt install colordiff
2 colordiff -u file1.txt file2.txt
```

文本编辑与处理是 Linux 命令行的核心能力。这些工具——从简单的 `cat`、`grep` 到强大的 `sed`、`awk`——构成了一个完整的文本处理工具箱。它们遵循 Unix 哲学：每个工具做好一件事，通过管道组合完成复杂任务。掌握这些工具，你就能高效地处理日志分析、配置修改、代码搜索、数据提取等各种文本相关的任务。在 RoboMaster 开发中，这些技能会在调试日志、分析数据、批量修改代码等场景中反复用到。

1.3.6. 用户与权限管理

Linux 从诞生之初就是一个多用户操作系统。多个用户可以同时登录同一台机器，每个用户有自己的文件和进程，彼此隔离。这种设计来自 Unix 的时分系统传统——一台昂贵的计算机需要被多人共享使用。虽然今天的个人电脑通常只有一个使用者，但多用户模型仍然是 Linux 安全架构的基础。理解用户和权限的概念，不仅是系统管理的必备知识，也是理解为什么某些操作需要 `sudo`、为什么脚本需要 `chmod +x` 的关键。

1.3.6.1. 用户与用户组

在 Linux 中，每个用户都有一个唯一的用户名和用户 ID（UID）。用户信息存储在 `/etc/passwd` 文件中：

```

1 cat /etc/passwd
2 root:x:0:0:root:/root:/bin/bash
3 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
4 ...
5 alice:x:1000:1000:Alice:/home/alice:/bin/bash
6 bob:x:1001:1001:Bob:/home/bob:/bin/bash

```

每行代表一个用户，字段用冒号分隔：用户名、密码占位符（实际密码在 `/etc/shadow`）、UID、GID（主组 ID）、用户描述、家目录、登录 Shell。

UID 0 是特殊的，它属于 root 用户——系统的超级管理员，拥有无限制的权限。普通用户的 UID 通常从 1000 开始。系统服务使用的用户（如 `daemon`、`www-data`）通常有较小的 UID，且登录 Shell 设为 `/usr/sbin/nologin`，表示这些账户不能交互式登录。

用户组是用户的集合，用于简化权限管理。每个用户属于一个主组（primary group），还可以属于多个附加组（supplementary groups）。组信息存储在 `/etc/group` 中：

```

1 cat /etc/group
2 root:x:0:
3 sudo:x:27:alice
4 audio:x:29:alice,bob
5 video:x:44:alice,bob
6 alice:x:1000:
7 bob:x:1001:

```

用户组让权限管理更加灵活。比如，你可以创建一个 `robomaster` 组，把所有队员加入这个组，然后设置项目目录对这个组可读写。这样所有队员都能访问项目文件，而其他用户不能。

查看当前用户和组信息：

```

1 # 查看当前用户
2 whoami
3 alice
4
5 # 查看当前用户的 UID 和所属组
6 id
7 uid=1000(alice)
  gid=1000(alice) groups=1000(alice),27(sudo),29(audio),44(video)
8
9 # 查看其他用户的信息
10 id bob
11 uid=1001(bob) gid=1001(bob) groups=1001(bob),29(audio),44(video)
12
13 # 查看当前用户所属的所有组
14 groups
15 alice sudo audio video

```

用户和组的管理命令（需要 root 权限）：

```

1 # 创建新用户
2 sudo useradd -m -s /bin/bash newuser
3 # -m 创建家目录
4 # -s 指定登录 Shell
5
6 # 设置密码
7 sudo passwd newuser
8
9 # 更友好的创建用户命令（交互式）
10 sudo adduser newuser
11

```

```

12 # 删除用户
13 sudo userdel -r olduser
14 # -r 同时删除家目录
15
16 # 创建组
17 sudo groupadd robomaster
18
19 # 将用户添加到组
20 sudo usermod -aG robomaster alice
21 # -a 追加（不覆盖现有组）
22 # -G 指定附加组
23
24 # 从组中移除用户
25 sudo gpasswd -d alice robomaster

```

修改用户组后，需要重新登录才能生效。这是因为组信息在登录时读取，会话期间不会自动更新。

1.3.6.2. root 与 sudo

root 是 Linux 系统的超级用户，UID 为 0，拥有对系统的完全控制权。root 可以读写任何文件、终止任何进程、修改任何配置。这种无限制的权力是危险的——一个误操作就可能破坏整个系统。因此，现代 Linux 实践是：日常工作使用普通用户，只在需要时临时获取 root 权限。

sudo (superuser do) 允许普通用户以 root 身份执行单个命令。使用 sudo 时需要输入当前用户的密码（不是 root 密码），验证是你本人在操作：

```

1 # 以 root 身份执行命令
2 sudo apt update
3
4 # 第一次使用会提示输入密码
5 [sudo] password for alice:
6 ...
7
8 # 短时间内再次使用不需要重新输入密码（默认 15 分钟）
9 sudo apt upgrade

```

sudo 的优势在于审计和控制。系统会记录谁在什么时候用 sudo 执行了什么命令（日志在 `/var/log/auth.log`），这对于安全审计很重要。管理员还可以精细控制哪些用户可以用 sudo 执行哪些命令，配置文件是 `/etc/sudoers`（用 **visudo** 编辑，它会检查语法错误）。

并非所有用户都能使用 sudo。在 Ubuntu 中，用户必须属于 **sudo** 组才能使用 sudo。安装系统时创建的第一个用户会自动加入这个组。

```

1 # 检查用户是否在 sudo 组
2 groups alice
3 alice sudo audio video
4
5 # 将用户加入 sudo 组（需要已有 sudo 权限的用户执行）
6 sudo usermod -aG sudo newuser

```

有时你需要以 root 身份执行多个命令，每次都输入 sudo 很繁琐。可以启动一个 root shell：

```

1 # 启动 root shell
2 sudo -i
3 root@hostname:~#
4 # 提示符从 $ 变成 #，表示现在是 root
5

```

```

6 # 完成后退出
7 root@hostname:~# exit
8 $
9
10 # 或者用 sudo -s 保持当前环境
11 sudo -s

```

但要谨慎使用 root shell——在 root 权限下，`rm -rf /` 这样的命令会真的删除整个系统，没有任何阻拦。一个好习惯是只在必要时使用 `sudo`，完成后立即退出 root shell。

另一个常见场景是以 root 身份编辑文件：

```

1 # 编辑系统配置文件
2 sudo nano /etc/ssh/sshd_config
3
4 # 或者
5 sudo vim /etc/hosts

```

有些用户会犯一个错误：用 `sudo` 运行图形程序或在家目录创建文件。这会导致文件的所有者变成 root，之后普通用户无法修改：

```

1 # 错误做法
2 sudo vim ~/my_script.sh
3 # 现在 my_script.sh 的所有者是 root
4
5 # 正确做法：只在必要时用 sudo
6 vim ~/my_script.sh

```

1.3.6.3. 文件权限：rwx 模型

Linux 的文件权限模型简单而有效。每个文件有三组权限，分别针对：所有者 (owner)、所属组 (group)、其他人 (others)。每组权限包含三种：读 (read)、写 (write)、执行 (execute)。

用 `ls -l` 查看文件权限：

```

1 ls -l
2 -rw-r--r-- 1 alice alice 1234 Jan 15 10:00 document.txt
3 -rwxr-xr-x 1 alice alice 5678 Jan 15 10:00 script.sh
4 drwxr-xr-x 2 alice alice 4096 Jan 15 10:00 mydir
5 lrwxrwxrwx 1 alice alice 10 Jan 15 10:00 link -> target

```

第一列就是权限字符串，让我们详细解读 `-rwxr-xr-x`：

- 第 1 个字符：文件类型。- 普通文件，`d` 目录，`l` 符号链接，`c` 字符设备，`b` 块设备。
- 第 2-4 个字符 (`rwx`)：所有者权限。这里是 `rwx`，表示所有者可以读、写、执行。
- 第 5-7 个字符 (`r-x`)：所属组权限。这里是 `r-x`，表示组成员可以读、执行，但不能写。
- 第 8-10 个字符 (`r-x`)：其他人权限。这里也是 `r-x`，表示其他人可以读、执行，但不能写。

权限字符串的含义：

- `r` (read, 值为 4):
 - 对于文件：可以读取内容
 - 对于目录：可以列出目录内容 (`ls`)
- `w` (write, 值为 2):
 - 对于文件：可以修改内容
 - 对于目录：可以在目录中创建、删除、重命名文件
- `x` (execute, 值为 1):

- 对于文件：可以作为程序执行
- 对于目录：可以进入目录 (cd)
- - : 没有该权限

目录权限的理解需要特别注意。要访问目录中的文件，你需要对目录有 `x` 权限(进入目录)。要列出目录内容，需要 `r` 权限。要在目录中创建或删除文件，需要 `w` 权限。一个常见的困惑是：即使你对某个文件有完全权限，如果对其所在目录没有 `x` 权限，你也无法访问该文件。

1.3.6.4. 数字表示法

除了 `rwX` 字符表示，权限还可以用数字表示。每种权限对应一个数值：`r=4`，`w=2`，`x=1`。一组权限的数字是各权限值的和：

- `rwX` = $4+2+1 = 7$
- `rw-` = $4+2+0 = 6$
- `r-x` = $4+0+1 = 5$
- `r--` = $4+0+0 = 4$
- `---` = $0+0+0 = 0$

三组权限组成三位数：

- `rwXr-Xr-X` = 755
- `rw-r--r--` = 644
- `rwX-----` = 700
- `rw-rw-r--` = 664

常见的权限设置：

- **755**：所有者完全控制，其他人可读可执行。用于可执行程序、脚本、公开目录。
- **644**：所有者可读写，其他人只读。用于普通文件、配置文件。
- **700**：只有所有者可访问。用于私密目录，如 `~/ .ssh`。
- **600**：只有所有者可读写。用于私密文件，如 SSH 私钥。
- **777**：所有人完全控制。通常不推荐，安全隐患。
- **666**：所有人可读写。同样不推荐。

理解这些数字对于设置权限和理解文档很重要。当你看到“将权限设为 755”或 `chmod 644` 时，就知道这意味着什么。

1.3.6.5. chmod：修改权限

`chmod` (change mode) 命令用于修改文件权限。它有两种语法：符号模式和数字模式。数字模式直接指定完整的权限：

```
1 # 设置权限为 755 (rwXr-Xr-X)
2 chmod 755 script.sh
3
4 # 设置权限为 644 (rw-r--r--)
5 chmod 644 document.txt
6
7 # 设置目录权限为 700 (只有所有者可访问)
8 chmod 700 private_dir
9
10 # 递归修改目录及其内容
11 chmod -R 755 public_dir
```

符号模式更灵活，可以增加或删除特定权限：

```

1 # 语法: chmod [ugoa][+ -=][rwx] file
2 # u=user(所有者), g=group(组), o=others(其他), a=all(所有)
3 # +=添加, -=删除, ==设置
4
5 # 给所有者添加执行权限
6 chmod u+x script.sh
7
8 # 给所有人添加读权限
9 chmod a+r document.txt
10
11 # 删除其他人的写权限
12 chmod o-w file.txt
13
14 # 给组添加读写权限
15 chmod g+rw shared_file.txt
16
17 # 组合操作
18 chmod u+x,g+r,o-w file.txt
19
20 # 设置精确权限 (覆盖现有)
21 chmod u=rwx,g=rx,o=rx script.sh # 等同于 755

```

符号模式的优势是可以只修改需要改变的部分, 而不影响其他权限。比如 `chmod u+x` 只添加所有者的执行权限, 不会改变其他权限位。

1.3.6.6. chown: 修改所有者

`chown` (change owner) 命令修改文件的所有者和所属组。这个命令通常需要 root 权限。

```

1 # 修改所有者
2 sudo chown bob file.txt
3
4 # 修改所有者和组
5 sudo chown bob:developers file.txt
6
7 # 只修改组 (注意冒号在前)
8 sudo chown :developers file.txt
9 # 或者用 chgrp
10 sudo chgrp developers file.txt
11
12 # 递归修改目录及其内容
13 sudo chown -R alice:robomaster project/

```

一个常见场景是修复被 `sudo` 创建的文件:

```

1 # 误用 sudo 创建了文件
2 sudo touch important.txt
3 ls -l important.txt
4 -rw-r--r-- 1 root root 0 Jan 15 10:00 important.txt
5
6 # 现在普通用户无法修改
7 echo "content" >> important.txt
8 bash: important.txt: Permission denied
9
10 # 修复所有权
11 sudo chown alice:alice important.txt
12 echo "content" >> important.txt # 现在可以了

```

1.3.6.7. 为什么脚本需要 chmod +x

当你写了一个 Shell 脚本, 尝试运行时可能会遇到这样的错误:


```
1 ./my_script.sh
2 bash: ./my_script.sh: Permission denied
```

这是因为新创建的文件默认没有执行权限：

```
1 ls -l my_script.sh
2 -rw-r--r-- 1 alice alice 100 Jan 15 10:00 my_script.sh
```

注意权限是 `rw-r--r--` (644)，没有 `x`。文件虽然内容是可执行的脚本，但操作系统不允许执行它。这是一个安全特性——防止你不小心把数据文件当作程序运行。

解决方法就是添加执行权限：

```
1 chmod +x my_script.sh
2 ls -l my_script.sh
3 -rwxr-xr-x 1 alice alice 100 Jan 15 10:00 my_script.sh
4
5 ./my_script.sh
6 Hello, World!
```

`chmod +x` 是 `chmod a+x` 的简写，给所有用户添加执行权限。如果你只想让自己能执行，可以用 `chmod u+x`。

有一个绕过方法是显式调用解释器：

```
1 bash my_script.sh      # 不需要执行权限
2 python3 script.py      # 同样不需要
```

这种方式不需要文件有执行权限，因为实际执行的是 `bash` 或 `python3`，脚本只是作为输入文件。但这不是推荐的做法——添加执行权限是正确的方式，也让文件的用途更明确。

对于编译型程序（如 C++ 编译出的二进制文件），编译器通常会自动设置执行权限：

```
1 g++ main.cpp -o program
2 ls -l program
3 -rwxr-xr-x 1 alice alice 12345 Jan 15 10:00 program
4
5 ./program # 直接可以运行
```

1.3.6.8. 特殊权限位

除了基本的 `rwx` 权限，还有三个特殊权限位：SUID、SGID 和 Sticky Bit。它们在某些场景下很有用，但也带来安全风险，需要谨慎使用。

SUID (Set User ID)：当可执行文件设置了 SUID 位，任何用户执行该文件时，进程会以文件所有者的身份运行，而不是执行者的身份。最典型的例子是 `passwd` 命令：

```
1 ls -l /usr/bin/passwd
2 -rwsr-xr-x 1 root root 68208 Jan 15 10:00 /usr/bin/passwd
```

注意所有者权限中的 `s`（而不是 `x`）。普通用户运行 `passwd` 时，进程以 `root` 身份运行，因此能够修改 `/etc/shadow` 文件（存储密码的地方）。没有 SUID，普通用户无法修改自己的密码。

设置 SUID：

```
1 chmod u+s program
```

```
2 chmod 4755 program # 4 表示 SUID
```

SGID (Set Group ID)：类似 SUID，但是以文件所属组的身份运行。对于目录，SGID 有特殊含义：在该目录下创建的新文件会继承目录的组，而不是创建者的主组。这对于团队共享目录很有用：

```
1 # 创建共享目录
2 sudo mkdir /shared/project
3 sudo chown :robomaster /shared/project
4 sudo chmod g+s /shared/project
5
6 # 现在在这个目录下创建的文件都属于 robomaster 组
7 touch /shared/project/newfile.txt
8 ls -l /shared/project/newfile.txt
9 -rw-r--r-- 1 alice robomaster 0 Jan 15 10:00 newfile.txt
```

Sticky Bit：对于目录，设置 Sticky Bit 后，只有文件所有者、目录所有者或 root 才能删除或重命名目录中的文件，即使其他用户对目录有写权限。`/tmp` 目录就设置了 Sticky Bit：

```
1 ls -ld /tmp
2 drwxrwxrwt 10 root root 4096 Jan 15 10:00 /tmp
```

注意其他人权限中的 **t**（而不是 **x**）。所有用户都可以在 `/tmp` 中创建文件，但只能删除自己的文件，不能删除别人的。

```
1 # 设置 Sticky Bit
2 chmod +t directory
3 chmod 1777 directory # 1 表示 Sticky Bit
```

1.3.6.9. umask：默认权限

当你创建新文件或目录时，系统会使用一个默认权限。这个默认值由 `umask` 控制。`umask` 指定了要从完整权限中“屏蔽”掉的位。

```
1 umask
2 0022
3
4 # 对于文件，完整权限是 666（不包括执行位）
5 # 666 - 022 = 644 (rw-r--r--)
6
7 # 对于目录，完整权限是 777
8 # 777 - 022 = 755 (rwxr-xr-x)
```

常见的 `umask` 值：

- **022**：文件 644，目录 755。这是大多数系统的默认值，其他用户可读。
- **077**：文件 600，目录 700。更私密，其他用户无法访问。
- **002**：文件 664，目录 775。同组用户可写，适合团队协作。

修改 `umask`：

```
1 # 临时修改
2 umask 077
3
4 # 永久修改（添加到 ~/.bashrc）
5 echo "umask 077" >> ~/.bashrc
```

1.3.6.10. 实际场景中的权限管理

让我们看一些 RoboMaster 开发中常见的权限相关场景。

访问串口设备: USB 串口设备 (如 `/dev/ttyUSB0`) 默认只有 root 和 dialout 组可以访问:

```
1 ls -l /dev/ttyUSB0
2 crw-rw---- 1 root dialout 188, 0 Jan 15 10:00 /dev/ttyUSB0
3
4 # 普通用户无法访问
5 cat /dev/ttyUSB0
6 cat: /dev/ttyUSB0: Permission denied
7
8 # 解决方法: 将用户加入 dialout 组
9 sudo usermod -aG dialout alice
10 # 重新登录后生效
11
12 groups
13 alice dialout ...
14 cat /dev/ttyUSB0 # 现在可以了
```

共享项目目录: 团队成员需要共同编辑项目文件:

```
1 # 创建项目目录
2 sudo mkdir -p /home/shared/rm_vision
3 sudo chown :robomaster /home/shared/rm_vision
4 sudo chmod 2775 /home/shared/rm_vision # SGID + rwxrwxr-x
5
6 # 确保团队成员都在 robomaster 组
7 sudo usermod -aG robomaster alice
8 sudo usermod -aG robomaster bob
```

保护 SSH 密钥: SSH 对私钥权限有严格要求:

```
1 chmod 700 ~/.ssh
2 chmod 600 ~/.ssh/id_rsa
3 chmod 644 ~/.ssh/id_rsa.pub
4 chmod 644 ~/.ssh/known_hosts
5 chmod 600 ~/.ssh/config
6
7 # 如果权限不对, SSH 会拒绝使用密钥
8 @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
9 @                WARNING: UNPROTECTED PRIVATE KEY FILE!                @
10 @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
11 Permissions 0644 for '/home/alice/.ssh/id_rsa' are too open.
```

脚本和程序:

```
1 # 创建脚本后添加执行权限
2 vim run_vision.sh
3 chmod +x run_vision.sh
4 ./run_vision.sh
5
6 # 安装自己的程序到 ~/.local/bin
7 cp my_tool ~/.local/bin/
8 chmod 755 ~/.local/bin/my_tool
```

用户与权限管理是 Linux 安全的基础。理解这些概念后, 你就能明白为什么某些操作需要 `sudo`, 为什么某些文件无法访问, 以及如何正确设置权限让团队协作更顺畅。记住一个原则: 给予最小必要的权限。不要随便 `chmod 777`, 不要长期在 root shell 中工作, 谨慎使用 SUID。这些习惯会让你的系统更加安全。

1.3.7. 软件包管理

在 Windows 上安装软件，你需要找到官网、下载安装程序、运行安装向导、处理各种“下一步”和捆绑软件的选项。在 Linux 上，这一切被包管理器彻底改变了。一条命令就能安装软件，依赖自动解决，更新一键完成，卸载干净彻底。包管理器是 Linux 发行版的核心组件之一，理解它的工作原理和使用方法，是高效使用 Linux 的关键。本节以 Ubuntu 使用的 APT 包管理器为主，介绍软件的安装、更新、卸载，以及如何添加软件源和从源码编译安装。

1.3.7.1. 包管理器的概念

软件包 (package) 是将程序文件、库、配置、文档等打包在一起的归档文件。在 Debian/Ubuntu 系统上，软件包的格式是 `.deb`。包管理器负责安装、升级、卸载这些软件包，并自动处理包之间的依赖关系。

依赖是指一个软件需要其他软件才能运行。比如，一个图形程序可能依赖 Qt 库，一个 Python 程序依赖 Python 解释器。在没有包管理器的年代，安装软件需要手动下载并安装所有依赖，依赖又可能有自己的依赖，形成“依赖地狱”。包管理器自动分析依赖关系，一次性安装所有需要的包，大大简化了软件安装。

软件源 (repository) 是存放软件包的服务器。Ubuntu 官方维护了庞大的软件源，包含数万个软件包。当你运行 `apt install` 时，包管理器会从软件源下载并安装软件。软件源的配置决定了你能安装哪些软件，以及获取软件的速度（选择地理位置近的镜像可以加快下载）。

1.3.7.2. APT 基础操作

APT (Advanced Package Tool) 是 Ubuntu 和其他 Debian 系发行版的高级包管理工具。它提供了友好的命令行界面，是日常软件管理的主要工具。

更新软件包列表

在安装或升级软件之前，应该先更新本地的软件包列表。这个列表记录了软件源中有哪些软件、什么版本：

```
1 sudo apt update
2 Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
3 Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
4 ...
5 Reading package lists... Done
6 Building dependency tree... Done
7 42 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

`apt update` 只是更新列表，不会安装或升级任何软件。它告诉你有多少包可以升级。

升级已安装的软件

更新列表后，可以升级所有已安装的软件到最新版本：

```
1 sudo apt upgrade
2 Reading package lists... Done
3 Building dependency tree... Done
4 Calculating upgrade... Done
5 The following packages will be upgraded:
6 package1 package2 package3 ...
7 42 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
8 Need to get 50.0 MB of archives.
9 Do you want to continue? [Y/n] y
10 ...
```

APT 会显示将要升级哪些包、下载量、磁盘空间变化，并询问确认。按 **Y** 或直接回车确认。

如果某些升级需要删除或安装新包（比如内核升级），**apt upgrade** 会跳过它们。要进行更激进的升级，使用 **apt full-upgrade**：

```
1 sudo apt full-upgrade
```

这是完整的系统更新命令，通常在重大版本升级时使用。

安装软件

安装软件是最常用的操作：

```
1 # 安装单个软件
2 sudo apt install vim
3
4 # 安装多个软件
5 sudo apt install git cmake build-essential
6
7 # 安装时不询问确认（脚本中常用）
8 sudo apt install -y package_name
9
10 # 重新安装已安装的软件
11 sudo apt reinstall package_name
```

build-essential 是一个元包 (metapackage)，它本身不包含软件，但依赖一组编译工具 (gcc、g++、make 等)。安装它会自动安装所有这些工具，是 C/C++ 开发的必备。

卸载软件

卸载软件有两种方式：

```
1 # 卸载软件，保留配置文件
2 sudo apt remove package_name
3
4 # 完全卸载，包括配置文件
5 sudo apt purge package_name
6
7 # 卸载后清理不再需要的依赖
8 sudo apt autoremove
```

remove 和 **purge** 的区别在于是否删除配置文件。如果你打算将来重新安装并保留配置，用 **remove**；如果想彻底清除，用 **purge**。

autoremove 清理那些作为依赖被安装、但现在不再被任何已安装软件需要的包。定期运行它可以释放磁盘空间。

搜索软件

不确定软件包的确切名称？用 **apt search**：

```
1 apt search opencv
2 Sorting... Done
3 Full Text Search... Done
4 libopencv-dev/jammy 4.5.4+dfsg-9ubuntu4 amd64
5   development files for opencv
6
7 python3-opencv/jammy 4.5.4+dfsg-9ubuntu4 amd64
8   Python 3 bindings for OpenCV
9 ...
```

搜索结果可能很多，可以用 **grep** 过滤：

```
1 apt search opencv | grep -i "dev"
```

查看软件包信息

安装前想了解软件包的详细信息：

```
1 apt show libopencv-dev
2 Package: libopencv-dev
3 Version: 4.5.4+dfsg-9ubuntu4
4 Priority: optional
5 Section: universe/libdevel
6 Source: opencv
7 Origin: Ubuntu
8 Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
9 Bugs: https://bugs.launchpad.net/ubuntu/+filebug
10 Installed-Size: 5,391 kB
11 Depends: libopencv-calib3d-dev, libopencv-contrib-dev, ...
12 Suggests: opencv-doc
13 Homepage: https://opencv.org/
14 Description: development files for opencv
15 ...
```

这显示了软件版本、大小、依赖关系、描述等信息。

列出已安装的软件

```
1 # 列出所有已安装的软件
2 apt list --installed
3
4 # 过滤特定软件
5 apt list --installed | grep opencv
6
7 # 列出可升级的软件
8 apt list --upgradable
```

清理缓存

APT 会缓存下载的 `.deb` 文件在 `/var/cache/apt/archives/`。时间长了可能占用大量空间：

```
1 # 查看缓存大小
2 du -sh /var/cache/apt/archives/
3 500M    /var/cache/apt/archives/
4
5 # 清理缓存（删除已安装软件的旧版本包）
6 sudo apt autoclean
7
8 # 清理所有缓存
9 sudo apt clean
```

1.3.7.3. dpkg：底层工具

`dpkg` 是 Debian 系统的底层包管理工具，APT 实际上是 `dpkg` 的前端。APT 处理依赖关系和从网络下载，`dpkg` 负责实际的包安装和管理。日常使用中你主要使用 APT，但有时需要直接用 `dpkg`。

安装本地 `.deb` 文件

有时你从网站下载了 `.deb` 文件，需要手动安装：

```
1 # 下载 deb 文件
2 wget https://example.com/software.deb
3
4 # 用 dpkg 安装
5 sudo dpkg -i software.deb
6
7 # 如果有依赖问题, 用 apt 修复
8 sudo apt install -f
```

`apt install -f` (fix broken) 会安装缺失的依赖。也可以直接用 APT 安装本地 deb 文件, 它会自动处理依赖:

```
1 sudo apt install ./software.deb
```

注意路径前的 `./`, 它告诉 APT 这是一个本地文件而不是软件源中的包名。

查询已安装的包

```
1 # 查看包是否安装
2 dpkg -l | grep package_name
3
4 # 查看包安装了哪些文件
5 dpkg -L package_name
6 /usr/bin/program
7 /usr/lib/libsomething.so
8 /usr/share/doc/package_name/README
9 ...
10
11 # 查看某个文件属于哪个包
12 dpkg -S /usr/bin/vim
13 vim: /usr/bin/vim
```

这在排查问题时很有用——比如某个命令不工作, 你可以查看它属于哪个包, 然后重新安装。

手动管理包

```
1 # 卸载包
2 sudo dpkg -r package_name
3
4 # 完全卸载 (包括配置)
5 sudo dpkg -P package_name
6
7 # 列出所有已安装的包
8 dpkg --get-selections
```

1.3.7.4. 软件源与 PPA

Ubuntu 官方软件源包含了大量软件, 但有时你需要的软件不在官方源中, 或者官方源的本太旧。这时可以添加第三方软件源。

软件源配置

软件源配置在 `/etc/apt/sources.list` 和 `/etc/apt/sources.list.d/` 目录下:

```
1 cat /etc/apt/sources.list
2 deb http://archive.ubuntu.com/ubuntu/ jammy main restricted universe
  multiverse
3 deb http://archive.ubuntu.com/ubuntu/ jammy-updates main restricted universe
  multiverse
```



```
4 deb http://archive.ubuntu.com/ubuntu/ jammy-security main restricted universe
multiverse
```

每行定义一个软件源：`deb` 表示二进制包源，URL 是服务器地址，后面是发行版代号和组件。组件的含义：

- `main`：Ubuntu 官方支持的自由软件
- `restricted`：官方支持的非自由软件（如显卡驱动）
- `universe`：社区维护的自由软件
- `multiverse`：非自由软件

更换镜像源

官方源服务器在国外，下载速度可能较慢。可以更换为国内镜像：

```
1 # 备份原配置
2 sudo cp /etc/apt/sources.list /etc/apt/sources.list.backup
3
4 # 编辑配置文件
5 sudo nano /etc/apt/sources.list
6 # 将 archive.ubuntu.com 替换为镜像地址，如：
7 # mirrors.aliyun.com
8 # mirrors.tuna.tsinghua.edu.cn
9 # mirrors.ustc.edu.cn
10
11 # 更新软件列表
12 sudo apt update
```

许多 Linux 发行版在安装时会提供镜像选择，或者有图形化的软件源设置工具。

PPA：个人软件包存档

PPA（Personal Package Archive）是 Ubuntu 的第三方软件源机制，允许开发者发布自己的软件包。很多最新版本的软件通过 PPA 提供。

```
1 # 添加 PPA
2 sudo add-apt-repository ppa:user/ppa-name
3 sudo apt update
4
5 # 例如，添加最新的 Git
6 sudo add-apt-repository ppa:git-core/ppa
7 sudo apt update
8 sudo apt install git
9
10 # 移除 PPA
11 sudo add-apt-repository --remove ppa:user/ppa-name
```

添加 PPA 时要谨慎——PPA 由第三方维护，软件质量和安全性没有官方保证。只添加你信任的 PPA。

手动添加软件源

有些软件（如 ROS 2、Docker）提供自己的软件源。添加过程通常包括：

```
1 # 1. 添加 GPG 密钥（验证软件包的签名）
2 curl -fsSL https://example.com/gpg.key | sudo gpg --dearmor -o /usr/share/
keyrings/example.gpg
3
4 # 2. 添加软件源
```



```
5 echo "deb [signed-by=/usr/share/keyrings/example.gpg] https://example.com/apt
  stable main" | sudo tee /etc/apt/sources.list.d/example.list
6
7 # 3. 更新并安装
8 sudo apt update
9 sudo apt install software
```

这个过程看起来复杂，但软件的官方文档通常会提供完整的命令，复制粘贴即可。

1.3.7.5. 从源码编译安装

有时候，软件源中没有你需要的软件，或者你需要特定版本、特定编译选项。这时需要从源码编译安装。

典型的编译安装流程

大多数 C/C++ 项目遵循类似的编译流程：

```
1 # 1. 下载源码
2 wget https://example.com/software-1.0.tar.gz
3 tar xzf software-1.0.tar.gz
4 cd software-1.0
5
6 # 2. 安装编译依赖
7 sudo apt install build-essential cmake # 基本工具
8 sudo apt install libxxx-dev           # 项目特定的依赖
9
10 # 3. 配置
11 mkdir build && cd build
12 cmake ..
13 # 或者对于使用 autotools 的项目
14 ./configure
15
16 # 4. 编译
17 make -j$(nproc) # 使用所有 CPU 核心并行编译
18
19 # 5. 安装（到系统目录）
20 sudo make install
21
22 # 或者安装到自定义位置
23 cmake -DCMAKE_INSTALL_PREFIX=/opt/software ..
24 make -j$(nproc)
25 sudo make install
```

安装编译依赖

编译软件通常需要开发版本的库（带 `-dev` 后缀）：

```
1 # 例如，编译使用 OpenCV 的程序
2 sudo apt install libopencv-dev
3
4 # 编译使用 Eigen 的程序
5 sudo apt install libeigen3-dev
6
7 # 编译使用 Qt 的程序
8 sudo apt install qtbase5-dev
```

开发包包含头文件和静态链接所需的文件，运行时包只包含动态库。编译时需要开发包，运行时只需要运行时包。

管理源码安装的软件

从源码安装的软件不受包管理器管理，有几个问题：

- 卸载麻烦：需要到源码目录运行 `sudo make uninstall`（如果支持的话）
- 升级麻烦：需要重新下载、编译、安装
- 可能与系统软件冲突

一些建议：

```
1 # 安装到 /opt 或 /usr/local, 与系统软件分开
2 cmake -DCMAKE_INSTALL_PREFIX=/opt/software-1.0 ..
3
4 # 或者使用 checkinstall 创建 deb 包
5 sudo apt install checkinstall
6 sudo checkinstall # 代替 sudo make install
7 # 这会创建一个 deb 包并安装, 之后可以用 dpkg 卸载
```

编译 OpenCV（示例）

OpenCV 是 RoboMaster 开发中最重要的库之一。虽然可以从软件源安装，但从源码编译可以启用更多功能（如 CUDA 加速）：

```
1 # 安装依赖
2 sudo apt install build-essential cmake git
3 sudo apt install libgtk-3-dev libavcodec-dev libavformat-dev libswscale-dev
4 sudo apt install libtbb-dev libjpeg-dev libpng-dev libtiff-dev
5 sudo apt install libv4l-dev libxvidcore-dev libx264-dev
6
7 # 下载源码
8 git clone https://github.com/opencv/opencv.git
9 git clone https://github.com/opencv/opencv_contrib.git
10 cd opencv
11 git checkout 4.8.0 # 切换到特定版本
12 cd ../opencv_contrib
13 git checkout 4.8.0
14
15 # 配置
16 cd ../opencv
17 mkdir build && cd build
18 cmake -DCMAKE_BUILD_TYPE=Release \
19     -DCMAKE_INSTALL_PREFIX=/usr/local \
20     -DOPENCV_EXTRA_MODULES_PATH=../../opencv_contrib/modules \
21     -DWITH_TBB=ON \
22     -DWITH_V4L=ON \
23     -DWITH_OPENGL=ON \
24     -DBUILD_EXAMPLES=OFF \
25     ..
26
27 # 编译（可能需要较长时间）
28 make -j$(nproc)
29
30 # 安装
31 sudo make install
32 sudo ldconfig # 更新库缓存
```

1.3.7.6. RoboMaster 常用软件安装

以下是 RoboMaster 开发中常用软件的安装方法：

基础开发工具

```
1 # 编译工具
2 sudo apt install build-essential cmake cmake-curses-gui ninja-build
3
```

```

4 # 版本控制
5 sudo apt install git git-lfs
6
7 # 编辑器和 IDE
8 sudo apt install vim
9 # VS Code 需要从官网下载或添加微软的软件源
10
11 # 调试工具
12 sudo apt install gdb valgrind
13
14 # 文档工具
15 sudo apt install doxygen graphviz

```

核心库

```

1 # OpenCV (从软件源安装)
2 sudo apt install libopencv-dev python3-opencv
3
4 # Eigen
5 sudo apt install libeigen3-dev
6
7 # fmt 和 spdlog
8 sudo apt install libfmt-dev libspdlog-dev
9
10 # yaml-cpp (配置文件解析)
11 sudo apt install libyaml-cpp-dev
12
13 # Ceres Solver (非线性优化)
14 sudo apt install libceres-dev
15
16 # Google Test
17 sudo apt install libgtest-dev

```

ROS 2 安装

ROS 2 的安装需要添加官方软件源:

```

1 # 设置 locale
2 sudo apt update && sudo apt install locales
3 sudo locale-gen en_US en_US.UTF-8
4 sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
5 export LANG=en_US.UTF-8
6
7 # 添加 ROS 2 软件源
8 sudo apt install software-properties-common
9 sudo add-apt-repository universe
10
11 sudo apt update && sudo apt install curl -y
12 sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.
key -o /usr/share/keyrings/ros-archive-keyring.gpg
13
14 echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/
ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-
release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/
ros2.list > /dev/null
15
16 # 安装 ROS 2
17 sudo apt update
18 sudo apt install ros-humble-desktop # 完整桌面版
19 # 或
20 sudo apt install ros-humble-ros-base # 基础版 (无 GUI 工具)
21
22 # 安装开发工具

```

```
23 sudo apt install ros-dev-tools
24
25 # 设置环境 (添加到 ~/.bashrc)
26 echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
27 source ~/.bashrc
```

常用 ROS 2 包

```
1 # 图像相关
2 sudo apt install ros-humble-cv-bridge ros-humble-image-transport
3 sudo apt install ros-humble-camera-info-manager
4
5 # TF2
6 sudo apt install ros-humble-tf2 ros-humble-tf2-ros ros-humble-tf2-geometry-
  msgs
7
8 # 可视化
9 sudo apt install ros-humble-rviz2 ros-humble-rqt*
10
11 # 仿真
12 sudo apt install ros-humble-gazebo-ros-pkgs
```

串口和硬件相关

```
1 # 串口通信库
2 sudo apt install libserial-dev
3
4 # USB 设备权限
5 sudo usermod -aG dialout $USER # 需要重新登录
6
7 # V4L2 相机工具
8 sudo apt install v4l-utils
9 v4l2-ctl --list-devices # 列出摄像头
```

NVIDIA 相关 (如果有 NVIDIA GPU)

```
1 # NVIDIA 驱动 (推荐使用 Ubuntu 的驱动管理器)
2 sudo ubuntu-drivers autoinstall
3 # 或手动安装特定版本
4 sudo apt install nvidia-driver-535
5
6 # CUDA (从 NVIDIA 官网下载或使用软件源)
7 # 详见 NVIDIA 官方文档
8
9 # cuDNN
10 # 需要从 NVIDIA 开发者网站下载
```

1.3.7.7. 常见问题解决

依赖问题

```
1 # 依赖不满足
2 sudo apt install -f
3
4 # 包冲突
5 sudo apt --fix-broken install
6
7 # 强制修复损坏的包
8 sudo dpkg --configure -a
```

锁文件问题

如果看到“无法获取锁”的错误，说明另一个程序正在使用包管理器：

```
1 E: Could not get lock /var/lib/dpkg/lock-frontent
2
3 # 等待其他程序完成，或者如果确定没有其他程序在运行
4 sudo rm /var/lib/dpkg/lock-frontent
5 sudo rm /var/lib/dpkg/lock
6 sudo dpkg --configure -a
```

包被保留

```
1 # 查看被保留的包
2 apt-mark showhold
3
4 # 取消保留
5 sudo apt-mark unhold package_name
```

清理系统

```
1 # 删除不需要的依赖
2 sudo apt autoremove
3
4 # 清理下载缓存
5 sudo apt clean
6
7 # 删除旧内核（保留当前和上一个）
8 sudo apt autoremove --purge
```

软件包管理是 Linux 系统维护的核心技能。APT 让软件安装变得简单可靠，理解其工作原理可以帮助你解决各种问题。记住定期运行 `apt update && apt upgrade` 保持系统更新，这不仅能获得新功能，更重要的是获得安全补丁。对于 RoboMaster 开发，掌握软件包管理让你能够快速搭建开发环境，这是项目顺利进行的基础。

1.3.8. 进程与系统监控

当你运行一个程序时，操作系统会创建一个进程来执行它。进程是程序的运行实例，拥有自己的内存空间、文件描述符和系统资源。Linux 是一个多任务操作系统，可以同时运行成百上千个进程。作为开发者，你需要了解如何查看系统中正在运行的进程、监控资源使用情况、终止行为异常的进程、管理后台任务。这些技能在调试程序、排查性能问题、管理机器人系统时都会用到。

1.3.8.1. 查看进程：ps

`ps` (process status) 命令显示系统中的进程信息。它有许多选项，最常用的是 `aux` 组合：

```
1 ps aux
2 USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
3 root         1  0.0  0.1 169584 13256 ?        Ss   Jan15    0:03 /sbin/init
4 root         2  0.0  0.0      0      0 ?        S    Jan15    0:00 [kthreadd]
5 ...
6 alice    12345  2.5  1.2 456789 98765 pts/0    Sl+  10:00    0:30 ./
7 alice    12400  0.0  0.0  12345   1234 pts/1    R+   10:30    0:00 ps aux
```

输出的各列含义：

- `USER`：进程所属用户
- `PID`：进程 ID，每个进程的唯一标识符
- `%CPU`：CPU 使用率

- **%MEM**：内存使用率
- **VSZ**：虚拟内存大小 (KB)
- **RSS**：实际使用的物理内存 (KB)
- **TTY**：关联的终端 (? 表示没有关联终端，通常是后台服务)
- **STAT**：进程状态
- **START**：启动时间
- **TIME**：累计 CPU 时间
- **COMMAND**：启动命令

进程状态 (STAT) 的常见值：

- **R**：运行中 (Running)
- **S**：睡眠 (Sleeping)，等待某个事件
- **D**：不可中断睡眠，通常在等待 I/O
- **T**：停止 (Stopped)
- **Z**：僵尸进程 (Zombie)，已终止但父进程未回收
- **+**：前台进程组
- **l**：多线程
- **s**：会话领导者

ps 的输出是瞬时快照，显示命令执行那一刻的状态。常用的 **ps** 命令变体：

```
1 # 显示所有进程的详细信息
2 ps aux
3
4 # 显示进程树，展示父子关系
5 ps auxf
6 # 或
7 pstree
8
9 # 只显示特定用户的进程
10 ps -u alice
11
12 # 显示特定进程
13 ps -p 12345
14
15 # 显示特定命令的进程
16 ps aux | grep detector
17 alice    12345   2.5   1.2 456789 98765 pts/0    Sl+  10:00   0:30 ./
18         detector_node
19 alice    12500   0.0   0.0  12345   1234 pts/1    S+   10:31   0:00 grep
20         detector
```

注意 **grep** 自己也会出现在结果中。可以用 **grep -v grep** 排除它，或者使用更优雅的技巧 **grep [d]etector**（方括号让 **grep** 本身的命令行不匹配这个模式）。

pgrep 是专门用于查找进程的工具，比 **ps | grep** 更方便：

```
1 # 查找名称包含 detector 的进程
2 pgrep -a detector
3 12345 ./detector_node
4
5 # 只输出 PID
6 pgrep detector
7 12345
8
9 # 查找特定用户的进程
```

```
10 pgrep -u alice python
```

1.3.8.2. 实时监控：top 和 htop

`ps` 显示的是静态快照，而 `top` 提供实时更新的进程视图。它是系统监控的标准工具，每隔几秒刷新一次：

```
1 top
2 top - 10:30:00 up 5 days, 12:30, 2 users, load average: 0.50, 0.45, 0.40
3 Tasks: 250 total, 1 running, 249 sleeping, 0 stopped, 0 zombie
4 %Cpu(s): 5.0 us, 2.0 sy, 0.0 ni, 92.5 id, 0.5 wa, 0.0 hi, 0.0 si,
  0.0 st
5 MiB Mem : 16000.0 total, 8000.0 free, 5000.0 used, 3000.0 buff/cache
6 MiB Swap: 2000.0 total, 2000.0 free, 0.0 used. 10500.0 avail Mem
7
8  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
9 12345 alice    20   0 456789  98765 12345 S   25.0   0.6   0:30.00 detector_node
10 12346 alice    20   0 234567  45678  8901 S   10.0   0.3   0:15.00 tracker_node
11    1 root      20   0 169584  13256  8456 S    0.0   0.1   0:03.00 systemd
12 ...
```

顶部的系统摘要显示了重要信息：

- **load average**：系统负载，三个数字分别是过去 1、5、15 分钟的平均值。在单核系统上，1.0 表示 CPU 满负载；在 4 核系统上，4.0 表示满负载。
- **Tasks**：进程统计——总数、运行中、睡眠、停止、僵尸
- **%Cpu(s)**：CPU 使用分解——us（用户空间）、sy（内核空间）、id（空闲）、wa（等待 I/O）等
- **Mem/Swap**：内存和交换空间使用情况

在 `top` 运行时，可以用按键交互：

- **q**：退出
- **h**：显示帮助
- **k**：杀死进程（输入 PID）
- **r**：调整进程优先级（renice）
- **M**：按内存使用排序
- **P**：按 CPU 使用排序
- **l**：显示每个 CPU 核心的使用情况
- **c**：显示完整命令行
- **f**：选择显示哪些列

`htop` 是 `top` 的增强版，提供彩色显示、鼠标支持、更直观的界面。它不是系统自带的，需要安装：

```
1 sudo apt install htop
2 htop
```

`htop` 的界面更加友好：顶部用彩色条形图显示每个 CPU 核心的使用率和内存使用情况，进程列表支持鼠标点击和滚动。底部显示常用操作的快捷键。

`htop` 的常用操作：

- **F1**：帮助

- **F2**：设置
- **F3**：搜索进程
- **F4**：过滤进程
- **F5**：树状视图
- **F6**：选择排序列
- **F9**：杀死进程
- **F10**：退出
- **Space**：标记进程（可以同时操作多个）
- **u**：只显示特定用户的进程
- **t**：切换树状/列表视图

对于 RoboMaster 开发，htop 是排查性能问题的利器。当机器人系统响应变慢时，打开 htop 可以立即看到哪个进程占用了大量 CPU 或内存。

1.3.8.3. 终止进程：kill、killall、pkill

当进程行为异常（如死循环、无响应）时，需要终止它。Linux 使用信号（signal）机制来控制进程。

kill 命令向指定 PID 的进程发送信号：

```
1 # 发送 SIGTERM (15)，请求进程优雅终止
2 kill 12345
3
4 # 发送 SIGKILL (9)，强制终止
5 kill -9 12345
6 # 或
7 kill -KILL 12345
8
9 # 发送 SIGHUP (1)，通常用于重新加载配置
10 kill -HUP 12345
```

常用的信号：

- **SIGTERM** (15)：终止信号，默认信号。进程可以捕获这个信号并进行清理后退出。
- **SIGKILL** (9)：强制杀死信号，进程无法捕获或忽略，立即终止。
- **SIGINT** (2)：中断信号，相当于 Ctrl+C。
- **SIGSTOP** (19)：暂停进程，相当于 Ctrl+Z。
- **SIGCONT** (18)：继续运行暂停的进程。
- **SIGHUP** (1)：挂起信号，常用于通知守护进程重新加载配置。

一般流程是先尝试 SIGTERM，给进程机会进行清理（关闭文件、释放资源）。如果进程不响应，再使用 SIGKILL 强制终止：

```
1 # 先温和地请求终止
2 kill 12345
3
4 # 等待几秒，如果进程还在
5 ps -p 12345
6 # 如果还在，强制杀死
7 kill -9 12345
```

killall 根据进程名称终止所有匹配的进程：

```
1 # 终止所有名为 detector_node 的进程
```



```

2 killall detector_node
3
4 # 强制终止
5 killall -9 detector_node
6
7 # 交互式确认
8 killall -i detector_node
9 Kill detector_node(12345)? (y/N)

```

pkill 更灵活，支持模式匹配：

```

1 # 终止名称包含 detector 的进程
2 pkill detector
3
4 # 终止特定用户的进程
5 pkill -u alice python
6
7 # 只终止最新启动的匹配进程
8 pkill -n detector
9
10 # 只终止最老的匹配进程
11 pkill -o detector

```

在 RoboMaster 开发中，经常需要终止 ROS 2 节点。可以使用 ROS 2 自己的命令，也可以用系统命令：

```

1 # ROS 2 方式
2 ros2 node list
3 /detector_node
4 /tracker_node
5
6 # 系统方式
7 pkill -f "ros2 run" # 终止所有 ros2 run 启动的进程
8 pkill detector_node

```

-f 选项让 pkill 匹配完整的命令行，而不仅仅是进程名。

1.3.8.4. 前台与后台

在终端中运行程序时，程序默认在前台运行，占据终端，你需要等它完成才能输入其他命令。但你可以让程序在后台运行，这样终端可以继续使用。

在命令末尾加 **&** 让程序在后台启动：

```

1 ./long_running_program &
2 [1] 12345
3 # 立即返回提示符，可以继续工作

```

[1] 是作业号，**12345** 是进程 ID。程序在后台运行，但它的输出仍然会显示在终端上（可能会打断你的输入）。

jobs 命令列出当前 Shell 的后台作业：

```

1 jobs
2 [1]+  Running                  ./long_running_program &
3 [2]-  Stopped                  vim file.txt

```

+ 表示当前作业（最近操作的），**-** 表示上一个作业。

fg (foreground) 把后台作业带回前台：

```
1 fg %1      # 把作业 1 带到前台
2 fg        # 把当前作业 (+) 带到前台
```

bg (background) 让暂停的作业在后台继续运行：

```
1 # 运行一个程序
2 ./program
3 # 按 Ctrl+Z 暂停
4 ^Z
5 [1]+  Stopped                  ./program
6
7 # 让它在后台继续运行
8 bg %1
9 [1]+ ./program &
```

这个技巧很有用：如果你运行了一个程序后发现它需要很长时间，不想开新终端，可以 Ctrl+Z 暂停，然后 bg 让它在后台继续。

后台进程仍然与终端关联。如果你关闭终端，后台进程也会收到 SIGHUP 信号并终止。要让进程在终端关闭后继续运行，使用 **nohup**：

```
1 nohup ./long_running_program &
2 nohup: ignoring input and appending output to 'nohup.out'
3 [1] 12345
```

nohup 让进程忽略 SIGHUP 信号，输出被重定向到 **nohup.out** 文件。

另一个选择是使用 **disown** 将已经在后台运行的作业与 Shell 解除关联：

```
1 ./program &
2 [1] 12345
3 disown %1
4 # 现在关闭终端也不会影响这个进程
```

对于需要长期运行的服务，更好的方式是使用 **systemd** 管理（后面会介绍），或者使用 **screen** / **tmux** 终端复用器。

1.3.8.5. 系统资源监控

内存：free

free 命令显示系统内存使用情况：

```
1 free -h
2              total        used        free      shared  buff/
3 Mem:         15Gi        5.0Gi        6.0Gi        500Mi        4.0Gi
4 Swap:         2.0Gi         0B         2.0Gi
```

-h 选项以人类可读的格式显示（GB、MB）。各列含义：

- **total**：总物理内存
- **used**：已使用内存
- **free**：完全空闲的内存
- **shared**：共享内存（如 tmpfs）
- **buff/cache**：缓冲区和缓存使用的内存
- **available**：可供新程序使用的内存（包括可释放的缓存）

Linux 会积极使用空闲内存作为缓存来加速磁盘访问，所以 `free` 值很低不一定表示内存紧张。关键是 `available` 值——只要它还有足够的余量，系统就不会有内存压力。

Swap 是磁盘上的交换空间，当物理内存不足时使用。如果 Swap 使用量很高，说明系统内存紧张，性能可能会下降。

磁盘空间：df

`df` (disk free) 显示文件系统的磁盘空间使用情况：

```
1 df -h
2 Filesystem      Size  Used Avail Use% Mounted on
3 /dev/sda1       100G   45G   50G   48% /
4 /dev/sda2       500G  200G  275G   43% /home
5 tmpfs           7.8G    0   7.8G    0% /dev/shm
6 /dev/sdb1       1.0T  500G  500G   50% /data
```

输出显示每个挂载的文件系统的大小、已用空间、可用空间和挂载点。

```
1 # 只显示本地文件系统，排除虚拟文件系统
2 df -h --local
3
4 # 显示特定目录所在的文件系统
5 df -h /home/alice/projects
```

目录大小：du

`du` (disk usage) 显示目录或文件占用的磁盘空间：

```
1 # 显示当前目录的总大小
2 du -sh .
3 2.5G .
4
5 # 显示各子目录的大小
6 du -sh */
7 500M  build/
8 1.5G  models/
9 200M  src/
10 300M  data/
11
12 # 显示所有文件和目录
13 du -ah
14
15 # 限制深度
16 du -h --max-depth=1
17
18 # 找出最大的目录/文件
19 du -sh * | sort -rh | head -10
```

当磁盘空间不足时，先用 `df` 确定哪个分区满了，再用 `du` 找出占用空间最多的目录：

```
1 df -h
2 # 发现 /home 分区快满了
3
4 cd /home
5 sudo du -sh */ | sort -rh | head
6 50G  alice/
7 20G  bob/
8 ...
9
10 cd alice
11 du -sh */ | sort -rh | head
```

```
12 30G      .cache/
13 10G      projects/
14 ...
```

综合监控工具

除了单独的命令，还有一些综合监控工具：

```
1 # vmstat: 虚拟内存统计
2 vmstat 1      # 每秒刷新一次
3 procs -----memory----- ---swap-- -----io---- -system-- -----
cpu-----
4  r  b   swpd   free   buff   cache   si   so    bi    bo    in   cs us sy id
wa st
5  1  0       0 6000000 200000 4000000   0   0     5    10   200   500   5   2 92
1  0
6
7 # iostat: I/O 统计
8 iostat -xh 1
9 Device      r/s      w/s    rkB/s    wkB/s  %util
10 sda         10.00    20.00   500.0k    1.0M    5.00
11
12 # iotop: 实时 I/O 监控（需要安装）
13 sudo apt install iotop
14 sudo iotop
15
16 # nethogs: 网络流量监控（按进程）
17 sudo apt install nethogs
18 sudo nethogs
```

1.3.8.6. 系统服务：systemd 和 systemctl

现代 Linux 发行版使用 systemd 作为初始化系统和服务管理器。系统启动时，systemd 是第一个运行的进程（PID 1），负责启动和管理所有其他服务。

systemctl 是与 systemd 交互的命令。服务在 systemd 中被称为“单元”（unit），服务单元的后缀是 .service。

查看服务状态

```
1 # 查看服务状态
2 systemctl status ssh
3 ● ssh.service - OpenBSD Secure Shell server
4   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor
         preset: enabled)
5   Active: active (running) since Mon 2024-01-15 10:00:00 CST; 5 days ago
6     Docs: man:sshd(8)
7           man:sshd_config(5)
8   Main PID: 1234 (sshd)
9     Tasks: 1 (limit: 18904)
10    Memory: 5.0M
11      CPU: 1.234s
12    CGroup: /system.slice/ssh.service
13            └─1234 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
14
15 Jan 15 10:00:00 hostname systemd[1]: Started OpenBSD Secure Shell server.
```

状态输出包含丰富的信息：服务是否启动、是否设置为开机自启、主进程 PID、资源使用、最近的日志等。

启动、停止、重启服务

```
1 # 启动服务
2 sudo systemctl start ssh
3
4 # 停止服务
5 sudo systemctl stop ssh
6
7 # 重启服务
8 sudo systemctl restart ssh
9
10 # 重新加载配置（不中断服务）
11 sudo systemctl reload ssh
12
13 # 如果不确定是否支持 reload
14 sudo systemctl reload-or-restart ssh
```

设置开机自启

```
1 # 设置开机自启
2 sudo systemctl enable ssh
3
4 # 取消开机自启
5 sudo systemctl disable ssh
6
7 # 同时启动并设置开机自启
8 sudo systemctl enable --now ssh
```

查看所有服务

```
1 # 列出所有服务
2 systemctl list-units --type=service
3
4 # 列出所有服务（包括未运行的）
5 systemctl list-units --type=service --all
6
7 # 列出启用的服务
8 systemctl list-unit-files --type=service --state=enabled
9
10 # 列出失败的服务
11 systemctl --failed
```

查看服务日志

systemd 有自己的日志系统 journald，用 `journalctl` 查看：

```
1 # 查看特定服务的日志
2 journalctl -u ssh
3
4 # 查看最近的日志
5 journalctl -u ssh -n 50
6
7 # 实时追踪日志
8 journalctl -u ssh -f
9
10 # 查看本次启动以来的日志
11 journalctl -u ssh -b
12
13 # 查看特定时间范围的日志
14 journalctl -u ssh --since "2024-01-15 10:00:00" --until "2024-01-15
15 12:00:00"
```

创建自己的服务

你可以创建自己的 systemd 服务来管理 RoboMaster 程序。服务文件放在 `/etc/systemd/system/` 目录下：

```
1 sudo nano /etc/systemd/system/rm-vision.service
```

服务文件内容：

```
1 [Unit]
2 Description=RoboMaster Vision System
3 After=network.target
4
5 [Service]
6 Type=simple
7 User=alice
8 WorkingDirectory=/home/alice/ros2_ws
9 ExecStart=/bin/bash -c 'source /opt/ros/humble/setup.bash && source install/
  setup.bash && ros2 launch rm_vision vision_launch.py'
10 Restart=on-failure
11 RestartSec=5
12
13 [Install]
14 WantedBy=multi-user.target
```

配置说明：

- `[Unit]` 部分描述服务和依赖关系
- `After=network.target` 表示在网络服务启动后再启动
- `[Service]` 部分定义如何运行服务
- `Type=simple` 表示 `ExecStart` 启动的进程就是主进程
- `User` 指定运行服务的用户
- `Restart=on-failure` 表示失败时自动重启
- `[Install]` 部分定义如何启用服务

启用并启动服务：

```
1 # 重新加载 systemd 配置
2 sudo systemctl daemon-reload
3
4 # 启动服务
5 sudo systemctl start rm-vision
6
7 # 查看状态
8 sudo systemctl status rm-vision
9
10 # 设置开机自启
11 sudo systemctl enable rm-vision
```

这样，你的视觉系统就会在机器人开机时自动启动，崩溃后自动重启，非常适合比赛环境。

常用系统服务

一些 RoboMaster 开发中可能遇到的系统服务：

```
1 # SSH 服务
2 sudo systemctl status ssh
3
4 # 网络管理
5 sudo systemctl status NetworkManager
6
```

```

7 # 时间同步
8 sudo systemctl status systemd-timesyncd
9
10 # Docker (如果安装了)
11 sudo systemctl status docker
12
13 # 查看所有 ROS 相关服务 (如果有)
14 systemctl list-units | grep ros

```

1.3.8.7. 实际应用场景

让我们看一些 RoboMaster 开发中的实际场景。

排查程序占用过多资源

```

1 # 发现系统变慢, 打开 htop 查看
2 htop
3 # 发现 detector_node 占用 100% CPU
4
5 # 查看进程详情
6 ps aux | grep detector
7 alice    12345 99.0  5.0 456789 400000 pts/0 R+ 10:00 30:00 ./detector_node
8
9 # 可能是死循环, 查看进程在做什么
10 strace -p 12345
11 # 或者用 gdb attach
12 gdb -p 12345
13
14 # 如果需要终止
15 kill 12345

```

监控 ROS 2 节点资源使用

```

1 # 查看所有 ROS 2 相关进程
2 ps aux | grep ros2
3 ps aux | grep -E "detector|tracker|aim"
4
5 # 实时监控
6 htop -p $(pgrep -d',' detector_node)

```

处理僵尸进程

```

1 # 查找僵尸进程
2 ps aux | grep Z
3 alice    12345  0.0  0.0      0      0 pts/0  Z   10:00   0:00 [defunct]
4
5 # 找到父进程
6 ps -o ppid= -p 12345
7 12300
8
9 # 通常需要终止父进程来清理僵尸进程
10 kill 12300

```

释放磁盘空间

```

1 # 检查磁盘使用
2 df -h
3 # /home 快满了
4
5 # 找大文件

```

```

6 du -sh ~/* | sort -rh | head
7 30G      /home/alice/ros2_ws
8 20G      /home/alice/.cache
9
10 # 清理 ROS 2 构建产物
11 cd ~/ros2_ws
12 rm -rf build/ install/ log/
13
14 # 清理系统缓存
15 sudo apt clean
16 sudo apt autoremove
17
18 # 清理用户缓存
19 rm -rf ~/.cache/pip

```

进程管理和系统监控是 Linux 系统管理的基础技能。掌握这些工具，你就能了解系统的运行状态，及时发现和解决问题。在 RoboMaster 开发中，这些技能会在调试性能问题、管理机器人服务、排查系统故障时反复用到。特别是 systemd 的使用，可以让你的机器人系统在开机时自动启动、异常时自动恢复，大大提高了系统的可靠性。

1.3.9. 网络与远程访问

机器人不会永远摆在你的桌面上，连着键盘和显示器等你操作。在实际的 RoboMaster 开发中，机器人上的计算机（如 Jetson 或工控机）通常没有直接连接的外设，你需要通过网络远程访问它。SSH（Secure Shell）是实现这一目标的核心工具——它让你能够从自己的笔记本连接到机器人的计算机，执行命令、编辑文件、运行程序，就像直接坐在那台机器前一样。本节将介绍网络基础知识、SSH 的使用方法、文件传输技巧，以及为什么掌握这些技能对机器人开发至关重要。

1.3.9.1. 查看网络信息

在连接网络之前，首先要了解本机的网络配置。`ip` 命令是现代 Linux 系统查看和配置网络的标准工具。

查看网络接口和 IP 地址：

```

1 ip addr
2 1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
   default qlen 1000
3     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4     inet 127.0.0.1/8 scope host lo
5         valid_lft forever preferred_lft forever
6
7 2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state
   UP group default qlen 1000
8     link/ether 08:00:27:12:34:56 brd ff:ff:ff:ff:ff:ff
9     inet 192.168.1.100/24 brd 192.168.1.255 scope global dynamic enp0s3
10        valid_lft 86400sec preferred_lft 86400sec
11
12 3: wlp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
   group default qlen 1000
13     link/ether a4:5e:60:ab:cd:ef brd ff:ff:ff:ff:ff:ff
14     inet 192.168.1.101/24 brd 192.168.1.255 scope global dynamic wlp2s0
15        valid_lft 86400sec preferred_lft 86400sec

```

输出中显示了三个网络接口：`lo` 是本地回环接口（localhost），`enp0s3` 是有线网卡，`wlp2s0` 是无线网卡。每个接口显示了 MAC 地址（`link/ether`）和 IP 地址（`inet`）。

常用的 `ip` 命令变体:

```
1 # 简洁显示 IP 地址
2 ip -br addr
3 lo UNKNOWN 127.0.0.1/8
4 enp0s3 UP 192.168.1.100/24
5 wlp2s0 UP 192.168.1.101/24
6
7 # 显示路由表
8 ip route
9 default via 192.168.1.1 dev enp0s3 proto dhcp metric 100
10 192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.100
11
12 # 显示网络接口的链路状态
13 ip link
```

老式的 `ifconfig` 命令在某些系统上仍然可用, 但已被 `ip` 取代。如果你看到用 `ifconfig` 的教程, 知道它做类似的事情即可。

`ping` 命令测试网络连通性, 通过发送 ICMP 数据包来检查目标主机是否可达:

```
1 ping 192.168.1.1
2 PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
3 64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.23 ms
4 64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.98 ms
5 64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=1.05 ms
6 ^C
7 --- 192.168.1.1 ping statistics ---
8 3 packets transmitted, 3 received, 0% packet loss, time 2003ms
9 rtt min/avg/max/mdev = 0.980/1.086/1.230/0.104 ms
```

Linux 的 `ping` 会持续运行直到你按 Ctrl+C 停止 (与 Windows 不同, Windows 默认只发 4 个包)。输出显示了每个数据包的往返时间 (RTT) 和丢包率。

```
1 # 只发送指定数量的包
2 ping -c 4 192.168.1.1
3
4 # ping 域名
5 ping -c 4 google.com
6
7 # 指定间隔时间 (秒)
8 ping -i 0.5 192.168.1.1
```

当网络连接出问题, `ping` 是最基本的诊断工具:

```
1 # 检查本地网络栈
2 ping 127.0.0.1
3
4 # 检查网关 (路由器)
5 ping 192.168.1.1
6
7 # 检查外部网络
8 ping 8.8.8.8
9
10 # 检查 DNS 解析
11 ping google.com
```

如果 `ping` 网关失败, 问题可能在本地网络配置或物理连接; 如果 `ping` 网关成功但外部失败, 问题可能在路由器或上游网络; 如果 IP 能 `ping` 通但域名不行, 问题在 DNS。

其他有用的网络诊断工具:

```

1 # 追踪到目标的路由路径
2 traceroute google.com
3
4 # 显示网络连接和监听端口
5 ss -tuln
6 # -t TCP, -u UDP, -l 监听, -n 数字显示
7
8 # 查看 DNS 解析
9 nslookup google.com
10 dig google.com
11
12 # 查看主机名
13 hostname
14 hostname -I # 显示所有 IP 地址

```

1.3.9.2. 下载文件：wget 和 curl

从网络下载文件是常见的任务，Linux 提供了两个强大的命令行工具。

wget 专注于下载文件，简单直接：

```

1 # 下载文件
2 wget https://example.com/file.tar.gz
3
4 # 下载并指定保存的文件名
5 wget -O myfile.tar.gz https://example.com/file.tar.gz
6
7 # 断点续传（下载中断后继续）
8 wget -c https://example.com/large_file.zip
9
10 # 后台下载
11 wget -b https://example.com/large_file.zip
12 # 日志保存在 wget-log
13
14 # 限速下载（避免占满带宽）
15 wget --limit-rate=1M https://example.com/file.zip
16
17 # 下载整个网站（镜像）
18 wget -r -np -k https://example.com/docs/

```

curl 功能更丰富，支持更多协议，常用于 API 调用和脚本：

```

1 # 下载文件（默认输出到标准输出）
2 curl https://example.com/file.txt
3
4 # 保存到文件
5 curl -o file.txt https://example.com/file.txt
6 curl -O https://example.com/file.txt # 使用远程文件名
7
8 # 跟随重定向
9 curl -L https://example.com/redirect
10
11 # 显示响应头
12 curl -I https://example.com
13
14 # POST 请求
15 curl -X POST -d "data=value" https://api.example.com/endpoint
16
17 # 发送 JSON 数据
18 curl -X POST -H "Content-Type: application/json" \
19     -d '{"key": "value"}' https://api.example.com/endpoint
20

```

```
21 # 带认证
22 curl -u username:password https://example.com/protected
```

在 RoboMaster 开发中，这些工具常用于：

```
1 # 下载预训练模型
2 wget https://github.com/xxx/releases/download/v1.0/model.onnx
3
4 # 下载 ROS 2 安装脚本
5 curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key |
  sudo gpg --dearmor -o /usr/share/keyrings/ros-archive-keyring.gpg
6
7 # 从 GitHub 下载源码
8 wget https://github.com/opencv/opencv/archive/4.8.0.tar.gz
```

1.3.9.3. SSH：远程连接的核心

SSH (Secure Shell) 是远程登录和管理 Linux 系统的标准工具。它提供加密的通信通道，让你能够安全地在网络上执行命令。

基本连接

```
1 # 连接到远程主机
2 ssh username@hostname
3 ssh alice@192.168.1.50
4
5 # 使用非默认端口
6 ssh -p 2222 alice@192.168.1.50
7
8 # 首次连接会提示确认主机指纹
9 The authenticity of host '192.168.1.50 (192.168.1.50)' can't be established.
10 ED25519 key fingerprint
11 is SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx.
12 Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

首次连接时，SSH 会显示远程主机的指纹并询问是否信任。确认后，主机的公钥会保存在 `~/.ssh/known_hosts` 中，之后连接不再询问。如果远程主机的密钥发生变化（可能是重装系统，也可能是中间人攻击），SSH 会警告并拒绝连接。

连接成功后，你会看到远程主机的 Shell 提示符，可以像在本地一样执行命令：

```
1 alice@robot:~$ ls
2 ros2_ws  projects
3
4 alice@robot:~$ ros2 node list
5 /detector_node
6 /tracker_node
7
8 alice@robot:~$ exit # 或 Ctrl+D 退出
9 Connection to 192.168.1.50 closed.
```

执行单个命令

不需要交互式 Shell 时，可以直接在 SSH 命令后附加要执行的命令：

```
1 # 远程执行单个命令
2 ssh alice@192.168.1.50 "ls -la"
3
4 # 远程执行多个命令
5 ssh alice@192.168.1.50 "cd ros2_ws && colcon build"
```

```
6
7 # 查看远程系统状态
8 ssh alice@192.168.1.50 "free -h && df -h"
```

SSH 密钥认证

每次连接都输入密码很麻烦，而且密码可能被暴力破解。SSH 密钥认证更安全也更方便——使用一对密钥（公钥和私钥）代替密码。

生成密钥对：

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
2 Generating public/private ed25519 key pair.
3 Enter file in which to save the key (/home/alice/.ssh/id_ed25519):
4 Enter passphrase (empty for no passphrase):
5 Enter same passphrase again:
6 Your identification has been saved in /home/alice/.ssh/id_ed25519
7 Your public key has been saved in /home/alice/.ssh/id_ed25519.pub
```

这会在 `~/.ssh/` 目录生成两个文件：`id_ed25519`（私钥，必须保密）和 `id_ed25519.pub`（公钥，可以分享）。passphrase 是可选的额外保护——即使私钥被盗，没有 passphrase 也无法使用。

将公钥复制到远程主机：

```
1 ssh-copy-id alice@192.168.1.50
2 /usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s)
3 ...
4 Number of key(s) added: 1
```

这会把你的公钥添加到远程主机的 `~/.ssh/authorized_keys` 文件。之后连接就不需要密码了：

```
1 ssh alice@192.168.1.50
2 # 直接进入，无需密码
3 alice@robot:~$
```

如果没有 `ssh-copy-id`，可以手动复制：

```
1 # 查看公钥
2 cat ~/.ssh/id_ed25519.pub
3 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIxxxx... your_email@example.com
4
5 # 在远程主机上
6 mkdir -p ~/.ssh
7 chmod 700 ~/.ssh
8 echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIxxxx..." >> ~/.ssh/authorized_keys
9 chmod 600 ~/.ssh/authorized_keys
```

权限设置很重要——SSH 会拒绝使用权限过于开放的密钥文件。

SSH 配置文件

频繁连接多台机器时，每次输入完整的用户名和地址很繁琐。可以在 `~/.ssh/config` 中配置别名：

```
1 nano ~/.ssh/config
```

```

1 # 机器人主机
2 Host robot
3     HostName 192.168.1.50
4     User alice
5     Port 22
6
7 # Jetson 开发板
8 Host jetson
9     HostName 192.168.1.60
10    User nvidia
11    IdentityFile ~/.ssh/jetson_key
12
13 # 跳板机访问内网服务器
14 Host internal
15     HostName 10.0.0.100
16     User admin
17     ProxyJump jumphost
18
19 # 所有主机的通用设置
20 Host *
21     ServerAliveInterval 60
22     ServerAliveCountMax 3

```

配置后，连接变得简单：

```

1 ssh robot    # 相当于 ssh alice@192.168.1.50
2 ssh jetson   # 相当于 ssh nvidia@192.168.1.60 -i ~/.ssh/jetson_key

```

`ServerAliveInterval` 让 SSH 定期发送心跳包，防止连接因空闲超时被断开。

SSH 端口转发

SSH 可以创建加密隧道，转发端口流量。这在访问远程服务时很有用。

本地端口转发——把远程服务映射到本地：

```

1 # 将远程的 ROS 2 话题可视化工具（运行在 8080 端口）映射到本地
2 ssh -L 8080:localhost:8080 alice@robot
3 # 现在访问 localhost:8080 就是访问远程的 8080
4
5 # 访问远程机器上的 Jupyter Notebook
6 ssh -L 8888:localhost:8888 alice@robot

```

远程端口转发——把本地服务映射到远程：

```

1 # 让远程机器能访问本地的服务
2 ssh -R 3000:localhost:3000 alice@robot

```

动态端口转发——创建 SOCKS 代理：

```

1 ssh -D 1080 alice@robot
2 # 本地 1080 端口成为 SOCKS 代理

```

1.3.9.4. 文件传输：scp 和 rsync

`scp` (secure copy) 通过 SSH 复制文件，用法类似 `cp`：

```

1 # 上传文件到远程
2 scp file.txt alice@192.168.1.50:/home/alice/
3 scp file.txt robot:~/ # 使用 SSH 配置的别名

```

```

4
5 # 下载文件到本地
6 scp alice@192.168.1.50:/home/alice/data.csv ./
7 scp robot:~/results.txt ./
8
9 # 复制整个目录 (-r 递归)
10 scp -r project/ robot:~/
11
12 # 使用非默认端口
13 scp -P 2222 file.txt robot:~/

```

scp 简单直接，但每次都会完整传输文件。对于大目录或频繁同步，**rsync** 更高效——它只传输有变化的部分。

```

1 # 同步目录到远程
2 rsync -avz project/ robot:~/project/
3 # -a 归档模式（保留权限、时间等）
4 # -v 显示详细信息
5 # -z 压缩传输
6
7 # 从远程同步到本地
8 rsync -avz robot:~/data/ ./data/
9
10 # 显示进度
11 rsync -avz --progress large_file.zip robot:~/
12
13 # 删除远程有但本地没有的文件（镜像同步）
14 rsync -avz --delete project/ robot:~/project/
15
16 # 排除某些文件
17 rsync -avz --exclude='build/' --exclude='*.log' project/ robot:~/project/
18
19 # 试运行（不实际传输，只显示会做什么）
20 rsync -avzn project/ robot:~/project/

```

rsync 的增量同步特别适合开发场景：

```

1 # 修改代码后，只同步变化的文件
2 rsync -avz ~/ros2_ws/src/ robot:~/ros2_ws/src/
3
4 # 同步后在远程编译
5 ssh robot "cd ros2_ws && colcon build"

```

对于更复杂的文件同步需求，可以考虑使用 **unison**（双向同步）或设置网络文件系统（NFS、SSHFS）。

SSHFS：挂载远程目录

SSHFS 可以把远程目录挂载到本地，像访问本地文件一样访问远程文件：

```

1 # 安装 sshfs
2 sudo apt install sshfs
3
4 # 创建挂载点
5 mkdir ~/remote_robot
6
7 # 挂载远程目录
8 sshfs robot:/home/alice/ros2_ws ~/remote_robot
9
10 # 现在可以直接访问
11 ls ~/remote_robot

```

```
12 build install log src
13
14 # 卸载
15 fusermount -u ~/remote_robot
```

这对于用本地 IDE（如 VS Code）编辑远程代码非常方便。VS Code 也有 Remote-SSH 扩展，提供更集成的体验。

1.3.9.5. 为什么机器人开发离不开 SSH

在 RoboMaster 开发中，SSH 不是可选的技能，而是必需的。这有几个原因。

首先，机器人上的计算机通常没有显示器和键盘。机器人需要轻量化，不可能带着显示器跑。即使在调试时临时接上显示器，操作起来也很不方便。SSH 让你可以舒适地坐在自己的工位上，用自己熟悉的键盘和屏幕操作机器人。

其次，机器人可能在运动中。你不能一边追着机器人跑，一边敲键盘。通过无线 SSH 连接，机器人可以自由移动，你可以远程观察它的状态、查看日志、调整参数。

第三，调试需要同时观察多个方面。你可能需要同时查看视觉节点的输出、控制节点的日志、系统资源使用情况。在本地终端开多个窗口，分别 SSH 到机器人执行不同的命令，比在机器人上切换窗口方便得多。

第四，团队协作。多人可能需要同时访问同一台机器人进行调试。SSH 天然支持多用户同时登录，每个人在自己的终端工作，互不干扰。

一个典型的调试场景：

```
1 # 终端 1: 查看检测节点的输出
2 ssh robot
3 ros2 topic echo /detector/armors
4
5 # 终端 2: 监控系统资源
6 ssh robot
7 htop
8
9 # 终端 3: 实时查看日志
10 ssh robot
11 tail -f ~/ros2_ws/log/latest/detector_node.log
12
13 # 终端 4: 运行可视化工具
14 ssh -X robot # -X 启用 X11 转发
15 rqt_image_view
```

SSH 的 X11 转发（-X 选项）允许在远程运行图形程序，窗口显示在本地。这对于运行 rqt、rviz 等可视化工具很有用，不过性能可能不如本地运行。

使用 `tmux` 或 `screen` 可以让远程会话在断开后继续运行：

```
1 # 在机器人上启动 tmux 会话
2 ssh robot
3 tmux new -s vision
4
5 # 运行程序
6 ros2 launch rm_vision vision_launch.py
7
8 # 按 Ctrl+B 然后 D 分离会话
9 # 断开 SSH 连接，程序继续运行
10
11 # 重新连接
```

```
12 ssh robot
13 tmux attach -t vision
```

这对于长时间运行的测试特别有用——即使网络断开，程序也不会中断。

1.3.9.6. 网络配置技巧

在 RoboMaster 场景中，网络配置有一些常见的模式。

固定 IP 配置

在比赛或测试环境中，通常需要给机器人分配固定 IP，以便可靠地连接：

```
1 # 使用 netplan (Ubuntu 18.04+)
2 sudo nano /etc/netplan/01-network-config.yaml
```

```
1 network:
2   version: 2
3   renderer: networkd
4   ethernets:
5     eth0:
6       addresses:
7         - 192.168.1.50/24
8       gateway4: 192.168.1.1
9       nameservers:
10        addresses:
11          - 8.8.8.8
12          - 8.8.4.4
```

```
1 sudo netplan apply
```

多机器人网络

多台机器人组成网络时，需要规划 IP 地址和 ROS 2 域 ID：

```
1 # 机器人 1: 192.168.1.50, ROS_DOMAIN_ID=1
2 # 机器人 2: 192.168.1.51, ROS_DOMAIN_ID=2
3 # 控制站: 192.168.1.100
4
5 # 在各机器的 .bashrc 中设置
6 export ROS_DOMAIN_ID=1
```

无线热点

机器人可以创建自己的无线热点，便于笔记本直接连接：

```
1 # 使用 nmcli 创建热点
2 sudo nmcli device wifi hotspot ssid "RoboMaster" password "password123"
```

SSH 是连接物理世界（机器人）和你的开发环境的桥梁。熟练使用 SSH，你就能随时随地访问和控制机器人，不受物理位置的限制。这是从“能让机器人动起来”到“能高效开发机器人”的关键一步。

1.3.10. Shell 脚本入门

当你发现自己反复输入相同的命令序列时，就是学习 Shell 脚本的时机了。Shell 脚本是将多个命令组合在一起的程序，可以自动执行重复性任务。编译项目、部署代码、备份文件、初始化环境——这些日常工作都可以用脚本自动化。脚本不仅节省时间，还能减少人为错误，让复杂

的操作变得可重复、可分享。本节将带你从零开始编写 Shell 脚本，掌握变量、条件、循环、函数等基本概念，最后通过几个实用示例展示如何用脚本解决实际问题。

1.3.10.1. 第一个脚本与 Shebang

让我们从最简单的脚本开始。用你喜欢的编辑器创建一个文件：

```
1 nano hello.sh
```

输入以下内容：

```
1 #!/bin/bash
2
3 echo "Hello, RoboMaster!"
4 echo "Current time: $(date)"
5 echo "Current directory: $(pwd)"
```

保存并退出。第一行 `#!/bin/bash` 称为 shebang (或 hashbang)，它告诉系统用哪个解释器执行这个脚本。`#!` 后面是解释器的路径，这里是 `/bin/bash`。没有 shebang，系统可能不知道如何执行你的脚本，或者用错误的解释器执行。

赋予脚本执行权限并运行：

```
1 chmod +x hello.sh
2 ./hello.sh
3 Hello, RoboMaster!
4 Current time: Mon Jan 15 10:30:00 CST 2024
5 Current directory: /home/alice/scripts
```

`$(command)` 是命令替换语法，它执行括号内的命令并将输出插入到该位置。这让你可以在脚本中动态获取信息。

你也可以显式用 `bash` 运行脚本，这时不需要执行权限：

```
1 bash hello.sh
```

但养成添加 shebang 和执行权限的习惯更好——这让脚本成为独立的可执行程序。

关于 shebang 的一些变体：

```
1 #!/bin/bash           # 使用 bash
2 #!/bin/sh             # 使用系统默认 shell (可能是 dash, 更严格)
3 #!/usr/bin/env bash   # 更可移植, 使用 PATH 中找到的 bash
4 #!/usr/bin/env python3 # Python 脚本也用同样的方式
```

`#!/usr/bin/env bash` 的好处是不硬编码 `bash` 的路径，在不同系统上更通用。

1.3.10.2. 变量

变量用于存储数据。在 Bash 中，变量赋值时等号两边不能有空格：

```
1 #!/bin/bash
2
3 # 变量赋值 (注意: 等号两边没有空格)
4 name="Alice"
5 project="rm_vision"
6 version=1.0
7
8 # 使用变量 (用 $ 前缀)
```

```

9 echo "Hello, $name!"
10 echo "Working on $project version $version"
11
12 # 花括号明确变量边界
13 echo "Project: ${project}_2024" # 输出: rm_vision_2024
14 echo "Project: $project_2024" # 错误: 会查找名为 project_2024 的变量

```

变量名区分大小写，习惯上普通变量用小写，环境变量用大写。

命令的输出可以赋值给变量：

```

1 #!/bin/bash
2
3 # 命令替换
4 current_date=$(date +%Y-%m-%d)
5 file_count=$(ls | wc -l)
6 git_branch=$(git branch --show-current 2>/dev/null || echo "not a git repo")
7
8 echo "Date: $current_date"
9 echo "Files in current directory: $file_count"
10 echo "Git branch: $git_branch"

```

一些特殊变量在脚本中很有用：

```

1 $0      # 脚本名称
2 $1, $2, ... # 命令行参数
3 $#      # 参数个数
4 $@      # 所有参数（作为独立的字符串）
5 $*      # 所有参数（作为一个字符串）
6 $?      # 上一个命令的退出状态
7 $$      # 当前脚本的进程 ID
8 $USER   # 当前用户名
9 $HOME   # 用户主目录
10 $PWD    # 当前工作目录

```

用户输入

`read` 命令从用户获取输入：

```

1 #!/bin/bash
2
3 echo -n "Enter your name: " # -n 不换行
4 read username
5
6 echo -n "Enter project name: "
7 read project
8
9 echo "Hello, $username! Setting up $project..."
10
11 # 带提示的读取
12 read -p "Continue? [y/n] " answer
13 if [[ "$answer" == "y" ]]; then
14     echo "Proceeding..."
15 fi
16
17 # 隐藏输入（用于密码）
18 read -s -p "Enter password: " password
19 echo # 换行

```

1.3.10.3. 条件判断

条件判断让脚本能根据情况执行不同的操作。

if 语句

```
1 #!/bin/bash
2
3 file="config.yaml"
4
5 if [[ -f "$file" ]]; then
6     echo "Config file exists"
7 elif [[ -d "$file" ]]; then
8     echo "It's a directory, not a file"
9 else
10     echo "Config file not found, creating default..."
11     touch "$file"
12 fi
```

`[[ ]>` 是 Bash 的条件测试语法，比传统的 `[ ]` 更强大，支持更多操作符，也更不容易出错（比如变量为空时不会报语法错误）。

文件测试操作符

```
1 -e file    # 文件存在
2 -f file    # 是普通文件
3 -d file    # 是目录
4 -r file    # 可读
5 -w file    # 可写
6 -x file    # 可执行
7 -s file    # 文件大小大于 0
8 -L file    # 是符号链接
```

示例：

```
1 #!/bin/bash
2
3 workspace="$HOME/ros2_ws"
4
5 if [[ ! -d "$workspace" ]]; then
6     echo "Workspace not found, creating..."
7     mkdir -p "$workspace/src"
8 fi
9
10 if [[ -x "/opt/ros/humble/setup.bash" ]]; then
11     source /opt/ros/humble/setup.bash
12 else
13     echo "ROS 2 not installed!"
14     exit 1
15 fi
```

字符串比较

```
1 [[ "$str1" == "$str2" ]]    # 相等
2 [[ "$str1" != "$str2" ]]    # 不相等
3 [[ -z "$str" ]]             # 字符串为空
4 [[ -n "$str" ]]             # 字符串非空
5 [[ "$str" == pattern* ]]    # 模式匹配（支持通配符）
6 [[ "$str" =~ regex ]]       # 正则表达式匹配
```

示例：

```
1 #!/bin/bash
2
3 read -p "Enter ROS distro [humble/iron]: " distro
```

```

4
5 if [[ -z "$distro" ]]; then
6     distro="humble"
7     echo "Using default: $distro"
8 fi
9
10 if [[ "$distro" == "humble" || "$distro" == "iron" ]]; then
11     echo "Setting up ROS 2 $distro..."
12     source "/opt/ros/$distro/setup.bash"
13 else
14     echo "Unknown distro: $distro"
15     exit 1
16 fi

```

数值比较

```

1 [[ $a -eq $b ]] # 等于
2 [[ $a -ne $b ]] # 不等于
3 [[ $a -lt $b ]] # 小于
4 [[ $a -le $b ]] # 小于等于
5 [[ $a -gt $b ]] # 大于
6 [[ $a -ge $b ]] # 大于等于
7
8 # 或者使用 (( )) 进行算术比较
9 (( a == b ))
10 (( a < b ))
11 (( a >= b ))

```

示例:

```

1 #!/bin/bash
2
3 cpu_usage=$(top -bn1 | grep "Cpu(s)" | awk '{print int($2)}')
4
5 if (( cpu_usage > 80 )); then
6     echo "Warning: High CPU usage: ${cpu_usage}%"
7 elif (( cpu_usage > 50 )); then
8     echo "CPU usage moderate: ${cpu_usage}%"
9 else
10     echo "CPU usage normal: ${cpu_usage}%"
11 fi

```

逻辑操作符

```

1 [[ condition1 && condition2 ]] # AND
2 [[ condition1 || condition2 ]] # OR
3 [[ ! condition ]] # NOT

```

1.3.10.4. 循环

for 循环

遍历列表:

```

1 #!/bin/bash
2
3 # 遍历列表
4 for fruit in apple banana orange; do
5     echo "I like $fruit"
6 done

```

```

7
8 # 遍历文件
9 for file in *.cpp; do
10     echo "Processing $file..."
11     # 对每个文件执行操作
12 done
13
14 # 遍历命令输出
15 for pkg in $(ros2 pkg list); do
16     echo "Found package: $pkg"
17 done
18
19 # 遍历数组
20 packages=("rm_vision" "rm_control" "rm_description")
21 for pkg in "${packages[@]"; do
22     echo "Building $pkg..."
23 done

```

C 风格的 for 循环:

```

1 #!/bin/bash
2
3 # 数字循环
4 for ((i = 1; i <= 10; i++)); do
5     echo "Iteration $i"
6 done
7
8 # 使用 seq 命令
9 for i in $(seq 1 10); do
10     echo "Number: $i"
11 done
12
13 # 使用花括号展开
14 for i in {1..10}; do
15     echo "Count: $i"
16 done
17
18 # 带步长
19 for i in {0..100..10}; do
20     echo "Step: $i"
21 done

```

while 循环

```

1 #!/bin/bash
2
3 # 基本 while 循环
4 count=1
5 while (( count <= 5 )); do
6     echo "Count: $count"
7     ((count++))
8 done
9
10 # 读取文件的每一行
11 while IFS= read -r line; do
12     echo "Line: $line"
13 done < "input.txt"
14
15 # 无限循环 (常用于监控脚本)
16 while true; do
17     echo "Checking system status..."
18     # 检查某些条件

```

```

19     sleep 5
20 done

```

等待条件满足:

```

1  #!/bin/bash
2
3  # 等待 ROS 2 节点启动
4  echo "Waiting for detector_node..."
5  while ! ros2 node list 2>/dev/null | grep -q "detector_node"; do
6      sleep 1
7  done
8  echo "detector_node is running!"
9
10 # 等待文件出现
11 while [[ ! -f "/tmp/ready.flag" ]]; do
12     echo "Waiting for initialization..."
13     sleep 2
14 done
15 echo "System ready!"

```

循环控制

```

1  #!/bin/bash
2
3  for i in {1..10}; do
4      if (( i == 5 )); then
5          continue # 跳过本次迭代
6      fi
7      if (( i == 8 )); then
8          break      # 退出循环
9      fi
10     echo "Number: $i"
11 done
12 # 输出: 1 2 3 4 6 7

```

1.3.10.5. 函数

函数让你可以将代码组织成可重用的块:

```

1  #!/bin/bash
2
3  # 定义函数
4  greet() {
5      echo "Hello, $1!" # $1 是第一个参数
6  }
7
8  # 调用函数
9  greet "Alice"
10 greet "Bob"
11
12 # 带多个参数的函数
13 log_message() {
14     local level="$1" # local 声明局部变量
15     local message="$2"
16     local timestamp=$(date +"%Y-%m-%d %H:%M:%S")
17     echo "[$timestamp] [$level] $message"
18 }
19
20 log_message "INFO" "Starting application"
21 log_message "ERROR" "Connection failed"

```

函数可以返回值，但只能返回 0-255 的整数作为退出状态。要返回其他值，用 `echo` 输出并捕获：

```
1 #!/bin/bash
2
3 # 返回退出状态
4 is_ros_installed() {
5     if [[ -d "/opt/ros" ]]; then
6         return 0 # 成功/真
7     else
8         return 1 # 失败/假
9     fi
10 }
11
12 if is_ros_installed; then
13     echo "ROS is installed"
14 fi
15
16 # 返回字符串值
17 get_ros_distro() {
18     if [[ -d "/opt/ros/humble" ]]; then
19         echo "humble"
20     elif [[ -d "/opt/ros/iron" ]]; then
21         echo "iron"
22     else
23         echo "unknown"
24     fi
25 }
26
27 distro=$(get_ros_distro)
28 echo "Detected ROS 2 distro: $distro"
29
30 # 返回计算结果
31 add_numbers() {
32     local a=$1
33     local b=$2
34     echo $((a + b))
35 }
36
37 result=$(add_numbers 5 3)
38 echo "5 + 3 = $result"
```

1.3.10.6. 命令行参数与退出状态

处理命令行参数让脚本更灵活：

```
1 #!/bin/bash
2
3 # 简单的参数处理
4 echo "Script name: $0"
5 echo "First argument: $1"
6 echo "Second argument: $2"
7 echo "All arguments: @$@"
8 echo "Number of arguments: $#"
```

```
9
10 # 检查参数
11 if [[ $# -lt 1 ]]; then
12     echo "Usage: $0 <project_name> [build_type]"
13     exit 1
14 fi
15
16 project_name="$1"
```

```

17 build_type="${2:-Release}" # 默认值
18
19 echo "Building $project_name in $build_type mode"

```

更复杂的参数解析可以用 `getopts` :

```

1  #!/bin/bash
2
3  # 默认值
4  verbose=false
5  output_dir="./output"
6  config_file=""
7
8  # 显示帮助
9  show_help() {
10     echo "Usage: $0 [-v] [-o output_dir] [-c config_file] <input>"
11     echo "  -v          Verbose mode"
12     echo "  -o output_dir  Output directory (default: ./output)"
13     echo "  -c config_file Configuration file"
14     echo "  -h          Show this help"
15 }
16
17 # 解析选项
18 while getopts "vo:c:h" opt; do
19     case $opt in
20         v)
21             verbose=true
22             ;;
23         o)
24             output_dir="$OPTARG"
25             ;;
26         c)
27             config_file="$OPTARG"
28             ;;
29         h)
30             show_help
31             exit 0
32             ;;
33         \?)
34             echo "Invalid option: -$OPTARG"
35             exit 1
36             ;;
37     esac
38 done
39
40 # 移除已处理的选项, 剩下的是位置参数
41 shift $((OPTIND - 1))
42
43 # 检查必需参数
44 if [[ $# -lt 1 ]]; then
45     echo "Error: Missing input file"
46     show_help
47     exit 1
48 fi
49
50 input_file="$1"
51
52 # 使用参数
53 echo "Input: $input_file"
54 echo "Output: $output_dir"
55 echo "Config: $config_file"
56 echo "Verbose: $verbose"

```


退出状态

每个命令执行后都有一个退出状态码（0 表示成功，非 0 表示失败）。脚本也应该返回适当的退出状态：

```
1 #!/bin/bash
2
3 # 检查命令是否成功
4 if ! colcon build; then
5     echo "Build failed!"
6     exit 1
7 fi
8
9 echo "Build successful!"
10 exit 0
```

使用 `set` 可以改变脚本的行为：

```
1 #!/bin/bash
2
3 set -e # 任何命令失败立即退出
4 set -u # 使用未定义变量时报错
5 set -o pipefail # 管道中任何命令失败则整体失败
6
7 # 通常组合使用
8 set -euo pipefail
9
10 # 现在任何错误都会导致脚本退出
11 cd /nonexistent_dir # 这里会失败并退出
12 echo "This won't be printed"
```

1.3.10.7. 实用脚本示例

让我们看几个 RoboMaster 开发中实用的脚本。

编译脚本

```
1 #!/bin/bash
2 # build.sh - ROS 2 工作空间编译脚本
3
4 set -euo pipefail
5
6 # 配置
7 WORKSPACE="${HOME}/ros2_ws"
8 ROS_DISTRO="${ROS_DISTRO:-humble}"
9 BUILD_TYPE="${1:-Release}"
10 PARALLEL_JOBS=$(nproc)
11
12 # 颜色输出
13 RED='\033[0;31m'
14 GREEN='\033[0;32m'
15 YELLOW='\033[1;33m'
16 NC='\033[0m' # No Color
17
18 log_info() {
19     echo -e "${GREEN}[INFO]${NC} $1"
20 }
21
22 log_warn() {
23     echo -e "${YELLOW}[WARN]${NC} $1"
24 }
25
```

```

26 log_error() {
27     echo -e "${RED}[ERROR]${NC} $1"
28 }
29
30 # 检查环境
31 check_environment() {
32     if [[ ! -d "/opt/ros/$ROS_DISTRO" ]]; then
33         log_error "ROS 2 $ROS_DISTRO not found!"
34         exit 1
35     fi
36
37     if [[ ! -d "$WORKSPACE" ]]; then
38         log_error "Workspace not found: $WORKSPACE"
39         exit 1
40     fi
41 }
42
43 # 清理构建
44 clean_build() {
45     log_info "Cleaning build directories..."
46     rm -rf "$WORKSPACE/build" "$WORKSPACE/install" "$WORKSPACE/log"
47 }
48
49 # 编译
50 build() {
51     log_info "Building workspace in $BUILD_TYPE mode..."
52
53     cd "$WORKSPACE"
54     source "/opt/ros/$ROS_DISTRO/setup.bash"
55
56     colcon build \
57         --cmake-args "-DCMAKE_BUILD_TYPE=$BUILD_TYPE" \
58         --parallel-workers "$PARALLEL_JOBS" \
59         --symlink-install
60
61     if [[ $? -eq 0 ]]; then
62         log_info "Build completed successfully!"
63     else
64         log_error "Build failed!"
65         exit 1
66     fi
67 }
68
69 # 主流程
70 main() {
71     log_info "ROS 2 Build Script"
72     log_info "Workspace: $WORKSPACE"
73     log_info "ROS Distro: $ROS_DISTRO"
74     log_info "Build Type: $BUILD_TYPE"
75
76     check_environment
77
78     if [[ "${2:-}" == "--clean" ]]; then
79         clean_build
80     fi
81
82     build
83
84     log_info "To use the workspace, run:"
85     echo "    source $WORKSPACE/install/setup.bash"
86 }
87
88 main "$@"

```

环境初始化脚本

```
1 #!/bin/bash
2 # setup_dev.sh - 开发环境初始化脚本
3
4 set -euo pipefail
5
6 echo "=====
7 echo "RoboMaster Development Environment Setup"
8 echo "=====
9
10 # 检查是否为 root
11 if [[ $EUID -eq 0 ]]; then
12     echo "Please don't run this script as root"
13     exit 1
14 fi
15
16 # 更新系统
17 echo "Updating system packages..."
18 sudo apt update && sudo apt upgrade -y
19
20 # 安装基础工具
21 echo "Installing development tools..."
22 sudo apt install -y \
23     build-essential \
24     cmake \
25     git \
26     vim \
27     curl \
28     wget \
29     htop \
30     tree \
31     python3-pip
32
33 # 安装 ROS 2 依赖
34 echo "Installing ROS 2 dependencies..."
35 sudo apt install -y \
36     libopencv-dev \
37     libeigen3-dev \
38     libfmt-dev \
39     libspdlog-dev \
40     libyaml-cpp-dev
41
42 # 检查 ROS 2
43 if [[ ! -d "/opt/ros/humble" ]]; then
44     echo "ROS 2 Humble not found. Please install it first."
45     echo "See: https://docs.ros.org/en/humble/Installation.html"
46 else
47     echo "ROS 2 Humble found."
48 fi
49
50 # 创建工作空间
51 WORKSPACE="$HOME/ros2_ws"
52 if [[ ! -d "$WORKSPACE" ]]; then
53     echo "Creating workspace at $WORKSPACE..."
54     mkdir -p "$WORKSPACE/src"
55 fi
56
57 # 配置 bashrc
58 if ! grep -q "source /opt/ros/humble/setup.bash" ~/.bashrc; then
59     echo "Configuring .bashrc..."
60     {
61         echo ""
62         echo "# ROS 2 Setup"
```

```

63         echo "source /opt/ros/humble/setup.bash"
64         echo "source $WORKSPACE/install/setup.bash 2>/dev/null || true"
65         echo "export ROS_DOMAIN_ID=0"
66     } >> ~/.bashrc
67 fi
68
69 # 配置 Git (如果未配置)
70 if [[ -z "$(git config --global user.name)" ]]; then
71     read -p "Enter your Git username: " git_name
72     read -p "Enter your Git email: " git_email
73     git config --global user.name "$git_name"
74     git config --global user.email "$git_email"
75 fi
76
77 # 添加用户到必要的组
78 sudo usermod -aG dialout "$USER"
79 sudo usermod -aG video "$USER"
80
81 echo ""
82 echo "=====
83 echo "Setup complete!"
84 echo "Please log out and log back in for group changes to take effect."
85 echo "Then run: source ~/.bashrc"
86 echo "=====

```

备份脚本

```

1  #!/bin/bash
2  # backup.sh - 项目备份脚本
3
4  set -euo pipefail
5
6  # 配置
7  SOURCE_DIR="${1:-$HOME/ros2_ws}"
8  BACKUP_DIR="${2:-$HOME/backups}"
9  MAX_BACKUPS=5
10
11 # 创建备份目录
12 mkdir -p "$BACKUP_DIR"
13
14 # 生成备份文件名
15 timestamp=$(date +%Y%m%d_%H%M%S)
16 project_name=$(basename "$SOURCE_DIR")
17 backup_file="$BACKUP_DIR/${project_name}_${timestamp}.tar.gz"
18
19 echo "Backing up $SOURCE_DIR..."
20 echo "Destination: $backup_file"
21
22 # 创建备份 (排除构建目录和大文件)
23 tar czf "$backup_file" \
24     --exclude='build' \
25     --exclude='install' \
26     --exclude='log' \
27     --exclude='.git' \
28     --exclude='*.bag' \
29     --exclude='*.onnx' \
30     -C "$(dirname "$SOURCE_DIR")" \
31     "$(basename "$SOURCE_DIR")"
32
33 # 显示备份大小
34 backup_size=$(du -h "$backup_file" | cut -f1)
35 echo "Backup created: $backup_file ($backup_size)"
36

```

```

37 # 清理旧备份
38 backup_count=$(ls -l "$BACKUP_DIR/${project_name}_".tar.gz 2>/dev/null | wc
-l)
39 if (( backup_count > MAX_BACKUPS )); then
40     echo "Cleaning old backups (keeping $MAX_BACKUPS)..."
41     ls -lt "$BACKUP_DIR/${project_name}_".tar.gz | tail -n +$((MAX_BACKUPS
+ 1)) | xargs rm -f
42 fi
43
44 echo "Backup complete!"

```

启动脚本

```

1 #!/bin/bash
2 # launch_robot.sh - 机器人系统启动脚本
3
4 set -euo pipefail
5
6 SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
7 LOG_DIR="$HOME/robot_logs"
8 TIMESTAMP=$(date +%Y%m%d_%H%M%S)
9
10 # 创建日志目录
11 mkdir -p "$LOG_DIR"
12
13 # 设置环境
14 source /opt/ros/humble/setup.bash
15 source "$HOME/ros2_ws/install/setup.bash"
16
17 # 清理函数
18 cleanup() {
19     echo "Shutting down..."
20     # 终止所有相关进程
21     pkill -f "ros2" || true
22     echo "Cleanup complete"
23 }
24
25 # 注册清理函数
26 trap cleanup EXIT
27
28 # 检查串口权限
29 if [[ ! -r "/dev/ttyUSB0" ]]; then
30     echo "Warning: Cannot access /dev/ttyUSB0"
31     echo "Run: sudo usermod -aG dialout $USER"
32 fi
33
34 # 启动节点
35 echo "Starting robot system..."
36 echo "Logs: $LOG_DIR/robot_${TIMESTAMP}.log"
37
38 ros2 launch rm_bringup robot_launch.py 2>&1 | tee "$LOG_DIR/
robot_${TIMESTAMP}.log"

```

Shell 脚本是提高工作效率的利器。从简单的命令组合开始，逐渐添加变量、条件、循环、函数，你可以构建出功能强大的自动化工具。好的脚本应该有清晰的注释、恰当的错误处理、友好的输出信息。随着经验积累，你会建立起自己的脚本库，让重复性工作变得轻松愉快。记住，如果某件事你需要做两次以上，就值得写一个脚本来自动化它。

1.3.11. 开发环境配置

工具决定效率。一个精心配置的开发环境可以让你的工作事半功倍，而一个混乱的环境会让简单的任务变得繁琐。本节将带你从 Shell 配置开始，逐步打造一个高效、舒适的 Linux 开发环境。我们会配置命令行提示符和别名、介绍 Zsh 和 Oh My Zsh 的美化方案、学习 tmux 终端复用、设置 VS Code 远程开发，最后梳理 CMake 项目的完整构建流程。这些配置一次到位，将长期提升你的开发体验。

1.3.11.1. Shell 配置

Shell 的行为由配置文件控制。对于 Bash，最重要的配置文件是 `~/.bashrc`，它在每次启动交互式 Shell 时执行。

理解配置文件

Bash 有几个配置文件，在不同场景下加载：

- `~/.bashrc`：交互式非登录 Shell（如打开终端窗口）
- `~/.bash_profile` 或 `~/.profile`：登录 Shell（如 SSH 登录、TTY 登录）
- `~/.bash_logout`：退出登录 Shell 时执行

大多数情况下，你只需要编辑 `~/.bashrc`。为了确保登录 Shell 也能加载它，通常在 `~/.profile` 中添加：

```
1 # ~/.profile
2 if [ -n "$BASH_VERSION" ]; then
3     if [ -f "$HOME/.bashrc" ]; then
4         . "$HOME/.bashrc"
5     fi
6 fi
```

定制 .bashrc

一个典型的 `.bashrc` 配置：

```
1 # ~/.bashrc
2
3 # 如果不是交互式 Shell，不加载配置
4 case $- in
5     *i*) ;;
6     *) return;;
7 esac
8
9 # ===== 历史记录配置 =====
10 HISTSIZE=10000 # 内存中保存的历史条数
11 HISTFILESIZE=20000 # 文件中保存的历史条数
12 HISTCONTROL=ignoreboth # 忽略重复和空白开头的命令
13 shopt -s histappend # 追加而不是覆盖历史文件
14
15 # ===== Shell 选项 =====
16 shopt -s checkwinsize # 自动调整窗口大小
17 shopt -s globstar # * 匹配多级目录
18 shopt -s cdspell # 自动纠正 cd 的拼写错误
19
20 # ===== 提示符配置 =====
21 # 带颜色的提示符：用户@主机:目录$
22 PS1='\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '
23
24 # 带 Git 分支的提示符
25 parse_git_branch() {
```

```

26     git branch 2>/dev/null | sed -e '/^[^*]/d' -e 's/* \(.*/ (\1)/'
27 }
28 PS1='\[\033[01;32m\]\u@h\[\033[00m\]:
   \[\033[01;34m\]\w\[\033[33m\]$(parse_git_branch)\[\033[00m\]\$ '
29
30 # ===== 别名 =====
31 # 文件操作
32 alias ll='ls -aLF'
33 alias la='ls -A'
34 alias l='ls -CF'
35 alias ls='ls --color=auto'
36
37 # 安全操作
38 alias rm='rm -i'
39 alias cp='cp -i'
40 alias mv='mv -i'
41
42 # 快捷操作
43 alias ..='cd ..'
44 alias ...='cd ../..'
45 alias ....='cd ../../..'
46
47 # Git 别名
48 alias gs='git status'
49 alias ga='git add'
50 alias gc='git commit'
51 alias gp='git push'
52 alias gl='git log --oneline -10'
53 alias gd='git diff'
54
55 # ROS 2 别名
56 alias cb='colcon build --symlink-install'
57 alias cbs='colcon build --symlink-install --packages-select'
58 alias cbd='colcon build --symlink-install --cmake-args -
   DCMAKE_BUILD_TYPE=Debug'
59 alias cbr='colcon build --symlink-install --cmake-args -
   DCMAKE_BUILD_TYPE=Release'
60 alias st='source install/setup.bash'
61
62 # 系统监控
63 alias ports='ss -tuln'
64 alias meminfo='free -h'
65 alias diskinfo='df -h'
66
67 # ===== 环境变量 =====
68 export EDITOR=vim
69 export VISUAL=vim
70 export PATH="$HOME/.local/bin:$PATH"
71
72 # ROS 2 环境
73 if [ -f /opt/ros/humble/setup.bash ]; then
74     source /opt/ros/humble/setup.bash
75 fi
76
77 # 工作空间环境
78 if [ -f ~/ros2_ws/install/setup.bash ]; then
79     source ~/ros2_ws/install/setup.bash
80 fi
81
82 export ROS_DOMAIN_ID=0
83
84 # ===== 自定义函数 =====
85 # 创建并进入目录

```

```

86 mkcd() {
87     mkdir -p "$1" && cd "$1"
88 }
89
90 # 快速查找文件
91 ff() {
92     find . -name "$1*"
93 }
94
95 # 快速查找内容
96 fg() {
97     grep -rn "$1" .
98 }
99
100 # ROS 2 工作空间快速切换
101 ws() {
102     local workspace="${1:-ros2_ws}"
103     cd "$HOME/$workspace" && source install/setup.bash 2>/dev/null
104     echo "Switched to $workspace"
105 }

```

修改 `.bashrc` 后, 执行 `source ~/.bashrc` 使更改生效, 或打开新终端。

别名的威力

别名是提高效率的简单方法。把常用的长命令定义成短别名:

```

1 # 项目特定的别名
2 alias rmv='cd ~/ros2_ws/src/rm_vision'
3 alias rmc='cd ~/ros2_ws/src/rm_control'
4
5 # 编译特定包
6 alias build_vision='colcon build --packages-select rm_vision rm_detector
  rm_tracker'
7
8 # 启动常用的 launch 文件
9 alias launch_robot='ros2 launch rm_bringup robot.launch.py'
10 alias launch_sim='ros2 launch rm_gazebo simulation.launch.py'
11
12 # 连接机器人
13 alias ssh_robot='ssh alice@192.168.1.50'
14
15 # 查看特定话题
16 alias echo_armor='ros2 topic echo /detector/armors'

```

1.3.11.2. Zsh 与 Oh My Zsh

Zsh (Z Shell) 是 Bash 的现代替代品, 提供更强大的自动补全、更好的脚本支持和丰富的自定义能力。Oh My Zsh 是一个 Zsh 配置框架, 让 Zsh 的配置变得简单。

安装 Zsh

```

1 # 安装 Zsh
2 sudo apt install zsh
3
4 # 验证安装
5 zsh --version
6 zsh 5.8.1
7
8 # 设为默认 Shell
9 chsh -s $(which zsh)
10 # 需要重新登录生效

```


安装 Oh My Zsh

```
1 sh -c "$(curl -fsSL https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

安装后，Zsh 配置文件是 `~/.zshrc`。Oh My Zsh 提供了主题和插件系统。

配置主题

Oh My Zsh 自带很多主题。编辑 `~/.zshrc`：

```
1 # 使用 agnoster 主题 (需要 Powerline 字体)
2 ZSH_THEME="agnoster"
3
4 # 或者使用简洁的 robbyrussell (默认)
5 ZSH_THEME="robbyrussell"
6
7 # 或者随机主题
8 ZSH_THEME="random"
```

推荐安装 Powerlevel10k 主题，它提供漂亮的提示符和丰富的信息显示：

```
1 # 安装 Powerlevel10k
2 git clone --depth=1 https://github.com/romkatv/powerlevel10k.git \
3     ${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
4
5 # 在 ~/.zshrc 中设置
6 ZSH_THEME="powerlevel10k/powerlevel10k"
7
8 # 重启终端，会自动启动配置向导
```

如果字符显示为方块，需要安装 Nerd Fonts：

```
1 # 下载并安装字体 (以 MesloLGS NF 为例)
2 mkdir -p ~/.local/share/fonts
3 cd ~/.local/share/fonts
4 curl -fLO https://github.com/romkatv/powerlevel10k-media/raw/master/MesloLGS%
5 20NF%20Regular.ttf
6 fc-cache -fv
7 # 然后在终端设置中选择这个字体
```

配置插件

Oh My Zsh 的插件提供额外的功能。编辑 `~/.zshrc`：

```
1 plugins=(
2     git                # Git 别名和补全
3     sudo               # 双击 Esc 在命令前加 sudo
4     history            # 历史命令相关
5     colored-man-pages  # 彩色 man 页面
6     command-not-found  # 提示安装缺失的命令
7     extract            # 智能解压 (x 命令)
8     z                  # 目录跳转 (z dirname)
9 )
```

两个强烈推荐的第三方插件：

```
1 # zsh-autosuggestions: 基于历史的命令建议
2 git clone https://github.com/zsh-users/zsh-autosuggestions \
3     ${ZSH_CUSTOM:-~/.oh-my-zsh/custom}/plugins/zsh-autosuggestions
```

```

4
5 # zsh-syntax-highlighting: 命令语法高亮
6 git clone https://github.com/zsh-users/zsh-syntax-highlighting.git \
7   ${ZSH_CUSTOM:~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting
8
9 # 添加到插件列表
10 plugins=(
11   git
12   sudo
13   z
14   zsh-autosuggestions
15   zsh-syntax-highlighting
16 )

```

迁移 Bash 配置

如果你从 Bash 迁移到 Zsh，大部分配置可以直接复制。在 `~/.zshrc` 末尾添加：

```

1 # 加载 ROS 2
2 source /opt/ros/humble/setup.zsh
3
4 # 工作空间
5 source ~/ros2_ws/install/setup.zsh 2>/dev/null
6
7 # 别名（大部分与 Bash 相同）
8 alias ll='ls -alF'
9 alias cb='colcon build --symlink-install'
10 # ...其他别名
11
12 # 自定义函数也可以直接复制
13 mkcd() {
14   mkdir -p "$1" && cd "$1"
15 }

```

注意 ROS 2 的 setup 文件有 `.zsh` 版本。

1.3.11.3. tmux 终端复用

tmux (Terminal Multiplexer) 让你在一个终端窗口中运行多个会话。更重要的是，会话可以在断开连接后继续运行——这对于远程开发至关重要。

安装和基本概念

```
1 sudo apt install tmux
```

tmux 有三个层次的概念：

- **会话 (Session)**：一组窗口的集合，可以独立于终端存在
- **窗口 (Window)**：类似于标签页，每个会话可以有多个窗口
- **面板 (Pane)**：窗口内的分割区域

基本操作

```

1 # 启动新会话
2 tmux
3
4 # 启动命名会话
5 tmux new -s robot
6
7 # 列出会话
8 tmux ls

```

```
9
10 # 连接到会话
11 tmux attach -t robot
12 # 或简写
13 tmux a -t robot
14
15 # 断开会话（会话继续运行）
16 # 在 tmux 中按 Ctrl+B 然后 D
17
18 # 关闭会话
19 tmux kill-session -t robot
```

tmux 的所有快捷键都以前缀键开始，默认是 **Ctrl+B**。按下前缀键后，再按其他键执行操作。

常用快捷键

会话管理（前缀 + 键）：

- **d**：断开当前会话
- **s**：列出会话并切换
- **\$**：重命名当前会话

窗口管理：

- **c**：创建新窗口
- **n**：下一个窗口
- **p**：上一个窗口
- **数字**：切换到指定窗口
- **,**：重命名当前窗口
- **&**：关闭当前窗口

面板管理：

- **%**：垂直分割（左右）
- **"**：水平分割（上下）
- **方向键**：在面板间移动
- **x**：关闭当前面板
- **z**：最大化/恢复当前面板
- **空格**：切换面板布局

其他：

- **?**：显示帮助
- **[**：进入复制模式（可滚动查看历史）
- **]**：粘贴

tmux 配置

创建 `~/.tmux.conf` 自定义 **tmux**：

```
1 # ~/.tmux.conf
2
3 # 更改前缀键为 Ctrl+A（更容易按）
4 unbind C-b
5 set -g prefix C-a
6 bind C-a send-prefix
7
8 # 启用鼠标支持
```

```

9  set -g mouse on
10
11 # 从 1 开始编号 (0 太远了)
12 set -g base-index 1
13 setw -g pane-base-index 1
14
15 # 更直观的分割快捷键
16 bind | split-window -h -c "#{pane_current_path}"
17 bind - split-window -v -c "#{pane_current_path}"
18
19 # 使用 Alt+方向键切换面板 (无需前缀)
20 bind -n M-Left select-pane -L
21 bind -n M-Right select-pane -R
22 bind -n M-Up select-pane -U
23 bind -n M-Down select-pane -D
24
25 # 快速重载配置
26 bind r source-file ~/.tmux.conf \; display "Config reloaded!"
27
28 # 256 色支持
29 set -g default-terminal "screen-256color"
30
31 # 增加历史记录
32 set -g history-limit 10000
33
34 # 减少命令延迟
35 set -sg escape-time 0
36
37 # 状态栏美化
38 set -g status-style bg=black,fg=white
39 set -g status-left '[#S] '
40 set -g status-right '%H:%M %d-%b-%y'

```

RoboMaster 开发的 tmux workflow

一个典型的机器人开发 tmux 布局:

```

1  # 创建会话
2  tmux new -s robot
3
4  # 窗口 1: 代码编辑 (默认窗口)
5  vim ~/ros2_ws/src/rm_vision/...
6
7  # 创建窗口 2: 编译和运行
8  Ctrl+B c
9  cd ~/ros2_ws && colcon build
10
11 # 创建窗口 3: 监控 (分割成多个面板)
12 Ctrl+B c
13 Ctrl+B % # 垂直分割
14 htop # 左侧: 系统监控
15 Ctrl+B 方向键切换到右侧
16 Ctrl+B " # 水平分割
17 ros2 topic echo /armors # 右上: 话题监控
18 Ctrl+B 方向键切换到右下
19 tail -f ~/ros2_ws/log/latest/*.log # 右下: 日志
20
21 # 断开会话 (会话保持运行)
22 Ctrl+B d
23
24 # 重新连接
25 tmux a -t robot

```

1.3.11.4. VS Code 远程开发

VS Code 的 Remote-SSH 扩展让你可以在本地编辑远程服务器上的代码, 同时享受完整的 IDE 功能。

安装和配置

1. 在本地 VS Code 中安装 “Remote - SSH” 扩展
2. 配置 SSH (确保已设置密钥认证):

```
1 # ~/.ssh/config
2 Host robot
3     HostName 192.168.1.50
4     User alice
5     IdentityFile ~/.ssh/id_ed25519
```

3. 在 VS Code 中连接:
 - 按 **F1** 或 **Ctrl+Shift+P**
 - 输入 “Remote-SSH: Connect to Host”
 - 选择配置的主机
4. 首次连接时, VS Code 会在远程主机安装服务器组件

远程开发 workflow

连接后, VS Code 的所有功能都在远程运行:

- 文件浏览器显示远程文件系统
- 终端是远程 Shell
- 扩展在远程执行 (需要在远程重新安装一些扩展)
- Git 集成使用远程仓库

推荐在远程安装的扩展:

- C/C++ (微软官方)
- CMake Tools
- Python
- ROS (如果开发 ROS 2)

配置 VS Code for ROS 2

在远程项目中创建 `.vscode/settings.json`:

```
1 {
2     "C_Cpp.default.configurationProvider": "ms-vscode.cmake-tools",
3     "cmake.configureOnOpen": true,
4     "cmake.buildDirectory": "${workspaceFolder}/build",
5     "files.associations": {
6         "*.launch.py": "python",
7         "*.xacro": "xml"
8     },
9     "python.analysis.extraPaths": [
10         "/opt/ros/humble/lib/python3.10/site-packages",
11         "${workspaceFolder}/install/lib/python3.10/site-packages"
12     ],
13     "terminal.integrated.env.linux": {
14         "PYTHONPATH": "/opt/ros/humble/lib/python3.10/site-packages:
15         ${workspaceFolder}/install/lib/python3.10/site-packages"
16     }
17 }
```

创建 `.vscode/c_cpp_properties.json` :

```
1 {
2     "configurations": [
3         {
4             "name": "Linux",
5             "includePath": [
6                 "${workspaceFolder}/*",
7                 "/opt/ros/humble/include/*",
8                 "/usr/include/*"
9             ],
10            "defines": [],
11            "compilerPath": "/usr/bin/g++",
12            "cStandard": "c17",
13            "cppStandard": "c++17",
14            "intelliSenseMode": "linux-gcc-x64",
15            "configurationProvider": "ms-vscode.cmake-tools"
16        }
17    ],
18    "version": 4
19 }
```

1.3.11.5. CMake 项目构建流程

CMake 是 C/C++ 项目的标准构建系统，理解其工作流程对于 RoboMaster 开发很重要。

标准 CMake 流程

```
1 # 1. 创建构建目录（外部构建）
2 mkdir build && cd build
3
4 # 2. 配置（生成构建系统）
5 cmake ..
6 # 或指定选项
7 cmake -DCMAKE_BUILD_TYPE=Release \
8       -DCMAKE_INSTALL_PREFIX=/usr/local \
9       ..
10
11 # 3. 编译
12 make -j$(nproc)
13 # 或使用 cmake 统一接口
14 cmake --build . --parallel
15
16 # 4. 安装（可选）
17 sudo make install
18 # 或
19 cmake --install .
```

常用 CMake 选项

```
1 # 构建类型
2 -DCMAKE_BUILD_TYPE=Debug|Release|RelWithDebInfo|MinSizeRel
3
4 # 安装路径
5 -DCMAKE_INSTALL_PREFIX=/path/to/install
6
7 # 指定编译器
8 -DCMAKE_C_COMPILER=gcc-11
9 -DCMAKE_CXX_COMPILER=g++-11
10
11 # 指定生成器
12 -G "Ninja"           # 使用 Ninja (比 Make 快)
```

```

13 -G "Unix Makefiles" # 默认
14
15 # 自定义选项（项目定义的）
16 -DWITH_CUDA=ON
17 -DBUILD_TESTS=OFF

```

ccmake 交互式配置

```

1 sudo apt install cmake-curses-gui
2 ccmake ..

```

ccmake 提供文本界面，让你可以浏览和修改所有 CMake 选项，非常方便探索项目的配置选项。

ROS 2 colcon 构建

ROS 2 使用 colcon 管理多包工作空间，它在底层调用 CMake：

```

1 # 进入工作空间
2 cd ~/ros2_ws
3
4 # 构建所有包
5 colcon build
6
7 # 常用选项
8 colcon build --symlink-install # 符号链接安装（便于开发）
9 colcon build --packages-select pkg1 pkg2 # 只构建指定包
10 colcon build --packages-up-to pkg1 # 构建包及其依赖
11 colcon build --cmake-args -DCMAKE_BUILD_TYPE=Debug
12
13 # 清理
14 rm -rf build/ install/ log/
15
16 # 构建后 source
17 source install/setup.bash

```

调试构建问题

```

1 # 详细输出
2 colcon build --event-handlers console_direct+
3
4 # 查看 CMake 配置输出
5 cat build/<package>/CMakeCache.txt
6
7 # 单独配置某个包（用于调试）
8 cd build/<package>
9 cmake ../../src/<package> -DCMAKE_BUILD_TYPE=Debug
10
11 # 查看编译命令
12 cmake --build . --verbose

```

1.3.11.6. 常见问题排查

找不到库或头文件

```

1 # 错误: fatal error: opencv2/opencv.hpp: No such file or directory
2
3 # 解决: 安装开发包
4 sudo apt install libopencv-dev
5

```

```

6 # 检查头文件位置
7 dpkg -L libopencv-dev | grep opencv.hpp
8
9 # 在 CMakeLists.txt 中正确 find_package
10 find_package(OpenCV REQUIRED)
11 target_link_libraries(my_target ${OpenCV_LIBS})
12 target_include_directories(my_target PRIVATE ${OpenCV_INCLUDE_DIRS})

```

链接错误

```

1 # 错误: undefined reference to `cv::imread(...)'
2
3 # 原因: 没有链接库
4 # 解决: 在 CMakeLists.txt 中添加 target_link_libraries
5
6 # 检查库是否安装
7 ldconfig -p | grep opencv

```

运行时找不到共享库

```

1 # 错误: error while loading shared libraries: libxxx.so
2
3 # 方法 1: 添加到 LD_LIBRARY_PATH
4 export LD_LIBRARY_PATH=/path/to/lib:$LD_LIBRARY_PATH
5
6 # 方法 2: 添加到系统库路径
7 echo "/path/to/lib" | sudo tee /etc/ld.so.conf.d/mylib.conf
8 sudo ldconfig
9
10 # 方法 3: 安装到标准位置
11 sudo make install
12 sudo ldconfig

```

ROS 2 包找不到

```

1 # 错误: Package 'xxx' not found
2
3 # 检查是否 source 了环境
4 echo $AMENT_PREFIX_PATH
5
6 # source ROS 2 和工作空间
7 source /opt/ros/humble/setup.bash
8 source ~/ros2_ws/install/setup.bash
9
10 # 检查包是否安装
11 ros2 pkg list | grep xxx
12
13 # 重新构建
14 colcon build --packages-select xxx

```

串口权限问题

```

1 # 错误: Permission denied: '/dev/ttyUSB0'
2
3 # 解决: 添加用户到 dialout 组
4 sudo usermod -aG dialout $USER
5 # 重新登录
6
7 # 临时解决
8 sudo chmod 666 /dev/ttyUSB0

```


SSH 连接问题

```
1 # 问题: Connection refused
2
3 # 检查 SSH 服务
4 sudo systemctl status ssh
5 sudo systemctl start ssh
6
7 # 检查防火墙
8 sudo ufw status
9 sudo ufw allow ssh
10
11 # 问题: Host key verification failed
12 # 解决: 删除旧的 known_hosts 条目
13 ssh-keygen -R hostname
```

内存不足导致编译失败

```
1 # 错误: c++: fatal error: Killed signal terminated program cclplus
2
3 # 原因: 内存不足, 编译器被 OOM killer 杀死
4
5 # 解决 1: 减少并行编译数
6 colcon build --parallel-workers 2
7
8 # 解决 2: 增加 swap
9 sudo fallocate -l 4G /swapfile
10 sudo chmod 600 /swapfile
11 sudo mkswap /swapfile
12 sudo swapon /swapfile
```

Git 子模块问题

```
1 # 克隆后子模块为空
2 git submodule update --init --recursive
3
4 # 子模块更新
5 git submodule update --remote
6
7 # 重置子模块
8 git submodule deinit -f .
9 git submodule update --init
```

一个配置良好的开发环境是高效工作的基础。花时间配置好 Shell、编辑器、终端复用，这些投入会在日后的开发中反复回报你。记住，最好的配置是适合你自己工作流程的配置——从这里的建议开始，逐渐调整成你喜欢的样子。

1.4. 计算机系统基础

1.4.1. 为什么需要了解计算机系统

1.4.2. 程序的执行：从源码到运行

1.4.3. 进程：程序的运行实例

1.4.4. 线程：轻量级执行单元

1.4.5. 内存层次结构

1.4.6. 虚拟内存

1.4.7. 进程的内存布局

1.4.8. 文件系统

1.4.9. 网络通信基础

1.4.10. 并发与同步

1.4.11. 操作系统的角色

1.4.12. 性能分析与调试工具

1.5. C++ 语言基础

1.5.1. C++ 基本程序结构

本章节介绍 C++ 程序的基本组成形式, 并说明源代码从编写到生成可执行文件的完整流程。内容包括程序结构、编译器 g++ 的使用方法以及构建系统 CMake 的基础用法。

一个最小的 C++ 程序通常由源文件 `/*.cpp` 和可选的头文件 `/*.h` 或 `/*.hpp` 组成。程序的入口由 `main()` 函数定义, 它是操作系统加载程序后第一个执行的函数。

下面是一个简单的 C++ 程序示例:

```
1 #include <iostream>           // 引入输入输出库
2 using namespace std;          // 使用标准命名空间
3 int main() {
4     cout << "Hello, RoboMaster!" << endl; // 输出文本
5     return 0;                 // 返回状态码
6 }
```

如果读者此前已经接触过 C 语言, 可以发现 C++ 与 C 语言的相似之处, 但必须说明的是: C++ 和 C 语言是两种不同的语言, C++ 既不是 C 语言的扩展, C 语言也不是 C++ 的简化。这一点将在后续的章节中越来越多地体现。对比 C++ 与 C 语言的 Hello RoboMaster 代码就能看出明显的区别, 这是 C 语言的 Hello World 代码, 能和上方代码输出相同的内容:

```
1 #include <stdio.h>             // 引入标准输入输出库
2 int main() {
3     printf("Hello, RoboMaster!\n"); // 输出文本
4     return 0;                 // 返回状态码
5 }
```

了解了 C 语言与 C++ 的不同之后，现在让我们来解释一下这段程序：

- **#include <iostream> :**

C++ 提供了一组标准头文件 (header files)，这些头文件包含了常用的类型、函数和类，程序只需包含相应头文件即可直接使用这些工具。就是其中用于“输入/输出”操作的头文件：I/O (Input/Output) 表示输入输出，stream (流) 表示数据按序列化地在程序内部或程序与外部设备之间传输的抽象。

在本程序中，我们需要借助标准输出能力将文本打印到终端，`<iostream>` 中正好提供了 `cout`、`cin`、`cerr` 等对象来完成这些工作。因此用 `#include <iostream>` 将相关声明引入到翻译单元中，编译器在预处理阶段会把头文件内容插入到源文件。

- **using namespace std; :**

C++ 使用命名空间 (namespace) 来组织符号，避免不同库之间的名称冲突。标准库中的大部分符号 (包括 `cout`、`endl`、`vector` 等) 都被放在 `std` 命名空间内。

`using namespace std;` 这行告诉编译器在当前作用域中可以不写 `std::` 前缀直接访问 `std` 下的符号。它方便了书写，但在较大型工程或头文件中滥用会增加命名冲突的风险。因此更推荐的做法是在需要时显式使用 `std::`：

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello World" << std::endl;
5     return 0;
6 }
```

这样更清晰，也更安全。

- **int main() :**

每个可独立运行的 C++ 程序都必须包含一个名为 `main()` 的函数：操作系统在启动程序时会调用它。`main` 的返回值类型通常为 `int`，代表程序的退出状态；返回 `0` 一般表示成功，非零值表示出现了错误或异常终止。

函数体由一对大括号 `{}` 包围，内部是**语句块** (statement block)，包含程序逻辑。

- **注释：// 与 /* ... */**

注释用于向阅读代码的人说明意图，编译器会忽略注释内容。C++ 支持两种注释形式：- `//` 单行注释：`//` 之后至行尾为注释。- `/* */` 多行注释：可跨行书写。

良好的注释能提高代码可读性，但注释内容应简洁、相关，避免重复说明代码本身能表达的事实。

- **cout << "Hello World" << endl; — 输出与“流”的概念**

这一行展示了 C++ 标准库 I/O 的核心风格：**流** (stream) 操作。`cout` (Console Output) 是一个输出流对象，负责将数据发送到标准输出 (通常是终端)。`<<` 是**插入运算符** (insertion operator)，表示“把右侧的值插入到左侧的流中”，可把多个项连续插入同一个流。`"Hello World"` 是一个字符串字面量，将被写入 `cout`。`endl` 是一个操作符，表示输出换行并刷新流缓冲区 (flush)，常用于确保输出及时显示。流操作的链式写法使得多个输出项可以按顺序组合，例如：

```
1 cout << "Hello" << " " << "World" << endl;
```

可以把这一过程理解为：字符串和 `endl` 按顺序“流向” `cout`，最终由 `cout` 将它们输出到终端。这个“流动”的抽象比直接调用格式化函数更面向对象，也更易于扩展（例如重载 `operator<<` 来支持自定义类型）。需要注意的一点是：`endl` 会强制刷新缓冲区，这在频繁输出的小段落中可能影响性能。若只需要换行而不刷新的话，可以使用 `'n'`。

- `return 0;`

`return 0;` 表示 `main` 正常结束并向操作系统返回成功状态。不同平台和约定会使用特定的非零值标识各种错误或异常退出原因，但通常情况下返回 0 即视为成功。

1.5.2. 编译一个 C++ 程序

在理解了程序结构之后，下一步需要了解：如何将源代码转换为计算机能够执行的程序。C++ 属于编译型语言，源代码必须经过编译器处理才能运行。本节将以实际示例为主，从一个文件的编译开始，逐步扩展到多文件项目和构建系统，帮助读者建立对工具链的完整认识。下面假设我们有一个最基本的程序 `main.cpp`：

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Hello, RoboMaster!" << endl;
6     return 0;
7 }
```

要将这个源文件编译成可执行文件，可以使用 GNU 编译器集合 (GCC) 中的 `g++` 命令行工具。假设读者已经在系统中安装了 `g++`，可以通过以下命令进行编译：

```
1 g++ main.cpp -o main
```

这里，`g++` 是调用 C++ 编译器的命令，`main.cpp` 是源文件名，`-o main` 指定输出文件名为 `main`（在 Windows 上会生成 `main.exe`）。执行该命令后，如果源代码没有语法错误，编译器会生成一个可执行文件 `main`。要运行生成的程序，可以在终端中输入：

```
1 ./main          # 在类 Unix 系统 (Linux、macOS) 上
```

至此，我们完成了一次最基础的 C++ 编译。

尽管我们只敲了一条命令，但编译器内部实际经历了四个阶段：

1. 预处理 Preprocessing

预处理器会处理以 `#` 开头的预处理指令，如 `#include`、`#define` 等。它会将包含的头文件内容插入到源文件中，展开宏定义等。预处理后的代码会生成一个临时文件，通常以 `.i` 结尾。我们可以用以下命令仅执行预处理阶段：

```
1 g++ -E main.cpp -o main.i
```

这时，`main.i` 文件中包含了展开后的代码。

2. 编译 Compilation

编译器将预处理后的代码转换为汇编代码。这个阶段会进行语法分析、语义检查和优化等。生成的汇编代码通常以 `.s` 结尾。可以用以下命令仅执行编译阶段：

```
1 g++ -S main.i -o main.s
```

输出文件 `main.s` 内是汇编语言。这时，我们可以查看汇编代码，了解编译器如何将 C++ 代码转换为底层指令。

3. 汇编 Assembly

汇编器将汇编代码转换为机器码，生成目标文件（object file），通常以 `.o` 结尾。目标文件包含了机器指令，但还不能独立运行。可以用以下命令仅执行汇编阶段：

```
1 g++ -c main.s -o main.o
```

这时，`main.o` 文件中包含了机器码，但还不能执行，因为它可能还依赖其他目标文件或库。

4. 链接 Linking

链接器将一个或多个目标文件和所需的库文件合并，生成最终的可执行文件。在这个阶段，链接器会解析符号引用，确保所有函数和变量都能正确连接。可以用以下命令仅执行链接阶段：

```
1 g++ main.o -o main
```

最终生成的 `main` 文件就是可执行程序，可以直接运行。

1.5.3. 变量与基本数据类型

程序的本质是处理数据。无论是计算电机转速、追踪目标位置，还是判断装甲板颜色，都需要在内存中存储和操作各种数据。C++ 通过变量来管理这些数据，而数据类型则决定了变量能够存储什么样的值、占用多少内存空间，以及可以进行哪些运算。

1.5.3.1. 变量的声明与初始化

变量是程序中用于存储数据的命名内存区域。在 C++ 中，使用变量之前必须先声明它的类型和名称。声明告诉编译器两件事：为这个变量分配多大的内存空间，以及如何解释这块内存中的二进制数据。

最基本的变量声明形式是“类型名 变量名”：

```
1 int score;           // 声明一个整数变量
2 double temperature; // 声明一个双精度浮点数变量
```

仅仅声明变量而不赋值是危险的做法。未初始化的变量包含的是内存中的随机数据，使用它们会导致不可预测的行为。因此，声明变量时应当同时进行初始化：

```
1 int score = 0;
2 double temperature = 36.5;
```

C++11 引入了统一初始化语法，使用花括号进行初始化。这种方式不仅适用于基本类型，也适用于复杂类型，并且能够防止某些隐式类型转换带来的精度损失：

```
1 int count{10};
2 double pi{3.14159};
3 int dangerous{3.14}; // 编译器会警告：浮点数转整数会丢失精度
```

当初始值的类型与变量类型不完全匹配时，花括号初始化会进行更严格的检查，这有助于在编译阶段发现潜在的错误。

1.5.3.2. 整数类型

整数是最常用的数据类型之一。C++ 提供了多种整数类型，它们的区别在于能够表示的数值范围和占用的内存大小。

`int` 是最常用的整数类型，在现代 64 位系统上通常占用 4 个字节（32 位），能够表示大约 ± 21 亿的范围。对于大多数日常计算，`int` 已经足够：

```
1 int motorSpeed = 3000;    // 电机转速 RPM
2 int bulletCount = 42;     // 弹丸数量
3 int targetDistance = 5000; // 目标距离 mm
```

当需要更大或更小的数值范围时，可以使用其他整数类型。`short` 通常占用 2 个字节，适合存储较小的数值以节省内存；`long` 和 `long long` 则提供更大的数值范围，后者保证至少 64 位，能够表示约 ± 922 亿亿的范围。

```
1 short sensorReading = 1024;    // 传感器读数
2 long long timestamp = 1703491200000; // 毫秒级时间戳
```

整数类型还分为有符号（signed）和无符号（unsigned）两种。有符号类型可以表示负数，无符号类型只能表示非负数，但正数的范围扩大一倍。默认情况下整数类型是有符号的，如果明确知道某个值不会为负（如数组索引、计数器），可以使用无符号类型：

```
1 unsigned int frameCount = 0; // 帧计数，不会为负
2 size_t arraySize = 100;     // size_t 是无符号类型，用于表示大小
```

需要注意的是，无符号整数在减法运算中可能产生意外结果。例如，`0u - 1` 的结果不是 `-1`，而是一个非常大的正数（无符号整数的最大值）。在 RoboMaster 开发中，涉及可能为负的计算时应当使用有符号类型。

1.5.3.3. 浮点类型

浮点类型用于表示带有小数部分的数值，在机器人控制中随处可见：角度、速度、PID 参数、坐标位置等都需要用浮点数表示。

C++ 提供三种浮点类型：`float`（单精度）、`double`（双精度）和 `long double`（扩展精度）。它们的区别在于精度和范围：

```
1 float angle = 45.0f;    // 单精度，约 7 位有效数字
2 double position = 1234.56789; // 双精度，约 15 位有效数字
```

`float` 占用 4 个字节，`double` 占用 8 个字节。虽然 `float` 节省内存，但其精度有限。在连续计算中，精度损失会逐渐累积，导致结果偏差。因此，除非有明确的内存或性能限制，一般推荐使用 `double`。

浮点数字面量默认是 `double` 类型。如果要表示 `float` 类型的字面量，需要在数字后加上 `f` 或 `F` 后缀：

```
1 float f1 = 3.14;    // 3.14 是 double，隐式转换为 float，可能有精度损失
2 float f2 = 3.14f;   // 3.14f 是 float，无需转换
```

浮点数的一个重要特性是它无法精确表示所有十进制小数。例如，0.1 在二进制浮点数中是一个无限循环小数，存储时会有微小的舍入误差。这意味着直接比较两个浮点数是否相等往往是不可靠的：


```

1 double a = 0.1 + 0.2;
2 double b = 0.3;
3 if (a == b) { // 危险! 可能不相等
4     // ...
5 }

```

正确的做法是判断两个浮点数的差值是否在可接受的误差范围内:

```

1 const double epsilon = 1e-9;
2 if (std::abs(a - b) < epsilon) {
3     // 认为相等
4 }

```

1.5.3.4. 字符类型

`char` 类型用于存储单个字符, 占用 1 个字节。字符字面量用单引号括起:

```

1 char grade = 'A';
2 char newline = '\n';
3 char tab = '\t';

```

在 C++ 中, `char` 本质上是一个小整数, 存储的是字符的编码值。在 ASCII 编码中, `'A'` 的值是 65, `'0'` 的值是 48。这一特性使得字符可以参与算术运算:

```

1 char c = 'A';
2 c = c + 1; // c 现在是 'B'
3
4 int digit = '7' - '0'; // 将字符 '7' 转换为整数 7

```

反斜杠 `\` 开头的是转义字符, 用于表示一些特殊字符: `\n` 表示换行, `\t` 表示制表符, `\\` 表示反斜杠本身, `\'` 表示单引号。

对于需要处理中文等非 ASCII 字符的场景, `char` 类型不再适用。C++11 引入了 `char16_t` 和 `char32_t` 用于 Unicode 字符, 但更常见的做法是使用 `std::string` 配合 UTF-8 编码处理多语言文本。

1.5.3.5. 布尔类型

`bool` 类型用于表示逻辑值, 只有两个可能的取值: `true` (真) 和 `false` (假)。布尔类型在条件判断和逻辑运算中广泛使用:

```

1 bool isTargetDetected = true;
2 bool isAutoAimEnabled = false;
3 bool shouldShoot = isTargetDetected && isAutoAimEnabled;

```

虽然 `bool` 在逻辑上只需要 1 位就能表示, 但由于内存对齐的原因, 它通常占用 1 个字节。

在 C++ 中, 布尔值可以隐式转换为整数 (`false` 转为 0, `true` 转为 1), 整数也可以隐式转换为布尔值 (0 转为 `false`, 非零值转为 `true`)。这种转换在条件语句中很常见:

```

1 int errorCode = someFunction();
2 if (errorCode) { // 非零值被视为 true
3     std::cout << "发生错误" << std::endl;
4 }

```

尽管这种隐式转换很方便，但过度依赖它会降低代码的可读性。在表达逻辑意图时，显式使用布尔表达式更为清晰。

1.5.3.6. auto 关键字

C++11 引入的 `auto` 关键字允许编译器根据初始值自动推断变量的类型。这在类型名称冗长或复杂时特别有用：

```
1 auto count = 10;           // 推断为 int
2 auto pi = 3.14159;         // 推断为 double
3 auto message = "Hello";    // 推断为 const char*
```

`auto` 的价值在复杂类型中体现得更为明显。例如，在使用标准库容器时，迭代器的类型往往非常冗长：

```
1 std::vector<int> numbers = {1, 2, 3, 4, 5};
2
3 // 不使用 auto
4 std::vector<int>::iterator it1 = numbers.begin();
5
6 // 使用 auto，简洁得多
7 auto it2 = numbers.begin();
```

使用 `auto` 时需要注意几点。首先，`auto` 声明的变量必须在声明时初始化，否则编译器无法推断类型。其次，`auto` 推断的类型可能与预期不同，特别是涉及引用和常量时。例如：

```
1 const int& ref = someValue;
2 auto a = ref; // a 的类型是 int，不是 const int&
```

如果需要保留引用或常量属性，应当显式声明：

```
1 const auto& b = ref; // b 的类型是 const int&
```

在 RoboMaster 开发中，`auto` 常用于 lambda 表达式、迭代器和复杂模板类型的声明，能够显著提高代码的简洁性。但对于基本类型，显式声明往往更清晰——`int count = 10` 比 `auto count = 10` 更直观地表达了变量的用途。

1.5.3.7. 类型转换

不同类型的数据在运算时可能需要相互转换。C++ 支持隐式转换和显式转换两种方式。

隐式转换由编译器自动完成，通常发生在混合类型运算中。基本原则是“向更宽的类型转换”，以避免数据丢失：

```
1 int a = 10;
2 double b = 3.14;
3 double c = a + b; // a 被隐式转换为 double，结果为 13.14
```

然而，隐式转换也可能导致意外的结果。例如，将浮点数赋值给整数变量时，小数部分会被截断：

```
1 int x = 3.99; // x 的值是 3，不是 4
```

当需要明确进行类型转换时，应当使用显式转换。C++ 提供了四种转换运算符，其中最常用的是 `static_cast`：


```

1 double pi = 3.14159;
2 int truncated = static_cast<int>(pi); // 显式截断为 3
3
4 int dividend = 7;
5 int divisor = 2;
6 double result = static_cast<double>(dividend) / divisor; // 结果为 3.5

```

`static_cast` 用于编译期可确定的安全转换，如数值类型之间的转换、向上转型等。其他三种转换运算符——`dynamic_cast`、`const_cast` 和 `reinterpret_cast`——用于更特殊的场景，将在后续章节介绍。

C 语言风格的强制转换（如 `(int)3.14`）在 C++ 中仍然有效，但不推荐使用。C++ 的转换运算符更加明确，也更容易在代码中搜索和审查。

1.5.3.8. 类型大小与 `sizeof` 运算符

不同平台上基本类型的大小可能有所不同。C++ 标准只规定了最小要求（如 `int` 至少 16 位），具体大小由编译器和平台决定。使用 `sizeof` 运算符可以查询类型或变量所占的字节数：

```

1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "char: " << sizeof(char) << " 字节" << endl;
6     cout << "short: " << sizeof(short) << " 字节" << endl;
7     cout << "int: " << sizeof(int) << " 字节" << endl;
8     cout << "long: " << sizeof(long) << " 字节" << endl;
9     cout << "long long: " << sizeof(long long) << " 字节" << endl;
10    cout << "float: " << sizeof(float) << " 字节" << endl;
11    cout << "double: " << sizeof(double) << " 字节" << endl;
12    cout << "bool: " << sizeof(bool) << " 字节" << endl;
13    return 0;
14 }

```

在典型的 64 位 Linux 系统上，输出通常为：

```

1 char: 1 字节
2 short: 2 字节
3 int: 4 字节
4 long: 8 字节
5 long long: 8 字节
6 float: 4 字节
7 double: 8 字节
8 bool: 1 字节

```

如果需要确切大小的整数类型，可以使用 `<cstdint>` 头文件中定义的定宽类型：

```

1 #include <cstdint>
2
3 int8_t a; // 精确 8 位有符号整数
4 int16_t b; // 精确 16 位有符号整数
5 int32_t c; // 精确 32 位有符号整数
6 int64_t d; // 精确 64 位有符号整数
7
8 uint8_t e; // 精确 8 位无符号整数
9 uint16_t f; // 精确 16 位无符号整数
10 uint32_t g; // 精确 32 位无符号整数
11 uint64_t h; // 精确 64 位无符号整数

```

在 RoboMaster 开发中, 与下位机通信时经常需要使用定宽类型来确保数据格式的一致性。例如, 串口协议中的帧头、数据长度、校验码等字段通常会明确规定使用多少字节, 此时定宽类型就显得尤为重要。

1.5.3.9. 常量与 `const` 关键字

在程序中, 有些值一旦确定就不应该被修改, 如数学常数、配置参数等。C++ 提供 `const` 关键字来声明常量:

```
1 const double PI = 3.14159265358979;  
2 const int MAX_BULLET_SPEED = 30; // 单位 m/s
```

使用 `const` 声明的变量必须在声明时初始化, 之后任何试图修改它的操作都会导致编译错误。这种编译期检查能够有效防止意外修改重要数据。

C++11 引入了 `constexpr` 关键字, 用于声明编译期常量。与 `const` 不同, `constexpr` 要求值必须在编译时就能确定:

```
1 constexpr double PI = 3.14159265358979;  
2 constexpr int ARRAY_SIZE = 100;  
3  
4 int arr[ARRAY_SIZE]; // 合法, ARRAY_SIZE 在编译期已知
```

对于简单的常量值, `const` 和 `constexpr` 的效果相似。但 `constexpr` 的语义更加明确——它强调这个值在编译时就已经确定, 可以用于模板参数、数组大小等需要编译期常量的场合。

相比于使用宏定义常量 (`#define PI 3.14159`), `const` 和 `constexpr` 具有类型安全性, 也遵循作用域规则, 是现代 C++ 推荐的做法。

1.5.4. 运算符与表达式

掌握了变量和数据类型之后, 下一步是学习如何操作这些数据。运算符是执行特定操作的符号, 表达式则是由运算符和操作数组而成的计算式。C++ 提供了丰富的运算符, 从基本的算术运算到复杂的位操作, 它们是构建程序逻辑的基础工具。

1.5.4.1. 算术运算符

算术运算符用于执行数学计算, 是最直观的一类运算符。C++ 支持加 (+)、减 (-)、乘 (*)、除 (/) 和取模 (%) 五种基本算术运算:

```
1 int a = 17, b = 5;  
2  
3 int sum = a + b; // 22  
4 int diff = a - b; // 12  
5 int prod = a * b; // 85  
6 int quot = a / b; // 3 (整数除法, 截断小数)  
7 int rem = a % b; // 2 (取模, 即余数)
```

整数除法是初学者常遇到的陷阱。当两个整数相除时, 结果仍然是整数, 小数部分被直接截断而非四舍五入。如果需要保留小数, 至少要将其中一个操作数转换为浮点类型:

```
1 int dividend = 7;  
2 int divisor = 2;  
3  
4 int result1 = dividend / divisor; // 结果为 3
```

```

5 double result2 = dividend / divisor;           // 结果仍为 3.0, 因为除法
   在赋值前已完成
6 double result3 = static_cast<double>(dividend) / divisor; // 结果为 3.5
7 double result4 = dividend / 2.0;               // 结果为 3.5, 2.0
   是 double

```

取模运算符 `%` 只能用于整数类型，返回除法的余数。它在许多场景下很有用，例如判断奇偶性、实现循环计数器、或者将数值限制在某个范围内：

```

1 bool isEven = (number % 2 == 0);           // 判断偶数
2 int arrayIndex = counter % arraySize;      // 循环索引
3 int angleInRange = angle % 360;           // 将角度限制在 0-359

```

C++ 还提供了自增 (`++`) 和自减 (`--`) 运算符，用于将变量的值加 1 或减 1。这两个运算符有前置和后置两种形式，区别在于返回值的时机：

```

1 int x = 5;
2 int y = ++x; // 前置：先加 1，再取值。x = 6, y = 6
3 int z = x++; // 后置：先取值，再加 1。x = 7, z = 6

```

前置形式返回修改后的值，后置形式返回修改前的值。在不需要使用返回值的情况下（如单独的 `i++` 语句），两者效果相同，但前置形式理论上效率略高，因为它不需要保存旧值。在 RoboMaster 代码中，循环变量的自增通常使用 `++i` 的形式。

1.5.4.2. 关系运算符

关系运算符用于比较两个值的大小关系，返回布尔值 `true` 或 `false`。C++ 提供六种关系运算符：

```

1 int a = 10, b = 20;
2
3 bool eq = (a == b); // 等于, false
4 bool ne = (a != b); // 不等于, true
5 bool lt = (a < b);   // 小于, true
6 bool le = (a <= b);  // 小于等于, true
7 bool gt = (a > b);   // 大于, false
8 bool ge = (a >= b);  // 大于等于, false

```

一个极其常见的错误是将相等判断 `==` 误写成赋值 `=`。这种错误往往不会产生编译错误，却会导致完全不同的行为：

```

1 int status = 0;
2
3 if (status = 1) { // 错误！这是赋值，status 变为 1，条件恒为 true
4     // 总是执行
5 }
6
7 if (status == 1) { // 正确，判断 status 是否等于 1
8     // 当 status 为 1 时执行
9 }

```

为了避免这类错误，有些程序员习惯将常量写在比较运算符的左侧（如 `1 == status`），这样误写成赋值时编译器会报错。不过，现代编译器通常会对条件语句中的赋值操作发出警告，开启足够的警告级别（如 `-Wall`）可以捕获大部分此类问题。

前面提到，浮点数的直接相等比较是不可靠的。在比较浮点数时，应当检查它们的差值是否在可接受的范围内：

```
1 double targetAngle = 45.0;
2 double currentAngle = computeAngle(); // 可能有浮点误差
3 double tolerance = 0.001;
4
5 if (std::abs(currentAngle - targetAngle) < tolerance) {
6     // 认为角度已到达目标
7 }
```

1.5.4.3. 逻辑运算符

逻辑运算符用于组合多个布尔表达式，构建复杂的条件判断。C++ 提供三种逻辑运算符：逻辑与（&&）、逻辑或（||）和逻辑非（!）。

逻辑与要求两个条件都为真，结果才为真：

```
1 bool canShoot = isTargetLocked && hasBullets && !isOverheated;
```

逻辑或只要有一个条件为真，结果就为真：

```
1 bool needWarning = isBatteryLow || isTemperatureHigh || hasError;
```

逻辑非将布尔值取反：

```
1 bool isValid = !isInvalid;
```

逻辑运算符具有短路求值（short-circuit evaluation）的特性。对于 &&，如果左侧表达式为假，右侧表达式不会被求值，因为结果已经确定为假；对于 ||，如果左侧为真，右侧同样不会被求值。这一特性不仅能提高效率，还可以用于避免潜在的错误：

```
1 // 安全的指针检查：先判断指针非空，再访问其成员
2 if (ptr != nullptr && ptr->isValid()) {
3     // 如果 ptr 为空，ptr->isValid() 不会被调用，避免空指针解引用
4 }
5
6 // 提供默认值
7 int value = (userInput > 0) ? userInput : defaultValue;
```

在复杂的条件表达式中，适当使用括号可以提高可读性，即使在运算符优先级正确的情况下：

```
1 // 不够清晰
2 if (a > 0 && b > 0 || c > 0 && d > 0) { }
3
4 // 更清晰
5 if ((a > 0 && b > 0) || (c > 0 && d > 0)) { }
```

1.5.4.4. 赋值运算符

最基本的赋值运算符是 =，它将右侧的值赋给左侧的变量。C++ 还提供了一系列复合赋值运算符，将算术运算和赋值合并为一步：

```
1 int x = 10;
2
```

```

3 x += 5; // 等价于 x = x + 5, x 变为 15
4 x -= 3; // 等价于 x = x - 3, x 变为 12
5 x *= 2; // 等价于 x = x * 2, x 变为 24
6 x /= 4; // 等价于 x = x / 4, x 变为 6
7 x %= 4; // 等价于 x = x % 4, x 变为 2

```

复合赋值运算符不仅使代码更简洁，在某些情况下还能避免重复计算左侧表达式。例如，`array[computeIndex()] += 1` 只调用一次 `computeIndex()`，而 `array[computeIndex()] = array[computeIndex()] + 1` 会调用两次。

赋值表达式本身也有值，其值为赋值后左侧变量的值。这使得链式赋值成为可能：

```

1 int a, b, c;
2 a = b = c = 0; // 从右向左执行，三个变量都被赋值为 0

```

然而，滥用赋值表达式的值会降低代码可读性，应当谨慎使用。

1.5.4.5. 位运算符

位运算符直接操作整数的二进制位，在底层编程和性能敏感的场景中很有用。C++ 提供六种位运算符：

```

1 int a = 0b1100; // 二进制 1100，即十进制 12
2 int b = 0b1010; // 二进制 1010，即十进制 10
3
4 int andResult = a & b; // 按位与：0b1000，即 8
5 int orResult = a | b; // 按位或：0b1110，即 14
6 int xorResult = a ^ b; // 按位异或：0b0110，即 6
7 int notResult = ~a; // 按位取反：所有位翻转
8
9 int leftShift = a << 2; // 左移 2 位：0b110000，即 48
10 int rightShift = a >> 2; // 右移 2 位：0b0011，即 3

```

在 RoboMaster 开发中，位运算常用于以下场景。

状态标志的管理是位运算的典型应用。使用单个整数的不同位表示不同的状态，可以高效地存储和检查多个布尔标志：

```

1 const uint8_t FLAG_MOTOR_READY = 0b00000001; // 第 0 位
2 const uint8_t FLAG_SENSOR_OK = 0b00000010; // 第 1 位
3 const uint8_t FLAG_COMM_ACTIVE = 0b00000100; // 第 2 位
4 const uint8_t FLAG_AUTO_AIM = 0b00001000; // 第 3 位
5
6 uint8_t robotStatus = 0;
7
8 // 设置标志
9 robotStatus |= FLAG_MOTOR_READY; // 将第 0 位置 1
10 robotStatus |= FLAG_SENSOR_OK; // 将第 1 位置 1
11
12 // 清除标志
13 robotStatus &= ~FLAG_AUTO_AIM; // 将第 3 位置 0
14
15 // 检查标志
16 if (robotStatus & FLAG_MOTOR_READY) {
17     // 电机已就绪
18 }
19
20 // 切换标志
21 robotStatus ^= FLAG_COMM_ACTIVE; // 翻转第 2 位

```

位移运算在处理通信协议时也很常见。例如，将多个字节组合成一个整数，或者从整数中提取特定字节：

```
1 // 将两个字节组合成 16 位整数（大端序）
2 uint8_t highByte = 0x12;
3 uint8_t lowByte = 0x34;
4 uint16_t combined = (highByte << 8) | lowByte; // 0x1234
5
6 // 从 32 位整数中提取各字节
7 uint32_t value = 0x12345678;
8 uint8_t byte0 = value & 0xFF; // 0x78
9 uint8_t byte1 = (value >> 8) & 0xFF; // 0x56
10 uint8_t byte2 = (value >> 16) & 0xFF; // 0x34
11 uint8_t byte3 = (value >> 24) & 0xFF; // 0x12
```

左移一位相当于乘以 2，右移一位相当于除以 2（对于无符号数或非负有符号数）。在某些性能关键的代码中，位移可能比乘除法更快，但现代编译器通常会自动进行这类优化，因此不必刻意使用位移替代乘除。

1.5.4.6. 条件运算符

条件运算符（?:）是 C++ 中唯一的三元运算符，它根据条件选择两个值中的一个：

```
1 int max = (a > b) ? a : b; // 如果 a > b，取 a；否则取 b
```

条件运算符可以替代简单的 if-else 语句，使代码更紧凑：

```
1 // 使用 if-else
2 std::string status;
3 if (isOnline) {
4     status = "在线";
5 } else {
6     status = "离线";
7 }
8
9 // 使用条件运算符
10 std::string status = isOnline ? "在线" : "离线";
```

条件运算符也可以嵌套，但过度嵌套会严重损害可读性：

```
1 // 勉强可以接受
2 int sign = (x > 0) ? 1 : (x < 0) ? -1 : 0;
3
4 // 嵌套过深，难以理解
5 int result = a ? b ? c : d : e ? f : g; // 不要这样写
```

当逻辑变得复杂时，应当使用普通的 if-else 语句。条件运算符最适合用于简单的二选一场景。

1.5.4.7. 运算符优先级

当一个表达式包含多个运算符时，运算符优先级决定了计算的顺序。C++ 定义了详细的优先级规则，以下是常用运算符按优先级从高到低的大致排列：

1. 后缀运算符：() [] -> . ++ (后置) -- (后置)
2. 一元运算符：++ (前置) -- (前置) ! ~ + (正号) - (负号) * (解引用) & (取地址)
3. 乘除模：* / %
4. 加减：+ -

5. 位移: `<<` `>>`
6. 关系: `<` `<=` `>` `>=`
7. 相等: `==` `!=`
8. 按位与: `&`
9. 按位异或: `^`
10. 按位或: `|`
11. 逻辑与: `&&`
12. 逻辑或: `||`
13. 条件: `?:`
14. 赋值: `=` `+=` `-=` 等
15. 逗号: `,`

记住所有优先级规则是困难的, 也没有必要。实践中的建议是: 对于不确定优先级的表达式, 使用括号明确计算顺序。括号不仅保证了正确性, 也提高了代码的可读性:

```
1 // 不确定优先级, 容易出错
2 int result = a + b * c >> d & e;
3
4 // 使用括号, 意图明确
5 int result = ((a + (b * c)) >> d) & e;
```

特别需要注意的是, 位运算符的优先级低于关系运算符, 这常常导致意外:

```
1 // 错误: 实际执行的是 flags & (FLAG_A == FLAG_A), 结果恒为 flags & 1
2 if (flags & FLAG_A == FLAG_A) { }
3
4 // 正确
5 if ((flags & FLAG_A) == FLAG_A) { }
```

1.5.4.8. 表达式与语句

理解表达式和语句的区别有助于写出更清晰的代码。表达式是可以求值的代码片段, 它产生一个结果; 语句是执行某种操作的完整指令, 以分号结束。

```
1 a + b           // 表达式, 值为 a 和 b 的和
2 x = a + b       // 也是表达式, 值为赋值后 x 的值
3 x = a + b;      // 语句, 执行赋值操作
4
5 int y = 10;     // 声明语句
6 if (x > 0) {}   // 控制语句
```

任何表达式加上分号都成为表达式语句。有些表达式语句是有意义的(如赋值、函数调用), 有些则毫无作用:

```
1 x = 5;          // 有意义, 修改了 x 的值
2 calculate();    // 有意义, 调用了函数
3 a + b;          // 无意义, 计算结果被丢弃 (编译器可能警告)
```

在 C++ 中, 某些看似语句的结构实际上是表达式。例如, 赋值操作返回被赋的值, 这使得 `a = b = c = 0` 这样的链式赋值成为可能。再如, 逗号运算符连接多个表达式, 从左到右依次求值, 整个表达式的值为最右侧表达式的值:

```
1 int x = (a = 1, b = 2, a + b); // x = 3
```


逗号运算符在 for 循环中偶尔有用（如同时更新多个变量），但在其他地方使用会降低代码可读性，应当避免。

理解了运算符和表达式的工作方式，就具备了编写计算逻辑的基础。下一节将介绍控制语句，它们决定了程序的执行流程——哪些代码会被执行，哪些会被跳过，哪些会重复执行。

1.5.5. 控制语句

程序并非总是从头到尾顺序执行。根据不同的条件选择不同的执行路径，或者重复执行某段代码直到满足特定条件，这些都是程序设计中的基本需求。控制语句正是实现这些功能的工具，它们决定了程序的执行流程。C++ 提供了三类控制语句：条件语句用于分支选择，循环语句用于重复执行，跳转语句用于改变执行顺序。

1.5.5.1. if 语句

`if` 语句是最基本的条件语句，它根据条件表达式的真假决定是否执行某段代码：

```
1 if (temperature > 60.0) {
2     std::cout << "警告: 电机温度过高! " << std::endl;
3     disableMotor();
4 }
```

当条件为真时，花括号内的代码块被执行；当条件为假时，代码块被跳过。花括号定义了 `if` 语句的作用范围，即使只有一条语句，也建议使用花括号，这样可以避免后续添加代码时遗漏花括号导致的逻辑错误：

```
1 // 危险的写法
2 if (error)
3     logError();
4     shutdown(); // 这行总是执行，不属于 if 语句!
5
6 // 安全的写法
7 if (error) {
8     logError();
9     shutdown(); // 现在正确地属于 if 语句
10 }
```

当需要在条件为假时执行另一段代码，可以使用 `else` 子句：

```
1 if (batteryLevel > 20) {
2     status = "正常";
3 } else {
4     status = "电量不足";
5     enablePowerSaving();
6 }
```

多个条件可以用 `else if` 串联起来，形成多路分支：

```
1 if (distance < 1000) {
2     range = "近距离";
3     adjustStrategy(CLOSE_RANGE);
4 } else if (distance < 3000) {
5     range = "中距离";
6     adjustStrategy(MID_RANGE);
7 } else if (distance < 5000) {
8     range = "远距离";
9     adjustStrategy(LONG_RANGE);
10 } else {
```



```

11     range = "超出范围";
12     adjustStrategy(OUT_OF_RANGE);
13 }

```

`else if` 链按顺序检查每个条件，一旦某个条件为真，执行对应的代码块后就跳出整个结构，后续条件不再检查。因此，条件的排列顺序很重要——应当将最可能满足或最需要优先处理的条件放在前面。

`if` 语句可以嵌套，但过深的嵌套会严重损害代码可读性。当嵌套超过两三层时，应当考虑重构代码，例如提取函数或使用提前返回（early return）：

```

1 // 嵌套过深，难以阅读
2 if (conditionA) {
3     if (conditionB) {
4         if (conditionC) {
5             doSomething();
6         }
7     }
8 }
9
10 // 使用提前返回，更清晰
11 if (!conditionA) return;
12 if (!conditionB) return;
13 if (!conditionC) return;
14 doSomething();

```

C++17 引入了带初始化的 `if` 语句，允许在条件判断前声明变量，该变量的作用域限于 `if` 语句内部：

```

1 if (auto result = computeSomething(); result > threshold) {
2     process(result);
3 } else {
4     handleFailure(result);
5 }
6 // result 在这里不可见

```

这种写法在需要检查函数返回值时特别有用，它将变量声明和条件检查合并，同时限制了变量的作用域。

1.5.5.2. switch 语句

当需要根据一个变量的不同取值执行不同的代码时，`switch` 语句比一连串的 `if-else if` 更加清晰：

```

1 enum class RobotState { Idle, Patrol, Attack, Retreat };
2
3 RobotState state = getCurrentState();
4
5 switch (state) {
6     case RobotState::Idle:
7         standby();
8         break;
9     case RobotState::Patrol:
10        moveAlongPath();
11        scanForTargets();
12        break;
13    case RobotState::Attack:
14        aimAtTarget();

```

```

15     fire();
16     break;
17     case RobotState::Retreat:
18         moveToSafeZone();
19         break;
20     default:
21         handleUnknownState();
22         break;
23 }

```

`switch` 语句的工作方式是：计算括号内表达式的值，然后跳转到匹配的 `case` 标签处开始执行。每个 `case` 后的 `break` 语句用于跳出 `switch` 结构；如果省略 `break`，程序会继续执行下一个 `case` 的代码，这称为“贯穿”（fall-through）。

贯穿行为有时是有意为之的，例如多个 `case` 共享同一段代码：

```

1 char grade = getGrade();
2
3 switch (grade) {
4     case 'A':
5     case 'B':
6     case 'C':
7         std::cout << "通过" << std::endl;
8         break;
9     case 'D':
10    case 'F':
11        std::cout << "未通过" << std::endl;
12        break;
13    default:
14        std::cout << "无效成绩" << std::endl;
15        break;
16 }

```

然而，无意的贯穿是常见的 bug 来源。C++17 引入了 `[[fallthrough]]` 属性，用于明确表示贯穿是有意的，同时让编译器在其他地方对遗漏的 `break` 发出警告：

```

1 switch (command) {
2     case CMD_INIT:
3         initialize();
4         [[fallthrough]]; // 明确表示继续执行下一个 case
5     case CMD_START:
6         start();
7         break;
8     // ...
9 }

```

`switch` 语句的表达式必须是整数类型或枚举类型，不能是浮点数或字符串。`case` 标签必须是编译期常量，不能是变量。`default` 标签处理所有未匹配的情况，虽然不是必需的，但添加 `default` 是良好的习惯，它可以捕获意外的输入值。

在 `case` 内部声明变量需要特别注意。由于 `switch` 的跳转特性，直接在 `case` 中声明变量可能导致跳过初始化，编译器会报错。解决方法是用花括号创建局部作用域：

```

1 switch (type) {
2     case TYPE_A: {
3         int localVar = 10; // 在局部作用域内声明
4         process(localVar);
5         break;
6     }

```

```

7     case TYPE_B: {
8         std::string msg = "hello";
9         display(msg);
10        break;
11    }
12 }

```

1.5.5.3. while 循环

`while` 循环在条件为真时重复执行代码块。它首先检查条件，如果为真则执行循环体，然后再次检查条件，如此反复直到条件为假：

```

1  int attempts = 0;
2  const int maxAttempts = 5;
3
4  while (attempts < maxAttempts && !connectionEstablished()) {
5      std::cout << "尝试连接... (" << attempts + 1 << "/" << maxAttempts << ")"
6      << std::endl;
7      tryConnect();
8      attempts++;
9      sleep(1000);
10 }
11 if (connectionEstablished()) {
12     std::cout << "连接成功" << std::endl;
13 } else {
14     std::cout << "连接失败" << std::endl;
15 }

```

`while` 循环适合用于事先不知道循环次数的场景，例如等待某个条件满足、处理不定长度的输入、或者实现重试机制。

编写 `while` 循环时必须确保循环条件最终会变为假，否则会产生无限循环。无限循环有时是有意为之的（如主事件循环），但意外的无限循环会导致程序挂起：

```

1  // 意外的无限循环：忘记更新 i
2  int i = 0;
3  while (i < 10) {
4      process(i);
5      // 忘记 i++, 循环永不结束
6  }
7
8  // 有意的无限循环：主控制循环
9  while (true) {
10     readSensors();
11     updateState();
12     sendCommands();
13
14     if (shutdownRequested()) {
15         break; // 使用 break 退出
16     }
17 }

```

1.5.5.4. do-while 循环

`do-while` 循环与 `while` 循环类似，但它先执行循环体，再检查条件。这保证了循环体至少执行一次：

```

1 int input;
2 do {
3     std::cout << "请输入一个正整数: ";
4     std::cin >> input;
5
6     if (input <= 0) {
7         std::cout << "输入无效, 请重试。" << std::endl;
8     }
9 } while (input <= 0);
10
11 std::cout << "你输入的是: " << input << std::endl;

```

`do-while` 的典型应用场景是需要先执行操作再判断是否继续的情况, 如用户输入验证、菜单交互等。由于循环体至少执行一次, 使用前应当确认这符合程序逻辑。

注意 `do-while` 语句以分号结尾, 这与其他循环不同, 容易遗漏。

1.5.5.5. for 循环

`for` 循环是最常用的循环结构, 它将初始化、条件检查和迭代更新集中在一行, 特别适合已知循环次数的场景:

```

1 // 遍历数组
2 int scores[] = {85, 92, 78, 96, 88};
3 int sum = 0;
4
5 for (int i = 0; i < 5; i++) {
6     sum += scores[i];
7 }
8
9 double average = static_cast<double>(sum) / 5;

```

`for` 语句的三个部分——初始化、条件、迭代——都是可选的。省略条件表达式等同于条件恒为真, 这是创建无限循环的常见方式:

```

1 for (;;) {
2     // 无限循环
3     if (shouldExit()) break;
4 }

```

`for` 循环的初始化部分可以声明多个同类型的变量, 迭代部分也可以包含多个表达式, 用逗号分隔:

```

1 // 双指针技术: 从两端向中间遍历
2 for (int left = 0, right = size - 1; left < right; left++, right--) {
3     if (array[left] > array[right]) {
4         std::swap(array[left], array[right]);
5     }
6 }

```

C++11 引入了范围 `for` 循环 (range-based for loop), 它提供了一种更简洁的方式遍历容器或数组中的元素:

```

1 std::vector<int> numbers = {1, 2, 3, 4, 5};
2
3 // 传统 for 循环
4 for (size_t i = 0; i < numbers.size(); i++) {
5     std::cout << numbers[i] << " ";
6 }

```

```

6 }
7
8 // 范围 for 循环
9 for (int num : numbers) {
10     std::cout << num << " ";
11 }

```

范围 `for` 循环自动处理迭代细节，代码更加简洁，也不容易出现索引越界等错误。如果需要修改元素，应当使用引用；如果元素较大且不需要修改，应当使用常量引用以避免复制开销：

```

1 std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
2
3 // 修改元素：使用引用
4 for (std::string& name : names) {
5     name = "Mr./Ms. " + name;
6 }
7
8 // 只读访问：使用常量引用，避免复制
9 for (const std::string& name : names) {
10     std::cout << name << std::endl;
11 }
12
13 // 使用 auto 简化类型声明
14 for (const auto& name : names) {
15     std::cout << name << std::endl;
16 }

```

在 RoboMaster 开发中，范围 `for` 循环常用于遍历检测到的目标列表、处理传感器数据数组等场景。

1.5.5.6. break 与 continue

`break` 语句用于立即退出最内层的循环或 `switch` 语句。当在循环中遇到某个条件需要提前结束时，`break` 非常有用：

```

1 // 在数组中查找目标值
2 int target = 42;
3 int foundIndex = -1;
4
5 for (int i = 0; i < arraySize; i++) {
6     if (array[i] == target) {
7         foundIndex = i;
8         break; // 找到后立即退出，无需继续搜索
9     }
10 }

```

`continue` 语句用于跳过当前迭代的剩余部分，直接进入下一次迭代。它适合用于在满足某些条件时跳过处理：

```

1 // 处理有效数据，跳过无效数据
2 for (const auto& reading : sensorReadings) {
3     if (!reading.isValid()) {
4         continue; // 跳过无效读数
5     }
6
7     processReading(reading);
8     updateStatistics(reading);
9 }

```

`break` 和 `continue` 只影响最内层的循环。在嵌套循环中，如果需要从内层循环跳出到外层，可以使用标志变量或将循环封装成函数并使用 `return`：

```
1 // 使用标志变量
2 bool found = false;
3 for (int i = 0; i < rows && !found; i++) {
4     for (int j = 0; j < cols; j++) {
5         if (matrix[i][j] == target) {
6             found = true;
7             break; // 只跳出内层循环
8         }
9     }
10 }
11
12 // 更优雅的方式：封装成函数
13 std::pair<int, int> findInMatrix(int target) {
14     for (int i = 0; i < rows; i++) {
15         for (int j = 0; j < cols; j++) {
16             if (matrix[i][j] == target) {
17                 return {i, j}; // 直接返回，退出所有循环
18             }
19         }
20     }
21     return {-1, -1}; // 未找到
22 }
```

1.5.5.7. 循环的选择

三种循环结构在功能上是等价的，任何一种都可以改写成另一种。选择哪种循环主要取决于代码的清晰度：

`for` 循环适合循环次数已知或有明确迭代模式的场景。它将循环的三个要素集中在一处，便于理解循环的行为。遍历数组、计数循环、有规律的迭代都适合用 `for`。

`while` 循环适合循环次数未知、依赖于运行时条件的场景。等待事件、处理流数据、重试机制等都适合用 `while`。

`do-while` 循环适合需要先执行再判断的场景，如用户输入验证。由于循环体必定执行一次，使用场景相对较少。

```
1 // 计数循环: for 最清晰
2 for (int i = 0; i < 10; i++) { }
3
4 // 等待条件: while 最清晰
5 while (!isReady()) {
6     wait();
7 }
8
9 // 输入验证: do-while 最清晰
10 do {
11     input = getInput();
12 } while (!isValid(input));
```

1.5.5.8. 控制语句的嵌套与组合

实际程序中，控制语句往往需要组合使用。条件语句可以嵌套在循环中，循环也可以嵌套在条件语句中。合理的嵌套能够表达复杂的逻辑，但过度嵌套会使代码难以理解和维护。

```
1 // RoboMaster 自动瞄准逻辑示例
```

```

2 void autoAimLoop() {
3     while (isAutoAimEnabled()) {
4         auto targets = detectTargets();
5
6         if (targets.empty()) {
7             continue; // 无目标, 进入下一帧
8         }
9
10        // 选择最优目标
11        Target* bestTarget = nullptr;
12        double bestScore = -1;
13
14        for (const auto& target : targets) {
15            if (!target.isValid()) {
16                continue;
17            }
18
19            double score = evaluateTarget(target);
20            if (score > bestScore) {
21                bestScore = score;
22                bestTarget = &target;
23            }
24        }
25
26        if (bestTarget != nullptr) {
27            aimAt(*bestTarget);
28
29            if (isAimStable() && canFire()) {
30                fire();
31            }
32        }
33
34        waitForNextFrame();
35    }
36 }

```

保持代码清晰的几个原则：控制嵌套深度，一般不超过三层；将复杂的条件表达式提取为命名良好的布尔变量或函数；使用提前返回或 `continue` 减少嵌套；将独立的逻辑块提取为函数。

控制语句是程序逻辑的骨架。掌握了变量、运算符和控制语句，就具备了编写基本程序的能力。然而，随着程序规模的增长，将所有代码写在 `main` 函数中会变得难以管理。下一节将介绍函数，它是组织和复用代码的基本单元。

1.5.6. 函数基础

随着程序规模的增长，将所有代码堆砌在 `main` 函数中会迅速变得不可维护。相似的代码片段可能在多处重复出现，修改一处逻辑需要同时修改多个位置，程序的整体结构也变得难以把握。函数是解决这些问题的基本工具——它将一段完成特定任务的代码封装起来，赋予一个名称，使其可以在需要时被调用。良好的函数设计能够提高代码的可读性、可维护性和复用性。

1.5.6.1. 函数的定义与调用

一个完整的函数定义包含返回类型、函数名、参数列表和函数体四个部分：

```

1 // 返回类型  函数名  参数列表
2 double average(int a, int b) {
3     // 函数体
4     return static_cast<double>(a + b) / 2;
5 }

```

返回类型指定函数返回值的类型。如果函数不返回任何值，使用 `void` 作为返回类型。函数名应当清晰地描述函数的功能，通常使用动词或动词短语。参数列表定义了函数接受的输入，多个参数用逗号分隔；如果函数不需要参数，参数列表为空。函数体是实际执行的代码，用花括号包围。

定义好函数后，通过函数名加括号的方式调用它：

```
1 int x = 10, y = 20;
2 double avg = average(x, y); // 调用函数，传入参数 x 和 y
3 std::cout << "平均值: " << avg << std::endl;
```

函数调用时，实际传入的值称为实参（argument），函数定义中的变量称为形参（parameter）。调用发生时，实参的值被复制给形参，函数在自己的作用域内使用这些副本进行计算。

`return` 语句用于从函数返回一个值并结束函数执行。对于返回 `void` 的函数，可以使用不带值的 `return`；提前退出，也可以让函数自然执行到末尾结束：

```
1 void printPositive(int value) {
2     if (value <= 0) {
3         return; // 提前退出，不打印非正数
4     }
5     std::cout << value << std::endl;
6 }
```

1.5.6.2. 函数声明与定义分离

在 C++ 中，函数必须在调用之前被声明。声明告诉编译器函数的存在及其接口（返回类型、名称、参数类型），而定义提供函数的具体实现。声明可以出现多次，但定义只能有一次。

```
1 // 函数声明（也称为函数原型）
2 double calculateDistance(double x1, double y1, double x2, double y2);
3
4 int main() {
5     // 可以调用，因为前面已经声明
6     double dist = calculateDistance(0, 0, 3, 4);
7     std::cout << "距离: " << dist << std::endl;
8     return 0;
9 }
10
11 // 函数定义
12 double calculateDistance(double x1, double y1, double x2, double y2) {
13     double dx = x2 - x1;
14     double dy = y2 - y1;
15     return std::sqrt(dx * dx + dy * dy);
16 }
```

将声明放在文件开头或头文件中，定义放在源文件中，是 C++ 项目组织代码的常见方式。这种分离使得多个源文件可以共享同一个函数，也使得编译器能够独立编译各个源文件。

声明中的参数名是可选的，只需要类型信息即可。但为了可读性，通常会保留有意义的参数名：

```
1 // 合法但可读性差
2 double calculateDistance(double, double, double, double);
3
4 // 更好的写法
```



```
5 double calculateDistance(double x1, double y1, double x2, double y2);
```

1.5.6.3. 参数传递

C++ 中参数传递有三种主要方式：值传递、引用传递和指针传递。理解它们的区别对于写出正确且高效的代码至关重要。

值传递是默认方式。调用函数时，实参的值被复制给形参，函数内部对形参的修改不会影响实参：

```
1 void increment(int n) {
2     n++; // 只修改了副本
3     std::cout << "函数内: " << n << std::endl;
4 }
5
6 int main() {
7     int value = 10;
8     increment(value);
9     std::cout << "函数外: " << value << std::endl; // 仍然是 10
10    return 0;
11 }
```

值传递的优点是安全——函数不会意外修改调用者的数据。缺点是对于大型对象，复制的开销可能很大。

引用传递通过在参数类型后加 `&` 实现。此时形参是实参的别名，函数内部对形参的修改会直接反映到实参上：

```
1 void increment(int& n) {
2     n++; // 直接修改原变量
3 }
4
5 int main() {
6     int value = 10;
7     increment(value);
8     std::cout << value << std::endl; // 输出 11
9     return 0;
10 }
```

引用传递有两个主要用途。一是允许函数修改调用者的变量，如上例所示。二是避免复制大型对象的开销——即使不需要修改，也可以使用常量引用传递：

```
1 // 传值：复制整个 vector，开销大
2 void processData(std::vector<double> data);
3
4 // 常量引用：不复制，不允许修改，安全高效
5 void processData(const std::vector<double>& data);
```

使用常量引用传递大型对象是 C++ 中的常见惯用法。对于基本类型（如 `int`、`double`），由于复制成本很低，通常直接值传递即可。

指针传递将在后续章节详细介绍。简单来说，它通过传递变量的地址来实现类似引用的效果，但语法更显式，且允许传递空指针表示“无值”。

1.5.6.4. 返回值

函数可以通过 `return` 语句返回一个值给调用者。返回值的类型必须与函数声明的返回类型兼容：

```

1 int findMax(int a, int b, int c) {
2     int max = a;
3     if (b > max) max = b;
4     if (c > max) max = c;
5     return max;
6 }

```

返回值可以是基本类型、对象、引用或指针。返回局部变量的值是安全的，因为返回时会复制该值；但返回局部变量的引用或指针是危险的，因为函数结束后局部变量被销毁，引用或指针将指向无效内存：

```

1 // 安全：返回值的副本
2 int getValue() {
3     int local = 42;
4     return local; // 返回 local 的副本
5 }
6
7 // 危险：返回局部变量的引用
8 int& getReference() {
9     int local = 42;
10    return local; // 错误！local 在函数返回后销毁
11 }

```

C++11 允许函数返回初始化列表，结合 `auto` 可以方便地返回多个值：

```

1 std::pair<double, double> getPosition() {
2     return {3.14, 2.71};
3 }
4
5 auto [x, y] = getPosition(); // C++17 结构化绑定

```

对于需要返回多个值的场景，除了返回 `std::pair` 或 `std::tuple`，也可以使用输出参数（通过引用或指针传递）：

```

1 // 方式一：返回结构体或 pair
2 struct Result {
3     bool success;
4     double value;
5 };
6
7 Result compute(double input);
8
9 // 方式二：输出参数
10 bool compute(double input, double& output);

```

两种方式各有优劣。返回结构体语义更清晰，输出参数在某些场景下（如需要区分返回值和错误码）更方便。现代 C++ 倾向于使用返回值而非输出参数。

1.5.6.5. 默认参数

函数参数可以指定默认值。调用时如果不提供该参数，就使用默认值：

```

1 void connect(const std::string& host, int port = 8080, int timeout = 5000) {
2     std::cout << "连接到 " << host << ":" << port
3               << ", 超时 " << timeout << "ms" << std::endl;
4 }
5
6 int main() {

```

```

7     connect("192.168.1.1");           // 使用默认端口和超时
8     connect("192.168.1.1", 9000);      // 自定义端口，默认超时
9     connect("192.168.1.1", 9000, 3000); // 全部自定义
10    return 0;
11 }

```

默认参数必须从右向左连续指定，不能跳过中间参数：

```

1 void func(int a, int b = 2, int c = 3); // 合法
2 void func(int a = 1, int b, int c = 3); // 非法，b 没有默认值但在有默认值的参数之后

```

如果函数声明和定义分离，默认参数应当只在声明中指定，不要在定义中重复：

```

1 // 头文件中的声明
2 void log(const std::string& message, int level = 1);
3
4 // 源文件中的定义
5 void log(const std::string& message, int level) { // 不重复默认值
6     // ...
7 }

```

默认参数在编译时确定，因此必须是编译期常量或能在编译时求值的表达式。

1.5.6.6. 函数重载

C++ 允许定义多个同名但参数列表不同的函数，这称为函数重载。编译器根据调用时提供的参数类型和数量选择合适的版本：

```

1 // 计算两个整数的平均值
2 double average(int a, int b) {
3     return static_cast<double>(a + b) / 2;
4 }
5
6 // 计算三个整数的平均值
7 double average(int a, int b, int c) {
8     return static_cast<double>(a + b + c) / 3;
9 }
10
11 // 计算浮点数数组的平均值
12 double average(const std::vector<double>& values) {
13     double sum = 0;
14     for (double v : values) sum += v;
15     return sum / values.size();
16 }
17
18 int main() {
19     std::cout << average(10, 20) << std::endl; // 调用第一个版本
20     std::cout << average(10, 20, 30) << std::endl; // 调用第二个版本
21     std::cout << average({1.5, 2.5, 3.5}) << std::endl; // 调用第三个版本
22     return 0;
23 }

```

重载函数必须在参数的数量或类型上有所不同。仅返回类型不同不构成有效的重载，因为编译器无法仅根据返回类型确定调用哪个版本：

```

1 int process(int x);
2 double process(int x); // 错误！仅返回类型不同

```

重载解析是编译器选择最佳匹配函数的过程。当存在多个可能的匹配时，编译器会选择“最佳匹配”——需要最少类型转换的版本。如果无法确定唯一的最佳匹配，编译器会报告歧义错误：

```
1 void print(int x);
2 void print(double x);
3
4 print(3.14f); // float 到 int 和 float 到 double 都需要转换，可能产生歧义
```

函数重载使得同一操作可以作用于不同类型的数据，提高了接口的一致性。在 RoboMaster 开发中，重载常见于各种初始化函数、数据转换函数等。

1.5.6.7. 内联函数

函数调用有一定的开销：保存当前状态、跳转到函数代码、执行、返回。对于非常短小且频繁调用的函数，这些开销可能超过函数体本身的执行时间。内联函数通过在调用处直接展开函数代码来消除这种开销：

```
1 inline int max(int a, int b) {
2     return (a > b) ? a : b;
3 }
```

`inline` 关键字是对编译器的建议而非强制命令。现代编译器会自行判断是否内联——即使没有 `inline` 关键字，编译器也可能内联短小的函数；即使有 `inline`，编译器也可能拒绝内联过大或递归的函数。

内联函数的定义通常放在头文件中，因为编译器需要在调用处看到完整的函数体才能展开。这与普通函数不同——普通函数的定义通常放在源文件中。

```
1 // math_utils.h
2 #ifndef MATH_UTILS_H
3 #define MATH_UTILS_H
4
5 inline int square(int x) {
6     return x * x;
7 }
8
9 #endif
```

在实践中，不必过度关注内联优化。编译器的优化能力很强，除非性能分析表明某个函数调用是瓶颈，否则让编译器自行决定即可。

1.5.6.8. 递归函数

函数可以调用自身，这称为递归。递归是一种强大的问题解决技术，适合处理具有自相似结构的问题：

```
1 // 计算阶乘: n! = n × (n-1)!
2 int factorial(int n) {
3     if (n <= 1) {
4         return 1; // 基本情况: 0! = 1! = 1
5     }
6     return n * factorial(n - 1); // 递归情况
7 }
```

每个递归函数都必须有基本情况 (base case) 和递归情况 (recursive case)。基本情况定义了不再递归的条件, 递归情况将问题分解为更小的子问题。缺少基本情况或递归未能向基本情况收敛会导致无限递归, 最终耗尽栈空间导致程序崩溃。

递归的经典例子包括树的遍历、快速排序、归并排序等。以下是一个在 RoboMaster 中可能用到的例子——路径搜索的简化版本:

```
1 bool findPath(int x, int y, int targetX, int targetY,
2               std::vector<std::pair<int,int>>& path) {
3     // 基本情况: 到达目标
4     if (x == targetX && y == targetY) {
5         path.push_back({x, y});
6         return true;
7     }
8
9     // 基本情况: 越界或遇到障碍
10    if (!isValid(x, y) || isObstacle(x, y) || isVisited(x, y)) {
11        return false;
12    }
13
14    markVisited(x, y);
15    path.push_back({x, y});
16
17    // 递归情况: 尝试四个方向
18    if (findPath(x + 1, y, targetX, targetY, path) ||
19        findPath(x - 1, y, targetX, targetY, path) ||
20        findPath(x, y + 1, targetX, targetY, path) ||
21        findPath(x, y - 1, targetX, targetY, path)) {
22        return true;
23    }
24
25    // 回溯
26    path.pop_back();
27    return false;
28 }
```

递归代码通常简洁优雅, 但也有缺点: 每次递归调用都会占用栈空间, 深度过大会导致栈溢出; 此外, 递归的函数调用开销可能影响性能。对于性能敏感的场景, 可以考虑将递归改写为迭代。

1.5.6.9. constexpr 函数

C++11 引入了 `constexpr` 函数, 它可以在编译期求值。如果传入的参数是编译期常量, 函数的结果也是编译期常量, 可用于数组大小、模板参数等需要常量的场合:

```
1 constexpr int square(int x) {
2     return x * x;
3 }
4
5 int main() {
6     constexpr int size = square(10); // 编译期计算, size = 100
7     int array[size]; // 合法, size 是编译期常量
8
9     int runtime = getUserInput();
10    int result = square(runtime); // 运行时计算, 仍然有效
11    return 0;
12 }
```

`constexpr` 函数有一些限制：在 C++11 中，函数体只能包含一条 `return` 语句；C++14 放宽了这一限制，允许使用局部变量、循环和条件语句。

```
1 // C++14 及以后
2 constexpr int factorial(int n) {
3     int result = 1;
4     for (int i = 2; i <= n; i++) {
5         result *= i;
6     }
7     return result;
8 }
9
10 constexpr int fact5 = factorial(5); // 编译期计算, fact5 = 120
```

`constexpr` 函数在 RoboMaster 开发中可用于计算固定的配置参数、查找表等，将计算从运行时移到编译期可以提高程序启动速度和运行效率。

1.5.6.10. 函数设计原则

良好的函数设计能够显著提高代码质量。以下是一些实践中总结的原则：

函数应当短小精悍，专注于完成单一任务。如果一个函数做了太多事情，应当拆分成多个函数。一个经验法则是：如果函数超过一屏（约 40-50 行），就应当考虑拆分。

函数名应当清晰描述其功能。好的命名使代码自文档化，减少注释的需要。动词开头的名称（如 `calculateDistance`、`isValid`、`sendCommand`）通常比名词更能表达函数的行为。

函数的参数不宜过多。超过三四个参数的函数难以记忆和使用，也往往表明函数承担了过多职责。可以考虑将相关参数组织成结构体，或者拆分函数。

优先使用返回值而非输出参数。返回值的语义更清晰，也更符合函数式编程的风格。现代 C++ 的移动语义使得返回大型对象的开销很小。

避免副作用。纯函数（相同输入总是产生相同输出，且不修改外部状态）更容易理解、测试和调试。当然，某些函数的目的就是产生副作用（如 I/O 函数），但应当明确标识这类函数的行为。

```
1 // 不好的设计：函数过长，职责不清
2 void processRobot() {
3     // 200 行代码，混合了传感器读取、状态更新、通信、日志...
4 }
5
6 // 好的设计：职责分离，函数短小
7 void updateRobotState() {
8     auto sensorData = readSensors();
9     auto newState = computeState(sensorData);
10    applyState(newState);
11    logStateChange(newState);
12 }
```

函数是组织代码的基本单元，也是抽象的基本工具。通过将复杂问题分解为一系列函数调用，程序的逻辑结构变得清晰，各部分也可以独立开发和测试。掌握函数的使用是迈向更大规模程序开发的关键一步。下一节将介绍数组和字符串，它们是处理批量数据的基础工具。

1.5.7. 数组与字符串

程序经常需要处理一组相关的数据：传感器的连续读数、检测到的多个目标、机器人的历史轨迹。逐个声明变量来存储这些数据既繁琐又不切实际——如果有一千个数据点，难道要声明一千个变量吗？数组正是为解决这一问题而设计的数据结构，它将多个相同类型的元素存储在连续

的内存空间中，通过索引访问各个元素。字符串则是字符数组的特殊形式，用于处理文本数据。本节将介绍 C 风格数组、现代 C++ 的 `std::array` 以及字符串的基本操作。

1.5.7.1. C 风格数组

C 风格数组是最基本的数组形式，它直接在栈上分配一块连续的内存。声明数组时需要指定元素类型和数组大小：

```
1 int scores[5];           // 声明一个包含 5 个整数的数组
2 double sensorReadings[100]; // 声明一个包含 100 个双精度浮点数的数组
```

数组大小必须是编译期常量，不能使用运行时才能确定的值（变长数组是 C99 的特性，C++ 标准不支持）。数组元素通过下标访问，下标从 0 开始：

```
1 int values[5] = {10, 20, 30, 40, 50};
2
3 int first = values[0]; // 10, 第一个元素
4 int third = values[2]; // 30, 第三个元素
5 int last = values[4];  // 50, 最后一个元素
6
7 values[1] = 25;        // 修改第二个元素
```

数组可以在声明时初始化。如果初始值的数量少于数组大小，剩余元素被初始化为零；如果提供了初始值但省略数组大小，编译器会根据初始值数量自动确定大小：

```
1 int a[5] = {1, 2, 3}; // a = {1, 2, 3, 0, 0}
2 int b[5] = {};        // b = {0, 0, 0, 0, 0}, 全部初始化为 0
3 int c[] = {1, 2, 3, 4, 5}; // 编译器推断大小为 5
```

数组的一个重要特性是它不进行边界检查。访问超出范围的索引不会产生编译错误，但会导致未定义行为——程序可能崩溃、产生错误结果，或者看似正常运行但实际上破坏了其他内存区域：

```
1 int arr[5] = {1, 2, 3, 4, 5};
2 int x = arr[10]; // 未定义行为！访问越界
3 arr[-1] = 0;     // 未定义行为！负索引
```

这种越界错误是 C/C++ 程序中最常见的 bug 来源之一。在 RoboMaster 开发中，数组越界可能导致机器人行为异常甚至失控，因此必须格外小心。

遍历数组通常使用 `for` 循环。由于数组本身不存储大小信息，需要单独跟踪数组长度：

```
1 const int SIZE = 5;
2 int values[SIZE] = {10, 20, 30, 40, 50};
3
4 // 传统 for 循环
5 for (int i = 0; i < SIZE; i++) {
6     std::cout << values[i] << " ";
7 }
8
9 // 范围 for 循环 (C++11)
10 for (int v : values) {
11     std::cout << v << " ";
12 }
```

多维数组用于表示表格、矩阵等结构。声明时指定每个维度的大小，访问时提供每个维度的索引：


```

1 // 3x4 的二维数组 (3 行 4 列)
2 int matrix[3][4] = {
3     {1, 2, 3, 4},
4     {5, 6, 7, 8},
5     {9, 10, 11, 12}
6 };
7
8 int element = matrix[1][2]; // 第 2 行第 3 列, 值为 7
9
10 // 遍历二维数组
11 for (int i = 0; i < 3; i++) {
12     for (int j = 0; j < 4; j++) {
13         std::cout << matrix[i][j] << " ";
14     }
15     std::cout << std::endl;
16 }

```

在内存中，多维数组按行优先（row-major）顺序存储，即同一行的元素在内存中相邻。遍历时按行优先顺序访问可以获得更好的缓存性能。

1.5.7.2. 数组与函数

将数组传递给函数时，数组会退化为指向首元素的指针，数组的大小信息丢失。因此，通常需要额外传递数组的大小：

```

1 // 数组参数实际上是指针
2 void printArray(int arr[], int size) {
3     for (int i = 0; i < size; i++) {
4         std::cout << arr[i] << " ";
5     }
6     std::cout << std::endl;
7 }
8
9 // 等价写法
10 void printArray(int* arr, int size);
11
12 int main() {
13     int values[] = {1, 2, 3, 4, 5};
14     printArray(values, 5);
15     return 0;
16 }

```

由于传递的是指针，函数内部对数组元素的修改会影响原数组。如果不希望函数修改数组内容，可以使用 `const` 修饰：

```

1 double average(const int arr[], int size) {
2     int sum = 0;
3     for (int i = 0; i < size; i++) {
4         sum += arr[i];
5     }
6     return static_cast<double>(sum) / size;
7 }

```

对于多维数组，除第一维外的其他维度大小必须在函数参数中明确指定：

```

1 void processMatrix(int matrix[][4], int rows) {
2     for (int i = 0; i < rows; i++) {
3         for (int j = 0; j < 4; j++) {
4             // 处理 matrix[i][j]

```



```

5     }
6   }
7 }

```

这一限制源于编译器需要知道每行的大小才能正确计算元素地址。

1.5.7.3. `std::array`

C++11 引入的 `std::array` 是对 C 风格数组的封装，提供了更安全、更方便的接口，同时保持与 C 风格数组相当的性能：

```

1 #include <array>
2
3 std::array<int, 5> values = {10, 20, 30, 40, 50};
4
5 int first = values[0];      // 下标访问，不检查边界
6 int second = values.at(1);  // at() 访问，检查边界，越界抛出异常
7
8 std::cout << "大小: " << values.size() << std::endl; // 5
9 std::cout << "是否为空: " << values.empty() << std::endl; // false

```

`std::array` 相比 C 风格数组的主要优势包括：

它知道自己的大小。`size()` 成员函数返回元素数量，无需单独维护大小变量。

它提供边界检查选项。`at()` 方法在越界时抛出 `std::out_of_range` 异常，便于调试。

它可以像普通对象一样复制和赋值。C 风格数组不能直接赋值，`std::array` 可以：

```

1 std::array<int, 3> a = {1, 2, 3};
2 std::array<int, 3> b = a; // 复制整个数组
3 b = {4, 5, 6};          // 赋值

```

它可以作为函数返回值。C 风格数组不能从函数返回，`std::array` 可以：

```

1 std::array<double, 3> getPosition() {
2     return {1.0, 2.0, 3.0};
3 }

```

它与标准库算法和容器兼容。`std::array` 提供迭代器，可以与 `std::sort`、`std::find` 等算法配合使用：

```

1 std::array<int, 5> arr = {5, 2, 8, 1, 9};
2 std::sort(arr.begin(), arr.end()); // 排序
3
4 auto it = std::find(arr.begin(), arr.end(), 8);
5 if (it != arr.end()) {
6     std::cout << "找到了 8" << std::endl;
7 }

```

`std::array` 的大小必须是编译期常量，这与 C 风格数组相同。如果需要运行时确定大小的数组，应当使用 `std::vector`（将在后续章节介绍）。

在 RoboMaster 开发中，`std::array` 适合存储固定大小的数据，如三维坐标、四元数、固定数量的电机参数等：

```

1 std::array<double, 3> position = {0.0, 0.0, 0.0};
2 std::array<double, 4> quaternion = {1.0, 0.0, 0.0, 0.0};
3 std::array<int, 4> motorIDs = {1, 2, 3, 4};

```

1.5.7.4. C 风格字符串

在 C 语言中，字符串是以空字符（`'\0'`）结尾的字符数组。C++ 继承了这种表示方式，称为 C 风格字符串：

```
1 char greeting[] = "Hello"; // 编译器自动添加 '\0'，实际大小为 6
2 char manual[] = {'H', 'e', 'l', 'l', 'o', '\0'}; // 等价写法
```

字符串字面量（如 `"Hello"`）的类型是 `const char[]`，存储在只读内存区域。尝试修改字符串字面量会导致未定义行为：

```
1 char* str = "Hello"; // 不推荐，应该用 const char*
2 str[0] = 'h';        // 未定义行为！修改只读内存
3
4 const char* str2 = "Hello"; // 正确的写法
```

C 风格字符串的操作函数定义在 `<cstring>` 头文件中：

```
1 #include <cstring>
2
3 char str1[20] = "Hello";
4 char str2[] = "World";
5
6 size_t len = strlen(str1); // 字符串长度（不含 '\0'），返回 5
7 strcpy(str1, str2);        // 复制 str2 到 str1
8 strcat(str1, "!");         // 连接字符串
9 int cmp = strcmp(str1, str2); // 比较字符串，相等返回 0
10 const char* pos = strstr(str1, "or"); // 查找子串
```

这些函数存在严重的安全隐患。`strcpy` 和 `strcat` 不检查目标缓冲区大小，容易导致缓冲区溢出——这是最常见的安全漏洞之一。现代 C++ 代码应当尽量避免使用这些函数，改用 `std::string`。

1.5.7.5. std::string

`std::string` 是 C++ 标准库提供的字符串类，它自动管理内存、提供丰富的操作接口，并且安全易用：

```
1 #include <string>
2
3 std::string greeting = "Hello";
4 std::string name = "RoboMaster";
5 std::string message = greeting + ", " + name + "!"; // 字符串连接
6
7 std::cout << message << std::endl; // 输出: Hello, RoboMaster!
8 std::cout << "长度: " << message.length() << std::endl; // 长度: 18
```

`std::string` 支持多种初始化方式：

```
1 std::string s1; // 空字符串
2 std::string s2 = "Hello"; // 从字符串字面量初始化
3 std::string s3("Hello"); // 等价写法
4 std::string s4(5, 'x'); // "xxxxx", 5 个 'x'
5 std::string s5 = s2; // 复制构造
6 std::string s6 = s2.substr(0, 3); // "Hel", 子串
```

访问单个字符可以使用下标运算符或 `at()` 方法：

```

1 std::string str = "Hello";
2
3 char c1 = str[0];    // 'H', 不检查边界
4 char c2 = str.at(1); // 'e', 检查边界
5
6 str[0] = 'h';        // 修改第一个字符

```

`std::string` 提供了丰富的操作方法:

```

1 std::string str = "Hello, World!";
2
3 // 查找
4 size_t pos = str.find("World");    // 返回 7
5 size_t pos2 = str.find("xyz");     // 返回 std::string::npos (未找到)
6
7 if (pos != std::string::npos) {
8     std::cout << "找到了, 位置: " << pos << std::endl;
9 }
10
11 // 子串
12 std::string sub = str.substr(7, 5); // "World", 从位置 7 开始取 5 个字符
13
14 // 替换
15 str.replace(7, 5, "C++");          // "Hello, C++!"
16
17 // 插入和删除
18 str.insert(7, "Dear ");             // "Hello, Dear C++!"
19 str.erase(7, 5);                   // "Hello, C++!"
20
21 // 追加
22 str.append(" is great");            // "Hello, C++! is great"
23 str += "!";                        // 等价于 append
24
25 // 清空和判空
26 str.clear();                       // 清空字符串
27 bool isEmpty = str.empty();        // true

```

字符串比较可以直接使用关系运算符, 按字典序比较:

```

1 std::string a = "apple";
2 std::string b = "banana";
3
4 if (a < b) {
5     std::cout << a << " 在 " << b << " 之前" << std::endl;
6 }
7
8 if (a == "apple") {
9     std::cout << "是苹果" << std::endl;
10 }

```

与 C 风格字符串的转换有时是必要的, 特别是与 C 语言库或系统 API 交互时:

```

1 std::string cppStr = "Hello";
2
3 // std::string 转 C 风格字符串
4 const char* cStr = cppStr.c_str();
5
6 // C 风格字符串转 std::string
7 const char* source = "World";
8 std::string newStr = source; // 自动转换

```

1.5.7.6. 字符串与数值转换

在处理配置文件、用户输入或通信协议时，经常需要在字符串和数值之间转换。C++11 提供了一组便捷的转换函数：

```
1 #include <string>
2
3 // 数值转字符串
4 int num = 42;
5 double pi = 3.14159;
6 std::string s1 = std::to_string(num);    // "42"
7 std::string s2 = std::to_string(pi);     // "3.141590"
8
9 // 字符串转数值
10 std::string str1 = "123";
11 std::string str2 = "3.14";
12 std::string str3 = "42abc";
13
14 int i = std::stoi(str1);                 // 123
15 double d = std::stod(str2);              // 3.14
16 int partial = std::stoi(str3);           // 42, 解析到非数字字符停止
17
18 // 其他转换函数
19 long l = std::stol("1234567890");
20 float f = std::stof("2.718");
```

这些函数在遇到无法解析的输入时会抛出异常（`std::invalid_argument` 或 `std::out_of_range`），使用时应当进行错误处理：

```
1 std::string input = "not a number";
2
3 try {
4     int value = std::stoi(input);
5 } catch (const std::invalid_argument& e) {
6     std::cerr << "无效输入: " << e.what() << std::endl;
7 } catch (const std::out_of_range& e) {
8     std::cerr << "数值超出范围: " << e.what() << std::endl;
9 }
```

1.5.7.7. 字符串流

`<sstream>` 头文件提供的字符串流允许像使用 `cin / cout` 一样操作字符串，这在格式化输出和解析输入时非常有用：

```
1 #include <sstream>
2
3 // 格式化输出到字符串
4 std::ostringstream oss;
5 oss << "Position: (" << 1.5 << ", " << 2.5 << ", " << 3.5 << ")";
6 std::string result = oss.str(); // "Position: (1.5, 2.5, 3.5)"
7
8 // 从字符串解析数据
9 std::string data = "100 200 300";
10 std::istringstream iss(data);
11 int x, y, z;
12 iss >> x >> y >> z; // x=100, y=200, z=300
```

字符串流在 RoboMaster 开发中常用于构建日志消息、解析配置文件或处理串口通信数据：

```
1 // 构建日志消息
```

```

2  std::ostringstream log;
3  log << "[" << timestamp << "]" Motor " << motorId
4    << ": speed=" << speed << " rpm, temp=" << temperature << "°C";
5  logger.write(log.str());
6
7  // 解析串口数据
8  std::string response = "OK 1234 5678";
9  std::istringstream parser(response);
10 std::string status;
11 int value1, value2;
12 parser >> status >> value1 >> value2;

```

1.5.7.8. 实践建议

在现代 C++ 开发中，应当优先使用 `std::array` 和 `std::string`，而非 C 风格数组和字符串。它们更安全、更易用，且性能开销可以忽略不计。

C 风格数组和字符串仍有其用武之地：与 C 语言库交互、嵌入式开发中的内存受限场景、或者需要精确控制内存布局时。但即使在这些场景下，也应当尽量将 C 风格代码封装在有限的范围内，对外提供现代 C++ 接口。

对于大小在运行时确定或需要动态增长的数组，应当使用 `std::vector`。它提供了动态数组的功能，是 C++ 中最常用的容器之一，将在后续章节详细介绍。

数组和字符串是处理数据的基础工具。然而，到目前为止我们操作的都是数据的“值”。下一节将介绍指针，它允许我们直接操作数据的“地址”，是理解 C++ 内存模型的关键。

1.5.8. 指针基础

到目前为止，我们操作的都是变量的值——读取它、修改它、传递它的副本。但有时候，我们需要直接操作变量在内存中的位置：让多个变量指向同一块数据、在函数中修改调用者的变量、动态分配内存、构建链表和树等复杂数据结构。指针正是实现这些功能的工具。理解指针是掌握 C++ 的关键一步，它揭示了程序与内存交互的底层机制。

1.5.8.1. 内存地址与指针的概念

计算机内存可以想象成一排连续编号的格子，每个格子存储一个字节的数据，格子的编号就是内存地址。当我们声明一个变量时，编译器会在内存中分配一块空间来存储它的值，这块空间有一个起始地址。

```

1  int x = 42;

```

假设 `x` 被分配在地址 `0x7ffd5e8c` 处，占用 4 个字节（`int` 类型的典型大小）。变量名 `x` 是这块内存的标签，`42` 是其中存储的值，`0x7ffd5e8c` 是它的地址。

取地址运算符 `&` 用于获取变量的内存地址：

```

1  int x = 42;
2  std::cout << "x 的值: " << x << std::endl;
3  std::cout << "x 的地址: " << &x << std::endl; // 输出类似 0x7ffd5e8c

```

指针是一种特殊的变量，它存储的不是普通数据，而是另一个变量的内存地址。声明指针时，在类型名后加上 `*`：

```

1  int x = 42;

```

```

2 int* ptr = &x; // ptr 是指向 int 的指针, 存储 x 的地址
3
4 std::cout << "ptr 的值: " << ptr << std::endl; // x 的地址
5 std::cout << "ptr 指向的值: " << *ptr << std::endl; // 42

```

这里 `int*` 表示“指向 `int` 的指针”类型。`ptr` 存储的是 `x` 的地址, 通过解引用运算符 `*` 可以访问 `ptr` 所指向的内存中的值。

指针声明的语法有几种等价写法, 选择哪种主要是风格问题:

```

1 int* ptr1; // 星号靠近类型, 强调 ptr1 的类型是 int*
2 int *ptr2; // 星号靠近变量名, C 语言传统风格
3 int * ptr3; // 两边都有空格, 较少使用

```

需要注意的是, 星号只与紧跟其后的变量名结合。在同一行声明多个变量时容易出错:

```

1 int* p1, p2; // 注意: p1 是指针, p2 是普通 int!
2 int *p3, *p4; // p3 和 p4 都是指针

```

为避免混淆, 建议每行只声明一个指针变量。

1.5.8.2. 解引用与指针运算

解引用运算符 `*` 用于访问指针所指向的内存。它既可以读取值, 也可以修改值:

```

1 int x = 10;
2 int* ptr = &x;
3
4 std::cout << *ptr << std::endl; // 读取: 输出 10
5
6 *ptr = 20; // 写入: 修改 ptr 指向的内存
7 std::cout << x << std::endl; // x 现在是 20

```

通过指针修改数据是指针最重要的用途之一。它使得函数能够修改调用者的变量, 也是实现复杂数据结构的基础。

指针支持算术运算, 但其行为与普通整数不同。指针加 1 不是地址值加 1, 而是移动到下一个元素的位置:

```

1 int arr[] = {10, 20, 30, 40, 50};
2 int* ptr = arr; // 指向数组首元素
3
4 std::cout << *ptr << std::endl; // 10
5 std::cout << *(ptr + 1) << std::endl; // 20
6 std::cout << *(ptr + 2) << std::endl; // 30
7
8 ptr++; // ptr 现在指向 arr[1]
9 std::cout << *ptr << std::endl; // 20

```

指针加 1 实际上使地址值增加了 `sizeof(int)` 字节 (通常是 4)。这种设计使得指针能够方便地遍历数组, 而无需关心元素的具体大小。

两个指向同一数组的指针可以相减, 结果是它们之间的元素个数:

```

1 int arr[] = {10, 20, 30, 40, 50};
2 int* p1 = &arr[1];
3 int* p2 = &arr[4];
4
5 std::ptrdiff_t diff = p2 - p1; // 3, 相差 3 个元素

```

1.5.8.3. 指针与数组

数组和指针的关系非常密切。在大多数表达式中，数组名会自动转换为指向首元素的指针：

```
1 int arr[] = {10, 20, 30, 40, 50};
2
3 int* ptr = arr;          // arr 退化为 &arr[0]
4 std::cout << *ptr << std::endl;    // 10
5 std::cout << ptr[2] << std::endl;   // 30, 等价于 *(ptr + 2)
6 std::cout << arr[2] << std::endl;   // 30, 等价于 *(arr + 2)
```

下标运算符 `[]` 实际上是指针算术的语法糖：`arr[i]` 完全等价于 `*(arr + i)`。这也解释了为什么数组下标从 0 开始——`arr[0]` 就是 `*(arr + 0)`，即首元素本身。

尽管数组名可以像指针一样使用，但它们并不完全相同。数组名是常量，不能被重新赋值；数组名使用 `sizeof` 返回整个数组的大小，而指针返回指针本身的大小：

```
1 int arr[5] = {1, 2, 3, 4, 5};
2 int* ptr = arr;
3
4 // arr = ptr; // 错误！数组名不能被赋值
5
6 std::cout << sizeof(arr) << std::endl; // 20 (5 个 int)
7 std::cout << sizeof(ptr) << std::endl; // 8 (64 位系统上指针的大小)
```

遍历数组可以使用下标，也可以使用指针：

```
1 int arr[] = {10, 20, 30, 40, 50};
2 int size = sizeof(arr) / sizeof(arr[0]);
3
4 // 下标方式
5 for (int i = 0; i < size; i++) {
6     std::cout << arr[i] << " ";
7 }
8
9 // 指针方式
10 for (int* p = arr; p < arr + size; p++) {
11     std::cout << *p << " ";
12 }
13
14 // 指针 + 下标混合
15 int* ptr = arr;
16 for (int i = 0; i < size; i++) {
17     std::cout << ptr[i] << " ";
18 }
```

这三种方式在效果上完全等价，选择哪种主要看个人偏好和具体场景。

1.5.8.4. 指针与函数参数

在前面的章节中，我们已经了解到 C++ 默认使用值传递——函数接收的是参数的副本，对副本的修改不影响原变量。通过指针传递，函数可以修改调用者的数据：

```
1 // 值传递：无法修改原变量
2 void incrementByValue(int n) {
3     n++; // 只修改了副本
4 }
5
6 // 指针传递：可以修改原变量
7 void incrementByPointer(int* ptr) {
```

```

8     (*ptr)++; // 修改指针指向的内存
9 }
10
11 int main() {
12     int x = 10;
13
14     incrementByValue(x);
15     std::cout << x << std::endl; // 仍然是 10
16
17     incrementByPointer(&x);
18     std::cout << x << std::endl; // 变成 11
19
20     return 0;
21 }

```

指针参数常用于以下场景：

需要修改调用者的变量时，如交换两个变量的值：

```

1 void swap(int* a, int* b) {
2     int temp = *a;
3     *a = *b;
4     *b = temp;
5 }
6
7 int x = 10, y = 20;
8 swap(&x, &y); // x = 20, y = 10

```

需要返回多个值时，可以通过指针参数“返回”额外的结果：

```

1 // 返回商和余数
2 void divide(int dividend, int divisor, int* quotient, int* remainder) {
3     *quotient = dividend / divisor;
4     *remainder = dividend % divisor;
5 }
6
7 int q, r;
8 divide(17, 5, &q, &r); // q = 3, r = 2

```

传递大型数据结构时，避免复制开销：

```

1 // 传值：复制整个结构体，开销大
2 void processData(LargeStruct data);
3
4 // 传指针：只传递地址，开销小
5 void processData(const LargeStruct* data);

```

当函数不需要修改指针指向的数据时，应当使用 `const` 修饰，这既表明了意图，也让编译器帮助检查错误。

1.5.8.5. 空指针与野指针

指针的一个重要特性是它可以不指向任何有效对象。空指针（null pointer）是一个特殊的指针值，表示“不指向任何东西”。在 C++11 之前，通常使用 `NULL` 或 `0` 表示空指针；C++11 引入了 `nullptr` 关键字，它是类型安全的空指针常量：

```

1 int* ptr1 = nullptr; // C++11 推荐写法
2 int* ptr2 = NULL;    // C 风格，仍然有效
3 int* ptr3 = 0;       // 也有效，但不推荐

```



```

4
5 if (ptr1 == nullptr) {
6     std::cout << "ptr1 是空指针" << std::endl;
7 }

```

解引用空指针会导致程序崩溃（通常是段错误）。因此，在使用指针之前，通常需要检查它是否为空：

```

1 void processTarget(Target* target) {
2     if (target == nullptr) {
3         std::cerr << "错误: 目标指针为空" << std::endl;
4         return;
5     }
6
7     // 安全地使用 target
8     target->track();
9 }

```

野指针（dangling pointer）是指向已释放或无效内存的指针，比空指针更危险。野指针的解引用可能不会立即崩溃，而是产生难以预测的行为：

```

1 int* createDanglingPointer() {
2     int local = 42;
3     return &local; // 危险！返回局部变量的地址
4 }
5
6 int main() {
7     int* ptr = createDanglingPointer();
8     // ptr 现在是野指针，local 已经被销毁
9     std::cout << *ptr << std::endl; // 未定义行为！
10    return 0;
11 }

```

避免野指针的原则包括：不返回局部变量的地址；释放内存后立即将指针置为 `nullptr`；使用智能指针（后续章节介绍）自动管理内存。

1.5.8.6. const 与指针

`const` 关键字与指针结合使用时，可以限制指针本身或指针所指向的数据。根据 `const` 的位置不同，有三种情况：

指向常量的指针（pointer to const）：不能通过指针修改所指向的数据，但指针本身可以改变指向：

```

1 int x = 10, y = 20;
2 const int* ptr = &x; // ptr 指向 const int
3
4 // *ptr = 30; // 错误！不能通过 ptr 修改数据
5 ptr = &y;      // 允许，ptr 可以指向其他地址

```

常量指针（const pointer）：指针本身不能改变指向，但可以通过指针修改数据：

```

1 int x = 10, y = 20;
2 int* const ptr = &x; // ptr 是 const，指向 int
3
4 *ptr = 30; // 允许，可以修改数据
5 // ptr = &y; // 错误！ptr 不能指向其他地址

```

指向常量的常量指针：两者都不能改变：

```
1 int x = 10;
2 const int* const ptr = &x;
3
4 // *ptr = 30; // 错误!
5 // ptr = &y; // 错误!
```

记忆技巧是从右向左读：`const int* ptr` 读作“ptr 是指针，指向 `const int`”；`int* const ptr` 读作“ptr 是 `const`，是指针，指向 `int`”。

在函数参数中，`const` 指针用于表明函数不会修改传入的数据：

```
1 // 承诺不修改数组内容
2 double calculateAverage(const int* arr, int size) {
3     double sum = 0;
4     for (int i = 0; i < size; i++) {
5         sum += arr[i];
6         // arr[i] = 0; // 错误! 不能修改
7     }
8     return sum / size;
9 }
```

1.5.8.7. 指针与动态内存

到目前为止，我们使用的变量都在栈上分配，它们的生命周期由作用域自动管理。但有时候，我们需要在运行时决定分配多少内存，或者让数据的生命周期超出创建它的函数。这时就需要动态内存分配。

C++ 使用 `new` 运算符在堆上分配内存，使用 `delete` 运算符释放内存：

```
1 // 分配单个对象
2 int* ptr = new int;           // 分配一个 int
3 *ptr = 42;
4 delete ptr;                   // 释放内存
5
6 // 分配并初始化
7 int* ptr2 = new int(100); // 分配并初始化为 100
8 delete ptr2;
9
10 // 分配数组
11 int* arr = new int[10];      // 分配 10 个 int 的数组
12 for (int i = 0; i < 10; i++) {
13     arr[i] = i * 10;
14 }
15 delete[] arr;                // 注意：数组用 delete[]
```

动态内存管理是 C++ 中最容易出错的领域之一。常见的问题包括：

内存泄漏：分配了内存但忘记释放，导致内存被持续占用：

```
1 void memoryLeak() {
2     int* ptr = new int[1000];
3     // 忘记 delete[], 函数返回后内存泄漏
4 }
```

重复释放：对同一块内存调用多次 `delete`，导致未定义行为：

```
1 int* ptr = new int;
2 delete ptr;
```

```
3 delete ptr; // 错误! 重复释放
```

使用已释放的内存：释放后继续使用指针：

```
1 int* ptr = new int(42);
2 delete ptr;
3 std::cout << *ptr << std::endl; // 未定义行为!
```

数组与单对象混淆：用 `delete` 释放 `new[]` 分配的内存，或反过来：

```
1 int* arr = new int[10];
2 delete arr; // 错误! 应该用 delete[]
```

为了避免这些问题，现代 C++ 推荐使用智能指针（`std::unique_ptr`、`std::shared_ptr`）来管理动态内存，它们会在适当的时机自动释放内存。智能指针将在后续章节详细介绍。

在 RoboMaster 开发中，动态内存分配常见于创建运行时大小确定的缓冲区、管理检测到的目标列表等场景。但由于实时性要求，应当尽量减少动态分配的频率，或在初始化阶段预分配所需内存。

1.5.8.8. 指针的常见用途

指针在 C++ 中有多种重要用途，以下是一些典型场景：

实现数据结构。链表、树、图等结构需要节点之间相互引用，指针是实现这种引用的自然方式：

```
1 struct ListNode {
2     int data;
3     ListNode* next;
4 };
```

多态与动态绑定。通过基类指针调用派生类的方法，是面向对象编程的核心技术（后续章节介绍）：

```
1 class Robot {
2 public:
3     virtual void move() = 0;
4 };
5
6 void controlRobot(Robot* robot) {
7     robot->move(); // 根据实际类型调用对应的实现
8 }
```

可选参数。指针可以为 `nullptr` 表示“无值”，这在引用无法做到的场景很有用：

```
1 void processData(const Config* optionalConfig = nullptr) {
2     if (optionalConfig != nullptr) {
3         // 使用配置
4     } else {
5         // 使用默认配置
6     }
7 }
```

与 C 语言库交互。许多系统 API 和第三方库使用 C 风格接口，需要通过指针传递数据。

1.5.8.9. 实践建议

指针是强大但危险的工具。以下是一些实践建议：

始终初始化指针。未初始化的指针包含随机地址，解引用会导致不可预测的行为。如果暂时没有有效地址，将指针初始化为 `nullptr`。

使用前检查空指针。特别是对于函数参数和函数返回值，不要假设指针一定有效。

释放后置空。`delete` 后立即将指针置为 `nullptr`，可以避免野指针和重复释放。

```
1 delete ptr;
2 ptr = nullptr;
```

优先使用引用。当不需要“可空”语义时，引用比指针更安全。

优先使用智能指针。对于动态内存管理，`std::unique_ptr` 和 `std::shared_ptr` 能够自动处理释放，大大减少内存错误。

理解指针是理解 C++ 内存模型的关键。它解释了为什么函数参数默认是值传递、为什么数组在传递时会丢失大小信息、为什么需要区分栈和堆。下一节将介绍引用，它提供了一种更安全、更简洁的方式来实现指针的部分功能。

1.5.9. 引用

在上一节中，我们学习了指针——一种直接操作内存地址的机制。指针功能强大，但使用起来需要格外小心：需要解引用才能访问数据，需要检查空指针，还要防范野指针。C++ 提供了另一种间接访问数据的方式——引用（reference）。引用可以看作是变量的别名，它提供了指针的部分功能，却更加安全和直观。理解引用与指针的区别，并在恰当的场合选择合适的工具，是写出清晰、安全的 C++ 代码的重要一步。

1.5.9.1. 引用的基本概念

引用是已存在变量的另一个名字。声明引用时，在类型名后加上 `&` 符号，并且必须在声明时初始化：

```
1 int x = 42;
2 int& ref = x; // ref 是 x 的引用，即 x 的别名
3
4 std::cout << x << std::endl; // 42
5 std::cout << ref << std::endl; // 42
6
7 ref = 100; // 通过引用修改
8 std::cout << x << std::endl; // 100, x 也变了
```

在这个例子中，`ref` 和 `x` 指向同一块内存。对 `ref` 的任何操作，实际上就是对 `x` 的操作。引用不是一个独立的对象，它只是现有对象的另一个名称。

与指针不同，引用一旦绑定到某个变量，就不能再绑定到其他变量。引用在声明时必须初始化，之后对引用的任何赋值都是在修改被引用对象的值，而非改变引用本身的绑定：

```
1 int a = 10;
2 int b = 20;
3 int& ref = a; // ref 绑定到 a
4
5 ref = b; // 这不是让 ref 绑定到 b，而是把 b 的值赋给 a
6 std::cout << a << std::endl; // 20, a 的值变成了 b 的值
7 std::cout << &a << std::endl; // a 的地址
```

```
8 std::cout << &ref << std::endl; // 相同的地址, ref 仍然绑定到 a
```

这一特性使得引用比指针更容易理解:一旦建立引用关系,就不会改变。使用引用时也无需像指针那样使用 `*` 解引用或 `&` 取地址,语法上与普通变量完全相同。

1.5.9.2. 引用与指针的对比

引用和指针都提供了间接访问数据的能力,但它们在设计理念和 Usage 方式上有本质区别。理解这些区别有助于在不同场景下做出正确的选择。

首先,引用必须在声明时初始化,且始终绑定到某个有效对象;指针可以不初始化,也可以指向 `nullptr` 表示“不指向任何对象”。这意味着引用更安全——使用引用时不需要检查它是否有效,而使用指针时通常需要进行空指针检查:

```
1 void processWithPointer(int* ptr) {
2     if (ptr == nullptr) { // 必须检查
3         return;
4     }
5     *ptr = 100;
6 }
7
8 void processWithReference(int& ref) {
9     // 无需检查, ref 一定绑定到有效对象
10    ref = 100;
11 }
```

其次,引用一旦绑定就不能改变,而指针可以随时指向其他对象。这在需要重新指向不同对象的场景下,指针更加灵活;而在希望保持绑定关系不变的场景下,引用更加安全。

再者,语法上的差异也很明显。指针需要 `*` 来解引用、`&` 来取地址,而引用的使用与普通变量无异:

```
1 int x = 42;
2
3 // 使用指针
4 int* ptr = &x;
5 *ptr = 100;
6 std::cout << *ptr << std::endl;
7
8 // 使用引用
9 int& ref = x;
10 ref = 100;
11 std::cout << ref << std::endl;
```

引用的语法更简洁,代码也更容易阅读。特别是在连续的操作中,不断地写 `*ptr` 会让代码显得杂乱。

此外,引用没有“引用的引用”或“引用的数组”这样的概念,而指针可以有级指针(如 `int*`)和指针数组。在需要复杂数据结构的场合,指针提供了更大的灵活性。

在实践中,应当优先考虑使用引用,除非以下情况需要使用指针:需要表示“可能不存在”(使用 `nullptr`)、需要在运行时改变所指对象、需要进行指针运算、或者需要与 C 语言接口交互。现代 C++ 的设计哲学是:能用引用就不用指针,能用智能指针就不用裸指针。

1.5.9.3. 引用作为函数参数

引用最常见的用途是作为函数参数。在前面的章节中，我们已经见过这种用法，现在来更深入地理解它的工作原理和优势。

当使用值传递时，函数接收的是实参的副本，对形参的修改不会影响实参。而使用引用传递时，形参是实参的别名，对形参的修改直接作用于实参：

```
1 void swapByValue(int a, int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5     // 交换的只是副本，对原变量没有影响
6 }
7
8 void swapByReference(int& a, int& b) {
9     int temp = a;
10    a = b;
11    b = temp;
12    // 直接操作原变量，交换成功
13 }
14
15 int main() {
16     int x = 10, y = 20;
17
18     swapByValue(x, y);
19     std::cout << x << ", " << y << std::endl; // 10, 20, 没有交换
20
21     swapByReference(x, y);
22     std::cout << x << ", " << y << std::endl; // 20, 10, 交换成功
23
24     return 0;
25 }
```

引用参数的第一个优势是允许函数修改调用者的变量。这在需要返回多个值的场景下特别有用：

```
1 // 计算同时返回商和余数
2 void divideWithRemainder(int dividend, int divisor,
3                          int& quotient, int& remainder) {
4     quotient = dividend / divisor;
5     remainder = dividend % divisor;
6 }
7
8 int main() {
9     int q, r;
10    divideWithRemainder(17, 5, q, r);
11    std::cout << "商: " << q << ", 余数: " << r << std::endl; // 商: 3, 余
    数: 2
12    return 0;
13 }
```

引用参数的第二个优势是避免复制开销。对于大型对象，值传递需要调用拷贝构造函数创建副本，这可能耗费大量时间和内存。通过引用传递，只传递对象的别名，完全消除了复制的开销：

```
1 // 值传递：每次调用都会复制整个 vector
2 double calculateAverageByValue(std::vector<double> data) {
3     double sum = 0;
4     for (double v : data) sum += v;
5     return sum / data.size();
6 }
```

```

6 }
7
8 // 引用传递：不复制，效率高
9 double calculateAverageByReference(std::vector<double>& data) {
10     double sum = 0;
11     for (double v : data) sum += v;
12     return sum / data.size();
13 }

```

对于包含成百上千个元素的 `std::vector`，两者的性能差异可能非常显著。在 RoboMaster 开发中，传感器数据、图像帧、目标列表等往往是大型对象，使用引用传递是标准做法。

引用参数与指针参数相比，调用时的语法更加自然。使用指针参数时，调用者需要显式取地址；使用引用参数时，直接传入变量名即可：

```

1 void updateWithPointer(int* ptr) { *ptr = 100; }
2 void updateWithReference(int& ref) { ref = 100; }
3
4 int x = 0;
5 updateWithPointer(&x); // 需要 &
6 updateWithReference(x); // 直接传变量名

```

然而，这种简洁性也带来一个潜在的问题：从调用代码来看，无法直接判断函数是否会修改参数。`updateWithReference(x)` 看起来像是值传递，但实际上 `x` 可能被修改。为了解决这个问题，当函数不需要修改参数时，应当使用常量引用。

1.5.9.4. 常量引用

常量引用（const reference）是指向常量的引用，通过它不能修改被引用的对象。声明常量引用时，在类型前加上 `const`：

```

1 int x = 42;
2 const int& cref = x;
3
4 std::cout << cref << std::endl; // 42, 可以读取
5 // cref = 100; // 错误！不能通过常量引用修改

```

常量引用最重要的用途是作为函数参数，表明函数只读取数据而不修改它。这既是一种性能优化（避免复制），也是一种文档说明（告诉调用者参数不会被修改）：

```

1 // 常量引用参数：承诺不修改 data
2 double calculateAverage(const std::vector<double>& data) {
3     double sum = 0;
4     for (double v : data) sum += v;
5     return sum / data.size();
6     // data.push_back(0); // 错误！不能修改
7 }

```

使用常量引用后，调用者可以放心地传入自己的数据，知道函数不会对其进行任何修改。编译器也会检查函数体内是否有违反承诺的操作，在编译阶段捕获潜在的 bug。

常量引用还有一个特殊性质：它可以绑定到临时对象（右值）。普通引用只能绑定到变量（左值），但常量引用例外：

```

1 // int& ref = 42; // 错误！普通引用不能绑定到字面量

```



```

2  const int& cref = 42; // 正确, 常量引用可以绑定到临时值
3
4  // 这使得以下调用成为可能
5  void printValue(const int& value) {
6      std::cout << value << std::endl;
7  }
8
9  printValue(100); // 可以传入字面量
10 printValue(x + y); // 可以传入表达式的结果

```

如果函数参数是非常量引用, 就不能传入临时值:

```

1  void modify(int& value) {
2      value = 100;
3  }
4
5  // modify(42); // 错误! 不能将临时值绑定到非常量引用

```

这个限制是合理的: 修改一个即将消失的临时值没有意义, 编译器通过这个规则帮助我们避免无意义甚至错误的操作。

在 RoboMaster 开发中, 常量引用是函数参数的常用形式。一条简单的经验法则是: 对于基本类型 (如 `int`、`double`), 由于它们很小, 直接值传递即可; 对于类类型和容器 (如 `std::string`、`std::vector`、自定义类), 如果函数不需要修改参数, 就使用常量引用传递:

```

1  // 基本类型: 值传递
2  int square(int x) {
3      return x * x;
4  }
5
6  // 类类型: 常量引用传递
7  void logMessage(const std::string& message) {
8      std::cout << "[LOG] " << message << std::endl;
9  }
10
11 // 容器: 常量引用传递
12 double processReadings(const std::vector<double>& readings) {
13     // 处理传感器读数
14 }
15
16 // 需要修改时: 非常量引用传递
17 void normalizeVector(std::vector<double>& vec) {
18     double sum = 0;
19     for (double v : vec) sum += v;
20     for (double& v : vec) v /= sum;
21 }

```

1.5.9.5. 引用作为返回值

函数不仅可以接受引用作为参数, 还可以返回引用。返回引用允许调用者直接访问和修改函数返回的对象, 而无需复制。这在操作容器元素时特别有用:

```

1  class Warehouse {
2  private:
3      std::vector<int> inventory;
4
5  public:
6      Warehouse() : inventory(10, 0) {}
7  }

```



```

8      // 返回引用, 允许外部修改库存
9      int& itemAt(size_t index) {
10         return inventory[index];
11     }
12
13     // 返回常量引用, 只允许读取
14     const int& itemAt(size_t index) const {
15         return inventory[index];
16     }
17 };
18
19 int main() {
20     Warehouse warehouse;
21     warehouse.itemAt(3) = 100; // 直接修改库存
22     std::cout << warehouse.itemAt(3) << std::endl; // 100
23     return 0;
24 }

```

标准库中的 `std::vector::operator[]` 和 `std::map::operator[]` 都返回引用, 这就是为什么我们可以写 `vec[0] = 10` 这样的代码。

然而, 返回引用时必须格外小心: 绝不能返回局部变量的引用。局部变量在函数返回后被销毁, 返回它的引用会导致未定义行为:

```

1 // 危险! 返回局部变量的引用
2 int& badFunction() {
3     int local = 42;
4     return local; // local 即将被销毁, 返回的引用是悬空的
5 }
6
7 int main() {
8     int& ref = badFunction(); // ref 是悬空引用
9     std::cout << ref << std::endl; // 未定义行为!
10    return 0;
11 }

```

安全的做法是只返回以下对象的引用: 成员变量、静态变量、通过引用参数传入的对象、或者动态分配的对象。简单的原则是: 确保被引用的对象在引用使用期间始终存在。

1.5.9.6. 引用的底层实现

从使用者的角度, 引用表现得像是变量的别名, 但在编译器的实现中, 引用通常是通过指针来实现的。编译器在底层将引用转换为常量指针 (指针本身不能改变指向), 并自动处理引用操作:

```

1 int x = 42;
2 int& ref = x;
3 ref = 100;
4
5 // 编译器可能将上述代码转换为类似:
6 int x = 42;
7 int* const ref_impl = &x; // 常量指针, 不能改变指向
8 *ref_impl = 100;          // 自动解引用

```

这解释了为什么引用在大多数情况下与指针具有相同的性能特征: 传递引用参数实际上传递的是地址, 返回引用实际上返回的是地址。引用的优势不在于运行效率, 而在于语法的简洁性和使用的安全性。

理解这一点也有助于解释引用的一些限制：引用不能为空（因为它必须绑定到有效对象，就像常量指针必须初始化一样）、引用不能重新绑定（因为底层指针是常量）、引用没有自己的地址（对引用取地址得到的是被引用对象的地址）。

1.5.9.7. 实践中的选择

掌握了引用和指针的区别后，在实际编程中应当根据具体需求选择合适的工具。以下是一些指导原则：

当需要表示“可能不存在”的语义时，使用指针。引用必须绑定到有效对象，无法表示空值；而指针可以为 `nullptr`，明确表示“当前没有指向任何对象”：

```
1 // 指针：可能找不到目标
2 Target* findTarget(const std::vector<Target>& targets) {
3     for (auto& t : targets) {
4         if (t.isValid()) return &t;
5     }
6     return nullptr; // 没有找到
7 }
```

当需要在函数中修改调用者的变量时，使用引用。引用语法更简洁，且调用者无需担心空指针问题：

```
1 void updatePosition(double& x, double& y, double dx, double dy) {
2     x += dx;
3     y += dy;
4 }
```

当传递大型对象且不需要修改时，使用常量引用。这既避免了复制开销，又明确了函数的意图：

```
1 void analyze(const SensorData& data) {
2     // 分析数据，不修改
3 }
```

当需要重新指向不同对象时，使用指针。引用一旦绑定就不能改变，而指针可以：

```
1 void processTargets(std::vector<Target>& targets) {
2     Target* current = nullptr;
3     for (auto& t : targets) {
4         current = &t; // 指针可以改变指向
5         process(*current);
6     }
7 }
```

当与 C 语言库或需要指针语义的 API 交互时，使用指针。许多系统调用和第三方库采用 C 风格接口，必须使用指针。

在 RoboMaster 开发中，引用广泛用于函数参数传递，而指针常见于与底层硬件接口、动态数据结构、以及可选参数的场景。良好的代码会根据语义需求选择最合适的工具，而不是一味地使用某一种方式。

引用是 C++ 相对于 C 语言的重要改进之一，它使得代码更加安全和易读。结合前面学习的指针知识，现在我们已经掌握了 C++ 中两种间接访问数据的机制。接下来，我们将把注意力从单个变量转向数据的组织方式，学习如何使用结构体将相关的数据组合在一起，这将为后续的面对象编程打下基础。

1.5.10. 结构体

到目前为止，我们处理的数据都是单一类型的：一个整数表示速度，一个浮点数表示角度，一个数组存储一系列读数。然而，现实世界中的实体往往具有多个不同类型的属性。以 RoboMaster 比赛中的目标为例，一个目标可能包含位置坐标（三个浮点数）、颜色（一个枚举值）、置信度（一个浮点数）、时间戳（一个整数）等信息。如果用独立的变量来表示这些属性，代码会变得零散且难以管理；如果用数组，又无法存储不同类型的数据。结构体（struct）正是为解决这一问题而设计的——它允许我们将多个相关的数据组合成一个自定义的复合类型。

1.5.10.1. 结构体的定义

结构体使用 `struct` 关键字定义，在花括号内列出各个成员变量（也称为字段）：

```
1 struct Target {
2     double x;
3     double y;
4     double z;
5     int color;
6     double confidence;
7     long long timestamp;
8 };
```

这段代码定义了一个名为 `Target` 的新类型。`Target` 类型的变量将包含六个成员：三个表示空间坐标的 `double`、一个表示颜色的 `int`、一个表示置信度的 `double`、以及一个表示时间戳的 `long long`。注意结构体定义末尾的分号是必需的，遗漏它是常见的语法错误。

定义好结构体类型后，就可以像使用内置类型一样声明该类型的变量：

```
1 Target enemy;           // 声明一个 Target 类型的变量
2 Target detectedTargets[10]; // Target 类型的数组
```

结构体将相关数据封装在一起，使代码的意图更加清晰。与使用六个独立变量相比，一个 `Target` 变量明确表达了“这些数据属于同一个目标”的语义关系。

1.5.10.2. 成员访问与初始化

访问结构体的成员使用点运算符（`.`）。通过点运算符，可以读取或修改结构体变量的各个字段：

```
1 Target enemy;
2 enemy.x = 1000.0;
3 enemy.y = 500.0;
4 enemy.z = 200.0;
5 enemy.color = 1; // 假设 1 表示红色
6 enemy.confidence = 0.95;
7 enemy.timestamp = 1703491200000;
8
9 std::cout << "目标位置: (" << enemy.x << ", " << enemy.y << ", " << enemy.z
10 << ")" << std::endl;
11 std::cout << "置信度: " << enemy.confidence << std::endl;
```

逐个赋值的方式有些繁琐，C++ 提供了多种更简洁的初始化语法。最直接的是使用花括号初始化列表，按照成员声明的顺序提供初始值：

```
1 Target enemy = {1000.0, 500.0, 200.0, 1, 0.95, 1703491200000};
```

C++11 引入了更灵活的初始化方式，可以省略等号，也可以使用指定初始化器（C++20）明确指定各字段的值：

```
1 // C++11 风格
2 Target enemy1{1000.0, 500.0, 200.0, 1, 0.95, 1703491200000};
3
4 // C++20 指定初始化器，更加清晰
5 Target enemy2{.x = 1000.0, .y = 500.0, .z = 200.0,
6               .color = 1, .confidence = 0.95, .timestamp = 1703491200000};
```

如果初始化列表提供的值少于成员数量，剩余成员将被零初始化。如果使用空的花括号 `{}`，所有成员都被零初始化：

```
1 Target partial = {100.0, 200.0}; // x=100, y=200, 其余为 0
2 Target zeroed = {};             // 所有成员为 0
```

结构体也可以在定义时为成员指定默认值，这是 C++11 引入的特性：

```
1 struct Target {
2     double x = 0.0;
3     double y = 0.0;
4     double z = 0.0;
5     int color = 0;
6     double confidence = 0.0;
7     long long timestamp = 0;
8 };
9
10 Target t; // 所有成员都使用默认值
```

默认值使得创建结构体变量时无需显式初始化每个成员，特别适合那些有合理默认状态的类型。

1.5.10.3. 结构体与函数

结构体可以作为函数的参数和返回值，这使得函数能够处理复合数据而无需传递大量独立参数。

将结构体作为参数传递时，默认是值传递——函数接收的是结构体的副本：

```
1 void printTarget(Target t) {
2     std::cout << "位置: (" << t.x << ", " << t.y << ", " << t.z << ")"
3     << std::endl;
4     std::cout << "置信度: " << t.confidence << std::endl;
5 }
6 Target enemy = {1000.0, 500.0, 200.0, 1, 0.95, 1703491200000};
7 printTarget(enemy); // 传递副本
```

然而，对于较大的结构体，值传递会产生复制开销。更高效的做法是使用引用传递。如果函数不需要修改结构体，应当使用常量引用：

```
1 // 常量引用：避免复制，且不会修改原数据
2 void printTarget(const Target& t) {
3     std::cout << "位置: (" << t.x << ", " << t.y << ", " << t.z << ")"
4     << std::endl;
5 }
6 // 引用：允许修改原数据
```

```

7 void updatePosition(Target& t, double dx, double dy, double dz) {
8     t.x += dx;
9     t.y += dy;
10    t.z += dz;
11 }

```

函数也可以返回结构体。这是从函数返回多个相关值的自然方式：

```

1 Target createTarget(double x, double y, double z, int color) {
2     Target t;
3     t.x = x;
4     t.y = y;
5     t.z = z;
6     t.color = color;
7     t.confidence = 1.0;
8     t.timestamp = getCurrentTime();
9     return t;
10 }
11
12 Target newTarget = createTarget(500.0, 300.0, 100.0, 1);

```

现代 C++ 编译器会对返回值进行优化（返回值优化，RVO），通常不会产生额外的复制开销，因此不必担心返回结构体的效率问题。

比较一下使用结构体前后的函数签名，可以明显感受到结构体带来的清晰度提升：

```

1 // 不使用结构体：参数众多，容易混淆顺序
2 void processTarget(double x, double y, double z, int color,
3                   double confidence, long long timestamp);
4
5 // 使用结构体：意图清晰，不易出错
6 void processTarget(const Target& target);

```

1.5.10.4. 结构体与指针

当通过指针访问结构体成员时，需要先解引用指针，再使用点运算符。由于运算符优先级的关系，必须使用括号：

```

1 Target enemy = {1000.0, 500.0, 200.0, 1, 0.95, 0};
2 Target* ptr = &enemy;
3
4 // 方式一：先解引用，再访问成员
5 (*ptr).x = 1500.0;
6
7 // 方式二：使用箭头运算符（更常用）
8 ptr->y = 600.0;

```

箭头运算符 `->` 是 `(*ptr).` 的简写形式，在实际代码中更为常见。它使得通过指针访问成员的语法更加简洁：

```

1 void updateTarget(Target* t) {
2     if (t == nullptr) return;
3
4     t->x += 10.0;
5     t->y += 10.0;
6     t->confidence *= 0.99; // 置信度衰减
7 }

```

在 RoboMaster 开发中，指向结构体的指针常用于动态分配的对象、链表节点、以及需要可选参数的场景。

1.5.10.5. 结构体的嵌套与组合

结构体的成员可以是另一个结构体，这使得我们能够构建层次化的数据结构。例如，可以先定义一个表示三维坐标的结构体，再在目标结构体中使用它：

```
1 struct Point3D {
2     double x;
3     double y;
4     double z;
5 };
6
7 struct Target {
8     Point3D position;
9     Point3D velocity;
10    int color;
11    double confidence;
12    long long timestamp;
13 };
```

访问嵌套成员时，连续使用点运算符：

```
1 Target enemy;
2 enemy.position.x = 1000.0;
3 enemy.position.y = 500.0;
4 enemy.position.z = 200.0;
5 enemy.velocity.x = 10.0;
6 enemy.velocity.y = -5.0;
7 enemy.velocity.z = 0.0;
8
9 std::cout << "位置: (" << enemy.position.x << ", "
10    << enemy.position.y << ", " << enemy.position.z << ")" <<
    std::endl;
```

初始化嵌套结构体时，使用嵌套的花括号：

```
1 Target enemy = {
2     {1000.0, 500.0, 200.0}, // position
3     {10.0, -5.0, 0.0},     // velocity
4     1,                      // color
5     0.95,                  // confidence
6     1703491200000          // timestamp
7 };
```

这种组合方式使得数据结构更加模块化。`Point3D` 可以在多处复用，而不必在每个需要三维坐标的地方重复定义三个浮点数。

结构体还可以包含数组作为成员，这在表示固定大小的数据集时很有用：

```
1 struct RobotState {
2     double jointAngles[6]; // 六个关节的角度
3     double motorCurrents[4]; // 四个电机的电流
4     bool sensorStatus[8]; // 八个传感器的状态
5     long long timestamp;
6 };
```

1.5.10.6. 结构体的比较与赋值

结构体支持整体赋值——将一个结构体变量的所有成员复制到另一个同类型的变量：

```
1 Target t1 = {1000.0, 500.0, 200.0, 1, 0.95, 0};
2 Target t2;
3
4 t2 = t1; // 复制所有成员
5
6 t2.x = 2000.0; // 修改 t2 不影响 t1
7 std::cout << t1.x << std::endl; // 仍然是 1000.0
```

这种赋值是成员逐一复制（memberwise copy），对于包含指针的结构体需要特别注意——复制的是指针值而非指针指向的数据，这可能导致两个结构体共享同一块动态内存（浅拷贝问题，将在后续章节详细讨论）。

然而，结构体默认不支持使用 `==` 或 `!=` 进行比较。如果需要比较两个结构体是否相等，必须逐个比较成员，或者自定义比较运算符（后续章节介绍运算符重载）：

```
1 // 不能直接比较
2 // if (t1 == t2) { } // 错误!
3
4 // 需要逐个成员比较
5 bool targetsEqual(const Target& a, const Target& b) {
6     return a.x == b.x && a.y == b.y && a.z == b.z &&
7           a.color == b.color && a.confidence == b.confidence &&
8           a.timestamp == b.timestamp;
9 }
```

C++20 引入了默认比较运算符的生成机制，可以使用 `= default` 让编译器自动生成比较函数，但这是较新的特性。

1.5.10.7. 结构体的内存布局

理解结构体在内存中的布局有助于优化性能和与底层系统交互。结构体的成员在内存中按声明顺序排列，但编译器可能会在成员之间插入填充字节（padding）以满足对齐要求。

大多数处理器要求特定类型的数据存储在特定对齐边界上。例如，4 字节的 `int` 通常需要存储在 4 的倍数的地址上，8 字节的 `double` 需要存储在 8 的倍数的地址上。为满足这些要求，编译器会在必要时插入填充：

```
1 struct Example1 {
2     char a; // 1 字节
3     int b; // 4 字节，但需要 4 字节对齐
4     char c; // 1 字节
5 };
6
7 // 内存布局可能是：
8 // a (1) + padding (3) + b (4) + c (1) + padding (3) = 12 字节
9 std::cout << sizeof(Example1) << std::endl; // 可能输出 12，而非 6
```

通过调整成员顺序，可以减少填充，节省内存：

```
1 struct Example2 {
2     int b; // 4 字节
3     char a; // 1 字节
4     char c; // 1 字节
5 };
6
```



```

7 // 内存布局: b (4) + a (1) + c (1) + padding (2) = 8 字节
8 std::cout << sizeof(Example2) << std::endl; // 可能输出 8

```

在 RoboMaster 开发中，当结构体用于与下位机通信时，内存布局尤为重要。通信双方必须对数据格式有一致的理解，否则会导致数据解析错误。可以使用编译器特定的指令（如 `#pragma pack`）来控制对齐方式，或者使用定宽类型（如 `int32_t`）确保成员大小一致。

1.5.10.8. 结构体的局限性

结构体为组织数据提供了有力的工具，但它也有一些局限性。

首先，结构体的所有成员默认都是公开的——任何代码都可以直接访问和修改结构体的任何成员。这种开放性在小型程序中不成问题，但在大型项目中可能导致数据被意外修改，难以追踪 bug 的来源：

```

1 Target t;
2 t.confidence = 2.0; // 置信度应该在 0-1 之间，但没有任何检查
3 t.x = -99999.0;     // 可能是无效坐标，但无法阻止

```

其次，结构体只是数据的集合，它不包含操作这些数据的行为。与数据相关的操作必须定义为独立的函数，数据和操作是分离的：

```

1 // 数据
2 struct Target {
3     double x, y, z;
4     double confidence;
5 };
6
7 // 操作数据的函数（与结构体分离）
8 double distanceToTarget(const Target& t) {
9     return std::sqrt(t.x * t.x + t.y * t.y + t.z * t.z);
10 }
11
12 void updateConfidence(Target& t, double factor) {
13     t.confidence *= factor;
14 }

```

这种分离在逻辑上不够自然——距离计算和置信度更新明显是属于“目标”这个概念的操作，却不得不定义在结构体外部。随着项目规模增长，相关的函数可能散落在各处，难以管理。

再者，结构体缺乏对初始化过程的控制。虽然可以为成员指定默认值，但无法在创建对象时执行复杂的初始化逻辑，如参数验证、资源分配、日志记录等。

这些局限性正是 C++ 引入“类”（class）的动机。类在结构体的基础上增加了访问控制（可以将某些成员设为私有）、成员函数（将数据和操作封装在一起）、构造函数和析构函数（控制对象的创建和销毁过程）等特性。实际上，在 C++ 中，`struct` 和 `class` 的唯一区别是默认的访问权限——`struct` 默认公开，`class` 默认私有。结构体可以看作是类的特例。

理解结构体是迈向面向对象编程的重要一步。结构体引入了自定义类型的概念，让我们能够按照问题领域的方式组织数据。下一节将正式介绍类与对象，学习如何将数据和操作封装在一起，构建更加健壮和可维护的代码。

1.5.11. 类与对象

上一节我们学习了结构体，它允许将多个相关数据组合成一个自定义类型。然而，结构体存在一些局限：成员完全暴露，任何代码都可以随意修改；数据和操作分离，相关的函数散落在各

处。这些问题在小型程序中尚可接受，但随着项目规模增长，代码会变得难以维护。C++ 的类（class）正是为解决这些问题而设计的。类将数据和操作封装在一起，并提供访问控制机制，是面向对象编程的核心概念。掌握类的使用，标志着从“面向过程”思维向“面向对象”思维的转变。

1.5.11.1. 从结构体到类

让我们从一个具体的例子开始。假设要表示 RoboMaster 比赛中的电机，使用结构体可能是这样：

```
1 struct Motor {
2     int id;
3     double speed;          // 当前转速 RPM
4     double targetSpeed;    // 目标转速
5     double temperature;    // 温度
6 };
7
8 // 操作电机的函数
9 void setTargetSpeed(Motor& m, double speed) {
10     m.targetSpeed = speed;
11 }
12
13 double getSpeedError(const Motor& m) {
14     return m.targetSpeed - m.speed;
15 }
16
17 bool isOverheated(const Motor& m) {
18     return m.temperature > 80.0;
19 }
```

这种写法的问题在于：`Motor` 只是数据的容器，而操作它的函数是独立存在的。从逻辑上讲，`setTargetSpeed`、`getSpeedError`、`isOverheated` 都是“电机”这个概念的一部分，但代码结构并未体现这种关联。此外，任何代码都可以直接修改 `speed` 或 `temperature`，即使这些值应该只由硬件反馈更新。

使用类可以将数据和操作封装在一起：

```
1 class Motor {
2 public:
3     void setTargetSpeed(double speed) {
4         targetSpeed = speed;
5     }
6
7     double getSpeedError() const {
8         return targetSpeed - speed;
9     }
10
11     bool isOverheated() const {
12         return temperature > 80.0;
13     }
14
15 private:
16     int id;
17     double speed;
18     double targetSpeed;
19     double temperature;
20 };
```

现在, `setTargetSpeed`、`getSpeedError`、`isOverheated` 成为了 `Motor` 类的成员函数, 它们与数据紧密绑定。`private` 关键字则保护了内部数据, 外部代码无法直接访问 `speed` 或 `temperature`。这就是封装——将数据和操作包装在一起, 并控制外部的访问权限。

1.5.11.2. 类的定义

类使用 `class` 关键字定义, 基本结构如下:

```
1 class ClassName {
2 public:
3     // 公开成员: 外部可以访问
4
5 private:
6     // 私有成员: 只有类内部可以访问
7
8 protected:
9     // 受保护成员: 类内部和派生类可以访问
10 };
```

与结构体类似, 类定义末尾需要分号。`public`、`private`、`protected` 是访问说明符 (access specifier), 用于控制成员的可见性。一个类定义中可以有多个访问说明符, 它们的作用范围从当前位置延续到下一个访问说明符或类定义结束。

类的成员包括成员变量 (也称为数据成员或属性) 和成员函数 (也称为方法)。成员变量存储对象的状态, 成员函数定义对象的行为:

```
1 class Target {
2 public:
3     // 成员函数
4     void setPosition(double newX, double newY, double newZ) {
5         x = newX;
6         y = newY;
7         z = newZ;
8     }
9
10    double distanceFromOrigin() const {
11        return std::sqrt(x * x + y * y + z * z);
12    }
13
14    void print() const {
15        std::cout << "Target at (" << x << ", " << y << ", " << z << ")"
16        << std::endl;
17    }
18 private:
19     // 成员变量
20     double x = 0.0;
21     double y = 0.0;
22     double z = 0.0;
23     double confidence = 0.0;
24 };
```

定义好类之后, 可以创建该类的对象 (也称为实例)。对象是类的具体化——类是蓝图, 对象是按照蓝图建造的房子:

```
1 Target t1;           // 创建一个 Target 对象
2 Target t2;           // 创建另一个 Target 对象
3 Target targets[10];   // Target 对象数组
4
```

```

5 t1.setPosition(100.0, 200.0, 50.0);
6 t2.setPosition(300.0, 400.0, 100.0);
7
8 std::cout << "t1 距原点: " << t1.distanceFromOrigin() << std::endl;
9 std::cout << "t2 距原点: " << t2.distanceFromOrigin() << std::endl;

```

每个对象都有自己独立的成员变量副本。t1 的位置与 t2 的位置互不影响，修改一个对象不会改变另一个对象的状态。

1.5.11.3. 成员函数

成员函数是定义在类内部的函数，它可以直接访问类的所有成员（包括私有成员），无需通过参数传递。调用成员函数时使用点运算符，语法与访问成员变量相同：

```

1 Target t;
2 t.setPosition(100.0, 200.0, 50.0); // 调用成员函数
3 double dist = t.distanceFromOrigin();
4 t.print();

```

成员函数可以在类定义内部直接实现（如上面的例子），也可以在类定义中只声明，然后在类外部实现。后者在大型项目中更为常见，它将接口（头文件中的类定义）与实现（源文件中的函数体）分离：

```

1 // Target.h - 头文件
2 class Target {
3 public:
4     void setPosition(double newX, double newY, double newZ);
5     double distanceFromOrigin() const;
6     void print() const;
7
8 private:
9     double x = 0.0;
10    double y = 0.0;
11    double z = 0.0;
12 };
13
14 // Target.cpp - 源文件
15 #include "Target.h"
16 #include <cmath>
17 #include <iostream>
18
19 void Target::setPosition(double newX, double newY, double newZ) {
20     x = newX;
21     y = newY;
22     z = newZ;
23 }
24
25 double Target::distanceFromOrigin() const {
26     return std::sqrt(x * x + y * y + z * z);
27 }
28
29 void Target::print() const {
30     std::cout << "Target at (" << x << ", " << y << ", " << z << ")"
31     << std::endl;
32 }

```

在类外部定义成员函数时，需要使用作用域解析运算符 `::` 指明函数属于哪个类。`Target::setPosition` 表示“Target 类的 setPosition 函数”。

注意某些成员函数声明末尾的 `const` 关键字。这表示该函数不会修改对象的状态——它是一个“只读”操作。常量成员函数可以被常量对象调用，而非常量成员函数则不行：

```
1 void analyzeTarget(const Target& t) {
2     t.print();           // 正确, print() 是 const 成员函数
3     double d = t.distanceFromOrigin(); // 正确, 也是 const
4     // t.setPosition(0, 0, 0); // 错误! setPosition() 不是 const, 不能通过
    const 引用调用
5 }
```

将不修改对象状态的成员函数声明为 `const` 是良好的编程习惯。它既是对函数行为的文档说明，也让编译器帮助检查意外的修改操作。

1.5.11.4. 访问控制

访问控制是类与结构体的关键区别之一。C++ 提供三个访问级别：

`public`（公开）成员可以在任何地方访问。类的公开成员构成了它的接口——外部代码通过公开成员与对象交互。通常，成员函数是公开的，它们定义了对象能够执行的操作。

`private`（私有）成员只能在类的内部访问，外部代码（包括其他类）无法直接访问。成员变量通常是私有的，这样可以保护数据不被随意修改，确保对象始终处于有效状态。

`protected`（受保护）成员介于两者之间：类内部和派生类（子类）可以访问，但其他代码不能。这在继承体系中有用，将在后续章节详细讨论。

```
1 class BankAccount {
2     public:
3         // 公开接口
4         void deposit(double amount) {
5             if (amount > 0) {
6                 balance += amount;
7                 transactionCount++;
8             }
9         }
10
11         bool withdraw(double amount) {
12             if (amount > 0 && amount <= balance) {
13                 balance -= amount;
14                 transactionCount++;
15                 return true;
16             }
17             return false;
18         }
19
20         double getBalance() const {
21             return balance;
22         }
23
24     private:
25         // 私有数据
26         double balance = 0.0;
27         int transactionCount = 0;
28         std::string accountNumber;
29 };
```

在这个例子中，`balance` 是私有的，外部代码不能直接修改它。存款和取款必须通过 `deposit` 和 `withdraw` 函数进行，这些函数可以包含验证逻辑（如检查金额是否为正、余额是否充足）。这种设计确保了账户余额不会出现非法状态（如负数余额）。

```

1 BankAccount account;
2 account.deposit(1000.0); // 正确, 通过公开接口操作
3 account.withdraw(500.0); // 正确
4 std::cout << account.getBalance() << std::endl; // 500.0
5
6 // account.balance = 1000000.0; // 错误! balance 是私有的
7 // account.transactionCount = 0; // 错误!

```

将数据设为私有、通过公开函数访问的模式称为“封装”。封装的好处不仅在于保护数据, 还在于隔离变化。如果将来需要修改 `balance` 的存储方式 (比如从 `double` 改为使用更精确的货币类型), 只需要修改类的内部实现, 外部代码无需任何改动, 因为它们只依赖于公开接口。

有时需要提供对私有成员的读取或写入能力, 这通常通过 `getter` 和 `setter` 函数实现:

```

1 class Motor {
2 public:
3     // Getter: 获取私有成员的值
4     double getSpeed() const { return speed; }
5     double getTemperature() const { return temperature; }
6     int getId() const { return id; }
7
8     // Setter: 设置私有成员的值 (可以包含验证逻辑)
9     void setId(int newId) {
10         if (newId > 0) {
11             id = newId;
12         }
13     }
14
15     void setTargetSpeed(double target) {
16         // 限制目标转速在合理范围内
17         if (target < -maxSpeed) target = -maxSpeed;
18         if (target > maxSpeed) target = maxSpeed;
19         targetSpeed = target;
20     }
21
22 private:
23     int id = 0;
24     double speed = 0.0;
25     double targetSpeed = 0.0;
26     double temperature = 0.0;
27     static constexpr double maxSpeed = 10000.0;
28 };

```

`getter` 和 `setter` 看似增加了代码量, 但它们提供了控制点。`getter` 可以在返回前进行计算或格式化, `setter` 可以验证输入、记录日志、触发其他操作。即使当前不需要这些功能, 保留这种结构也为将来的扩展留出了空间。

1.5.11.5. struct 与 class 的区别

在 C++ 中, `struct` 和 `class` 几乎是相同的, 唯一的区别是默认访问级别。`struct` 的成员默认是 `public`, 而 `class` 的成员默认是 `private`:

```

1 struct Point {
2     double x; // 默认 public
3     double y;
4 };
5
6 class Vector {
7     double x; // 默认 private

```

```
8     double y;
9 };
```

因此，以下两个定义是等价的：

```
1 struct A {
2     private:
3         int data;
4     public:
5         void func();
6 };
7
8 class B {
9     int data; // 默认 private
10    public:
11        void func();
12};
```

由于这种区别只是默认值不同，技术上可以用 `struct` 实现任何 `class` 能实现的功能。然而，惯例上两者有不同的使用场景：`struct` 通常用于简单的数据聚合，成员都是公开的，很少或没有成员函数；`class` 用于需要封装的情况，有私有数据和公开接口。遵循这一惯例可以让代码的意图更加清晰。

```
1 // 使用 struct: 简单数据聚合, 全部公开
2 struct Point3D {
3     double x, y, z;
4 };
5
6 // 使用 class: 有封装需求, 私有数据 + 公开接口
7 class Robot {
8     public:
9         void move(double dx, double dy);
10        void rotate(double angle);
11        Point3D getPosition() const;
12
13    private:
14        Point3D position;
15        double heading;
16        // ...
17};
```

1.5.11.6. this 指针

在成员函数内部，如何访问调用该函数的对象本身？C++ 提供了 `this` 指针来解决这个问题。`this` 是一个隐含的指针，指向调用成员函数的那个对象。

当我们写 `t.print()` 时，实际上编译器将 `&t` 作为隐含参数传递给 `print` 函数。在 `print` 内部，`this` 就指向 `t`。通常情况下，访问成员变量时不需要显式使用 `this` ——直接写 `x` 等价于 `this->x`。但在某些情况下，显式使用 `this` 是必要或有益的。

第一种情况是当成员变量与参数同名时。如果不使用 `this`，参数名会遮蔽成员变量名：

```
1 class Rectangle {
2     public:
3         void setDimensions(double width, double height) {
4             // 参数 width 遮蔽了成员变量 width
5             // 直接写 width = width 没有意义
6             this->width = width;
```

```

7         this->height = height;
8     }
9
10    private:
11        double width;
12        double height;
13 };

```

一种避免命名冲突的做法是给成员变量加前缀（如 `m_width`）或后缀（如 `width_`），这样就不需要 `this`：

```

1 class Rectangle {
2 public:
3     void setDimensions(double width, double height) {
4         m_width = width;
5         m_height = height;
6     }
7
8 private:
9     double m_width;
10    double m_height;
11 };

```

第二种情况是需要返回对象自身的引用，这在实现链式调用时很常见：

```

1 class QueryBuilder {
2 public:
3     QueryBuilder& select(const std::string& columns) {
4         query += "SELECT " + columns + " ";
5         return *this; // 返回对象自身的引用
6     }
7
8     QueryBuilder& from(const std::string& table) {
9         query += "FROM " + table + " ";
10        return *this;
11    }
12
13    QueryBuilder& where(const std::string& condition) {
14        query += "WHERE " + condition + " ";
15        return *this;
16    }
17
18    std::string build() const {
19        return query;
20    }
21
22 private:
23     std::string query;
24 };
25
26 // 链式调用
27 QueryBuilder builder;
28 std::string sql = builder.select("*")
29                     .from("targets")
30                     .where("confidence > 0.9")
31                     .build();

```

每个成员函数返回 `*this`（对象自身的引用），使得可以连续调用多个函数。这种模式在许多库中都有应用，如流操作符 `cout << a << b << c` 就是链式调用的例子。

第三种情况是需要将对象自身传递给其他函数：

```

1  class Observer;
2
3  class Subject {
4  public:
5      void registerObserver(Observer* obs) {
6          observers.push_back(obs);
7      }
8
9      void notifyObservers();
10
11 private:
12     std::vector<Observer*> observers;
13 };
14
15 class Observer {
16 public:
17     void observe(Subject* subject) {
18         subject->registerObserver(this); // 将自身注册到被观察对象
19     }
20
21     virtual void onNotify() = 0;
22 };

```

理解 `this` 指针有助于理解成员函数的工作原理。从底层看，成员函数与普通函数的主要区别就是隐含的 `this` 参数。`obj.func(arg)` 在概念上类似于 `func(&obj, arg)`，只不过 C++ 的语法将这一细节隐藏了起来。

1.5.11.7. 一个完整的例子

让我们用一个完整的例子来综合运用本节学习的概念。以下是一个简化的 PID 控制器类，它在 RoboMaster 开发中非常常见：

```

1  #include <algorithm>
2
3  class PIDController {
4  public:
5      // 设置 PID 参数
6      void setGains(double kp, double ki, double kd) {
7          this->kp = kp;
8          this->ki = ki;
9          this->kd = kd;
10     }
11
12     // 设置输出限幅
13     void setOutputLimits(double minOutput, double maxOutput) {
14         this->minOutput = minOutput;
15         this->maxOutput = maxOutput;
16     }
17
18     // 设置积分限幅（防止积分饱和）
19     void setIntegralLimits(double minIntegral, double maxIntegral) {
20         this->minIntegral = minIntegral;
21         this->maxIntegral = maxIntegral;
22     }
23
24     // 计算控制输出
25     double compute(double setpoint, double measurement, double dt) {
26         double error = setpoint - measurement;
27
28         // 比例项
29         double pTerm = kp * error;

```



```

30
31     // 积分项
32     integral += error * dt;
33     integral = std::clamp(integral, minIntegral, maxIntegral);
34     double iTerm = ki * integral;
35
36     // 微分项
37     double derivative = (error - lastError) / dt;
38     double dTerm = kd * derivative;
39
40     lastError = error;
41
42     // 计算总输出并限幅
43     double output = pTerm + iTerm + dTerm;
44     return std::clamp(output, minOutput, maxOutput);
45 }
46
47 // 重置控制器状态
48 void reset() {
49     integral = 0.0;
50     lastError = 0.0;
51 }
52
53 // Getter 函数
54 double getKp() const { return kp; }
55 double getKi() const { return ki; }
56 double getKd() const { return kd; }
57 double getIntegral() const { return integral; }
58
59 private:
60     // PID 增益
61     double kp = 0.0;
62     double ki = 0.0;
63     double kd = 0.0;
64
65     // 输出限幅
66     double minOutput = -1e9;
67     double maxOutput = 1e9;
68
69     // 积分限幅
70     double minIntegral = -1e9;
71     double maxIntegral = 1e9;
72
73     // 状态变量
74     double integral = 0.0;
75     double lastError = 0.0;
76 };

```

这个类展示了封装的价值：`integral` 和 `lastError` 是控制器的内部状态，外部代码不应该直接修改它们，否则会破坏控制器的正确行为。通过将它们设为私有，我们确保状态只能通过 `compute` 和 `reset` 函数正确地更新。

使用这个类：

```

1 int main() {
2     PIDController speedController;
3     speedController.setGains(1.5, 0.1, 0.05);
4     speedController.setOutputLimits(-1000.0, 1000.0);
5     speedController.setIntegralLimits(-500.0, 500.0);
6
7     double targetSpeed = 3000.0; // 目标转速

```

```

8     double currentSpeed = 0.0;    // 当前转速
9     double dt = 0.001;           // 控制周期 1ms
10
11     for (int i = 0; i < 1000; i++) {
12         double output = speedController.compute(targetSpeed, currentSpeed,
13 dt);
14
15         // 简化的电机模型：输出直接影响速度变化
16         currentSpeed += output * 0.01;
17
18         if (i % 100 == 0) {
19             std::cout << "Step " << i << ": speed = " << currentSpeed
20 << std::endl;
21         }
22     }
23     return 0;
24 }

```

1.5.11.8. 类设计的基本原则

设计良好的类应当遵循一些基本原则。

首先是单一职责原则：一个类应该只负责一件事情。如果一个类承担了过多职责，它会变得庞大而难以维护。当发现类变得过于复杂时，应当考虑将其拆分为多个更小、更专注的类。

其次是信息隐藏：尽可能将成员设为私有，只暴露必要的接口。公开的成员越少，类与外部代码的耦合就越松，将来修改内部实现时需要调整的地方也越少。

再者是接口设计：公开接口应当简洁、直观、难以误用。函数名应当清晰表达其功能，参数顺序应当符合直觉，异常情况应当有合理的处理方式。

最后是保持一致性：类的成员函数在命名风格、参数顺序、返回值约定等方面应当保持一致。一致的风格使得类更容易学习和使用。

类是 C++ 面向对象编程的基石。通过封装，我们将数据和操作绑定在一起，创建出具有清晰边界和明确职责的代码模块。然而，目前我们创建的对象都使用默认的初始化方式，还没有办法在创建对象时执行自定义的初始化逻辑。下一节将介绍构造函数和析构函数，它们控制着对象的“生”与“死”——如何创建和如何销毁。

1.5.12. 构造函数与析构函数

在上一节中，我们学习了如何定义类和创建对象。但仔细观察会发现，我们创建对象时要么依赖成员变量的默认值，要么在创建后逐一调用 setter 函数进行初始化。这种方式不仅繁琐，还容易遗漏某些必要的初始化步骤，导致对象处于不完整或无效的状态。C++ 提供了构造函数 (constructor) 来解决这一问题——它是对象创建时自动调用的特殊函数，负责将对象初始化到有效状态。与之对应的是析构函数 (destructor)，它在对象销毁时自动调用，负责清理对象占用的资源。理解构造函数和析构函数，就是理解对象的生命周期——从诞生到消亡的完整过程。

1.5.12.1. 对象的生命周期

每个对象都有其生命周期：创建、使用、销毁。对象的存储位置决定了它生命周期特征。

栈上的对象（局部变量）在进入作用域时创建，离开作用域时销毁。它们的生命周期由编译器自动管理：

```

1 void processTarget() {

```

```

2     Target t; // t 在这里被创建
3     t.setPosition(100, 200, 50);
4     // 使用 t...
5 } // t 在这里被销毁, 离开作用域
6
7 int main() {
8     processTarget();
9     // 此时 t 已经不存在了
10    return 0;
11 }

```

堆上的对象（通过 `new` 创建）在调用 `new` 时创建，调用 `delete` 时销毁。程序员必须手动管理它们的生命周期：

```

1 void dynamicExample() {
2     Target* ptr = new Target(); // 在堆上创建对象
3     ptr->setPosition(100, 200, 50);
4     // 使用 ptr...
5     delete ptr; // 手动销毁对象
6 }

```

全局对象和静态对象在程序启动时创建，在程序结束时销毁。

无论对象在哪里创建，构造函数都会在创建时被调用，析构函数都会在销毁时被调用。这种自动调用机制是 C++ 资源管理的基础。

1.5.12.2. 默认构造函数

构造函数是一种特殊的成员函数，它的名称与类名相同，没有返回类型（连 `void` 都没有）。最简单的构造函数是默认构造函数，它不接受任何参数：

```

1 class Motor {
2 public:
3     Motor() {
4         std::cout << "Motor 对象被创建" << std::endl;
5         id = 0;
6         speed = 0.0;
7         temperature = 25.0;
8     }
9
10 private:
11     int id;
12     double speed;
13     double temperature;
14 };
15
16 int main() {
17     Motor m; // 自动调用默认构造函数
18     return 0;
19 }

```

当我们写 `Motor m;` 时，编译器自动调用 `Motor()` 构造函数。构造函数中的代码被执行，成员变量被初始化为指定的值。构造函数确保了对象一旦创建就处于有效状态，使用者无需额外的初始化步骤。

如果类没有定义任何构造函数，编译器会自动生成一个默认构造函数。这个编译器生成的默认构造函数不做任何事情（对于没有默认值的成员变量，它们将包含未定义的值）。但一旦我们定义了任何构造函数，编译器就不再自动生成默认构造函数：

```

1 class Point {
2 public:
3     Point(double x, double y) { // 只定义了带参数的构造函数
4         this->x = x;
5         this->y = y;
6     }
7
8 private:
9     double x, y;
10 };
11
12 int main() {
13     Point p1(1.0, 2.0); // 正确, 调用 Point(double, double)
14     // Point p2;        // 错误! 没有默认构造函数
15     return 0;
16 }

```

如果希望在定义了其他构造函数的同时保留默认构造函数, 可以显式声明它:

```

1 class Point {
2 public:
3     Point() = default; // 显式要求编译器生成默认构造函数
4     Point(double x, double y) : x(x), y(y) {}
5
6 private:
7     double x = 0.0;
8     double y = 0.0;
9 };

```

`= default` 是 C++11 引入的语法, 它告诉编译器生成该函数的默认版本。

1.5.12.3. 带参数的构造函数

更常见的情况是构造函数需要接受参数, 以便根据不同的输入创建不同状态的对象:

```

1 class Target {
2 public:
3     Target(double x, double y, double z, int color) {
4         this->x = x;
5         this->y = y;
6         this->z = z;
7         this->color = color;
8         this->confidence = 1.0;
9         this->timestamp = getCurrentTime();
10    }
11
12    void print() const {
13        std::cout << "Target at (" << x << ", " << y << ", " << z << ")"
14        << std::endl;
15    }
16 private:
17     double x, y, z;
18     int color;
19     double confidence;
20     long long timestamp;
21
22     long long getCurrentTime() {
23         // 返回当前时间戳
24         return 0; // 简化示例
25    }

```

```

26 };
27
28 int main() {
29     Target t1(100.0, 200.0, 50.0, 1); // 直接初始化
30     Target t2 = Target(300.0, 400.0, 100.0, 2); // 显式调用构造函数
31     Target t3{500.0, 600.0, 150.0, 1}; // C++11 花括号初始化
32
33     t1.print();
34     t2.print();
35     t3.print();
36
37     return 0;
38 }

```

带参数的构造函数使得创建对象和初始化可以在一步完成，避免了“创建后再设置”的繁琐模式。构造函数还可以包含验证逻辑，确保对象以合法状态创建：

```

1 class Motor {
2 public:
3     Motor(int id, double maxSpeed) {
4         if (id <= 0) {
5             throw std::invalid_argument("电机 ID 必须为正数");
6         }
7         if (maxSpeed <= 0) {
8             throw std::invalid_argument("最大转速必须为正数");
9         }
10
11         this->id = id;
12         this->maxSpeed = maxSpeed;
13         this->currentSpeed = 0.0;
14     }
15
16 private:
17     int id;
18     double maxSpeed;
19     double currentSpeed;
20 };

```

1.5.12.4. 构造函数重载

与普通函数一样，构造函数也可以重载——定义多个同名但参数列表不同的构造函数，为对象创建提供多种方式：

```

1 class PIDController {
2 public:
3     // 默认构造函数：使用默认参数
4     PIDController() {
5         kp = 1.0;
6         ki = 0.0;
7         kd = 0.0;
8         reset();
9     }
10
11     // 只设置 P 参数
12     PIDController(double kp) {
13         this->kp = kp;
14         this->ki = 0.0;
15         this->kd = 0.0;
16         reset();
17     }

```

```

18
19 // 设置 PID 三个参数
20 PIDController(double kp, double ki, double kd) {
21     this->kp = kp;
22     this->ki = ki;
23     this->kd = kd;
24     reset();
25 }
26
27 void reset() {
28     integral = 0.0;
29     lastError = 0.0;
30 }
31
32 private:
33     double kp, ki, kd;
34     double integral;
35     double lastError;
36 };
37
38 int main() {
39     PIDController pid1;                // 调用默认构造函数
40     PIDController pid2(2.0);           // 调用 PIDController(double)
41     PIDController pid3(1.5, 0.1, 0.05); // 调用 PIDController(double,
double, double)
42
43     return 0;
44 }

```

编译器根据调用时提供的参数自动选择匹配的构造函数版本。

另一种实现类似效果的方法是使用默认参数：

```

1 class PIDController {
2 public:
3     PIDController(double kp = 1.0, double ki = 0.0, double kd = 0.0) {
4         this->kp = kp;
5         this->ki = ki;
6         this->kd = kd;
7         integral = 0.0;
8         lastError = 0.0;
9     }
10
11 private:
12     double kp, ki, kd;
13     double integral, lastError;
14 };
15
16 int main() {
17     PIDController pid1;                // kp=1.0, ki=0.0, kd=0.0
18     PIDController pid2(2.0);           // kp=2.0, ki=0.0, kd=0.0
19     PIDController pid3(1.5, 0.1);      // kp=1.5, ki=0.1, kd=0.0
20     PIDController pid4(1.5, 0.1, 0.05); // kp=1.5, ki=0.1, kd=0.05
21
22     return 0;
23 }

```

默认参数使得一个构造函数可以处理多种调用方式，减少了代码重复。

1.5.12.5. 成员初始化列表

在前面的例子中，我们在构造函数体内对成员变量进行赋值。C++ 提供了一种更高效的初始化方式——成员初始化列表（member initializer list）。它出现在构造函数参数列表之后、函数体之前，以冒号开头：

```
1 class Target {
2 public:
3     Target(double x, double y, double z, int color)
4         : x(x), y(y), z(z), color(color), confidence(1.0), timestamp(0)
5     {
6         // 构造函数体，可以为空
7     }
8
9 private:
10     double x, y, z;
11     int color;
12     double confidence;
13     long long timestamp;
14 };
```

初始化列表中，每个成员的初始化形式为 `成员名(初始值)`，多个成员之间用逗号分隔。这种语法与函数体内赋值有本质区别：初始化列表是真正的初始化，而函数体内是赋值。

对于基本类型，两者的效果通常相同。但对于某些情况，必须使用初始化列表：

常量成员必须在初始化列表中初始化，因为常量一旦创建就不能被赋值：

```
1 class Config {
2 public:
3     Config(int maxConnections)
4         : MAX_CONNECTIONS(maxConnections) // 必须在初始化列表中
5     {
6         // MAX_CONNECTIONS = maxConnections; // 错误！不能给 const 成员赋值
7     }
8
9 private:
10     const int MAX_CONNECTIONS;
11 };
```

引用成员也必须在初始化列表中初始化，因为引用必须在创建时绑定：

```
1 class Observer {
2 public:
3     Observer(Subject& subject)
4         : subject(subject) // 引用必须在初始化列表中绑定
5     {
6     }
7
8 private:
9     Subject& subject;
10 };
```

没有默认构造函数的成员对象必须在初始化列表中初始化：

```
1 class Engine {
2 public:
3     Engine(int power) : power(power) {} // 没有默认构造函数
4 private:
5     int power;
6 };
```

```

7
8 class Car {
9 public:
10     Car(int enginePower)
11         : engine(enginePower) // 必须在初始化列表中初始化 engine
12     {
13     }
14
15 private:
16     Engine engine;
17 };

```

即使不是必须使用初始化列表的情况，使用它也是推荐的做法。对于类类型的成员，初始化列表直接调用构造函数进行初始化，而函数体内赋值会先调用默认构造函数，再调用赋值运算符——多了一步不必要的操作：

```

1 class Example {
2 public:
3     // 低效：先默认构造 name，再赋值
4     Example(const std::string& n) {
5         name = n; // 调用 string 的赋值运算符
6     }
7
8     // 高效：直接用 n 构造 name
9     Example(const std::string& n)
10         : name(n) // 调用 string 的拷贝构造函数
11     {
12     }
13
14 private:
15     std::string name;
16 };

```

初始化列表中成员的初始化顺序由它们在类中的声明顺序决定，而非初始化列表中的书写顺序。为避免混淆，建议初始化列表的顺序与成员声明顺序保持一致：

```

1 class Example {
2 public:
3     Example(int val)
4         : a(val), b(a + 1) // 正确：a 先于 b 声明，所以 a 先初始化
5     {
6     }
7
8 private:
9     int a; // 先声明
10    int b; // 后声明
11 };

```

1.5.12.6. 委托构造函数

C++11 引入了委托构造函数 (delegating constructor)，允许一个构造函数调用同一类的另一个构造函数，减少代码重复：

```

1 class Motor {
2 public:
3     // 主构造函数
4     Motor(int id, double maxSpeed, double gearRatio)
5         : id(id), maxSpeed(maxSpeed), gearRatio(gearRatio),
6           currentSpeed(0.0), temperature(25.0)

```



```

7     {
8         validateParameters();
9     }
10
11     // 委托构造函数: 使用默认齿轮比
12     Motor(int id, double maxSpeed)
13         : Motor(id, maxSpeed, 1.0) // 委托给主构造函数
14     {
15     }
16
17     // 委托构造函数: 使用默认参数
18     Motor(int id)
19         : Motor(id, 10000.0, 1.0) // 委托给主构造函数
20     {
21     }
22
23 private:
24     void validateParameters() {
25         if (id <= 0 || maxSpeed <= 0 || gearRatio <= 0) {
26             throw std::invalid_argument("无效参数");
27         }
28     }
29
30     int id;
31     double maxSpeed;
32     double gearRatio;
33     double currentSpeed;
34     double temperature;
35 };

```

委托构造函数在初始化列表中调用目标构造函数, 调用形式与普通成员初始化类似。使用委托构造函数后, 初始化逻辑集中在一个主构造函数中, 其他构造函数只需要提供不同的参数组合, 代码更加简洁且易于维护。

1.5.12.7. 析构函数

析构函数是与构造函数对应的特殊成员函数, 它在对象销毁时自动调用。析构函数的名称是类名前加波浪号 `~`, 没有返回类型, 也不接受任何参数:

```

1 class Logger {
2 public:
3     Logger(const std::string& filename) {
4         file.open(filename, std::ios::app);
5         std::cout << "Logger 创建, 打开文件: " << filename << std::endl;
6     }
7
8     ~Logger() {
9         if (file.is_open()) {
10             file.close();
11         }
12         std::cout << "Logger 销毁, 关闭文件" << std::endl;
13     }
14
15     void log(const std::string& message) {
16         file << message << std::endl;
17     }
18
19 private:
20     std::ofstream file;
21 };
22

```

```

23 void processData() {
24     Logger logger("app.log"); // 构造函数被调用
25     logger.log("开始处理");
26     logger.log("处理完成");
27 } // logger 离开作用域, 析构函数被调用
28
29 int main() {
30     std::cout << "程序开始" << std::endl;
31     processData();
32     std::cout << "程序结束" << std::endl;
33     return 0;
34 }

```

输出将是:

```

1  程序开始
2  Logger 创建, 打开文件: app.log
3  Logger 销毁, 关闭文件
4  程序结束

```

析构函数的主要职责是释放对象持有的资源。这些资源可能包括:

动态分配的内存 (通过 `new` 分配的) 需要在析构函数中 `delete` :

```

1  class DynamicArray {
2  public:
3      DynamicArray(size_t size)
4          : size(size), data(new int[size])
5      {
6          std::fill(data, data + size, 0);
7      }
8
9      ~DynamicArray() {
10         delete[] data; // 释放动态分配的内存
11     }
12
13     int& operator[](size_t index) {
14         return data[index];
15     }
16
17 private:
18     size_t size;
19     int* data;
20 };

```

打开的文件需要关闭, 网络连接需要断开, 锁需要释放——所有这些清理工作都应该在析构函数中完成。

与构造函数不同, 析构函数不能重载, 每个类只能有一个析构函数。如果没有显式定义析构函数, 编译器会自动生成一个, 它会调用每个成员对象的析构函数, 但不会做其他事情。对于持有原始指针等需要手动释放的资源的类, 必须显式定义析构函数。

1.5.12.8. RAII 思想

RAII (Resource Acquisition Is Initialization, 资源获取即初始化) 是 C++ 中最重要的编程惯用法之一。它的核心思想是: 将资源的生命周期与对象的生命周期绑定——在构造函数中获取资源, 在析构函数中释放资源。

这个思想解决了资源管理的根本问题: 确保资源总是被正确释放, 即使在异常发生时也是如此。

考虑一个不使用 RAII 的例子：

```
1 void processFile(const std::string& filename) {
2     FILE* file = fopen(filename.c_str(), "r");
3     if (!file) return;
4
5     // 处理文件...
6     if (someError) {
7         fclose(file); // 必须记得关闭
8         return;
9     }
10
11    // 更多处理...
12    if (anotherError) {
13        fclose(file); // 每个退出点都要关闭
14        return;
15    }
16
17    fclose(file); // 正常退出也要关闭
18 }
```

每个可能的退出点都需要记得关闭文件，容易遗漏。如果中间的代码抛出异常，文件更是无法被关闭。

使用 RAII 重写：

```
1 class FileHandle {
2 public:
3     FileHandle(const std::string& filename, const char* mode)
4         : file(fopen(filename.c_str(), mode))
5     {
6         if (!file) {
7             throw std::runtime_error("无法打开文件");
8         }
9     }
10
11    ~FileHandle() {
12        if (file) {
13            fclose(file);
14        }
15    }
16
17    FILE* get() const { return file; }
18
19 private:
20     FILE* file;
21 };
22
23 void processFile(const std::string& filename) {
24     FileHandle handle(filename, "r"); // 构造函数打开文件
25
26     // 处理文件...
27     if (someError) {
28         return; // 不需要手动关闭，析构函数会处理
29     }
30
31    // 更多处理...
32    if (anotherError) {
33        return; // 同样，析构函数会处理
34    }
35
36    // 正常退出，析构函数自动关闭文件
```

无论函数如何退出——正常返回、提前返回、还是抛出异常——`FileHandle` 的析构函数都会被调用, 文件都会被正确关闭。这就是 RAII 的威力: 它将资源管理变成了自动的、异常安全的。

RAII 的应用无处不在:

```

1 // 标准库中的 RAII 示例
2 void examples() {
3     // std::string 管理字符数组的内存
4     std::string text = "Hello"; // 构造时分配内存
5     // 离开作用域时自动释放
6
7     // std::vector 管理动态数组
8     std::vector<int> numbers(100); // 构造时分配
9     // 离开作用域时自动释放
10
11    // std::fstream 管理文件
12    std::ofstream file("output.txt"); // 构造时打开
13    file << "Hello";
14    // 离开作用域时自动关闭
15
16    // std::lock_guard 管理互斥锁
17    std::mutex mtx;
18    {
19        std::lock_guard<std::mutex> lock(mtx); // 构造时加锁
20        // 临界区代码
21    } // 离开作用域时自动解锁
22
23    // std::unique_ptr 管理动态分配的对象
24    std::unique_ptr<Motor> motor(new Motor(1, 10000.0)); // 构造时获得所有权
25    // 离开作用域时自动 delete
26 }
```

在 RoboMaster 开发中, RAII 思想应用于串口连接、相机资源、线程管理等各个方面:

```

1 class SerialPort {
2 public:
3     SerialPort(const std::string& device, int baudrate)
4         : fd(open(device.c_str(), O_RDWR | O_NOCTTY))
5     {
6         if (fd < 0) {
7             throw std::runtime_error("无法打开串口");
8         }
9         configure(baudrate);
10        std::cout << "串口已打开: " << device << std::endl;
11    }
12
13    ~SerialPort() {
14        if (fd >= 0) {
15            close(fd);
16            std::cout << "串口已关闭" << std::endl;
17        }
18    }
19
20    ssize_t write(const void* data, size_t len) {
21        return ::write(fd, data, len);
22    }
23
24    ssize_t read(void* buffer, size_t len) {
25        return ::read(fd, buffer, len);
26    }
27 }
```

```

26     }
27
28 private:
29     void configure(int baudrate);
30     int fd;
31 };
32
33 void communicate() {
34     SerialPort port("/dev/ttyUSB0", 115200);
35
36     uint8_t command[] = {0xA5, 0x01, 0x02, 0x03};
37     port.write(command, sizeof(command));
38
39     uint8_t response[256];
40     port.read(response, sizeof(response));
41
42     // 不需要手动关闭串口
43 }

```

1.5.12.9. 拷贝控制预览

当我们定义了管理资源的类时，还需要考虑对象被拷贝时会发生什么。默认情况下，C++ 会逐成员复制对象，这对于持有指针的类可能导致问题：

```

1  class DynamicArray {
2  public:
3      DynamicArray(size_t size)
4          : size(size), data(new int[size]) {}
5
6      ~DynamicArray() {
7          delete[] data;
8      }
9
10 private:
11     size_t size;
12     int* data;
13 };
14
15 void problematic() {
16     DynamicArray a(10);
17     DynamicArray b = a; // 默认拷贝: b.data 指向与 a.data 相同的内存
18
19 } // 问题! a 和 b 的析构函数都会 delete 同一块内存

```

这种默认的浅拷贝导致两个对象共享同一块内存，当它们都被销毁时，会重复释放同一块内存，这是严重的错误。

解决方案是定义拷贝构造函数和拷贝赋值运算符来实现深拷贝，或者禁止拷贝。这些内容将在下一节“拷贝控制”中详细讨论。

1.5.12.10. 构造与析构的顺序

理解构造和析构的调用顺序对于正确使用类层次结构很重要。

对于包含成员对象的类，成员按照声明顺序构造，按相反顺序析构：

```

1  class Component {
2  public:
3      Component(int id) : id(id) {
4          std::cout << "Component " << id << " 构造" << std::endl;

```

```

5     }
6     ~Component() {
7         std::cout << "Component " << id << " 析构" << std::endl;
8     }
9     private:
10        int id;
11    };
12
13    class System {
14    public:
15        System() : c1(1), c2(2), c3(3) {
16            std::cout << "System 构造" << std::endl;
17        }
18        ~System() {
19            std::cout << "System 析构" << std::endl;
20        }
21    private:
22        Component c1;
23        Component c2;
24        Component c3;
25    };
26
27    int main() {
28        System sys;
29        return 0;
30    }

```

输出:

```

1 Component 1 构造
2 Component 2 构造
3 Component 3 构造
4 System 构造
5 System 析构
6 Component 3 析构
7 Component 2 析构
8 Component 1 析构

```

成员对象在包含它们的对象之前构造，在之后析构。这个顺序确保了在 `System` 的构造函数和析构函数中，所有成员对象都处于有效状态。

对于数组中的对象，按索引顺序构造，按相反顺序析构：

```

1 void arrayExample() {
2     Motor motors[3] = {Motor(1), Motor(2), Motor(3)};
3     // Motor(1) 先构造, Motor(3) 最后构造
4 } // Motor(3) 先析构, Motor(1) 最后析构

```

1.5.12.11. 实践建议

编写构造函数和析构函数时，以下原则值得遵循：

优先使用成员初始化列表。它不仅在某些情况下是必需的，而且通常更高效。养成使用初始化列表的习惯，可以避免很多潜在问题。

确保构造函数将对象初始化到有效状态。对象一旦创建，就应该可以安全使用。如果初始化可能失败，考虑抛出异常。

如果类管理资源，记得定义析构函数释放资源。同时考虑是否需要自定义拷贝行为——这是“三五法则”的一部分，将在下一节详细讨论。

拥抱 RAII。将资源管理封装在类中，让构造函数和析构函数自动处理资源的获取和释放。这不仅使代码更简洁，也使其更加异常安全。

避免在构造函数和析构函数中调用虚函数。这是一个微妙的陷阱，将在多态相关章节中解释。

构造函数和析构函数是类的生命周期管理机制。通过它们，我们可以精确控制对象从创建到销毁的整个过程，确保资源被正确获取和释放。然而，当对象被拷贝或赋值时，还需要额外的控制。下一节将深入探讨拷贝控制——如何正确处理对象的复制和赋值操作。

1.5.13. 拷贝控制

上一节末尾，我们遇到了一个棘手的问题：当一个管理动态内存的对象被拷贝时，默认的逐成员复制会导致两个对象共享同一块内存，最终在析构时重复释放，引发严重错误。这揭示了一个更深层的问题——对象的拷贝行为并非总是简单的内存复制。C++ 允许我们自定义对象在拷贝和赋值时的行为，这就是拷贝控制（copy control）。掌握拷贝控制是编写健壮的资源管理类的关键，也是理解 C++ 对象语义的重要一步。

1.5.13.1. 拷贝的发生时机

在深入拷贝控制的细节之前，先来了解拷贝何时发生。C++ 中对象拷贝的情况比想象中更加普遍。

最直观的是用一个对象初始化另一个对象：

```
1 Motor m1(1, 10000.0);
2 Motor m2 = m1;           // 拷贝初始化
3 Motor m3(m1);            // 直接初始化，也是拷贝
4 Motor m4{m1};            // 列表初始化，也是拷贝
```

函数参数按值传递时会发生拷贝：

```
1 void process(Motor m) { // m 是实参的拷贝
2     // ...
3 }
4
5 Motor motor(1, 10000.0);
6 process(motor); // 拷贝 motor 到参数 m
```

函数按值返回时也会发生拷贝（尽管编译器常常会优化掉）：

```
1 Motor createMotor() {
2     Motor m(1, 10000.0);
3     return m; // 返回时可能拷贝
4 }
5
6 Motor result = createMotor(); // 接收返回值时可能拷贝
```

将对象放入容器、从容器中取出、容器扩容时重新分配空间，这些操作也可能涉及拷贝。

理解拷贝的普遍性，就能理解为什么正确实现拷贝控制如此重要——错误的拷贝行为会在程序中各处引发问题。

1.5.13.2. 拷贝构造函数

拷贝构造函数是一种特殊的构造函数，它使用同类型的另一个对象来初始化新对象。拷贝构造函数的第一个参数必须是本类类型的引用（通常是常量引用），且不应有其他非默认参数：

```

1 class Motor {
2 public:
3     // 普通构造函数
4     Motor(int id, double maxSpeed)
5         : id(id), maxSpeed(maxSpeed), currentSpeed(0.0)
6     {
7         std::cout << "构造 Motor " << id << std::endl;
8     }
9
10    // 拷贝构造函数
11    Motor(const Motor& other)
12        : id(other.id), maxSpeed(other.maxSpeed),
13        currentSpeed(other.currentSpeed)
14    {
15        std::cout << "拷贝构造 Motor " << id << std::endl;
16    }
17 private:
18     int id;
19     double maxSpeed;
20     double currentSpeed;
21 };
22
23 int main() {
24     Motor m1(1, 10000.0); // 调用普通构造函数
25     Motor m2 = m1;        // 调用拷贝构造函数
26     Motor m3(m1);         // 调用拷贝构造函数
27     return 0;
28 }

```

拷贝构造函数的参数必须是引用，这一点至关重要。如果参数是值传递，那么传递参数时就需要拷贝，而拷贝又需要调用拷贝构造函数，这会导致无限递归。

如果我们没有定义拷贝构造函数，编译器会自动生成一个。编译器生成的拷贝构造函数执行逐成员拷贝——对每个成员调用其拷贝构造函数（对于基本类型，就是简单的值复制）。对于只包含基本类型和标准库类型的简单类，编译器生成的版本通常足够：

```

1 class Point {
2 public:
3     Point(double x, double y) : x(x), y(y) {}
4     // 编译器自动生成的拷贝构造函数足以正确工作
5
6 private:
7     double x, y;
8 };
9
10 class Target {
11 public:
12     Target(const std::string& name, Point pos)
13         : name(name), position(pos) {}
14     // 编译器生成的版本会正确拷贝 string 和 Point
15
16 private:
17     std::string name;
18     Point position;
19 };

```

`std::string` 和 `Point` 都有正确的拷贝行为，因此包含它们的 `Target` 也能正确拷贝。

1.5.13.3. 浅拷贝的问题

问题出现在类包含指向动态分配内存的指针时。编译器生成的拷贝构造函数只复制指针的值（即地址），而不复制指针指向的数据。这称为浅拷贝（shallow copy）：

```
1 class DynamicArray {
2 public:
3     DynamicArray(size_t size)
4         : size(size), data(new int[size])
5     {
6         std::fill(data, data + size, 0);
7         std::cout << "构造, 分配内存: " << data << std::endl;
8     }
9
10    ~DynamicArray() {
11        std::cout << "析构, 释放内存: " << data << std::endl;
12        delete[] data;
13    }
14
15    int& operator[](size_t index) { return data[index]; }
16    size_t getSize() const { return size; }
17
18 private:
19     size_t size;
20     int* data; // 指向动态分配的内存
21 };
22
23 int main() {
24     DynamicArray a(5);
25     a[0] = 100;
26
27     DynamicArray b = a; // 浅拷贝: b.data 与 a.data 指向同一块内存
28
29     std::cout << "a[0] = " << a[0] << std::endl; // 100
30     std::cout << "b[0] = " << b[0] << std::endl; // 100
31
32     b[0] = 200; // 修改 b 也会影响 a
33     std::cout << "a[0] = " << a[0] << std::endl; // 200!
34
35     return 0;
36 } // 灾难: a 和 b 的析构函数都会 delete 同一块内存
```

运行这段代码，程序可能崩溃，或者产生不可预测的行为。这是因为：

1. `b = a` 时，`b.data` 被赋值为 `a.data` 的值，两个指针指向同一块内存
2. 修改 `b[0]` 实际上修改的是 `a` 和 `b` 共享的那块内存
3. 当 `b` 被销毁时，它的析构函数释放了那块内存
4. 当 `a` 被销毁时，它的析构函数试图释放已经被释放的内存——这是未定义行为

浅拷贝的问题本质上是所有权的混乱：两个对象都认为自己拥有同一块内存，都试图管理它的生命周期。

1.5.13.4. 深拷贝

解决方案是实现深拷贝（deep copy）：不仅复制指针，还复制指针指向的数据，使每个对象拥有自己独立的资源副本：

```
1 class DynamicArray {
2 public:
3     DynamicArray(size_t size)
```

```

4         : size(size), data(new int[size])
5     {
6         std::fill(data, data + size, 0);
7         std::cout << "构造, 分配内存: " << data << std::endl;
8     }
9
10    // 深拷贝的拷贝构造函数
11    DynamicArray(const DynamicArray& other)
12        : size(other.size), data(new int[other.size]) // 分配新内存
13    {
14        std::copy(other.data, other.data + size, data); // 复制数据
15        std::cout << "拷贝构造, 分配新内存: " << data << std::endl;
16    }
17
18    ~DynamicArray() {
19        std::cout << "析构, 释放内存: " << data << std::endl;
20        delete[] data;
21    }
22
23    int& operator[](size_t index) { return data[index]; }
24    size_t getSize() const { return size; }
25
26    private:
27        size_t size;
28        int* data;
29    };
30
31    int main() {
32        DynamicArray a(5);
33        a[0] = 100;
34
35        DynamicArray b = a; // 深拷贝: b 拥有自己的内存
36
37        std::cout << "a[0] = " << a[0] << std::endl; // 100
38        std::cout << "b[0] = " << b[0] << std::endl; // 100
39
40        b[0] = 200; // 修改 b 不影响 a
41        std::cout << "a[0] = " << a[0] << std::endl; // 100, 不变
42        std::cout << "b[0] = " << b[0] << std::endl; // 200
43
44        return 0;
45    } // 正常: a 和 b 各自释放自己的内存

```

现在每个 `DynamicArray` 对象都拥有自己独立的内存块, 修改一个不会影响另一个, 销毁时也各自释放自己的资源, 不会产生冲突。

1.5.13.5. 拷贝赋值运算符

拷贝构造函数处理的是用已有对象初始化新对象的情况。但还有另一种拷贝场景: 将一个已存在的对象赋值给另一个已存在的对象。这由拷贝赋值运算符 (copy assignment operator) 处理:

```

1 DynamicArray a(5);
2 DynamicArray b(10);
3
4 b = a; // 赋值, 不是初始化! b 已经存在

```

拷贝赋值运算符是 `operator=` 的重载, 它接受同类型的常量引用作为参数, 返回对象自身的引用 (以支持链式赋值):

```

1 class DynamicArray {
2 public:
3     // ... 构造函数和拷贝构造函数 ...
4
5     // 拷贝赋值运算符
6     DynamicArray& operator=(const DynamicArray& other) {
7         std::cout << "拷贝赋值" << std::endl;
8
9         if (this == &other) { // 自赋值检查
10             return *this;
11         }
12
13         // 释放当前资源
14         delete[] data;
15
16         // 分配新资源并复制
17         size = other.size;
18         data = new int[size];
19         std::copy(other.data, other.data + size, data);
20
21         return *this;
22     }
23
24     // ... 其他成员 ...
25 };

```

拷贝赋值运算符与拷贝构造函数的关键区别在于：赋值时，目标对象已经存在，可能已经持有资源。因此，赋值运算符必须先释放旧资源，再获取新资源。

自赋值检查（`if (this == &other)`）看似多余，但在某些情况下是必要的。考虑 `a = a` 这种情况：如果没有检查，我们会先 `delete[] data`，然后试图从已释放的 `other.data`（实际上就是刚释放的 `data`）复制数据——这显然是错误的。

实际上，上面的实现还有一个问题：如果 `new int[size]` 抛出异常，对象会处于不一致状态（`data` 已被删除但尚未重新分配）。更安全的实现是先分配新资源，再释放旧资源：

```

1 DynamicArray& operator=(const DynamicArray& other) {
2     if (this == &other) {
3         return *this;
4     }
5
6     // 先分配新资源
7     int* newData = new int[other.size];
8     std::copy(other.data, other.data + other.size, newData);
9
10    // 成功后再释放旧资源
11    delete[] data;
12    data = newData;
13    size = other.size;
14
15    return *this;
16 }

```

如果 `new` 失败并抛出异常，旧资源仍然完好，对象状态不变。这种技术称为“先复制后交换”（copy-and-swap），可以进一步改进。

1.5.13.6. copy-and-swap 惯用法

copy-and-swap 是实现拷贝赋值运算符的优雅且异常安全的方法。它结合使用拷贝构造函数和 `swap` 函数：

```

1  class DynamicArray {
2  public:
3      DynamicArray(size_t size)
4          : size(size), data(new int[size])
5      {
6          std::fill(data, data + size, 0);
7      }
8
9      DynamicArray(const DynamicArray& other)
10         : size(other.size), data(new int[other.size])
11     {
12         std::copy(other.data, other.data + size, data);
13     }
14
15     ~DynamicArray() {
16         delete[] data;
17     }
18
19     // swap 函数: 交换两个对象的内部状态
20     friend void swap(DynamicArray& first, DynamicArray& second) noexcept {
21         using std::swap;
22         swap(first.size, second.size);
23         swap(first.data, second.data);
24     }
25
26     // copy-and-swap 实现的赋值运算符
27     DynamicArray& operator=(DynamicArray other) { // 注意: 按值传递
28         swap(*this, other);
29         return *this;
30     }
31
32     int& operator[](size_t index) { return data[index]; }
33     size_t getSize() const { return size; }
34
35 private:
36     size_t size;
37     int* data;
38 };

```

这个实现的精妙之处在于:

参数 `other` 是按值传递的, 这意味着在进入函数时, 拷贝构造函数已经被调用, `other` 是实参的一个副本。如果拷贝构造失败 (如内存分配异常), 函数根本不会执行, 原对象不受影响。

然后, 我们将当前对象与这个副本交换内部状态。交换之后, `*this` 拥有了副本的资源 (即新值), 而 `other` 拥有了原来的资源 (即旧值)。

当函数返回时, `other` 被销毁, 它的析构函数释放旧资源。

这种方法自动处理了自赋值 (`a = a` 时, 只是与自己的副本交换, 没有问题), 代码简洁, 且提供强异常安全保证。

1.5.13.7. 三五法则

C++ 有一条著名的经验法则, 用于指导何时需要自定义拷贝控制成员。

三法则 (Rule of Three) 指出: 如果一个类需要自定义析构函数、拷贝构造函数或拷贝赋值运算符中的任何一个, 那么通常也需要自定义另外两个。

这是因为需要自定义这些函数的情况通常意味着类管理着某种资源。如果析构函数需要释放资源, 那么拷贝时就需要决定如何处理资源 (深拷贝还是共享); 如果拷贝构造函数需要复制资源, 那么赋值运算符同样需要。

C++11 引入了移动语义后，三法则扩展为五法则（Rule of Five）：如果需要自定义析构函数、拷贝构造函数、拷贝赋值运算符、移动构造函数或移动赋值运算符中的任何一个，通常应该考虑全部五个。移动语义将在后续章节介绍。

还有一条相关的零法则（Rule of Zero）：如果可能，设计类时应该不需要自定义任何拷贝控制成员。通过使用智能指针和标准库容器等自动管理资源的类型，可以让编译器生成的默认版本正确工作：

```
1 // 遵循零法则的设计
2 class ModernArray {
3 public:
4     ModernArray(size_t size)
5         : data(size, 0) // vector 自动管理内存
6     {
7     }
8
9     // 不需要自定义析构函数、拷贝构造函数、拷贝赋值运算符
10    // 编译器生成的版本会正确调用 vector 的相应函数
11
12    int& operator[](size_t index) { return data[index]; }
13    size_t getSize() const { return data.size(); }
14
15 private:
16    std::vector<int> data; // 让 vector 管理内存
17 };
```

`std::vector` 已经正确实现了拷贝控制，因此包含 `std::vector` 的类也自动具有正确的拷贝行为。这种设计更简洁、更安全、更不容易出错。

1.5.13.8. 禁止拷贝

有些类根本不应该被拷贝。例如，表示唯一资源的类（如线程、互斥锁、文件句柄的独占所有权）、单例模式的类，或者拷贝成本过高且没有意义的类。

C++11 之前，禁止拷贝的方法是将拷贝构造函数和拷贝赋值运算符声明为私有且不实现：

```
1 // C++11 之前的做法
2 class NonCopyable {
3 private:
4     NonCopyable(const NonCopyable&); // 只声明，不实现
5     NonCopyable& operator=(const NonCopyable&); // 只声明，不实现
6 public:
7     NonCopyable() {}
8 };
```

C++11 引入了更清晰的语法——使用 `= delete` 显式禁止函数：

```
1 class UniqueResource {
2 public:
3     UniqueResource() : handle(acquireResource()) {}
4     ~UniqueResource() { releaseResource(handle); }
5
6     // 禁止拷贝
7     UniqueResource(const UniqueResource&) = delete;
8     UniqueResource& operator=(const UniqueResource&) = delete;
9
10    void use() { /* 使用资源 */ }
11
12 private:
```

```

13     ResourceHandle handle;
14 };
15
16 int main() {
17     UniqueResource r1;
18     // UniqueResource r2 = r1; // 编译错误: 拷贝构造函数被删除
19     // UniqueResource r3;
20     // r3 = r1; // 编译错误: 拷贝赋值运算符被删除
21     return 0;
22 }

```

`= delete` 不仅适用于拷贝控制成员，还可以用于禁止任何不希望被调用的函数重载。编译器在尝试调用被删除的函数时会产生清晰的错误信息。

1.5.13.9. 显式默认

与 `= delete` 相对的是 `= default`，它显式要求编译器生成默认版本的特殊成员函数：

```

1 class Example {
2 public:
3     Example() = default; // 显式使用默认构造函数
4     Example(int value) : value(value) {}
5
6     Example(const Example&) = default; // 显式使用默认拷贝构造
7     Example& operator=(const Example&) = default; // 显式使用默认拷贝赋值
8     ~Example() = default; // 显式使用默认析构
9
10 private:
11     int value = 0;
12 };

```

使用 `= default` 的好处包括：

明确表达意图——告诉读者“这里使用默认行为是有意的，不是遗忘”。

即使定义了其他构造函数，也可以保留默认构造函数。

在某些情况下，编译器生成的版本可能比手写的更高效（如可以是 trivial 的）。

1.5.13.10. 一个完整的例子

让我们用一个完整的例子来综合运用本节的知识。以下是一个简化的字符串类实现：

```

1 #include <iostream>
2 #include <cstring>
3 #include <algorithm>
4
5 class MyString {
6 public:
7     // 默认构造函数
8     MyString() : len(0), data(new char[1]) {
9         data[0] = '\0';
10    }
11
12    // 从 C 字符串构造
13    MyString(const char* str) {
14        len = std::strlen(str);
15        data = new char[len + 1];
16        std::strcpy(data, str);
17    }
18
19    // 拷贝构造函数（深拷贝）

```

```

20     MyString(const MyString& other) : len(other.len), data(new
char[other.len + 1]) {
21         std::strcpy(data, other.data);
22         std::cout << "拷贝构造: \" << data << \"\" << std::endl;
23     }
24
25     // 析构函数
26     ~MyString() {
27         delete[] data;
28     }
29
30     // swap 函数
31     friend void swap(MyString& first, MyString& second) noexcept {
32         using std::swap;
33         swap(first.len, second.len);
34         swap(first.data, second.data);
35     }
36
37     // 拷贝赋值运算符 (copy-and-swap)
38     MyString& operator=(MyString other) {
39         std::cout << "拷贝赋值: \" << other.data << \"\" << std::endl;
40         swap(*this, other);
41         return *this;
42     }
43
44     // 访问接口
45     size_t length() const { return len; }
46     const char* c_str() const { return data; }
47
48     char& operator[](size_t index) { return data[index]; }
49     char operator[](size_t index) const { return data[index]; }
50
51     // 字符串连接
52     MyString operator+(const MyString& other) const {
53         MyString result;
54         delete[] result.data;
55
56         result.len = len + other.len;
57         result.data = new char[result.len + 1];
58         std::strcpy(result.data, data);
59         std::strcat(result.data, other.data);
60
61         return result;
62     }
63
64     // 输出运算符
65     friend std::ostream& operator<<(std::ostream& os, const MyString& str) {
66         return os << str.data;
67     }
68
69 private:
70     size_t len;
71     char* data;
72 };
73
74 int main() {
75     MyString s1("Hello");
76     MyString s2(" World");
77
78     std::cout << "s1: " << s1 << std::endl;
79     std::cout << "s2: " << s2 << std::endl;
80
81     MyString s3 = s1; // 拷贝构造

```



```

82     std::cout << "s3: " << s3 << std::endl;
83
84     MyString s4;
85     s4 = s1 + s2; // 赋值
86     std::cout << "s4: " << s4 << std::endl;
87
88     s1[0] = 'h';
89     std::cout << "修改后 s1: " << s1 << std::endl;
90     std::cout << "s3 不变: " << s3 << std::endl; // 深拷贝, s1 的修改不影响 s3
91
92     return 0;
93 }

```

这个例子展示了：

深拷贝的拷贝构造函数，确保每个对象拥有独立的数据副本。

使用 copy-and-swap 惯用法的拷贝赋值运算符，既简洁又异常安全。

正确的析构函数释放动态分配的内存。

遵循三法则，三个函数配合工作，确保对象在整个生命周期内正确管理资源。

1.5.13.11. 实践建议

编写拷贝控制成员时，以下原则值得遵循：

优先使用零法则。如果可能，使用 `std::string`、`std::vector`、智能指针等标准库类型来管理资源，让编译器生成的默认函数正确工作。这是最简单也最不容易出错的方式。

如果必须管理资源，遵循三/五法则。自定义析构函数、拷贝构造函数和拷贝赋值运算符（C++11 后还包括移动操作），确保它们协调一致。

使用 copy-and-swap 实现赋值运算符。这种方式代码简洁、自动处理自赋值、提供强异常安全保证。

对于不应拷贝的类，显式删除拷贝操作。使用 `= delete` 明确表达意图，让编译器帮助捕获错误使用。

测试拷贝行为。编写测试用例验证拷贝后的对象与原对象独立、修改一个不影响另一个、两个对象都能正确析构。

拷贝控制是 C++ 对象模型的核心部分。正确实现拷贝语义，能够让自定义类型像内置类型一样自然地使用——可以放入容器、作为参数传递、作为返回值返回。然而，拷贝有时意味着性能开销，特别是对于大型对象。C++11 引入的移动语义提供了一种优化方案，允许“窃取”临时对象的资源而非复制它们。但在学习移动语义之前，我们先来探索类的另一个重要特性——继承，它使得代码复用和类型层次结构成为可能。

1.5.14. 类的继承

在前面的章节中，我们学习了如何定义类、如何控制对象的生命周期、如何正确处理对象的拷贝。这些都是构建单个类的基础技术。然而，现实世界中的概念往往存在层次关系：电机是一种执行器，步兵机器人是一种机器人，自动瞄准系统是一种瞄准系统。这些“是一种”（is-a）的关系可以用继承（inheritance）来表达。继承允许我们基于已有的类创建新类，新类自动获得已有类的特性，同时可以添加新特性或修改已有行为。这不仅实现了代码复用，更重要的是建立了类型之间的层次结构，为多态奠定了基础。

1.5.14.1. 继承的基本概念

假设我们正在开发 RoboMaster 机器人的控制系统，需要处理不同类型的机器人：步兵、英雄、工程、哨兵等。这些机器人有很多共同的特性——都有底盘、都能移动、都有血量——但也有各自独特的能力。如果为每种机器人都从头编写一个类，会有大量重复代码。

继承提供了一种更好的方式：先定义一个包含共同特性的基类（base class，也称父类或超类），然后让各种具体的机器人类从基类派生（derive），自动继承共同特性，同时添加各自的独特功能。

```
1 // 基类：通用机器人
2 class Robot {
3 public:
4     Robot(int id, int maxHealth)
5         : id(id), health(maxHealth), maxHealth(maxHealth)
6     {
7     }
8
9     void takeDamage(int damage) {
10         health -= damage;
11         if (health < 0) health = 0;
12     }
13
14     void heal(int amount) {
15         health += amount;
16         if (health > maxHealth) health = maxHealth;
17     }
18
19     bool isAlive() const {
20         return health > 0;
21     }
22
23     int getHealth() const { return health; }
24     int getId() const { return id; }
25
26 protected:
27     int id;
28     int health;
29     int maxHealth;
30 };
31
32 // 派生类：步兵机器人
33 class Infantry : public Robot {
34 public:
35     Infantry(int id)
36         : Robot(id, 500) // 调用基类构造函数，步兵血量 500
37     {
38     }
39
40     void shoot() {
41         std::cout << "步兵 " << id << " 发射弹丸" << std::endl;
42     }
43 };
44
45 // 派生类：英雄机器人
46 class Hero : public Robot {
47 public:
48     Hero(int id)
49         : Robot(id, 600) // 英雄血量 600
50     {
51     }
52 }
```

```

53     void shootLarge() {
54         std::cout << "英雄 " << id << " 发射大弹丸" << std::endl;
55     }
56 };

```

在这个例子中，`Robot` 是基类，`Infantry` 和 `Hero` 是派生类。派生类使用 `: public Robot` 语法声明它们继承自 `Robot`。派生类自动拥有基类的所有成员（`id`、`health`、`takeDamage()` 等），同时可以定义自己的成员（如 `Infantry::shoot()`、`Hero::shootLarge()`）。

使用这些类：

```

1  int main() {
2      Infantry infantry(1);
3      Hero hero(2);
4
5      // 派生类对象可以使用基类的成员函数
6      infantry.takeDamage(100);
7      hero.takeDamage(150);
8
9      std::cout << "步兵血量: " << infantry.getHealth() << std::endl; // 400
10     std::cout << "英雄血量: " << hero.getHealth() << std::endl;      // 450
11
12     // 派生类对象可以使用自己的成员函数
13     infantry.shoot();
14     hero.shootLarge();
15
16     return 0;
17 }

```

继承建立了“是一种”的关系：`Infantry` 是一种 `Robot`，`Hero` 也是一种 `Robot`。这种关系不仅是概念上的，在 C++ 类型系统中也有体现——派生类对象可以被当作基类对象使用，这是多态的基础。

1.5.14.2. 继承的语法

派生类的定义语法如下：

```

1  class 派生类名 : 访问说明符 基类名 {
2      // 派生类成员
3  };

```

访问说明符可以是 `public`、`protected` 或 `private`，它决定了基类成员在派生类中的访问权限。最常用的是 `public` 继承，它保持基类成员的原有访问级别：

```

1  class Infantry : public Robot {
2      // Robot 的 public 成员在 Infantry 中仍是 public
3      // Robot 的 protected 成员在 Infantry 中仍是 protected
4      // Robot 的 private 成员在 Infantry 中不可直接访问
5  };

```

`protected` 继承和 `private` 继承较少使用，它们会降低基类成员的访问级别。在 `protected` 继承中，基类的 `public` 成员变为 `protected`；在 `private` 继承中，基类的所有可访问成员都变为 `private`。这些形式更多用于实现细节的封装，而非表达“是一种”的关系。

C++ 支持多重继承——一个类可以从多个基类派生：

```

1 class FlyingRobot : public Robot, public Flyable {
2     // 同时继承 Robot 和 Flyable 的特性
3 };

```

多重继承功能强大但也容易引入复杂性（如菱形继承问题），在实践中应谨慎使用。本节主要讨论单继承。

1.5.14.3. 访问控制与继承

理解继承中的访问控制对于正确设计类层次结构至关重要。C++ 的三个访问级别在继承中有不同的表现：

public 成员对所有代码可见。基类的公开成员在派生类中仍然是公开的，外部代码可以通过派生类对象访问这些成员。

private 成员只对定义它的类可见。基类的私有成员对派生类也是不可见的——派生类不能直接访问基类的私有成员，只能通过基类提供的公开或受保护接口间接访问。

protected 成员介于两者之间。它对外部代码不可见（像 **private**），但对派生类可见（不像 **private**）。**protected** 专为继承设计，用于那些需要在派生类中访问但不想暴露给外部的成员。

```

1 class Base {
2 public:
3     int publicData;
4     void publicFunc() {}
5
6 protected:
7     int protectedData;
8     void protectedFunc() {}
9
10 private:
11     int privateData;
12     void privateFunc() {}
13 };
14
15 class Derived : public Base {
16 public:
17     void accessBaseMembers() {
18         publicData = 1;    // 正确, public 成员可访问
19         publicFunc();      // 正确
20
21         protectedData = 2; // 正确, protected 成员对派生类可访问
22         protectedFunc();   // 正确
23
24         // privateData = 3; // 错误! private 成员不可访问
25         // privateFunc();   // 错误!
26     }
27 };
28
29 int main() {
30     Derived d;
31     d.publicData = 1;    // 正确, public 成员
32     d.publicFunc();      // 正确
33
34     // d.protectedData = 2; // 错误! protected 对外部不可见
35     // d.privateData = 3;   // 错误! private 对外部不可见
36
37     return 0;
38 }

```

在设计类层次时，需要仔细考虑每个成员的访问级别。一般原则是：

数据成员通常设为 `private` 或 `protected`。如果派生类需要直接访问，使用 `protected`；否则使用 `private` 并提供访问接口。

对外接口设为 `public`。这些是类的使用者（包括派生类对象的使用者）可以调用的方法。

供派生类扩展或修改的方法设为 `protected` 或 `public`。这取决于是否也想让外部代码调用。

在前面 `Robot` 的例子中，我们将 `id`、`health`、`maxHealth` 设为 `protected`，使得派生类可以直接访问这些数据（如 `Infantry::shoot()` 中使用 `id`），同时对外部代码隐藏这些实现细节。

1.5.14.4. 派生类的构造函数

派生类对象包含从基类继承的成员和派生类自己定义的成员。构造派生类对象时，这些成员都需要被初始化。C++ 的规则是：先构造基类部分，再构造派生类部分。

派生类构造函数通过成员初始化列表调用基类构造函数：

```
1 class Robot {
2 public:
3     Robot(int id, int maxHealth)
4         : id(id), health(maxHealth), maxHealth(maxHealth)
5     {
6         std::cout << "Robot 构造: id=" << id << std::endl;
7     }
8
9 protected:
10     int id;
11     int health;
12     int maxHealth;
13 };
14
15 class Infantry : public Robot {
16 public:
17     Infantry(int id, int ammo)
18         : Robot(id, 500), // 先调用基类构造函数
19         ammoCount(ammo) // 再初始化派生类成员
20     {
21         std::cout << "Infantry 构造: ammo=" << ammoCount << std::endl;
22     }
23
24 private:
25     int ammoCount;
26 };
27
28 int main() {
29     Infantry infantry(1, 100);
30     return 0;
31 }
```

输出：

```
1 Robot 构造: id=1
2 Infantry 构造: ammo=100
```

基类构造函数必须在初始化列表中调用，不能在构造函数体内调用。如果不显式调用基类构造函数，编译器会尝试调用基类的默认构造函数。如果基类没有默认构造函数，则会产生编译错误：

```

1 class Base {
2 public:
3     Base(int x) : value(x) {} // 只有带参数的构造函数，没有默认构造函数
4 private:
5     int value;
6 };
7
8 class Derived : public Base {
9 public:
10     Derived() // 错误！没有调用 Base 的构造函数，且 Base 没有默认构造函数
11     {
12     }
13
14     Derived(int x) : Base(x) {} // 正确，显式调用 Base(int)
15 };

```

派生类构造函数可以接受额外的参数，用于初始化派生类特有的成员，同时将必要的参数传递给基类构造函数：

```

1 class Engineer : public Robot {
2 public:
3     Engineer(int id, int maxHealth, int repairRate)
4         : Robot(id, maxHealth), // 传递给基类
5           repairRate(repairRate) // 初始化派生类成员
6     {
7     }
8
9     void repairAlly(Robot& ally) {
10         ally.heal(repairRate);
11         std::cout << "工程机器人修理了友军" << std::endl;
12     }
13
14 private:
15     int repairRate;
16 };

```

1.5.14.5. 析构函数与继承

与构造顺序相反，析构时先执行派生类析构函数，再执行基类析构函数。这个顺序确保了在派生类析构函数执行期间，基类部分仍然有效：

```

1 class Robot {
2 public:
3     Robot(int id) : id(id) {
4         std::cout << "Robot 构造: " << id << std::endl;
5     }
6
7     ~Robot() {
8         std::cout << "Robot 析构: " << id << std::endl;
9     }
10
11 protected:
12     int id;
13 };
14
15 class Infantry : public Robot {
16 public:
17     Infantry(int id) : Robot(id) {
18         std::cout << "Infantry 构造" << std::endl;
19     }

```

```

20
21     ~Infantry() {
22         std::cout << "Infantry 析构" << std::endl;
23     }
24 };
25
26 int main() {
27     Infantry infantry(1);
28     return 0;
29 }

```

输出:

```

1 Robot 构造: 1
2 Infantry 构造
3 Infantry 析构
4 Robot 析构: 1

```

这种“先进后出”的顺序与栈的行为一致：基类是“先进”的（先构造），所以“后出”（后析构）。

派生类的析构函数会自动调用基类的析构函数，不需要（也不应该）显式调用。如果派生类管理了资源，只需在派生类析构函数中释放派生类自己的资源，基类的资源由基类析构函数负责：

```

1 class ResourceHolder : public Robot {
2 public:
3     ResourceHolder(int id) : Robot(id), buffer(new char[1024]) {
4         std::cout << "分配缓冲区" << std::endl;
5     }
6
7     ~ResourceHolder() {
8         delete[] buffer; // 只释放派生类的资源
9         std::cout << "释放缓冲区" << std::endl;
10        // 基类析构函数会自动被调用
11    }
12
13 private:
14     char* buffer;
15 };

```

关于析构函数与继承，还有一个重要的话题——虚析构函数，这将在下一节“多态与虚函数”中详细讨论。简单来说，如果一个类可能被继承，其析构函数通常应该声明为 `virtual`。

1.5.14.6. 在派生类中访问基类成员

派生类可以直接访问基类的 `public` 和 `protected` 成员，就像它们是派生类自己的成员一样。但有时需要明确指定访问的是基类的成员，特别是当派生类定义了同名成员时。

使用作用域解析运算符 `::` 可以明确指定访问基类成员：

```

1 class Base {
2 public:
3     void print() {
4         std::cout << "Base::print()" << std::endl;
5     }
6
7 protected:
8     int value = 10;
9 };

```

```

10
11 class Derived : public Base {
12 public:
13     void print() { // 隐藏了基类的 print()
14         std::cout << "Derived::print()" << std::endl;
15     }
16
17     void printBoth() {
18         print(); // 调用 Derived::print()
19         Base::print(); // 明确调用 Base::print()
20     }
21
22     void showValues() {
23         int value = 20; // 局部变量, 隐藏了成员
24         std::cout << "局部 value: " << value << std::endl; // 20
25         std::cout << "成员 value: " << this->value << std::endl; // 10 (继承
    自 Base)
26         std::cout << "Base::value: " << Base::value << std::endl; // 10
27     }
28 };

```

当派生类定义了与基类同名的成员（函数或变量）时，基类成员被“隐藏”（hidden）。这与后面要介绍的“覆盖”（override）不同——隐藏是名称查找层面的概念，覆盖是多态行为层面的概念。

1.5.14.7. 派生类与基类的类型关系

继承建立的“是一种”关系在类型系统中有重要体现：派生类对象可以被当作基类对象使用。具体来说，可以将派生类对象绑定到基类引用，或将派生类对象的地址赋给基类指针：

```

1 void displayHealth(const Robot& robot) {
2     std::cout << "血量: " << robot.getHealth() << std::endl;
3 }
4
5 int main() {
6     Infantry infantry(1);
7     Hero hero(2);
8
9     displayHealth(infantry); // 正确, Infantry 是 Robot
10    displayHealth(hero); // 正确, Hero 是 Robot
11
12    Robot* robotPtr = &infantry; // 基类指针指向派生类对象
13    robotPtr->takeDamage(50); // 通过基类指针调用基类成员
14
15    Robot& robotRef = hero; // 基类引用绑定到派生类对象
16    robotRef.heal(100); // 通过基类引用调用基类成员
17
18    return 0;
19 }

```

这种向上转型（upcasting）是安全的，因为派生类对象包含基类的所有成员。通过基类指针或引用访问对象时，只能访问基类中定义的成员：

```

1 Robot* ptr = &infantry;
2 ptr->takeDamage(50); // 正确, takeDamage 是 Robot 的成员
3 // ptr->shoot(); // 错误! Robot 没有 shoot() 方法

```

反向的转换——将基类对象当作派生类对象——称为向下转型（downcasting），这通常是不安全的，需要使用特殊的转型操作符，将在后续章节讨论。

1.5.14.8. 对象切片

当派生类对象被赋值给基类对象（而非指针或引用）时，会发生对象切片（object slicing）：派生类特有的成员被“切掉”，只保留基类部分：

```
1 Infantry infantry(1);
2 infantry.takeDamage(100);
3
4 Robot robot = infantry; // 对象切片!
5 // robot 只有 Robot 的成员, Infantry 特有的成员丢失了
```

对象切片通常不是预期的行为，它会导致信息丢失。要保持派生类的完整信息，应该使用指针或引用：

```
1 Infantry infantry(1);
2
3 Robot& ref = infantry; // 引用, 无切片
4 Robot* ptr = &infantry; // 指针, 无切片
5
6 // ref 和 ptr 仍然指向完整的 Infantry 对象
```

在 RoboMaster 开发中，经常需要将不同类型的机器人统一管理。应该使用指针或引用的容器，而非对象容器：

```
1 // 错误的做法：会发生对象切片
2 std::vector<Robot> robots;
3 robots.push_back(Infantry(1)); // 切片！只保存了 Robot 部分
4
5 // 正确的做法：使用指针
6 std::vector<Robot*> robots;
7 Infantry* infantry = new Infantry(1);
8 Hero* hero = new Hero(2);
9 robots.push_back(infantry);
10 robots.push_back(hero);
11
12 // 更好的做法：使用智能指针
13 std::vector<std::unique_ptr<Robot>> robots;
14 robots.push_back(std::make_unique<Infantry>(1));
15 robots.push_back(std::make_unique<Hero>(2));
```

1.5.14.9. 继承层次的设计

设计良好的继承层次应该反映真实的“是一种”关系。如果 B 继承自 A，那么 B 应该在概念上是 A 的一种特殊情况，B 的对象可以在任何需要 A 的地方使用。这称为里氏替换原则（Liskov Substitution Principle）。

```
1 // 良好的继承设计
2 class Shape {
3 public:
4     virtual double area() const = 0;
5 };
6
7 class Rectangle : public Shape {
8 public:
9     Rectangle(double w, double h) : width(w), height(h) {}
10    double area() const override { return width * height; }
11 private:
12    double width, height;
13 };
```



```

14
15 class Circle : public Shape {
16 public:
17     Circle(double r) : radius(r) {}
18     double area() const override { return 3.14159 * radius * radius; }
19 private:
20     double radius;
21 };
22
23 // Rectangle 是一种 Shape, Circle 也是一种 Shape
24 // 可以统一处理各种形状
25 void printArea(const Shape& shape) {
26     std::cout << "面积: " << shape.area() << std::endl;
27 }

```

不应该仅仅为了复用代码而使用继承。如果两个类之间的关系是“有一个”(has-a)而非“是一种”(is-a)，应该使用组合而非继承：

```

1 // 错误: Engine 不是 Robot 的一种
2 class Engine : public Robot { }; // 不合理
3
4 // 正确: Robot 有一个 Engine
5 class Robot {
6 private:
7     Engine engine; // 组合关系
8 };

```

继承层次不宜过深。过深的层次增加复杂性，使得理解和维护变得困难。如果发现层次超过三四层，应该考虑是否可以通过组合或重新设计来简化。

1.5.14.10. 一个完整的例子

让我们用一个更完整的例子来综合运用继承的各个方面：

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <memory>
5
6 // 基类: 机器人
7 class Robot {
8 public:
9     Robot(int id, const std::string& name, int maxHealth)
10         : id(id), name(name), health(maxHealth), maxHealth(maxHealth)
11     {
12         std::cout << "创建机器人: " << name << " (ID: " << id << ")"
13         << std::endl;
14     }
15     virtual ~Robot() {
16         std::cout << "销毁机器人: " << name << std::endl;
17     }
18     void takeDamage(int damage) {
19         health -= damage;
20         if (health < 0) health = 0;
21         std::cout << name << " 受到 " << damage << " 点伤害, 剩余血量: " <<
22         health << std::endl;
23     }
24 }

```

```

25     void heal(int amount) {
26         health += amount;
27         if (health > maxHealth) health = maxHealth;
28     }
29
30     bool isAlive() const { return health > 0; }
31     int getHealth() const { return health; }
32     const std::string& getName() const { return name; }
33
34     virtual void performAction() {
35         std::cout << name << " 待机中..." << std::endl;
36     }
37
38 protected:
39     int id;
40     std::string name;
41     int health;
42     int maxHealth;
43 };
44
45 // 派生类: 步兵机器人
46 class Infantry : public Robot {
47 public:
48     Infantry(int id)
49         : Robot(id, "步兵-" + std::to_string(id), 500),
50           ammo(300), fireRate(10)
51     {
52     }
53
54     ~Infantry() override {
55         std::cout << "步兵专用资源已释放" << std::endl;
56     }
57
58     void shoot() {
59         if (ammo > 0) {
60             ammo--;
61             std::cout << getName() << " 发射弹丸 (剩余: " << ammo << ")"
62             << std::endl;
63         } else {
64             std::cout << getName() << " 弹药耗尽! " << std::endl;
65         }
66     }
67
68     void reload(int amount) {
69         ammo += amount;
70         std::cout << getName() << " 装填弹药, 当前: " << ammo << std::endl;
71     }
72
73     void performAction() override {
74         shoot();
75     }
76 private:
77     int ammo;
78     int fireRate;
79 };
80
81 // 派生类: 工程机器人
82 class Engineer : public Robot {
83 public:
84     Engineer(int id)
85         : Robot(id, "工程-" + std::to_string(id), 600),
86           repairRate(50)

```

```

87     {
88     }
89
90     void repair(Robot& target) {
91         if (&target == this) {
92             std::cout << getName() << " 无法修理自己" << std::endl;
93             return;
94         }
95         target.heal(repairRate);
96         std::cout << getName() << " 修理了 " << target.getName()
97             << ", 恢复 " << repairRate << " 点血量" << std::endl;
98     }
99
100    void performAction() override {
101        std::cout << getName() << " 准备进行工程作业" << std::endl;
102    }
103
104    private:
105        int repairRate;
106    };
107
108    int main() {
109        // 使用智能指针管理机器人
110        std::vector<std::unique_ptr<Robot>> team;
111
112        team.push_back(std::make_unique<Infantry>(1));
113        team.push_back(std::make_unique<Infantry>(2));
114        team.push_back(std::make_unique<Engineer>(3));
115
116        std::cout << "\n--- 战斗开始 ---\n" << std::endl;
117
118        // 通过基类指针统一操作
119        for (auto& robot : team) {
120            robot->performAction();
121        }
122
123        std::cout << "\n--- 受到攻击 ---\n" << std::endl;
124
125        team[0]->takeDamage(200);
126        team[1]->takeDamage(150);
127
128        std::cout << "\n--- 工程机器人修理 ---\n" << std::endl;
129
130        // 需要向下转型才能调用派生类特有方法
131        Engineer* engineer = dynamic_cast<Engineer*>(team[2].get());
132        if (engineer) {
133            engineer->repair(*team[0]);
134            engineer->repair(*team[1]);
135        }
136
137        std::cout << "\n--- 程序结束 ---\n" << std::endl;
138
139        return 0;
140    }

```

这个例子展示了继承的多个方面：基类定义共同接口，派生类扩展功能；构造函数链式调用；通过基类指针统一管理不同类型的对象；虚函数实现多态行为（将在下一节详细介绍）。

继承是面向对象编程的三大支柱之一（另外两个是封装和多态）。通过继承，我们可以建立类型层次结构，实现代码复用，并为多态奠定基础。然而，继承本身并不能实现真正的运行时多态——当通过基类指针调用成员函数时，调用的总是基类版本。要让派生类能够替换基类的行为，需要使用虚函数。下一节将深入探讨多态与虚函数，揭示面向对象编程最强大的特性。

1.5.15. 多态与虚函数

上一节我们学习了继承，它允许派生类复用基类的代码，并通过基类指针或引用统一管理不同类型的对象。然而，当通过基类指针调用成员函数时，调用的始终是基类的版本，即使指针实际指向的是派生类对象。这在很多场景下并非我们想要的行为——我们希望调用的是派生类重新定义的版本，让不同类型的对象表现出不同的行为。这种“同一操作作用于不同对象产生不同效果”的能力称为多态（polymorphism），而虚函数（virtual function）正是 C++ 实现运行时多态的机制。多态是面向对象编程最强大的特性之一，它使得程序能够以统一的方式处理各种具体类型，极大地提高了代码的灵活性和可扩展性。

1.5.15.1. 从问题出发

让我们先看一个没有使用虚函数的例子，理解为什么需要多态：

```
1  class Robot {
2  public:
3      Robot(const std::string& name) : name(name) {}
4
5      void performAction() {
6          std::cout << name << " 执行默认动作" << std::endl;
7      }
8
9  protected:
10     std::string name;
11 };
12
13 class Infantry : public Robot {
14 public:
15     Infantry(const std::string& name) : Robot(name) {}
16
17     void performAction() { // 重新定义了 performAction
18         std::cout << name << " 发射弹丸" << std::endl;
19     }
20 };
21
22 class Engineer : public Robot {
23 public:
24     Engineer(const std::string& name) : Robot(name) {}
25
26     void performAction() { // 重新定义了 performAction
27         std::cout << name << " 进行修理" << std::endl;
28     }
29 };
30
31 void commandRobot(Robot& robot) {
32     robot.performAction(); // 调用哪个版本?
33 }
34
35 int main() {
36     Infantry infantry("步兵1号");
37     Engineer engineer("工程1号");
38
39     commandRobot(infantry); // 输出: 步兵1号 执行默认动作
40     commandRobot(engineer); // 输出: 工程1号 执行默认动作
41
42     return 0;
43 }
```

尽管 Infantry 和 Engineer 都重新定义了 performAction()，但通过基类引用调用时，执行的始终是基类 Robot 的版本。这是因为编译器在编译时根据引用的静态类型

(Robot&) 决定调用哪个函数，而非根据实际对象的类型。这种绑定方式称为静态绑定或早绑定 (static/early binding)。

我们真正需要的是：让 `commandRobot` 函数根据传入对象的实际类型调用相应的 `performAction()` 版本。这就是动态绑定或晚绑定 (dynamic/late binding)，需要通过虚函数来实现。

1.5.15.2. 虚函数

将基类中的成员函数声明为虚函数，只需在函数声明前加上 `virtual` 关键字：

```
1 class Robot {
2 public:
3     Robot(const std::string& name) : name(name) {}
4
5     virtual void performAction() { // 声明为虚函数
6         std::cout << name << " 执行默认动作" << std::endl;
7     }
8
9 protected:
10     std::string name;
11 };
12
13 class Infantry : public Robot {
14 public:
15     Infantry(const std::string& name) : Robot(name) {}
16
17     void performAction() override { // 覆盖基类虚函数
18         std::cout << name << " 发射弹丸" << std::endl;
19     }
20 };
21
22 class Engineer : public Robot {
23 public:
24     Engineer(const std::string& name) : Robot(name) {}
25
26     void performAction() override { // 覆盖基类虚函数
27         std::cout << name << " 进行修理" << std::endl;
28     }
29 };
30
31 void commandRobot(Robot& robot) {
32     robot.performAction(); // 现在会根据实际类型调用
33 }
34
35 int main() {
36     Infantry infantry("步兵1号");
37     Engineer engineer("工程1号");
38
39     commandRobot(infantry); // 输出: 步兵1号 发射弹丸
40     commandRobot(engineer); // 输出: 工程1号 进行修理
41
42     return 0;
43 }
```

现在，`commandRobot` 函数会根据传入对象的实际类型调用相应的 `performAction()` 版本。这就是多态的魔力：同一段代码 (`robot.performAction()`) 作用于不同对象时产生不同的行为。

派生类中重新定义虚函数称为覆盖（override）。`override` 关键字是 C++11 引入的，它告诉编译器这个函数意图覆盖基类的虚函数。如果基类中没有匹配的虚函数（比如函数名拼写错误或参数列表不匹配），编译器会报错，帮助我们捕获潜在的 bug：

```
1 class Infantry : public Robot {
2 public:
3     // 错误示例：函数名拼写错误
4     void preformAction() override { // 编译错误！基类没有 preformAction
5         std::cout << "发射" << std::endl;
6     }
7
8     // 错误示例：参数列表不匹配
9     void performAction(int times) override { // 编译错误！签名不匹配
10         std::cout << "发射 " << times << " 次" << std::endl;
11     }
12 };
```

虽然 `override` 不是必需的，但强烈建议在所有覆盖虚函数时使用它，这是现代 C++ 的最佳实践。

1.5.15.3. 动态绑定的工作原理

虚函数的多态行为是如何实现的？编译器通过虚函数表（virtual table，简称 vtable）机制来实现动态绑定。

当一个类包含虚函数时，编译器会为该类创建一个虚函数表，其中存储了该类所有虚函数的地址。每个包含虚函数的对象内部都有一个隐藏的指针（通常称为 `vptr`），指向其所属类的虚函数表。

当通过基类指针或引用调用虚函数时，程序会：

1. 通过对象的 `vptr` 找到虚函数表
2. 在虚函数表中查找要调用的函数地址
3. 调用该地址处的函数

由于不同类型的对象有不同的虚函数表，即使通过相同的基类指针访问，也会调用到各自类型对应的函数版本。

```
1 // 概念性地展示 vtable（实际实现由编译器处理）
2
3 // Robot 的 vtable:
4 // [0] -> Robot::performAction()
5
6 // Infantry 的 vtable:
7 // [0] -> Infantry::performAction() // 覆盖了基类版本
8
9 // Engineer 的 vtable:
10 // [0] -> Engineer::performAction() // 覆盖了基类版本
```

这种机制有一些性能影响：每个多态对象需要额外存储一个 `vptr`（通常 8 字节），每次虚函数调用需要一次间接寻址。在大多数应用中这点开销可以忽略不计，但在极端性能敏感的场景（如每秒调用百万次的紧密循环）可能需要考虑。

需要特别注意的是，动态绑定只在通过指针或引用调用虚函数时发生。如果通过对象直接调用，或者在构造函数/析构函数中调用虚函数，使用的是静态绑定：

```
1 Infantry infantry("步兵");
2
```

```

3 infantry.performAction(); // 直接调用, 静态绑定, 调用 Infantry::performAction
4
5 Robot robot = infantry;    // 对象切片!
6 robot.performAction();     // 静态绑定, 调用 Robot::performAction
7
8 Robot& ref = infantry;
9 ref.performAction();       // 动态绑定, 调用 Infantry::performAction

```

1.5.15.4. 纯虚函数与抽象类

有时候, 基类中的某个虚函数没有合理的默认实现——它的存在只是为了定义接口, 具体实现必须由派生类提供。这种情况下可以将虚函数声明为纯虚函数 (pure virtual function), 通过在声明末尾加上 `= 0` 来实现:

```

1 class Robot {
2 public:
3     Robot(const std::string& name) : name(name) {}
4     virtual ~Robot() = default;
5
6     // 纯虚函数: 没有默认实现, 派生类必须覆盖
7     virtual void performAction() = 0;
8
9     // 普通虚函数: 有默认实现, 派生类可以选择覆盖
10    virtual void reportStatus() {
11        std::cout << name << " 状态正常" << std::endl;
12    }
13
14 protected:
15     std::string name;
16 };

```

包含纯虚函数的类称为抽象类 (abstract class)。抽象类不能被实例化——只能作为基类使用, 由派生类提供纯虚函数的实现:

```

1 int main() {
2     // Robot robot("测试"); // 编译错误! Robot 是抽象类, 不能实例化
3
4     Infantry infantry("步兵"); // 正确, Infantry 实现了所有纯虚函数
5     Robot* ptr = &infantry;    // 可以使用抽象类的指针
6     ptr->performAction();       // 调用 Infantry::performAction
7
8     return 0;
9 }

```

如果派生类没有实现基类的所有纯虚函数, 那么派生类也是抽象类:

```

1 class SpecialRobot : public Robot {
2 public:
3     SpecialRobot(const std::string& name) : Robot(name) {}
4     // 没有实现 performAction(), 所以 SpecialRobot 也是抽象类
5 };
6
7 // SpecialRobot sr("特殊"); // 编译错误!

```

抽象类的典型用途是定义接口。在 RoboMaster 开发中, 可以定义各种抽象接口:

```

1 // 可射击接口
2 class Shootable {

```

```

3 public:
4     virtual ~Shootable() = default;
5     virtual void shoot() = 0;
6     virtual int getAmmo() const = 0;
7     virtual void reload(int amount) = 0;
8 };
9
10 // 可移动接口
11 class Movable {
12 public:
13     virtual ~Movable() = default;
14     virtual void move(double x, double y) = 0;
15     virtual void stop() = 0;
16     virtual double getSpeed() const = 0;
17 };
18
19 // 步兵机器人实现多个接口
20 class Infantry : public Robot, public Shootable, public Movable {
21 public:
22     Infantry(const std::string& name) : Robot(name), ammo(300), speed(0) {}
23
24     // 实现 Shootable 接口
25     void shoot() override {
26         if (ammo > 0) {
27             ammo--;
28             std::cout << name << " 开火! " << std::endl;
29         }
30     }
31     int getAmmo() const override { return ammo; }
32     void reload(int amount) override { ammo += amount; }
33
34     // 实现 Movable 接口
35     void move(double x, double y) override {
36         speed = 5.0;
37         std::cout << name << " 移动到 (" << x << ", " << y << ")" <<
std::endl;
38     }
39     void stop() override { speed = 0; }
40     double getSpeed() const override { return speed; }
41
42     // 实现 Robot 的纯虚函数
43     void performAction() override { shoot(); }
44
45 private:
46     int ammo;
47     double speed;
48 };

```

通过纯虚函数定义接口，可以编写不依赖具体实现的通用代码：

```

1 void fireAll(std::vector<Shootable*>& shooters) {
2     for (auto* shooter : shooters) {
3         shooter->shoot();
4     }
5 }
6
7 void moveSquad(std::vector<Movable*>& units, double x, double y) {
8     for (auto* unit : units) {
9         unit->move(x, y);
10    }
11 }

```

这些函数可以处理任何实现了相应接口的对象，而无需知道它们的具体类型。

1.5.15.5. 虚析构造函数

当通过基类指针删除派生类对象时, 如果基类析构函数不是虚函数, 只有基类的析构函数会被调用, 派生类的析构函数不会执行。这可能导致资源泄漏:

```
1 class Base {
2 public:
3     Base() { std::cout << "Base 构造" << std::endl; }
4     ~Base() { std::cout << "Base 析构" << std::endl; } // 非虚析构函数
5 };
6
7 class Derived : public Base {
8 public:
9     Derived() {
10         data = new int[100];
11         std::cout << "Derived 构造, 分配内存" << std::endl;
12     }
13     ~Derived() {
14         delete[] data;
15         std::cout << "Derived 析构, 释放内存" << std::endl;
16     }
17 private:
18     int* data;
19 };
20
21 int main() {
22     Base* ptr = new Derived();
23     delete ptr; // 只调用 Base::~~Base(), Derived::~~Derived() 不会被调用!
24     // 内存泄漏!
25     return 0;
26 }
```

输出:

```
1 Base 构造
2 Derived 构造, 分配内存
3 Base 析构
```

`Derived` 的析构函数没有被调用, `data` 指向的内存泄漏了。

解决方案是将基类析构函数声明为虚函数:

```
1 class Base {
2 public:
3     Base() { std::cout << "Base 构造" << std::endl; }
4     virtual ~Base() { std::cout << "Base 析构" << std::endl; } // 虚析构函数
5 };
```

现在输出变为:

```
1 Base 构造
2 Derived 构造, 分配内存
3 Derived 析构, 释放内存
4 Base 析构
```

派生类和基类的析构函数都被正确调用, 资源被正确释放。

这引出了一条重要的规则: 如果一个类可能被继承 (特别是可能通过基类指针删除派生类对象), 它的析构函数应该是虚函数。更简单的经验法则是: 如果类中有任何虚函数, 析构函数也应该是虚函数。

C++11 的 `override` 关键字也可以用于析构函数，尽管析构函数的覆盖是隐式的：

```
1 class Base {
2 public:
3     virtual ~Base() = default;
4 };
5
6 class Derived : public Base {
7 public:
8     ~Derived() override = default; // 显式标记覆盖
9 };
```

1.5.15.6. final 关键字

C++11 引入的 `final` 关键字可以阻止类被继承或虚函数被覆盖。

将类标记为 `final` 表示该类不能被继承：

```
1 class FinalRobot final : public Robot {
2 public:
3     FinalRobot() : Robot("终极机器人") {}
4     void performAction() override {
5         std::cout << "终极攻击! " << std::endl;
6     }
7 };
8
9 // class SuperRobot : public FinalRobot { }; // 编译错误! FinalRobot 是 final
   的
```

将虚函数标记为 `final` 表示该函数不能在派生类中被覆盖：

```
1 class Infantry : public Robot {
2 public:
3     Infantry(const std::string& name) : Robot(name) {}
4
5     void performAction() override final { // 这个实现是最终的
6         std::cout << name << " 发射弹丸" << std::endl;
7     }
8 };
9
10 class EliteInfantry : public Infantry {
11 public:
12     EliteInfantry(const std::string& name) : Infantry(name) {}
13
14     // void performAction() override { } // 编译错误! Infantry::performAction
   是 final 的
15 };
```

`final` 的用途包括：防止意外的继承或覆盖、表达设计意图、以及在某些情况下帮助编译器进行优化（编译器知道虚函数不会被进一步覆盖，可能可以去虚拟化）。

1.5.15.7. 多态的实际应用

多态在实际开发中有广泛的应用。让我们看几个 RoboMaster 相关的例子。

第一个例子是统一的目标处理系统。不同类型的目标（装甲板、能量机关、基地）需要不同的处理方式，但可以通过统一的接口进行管理：

```
1 class Target {
2 public:
```

```

3     virtual ~Target() = default;
4
5     virtual void track() = 0;           // 追踪目标
6     virtual double getPriority() const = 0; // 获取优先级
7     virtual bool isValid() const = 0;     // 检查是否有效
8
9     virtual void render(cv::Mat& frame) const = 0; // 在图像上绘制
10 };
11
12 class ArmorPlate : public Target {
13 public:
14     ArmorPlate(const cv::Rect& bbox, int color)
15         : boundingBox(bbox), armorColor(color), lastSeen(0) {}
16
17     void track() override {
18         // 装甲板追踪算法
19         std::cout << "追踪装甲板" << std::endl;
20     }
21
22     double getPriority() const override {
23         // 根据距离、角度等计算优先级
24         return 1.0 / (boundingBox.area() + 1);
25     }
26
27     bool isValid() const override {
28         return boundingBox.area() > 100;
29     }
30
31     void render(cv::Mat& frame) const override {
32         cv::rectangle(frame, boundingBox, cv::Scalar(0, 255, 0), 2);
33     }
34
35 private:
36     cv::Rect boundingBox;
37     int armorColor;
38     int64_t lastSeen;
39 };
40
41 class EnergyMechanism : public Target {
42 public:
43     void track() override {
44         // 能量机关追踪算法，与装甲板不同
45         std::cout << "追踪能量机关" << std::endl;
46     }
47
48     double getPriority() const override {
49         return 2.0; // 能量机关优先级更高
50     }
51
52     bool isValid() const override {
53         return activated;
54     }
55
56     void render(cv::Mat& frame) const override {
57         // 绘制能量机关的特殊标记
58     }
59
60 private:
61     bool activated = false;
62 };
63
64 // 统一处理所有目标
65 void processTargets(std::vector<std::unique_ptr<Target>>& targets) {

```

```

66     // 移除无效目标
67     targets.erase(
68         std::remove_if(targets.begin(), targets.end(),
69             [](const auto& t) { return !t->isValid(); }),
70         targets.end()
71     );
72
73     // 按优先级排序
74     std::sort(targets.begin(), targets.end(),
75         [](const auto& a, const auto& b) {
76             return a->getPriority() > b->getPriority();
77         });
78
79     // 追踪最高优先级目标
80     if (!targets.empty()) {
81         targets[0]->track();
82     }
83 }

```

第二个例子是控制器系统。不同的控制算法（PID、模糊控制、模型预测控制）可以通过统一接口使用：

```

1  class Controller {
2  public:
3      virtual ~Controller() = default;
4
5      virtual double compute(double setpoint, double measurement, double dt)
6      = 0;
7      virtual void reset() = 0;
8      virtual std::string getName() const = 0;
9  };
10 class PIDController : public Controller {
11 public:
12     PIDController(double kp, double ki, double kd)
13         : kp(kp), ki(ki), kd(kd), integral(0), lastError(0) {}
14
15     double compute(double setpoint, double measurement, double dt) override
16     {
17         double error = setpoint - measurement;
18         integral += error * dt;
19         double derivative = (error - lastError) / dt;
20         lastError = error;
21         return kp * error + ki * integral + kd * derivative;
22     }
23
24     void reset() override {
25         integral = 0;
26         lastError = 0;
27     }
28
29     std::string getName() const override {
30         return "PID Controller";
31     }
32 private:
33     double kp, ki, kd;
34     double integral, lastError;
35 };
36
37 class FuzzyController : public Controller {
38 public:

```

```

39     double compute(double setpoint, double measurement, double dt) override
40     {
41         // 模糊控制算法
42         return 0; // 简化
43     }
44     void reset() override { }
45
46     std::string getName() const override {
47         return "Fuzzy Controller";
48     }
49 };
50
51 // 使用控制器的代码不需要知道具体类型
52 class MotorDriver {
53 public:
54     MotorDriver(std::unique_ptr<Controller> ctrl)
55         : controller(std::move(ctrl)) {}
56
57     void setController(std::unique_ptr<Controller> ctrl) {
58         controller = std::move(ctrl);
59         controller->reset();
60         std::cout << "切换到 " << controller->getName() << std::endl;
61     }
62
63     void update(double targetSpeed, double currentSpeed, double dt) {
64         double output = controller->compute(targetSpeed, currentSpeed, dt);
65         applyOutput(output);
66     }
67
68 private:
69     void applyOutput(double output) {
70         // 应用控制输出到电机
71     }
72
73     std::unique_ptr<Controller> controller;
74 };

```

这种设计的优势在于：`MotorDriver` 可以使用任何类型的控制器，甚至可以在运行时切换控制器，而无需修改 `MotorDriver` 的代码。添加新的控制算法只需要创建新的派生类，现有代码完全不受影响。

1.5.15.8. 避免常见陷阱

使用多态时需要注意一些常见的陷阱。

第一个陷阱是在构造函数和析构函数中调用虚函数。在这两个函数中，虚函数机制不会正常工作——调用的总是当前类的版本，而非派生类的版本：

```

1 class Base {
2 public:
3     Base() {
4         initialize(); // 危险! 总是调用 Base::initialize()
5     }
6     virtual void initialize() {
7         std::cout << "Base 初始化" << std::endl;
8     }
9 };
10
11 class Derived : public Base {
12 public:

```

```

13     Derived() : Base() {}
14     void initialize() override {
15         std::cout << "Derived 初始化" << std::endl;
16     }
17 };
18
19 int main() {
20     Derived d; // 输出 "Base 初始化", 而非 "Derived 初始化"
21     return 0;
22 }

```

这是因为在 `Base` 的构造函数执行时，对象的 `Derived` 部分尚未构造，将其当作 `Derived` 对象处理是不安全的。解决方案是避免在构造函数中调用虚函数，或者使用两阶段初始化。

第二个陷阱是忘记将析构函数声明为虚函数。如前所述，这会导致通过基类指针删除派生类对象时资源泄漏。

第三个陷阱是参数类型不匹配导致隐藏而非覆盖：

```

1  class Base {
2  public:
3      virtual void process(int x) {
4          std::cout << "Base::process(int)" << std::endl;
5      }
6  };
7
8  class Derived : public Base {
9  public:
10     void process(double x) { // 注意：参数类型不同，这是隐藏，不是覆盖！
11         std::cout << "Derived::process(double)" << std::endl;
12     }
13 };
14
15 int main() {
16     Derived d;
17     Base* ptr = &d;
18     ptr->process(42); // 调用 Base::process(int)，不是
19                     // Derived::process(double)
20     return 0;
21 }

```

使用 `override` 关键字可以避免这个错误——如果签名不匹配，编译器会报错。

1.5.15.9. 性能考虑

虚函数调用比普通函数调用略慢，因为需要通过虚函数表间接调用。在大多数应用中，这点开销可以忽略不计。但在性能极其敏感的代码路径中（如每帧执行数百万次的图像处理内循环），可能需要考虑：

使用 `final` 关键字帮助编译器去虚拟化。如果编译器能确定虚函数不会被进一步覆盖，可能会将虚调用优化为直接调用。

考虑使用模板实现编译时多态（静态多态），它没有虚函数的运行时开销。

在热点代码中避免不必要的虚函数调用，可以将结果缓存或使用其他设计。

不过，在优化之前应当先进行性能分析，确认虚函数确实是瓶颈。过早优化会增加代码复杂性，得不偿失。

多态是面向对象编程的精髓。它使得代码能够以抽象的方式工作，处理“概念上的对象”而非“具体的对象”。通过虚函数和继承，我们可以编写灵活、可扩展的系统——新增功能只需添加新的派生类，而无需修改现有代码。这正是开闭原则（对扩展开放，对修改关闭）的体现。下一节将学习运算符重载，它允许我们为自定义类型定义运算符的行为，使得自定义类型能够像内置类型一样自然地使用。

1.5.16. 运算符重载

在前面的章节中，我们学习了如何定义类，如何控制对象的生命周期、拷贝行为，以及如何通过继承和多态建立类型层次结构。现在，让我们思考一个问题：对于自定义类型，如何让它们像内置类型一样自然地使用运算符？

考虑一个表示三维向量的类。我们当然可以定义 `add`、`subtract`、`multiply` 等成员函数来实现向量运算，但这样的代码读起来并不自然：

```
1 Vector3D result = v1.add(v2.multiply(scalar)); // 不够直观
```

如果能像使用内置类型那样使用运算符，代码会清晰得多：

```
1 Vector3D result = v1 + v2 * scalar; // 直观、自然
```

C++ 的运算符重载 (operator overloading) 正是为此设计的。它允许我们为自定义类型定义运算符的行为，使得自定义类型能够融入语言的表达体系，与内置类型一样自然地参与运算。这不仅提高了代码的可读性，也使得泛型编程成为可能——同样的算法可以同时作用于内置类型和自定义类型。

1.5.16.1. 运算符重载的基本语法

运算符重载本质上是定义一个特殊名称的函数，函数名由 `operator` 关键字后跟要重载的运算符组成。例如，重载加法运算符的函数名是 `operator+`，重载等于运算符的函数名是 `operator==`。

运算符可以作为成员函数或非成员函数重载。作为成员函数时，左操作数是调用对象本身；作为非成员函数时，所有操作数都通过参数传递。

让我们从一个简单的例子开始——为三维向量类重载加法运算符：

```
1 class Vector3D {
2 public:
3     Vector3D(double x = 0, double y = 0, double z = 0)
4         : x(x), y(y), z(z) {}
5
6     // 作为成员函数重载 +
7     Vector3D operator+(const Vector3D& other) const {
8         return Vector3D(x + other.x, y + other.y, z + other.z);
9     }
10
11     double getX() const { return x; }
12     double getY() const { return y; }
13     double getZ() const { return z; }
14
15 private:
16     double x, y, z;
17 };
18
19 int main() {
```

```

20     Vector3D v1(1.0, 2.0, 3.0);
21     Vector3D v2(4.0, 5.0, 6.0);
22
23     Vector3D v3 = v1 + v2; // 调用 v1.operator+(v2)
24
25     std::cout << "v3 = (" << v3.getX() << ", " << v3.getY() << ", " <<
v3.getZ() << ")" << std::endl;
26     // 输出: v3 = (5, 7, 9)
27
28     return 0;
29 }

```

当编译器遇到 `v1 + v2` 时，它会将其转换为 `v1.operator+(v2)` 的函数调用。成员函数版本的 `operator+` 接受一个参数（右操作数），左操作数就是调用该函数的对象（`*this`）。

注意 `operator+` 被声明为 `const` 成员函数，因为加法运算不应该修改任何一个操作数。它返回一个新的 `Vector3D` 对象，包含运算结果。这与内置类型的加法行为一致——`a + b` 不会修改 `a` 或 `b`。

1.5.16.2. 算术运算符

算术运算符是最常重载的一类运算符。除了加法，我们还可以重载减法、乘法、除法等。以下是 `Vector3D` 类的完整算术运算符实现：

```

1  class Vector3D {
2  public:
3      Vector3D(double x = 0, double y = 0, double z = 0)
4          : x(x), y(y), z(z) {}
5
6      // 向量加法
7      Vector3D operator+(const Vector3D& other) const {
8          return Vector3D(x + other.x, y + other.y, z + other.z);
9      }
10
11     // 向量减法
12     Vector3D operator-(const Vector3D& other) const {
13         return Vector3D(x - other.x, y - other.y, z - other.z);
14     }
15
16     // 标量乘法（向量 * 标量）
17     Vector3D operator*(double scalar) const {
18         return Vector3D(x * scalar, y * scalar, z * scalar);
19     }
20
21     // 标量除法
22     Vector3D operator/(double scalar) const {
23         return Vector3D(x / scalar, y / scalar, z / scalar);
24     }
25
26     // 点积（使用 * 运算符，两个向量相乘）
27     double operator*(const Vector3D& other) const {
28         return x * other.x + y * other.y + z * other.z;
29     }
30
31     // 一元负号（取反）
32     Vector3D operator-() const {
33         return Vector3D(-x, -y, -z);
34     }
35
36 private:

```



```

37     double x, y, z;
38 };
39
40 int main() {
41     Vector3D v1(1.0, 2.0, 3.0);
42     Vector3D v2(4.0, 5.0, 6.0);
43
44     Vector3D sum = v1 + v2;           // (5, 7, 9)
45     Vector3D diff = v1 - v2;         // (-3, -3, -3)
46     Vector3D scaled = v1 * 2.0;      // (2, 4, 6)
47     Vector3D divided = v2 / 2.0;     // (2, 2.5, 3)
48     double dot = v1 * v2;            // 1*4 + 2*5 + 3*6 = 32
49     Vector3D negated = -v1;          // (-1, -2, -3)
50
51     return 0;
52 }

```

注意一元负号运算符 `operator-()` 不接受参数，它作用于单个操作数（调用对象本身）。这与二元减法运算符 `operator-(const Vector3D&)` 是不同的重载。

上面的标量乘法 `v1 * 2.0` 可以工作，但 `2.0 * v1` 却不行——因为成员函数版本要求左操作数是 `Vector3D` 类型。要支持 `2.0 * v1` 这种写法，需要定义非成员函数版本的运算符。

1.5.16.3. 友元函数

非成员函数不能直接访问类的私有成员。如果运算符函数需要访问私有成员，有两种选择：通过公开的 `getter` 函数访问，或者将运算符函数声明为类的友元（`friend`）。

友元函数不是类的成员，但被授予了访问类私有成员的权限。在类定义内部使用 `friend` 关键字声明友元：

```

1  class Vector3D {
2  public:
3      Vector3D(double x = 0, double y = 0, double z = 0)
4          : x(x), y(y), z(z) {}
5
6      // 成员函数版本：向量 * 标量
7      Vector3D operator*(double scalar) const {
8          return Vector3D(x * scalar, y * scalar, z * scalar);
9      }
10
11     // 友元函数声明：标量 * 向量
12     friend Vector3D operator*(double scalar, const Vector3D& v);
13
14 private:
15     double x, y, z;
16 };
17
18 // 友元函数定义（在类外部）
19 Vector3D operator*(double scalar, const Vector3D& v) {
20     return Vector3D(v.x * scalar, v.y * scalar, v.z * scalar); // 可以访问私有成员
21 }
22
23 int main() {
24     Vector3D v(1.0, 2.0, 3.0);
25
26     Vector3D r1 = v * 2.0;           // 调用成员函数 v.operator*(2.0)
27     Vector3D r2 = 2.0 * v;           // 调用友元函数 operator*(2.0, v)
28
29     return 0;

```

友元声明可以放在类的任何访问区域（`public`、`private` 或 `protected`），访问说明符对友元声明没有影响——友元总是可以访问所有成员。

一种常见的做法是在类内部直接定义友元函数，这样可以将声明和定义放在一起：

```
1 class Vector3D {
2 public:
3     // ...
4
5     // 在类内部定义友元函数
6     friend Vector3D operator*(double scalar, const Vector3D& v) {
7         return Vector3D(v.x * scalar, v.y * scalar, v.z * scalar);
8     }
9
10 private:
11     double x, y, z;
12 };
```

虽然这个函数定义在类的花括号内，但它仍然是非成员函数。`friend` 关键字既声明了友元关系，又允许在此处定义函数。

1.5.16.4. 比较运算符

比较运算符用于判断两个对象之间的关系。最常用的是相等（`==`）和不等（`!=`）运算符：

```
1 class Vector3D {
2 public:
3     Vector3D(double x = 0, double y = 0, double z = 0)
4         : x(x), y(y), z(z) {}
5
6     bool operator==(const Vector3D& other) const {
7         const double epsilon = 1e-9;
8         return std::abs(x - other.x) < epsilon &&
9             std::abs(y - other.y) < epsilon &&
10             std::abs(z - other.z) < epsilon;
11     }
12
13     bool operator!=(const Vector3D& other) const {
14         return !(*this == other); // 利用已定义的 ==
15     }
16
17 private:
18     double x, y, z;
19 };
```

对于浮点数比较，直接使用 `==` 可能因精度问题产生错误结果，因此使用一个小的容差值（`epsilon`）进行比较。`operator!=` 通常可以基于 `operator==` 实现，避免代码重复。

对于需要排序的类型，还需要重载关系运算符。例如，表示分数的类：

```
1 class Fraction {
2 public:
3     Fraction(int num = 0, int den = 1) : numerator(num), denominator(den) {
4         normalize();
5     }
6
7     bool operator==(const Fraction& other) const {
```

```

8         return numerator == other.numerator && denominator ==
other.denominator;
9     }
10
11     bool operator!=(const Fraction& other) const {
12         return !(*this == other);
13     }
14
15     bool operator<(const Fraction& other) const {
16         return numerator * other.denominator < other.numerator *
denominator;
17     }
18
19     bool operator>(const Fraction& other) const {
20         return other < *this;
21     }
22
23     bool operator<=(const Fraction& other) const {
24         return !(other < *this);
25     }
26
27     bool operator>=(const Fraction& other) const {
28         return !(*this < other);
29     }
30
31 private:
32     void normalize(); // 约分并处理符号
33     int numerator;
34     int denominator;
35 };

```

C++20 引入了三路比较运算符（`<=>`，也称为“太空船运算符”），可以一次性定义所有比较运算符。但在 C++20 之前，通常需要手动实现六个比较运算符，其中 `==` 和 `<` 是基础，其他四个可以基于它们实现。

1.5.16.5. 复合赋值运算符

复合赋值运算符（如 `+=`、`-=`、`*=`）结合了算术运算和赋值。它们修改左操作数并返回其引用：

```

1 class Vector3D {
2 public:
3     // ...
4
5     Vector3D& operator+=(const Vector3D& other) {
6         x += other.x;
7         y += other.y;
8         z += other.z;
9         return *this;
10    }
11
12    Vector3D& operator-=(const Vector3D& other) {
13        x -= other.x;
14        y -= other.y;
15        z -= other.z;
16        return *this;
17    }
18
19    Vector3D& operator*=(double scalar) {
20        x *= scalar;

```

```

21     y *= scalar;
22     z *= scalar;
23     return *this;
24 }
25
26 Vector3D& operator/=(double scalar) {
27     x /= scalar;
28     y /= scalar;
29     z /= scalar;
30     return *this;
31 }
32
33 private:
34     double x, y, z;
35 };

```

返回 `*this` 的引用允许链式操作：`v1 += v2 += v3`（从右向左执行）。

一种良好的实践是基于复合赋值运算符实现普通算术运算符，这样可以避免代码重复：

```

1  class Vector3D {
2  public:
3      // 复合赋值运算符（修改自身）
4      Vector3D& operator+=(const Vector3D& other) {
5          x += other.x;
6          y += other.y;
7          z += other.z;
8          return *this;
9      }
10
11     // 基于 += 实现 +
12     Vector3D operator+(const Vector3D& other) const {
13         Vector3D result = *this; // 复制当前对象
14         result += other;         // 使用 +=
15         return result;
16     }
17
18     // 或者使用更简洁的写法
19     Vector3D operator-(const Vector3D& other) const {
20         return Vector3D(*this) -= other;
21     }
22
23 private:
24     double x, y, z;
25 };

```

这种方式确保了 `+` 和 `+=` 的行为一致，修改一处逻辑时另一处自动更新。

1.5.16.6. 下标运算符

下标运算符 `operator[]` 使对象能够像数组一样通过索引访问元素。它必须作为成员函数定义，通常需要提供 `const` 和非 `const` 两个版本：

```

1  class Matrix {
2  public:
3      Matrix(size_t rows, size_t cols)
4          : rows(rows), cols(cols), data(rows * cols, 0.0) {}
5
6      // 非 const 版本：可以修改元素
7      double& operator()(size_t row, size_t col) {
8          return data[row * cols + col];
9      }
9

```

```

9     }
10
11     // const 版本: 只读访问
12     const double& operator()(size_t row, size_t col) const {
13         return data[row * cols + col];
14     }
15
16     size_t getRows() const { return rows; }
17     size_t getCols() const { return cols; }
18
19 private:
20     size_t rows, cols;
21     std::vector<double> data;
22 };
23
24 int main() {
25     Matrix m(3, 3);
26
27     m(0, 0) = 1.0; // 使用非 const 版本
28     m(1, 1) = 1.0;
29     m(2, 2) = 1.0;
30
31     const Matrix& cm = m;
32     double val = cm(0, 0); // 使用 const 版本
33     // cm(0, 0) = 2.0;      // 错误! const 版本返回 const 引用
34
35     return 0;
36 }

```

注意这里使用了 `operator()` 而非 `operator[]`，因为 `operator[]` 只能接受一个参数，而矩阵访问需要行和列两个索引。使用函数调用运算符 `operator()` 可以接受任意数量的参数。

对于一维容器，可以使用标准的 `operator[]`：

```

1 class DynamicArray {
2 public:
3     DynamicArray(size_t size) : data(size) {}
4
5     double& operator[](size_t index) {
6         return data[index];
7     }
8
9     const double& operator[](size_t index) const {
10        return data[index];
11    }
12
13    size_t size() const { return data.size(); }
14
15 private:
16    std::vector<double> data;
17 };

```

是否进行边界检查是一个设计选择。标准库的 `vector::operator[]` 不检查边界（为了性能），而 `vector::at()` 会检查并在越界时抛出异常。自定义类型可以根据需求选择合适的策略。

1.5.16.7. 输入输出运算符

流插入运算符 `<<` 和流提取运算符 `>>` 使得自定义类型可以与 `iostream` 无缝配合。由于左操作数是流对象（`std::ostream` 或 `std::istream`），这些运算符必须作为非成员函数实现：

```
1 class Vector3D {
2 public:
3     Vector3D(double x = 0, double y = 0, double z = 0)
4         : x(x), y(y), z(z) {}
5
6     // 声明友元
7     friend std::ostream& operator<<(std::ostream& os, const Vector3D& v);
8     friend std::istream& operator>>(std::istream& is, Vector3D& v);
9
10 private:
11     double x, y, z;
12 };
13
14 // 输出运算符
15 std::ostream& operator<<(std::ostream& os, const Vector3D& v) {
16     os << "(" << v.x << ", " << v.y << ", " << v.z << ")";
17     return os;
18 }
19
20 // 输入运算符
21 std::istream& operator>>(std::istream& is, Vector3D& v) {
22     is >> v.x >> v.y >> v.z;
23     return is;
24 }
25
26 int main() {
27     Vector3D v1(1.0, 2.0, 3.0);
28     std::cout << "v1 = " << v1 << std::endl; // 输出: v1 = (1, 2, 3)
29
30     Vector3D v2;
31     std::cout << "输入向量 (x y z): ";
32     std::cin >> v2;
33     std::cout << "你输入的向量是: " << v2 << std::endl;
34
35     return 0;
36 }
```

输出运算符接受 `const` 引用参数，因为输出不应修改对象；输入运算符接受非 `const` 引用，因为需要修改对象。两者都返回流的引用，以支持链式调用：`std::cout << v1 << " and " << v2`。

输入运算符还应该处理输入失败的情况。更健壮的实现会检查流状态：

```
1 std::istream& operator>>(std::istream& is, Vector3D& v) {
2     double x, y, z;
3     if (is >> x >> y >> z) {
4         v = Vector3D(x, y, z);
5     }
6     // 如果输入失败，流的失败状态会保持，v 不变
7     return is;
8 }
```

1.5.16.8. 递增递减运算符

递增 `++` 和递减 `--` 运算符有前置和后置两种形式，它们的行为不同：前置返回修改后的值，后置返回修改前的值。为了区分这两种形式，C++ 使用一个哑参数 (dummy parameter)：

```
1  class Counter {
2  public:
3      Counter(int value = 0) : value(value) {}
4
5      // 前置递增: ++c
6      Counter& operator++() {
7          ++value;
8          return *this;
9      }
10
11     // 后置递增: c++
12     Counter operator++(int) { // int 参数仅用于区分, 不使用
13         Counter old = *this; // 保存旧值
14         ++(*this);           // 使用前置递增
15         return old;          // 返回旧值
16     }
17
18     // 前置递减: --c
19     Counter& operator--() {
20         --value;
21         return *this;
22     }
23
24     // 后置递减: c--
25     Counter operator--(int) {
26         Counter old = *this;
27         --(*this);
28         return old;
29     }
30
31     int getValue() const { return value; }
32
33 private:
34     int value;
35 };
36
37 int main() {
38     Counter c(5);
39
40     std::cout << ++c.getValue() << std::endl; // 6 (先加后用)
41     std::cout << c++.getValue() << std::endl; // 6 (先用后加, 但这里有问题...)
42
43     // 正确的测试方式
44     Counter c1(5);
45     Counter c2 = ++c1; // c1 = 6, c2 = 6
46     Counter c3 = c1++; // c1 = 7, c3 = 6 (c3 是旧值)
47
48     return 0;
49 }
```

前置版本返回引用，效率更高；后置版本必须创建副本保存旧值，然后返回这个副本。这就是为什么在不需要旧值的情况下，`++i` 比 `i++` 更推荐使用。

后置版本通常基于前置版本实现，确保行为一致。

1.5.16.9. 类型转换运算符

类型转换运算符允许对象隐式或显式转换为其他类型。转换运算符的名称是 `operator` 后跟目标类型，没有返回类型声明（因为返回类型就是运算符名称中的类型）：

```
1 class Fraction {
2 public:
3     Fraction(int num = 0, int den = 1) : numerator(num), denominator(den) {}
4
5     // 转换为 double
6     operator double() const {
7         return static_cast<double>(numerator) / denominator;
8     }
9
10    // 转换为 bool (判断是否为零)
11    explicit operator bool() const {
12        return numerator != 0;
13    }
14
15 private:
16     int numerator;
17     int denominator;
18 };
19
20 int main() {
21     Fraction f(3, 4);
22
23     double d = f; // 隐式转换, d = 0.75
24     double e = static_cast<double>(f); // 显式转换
25
26     if (f) { // explicit 转换在条件语句中可以隐式使用
27         std::cout << "分数非零" << std::endl;
28     }
29
30     // bool b = f; // 错误! explicit 禁止了这种隐式转换
31     bool b = static_cast<bool>(f); // 正确, 显式转换
32
33     return 0;
34 }
```

`explicit` 关键字防止意外的隐式转换，这是 C++11 之后的推荐做法。不加 `explicit` 的转换运算符可能导致意想不到的类型转换，引发难以调试的问题。

除了定义从自定义类型到其他类型的转换，还可以通过接受单个参数的构造函数定义从其他类型到自定义类型的转换：

```
1 class Vector3D {
2 public:
3     // 从 double 转换: 创建 (d, d, d) 向量
4     Vector3D(double d) : x(d), y(d), z(d) {}
5
6     // 阻止隐式转换
7     explicit Vector3D(int i) : x(i), y(i), z(i) {}
8
9 private:
10     double x, y, z;
11 };
12
13 int main() {
14     Vector3D v1 = 1.5; // 隐式转换, 调用 Vector3D(double)
15     Vector3D v2(3.0); // 直接构造
16 }
```



```

17     // Vector3D v3 = 42; // 错误! Vector3D(int) 是 explicit 的
18     Vector3D v4(42);    // 正确, 显式构造
19
20     return 0;
21 }

```

1.5.16.10. 函数调用运算符

函数调用运算符 `operator()` 使对象可以像函数一样被调用。具有这种能力的对象称为函数对象或仿函数 (functor):

```

1  class Adder {
2  public:
3      Adder(int increment) : increment(increment) {}
4
5      int operator()(int value) const {
6          return value + increment;
7      }
8
9  private:
10     int increment;
11 };
12
13 class DistanceCalculator {
14 public:
15     DistanceCalculator(double x, double y, double z)
16         : originX(x), originY(y), originZ(z) {}
17
18     double operator()(double x, double y, double z) const {
19         double dx = x - originX;
20         double dy = y - originY;
21         double dz = z - originZ;
22         return std::sqrt(dx*dx + dy*dy + dz*dz);
23     }
24
25 private:
26     double originX, originY, originZ;
27 };
28
29 int main() {
30     Adder addFive(5);
31     std::cout << addFive(10) << std::endl; // 15
32     std::cout << addFive(20) << std::endl; // 25
33
34     DistanceCalculator distFromOrigin(0, 0, 0);
35     std::cout << distFromOrigin(3, 4, 0) << std::endl; // 5
36
37     return 0;
38 }

```

函数对象比普通函数更灵活, 因为它们可以携带状态 (如上例中的 `increment` 和原点坐标)。它们在标准库算法中广泛使用:

```

1  std::vector<int> numbers = {1, 2, 3, 4, 5};
2
3  // 使用函数对象作为谓词
4  struct IsEven {
5      bool operator()(int n) const {
6          return n % 2 == 0;
7      }

```

```

8 };
9
10 auto it = std::find_if(numbers.begin(), numbers.end(), IsEven());
11
12 // 使用函数对象进行变换
13 Adder addTen(10);
14 std::transform(numbers.begin(), numbers.end(), numbers.begin(), addTen);
15 // numbers 现在是 {11, 12, 13, 14, 15}

```

C++11 引入的 lambda 表达式在很多场景下可以替代函数对象，但函数对象在需要复杂状态或多次复用时仍然有用。

1.5.16.11. 运算符重载的原则与限制

并非所有运算符都可以重载。以下运算符不能重载：

- `::` 作用域解析运算符
- `.` 成员访问运算符
- `.*` 成员指针访问运算符
- `?:` 条件运算符
- `sizeof` 大小运算符
- `typeid` 类型信息运算符

此外，还有一些重要的设计原则：

不要改变运算符的基本语义。`+` 应该表示某种“相加”的概念，`==` 应该表示某种“相等”的概念。如果 `a + b` 的含义与加法毫无关系，会让代码极其难以理解。

保持与内置类型的一致性。如果 `a + b` 有效，通常 `a += b` 也应该有效且行为一致。如果 `a == b`，那么 `a != b` 应该返回相反的结果。

对于二元运算符，考虑是否需要支持混合类型操作。例如，`Vector3D * double` 和 `double * Vector3D` 都应该有效。

优先使用成员函数还是非成员函数？一般原则是：

- 赋值（`=`）、下标（`[]`）、调用（`()`）、成员访问（`->`）必须是成员函数
- 复合赋值运算符（`+=` 等）通常是成员函数
- 算术运算符（`+` 等）如果需要左操作数可以隐式转换，应该是非成员函数
- 输入输出运算符（`<<`、`>>`）必须是非成员函数

1.5.16.12. 一个完整的例子

让我们综合运用本节的知识，实现一个完整的复数类：

```

1  #include <iostream>
2  #include <cmath>
3
4  class Complex {
5  public:
6      // 构造函数
7      Complex(double real = 0, double imag = 0)
8          : real(real), imag(imag) {}
9
10     // Getter
11     double getReal() const { return real; }
12     double getImag() const { return imag; }
13

```

```

14 // 模 (绝对值)
15 double abs() const {
16     return std::sqrt(real * real + imag * imag);
17 }
18
19 // 复合赋值运算符
20 Complex& operator+=(const Complex& other) {
21     real += other.real;
22     imag += other.imag;
23     return *this;
24 }
25
26 Complex& operator-=(const Complex& other) {
27     real -= other.real;
28     imag -= other.imag;
29     return *this;
30 }
31
32 Complex& operator*=(const Complex& other) {
33     double newReal = real * other.real - imag * other.imag;
34     double newImag = real * other.imag + imag * other.real;
35     real = newReal;
36     imag = newImag;
37     return *this;
38 }
39
40 Complex& operator/=(const Complex& other) {
41     double denom = other.real * other.real + other.imag * other.imag;
42     double newReal = (real * other.real + imag * other.imag) / denom;
43     double newImag = (imag * other.real - real * other.imag) / denom;
44     real = newReal;
45     imag = newImag;
46     return *this;
47 }
48
49 // 一元运算符
50 Complex operator-() const {
51     return Complex(-real, -imag);
52 }
53
54 Complex operator+() const {
55     return *this;
56 }
57
58 // 比较运算符
59 bool operator==(const Complex& other) const {
60     const double epsilon = 1e-9;
61     return std::abs(real - other.real) < epsilon &&
62           std::abs(imag - other.imag) < epsilon;
63 }
64
65 bool operator!=(const Complex& other) const {
66     return !(*this == other);
67 }
68
69 // 友元声明
70 friend Complex operator+(const Complex& a, const Complex& b);
71 friend Complex operator-(const Complex& a, const Complex& b);
72 friend Complex operator*(const Complex& a, const Complex& b);
73 friend Complex operator/(const Complex& a, const Complex& b);
74 friend std::ostream& operator<<(std::ostream& os, const Complex& c);
75 friend std::istream& operator>>(std::istream& is, Complex& c);
76

```

```

77 private:
78     double real;
79     double imag;
80 };
81
82 // 算术运算符（非成员函数，支持隐式转换）
83 Complex operator+(const Complex& a, const Complex& b) {
84     return Complex(a.real + b.real, a.imag + b.imag);
85 }
86
87 Complex operator-(const Complex& a, const Complex& b) {
88     return Complex(a.real - b.real, a.imag - b.imag);
89 }
90
91 Complex operator*(const Complex& a, const Complex& b) {
92     return Complex(a.real * b.real - a.imag * b.imag,
93                     a.real * b.imag + a.imag * b.real);
94 }
95
96 Complex operator/(const Complex& a, const Complex& b) {
97     double denom = b.real * b.real + b.imag * b.imag;
98     return Complex((a.real * b.real + a.imag * b.imag) / denom,
99                     (a.imag * b.real - a.real * b.imag) / denom);
100 }
101
102 // 输入输出运算符
103 std::ostream& operator<<(std::ostream& os, const Complex& c) {
104     os << c.real;
105     if (c.imag >= 0) os << "+";
106     os << c.imag << "i";
107     return os;
108 }
109
110 std::istream& operator>>(std::istream& is, Complex& c) {
111     is >> c.real >> c.imag;
112     return is;
113 }
114
115 int main() {
116     Complex c1(3, 4);
117     Complex c2(1, -2);
118
119     std::cout << "c1 = " << c1 << std::endl;           // 3+4i
120     std::cout << "c2 = " << c2 << std::endl;           // 1-2i
121     std::cout << "|c1| = " << c1.abs() << std::endl;    // 5
122
123     std::cout << "c1 + c2 = " << (c1 + c2) << std::endl; // 4+2i
124     std::cout << "c1 - c2 = " << (c1 - c2) << std::endl; // 2+6i
125     std::cout << "c1 * c2 = " << (c1 * c2) << std::endl; // 11-2i
126     std::cout << "c1 / c2 = " << (c1 / c2) << std::endl; // -1+2i
127
128     // 与实数混合运算（利用隐式转换）
129     Complex c3 = c1 + 5.0; // 5.0 隐式转换为 Complex(5, 0)
130     std::cout << "c1 + 5 = " << c3 << std::endl; // 8+4i
131
132     Complex c4 = 2.0 * c1; // 同样利用隐式转换
133     std::cout << "2 * c1 = " << c4 << std::endl; // 6+8i
134
135     return 0;
136 }

```

这个复数类展示了运算符重载的多个方面：算术运算符、复合赋值运算符、比较运算符、一元运算符、输入输出运算符，以及通过友元函数实现的非成员运算符。它像内置类型一样自然地使用，同时保持了良好的封装。

运算符重载是 C++ 强大而灵活的特性。合理使用它可以使代码更加直观和优雅，但过度或不当使用会让代码变得难以理解。关键在于保持运算符的自然语义，让使用者能够凭直觉正确使用你的类型。下一节我们将学习 STL 容器，这些标准库类型正是运算符重载的典范——它们通过重载下标、迭代器运算符等，提供了直观而强大的接口。

1.5.17. STL 容器

在前面的章节中，我们学习了数组——一种存储固定数量同类型元素的基本数据结构。然而，实际开发中常常面临更复杂的需求：元素数量在运行时才能确定、需要频繁在中间位置插入删除、需要快速查找某个元素是否存在、需要按键值对存储数据。如果每次都从头实现这些数据结构，不仅费时费力，还容易出错。

C++ 标准模板库（Standard Template Library, STL）提供了一套精心设计、经过充分测试的容器类，涵盖了最常用的数据结构。这些容器不仅功能强大、性能优异，而且接口统一、易于学习。掌握 STL 容器是每个 C++ 程序员的必备技能，它们将成为你日常开发中最常使用的工具。

1.5.17.1. 容器的分类

STL 容器可以分为三大类：

顺序容器 (Sequence Containers) 按照元素插入的顺序存储数据，包括 `vector`、`deque`、`list`、`array`、`forward_list` 等。它们的主要区别在于内存布局和各种操作的性能特征。

关联容器 (Associative Containers) 按照键的顺序存储数据，内部使用平衡二叉树实现，包括 `set`、`map`、`multiset`、`multimap`。它们提供基于键的快速查找，元素按键自动排序。

无序关联容器 (Unordered Associative Containers) 使用哈希表实现，包括 `unordered_set`、`unordered_map`、`unordered_multiset`、`unordered_multimap`。它们提供平均常数时间的查找，但元素没有特定顺序。

此外还有容器适配器 (Container Adapters)，如 `stack`、`queue`、`priority_queue`，它们基于其他容器实现，提供特定的访问模式。

让我们从最常用的容器开始，逐一探索它们的特点和用法。

1.5.17.2. vector：动态数组

`std::vector` 是最常用的 STL 容器，可以看作是能够自动增长的数组。它在内存中连续存储元素，支持随机访问，在末尾添加和删除元素非常高效。

使用 `vector` 需要包含 `<vector>` 头文件：

```
1 #include <vector>
2 #include <iostream>
3
4 int main() {
5     // 创建空 vector
6     std::vector<int> numbers;
7
8     // 创建包含 5 个元素的 vector，初始值为 0
9     std::vector<int> zeros(5);
10 }
```

```

11 // 创建包含 5 个元素的 vector, 初始值为 10
12 std::vector<int> tens(5, 10);
13
14 // 使用初始化列表创建
15 std::vector<int> primes = {2, 3, 5, 7, 11};
16
17 // 复制另一个 vector
18 std::vector<int> primesCopy = primes;
19
20 return 0;
21 }

```

`vector` 模板参数指定元素类型。`std::vector<int>` 存储整数, `std::vector<double>` 存储双精度浮点数, `std::vector<std::string>` 存储字符串, `std::vector<Target>` 存储自定义的 `Target` 对象。

访问元素可以使用下标运算符或 `at()` 方法。下标运算符不检查边界, `at()` 会在越界时抛出异常:

```

1 std::vector<int> v = {10, 20, 30, 40, 50};
2
3 int a = v[0]; // 10, 不检查边界
4 int b = v.at(1); // 20, 检查边界
5
6 v[2] = 35; // 修改元素
7 // v[10] = 100; // 未定义行为! 越界但不报错
8 // v.at(10) = 100; // 抛出 std::out_of_range 异常
9
10 int first = v.front(); // 第一个元素
11 int last = v.back(); // 最后一个元素

```

在 `vector` 末尾添加和删除元素是最常用的操作:

```

1 std::vector<int> v;
2
3 v.push_back(10); // 添加到末尾: {10}
4 v.push_back(20); // {10, 20}
5 v.push_back(30); // {10, 20, 30}
6
7 v.pop_back(); // 移除末尾元素: {10, 20}
8
9 // C++11 引入的 emplace_back, 直接在容器内构造对象, 避免拷贝
10 v.emplace_back(40); // {10, 20, 40}

```

`push_back` 和 `pop_back` 的时间复杂度是摊销常数 (amortized constant), 非常高效。这是因为 `vector` 会预先分配额外的内存空间, 只有当空间用完时才需要重新分配。

`vector` 的大小和容量是两个不同的概念。大小 (size) 是当前存储的元素数量, 容量 (capacity) 是不需要重新分配内存就能存储的最大元素数量:

```

1 std::vector<int> v;
2
3 std::cout << "Size: " << v.size() << std::endl; // 0
4 std::cout << "Capacity: " << v.capacity() << std::endl; // 可能是 0
5
6 v.reserve(100); // 预分配空间, 避免频繁重新分配
7 std::cout << "After reserve - Size: " << v.size() << std::endl; // 仍然是 0
8 std::cout << "After reserve - Capacity: " << v.capacity() << std::endl; // 至少 100

```

```

9
10 for (int i = 0; i < 50; i++) {
11     v.push_back(i);
12 }
13 std::cout << "After pushing - Size: " << v.size() << std::endl;    // 50
14 std::cout << "After pushing - Capacity: " << v.capacity() << std::endl; //
    仍然至少 100
15
16 v.shrink_to_fit(); // 释放多余的容量（非强制）

```

在 RoboMaster 开发中，如果预先知道大致需要多少元素，使用 `reserve()` 预分配空间可以避免多次内存重新分配，提高性能。

遍历 `vector` 有多种方式：

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 // 方式一：基于索引
4 for (size_t i = 0; i < v.size(); i++) {
5     std::cout << v[i] << " ";
6 }
7
8 // 方式二：范围 for 循环（推荐）
9 for (int x : v) {
10     std::cout << x << " ";
11 }
12
13 // 方式三：使用引用避免拷贝（对于大型元素）
14 for (const int& x : v) {
15     std::cout << x << " ";
16 }
17
18 // 方式四：使用迭代器
19 for (auto it = v.begin(); it != v.end(); ++it) {
20     std::cout << *it << " ";
21 }

```

在中间位置插入和删除元素需要移动后续所有元素，时间复杂度是 $O(n)$ ：

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 // 在位置 2（第三个元素前）插入 100
4 v.insert(v.begin() + 2, 100); // {1, 2, 100, 3, 4, 5}
5
6 // 删除位置 3 的元素
7 v.erase(v.begin() + 3); // {1, 2, 100, 4, 5}
8
9 // 删除范围 [begin+1, begin+3)
10 v.erase(v.begin() + 1, v.begin() + 3); // {1, 4, 5}
11
12 // 清空所有元素
13 v.clear(); // {}
14 std::cout << "Empty: " << v.empty() << std::endl; // true

```

1.5.17.3. deque：双端队列

`std::deque`（double-ended queue）支持在两端高效地插入和删除元素。与 `vector` 不同，`deque` 的内存不是完全连续的，它由多个固定大小的内存块组成。

```

1 #include <deque>

```

```

2
3 std::deque<int> dq = {2, 3, 4};
4
5 // 在两端操作都是高效的
6 dq.push_front(1); // {1, 2, 3, 4}
7 dq.push_back(5);  // {1, 2, 3, 4, 5}
8
9 dq.pop_front();    // {2, 3, 4, 5}
10 dq.pop_back();     // {2, 3, 4}
11
12 // 随机访问
13 int x = dq[1];      // 3
14 int y = dq.at(2);   // 4

```

`deque` 与 `vector` 的主要区别在于：`deque` 在头部插入删除是常数时间，`vector` 则是线性时间；`deque` 的内存不完全连续，不能直接将其数据传给需要连续内存的 C 函数；`deque` 没有 `capacity()` 和 `reserve()` 方法。

在需要频繁在两端操作的场景下，`deque` 比 `vector` 更合适。

1.5.17.4. list: 双向链表

`std::list` 是双向链表，每个元素都包含指向前一个和后一个元素的指针。它支持在任意位置常数时间的插入和删除，但不支持随机访问。

```

1 #include <list>
2
3 std::list<int> lst = {1, 2, 3, 4, 5};
4
5 // 在两端操作
6 lst.push_front(0); // {0, 1, 2, 3, 4, 5}
7 lst.push_back(6);  // {0, 1, 2, 3, 4, 5, 6}
8
9 // 不支持随机访问
10 // int x = lst[2]; // 错误! list 没有 operator[]
11
12 // 必须通过迭代器访问
13 auto it = lst.begin();
14 std::advance(it, 2); // 移动到第三个元素
15 std::cout << *it << std::endl; // 2
16
17 // 在任意位置插入删除都是常数时间（找到位置后）
18 lst.insert(it, 100); // 在 it 前插入 100
19 lst.erase(it);      // 删除 it 指向的元素

```

`list` 提供了一些特有的操作：

```

1 std::list<int> lst1 = {1, 3, 5};
2 std::list<int> lst2 = {2, 4, 6};
3
4 // 合并两个已排序的链表
5 lst1.merge(lst2); // lst1 = {1, 2, 3, 4, 5, 6}, lst2 = {}
6
7 // 排序 (list 不能使用 std::sort, 必须用成员函数)
8 std::list<int> unsorted = {5, 2, 8, 1, 9};
9 unsorted.sort(); // {1, 2, 5, 8, 9}
10
11 // 去除连续重复元素
12 std::list<int> dups = {1, 1, 2, 2, 2, 3, 3};
13 dups.unique(); // {1, 2, 3}
14

```



```

15 // 移除特定值
16 std::list<int> vals = {1, 2, 3, 2, 4, 2};
17 vals.remove(2); // {1, 3, 4}
18
19 // 翻转
20 std::list<int> rev = {1, 2, 3};
21 rev.reverse(); // {3, 2, 1}

```

`list` 适用于需要频繁在中间插入删除的场景，但由于缓存不友好（元素在内存中不连续），遍历性能不如 `vector`。

1.5.17.5. set: 有序集合

`std::set` 存储唯一元素的有序集合。它基于红黑树实现，查找、插入、删除的时间复杂度都是 $O(\log n)$ 。元素按照比较函数（默认是 `<`）自动排序。

```

1  #include <set>
2
3  std::set<int> s;
4
5  // 插入元素
6  s.insert(30);
7  s.insert(10);
8  s.insert(20);
9  s.insert(10); // 重复元素，不会插入
10
11 // 遍历（自动排序）
12 for (int x : s) {
13     std::cout << x << " "; // 10 20 30
14 }
15 std::cout << std::endl;
16
17 // 查找
18 auto it = s.find(20);
19 if (it != s.end()) {
20     std::cout << "找到: " << *it << std::endl;
21 }
22
23 // 检查是否存在
24 if (s.count(20) > 0) { // count 返回 0 或 1
25     std::cout << "20 存在" << std::endl;
26 }
27
28 // C++20 提供了更直观的 contains
29 // if (s.contains(20)) { ... }
30
31 // 删除
32 s.erase(20); // 删除值为 20 的元素
33 s.erase(s.begin()); // 删除第一个元素
34
35 std::cout << "Size: " << s.size() << std::endl;

```

`set` 中的元素一旦插入就不能修改（因为修改可能破坏排序），只能删除后重新插入。

在 RoboMaster 开发中，`set` 可用于存储已处理的目标 ID、维护不重复的事件列表等：

```

1  std::set<int> processedTargetIds;
2
3  void processTarget(const Target& target) {
4      // 检查是否已处理过
5      if (processedTargetIds.count(target.id) > 0) {

```

```

6         return; // 已处理, 跳过
7     }
8
9     // 处理目标...
10
11     processedTargetIds.insert(target.id);
12 }

```

1.5.17.6. map: 有序键值对

`std::map` 存储键值对 (key-value pairs), 按键排序。每个键唯一, 对应一个值。它同样基于红黑树实现, 各操作时间复杂度为 $O(\log n)$ 。

```

1  #include <map>
2
3  std::map<std::string, int> ages;
4
5  // 插入键值对
6  ages["Alice"] = 25;
7  ages["Bob"] = 30;
8  ages.insert({"Charlie", 35});
9  ages.insert(std::make_pair("David", 28));
10
11 // 访问
12 std::cout << "Alice's age: " << ages["Alice"] << std::endl; // 25
13
14 // 注意: 使用 [] 访问不存在的键会自动插入
15 std::cout << "Eve's age: " << ages["Eve"] << std::endl; // 0 (自动插入)
16
17 // 安全访问: 使用 at(), 键不存在时抛出异常
18 try {
19     int age = ages.at("Frank");
20 } catch (const std::out_of_range& e) {
21     std::cout << "Frank not found" << std::endl;
22 }
23
24 // 检查键是否存在
25 if (ages.count("Alice") > 0) {
26     std::cout << "Alice exists" << std::endl;
27 }
28
29 // 使用 find 查找
30 auto it = ages.find("Bob");
31 if (it != ages.end()) {
32     std::cout << it->first << " is " << it->second << " years old" <<
33     std::endl;
34 }
35
36 // 遍历 (按键排序)
37 for (const auto& pair : ages) {
38     std::cout << pair.first << ": " << pair.second << std::endl;
39 }
40
41 // C++17 结构化绑定
42 for (const auto& [name, age] : ages) {
43     std::cout << name << ": " << age << std::endl;
44 }
45
46 // 删除
47 ages.erase("Eve");

```

`map` 在 RoboMaster 开发中非常有用，例如存储电机 ID 与电机对象的映射、目标 ID 与跟踪状态的映射等：

```
1 std::map<int, Motor> motors;
2 motors[1] = Motor(1, 10000.0);
3 motors[2] = Motor(2, 10000.0);
4 motors[3] = Motor(3, 8000.0);
5 motors[4] = Motor(4, 8000.0);
6
7 void updateMotor(int id, double speed) {
8     auto it = motors.find(id);
9     if (it != motors.end()) {
10         it->second.setSpeed(speed);
11     }
12 }
```

1.5.17.7. unordered_set 和 unordered_map

`std::unordered_set` 和 `std::unordered_map` 是基于哈希表的容器，提供平均 $O(1)$ 的查找、插入、删除操作，但元素没有特定顺序。

```
1 #include <unordered_set>
2 #include <unordered_map>
3
4 // unordered_set
5 std::unordered_set<int> us = {3, 1, 4, 1, 5, 9};
6 // 元素唯一，但顺序不确定
7 for (int x : us) {
8     std::cout << x << " "; // 顺序不确定
9 }
10 std::cout << std::endl;
11
12 us.insert(2);
13 us.erase(4);
14 bool found = (us.find(5) != us.end());
15
16 // unordered_map
17 std::unordered_map<std::string, int> scores;
18 scores["Alice"] = 95;
19 scores["Bob"] = 87;
20 scores["Charlie"] = 92;
21
22 // 接口与 map 类似，但元素无序
23 for (const auto& [name, score] : scores) {
24     std::cout << name << ": " << score << std::endl; // 顺序不确定
25 }
```

无序容器通常比有序容器更快，但有一些注意事项：

哈希冲突会影响性能。在最坏情况下，操作可能退化为 $O(n)$ 。

元素类型必须支持哈希。对于自定义类型，需要提供哈希函数：

```
1 struct Point {
2     int x, y;
3
4     bool operator==(const Point& other) const {
5         return x == other.x && y == other.y;
6     }
7 };
8
```

```

9 // 自定义哈希函数
10 struct PointHash {
11     size_t operator()(const Point& p) const {
12         return std::hash<int>()(p.x) ^ (std::hash<int>()(p.y) << 1);
13     }
14 };
15
16 std::unordered_set<Point, PointHash> points;
17 points.insert({1, 2});
18 points.insert({3, 4});

```

在需要快速查找且不关心元素顺序的场景下，优先选择无序容器。

1.5.17.8. stack: 栈

`std::stack` 是容器适配器，提供后进先出（LIFO）的访问模式。它默认基于 `deque` 实现，也可以指定其他底层容器。

```

1 #include <stack>
2
3 std::stack<int> st;
4
5 // 压栈
6 st.push(10);
7 st.push(20);
8 st.push(30);
9
10 // 查看栈顶
11 std::cout << "Top: " << st.top() << std::endl; // 30
12
13 // 出栈
14 st.pop(); // 移除 30
15 std::cout << "After pop: " << st.top() << std::endl; // 20
16
17 // 检查是否为空
18 while (!st.empty()) {
19     std::cout << st.top() << " ";
20     st.pop();
21 }
22 // 输出: 20 10

```

`stack` 只能访问栈顶元素，不能遍历。它适用于需要“最近添加的最先处理”语义的场景，如括号匹配、表达式求值、深度优先搜索等。

```

1 // 括号匹配示例
2 bool isBalanced(const std::string& expr) {
3     std::stack<char> st;
4
5     for (char c : expr) {
6         if (c == '(' || c == '[' || c == '{') {
7             st.push(c);
8         } else if (c == ')' || c == ']' || c == '}') {
9             if (st.empty()) return false;
10
11             char top = st.top();
12             if ((c == ')' && top != '(') ||
13                 (c == ']' && top != '[') ||
14                 (c == '}' && top != '{')) {
15                 return false;
16             }
17             st.pop();
18         }
19     }
20     return st.empty();
21 }

```

```

18     }
19 }
20
21     return st.empty();
22 }

```

1.5.17.9. queue: 队列

`std::queue` 提供先进先出（FIFO）的访问模式，同样是容器适配器。

```

1  #include <queue>
2
3  std::queue<int> q;
4
5  // 入队
6  q.push(10);
7  q.push(20);
8  q.push(30);
9
10 // 查看队首和队尾
11 std::cout << "Front: " << q.front() << std::endl; // 10
12 std::cout << "Back: " << q.back() << std::endl;   // 30
13
14 // 出队
15 q.pop(); // 移除 10
16 std::cout << "After pop: " << q.front() << std::endl; // 20
17
18 // 大小
19 std::cout << "Size: " << q.size() << std::endl; // 2

```

`queue` 适用于“先来先服务”的场景，如任务调度、广度优先搜索、消息队列等。

在 RoboMaster 开发中，队列可用于缓存传感器数据、管理待处理的命令等：

```

1  std::queue<SensorData> sensorQueue;
2
3  // 生产者线程
4  void sensorCallback(const SensorData& data) {
5      sensorQueue.push(data);
6  }
7
8  // 消费者线程
9  void processLoop() {
10     while (running) {
11         if (!sensorQueue.empty()) {
12             SensorData data = sensorQueue.front();
13             sensorQueue.pop();
14             process(data);
15         }
16     }
17 }

```

1.5.17.10. priority_queue: 优先队列

`std::priority_queue` 是一种特殊的队列，每次取出的是优先级最高的元素（默认是最大值）。它基于堆实现。

```

1  #include <queue>
2
3  // 默认是最大堆

```

```

4 std::priority_queue<int> maxPq;
5 maxPq.push(30);
6 maxPq.push(10);
7 maxPq.push(20);
8
9 std::cout << maxPq.top() << std::endl; // 30
10 maxPq.pop();
11 std::cout << maxPq.top() << std::endl; // 20
12
13 // 最小堆需要指定比较函数
14 std::priority_queue<int, std::vector<int>, std::greater<int>> minPq;
15 minPq.push(30);
16 minPq.push(10);
17 minPq.push(20);
18
19 std::cout << minPq.top() << std::endl; // 10

```

优先队列在 RoboMaster 开发中可用于目标优先级排序：

```

1 struct Target {
2     int id;
3     double priority;
4
5     // 定义比较（优先级高的排前面）
6     bool operator<(const Target& other) const {
7         return priority < other.priority; // 注意：priority_queue 是最大堆
8     }
9 };
10
11 std::priority_queue<Target> targetQueue;
12
13 void addTarget(const Target& t) {
14     targetQueue.push(t);
15 }
16
17 Target getHighestPriorityTarget() {
18     Target t = targetQueue.top();
19     targetQueue.pop();
20     return t;
21 }

```

1.5.17.11. 如何选择容器

选择合适的容器是编写高效代码的关键。以下是一些指导原则：

如果需要随机访问元素，且主要在末尾添加删除，选择 `vector`。它是最通用的容器，在大多数情况下都是首选。`vector` 的内存连续，缓存友好，遍历性能极佳。

如果需要在两端频繁添加删除，选择 `deque`。它在头尾操作都是常数时间，适合实现双端队列。

如果需要在中间频繁插入删除，且不需要随机访问，选择 `list`。但要注意 `list` 的缓存性能较差，实际上在很多场景下 `vector`（即使需要移动元素）反而更快。

如果需要元素自动排序且唯一，选择 `set`。如果允许重复元素，选择 `multiset`。

如果需要键值对且按键排序，选择 `map`。如果允许重复键，选择 `multimap`。

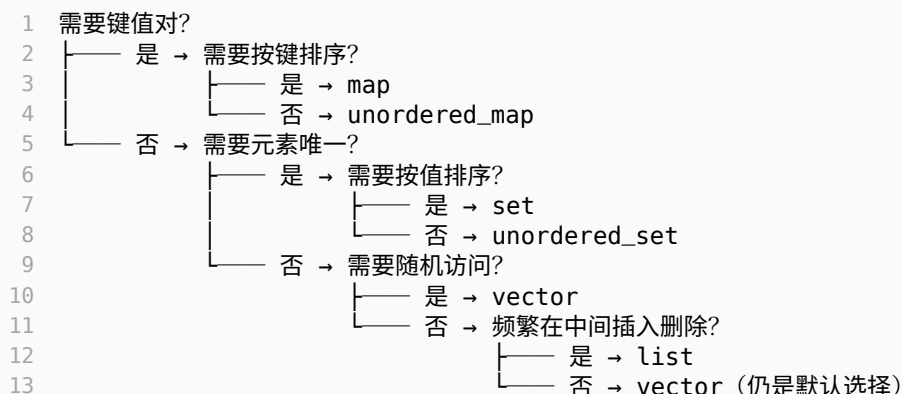
如果只需要快速查找、不关心顺序，选择 `unordered_set` 或 `unordered_map`。它们的平均性能优于有序版本。

如果需要后进先出语义，选择 `stack`。

如果需要先进先出语义，选择 `queue`。

如果需要按优先级处理元素，选择 `priority_queue`。

以下是一个简单的决策流程：



1.5.17.12. 容器的通用操作

STL 容器共享许多通用操作，这种一致性使得学习新容器变得简单：

```
1 // 以 vector 为例，但这些操作适用于大多数容器
2
3 std::vector<int> v = {1, 2, 3, 4, 5};
4
5 // 大小相关
6 v.size();      // 元素数量
7 v.empty();     // 是否为空
8 v.max_size();  // 理论最大容量
9
10 // 迭代器
11 v.begin();     // 指向第一个元素
12 v.end();       // 指向最后一个元素之后
13 v.rbegin();    // 反向迭代器，指向最后一个元素
14 v.rend();     // 反向迭代器，指向第一个元素之前
15
16 // 修改
17 v.clear();     // 清空所有元素
18 v.swap(other); // 与另一个容器交换内容
19
20 // 比较（字典序）
21 v1 == v2;     // 相等
22 v1 < v2;      // 小于
23 // 等等
```

这种统一的接口设计是 STL 的重要特征，它使得通用算法可以作用于任何容器。

1.5.17.13. 实践中的性能考虑

选择容器时，除了功能匹配，还要考虑性能特征。以下是各种操作的时间复杂度对比：

操作	vector	deque	list	set/map	unordered_set/map
随机访问	$O(1)$	$O(1)$	$O(n)$	$O(\log n)$	$O(1)$ 平均
头部插入	$O(n)$	$O(1)$	$O(1)$	-	-
尾部插入	$O(1)^*$	$O(1)$	$O(1)$	-	-
中间插入	$O(n)$	$O(n)$	$O(1)^*$	$O(\log n)$	$O(1)$ 平均
查找	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$ 平均
* 摊销常数时间					

然而, 时间复杂度只是故事的一部分。`vector` 的缓存局部性使得它在很多场景下即使时间复杂度较高 (如 $O(n)$ 的中间插入), 实际性能也优于理论上更快的 `list`。现代 CPU 的缓存机制使得连续内存访问比离散内存访问快得多。

在 RoboMaster 开发中的一般建议是: 从 `vector` 开始, 只有在性能分析表明它是瓶颈时才考虑其他容器。过早优化选择复杂的数据结构可能反而降低性能, 同时增加代码复杂性。

STL 容器是 C++ 程序员最强大的工具之一。它们经过精心设计和优化, 提供了丰富的功能和优异的性能。掌握它们, 可以让你专注于解决实际问题, 而不是重复实现基础数据结构。下一节我们将学习 STL 算法和迭代器, 它们与容器配合使用, 提供了强大的数据处理能力。

1.5.18. STL 算法与迭代器

上一节我们学习了 STL 容器, 它们提供了多种数据存储方式。然而, 仅有容器是不够的——我们还需要对容器中的数据进行各种操作: 排序、查找、变换、统计等。如果每种容器都需要单独实现这些操作, 代码量将会非常庞大。STL 的设计者采用了一种优雅的方案: 通过迭代器 (iterator) 将容器与算法解耦。迭代器是一种抽象的“指针”, 提供了统一的元素访问方式; 算法则基于迭代器工作, 不关心底层容器的具体类型。这种设计使得少量的算法可以作用于任意容器, 极大地提高了代码的复用性。

1.5.18.1. 迭代器的概念

迭代器是容器与算法之间的桥梁。从概念上讲, 迭代器是一种行为类似指针的对象, 它指向容器中的某个元素, 可以通过解引用访问该元素, 通过递增移动到下一个元素。

每个 STL 容器都定义了自己的迭代器类型, 可以通过 `begin()` 和 `end()` 成员函数获取:

```
1  #include <vector>
2  #include <iostream>
3
4  int main() {
5      std::vector<int> v = {10, 20, 30, 40, 50};
6
7      // 获取迭代器
8      std::vector<int>::iterator it = v.begin(); // 指向第一个元素
9      std::vector<int>::iterator end = v.end();   // 指向最后一个元素之后
10
11     // 使用 auto 简化类型声明
12     auto it2 = v.begin();
13
14     // 解引用访问元素
15     std::cout << *it << std::endl; // 10
16
17     // 递增移动到下一个元素
18     ++it;
19     std::cout << *it << std::endl; // 20
20
21     // 遍历容器
22     for (auto iter = v.begin(); iter != v.end(); ++iter) {
23         std::cout << *iter << " ";
24     }
25     std::cout << std::endl; // 10 20 30 40 50
26
27     return 0;
28 }
```


`begin()` 返回指向第一个元素的迭代器，`end()` 返回指向“尾后”位置的迭代器——这是一个不存在的位置，表示容器的结束。这种半开区间 `[begin, end)` 的设计使得空容器可以用 `begin() == end()` 来判断，也使得遍历逻辑更加统一。

迭代器支持的操作取决于其类别。STL 定义了五种迭代器类别，从弱到强依次是：

输入迭代器 (Input Iterator) 只能向前移动，只能读取元素，且只能遍历一次。典型例子是从输入流读取数据的迭代器。

输出迭代器 (Output Iterator) 只能向前移动，只能写入元素。典型例子是向输出流写入数据的迭代器。

前向迭代器 (Forward Iterator) 可以多次遍历，支持读写。`forward_list` 的迭代器属于这一类。

双向迭代器 (Bidirectional Iterator) 可以向前和向后移动。`list`、`set`、`map` 的迭代器属于这一类。

随机访问迭代器 (Random Access Iterator) 支持所有指针运算，包括直接跳转到任意位置。`vector`、`deque`、`array` 的迭代器以及原生指针属于这一类。

```
1 std::vector<int> v = {10, 20, 30, 40, 50};
2 auto it = v.begin();
3
4 // 随机访问迭代器支持的操作
5 ++it;           // 前进一步
6 --it;           // 后退一步
7 it += 3;        // 前进三步
8 it -= 2;        // 后退两步
9 auto it2 = it + 2; // 获取前方第二个位置的迭代器
10 int diff = it2 - it; // 计算两个迭代器之间的距离
11 bool less = it < it2; // 比较位置
12
13 std::cout << *it << std::endl;    // 20
14 std::cout << it[2] << std::endl;  // 40 (相当于 *(it + 2))
15
16 // list 的双向迭代器不支持随机访问
17 std::list<int> lst = {10, 20, 30};
18 auto lit = lst.begin();
19 ++lit;    // 正确
20 --lit;    // 正确
21 // lit += 2; // 错误! 双向迭代器不支持 +=
22 // lit[1];   // 错误! 双向迭代器不支持 []
```

不同算法对迭代器有不同的要求。例如，`std::sort` 需要随机访问迭代器，因此不能用于 `list`；而 `std::find` 只需要输入迭代器，可以用于任何容器。

1.5.18.2. 迭代器的种类

除了普通迭代器，STL 还提供了几种特殊的迭代器。

常量迭代器 (`const_iterator`) 不允许通过迭代器修改元素。当容器是 `const` 的，或者只需要读取元素时，应使用常量迭代器：

```
1 std::vector<int> v = {10, 20, 30};
2
3 // 普通迭代器，可以修改元素
4 std::vector<int>::iterator it = v.begin();
5 *it = 100; // 正确
6
```

```

7 // 常量迭代器, 不能修改元素
8 std::vector<int>::const_iterator cit = v.cbegin();
9 // *cit = 200; // 错误! 不能通过 const_iterator 修改
10
11 // const 容器只能使用常量迭代器
12 const std::vector<int>& cv = v;
13 auto cit2 = cv.cbegin(); // 自动推导为 const_iterator

```

`cbegin()` 和 `cend()` 显式返回常量迭代器, 即使容器本身不是 `const` 的。

反向迭代器 (`reverse_iterator`) 从后向前遍历容器:

```

1 std::vector<int> v = {10, 20, 30, 40, 50};
2
3 // 反向遍历
4 for (auto rit = v.rbegin(); rit != v.rend(); ++rit) {
5     std::cout << *rit << " ";
6 }
7 std::cout << std::endl; // 50 40 30 20 10
8
9 // 常量反向迭代器
10 for (auto crit = v.crbegin(); crit != v.crend(); ++crit) {
11     std::cout << *crit << " ";
12 }

```

插入迭代器 (`insert iterator`) 在写入时自动向容器插入新元素, 而非覆盖现有元素:

```

1 #include <iterator>
2
3 std::vector<int> v = {1, 2, 3};
4 std::vector<int> result;
5
6 // back_inserter: 在末尾插入
7 std::copy(v.begin(), v.end(), std::back_inserter(result));
8 // result = {1, 2, 3}
9
10 // front_inserter: 在开头插入 (需要支持 push_front 的容器)
11 std::list<int> lst;
12 std::copy(v.begin(), v.end(), std::front_inserter(lst));
13 // lst = {3, 2, 1} (注意顺序)
14
15 // inserter: 在指定位置插入
16 std::vector<int> v2 = {10, 20};
17 std::copy(v.begin(), v.end(), std::inserter(v2, v2.begin() + 1));
18 // v2 = {10, 1, 2, 3, 20}

```

1.5.18.3. 常用算法概览

STL 算法定义在 `<algorithm>` 头文件中 (部分数值算法在 `<numeric>` 中)。它们接受迭代器作为参数, 对指定范围内的元素进行操作。

让我们从最常用的算法开始。

排序算法 `std::sort` 对元素进行升序排序:

```

1 #include <algorithm>
2 #include <vector>
3
4 std::vector<int> v = {30, 10, 50, 20, 40};
5
6 std::sort(v.begin(), v.end()); // 升序排序

```

```

7 // v = {10, 20, 30, 40, 50}
8
9 // 降序排序: 使用比较函数
10 std::sort(v.begin(), v.end(), std::greater<int>());
11 // v = {50, 40, 30, 20, 10}
12
13 // 自定义比较函数
14 std::sort(v.begin(), v.end(), [](int a, int b) {
15     return a > b; // 降序
16 });
17
18 // 对部分范围排序
19 std::vector<int> v2 = {5, 3, 1, 4, 2};
20 std::sort(v2.begin() + 1, v2.begin() + 4); // 只排序索引 1-3
21 // v2 = {5, 1, 3, 4, 2}

```

`std::sort` 使用快速排序的变体 (IntroSort), 平均时间复杂度 $O(n \log n)$, 需要随机访问迭代器。对于 `list`, 应使用其成员函数 `list::sort()`。

稳定排序 `std::stable_sort` 保持相等元素的相对顺序:

```

1 struct Student {
2     std::string name;
3     int score;
4 };
5
6 std::vector<Student> students = {
7     {"Alice", 90}, {"Bob", 85}, {"Charlie", 90}, {"David", 85}
8 };
9
10 // 按分数排序, 相同分数保持原顺序
11 std::stable_sort(students.begin(), students.end(),
12     [](const Student& a, const Student& b) {
13         return a.score > b.score;
14     });
15 // Alice(90), Charlie(90), Bob(85), David(85)
16 // 相同分数的学生保持了原来的相对顺序

```

查找算法 `std::find` 在范围内查找特定值:

```

1 std::vector<int> v = {10, 20, 30, 40, 50};
2
3 auto it = std::find(v.begin(), v.end(), 30);
4 if (it != v.end()) {
5     std::cout << "找到: " << *it << std::endl;
6     std::cout << "索引: " << (it - v.begin()) << std::endl; // 2
7 } else {
8     std::cout << "未找到" << std::endl;
9 }
10
11 // 查找满足条件的第一个元素
12 auto it2 = std::find_if(v.begin(), v.end(), [](int x) {
13     return x > 25;
14 });
15 if (it2 != v.end()) {
16     std::cout << "第一个大于25的数: " << *it2 << std::endl; // 30
17 }
18
19 // 查找不满足条件的第一个元素
20 auto it3 = std::find_if_not(v.begin(), v.end(), [](int x) {
21     return x < 35;
22 });

```

```
23 // *it3 = 40
```

对于已排序的容器，`std::binary_search` 使用二分查找，效率更高：

```
1 std::vector<int> v = {10, 20, 30, 40, 50}; // 必须已排序
2
3 bool found = std::binary_search(v.begin(), v.end(), 30); // true
4 bool notFound = std::binary_search(v.begin(), v.end(), 35); // false
5
6 // 获取位置使用 lower_bound / upper_bound
7 auto lb = std::lower_bound(v.begin(), v.end(), 30); // 指向 30
8 auto ub = std::upper_bound(v.begin(), v.end(), 30); // 指向 40
```

计数算法 `std::count` 统计特定值的出现次数：

```
1 std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};
2
3 int count = std::count(v.begin(), v.end(), 2);
4 std::cout << "2 出现了 " << count << " 次" << std::endl; // 3
5
6 // 统计满足条件的元素数量
7 int countEven = std::count_if(v.begin(), v.end(), [](int x) {
8     return x % 2 == 0;
9 });
10 std::cout << "偶数有 " << countEven << " 个" << std::endl; // 4
```

1.5.18.4. 变换与修改算法

`std::transform` 对每个元素应用函数，将结果写入目标范围：

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2 std::vector<int> result(v.size());
3
4 // 每个元素乘以 2
5 std::transform(v.begin(), v.end(), result.begin(),
6     [](int x) { return x * 2; });
7 // result = {2, 4, 6, 8, 10}
8
9 // 原地变换
10 std::transform(v.begin(), v.end(), v.begin(),
11     [](int x) { return x * x; });
12 // v = {1, 4, 9, 16, 25}
13
14 // 两个范围的元素配对运算
15 std::vector<int> a = {1, 2, 3};
16 std::vector<int> b = {4, 5, 6};
17 std::vector<int> sum(3);
18
19 std::transform(a.begin(), a.end(), b.begin(), sum.begin(),
20     [](int x, int y) { return x + y; });
21 // sum = {5, 7, 9}
```

`std::for_each` 对每个元素执行操作（不产生新值）：

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 // 打印每个元素
4 std::for_each(v.begin(), v.end(), [](int x) {
5     std::cout << x << " ";
6 });
```

```

7  std::cout << std::endl;
8
9  // 修改元素 (通过引用)
10 std::for_each(v.begin(), v.end(), [](int& x) {
11     x *= 2;
12 });
13 // v = {2, 4, 6, 8, 10}

```

`std::copy` 复制元素到另一个范围:

```

1  std::vector<int> src = {1, 2, 3, 4, 5};
2  std::vector<int> dst(5);
3
4  std::copy(src.begin(), src.end(), dst.begin());
5  // dst = {1, 2, 3, 4, 5}
6
7  // 条件复制
8  std::vector<int> evens;
9  std::copy_if(src.begin(), src.end(), std::back_inserter(evens),
10             [](int x) { return x % 2 == 0; });
11 // evens = {2, 4}

```

`std::fill` 将范围内所有元素设为指定值:

```

1  std::vector<int> v(10);
2  std::fill(v.begin(), v.end(), 42);
3  // v = {42, 42, 42, 42, 42, 42, 42, 42, 42, 42}
4
5  // 只填充部分
6  std::fill(v.begin(), v.begin() + 5, 0);
7  // v = {0, 0, 0, 0, 0, 42, 42, 42, 42, 42}

```

`std::replace` 替换特定值:

```

1  std::vector<int> v = {1, 2, 3, 2, 4, 2};
2
3  std::replace(v.begin(), v.end(), 2, 99);
4  // v = {1, 99, 3, 99, 4, 99}
5
6  // 条件替换
7  std::vector<int> v2 = {1, 2, 3, 4, 5};
8  std::replace_if(v2.begin(), v2.end(),
9                [](int x) { return x % 2 == 0; }, 0);
10 // v2 = {1, 0, 3, 0, 5}

```

`std::remove` 和 `std::remove_if` “移除”满足条件的元素。注意，它们实际上不会改变容器大小，只是将不需要移除的元素移到前面，返回新的逻辑结尾:

```

1  std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};
2
3  // remove 返回新的逻辑结尾
4  auto newEnd = std::remove(v.begin(), v.end(), 2);
5  // v 可能是 {1, 3, 4, 5, ?, ?, ?}, newEnd 指向第一个 ?
6
7  // 真正删除需要配合 erase
8  v.erase(newEnd, v.end());
9  // v = {1, 3, 4, 5}
10
11 // 常见的 erase-remove 惯用法
12 std::vector<int> v2 = {1, 2, 3, 4, 5, 6};

```

```

13 v2.erase(std::remove_if(v2.begin(), v2.end(),
14     [](int x) { return x % 2 == 0; }), v2.end());
15 // v2 = {1, 3, 5}

```

`std::unique` 移除连续的重复元素:

```

1 std::vector<int> v = {1, 1, 2, 2, 2, 3, 3, 4};
2
3 auto newEnd = std::unique(v.begin(), v.end());
4 v.erase(newEnd, v.end());
5 // v = {1, 2, 3, 4}
6
7 // 对于非连续的重复, 需要先排序
8 std::vector<int> v2 = {3, 1, 2, 1, 3, 2, 1};
9 std::sort(v2.begin(), v2.end());
10 v2.erase(std::unique(v2.begin(), v2.end()), v2.end());
11 // v2 = {1, 2, 3}

```

`std::reverse` 反转元素顺序:

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 std::reverse(v.begin(), v.end());
3 // v = {5, 4, 3, 2, 1}

```

1.5.18.5. 数值算法

`<numeric>` 头文件提供了一些数值算法。

`std::accumulate` 计算范围内元素的累积值:

```

1 #include <numeric>
2
3 std::vector<int> v = {1, 2, 3, 4, 5};
4
5 // 求和
6 int sum = std::accumulate(v.begin(), v.end(), 0); // 15
7
8 // 求积
9 int product = std::accumulate(v.begin(), v.end(), 1,
10     [](int acc, int x) { return acc * x; }); // 120
11
12 // 字符串连接
13 std::vector<std::string> words = {"Hello", " ", "World"};
14 std::string sentence = std::accumulate(words.begin(), words.end(),
15     std::string("")); // "Hello World"

```

`std::inner_product` 计算两个范围的内积:

```

1 std::vector<double> a = {1.0, 2.0, 3.0};
2 std::vector<double> b = {4.0, 5.0, 6.0};
3
4 double dot = std::inner_product(a.begin(), a.end(), b.begin(), 0.0);
5 // 1*4 + 2*5 + 3*6 = 32

```

`std::partial_sum` 计算前缀和:

```

1 std::vector<int> v = {1, 2, 3, 4, 5};
2 std::vector<int> prefix(v.size());
3
4 std::partial_sum(v.begin(), v.end(), prefix.begin());

```

```
5 // prefix = {1, 3, 6, 10, 15}
```

`std::iota` 填充递增序列:

```
1 std::vector<int> v(10);
2 std::iota(v.begin(), v.end(), 1);
3 // v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

1.5.18.6. 范围 for 循环

C++11 引入的范围 for 循环 (range-based for loop) 大大简化了容器遍历的语法。它内部使用迭代器, 但隐藏了迭代器的细节:

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 // 传统迭代器方式
4 for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
5     std::cout << *it << " ";
6 }
7
8 // 范围 for 循环
9 for (int x : v) {
10     std::cout << x << " ";
11 }
```

范围 for 循环有几种形式, 适用于不同场景:

```
1 std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
2
3 // 值拷贝: 适用于小型类型, 不修改原容器
4 for (std::string name : names) {
5     std::cout << name << std::endl;
6 }
7
8 // 常量引用: 避免拷贝, 只读访问 (推荐用于大型对象)
9 for (const std::string& name : names) {
10     std::cout << name << std::endl;
11 }
12
13 // 引用: 需要修改元素时使用
14 for (std::string& name : names) {
15     name = "Mr. " + name;
16 }
17 // names = {"Mr. Alice", "Mr. Bob", "Mr. Charlie"}
18
19 // auto 自动推导类型
20 for (const auto& name : names) {
21     std::cout << name << std::endl;
22 }
```

范围 for 循环可以用于任何提供 `begin()` 和 `end()` 的对象, 包括:

```
1 // STL 容器
2 std::vector<int> vec = {1, 2, 3};
3 for (int x : vec) { /* ... */ }
4
5 // 原生数组
6 int arr[] = {1, 2, 3, 4, 5};
7 for (int x : arr) { /* ... */ }
8
```

```

9 // 初始化列表
10 for (int x : {1, 2, 3, 4, 5}) { /* ... */ }
11
12 // std::string
13 std::string str = "Hello";
14 for (char c : str) { /* ... */ }
15
16 // map 等关联容器
17 std::map<std::string, int> ages = {"Alice", 25}, {"Bob", 30};
18 for (const auto& pair : ages) {
19     std::cout << pair.first << ": " << pair.second << std::endl;
20 }
21
22 // C++17 结构化绑定
23 for (const auto& [name, age] : ages) {
24     std::cout << name << ": " << age << std::endl;
25 }

```

范围 for 循环的一个限制是无法直接获取当前索引或迭代器。如果需要索引, 可以使用传统循环或额外维护一个计数器:

```

1 std::vector<int> v = {10, 20, 30, 40, 50};
2
3 // 需要索引时的选择
4 // 方式一: 传统循环
5 for (size_t i = 0; i < v.size(); i++) {
6     std::cout << i << ": " << v[i] << std::endl;
7 }
8
9 // 方式二: 手动维护索引
10 size_t index = 0;
11 for (const auto& x : v) {
12     std::cout << index << ": " << x << std::endl;
13     index++;
14 }

```

1.5.18.7. 算法与谓词

许多 STL 算法接受谓词 (predicate) 作为参数。谓词是返回布尔值的函数或函数对象, 用于自定义算法的行为。

谓词可以是普通函数:

```

1 bool isPositive(int x) {
2     return x > 0;
3 }
4
5 std::vector<int> v = {-2, -1, 0, 1, 2};
6 int count = std::count_if(v.begin(), v.end(), isPositive); // 2

```

也可以是函数对象:

```

1 struct GreaterThan {
2     int threshold;
3     GreaterThan(int t) : threshold(t) {}
4     bool operator()(int x) const {
5         return x > threshold;
6     }
7 };
8

```



```

9  std::vector<int> v = {1, 5, 3, 8, 2, 9, 4};
10 auto it = std::find_if(v.begin(), v.end(), GreaterThan(6));
11 // *it = 8

```

最常用的是 lambda 表达式，它提供了简洁的内联定义方式：

```

1  std::vector<int> v = {1, 5, 3, 8, 2, 9, 4};
2
3  // 基本 lambda
4  auto it = std::find_if(v.begin(), v.end(), [](int x) {
5      return x > 6;
6  });
7
8  // 捕获外部变量
9  int threshold = 6;
10 auto it2 = std::find_if(v.begin(), v.end(), [threshold](int x) {
11     return x > threshold;
12 });
13
14 // 按引用捕获
15 int sum = 0;
16 std::for_each(v.begin(), v.end(), [&sum](int x) {
17     sum += x;
18 });
19
20 // 捕获所有局部变量（按值）
21 // [=](int x) { ... }
22
23 // 捕获所有局部变量（按引用）
24 // [&](int x) { ... }

```

STL 还提供了一些预定义的函数对象，定义在 `<functional>` 头文件中：

```

1  #include <functional>
2
3  std::vector<int> v = {30, 10, 50, 20, 40};
4
5  // 降序排序
6  std::sort(v.begin(), v.end(), std::greater<int>());
7  // v = {50, 40, 30, 20, 10}
8
9  // 其他常用函数对象
10 // std::less<T>      小于比较（默认）
11 // std::greater<T>   大于比较
12 // std::plus<T>      加法
13 // std::minus<T>     减法
14 // std::multiplies<T> 乘法
15 // std::divides<T>   除法
16 // std::negate<T>    取负
17 // std::equal_to<T>  相等比较
18 // std::not_equal_to<T> 不等比较

```

1.5.18.8. 在 RoboMaster 开发中的应用

让我们看几个 STL 算法在 RoboMaster 开发中的实际应用。

目标筛选与排序：

```

1  struct Target {
2      int id;
3      double distance;

```

```

4     double confidence;
5     bool isEnemy;
6 };
7
8 std::vector<Target> targets;
9
10 // 筛选敌方目标
11 std::vector<Target> enemies;
12 std::copy_if(targets.begin(), targets.end(), std::back_inserter(enemies),
13     [](const Target& t) { return t.isEnemy; });
14
15 // 按距离排序
16 std::sort(enemies.begin(), enemies.end(),
17     [](const Target& a, const Target& b) {
18         return a.distance < b.distance;
19     });
20
21 // 找最高置信度的目标
22 auto bestTarget = std::max_element(enemies.begin(), enemies.end(),
23     [](const Target& a, const Target& b) {
24         return a.confidence < b.confidence;
25     });
26
27 if (bestTarget != enemies.end()) {
28     std::cout << "最佳目标 ID: " << bestTarget->id << std::endl;
29 }
30
31 // 移除低置信度目标
32 enemies.erase(
33     std::remove_if(enemies.begin(), enemies.end(),
34         [](const Target& t) { return t.confidence < 0.5; }),
35     enemies.end());

```

传感器数据处理:

```

1 struct SensorReading {
2     double timestamp;
3     double value;
4 };
5
6 std::vector<SensorReading> readings;
7
8 // 计算平均值
9 double sum = std::accumulate(readings.begin(), readings.end(), 0.0,
10     [](double acc, const SensorReading& r) {
11         return acc + r.value;
12     });
13 double average = readings.empty() ? 0.0 : sum / readings.size();
14
15 // 找最大最小值
16 auto [minIt, maxIt] = std::minmax_element(readings.begin(), readings.end(),
17     [](const SensorReading& a, const SensorReading& b) {
18         return a.value < b.value;
19     });
20
21 // 检查是否所有读数都在有效范围内
22 bool allValid = std::all_of(readings.begin(), readings.end(),
23     [](const SensorReading& r) {
24         return r.value >= 0.0 && r.value <= 100.0;
25     });
26
27 // 检查是否有任何异常值
28 bool hasAnomaly = std::any_of(readings.begin(), readings.end(),

```

```

29     [average](const SensorReading& r) {
30         return std::abs(r.value - average) > 3.0 * stddev; // 假设 stddev 已
    计算
31     });

```

电机控制数组操作：

```

1  std::array<double, 4> motorSpeeds = {1000, 2000, 1500, 1800};
2  std::array<double, 4> targetSpeeds = {1200, 2200, 1600, 2000};
3
4  // 计算误差
5  std::array<double, 4> errors;
6  std::transform(targetSpeeds.begin(), targetSpeeds.end(),
7                motorSpeeds.begin(), errors.begin(),
8                [](double target, double current) {
9                    return target - current;
10               });
11
12 // 限幅处理
13 const double maxSpeed = 5000.0;
14 std::transform(motorSpeeds.begin(), motorSpeeds.end(), motorSpeeds.begin(),
15               [maxSpeed](double speed) {
16                   return std::clamp(speed, -maxSpeed, maxSpeed);
17               });
18
19 // 检查是否所有电机都接近目标
20 bool allOnTarget = std::all_of(errors.begin(), errors.end(),
21                                 [](double e) { return std::abs(e) < 10.0; });

```

1.5.18.9. 算法的效率考虑

使用 STL 算法时，了解其复杂度很重要：

1 算法	时间复杂度	说明
2 sort	$O(n \log n)$	快速排序变体
3 stable_sort	$O(n \log n)$	归并排序
4 find	$O(n)$	线性查找
5 binary_search	$O(\log n)$	需要已排序
6 count	$O(n)$	线性扫描
7 transform	$O(n)$	线性变换
8 copy	$O(n)$	线性复制
9 unique	$O(n)$	线性扫描
10 reverse	$O(n)$	线性反转
11 min/max_element	$O(n)$	线性扫描
12 accumulate	$O(n)$	线性累加

一些优化建议：

对于需要频繁查找的数据，考虑使用 `set`、`map` 或无序容器，而非在 `vector` 上使用 `find`。

如果需要多次查找，先排序再使用 `binary_search` 或 `lower_bound`。

避免在循环中重复调用 `size()` 或 `end()`，虽然现代编译器通常会优化这些调用。

使用 `reserve()` 预分配空间，避免多次重新分配。

对于大型对象，使用移动语义（后续章节介绍）减少拷贝开销。

1.5.18.10. 自定义类型与算法

要让自定义类型与 STL 算法配合使用，需要满足某些要求。

对于排序和查找算法，类型需要支持比较操作：

```
1 struct Point {
2     double x, y;
3
4     // 方式一: 重载 < 运算符
5     bool operator<(const Point& other) const {
6         if (x != other.x) return x < other.x;
7         return y < other.y;
8     }
9
10    // 方式二: 重载 == 运算符 (用于 find 等)
11    bool operator==(const Point& other) const {
12        return x == other.x && y == other.y;
13    }
14 };
15
16 std::vector<Point> points = {{1, 2}, {3, 1}, {2, 3}};
17 std::sort(points.begin(), points.end()); // 使用 operator<
18
19 Point target = {3, 1};
20 auto it = std::find(points.begin(), points.end(), target); // 使用
    operator==
```

也可以不修改类型，而是提供自定义比较器：

```
1 struct Point {
2     double x, y;
3 };
4
5 // 按到原点距离排序
6 std::sort(points.begin(), points.end(),
7     [](const Point& a, const Point& b) {
8         double distA = a.x * a.x + a.y * a.y;
9         double distB = b.x * b.x + b.y * b.y;
10        return distA < distB;
11    });
```

STL 算法与迭代器的组合提供了强大而灵活的数据处理能力。通过学习这些工具，你可以用更少的代码完成更多的工作，同时保持代码的清晰和效率。算法表达了“做什么”，而不是“怎么做”，这使得代码更易于理解和维护。下一节我们将学习模板基础，了解 STL 容器和算法背后的泛型编程技术。

1.5.19. 模板基础

在前面的章节中，我们大量使用了 STL 容器和算法：`vector<int>`、`map<string, double>`、`sort`、`find` 等。你可能已经注意到，这些组件可以处理任意类型的数据——`vector` 可以存储 `int`、`double`、`string` 甚至自定义类型，`sort` 可以对任何支持比较操作的类型进行排序。这种“一次编写，处理多种类型”的能力正是 C++ 模板 (template) 提供的。模板是 C++ 泛型编程的基础，它允许我们编写与类型无关的代码，由编译器在使用时根据具体类型生成相应的实现。理解模板不仅能帮助你更好地使用 STL，还能让你编写出更加通用、可复用的代码。

1.5.19.1. 为什么需要模板

假设我们需要编写一个函数来交换两个整数的值：

```

1 void swapInt(int& a, int& b) {
2     int temp = a;
3     a = b;
4     b = temp;
5 }

```

现在又需要交换两个 `double` 类型的值。由于 C++ 是强类型语言，我们必须编写另一个函数：

```

1 void swapDouble(double& a, double& b) {
2     double temp = a;
3     a = b;
4     b = temp;
5 }

```

如果还需要交换 `string`、`Vector3D`、`Motor` 等类型呢？每种类型都写一个几乎相同的函数，这显然不是好的做法——代码重复、维护困难、容易出错。

函数重载可以让这些函数同名：

```

1 void swap(int& a, int& b) { /* ... */ }
2 void swap(double& a, double& b) { /* ... */ }
3 void swap(std::string& a, std::string& b) { /* ... */ }

```

但这仍然没有解决代码重复的问题。而且，对于用户自定义的类型，我们无法预先提供重载版本。

模板提供了根本性的解决方案：编写一个通用的“模板”，让编译器根据实际使用的类型自动生成具体的函数。

1.5.19.2. 函数模板

函数模板使用 `template` 关键字定义，后跟模板参数列表。模板参数通常是类型参数，用 `typename` 或 `class` 关键字声明：

```

1 template <typename T>
2 void swap(T& a, T& b) {
3     T temp = a;
4     a = b;
5     b = temp;
6 }

```

这里 `T` 是类型参数，代表任意类型。当我们调用这个函数时，编译器会根据实参类型推导出 `T` 的具体类型，并生成相应的函数实例：

```

1 int main() {
2     int x = 10, y = 20;
3     swap(x, y); // T 被推导为 int, 生成 swap<int>
4
5     double a = 1.5, b = 2.5;
6     swap(a, b); // T 被推导为 double, 生成 swap<double>
7
8     std::string s1 = "hello", s2 = "world";
9     swap(s1, s2); // T 被推导为 std::string
10
11     return 0;
12 }

```

编译器为每种使用到的类型生成一个独立的函数版本，这个过程称为模板实例化 (instantiation)。生成的代码与手写的类型特定版本完全相同，没有任何运行时开销。

也可以显式指定模板参数：

```
1 swap<int>(x, y);      // 显式指定 T 为 int
2 swap<double>(a, b);   // 显式指定 T 为 double
```

当编译器无法自动推导类型，或者需要强制使用特定类型时，显式指定是必要的。

函数模板可以有多个类型参数：

```
1 template <typename T, typename U>
2 void printPair(const T& first, const U& second) {
3     std::cout << "(" << first << ", " << second << ")" << std::endl;
4 }
5
6 int main() {
7     printPair(10, 3.14);           // T=int, U=double
8     printPair("hello", 42);        // T=const char*, U=int
9     printPair(std::string("hi"), std::vector<int>{1,2,3}); // 复杂类型也可以
10    return 0;
11 }
```

1.5.19.3. 类模板

类模板允许定义通用的类，其成员的类型可以参数化。STL 容器就是类模板的典型例子。

定义一个简单的类模板：

```
1 template <typename T>
2 class Container {
3 public:
4     Container(const T& value) : data(value) {}
5
6     T getData() const { return data; }
7     void setData(const T& value) { data = value; }
8
9 private:
10    T data;
11 };
```

使用类模板时，必须显式指定类型参数（C++17 之前）：

```
1 int main() {
2     Container<int> intContainer(42);
3     std::cout << intContainer.getData() << std::endl; // 42
4
5     Container<std::string> strContainer("hello");
6     std::cout << strContainer.getData() << std::endl; // hello
7
8     Container<double> dblContainer(3.14);
9     dblContainer.setData(2.71);
10
11    return 0;
12 }
```

C++17 引入了类模板参数推导 (CTAD)，允许编译器从构造函数参数推导类型：

```
1 // C++17 及以后
```

```

2 Container intContainer(42);           // 推导为 Container<int>
3 Container strContainer("hello");      // 推导为 Container<const char*>
4 Container dblContainer(3.14);         // 推导为 Container<double>

```

让我们实现一个更实用的类模板——简化版的动态数组：

```

1  template <typename T>
2  class DynamicArray {
3  public:
4      DynamicArray() : data(nullptr), size(0), capacity(0) {}
5
6      ~DynamicArray() {
7          delete[] data;
8      }
9
10     void push_back(const T& value) {
11         if (size >= capacity) {
12             grow();
13         }
14         data[size++] = value;
15     }
16
17     T& operator[](size_t index) {
18         return data[index];
19     }
20
21     const T& operator[](size_t index) const {
22         return data[index];
23     }
24
25     size_t getSize() const { return size; }
26     bool empty() const { return size == 0; }
27
28 private:
29     void grow() {
30         size_t newCapacity = (capacity == 0) ? 1 : capacity * 2;
31         T* newData = new T[newCapacity];
32         for (size_t i = 0; i < size; i++) {
33             newData[i] = data[i];
34         }
35         delete[] data;
36         data = newData;
37         capacity = newCapacity;
38     }
39
40     T* data;
41     size_t size;
42     size_t capacity;
43 };
44
45 int main() {
46     DynamicArray<int> numbers;
47     numbers.push_back(10);
48     numbers.push_back(20);
49     numbers.push_back(30);
50
51     for (size_t i = 0; i < numbers.getSize(); i++) {
52         std::cout << numbers[i] << " ";
53     }
54     std::cout << std::endl; // 10 20 30
55
56     DynamicArray<std::string> words;

```

```

57     words.push_back("hello");
58     words.push_back("world");
59
60     return 0;
61 }

```

这个简化版展示了类模板的基本结构。真正的 `std::vector` 实现要复杂得多，包括异常安全、移动语义、分配器支持等。

1.5.19.4. 成员函数模板

类（无论是否是模板类）可以拥有成员函数模板：

```

1  class Printer {
2  public:
3      template <typename T>
4      void print(const T& value) {
5          std::cout << value << std::endl;
6      }
7
8      template <typename T, typename U>
9      void printPair(const T& first, const U& second) {
10         std::cout << first << ", " << second << std::endl;
11     }
12 };
13
14 int main() {
15     Printer p;
16     p.print(42);           // 实例化 print<int>
17     p.print(3.14);         // 实例化 print<double>
18     p.print("hello");      // 实例化 print<const char*>
19     p.printPair(1, "one"); // 实例化 printPair<int, const char*>
20     return 0;
21 }

```

类模板也可以有成员函数模板，这在实现通用接口时很有用：

```

1  template <typename T>
2  class Container {
3  public:
4      Container(const T& value) : data(value) {}
5
6      // 成员函数模板：允许从不同类型的 Container 构造
7      template <typename U>
8      Container(const Container<U>& other) : data(other.getData()) {}
9
10     T getData() const { return data; }
11
12 private:
13     T data;
14 };
15
16 int main() {
17     Container<int> intCont(42);
18     Container<double> dblCont(intCont); // 从 Container<int> 构造
19     std::cout << dblCont.getData() << std::endl; // 42.0
20     return 0;
21 }

```


1.5.19.5. 非类型模板参数

除了类型参数，模板还可以有非类型参数（non-type template parameter）。非类型参数必须是编译时常量，可以是整数、枚举、指针或引用：

```
1  template <typename T, size_t N>
2  class FixedArray {
3  public:
4      T& operator[](size_t index) {
5          return data[index];
6      }
7
8      const T& operator[](size_t index) const {
9          return data[index];
10     }
11
12     constexpr size_t size() const { return N; }
13
14 private:
15     T data[N];
16 };
17
18 int main() {
19     FixedArray<int, 5> arr1;      // 5 个 int 的数组
20     FixedArray<double, 10> arr2; // 10 个 double 的数组
21
22     for (size_t i = 0; i < arr1.size(); i++) {
23         arr1[i] = i * 10;
24     }
25
26     // arr1 和 arr2 是不同的类型
27     // FixedArray<int, 5> 与 FixedArray<int, 6> 也是不同的类型
28
29     return 0;
30 }
```

`std::array` 就是使用非类型模板参数的典型例子：`std::array<int, 10>` 表示包含 10 个整数的固定大小数组。

非类型参数还可以用于编译时计算：

```
1  template <int N>
2  struct Factorial {
3      static constexpr int value = N * Factorial<N - 1>::value;
4  };
5
6  template <>
7  struct Factorial<0> {
8      static constexpr int value = 1;
9  };
10
11 int main() {
12     constexpr int fact5 = Factorial<5>::value; // 编译时计算 5! = 120
13     std::cout << "5! = " << fact5 << std::endl;
14     return 0;
15 }
```

这种技术称为模板元编程（template metaprogramming），它在编译时进行计算，可以实现许多高级功能。

1.5.19.6. 默认模板参数

模板参数可以有默认值：

```
1  template <typename T = int, size_t N = 10>
2  class Buffer {
3  public:
4      T& operator[](size_t index) { return data[index]; }
5      size_t size() const { return N; }
6  private:
7      T data[N];
8  };
9
10 int main() {
11     Buffer<> buf1;           // T=int, N=10
12     Buffer<double> buf2;     // T=double, N=10
13     Buffer<char, 256> buf3;  // T=char, N=256
14     return 0;
15 }
```

STL 容器广泛使用默认模板参数。例如，`std::vector` 的完整声明是：

```
1  template <typename T, typename Allocator = std::allocator<T>>
2  class vector;
```

大多数情况下我们只需要指定 `T`，分配器使用默认值。

1.5.19.7. 模板特化

有时候，通用的模板实现对某些特定类型不适用或效率不高。模板特化（template specialization）允许为特定类型提供定制实现。

全特化（full specialization）为所有模板参数指定具体类型：

```
1  // 通用版本
2  template <typename T>
3  class TypeInfo {
4  public:
5      static std::string name() { return "unknown"; }
6  };
7
8  // 针对 int 的特化
9  template <>
10 class TypeInfo<int> {
11 public:
12     static std::string name() { return "int"; }
13 };
14
15 // 针对 double 的特化
16 template <>
17 class TypeInfo<double> {
18 public:
19     static std::string name() { return "double"; }
20 };
21
22 // 针对 std::string 的特化
23 template <>
24 class TypeInfo<std::string> {
25 public:
26     static std::string name() { return "std::string"; }
27 };
28
```

```

29 int main() {
30     std::cout << TypeInfo<int>::name() << std::endl;           // int
31     std::cout << TypeInfo<double>::name() << std::endl;        // double
32     std::cout << TypeInfo<std::string>::name() << std::endl;    // std::string
33     std::cout << TypeInfo<char>::name() << std::endl;          // unknown
34     return 0;
35 }

```

偏特化 (partial specialization) 只特化部分参数, 或对参数施加约束:

```

1 // 通用版本
2 template <typename T>
3 class Container {
4 public:
5     void info() { std::cout << "Generic container" << std::endl; }
6 };
7
8 // 针对指针类型的偏特化
9 template <typename T>
10 class Container<T*> {
11 public:
12     void info() { std::cout << "Pointer container" << std::endl; }
13 };
14
15 // 针对数组类型的偏特化
16 template <typename T, size_t N>
17 class Container<T[N]> {
18 public:
19     void info() { std::cout << "Array container of size " << N <<
20         std::endl; }
21 };
22
23 int main() {
24     Container<int> c1;
25     Container<int*> c2;
26     Container<double[10]> c3;
27
28     c1.info(); // Generic container
29     c2.info(); // Pointer container
30     c3.info(); // Array container of size 10
31
32     return 0;
33 }

```

注意: 函数模板不支持偏特化, 但可以通过重载达到类似效果。

1.5.19.8. 模板与编译

理解模板的编译模型对于避免常见错误很重要。

模板本身不是代码, 而是生成代码的蓝图。只有当模板被实例化时, 编译器才会生成实际的代码。这意味着:

模板定义通常放在头文件中。由于编译器需要看到完整的模板定义才能实例化, 模板的声明和定义通常都放在头文件中, 而不像普通函数那样声明在头文件、定义在源文件。

```

1 // MyTemplate.h
2 #ifndef MY_TEMPLATE_H
3 #define MY_TEMPLATE_H
4
5 template <typename T>

```

```

6 class MyTemplate {
7 public:
8     void foo();
9     void bar();
10 };
11
12 // 成员函数定义也在头文件中
13 template <typename T>
14 void MyTemplate<T>::foo() {
15     // 实现
16 }
17
18 template <typename T>
19 void MyTemplate<T>::bar() {
20     // 实现
21 }
22
23 #endif

```

未使用的模板代码不会被编译。如果模板中有语法错误, 但该模板从未被实例化, 错误可能不会被发现。

类型检查发生在实例化时。模板代码对类型的要求(如需要 `operator<`)只有在实例化时才会被检查:

```

1 template <typename T>
2 T maximum(const T& a, const T& b) {
3     return (a > b) ? a : b; // 要求 T 支持 operator>
4 }
5
6 struct NoCompare {
7     int value;
8 };
9
10 int main() {
11     maximum(10, 20); // 正确, int 支持 >
12
13     NoCompare nc1{1}, nc2{2};
14     // maximum(nc1, nc2); // 错误! NoCompare 不支持 >
15
16     return 0;
17 }

```

1.5.19.9. 实际应用示例

让我们看几个模板在 RoboMaster 开发中的实际应用。

通用的数值范围限制函数:

```

1 template <typename T>
2 T clamp(const T& value, const T& minVal, const T& maxVal) {
3     if (value < minVal) return minVal;
4     if (value > maxVal) return maxVal;
5     return value;
6 }
7
8 // 使用
9 double speed = clamp(rawSpeed, -1000.0, 1000.0);
10 int pwm = clamp(calculatedPwm, 0, 255);

```

通用的 PID 控制器:

```

1  template <typename T>
2  class PIDController {
3  public:
4      PIDController(T kp, T ki, T kd)
5          : kp(kp), ki(ki), kd(kd), integral(0), lastError(0) {}
6
7      T compute(T setpoint, T measurement, T dt) {
8          T error = setpoint - measurement;
9          integral += error * dt;
10         T derivative = (error - lastError) / dt;
11         lastError = error;
12         return kp * error + ki * integral + kd * derivative;
13     }
14
15     void reset() {
16         integral = 0;
17         lastError = 0;
18     }
19
20 private:
21     T kp, ki, kd;
22     T integral;
23     T lastError;
24 };
25
26 // 使用 double 精度
27 PIDController<double> speedController(1.5, 0.1, 0.05);
28
29 // 使用 float 精度 (嵌入式系统中常见)
30 PIDController<float> angleController(2.0f, 0.0f, 0.1f);

```

通用的环形缓冲区:

```

1  template <typename T, size_t N>
2  class RingBuffer {
3  public:
4      RingBuffer() : head(0), tail(0), count(0) {}
5
6      bool push(const T& value) {
7          if (count >= N) return false; // 缓冲区满
8          data[tail] = value;
9          tail = (tail + 1) % N;
10         count++;
11         return true;
12     }
13
14     bool pop(T& value) {
15         if (count == 0) return false; // 缓冲区空
16         value = data[head];
17         head = (head + 1) % N;
18         count--;
19         return true;
20     }
21
22     bool empty() const { return count == 0; }
23     bool full() const { return count >= N; }
24     size_t size() const { return count; }
25
26 private:
27     T data[N];
28     size_t head;
29     size_t tail;

```

```

30     size_t count;
31 };
32
33 // 用于存储最近 100 个传感器读数
34 RingBuffer<SensorReading, 100> sensorHistory;
35
36 // 用于通信数据缓冲
37 RingBuffer<uint8_t, 256> rxBuffer;

```

通用的单例模式：

```

1  template <typename T>
2  class Singleton {
3  public:
4      static T& getInstance() {
5          static T instance;
6          return instance;
7      }
8
9      // 禁止拷贝和赋值
10     Singleton(const Singleton&) = delete;
11     Singleton& operator=(const Singleton&) = delete;
12
13 protected:
14     Singleton() = default;
15     ~Singleton() = default;
16 };
17
18 // 使用单例
19 class ConfigManager : public Singleton<ConfigManager> {
20     friend class Singleton<ConfigManager>;
21 public:
22     void loadConfig(const std::string& filename);
23     int getValue(const std::string& key);
24 private:
25     ConfigManager() = default;
26     std::map<std::string, int> config;
27 };
28
29 // 访问单例
30 ConfigManager::getInstance().loadConfig("robot.cfg");

```

1.5.19.10. 模板的优缺点

模板是强大的工具，但也有其局限性。

优点包括：

代码复用：一次编写，适用于多种类型。

类型安全：编译时类型检查，不像 `void*` 那样丢失类型信息。

零运行时开销：模板实例化发生在编译时，生成的代码与手写的特定类型代码效率相同。

强大的抽象能力：可以表达复杂的类型关系和约束。

缺点包括：

编译时间增加：每个翻译单元都可能实例化相同的模板，增加编译时间。

代码膨胀：每种类型实例化都会生成独立的代码，可能增加可执行文件大小。

错误信息复杂：模板相关的编译错误信息通常很长且难以理解。

调试困难：模板代码的调试比普通代码更具挑战性。

1.5.19.11. 现代 C++ 中的模板改进

C++11 及以后的标准对模板进行了许多改进。

变参模板 (variadic templates) 允许模板接受任意数量的参数:

```
1 // 递归终止
2 void print() {
3     std::cout << std::endl;
4 }
5
6 // 变参模板
7 template <typename T, typename... Args>
8 void print(const T& first, const Args&... rest) {
9     std::cout << first << " ";
10    print(rest...); // 递归展开
11 }
12
13 int main() {
14     print(1, 2.5, "hello", 'c'); // 1 2.5 hello c
15     return 0;
16 }
```

`auto` 和 `decltype` 简化了复杂类型的声明:

```
1 template <typename T, typename U>
2 auto add(const T& a, const U& b) -> decltype(a + b) {
3     return a + b;
4 }
5
6 // C++14 可以省略尾置返回类型
7 template <typename T, typename U>
8 auto multiply(const T& a, const U& b) {
9     return a * b;
10 }
```

C++20 引入了概念 (concepts), 可以对模板参数施加约束, 提供更好的错误信息:

```
1 // C++20 概念
2 template <typename T>
3 concept Comparable = requires(T a, T b) {
4     { a < b } -> std::convertible_to<bool>;
5     { a > b } -> std::convertible_to<bool>;
6 };
7
8 template <Comparable T>
9 T maximum(const T& a, const T& b) {
10     return (a > b) ? a : b;
11 }
```

模板是 C++ 最强大的特性之一, 也是 STL 的基础。通过本节的学习, 你应该能够理解 STL 容器和算法的工作原理, 并能够编写自己的模板代码。模板的高级用法 (如 SFINAE、模板元编程、类型萃取等) 超出了本教程的范围, 但掌握基础知识已经足以应对大多数实际需求。下一节我们将学习智能指针, 它们是现代 C++ 内存管理的核心工具。

1.5.20. 智能指针

在前面的章节中, 我们多次提到动态内存管理的挑战: 使用 `new` 分配的内存必须用 `delete` 释放, 否则会造成内存泄漏; 如果释放得太早或重复释放, 程序会崩溃或产生未定义行

为。手动管理内存不仅繁琐，还极易出错，特别是在存在多个执行路径（如异常、提前返回）的复杂代码中。

C++11 引入的智能指针（smart pointer）从根本上解决了这个问题。智能指针是一种封装了原始指针的类模板，它在适当的时机自动释放所管理的内存，遵循 RAII 原则。使用智能指针，你几乎不再需要直接调用 `new` 和 `delete`，内存管理变得安全而自动。现代 C++ 代码中，智能指针已经成为管理动态分配对象的标准方式。

1.5.20.1. 原始指针的问题

让我们先回顾一下原始指针管理动态内存时面临的问题：

```
1 void processData() {
2     int* data = new int[1000];
3
4     // 处理数据...
5
6     if (someError) {
7         return; // 忘记 delete! 内存泄漏
8     }
9
10    // 更多处理...
11
12    if (anotherError) {
13        throw std::runtime_error("处理失败"); // 异常导致 delete 不执行
14    }
15
16    delete[] data; // 只有正常执行到这里才会释放
17 }
```

这段代码有多问题：提前返回和异常都会导致内存泄漏。要正确处理，需要在每个退出点都添加 `delete`：

```
1 void processData() {
2     int* data = new int[1000];
3
4     try {
5         if (someError) {
6             delete[] data;
7             return;
8         }
9
10        // ...
11
12        if (anotherError) {
13            delete[] data;
14            throw std::runtime_error("处理失败");
15        }
16
17        delete[] data;
18    } catch (...) {
19        delete[] data;
20        throw;
21    }
22 }
```

代码变得冗长且容易出错。智能指针通过 RAII 优雅地解决了这个问题：当智能指针离开作用域时，析构函数自动释放资源，无论是正常退出、提前返回还是异常抛出。

1.5.20.2. unique_ptr: 独占所有权

`std::unique_ptr` 表示对动态分配对象的独占所有权。同一时刻只有一个 `unique_ptr` 可以指向特定对象，当 `unique_ptr` 被销毁时，它所管理的对象也随之被删除。

基本用法:

```
1 #include <memory>
2 #include <iostream>
3
4 class Motor {
5 public:
6     Motor(int id) : id(id) {
7         std::cout << "Motor " << id << " 创建" << std::endl;
8     }
9     ~Motor() {
10         std::cout << "Motor " << id << " 销毁" << std::endl;
11     }
12     void run() {
13         std::cout << "Motor " << id << " 运行中" << std::endl;
14     }
15 private:
16     int id;
17 };
18
19 int main() {
20     // 创建 unique_ptr
21     std::unique_ptr<Motor> motor1(new Motor(1));
22
23     // 推荐: 使用 make_unique (C++14)
24     auto motor2 = std::make_unique<Motor>(2);
25
26     // 使用智能指针 (像普通指针一样)
27     motor1->run();
28     motor2->run();
29
30     // 检查是否为空
31     if (motor1) {
32         std::cout << "motor1 有效" << std::endl;
33     }
34
35     return 0;
36 } // motor1 和 motor2 离开作用域, 自动销毁所管理的对象
```

输出:

```
1 Motor 1 创建
2 Motor 2 创建
3 Motor 1 运行中
4 Motor 2 运行中
5 motor1 有效
6 Motor 2 销毁
7 Motor 1 销毁
```

`std::make_unique` 是创建 `unique_ptr` 的推荐方式，它更安全（异常安全）、更简洁，且避免了显式使用 `new`。

`unique_ptr` 的核心特性是不可复制，这确保了独占所有权的语义：

```
1 auto ptr1 = std::make_unique<Motor>(1);
2
3 // std::unique_ptr<Motor> ptr2 = ptr1; // 错误! 不能复制
```

```

4 // auto ptr3 = ptr1; // 错误! 不能复制
5
6 // 但可以移动
7 std::unique_ptr<Motor> ptr2 = std::move(ptr1); // 所有权转移
8 // 现在 ptr1 为空, ptr2 拥有对象
9
10 if (!ptr1) {
11     std::cout << "ptr1 现在为空" << std::endl;
12 }

```

移动后, 原来的 `unique_ptr` 变为空 (`nullptr`), 所有权完全转移到新的 `unique_ptr`。

`unique_ptr` 常用的操作:

```

1 auto ptr = std::make_unique<Motor>(1);
2
3 // 获取原始指针 (不转移所有权)
4 Motor* raw = ptr.get();
5
6 // 释放所有权, 返回原始指针 (调用者负责删除)
7 Motor* released = ptr.release();
8 // ptr 现在为空, 调用者必须手动 delete released
9
10 // 重置: 删除当前对象, 可选地管理新对象
11 ptr.reset(); // 删除并置空
12 ptr.reset(new Motor(2)); // 删除旧对象, 管理新对象
13
14 // 交换
15 auto ptr2 = std::make_unique<Motor>(3);
16 ptr.swap(ptr2);

```

`unique_ptr` 可以管理数组:

```

1 // 管理动态数组
2 auto arr = std::make_unique<int[]>(100);
3 arr[0] = 10;
4 arr[99] = 20;
5 // 自动调用 delete[]

```

在函数参数和返回值中使用 `unique_ptr`:

```

1 // 工厂函数: 返回 unique_ptr 表示调用者获得所有权
2 std::unique_ptr<Motor> createMotor(int id) {
3     return std::make_unique<Motor>(id);
4 }
5
6 // 接收 unique_ptr: 转移所有权 (接收者负责生命周期)
7 void takeOwnership(std::unique_ptr<Motor> motor) {
8     motor->run();
9 } // motor 在这里被销毁
10
11 // 只使用不获取所有权: 传递引用或原始指针
12 void useMotor(Motor& motor) {
13     motor.run();
14 }
15
16 void useMotorPtr(Motor* motor) {
17     if (motor) motor->run();
18 }
19
20 int main() {
21     auto motor = createMotor(1);

```

```

22
23     useMotor(*motor);           // 传递引用
24     useMotorPtr(motor.get());   // 传递原始指针
25
26     takeOwnership(std::move(motor)); // 转移所有权
27     // motor 现在为空
28
29     return 0;
30 }

```

1.5.20.3. shared_ptr: 共享所有权

`std::shared_ptr` 允许多个指针共享同一个对象的所有权。它使用引用计数来跟踪有多少个 `shared_ptr` 指向同一对象，当最后一个 `shared_ptr` 被销毁时，对象才被删除。

```

1  #include <memory>
2  #include <iostream>
3
4  class Sensor {
5  public:
6      Sensor(const std::string& name) : name(name) {
7          std::cout << "Sensor " << name << " 创建" << std::endl;
8      }
9      ~Sensor() {
10         std::cout << "Sensor " << name << " 销毁" << std::endl;
11     }
12     std::string getName() const { return name; }
13 private:
14     std::string name;
15 };
16
17 int main() {
18     std::shared_ptr<Sensor> sensor1 = std::make_shared<Sensor>("IMU");
19     std::cout << "引用计数: " << sensor1.use_count() << std::endl; // 1
20
21     {
22         std::shared_ptr<Sensor> sensor2 = sensor1; // 共享所有权
23         std::cout << "引用计数: " << sensor1.use_count() << std::endl; // 2
24
25         std::shared_ptr<Sensor> sensor3 = sensor1;
26         std::cout << "引用计数: " << sensor1.use_count() << std::endl; // 3
27     } // sensor2 和 sensor3 销毁，但对象还在
28
29     std::cout << "引用计数: " << sensor1.use_count() << std::endl; // 1
30
31     return 0;
32 } // sensor1 销毁，引用计数变为 0，对象被删除

```

输出:

```

1  Sensor IMU 创建
2  引用计数: 1
3  引用计数: 2
4  引用计数: 3
5  引用计数: 1
6  Sensor IMU 销毁

```

与 `unique_ptr` 不同，`shared_ptr` 可以复制:

```

1 auto ptr1 = std::make_shared<Sensor>("Camera");
2 auto ptr2 = ptr1; // 复制, 共享所有权
3 auto ptr3 = ptr1; // 再复制
4
5 // 三个指针指向同一个对象
6 std::cout << ptr1.get() << std::endl;
7 std::cout << ptr2.get() << std::endl; // 相同地址
8 std::cout << ptr3.get() << std::endl; // 相同地址

```

`std::make_shared` 是创建 `shared_ptr` 的推荐方式。它比直接使用 `new` 更高效, 因为它只进行一次内存分配 (同时分配对象和控制块), 而 `shared_ptr<T>(new T(...))` 需要两次分配:

```

1 // 推荐
2 auto ptr1 = std::make_shared<Sensor>("Lidar");
3
4 // 不推荐 (两次内存分配)
5 std::shared_ptr<Sensor> ptr2(new Sensor("Radar"));

```

`shared_ptr` 在多个对象需要共同访问同一资源时非常有用:

```

1 class TargetTracker {
2 public:
3     void setTarget(std::shared_ptr<Target> t) {
4         target = t;
5     }
6
7     void track() {
8         if (target) {
9             // 跟踪目标
10        }
11    }
12
13 private:
14     std::shared_ptr<Target> target;
15 };
16
17 class WeaponSystem {
18 public:
19     void setTarget(std::shared_ptr<Target> t) {
20         target = t;
21     }
22
23     void aim() {
24         if (target) {
25             // 瞄准目标
26        }
27    }
28
29 private:
30     std::shared_ptr<Target> target;
31 };
32
33 int main() {
34     auto target = std::make_shared<Target>(100, 200, 50);
35
36     TargetTracker tracker;
37     WeaponSystem weapon;
38
39     // 两个系统共享同一个目标
40     tracker.setTarget(target);

```

```

41     weapon.setTarget(target);
42
43     // 即使 target 变量被销毁, tracker 和 weapon 仍持有目标
44     target.reset();
45
46     // 目标仍然有效, 直到 tracker 和 weapon 都释放
47     tracker.track();
48     weapon.aim();
49
50     return 0;
51 }

```

1.5.20.4. weak_ptr: 弱引用

`std::weak_ptr` 是 `shared_ptr` 的补充。它持有对象的非拥有（“弱”）引用——不增加引用计数，不影响对象的生命周期。`weak_ptr` 主要用于解决 `shared_ptr` 的循环引用问题，以及在不延长对象生命周期的情况下观察对象。

循环引用问题：

```

1  class Node {
2  public:
3      std::string name;
4      std::shared_ptr<Node> next; // 指向下一个节点
5      std::shared_ptr<Node> prev; // 指向上一个节点——问题所在!
6
7      Node(const std::string& n) : name(n) {
8          std::cout << "Node " << name << " 创建" << std::endl;
9      }
10     ~Node() {
11         std::cout << "Node " << name << " 销毁" << std::endl;
12     }
13 };
14
15 int main() {
16     auto node1 = std::make_shared<Node>("A");
17     auto node2 = std::make_shared<Node>("B");
18
19     node1->next = node2;
20     node2->prev = node1; // 循环引用!
21
22     return 0;
23 } // node1 和 node2 离开作用域, 但对象不会被销毁!
24 // 因为 node1 被 node2->prev 引用, node2 被 node1->next 引用
25 // 两个对象的引用计数都是 1, 永远不会变成 0

```

使用 `weak_ptr` 打破循环：

```

1  class Node {
2  public:
3      std::string name;
4      std::shared_ptr<Node> next;
5      std::weak_ptr<Node> prev; // 使用 weak_ptr 打破循环
6
7      Node(const std::string& n) : name(n) {
8          std::cout << "Node " << name << " 创建" << std::endl;
9      }
10     ~Node() {
11         std::cout << "Node " << name << " 销毁" << std::endl;
12     }

```

```

13 };
14
15 int main() {
16     auto node1 = std::make_shared<Node>("A");
17     auto node2 = std::make_shared<Node>("B");
18
19     node1->next = node2;
20     node2->prev = node1; // weak_ptr 不增加引用计数
21
22     return 0;
23 } // 正确销毁

```

输出:

```

1 Node A 创建
2 Node B 创建
3 Node B 销毁
4 Node A 销毁

```

`weak_ptr` 不能直接访问对象, 必须先转换为 `shared_ptr`。这是因为 `weak_ptr` 所指向的对象可能已经被删除:

```

1 std::weak_ptr<Sensor> weakSensor;
2
3 {
4     auto sensor = std::make_shared<Sensor>("GPS");
5     weakSensor = sensor;
6
7     // 方式一: lock() 返回 shared_ptr, 如果对象已删除则返回空
8     if (auto sp = weakSensor.lock()) {
9         std::cout << "Sensor 有效: " << sp->getName() << std::endl;
10    }
11
12    // 方式二: expired() 检查对象是否已删除
13    if (!weakSensor.expired()) {
14        auto sp = weakSensor.lock();
15        // 使用 sp...
16    }
17
18 } // sensor 销毁
19
20 // 现在对象已删除
21 if (weakSensor.expired()) {
22     std::cout << "Sensor 已失效" << std::endl;
23 }
24
25 if (auto sp = weakSensor.lock()) {
26     // 不会执行
27 } else {
28     std::cout << "无法获取 Sensor" << std::endl;
29 }

```

`weak_ptr` 的另一个用途是缓存: 持有对象的弱引用, 如果对象仍存在则复用, 否则重新创建:

```

1 class ResourceCache {
2 public:
3     std::shared_ptr<Texture> getTexture(const std::string& name) {
4         // 检查缓存
5         auto it = cache.find(name);

```

```

6         if (it != cache.end()) {
7             // 尝试获取已缓存的纹理
8             if (auto sp = it->second.lock()) {
9                 std::cout << "缓存命中: " << name << std::endl;
10                return sp;
11            }
12        }
13
14        // 缓存未命中或已过期, 加载新纹理
15        std::cout << "加载纹理: " << name << std::endl;
16        auto texture = std::make_shared<Texture>(name);
17        cache[name] = texture;
18        return texture;
19    }
20
21 private:
22     std::unordered_map<std::string, std::weak_ptr<Texture>> cache;
23 };

```

1.5.20.5. 智能指针与自定义删除器

默认情况下, 智能指针使用 `delete` 或 `delete[]` 释放资源。但有时需要自定义释放方式, 比如关闭文件句柄、释放网络连接等。可以通过自定义删除器实现:

```

1 // 自定义删除器: 函数
2 void closeFile(FILE* fp) {
3     if (fp) {
4         std::cout << "关闭文件" << std::endl;
5         fclose(fp);
6     }
7 }
8
9 // 使用函数作为删除器
10 std::unique_ptr<FILE, decltype(&closeFile)> file(fopen("data.txt", "r"),
11         closeFile);
12
13 // 使用 lambda 作为删除器
14 auto file2 = std::unique_ptr<FILE, void(*)(FILE*)>(
15     fopen("log.txt", "w"),
16     [](FILE* fp) {
17         if (fp) {
18             std::cout << "关闭日志文件" << std::endl;
19             fclose(fp);
20         }
21     });
22
23 // shared_ptr 的删除器更简单 (类型擦除)
24 std::shared_ptr<FILE> file3(
25     fopen("config.txt", "r"),
26     [](FILE* fp) {
27         if (fp) fclose(fp);
28     });
29 );

```

在 RoboMaster 开发中, 自定义删除器可用于管理硬件资源:

```

1 class SerialPort {
2 public:
3     static std::unique_ptr<SerialPort, void(*)(SerialPort*)> open(const
        std::string& device) {

```

```

4      SerialPort* port = new SerialPort(device);
5      if (!port->isOpen()) {
6          delete port;
7          return {nullptr, [](SerialPort*){}};
8      }
9      return {port, [](SerialPort* p) {
10         if (p) {
11             p->close();
12             delete p;
13         }
14     }};
15 }
16
17 private:
18     SerialPort(const std::string& device);
19     void close();
20     bool isOpen() const;
21     int fd;
22 };
23
24 // 使用
25 auto port = SerialPort::open("/dev/ttyUSB0");
26 if (port) {
27     // 使用串口...
28 } // 自动关闭并释放

```

1.5.20.6. enable_shared_from_this

有时对象内部需要获取指向自身的 `shared_ptr`。直接从 `this` 创建 `shared_ptr` 是错误的，因为这会创建独立的引用计数：

```

1 class BadExample {
2 public:
3     std::shared_ptr<BadExample> getShared() {
4         return std::shared_ptr<BadExample>(this); // 错误!
5     }
6 };
7
8 auto ptr1 = std::make_shared<BadExample>();
9 auto ptr2 = ptr1->getShared(); // ptr2 有独立的引用计数
10 // 当 ptr1 或 ptr2 销毁时，对象会被重复删除!

```

正确的做法是继承 `std::enable_shared_from_this`：

```

1 class GoodExample : public std::enable_shared_from_this<GoodExample> {
2 public:
3     std::shared_ptr<GoodExample> getShared() {
4         return shared_from_this(); // 正确!
5     }
6
7     void registerCallback() {
8         // 将自身注册到某个系统
9         callbackManager.register(shared_from_this());
10    }
11 };
12
13 auto ptr1 = std::make_shared<GoodExample>();
14 auto ptr2 = ptr1->getShared(); // ptr2 共享 ptr1 的引用计数
15 std::cout << ptr1.use_count() << std::endl; // 2

```


注意: `shared_from_this()` 只能在对象已经被 `shared_ptr` 管理时调用。在构造函数中调用会导致异常。

1.5.20.7. 如何选择智能指针

选择正确的智能指针对于编写清晰、高效的代码至关重要。

使用 `unique_ptr` 当:

- 只有一个所有者需要管理对象的生命周期
- 需要工厂函数返回动态分配的对象
- 需要将对象存储在容器中, 但保持独占所有权
- 作为类的成员变量, 管理该类独占的资源

```
1 class Robot {
2 public:
3     Robot() : controller(std::make_unique<PIDController>(1.0, 0.1, 0.05)) {}
4
5 private:
6     std::unique_ptr<PIDController> controller; // Robot 独占 controller
7 };
8
9 std::unique_ptr<Motor> createMotor(int id) {
10     return std::make_unique<Motor>(id); // 工厂函数
11 }
```

使用 `shared_ptr` 当:

- 多个对象需要共享同一资源的所有权
- 不确定哪个对象最后使用资源
- 需要将指针存储在多个容器或对象中

```
1 class Scene {
2 public:
3     void addObject(std::shared_ptr<GameObject> obj) {
4         objects.push_back(obj);
5     }
6
7 private:
8     std::vector<std::shared_ptr<GameObject>> objects;
9 };
10
11 // 同一个对象可以被多个场景共享
12 auto player = std::make_shared<Player>();
13 scene1.addObject(player);
14 scene2.addObject(player);
```

使用 `weak_ptr` 当:

- 需要打破 `shared_ptr` 的循环引用
- 需要观察对象但不想延长其生命周期
- 实现缓存, 允许对象在不再使用时被释放

```
1 class Observer {
2 public:
3     void observe(std::shared_ptr<Subject> subject) {
4         this->subject = subject; // weak_ptr 不延长 subject 的生命周期
5     }
6 }
```

```

7     void notify() {
8         if (auto sp = subject.lock()) {
9             // subject 仍然存在
10        }
11    }
12
13 private:
14     std::weak_ptr<Subject> subject;
15 };

```

使用原始指针（非拥有）当：

- 函数只是使用对象，不管理其生命周期
- 生命周期由调用者明确保证
- 需要表示可选的引用（可为 `nullptr`）

```

1 void processMotor(Motor* motor) { // 只使用，不拥有
2     if (motor) {
3         motor->run();
4     }
5 }
6
7 auto motor = std::make_unique<Motor>(1);
8 processMotor(motor.get()); // 传递原始指针

```

使用引用当：

- 对象一定存在（不可为空）
- 只是使用对象，不管理生命周期

```

1 void processMotor(Motor& motor) { // 对象必须存在
2     motor.run();
3 }
4
5 auto motor = std::make_unique<Motor>(1);
6 processMotor(*motor); // 传递引用

```

1.5.20.8. 性能考虑

智能指针有一些性能开销需要了解：

`unique_ptr` 几乎零开销。它的大小通常与原始指针相同（除非使用了有状态的删除器），所有操作都可以被编译器优化为与原始指针等效的代码。

`shared_ptr` 有一定开销：

- 额外的内存用于存储引用计数（控制块）
- 复制时需要原子操作更新引用计数（线程安全但有开销）
- 比 `unique_ptr` 多一次间接寻址

```

1 // 大小比较
2 std::cout << sizeof(int*) << std::endl; // 8 (64位系统)
3 std::cout << sizeof(std::unique_ptr<int>) << std::endl; // 8
4 std::cout << sizeof(std::shared_ptr<int>) << std::endl; // 16 (包含控制块
   指针)
5 std::cout << sizeof(std::weak_ptr<int>) << std::endl; // 16

```

优化建议：

- 优先使用 `unique_ptr`，只在确实需要共享所有权时使用 `shared_ptr`

- 传递智能指针时，如果不涉及所有权转移，传递引用或原始指针
- 避免不必要的 `shared_ptr` 复制，使用 `const shared_ptr<T>&` 传参
- 使用 `make_unique` 和 `make_shared` 而非直接 `new`

```

1 // 不好: 不必要地复制 shared_ptr
2 void process(std::shared_ptr<Data> data) { // 复制, 增加引用计数
3     // ...
4 }
5
6 // 好: 如果不需要共享所有权, 传递引用
7 void process(const std::shared_ptr<Data>& data) { // 不复制
8     // ...
9 }
10
11 // 更好: 如果只是使用, 传递原始引用
12 void process(const Data& data) { // 不涉及智能指针
13     // ...
14 }

```

1.5.20.9. 实际应用示例

让我们看一个综合示例，展示智能指针在 RoboMaster 开发中的应用：

```

1 #include <memory>
2 #include <vector>
3 #include <iostream>
4
5 // 前向声明
6 class RobotSystem;
7
8 // 传感器基类
9 class Sensor {
10 public:
11     virtual ~Sensor() = default;
12     virtual void update() = 0;
13     virtual std::string getName() const = 0;
14 };
15
16 // 具体传感器
17 class IMU : public Sensor {
18 public:
19     IMU() { std::cout << "IMU 初始化" << std::endl; }
20     ~IMU() { std::cout << "IMU 关闭" << std::endl; }
21
22     void update() override {
23         std::cout << "IMU 更新数据" << std::endl;
24     }
25
26     std::string getName() const override { return "IMU"; }
27 };
28
29 class Camera : public Sensor {
30 public:
31     Camera() { std::cout << "Camera 初始化" << std::endl; }
32     ~Camera() { std::cout << "Camera 关闭" << std::endl; }
33
34     void update() override {
35         std::cout << "Camera 捕获图像" << std::endl;
36     }
37
38     std::string getName() const override { return "Camera"; }

```

```

39 };
40
41 // 目标（可能被多个系统共享）
42 class Target : public std::enable_shared_from_this<Target> {
43 public:
44     Target(double x, double y) : x(x), y(y) {
45         std::cout << "Target 创建于 (" << x << ", " << y << ")" <<
std::endl;
46     }
47
48     ~Target() {
49         std::cout << "Target 销毁" << std::endl;
50     }
51
52     void updatePosition(double newX, double newY) {
53         x = newX;
54         y = newY;
55     }
56
57     std::shared_ptr<Target> getPtr() {
58         return shared_from_this();
59     }
60
61 private:
62     double x, y;
63 };
64
65 // 追踪器（观察目标但不拥有）
66 class Tracker {
67 public:
68     void setTarget(std::shared_ptr<Target> t) {
69         target = t; // weak_ptr, 不延长目标生命周期
70     }
71
72     void track() {
73         if (auto t = target.lock()) {
74             std::cout << "正在追踪目标" << std::endl;
75         } else {
76             std::cout << "目标已丢失" << std::endl;
77         }
78     }
79
80 private:
81     std::weak_ptr<Target> target;
82 };
83
84 // 机器人系统
85 class RobotSystem {
86 public:
87     RobotSystem() {
88         std::cout << "=== 机器人系统启动 ===" << std::endl;
89     }
90
91     ~RobotSystem() {
92         std::cout << "=== 机器人系统关闭 ===" << std::endl;
93     }
94
95     // 添加传感器（独占所有权）
96     void addSensor(std::unique_ptr<Sensor> sensor) {
97         std::cout << "添加传感器: " << sensor->getName() << std::endl;
98         sensors.push_back(std::move(sensor));
99     }
100

```

```

101 // 设置当前目标（共享所有权）
102 void setTarget(std::shared_ptr<Target> t) {
103     currentTarget = t;
104     tracker.setTarget(t);
105 }
106
107 // 更新所有传感器
108 void updateSensors() {
109     for (auto& sensor : sensors) {
110         sensor->update();
111     }
112 }
113
114 // 追踪目标
115 void trackTarget() {
116     tracker.track();
117 }
118
119 private:
120     std::vector<std::unique_ptr<Sensor>> sensors; // 独占传感器
121     std::shared_ptr<Target> currentTarget;      // 共享目标
122     Tracker tracker;
123 };
124
125 int main() {
126     auto robot = std::make_unique<RobotSystem>();
127
128     // 添加传感器（转移所有权给 robot）
129     robot->addSensor(std::make_unique<IMU>());
130     robot->addSensor(std::make_unique<Camera>());
131
132     // 创建共享目标
133     auto target = std::make_shared<Target>(100, 200);
134     robot->setTarget(target);
135
136     // 运行
137     robot->updateSensors();
138     robot->trackTarget();
139
140     std::cout << "\n--- 目标引用计数: " << target.use_count() << " ---\n"
141 << std::endl;
142
143     // 释放外部目标引用
144     target.reset();
145     std::cout << "外部目标引用已释放" << std::endl;
146
147     // 目标仍然存在（robot 持有）
148     robot->trackTarget();
149
150     std::cout << "\n--- 关闭机器人 ---\n" << std::endl;
151     robot.reset(); // 销毁机器人，所有资源自动释放
152
153     return 0;
154 }

```

输出:

```

1 === 机器人系统启动 ===
2 IMU 初始化
3 添加传感器: IMU
4 Camera 初始化
5 添加传感器: Camera

```

```

6 Target 创建于 (100, 200)
7 IMU 更新数据
8 Camera 捕获图像
9 正在追踪目标
10
11 --- 目标引用计数: 2 ---
12
13 外部目标引用已释放
14 正在追踪目标
15
16 --- 关闭机器人 ---
17
18 === 机器人系统关闭 ===
19 Target 销毁
20 Camera 关闭
21 IMU 关闭

```

这个示例展示了三种智能指针的典型用法：`unique_ptr` 管理独占资源（传感器），`shared_ptr` 管理共享资源（目标），`weak_ptr` 观察资源而不影响生命周期（追踪器）。

智能指针是现代 C++ 内存管理的基石。通过使用智能指针，你可以编写出更安全、更清晰的代码，几乎完全消除内存泄漏和悬空指针的风险。记住：优先使用 `unique_ptr`，需要共享时使用 `shared_ptr`，打破循环或观察时使用 `weak_ptr`。下一节我们将学习移动语义，它与智能指针密切相关，是理解现代 C++ 性能优化的关键。

1.5.21. 右值引用与移动语义

在前面的章节中，我们多次提到“移动”这个概念：`unique_ptr` 不能复制但可以移动，`std::move` 可以将对象的所有权转移给另一个对象。那么，移动到底是什么？为什么需要移动？这一节将深入探讨 C++11 引入的右值引用（rvalue reference）和移动语义（move semantics），它们是现代 C++ 性能优化的核心技术。

1.5.21.1. 从问题出发

假设有一个存储大量数据的类：

```

1 class BigData {
2 public:
3     BigData(size_t size) : size(size), data(new int[size]) {
4         std::cout << "构造: 分配 " << size << " 个整数" << std::endl;
5     }
6
7     // 拷贝构造函数
8     BigData(const BigData& other) : size(other.size), data(new
9 int[other.size]) {
10         std::copy(other.data, other.data + size, data);
11         std::cout << "拷贝构造: 复制 " << size << " 个整数" << std::endl;
12     }
13
14     ~BigData() {
15         delete[] data;
16         std::cout << "析构: 释放内存" << std::endl;
17     }
18 private:
19     size_t size;
20     int* data;
21 };
22

```

```

23  BigInt createBigInt() {
24      BigInt temp(1000000); // 创建临时对象
25      return temp;          // 返回时会拷贝
26  }
27
28  int main() {
29      BigInt data = createBigInt(); // 又一次拷贝?
30      return 0;
31  }

```

在 C++11 之前，这段代码可能涉及多次昂贵的深拷贝。createBigInt 返回时，temp 的内容被复制到返回值；然后返回值又被复制到 data。对于包含百万个整数的对象，这是巨大的浪费。

问题的关键在于：temp 是一个即将销毁的临时对象，我们完全可以“偷走”它的数据，而不是复制。这就是移动语义的核心思想——对于即将消亡的对象，转移其资源而非复制。

1.5.21.2. 左值与右值

要理解移动语义，首先需要理解左值（lvalue）和右值（rvalue）的概念。

简单来说，左值是有持久身份的表达式——它有名字，可以取地址，在表达式结束后仍然存在。右值是临时的、即将消亡的表达式——通常没有名字，不能取地址，表达式结束后就不存在了。

```

1  int x = 10;           // x 是左值
2  int y = x + 5;        // x 是左值，x + 5 是右值，y 是左值
3  int z = x + y;        // x 和 y 是左值，x + y 是右值
4
5  int* p = &x;          // 正确，x 是左值，可以取地址
6  // int* q = &(x + 5); // 错误！x + 5 是右值，不能取地址
7
8  std::string s1 = "hello"; // s1 是左值
9  std::string s2 = s1 + " world"; // s1 是左值，s1 + " world" 是右值
10 std::string s3 = std::string("hi"); // std::string("hi") 是右值

```

函数返回值（非引用）是右值：

```

1  std::string getName() {
2      return "Robot";
3  }
4
5  std::string name = getName(); // getName() 返回的是右值

```

直观判断方法：能放在赋值运算符左边的通常是左值，只能放在右边的是右值。能取地址的是左值，不能取地址的是右值。

1.5.21.3. 右值引用

C++11 引入了右值引用，用 && 表示。右值引用只能绑定到右值：

```

1  int x = 10;
2
3  int& lref = x;        // 左值引用，绑定到左值
4  // int& lref2 = 10;   // 错误！左值引用不能绑定到右值
5
6  int&& rref = 10;       // 右值引用，绑定到右值
7  int&& rref2 = x + 5;   // 正确，x + 5 是右值
8  // int&& rref3 = x;    // 错误！右值引用不能绑定到左值
9

```

```

10 // 特殊情况: const 左值引用可以绑定到右值
11 const int& cref = 10; // 正确, 这是历史遗留特性

```

右值引用的意义在于:它让我们能够区分“传入的是即将消亡的临时对象”和“传入的是需要保留的持久对象”。基于这个区分,我们可以对临时对象采取不同的处理策略——移动而非复制。

1.5.21.4. 移动构造函数与移动赋值运算符

有了右值引用,我们可以为类定义移动构造函数和移动赋值运算符。它们接受右值引用参数,在实现中“窃取”源对象的资源:

```

1  class BigData {
2  public:
3      // 普通构造函数
4      BigData(size_t size) : size(size), data(new int[size]) {
5          std::cout << "构造: 分配 " << size << " 个整数" << std::endl;
6      }
7
8      // 拷贝构造函数
9      BigData(const BigData& other) : size(other.size), data(new
10 int[other.size]) {
11      std::copy(other.data, other.data + size, data);
12      std::cout << "拷贝构造: 复制 " << size << " 个整数" << std::endl;
13  }
14
15      // 移动构造函数
16      BigData(BigData&& other) noexcept
17      : size(other.size), data(other.data) // 直接接管资源
18      {
19          other.size = 0;
20          other.data = nullptr; // 将源对象置于有效但空的状态
21          std::cout << "移动构造: 转移资源" << std::endl;
22      }
23
24      // 拷贝赋值运算符
25      BigData& operator=(const BigData& other) {
26          if (this != &other) {
27              delete[] data;
28              size = other.size;
29              data = new int[size];
30              std::copy(other.data, other.data + size, data);
31              std::cout << "拷贝赋值: 复制 " << size << " 个整数" << std::endl;
32          }
33          return *this;
34      }
35
36      // 移动赋值运算符
37      BigData& operator=(BigData&& other) noexcept {
38          if (this != &other) {
39              delete[] data; // 释放自己的资源
40              size = other.size; // 接管源对象的资源
41              data = other.data;
42              other.size = 0; // 将源对象置于有效但空的状态
43              other.data = nullptr;
44              std::cout << "移动赋值: 转移资源" << std::endl;
45          }
46          return *this;
47      }
48
49      ~BigData() {
50          delete[] data;
51      }
52  };

```



```

50         if (size > 0) {
51             std::cout << "析构: 释放 " << size << " 个整数" << std::endl;
52         } else {
53             std::cout << "析构: 空对象" << std::endl;
54         }
55     }
56
57 private:
58     size_t size;
59     int* data;
60 };

```

移动操作的关键点:

1. 参数是右值引用 T&&
2. 直接接管源对象的资源（指针、句柄等），而不是复制
3. 将源对象置于“有效但未指定”的状态（通常是空状态），确保其析构函数能安全执行
4. 标记为 `noexcept`，这对于标准库容器的优化很重要

现在让我们看看移动语义的效果:

```

1  BigInt createBigInt() {
2      BigInt temp(1000000);
3      return temp;
4  }
5
6  int main() {
7      std::cout << "=== 创建临时对象 ===" << std::endl;
8      BigInt data = createBigInt();
9
10     std::cout << "\n=== 从右值移动 ===" << std::endl;
11     BigInt data2 = BigInt(500000); // 临时对象，触发移动构造
12
13     std::cout << "\n=== 移动赋值 ===" << std::endl;
14     data2 = BigInt(200000); // 临时对象，触发移动赋值
15
16     std::cout << "\n=== 程序结束 ===" << std::endl;
17     return 0;
18 }

```

移动操作只是转移指针，时间复杂度 $O(1)$ ，而拷贝操作需要分配内存和复制数据，时间复杂度 $O(n)$ 。对于大型对象，性能差异是巨大的。

1.5.21.5. std::move

前面的例子中，移动语义自动应用于临时对象（右值）。但有时我们想对左值也使用移动语义——比如我们知道某个对象之后不再使用，想把它资源转移出去。

`std::move` 的作用是将左值转换为右值引用，从而允许移动操作:

```

1  #include <utility> // std::move
2
3  int main() {
4      BigInt data1(1000000);
5
6      // BigInt data2 = data1; // 拷贝构造
7
8      BigInt data2 = std::move(data1); // 移动构造
9      // data1 现在处于有效但未指定的状态（通常是空）
10     // 不应该再使用 data1 的值

```

```

11
12     return 0;
13 }

```

理解 `std::move` 的关键：它本身不移动任何东西！它只是一个类型转换，将表达式的类型从左值转换为右值引用。真正的移动发生在移动构造函数或移动赋值运算符中。

```

1 // std::move 的简化实现
2 template <typename T>
3 typename std::remove_reference<T>::type&& move(T&& arg) noexcept {
4     return static_cast<typename std::remove_reference<T>::type&&>(arg);
5 }

```

使用 `std::move` 后的对象处于“有效但未指定”的状态。这意味着：

- 对象仍然有效，可以被销毁、被赋新值
- 但不应该依赖它的值，因为值已经被转移走了

```

1 std::string s1 = "Hello, World!";
2 std::string s2 = std::move(s1);
3
4 std::cout << s2 << std::endl; // "Hello, World!"
5 // std::cout << s1 << std::endl; // 可能输出空字符串，但不要依赖这个行为
6
7 s1 = "New Value"; // 可以重新赋值
8 std::cout << s1 << std::endl; // "New Value"

```

1.5.21.6. 何时使用 `std::move`

`std::move` 应该谨慎使用。以下是一些适当使用的场景：

将对象传递给会“消费”它的函数：

```

1 void takeOwnership(std::unique_ptr<Motor> motor) {
2     // 使用 motor...
3 }
4
5 auto motor = std::make_unique<Motor>(1);
6 takeOwnership(std::move(motor)); // 转移所有权
7 // motor 现在为空

```

从函数返回局部对象时通常不需要 `std::move`，编译器会自动应用返回值优化（RVO）或移动：

```

1 BigData createData() {
2     BigData data(1000);
3     return data; // 不要写 return std::move(data)，这可能阻止 RVO
4 }

```

将容器中的元素移出：

```

1 std::vector<std::string> names = {"Alice", "Bob", "Charlie"};
2 std::string first = std::move(names[0]);
3 // names[0] 现在是空字符串（或未指定状态）

```

在类的成员函数中转移成员：

```

1 class Container {

```

```

2 public:
3     std::vector<int> extractData() {
4         return std::move(data); // 转移而非复制
5         // 之后 data 为空, 对象状态改变
6     }
7
8 private:
9     std::vector<int> data;
10 };

```

不应该对 `const` 对象使用 `std::move` —— 结果仍然是 `const`，会匹配拷贝构造函数而非移动构造函数：

```

1 const std::string s1 = "Hello";
2 std::string s2 = std::move(s1); // 实际上是拷贝！

```

1.5.21.7. 完美转发与万能引用

在模板编程中，`T&&` 不一定是右值引用。当 `T` 是模板参数时，`T&&` 是“万能引用”（也称“转发引用”），它可以绑定到左值或右值：

```

1 template <typename T>
2 void wrapper(T&& arg) { // T&& 是万能引用
3     // arg 是左值还是右值取决于调用时传入的参数
4 }
5
6 int x = 10;
7 wrapper(x); // T 推导为 int&, arg 类型是 int& (左值引用)
8 wrapper(10); // T 推导为 int, arg 类型是 int&& (右值引用)
9 wrapper(x + 5); // T 推导为 int, arg 类型是 int&&

```

在模板中，为了保持参数的左值/右值属性进行转发，使用 `std::forward`：

```

1 template <typename T>
2 void relay(T&& arg) {
3     process(std::forward<T>(arg)); // 完美转发
4 }
5
6 void process(int& x) { std::cout << "左值版本" << std::endl; }
7 void process(int&& x) { std::cout << "右值版本" << std::endl; }
8
9 int main() {
10     int x = 10;
11     relay(x); // 调用 process(int&)
12     relay(10); // 调用 process(int&&)
13     relay(x + 5); // 调用 process(int&&)
14     return 0;
15 }

```

`std::forward` 与 `std::move` 的区别：

- `std::move` 无条件地将参数转为右值引用
- `std::forward` 根据模板参数有条件地转换，保持原来的左值/右值属性

这在实现通用工厂函数或包装器时非常有用：

```

1 template <typename T, typename... Args>
2 std::unique_ptr<T> makeUnique(Args&&... args) {
3     return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
4 }

```

```
4 }
```

1.5.21.8. 移动语义与标准库

标准库的容器和类型都支持移动语义，这使得它们在很多场景下效率更高。

`std::vector` 在扩容时，如果元素类型的移动构造函数是 `noexcept` 的，会使用移动而非复制：

```
1 std::vector<BigData> vec;
2 vec.reserve(2);
3
4 vec.push_back(BigData(1000)); // 移动临时对象
5 vec.push_back(BigData(2000)); // 移动临时对象
6
7 // 如果需要扩容，且 BigData 的移动构造是 noexcept
8 // 则现有元素会被移动而非复制
9 vec.push_back(BigData(3000));
```

这就是为什么移动构造函数应该标记为 `noexcept` ——它允许标准库进行优化。

`std::string` 和其他标准库类型也支持移动：

```
1 std::string s1 = "A very long string that exceeds small string optimization";
2 std::string s2 = std::move(s1); // 0(1) 移动，而非 0(n) 复制
3
4 std::vector<int> v1 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
5 std::vector<int> v2 = std::move(v1); // 0(1) 移动
```

1.5.21.9. 五法则

C++11 之后，“三法则”扩展为“五法则”：如果你需要自定义析构函数、拷贝构造函数或拷贝赋值运算符中的任何一个，你可能也需要自定义其他四个（包括移动构造函数和移动赋值运算符）。

完整的五法则示例：

```
1 class Resource {
2 public:
3     // 构造函数
4     Resource(size_t size) : size(size), data(new int[size]) {}
5
6     // 析构函数
7     ~Resource() {
8         delete[] data;
9     }
10
11     // 拷贝构造函数
12     Resource(const Resource& other)
13         : size(other.size), data(new int[other.size])
14     {
15         std::copy(other.data, other.data + size, data);
16     }
17
18     // 拷贝赋值运算符
19     Resource& operator=(const Resource& other) {
20         if (this != &other) {
21             delete[] data;
22             size = other.size;
```

```

23         data = new int[size];
24         std::copy(other.data, other.data + size, data);
25     }
26     return *this;
27 }
28
29 // 移动构造函数
30 Resource(Resource&& other) noexcept
31     : size(other.size), data(other.data)
32 {
33     other.size = 0;
34     other.data = nullptr;
35 }
36
37 // 移动赋值运算符
38 Resource& operator=(Resource&& other) noexcept {
39     if (this != &other) {
40         delete[] data;
41         size = other.size;
42         data = other.data;
43         other.size = 0;
44         other.data = nullptr;
45     }
46     return *this;
47 }
48
49 private:
50     size_t size;
51     int* data;
52 };

```

或者使用 copy-and-swap 惯用法统一处理拷贝和移动赋值：

```

1  class Resource {
2  public:
3      Resource(size_t size) : size(size), data(new int[size]) {}
4      ~Resource() { delete[] data; }
5
6      Resource(const Resource& other)
7          : size(other.size), data(new int[other.size])
8      {
9          std::copy(other.data, other.data + size, data);
10     }
11
12     Resource(Resource&& other) noexcept
13         : size(other.size), data(other.data)
14     {
15         other.size = 0;
16         other.data = nullptr;
17     }
18
19     // 统一的赋值运算符（拷贝和移动）
20     Resource& operator=(Resource other) noexcept { // 按值传递
21         swap(*this, other);
22         return *this;
23     }
24
25     friend void swap(Resource& a, Resource& b) noexcept {
26         using std::swap;
27         swap(a.size, b.size);
28         swap(a.data, b.data);
29     }

```

```

30
31 private:
32     size_t size;
33     int* data;
34 };

```

1.5.21.10. 实际应用示例

让我们看看移动语义在 RoboMaster 开发中的应用。

高效的图像处理管道：

```

1  class Frame {
2  public:
3      Frame(int width, int height)
4          : width(width), height(height),
5            data(new uint8_t[width * height * 3])
6      {
7          std::cout << "分配帧: " << width << "x" << height << std::endl;
8      }
9
10     Frame(Frame&& other) noexcept
11         : width(other.width), height(other.height), data(other.data)
12     {
13         other.width = 0;
14         other.height = 0;
15         other.data = nullptr;
16         std::cout << "移动帧" << std::endl;
17     }
18
19     Frame& operator=(Frame&& other) noexcept {
20         if (this != &other) {
21             delete[] data;
22             width = other.width;
23             height = other.height;
24             data = other.data;
25             other.width = 0;
26             other.height = 0;
27             other.data = nullptr;
28         }
29         return *this;
30     }
31
32     // 禁止拷贝（图像数据太大）
33     Frame(const Frame&) = delete;
34     Frame& operator=(const Frame&) = delete;
35
36     ~Frame() { delete[] data; }
37
38 private:
39     int width, height;
40     uint8_t* data;
41 };
42
43 class FrameProcessor {
44 public:
45     Frame process(Frame frame) { // 按值传递，自动移动
46         // 处理帧...
47         return frame; // 返回处理后的帧，自动移动
48     }
49 };
50

```

```

51 // 使用
52 Frame captureFrame() {
53     return Frame(1920, 1080);
54 }
55
56 int main() {
57     FrameProcessor processor;
58
59     Frame frame = captureFrame();           // 移动 (或 RVO)
60     frame = processor.process(std::move(frame)); // 移动进, 移动出
61
62     return 0;
63 }

```

消息队列的高效传递:

```

1  class Message {
2  public:
3      Message(std::string topic, std::vector<uint8_t> payload)
4          : topic(std::move(topic)), payload(std::move(payload)),
5            timestamp(std::chrono::steady_clock::now())
6      {
7      }
8
9      // 默认移动操作足够
10     Message(Message&&) = default;
11     Message& operator=(Message&&) = default;
12
13     // 禁止拷贝
14     Message(const Message&) = delete;
15     Message& operator=(const Message&) = delete;
16
17     const std::string& getTopic() const { return topic; }
18     const std::vector<uint8_t>& getPayload() const { return payload; }
19
20 private:
21     std::string topic;
22     std::vector<uint8_t> payload;
23     std::chrono::steady_clock::time_point timestamp;
24 };
25
26 class MessageQueue {
27 public:
28     void push(Message msg) { // 按值传递, 调用者可以移动或拷贝
29         std::lock_guard<std::mutex> lock(mutex);
30         queue.push(std::move(msg)); // 移动到队列
31     }
32
33     std::optional<Message> pop() {
34         std::lock_guard<std::mutex> lock(mutex);
35         if (queue.empty()) return std::nullopt;
36
37         Message msg = std::move(queue.front()); // 移动出队列
38         queue.pop();
39         return msg; // 移动返回
40     }
41
42 private:
43     std::queue<Message> queue;
44     std::mutex mutex;
45 };
46

```

```

47 // 使用
48 void producer(MessageQueue& mq) {
49     std::vector<uint8_t> data(1024);
50     mq.push(Message("sensor/imu", std::move(data))); // 高效传递
51 }
52
53 void consumer(MessageQueue& mq) {
54     if (auto msg = mq.pop()) {
55         // 处理消息...
56     }
57 }

```

高效的返回复杂对象:

```

1 struct DetectionResult {
2     std::vector<Target> targets;
3     Frame processedFrame;
4     double processingTime;
5
6     // 默认移动操作
7     DetectionResult(DetectionResult&&) = default;
8     DetectionResult& operator=(DetectionResult&&) = default;
9 };
10
11 DetectionResult detectTargets(Frame frame) {
12     DetectionResult result;
13
14     // 处理...
15     result.processedFrame = std::move(frame);
16     result.targets.emplace_back(/* ... */);
17     result.processingTime = 0.016;
18
19     return result; // 移动返回 (或 NRVO)
20 }
21
22 int main() {
23     Frame frame = captureFrame();
24     DetectionResult result = detectTargets(std::move(frame));
25     // frame 已被移动, result 包含处理结果
26     return 0;
27 }

```

1.5.21.11. 常见错误与最佳实践

不要过度使用 `std::move`。以下情况不需要 `std::move`：

```

1 // 错误: 返回局部变量不需要 move, 可能阻止 RV0
2 std::string getName() {
3     std::string name = "Robot";
4     return std::move(name); // 不要这样做!
5 }
6
7 // 正确
8 std::string getName() {
9     std::string name = "Robot";
10    return name; // 编译器会自动优化
11 }

```

不要移动后继续使用:


```

1 std::vector<int> v1 = {1, 2, 3};
2 std::vector<int> v2 = std::move(v1);
3
4 // 危险! v1 的状态未指定
5 // for (int x : v1) { ... } // 不要这样做
6
7 // 可以重新赋值
8 v1 = {4, 5, 6}; // 现在 v1 又有效了

```

记得标记 `noexcept` :

```

1 // 好: 标记 noexcept, 允许标准库优化
2 Resource(Resource&& other) noexcept;
3
4 // 不好: 没有 noexcept, vector 扩容时会复制而非移动
5 Resource(Resource&& other);

```

使用 `= default` 生成移动操作 (如果合适):

```

1 class Simple {
2 public:
3     Simple() = default;
4     Simple(Simple&&) = default; // 编译器生成移动构造
5     Simple& operator=(Simple&&) = default; // 编译器生成移动赋值
6
7 private:
8     std::string name;
9     std::vector<int> data;
10    // 所有成员都支持移动, 默认实现就够了
11 };

```

移动语义是现代 C++ 的重要特性, 它使得资源密集型对象的传递变得高效。理解左值、右值、右值引用和 `std::move` 的含义, 可以帮助你写出更高效的代码, 并更好地理解标准库的行为。在 RoboMaster 开发中, 图像处理、消息传递、大型数据结构等场景都能从移动语义中受益。

1.5.22. Lambda 表达式

在前面的章节中, 我们已经多次使用 lambda 表达式: 作为 STL 算法的谓词、作为线程的执行函数、作为条件变量的等待条件。Lambda 表达式是 C++11 引入的重要特性, 它允许在代码中就地定义匿名函数, 使代码更加简洁和直观。在 RoboMaster 开发中, lambda 表达式尤其重要——ROS 的回调机制大量使用 lambda 来处理消息、定时器和服务响应。掌握 lambda 表达式, 是编写现代 C++ 代码和 ROS 程序的必备技能。

1.5.22.1. 为什么需要 Lambda

假设我们要对一组目标按距离排序。使用传统方法, 需要单独定义比较函数:

```

1 bool compareByDistance(const Target& a, const Target& b) {
2     return a.distance < b.distance;
3 }
4
5 void processTargets(std::vector<Target>& targets) {
6     std::sort(targets.begin(), targets.end(), compareByDistance);
7 }

```

这种方式有几个问题：比较函数定义在使用位置之外，阅读代码时需要跳转查看；如果比较逻辑只用一次，单独定义函数显得冗余；如果需要访问局部变量（如阈值），函数就无法直接使用。

Lambda 表达式解决了这些问题：

```
1 void processTargets(std::vector<Target>& targets) {
2     std::sort(targets.begin(), targets.end(),
3         [](const Target& a, const Target& b) {
4             return a.distance < b.distance;
5         });
6 }
```

比较逻辑就在使用的地方定义，代码意图一目了然。

1.5.22.2. Lambda 的基本语法

Lambda 表达式的完整语法如下：

```
1 [捕获列表](参数列表) mutable 异常说明 -> 返回类型 { 函数体 }
```

其中很多部分是可选的，最简形式只需要捕获列表和函数体：

```
1 []{ std::cout << "Hello, Lambda!" << std::endl; }
```

让我们逐一了解各个部分：

```
1 // 完整示例
2 auto lambda = [x, &y](int a, int b) mutable noexcept -> int {
3     x++; // mutable 允许修改按值捕获的变量
4     return a + b + x + y;
5 };
6
7 // 各部分说明：
8 // [x, &y]      - 捕获列表：x 按值捕获，y 按引用捕获
9 // (int a, int b) - 参数列表：接受两个 int 参数
10 // mutable     - 允许修改按值捕获的变量（可选）
11 // noexcept    - 异常说明（可选）
12 // -> int      - 返回类型（通常可省略，编译器自动推导）
13 // { ... }     - 函数体
```

常见的 lambda 形式：

```
1 // 无参数、无捕获
2 auto greet = []() { std::cout << "Hello!" << std::endl; };
3 greet();
4
5 // 参数列表，无捕获
6 auto add = [](int a, int b) { return a + b; };
7 int sum = add(3, 4); // 7
8
9 // 捕获局部变量
10 int factor = 10;
11 auto multiply = [factor](int x) { return x * factor; };
12 int result = multiply(5); // 50
13
14 // 省略空参数列表
15 auto sayHi = [] { std::cout << "Hi!" << std::endl; };
16 sayHi();
```

返回类型通常可以省略，编译器会自动推导。但在某些复杂情况下需要显式指定：

```

1 // 编译器自动推导返回类型为 int
2 auto simple = [](int x) { return x * 2; };
3
4 // 多个 return 语句且类型不一致时, 需要显式指定
5 auto conditional = [](bool flag) -> double {
6     if (flag) return 1;           // int
7     else return 3.14;           // double
8 };

```

1.5.22.3. 捕获列表

捕获列表是 lambda 最重要的特性之一, 它决定了 lambda 如何访问外部作用域的变量。按值捕获创建变量的副本, lambda 内部使用的是副本, 原变量不受影响:

```

1 int x = 10;
2
3 auto byValue = [x]() {
4     // x = 20; // 错误! 默认情况下不能修改按值捕获的变量
5     std::cout << x << std::endl; // 10
6 };
7
8 x = 100;
9 byValue(); // 仍然输出 10, 因为捕获的是副本

```

按引用捕获使用变量的引用, 修改会影响原变量:

```

1 int x = 10;
2
3 auto byRef = [&x]() {
4     x = 20; // 修改原变量
5 };
6
7 byRef();
8 std::cout << x << std::endl; // 20

```

可以混合使用多种捕获方式:

```

1 int a = 1, b = 2, c = 3;
2
3 // 分别指定每个变量的捕获方式
4 auto mixed = [a, &b, c]() {
5     // a 是值捕获
6     // b 是引用捕获
7     // c 是值捕获
8 };
9
10 // 默认按值捕获所有变量, b 按引用
11 auto defaultValue = [=, &b]() {
12     // 所有变量按值捕获, 除了 b 按引用
13 };
14
15 // 默认按引用捕获所有变量, a 按值
16 auto defaultRef = [&, a]() {
17     // 所有变量按引用捕获, 除了 a 按值
18 };

```

捕获列表的各种形式:

```

1 [] // 不捕获任何变量

```

```

2 [x]          // 按值捕获 x
3 [&x]         // 按引用捕获 x
4 [=]          // 按值捕获所有使用的局部变量
5 [&]          // 按引用捕获所有使用的局部变量
6 [=, &x]     // 默认按值, x 按引用
7 [&, x]      // 默认按引用, x 按值
8 [this]       // 捕获当前对象的 this 指针
9 [*this]      // C++17: 按值捕获当前对象 (复制)

```

使用 `mutable` 关键字可以修改按值捕获的变量（修改的是副本）：

```

1 int x = 10;
2
3 auto mutableLambda = [x]() mutable {
4     x++; // 允许修改, 但修改的是副本
5     return x;
6 };
7
8 std::cout << mutableLambda() << std::endl; // 11
9 std::cout << mutableLambda() << std::endl; // 12 (lambda 内部的 x 保持状态)
10 std::cout << x << std::endl; // 10 (原变量不变)

```

1.5.22.4. 捕获的注意事项

捕获引用时要特别注意变量的生命周期。如果 lambda 存活时间超过被捕获变量，会产生悬空引用：

```

1 std::function<int()> createCounter() {
2     int count = 0;
3     // 危险! count 在函数返回后被销毁
4     return [&count]() { return ++count; };
5 }
6
7 auto counter = createCounter();
8 // counter(); // 未定义行为! count 已经不存在

```

解决方案是按值捕获，或者使用 `mutable`：

```

1 std::function<int()> createCounter() {
2     int count = 0;
3     // 安全: 按值捕获, 使用 mutable 允许修改
4     return [count]() mutable { return ++count; };
5 }
6
7 auto counter = createCounter();
8 std::cout << counter() << std::endl; // 1
9 std::cout << counter() << std::endl; // 2
10 std::cout << counter() << std::endl; // 3

```

在类的成员函数中，捕获 `this` 可以访问成员变量：

```

1 class Robot {
2 public:
3     void startMonitoring() {
4         // 捕获 this, 可以访问成员变量
5         auto monitor = [this]() {
6             while (running) {
7                 std::cout << "Position: " << position << std::endl;
8                 std::this_thread::sleep_for(std::chrono::seconds(1));

```

```

9         }
10    };
11
12    monitorThread = std::thread(monitor);
13 }
14
15 void stop() {
16     running = false;
17     if (monitorThread.joinable()) {
18         monitorThread.join();
19     }
20 }
21
22 private:
23     std::atomic<bool> running{true};
24     double position = 0.0;
25     std::thread monitorThread;
26 };

```

C++17 的 `[*this]` 按值捕获整个对象，这在对象可能在 lambda 执行前被销毁的情况下很有用：

```

1 class Processor {
2 public:
3     auto createTask() {
4         // C++17: 复制整个对象
5         return [*this]() {
6             // 使用对象的副本，即使原对象已销毁也安全
7             return this->process();
8         };
9     }
10
11 private:
12     int process() const { return data * 2; }
13     int data = 42;
14 };

```

1.5.22.5. Lambda 的类型与存储

每个 lambda 表达式都有唯一的、无名的类型（闭包类型）。即使两个 lambda 看起来完全相同，它们的类型也不同：

```

1 auto lambda1 = [](int x) { return x * 2; };
2 auto lambda2 = [](int x) { return x * 2; };
3
4 // lambda1 和 lambda2 的类型不同!
5 // decltype(lambda1) != decltype(lambda2)

```

因此，通常使用 `auto` 来声明 lambda 变量。如果需要存储 lambda 或将其作为参数传递，可以使用 `std::function`：

```

1 #include <functional>
2
3 // std::function 可以存储任何可调用对象
4 std::function<int(int)> func;
5
6 func = [](int x) { return x * 2; };
7 std::cout << func(5) << std::endl; // 10
8

```

```

9 func = [](int x) { return x * x; };
10 std::cout << func(5) << std::endl; // 25
11
12 // 作为函数参数
13 void applyToAll(std::vector<int>& vec, std::function<int(int)> transform) {
14     for (int& x : vec) {
15         x = transform(x);
16     }
17 }

```

不捕获任何变量的 lambda 可以转换为函数指针：

```

1 // 无捕获 lambda 可以转换为函数指针
2 int (*funcPtr)(int, int) = [](int a, int b) { return a + b; };
3
4 std::cout << funcPtr(3, 4) << std::endl; // 7
5
6 // 有捕获的 lambda 不能转换为函数指针
7 int factor = 2;
8 // int (*ptr)(int) = [factor](int x) { return x * factor; }; // 错误!

```

1.5.22.6. 泛型 Lambda (C++14)

C++14 允许 lambda 的参数使用 `auto`，创建泛型 lambda：

```

1 // 泛型 lambda: 参数类型自动推导
2 auto print = [](const auto& x) {
3     std::cout << x << std::endl;
4 };
5
6 print(42);           // int
7 print(3.14);         // double
8 print("hello");      // const char*
9 print(std::string("world")); // std::string
10
11 // 多个 auto 参数
12 auto add = [](auto a, auto b) {
13     return a + b;
14 };
15
16 std::cout << add(1, 2) << std::endl; // 3 (int)
17 std::cout << add(1.5, 2.5) << std::endl; // 4.0 (double)
18 std::cout << add(std::string("Hello, "), "World!") << std::endl; // "Hello, World!"

```

泛型 lambda 本质上是带有模板化 `operator()` 的函数对象。

1.5.22.7. ROS 回调中的 Lambda

在 ROS (Robot Operating System) 开发中，lambda 表达式是处理回调的首选方式。相比传统的成员函数回调，lambda 更加灵活和直观。

ROS 2 订阅器回调：

```

1 #include <rclcpp/rclcpp.hpp>
2 #include <sensor_msgs/msg/imu.hpp>
3 #include <geometry_msgs/msg/twist.hpp>
4
5 class RobotNode : public rclcpp::Node {
6 public:

```

```

7   RobotNode() : Node("robot_node") {
8       // 使用 lambda 作为订阅回调
9       imu_sub_ = this->create_subscription<sensor_msgs::msg::Imu>(
10          "imu/data", 10,
11          [this](const sensor_msgs::msg::Imu::SharedPtr msg) {
12              // 直接访问成员变量
13              latest_imu_ = *msg;
14              processIMU(msg);
15          }
16      );
17
18      // 另一个订阅器
19      cmd_sub_ = this->create_subscription<geometry_msgs::msg::Twist>(
20          "cmd_vel", 10,
21          [this](const geometry_msgs::msg::Twist::SharedPtr msg) {
22              target_velocity_ = *msg;
23          }
24      );
25
26      // 定时器回调
27      timer_ = this->create_wall_timer(
28          std::chrono::milliseconds(10),
29          [this]() {
30              controlLoop();
31          }
32      );
33  }
34
35 private:
36     void processIMU(const sensor_msgs::msg::Imu::SharedPtr& msg) {
37         // 处理 IMU 数据
38     }
39
40     void controlLoop() {
41         // 控制循环
42     }
43
44     sensor_msgs::msg::Imu latest_imu_;
45     geometry_msgs::msg::Twist target_velocity_;
46
47     rclcpp::Subscription<sensor_msgs::msg::Imu>::SharedPtr imu_sub_;
48     rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr cmd_sub_;
49     rclcpp::TimerBase::SharedPtr timer_;
50 };

```

服务回调:

```

1  #include <example_interfaces/srv/add_two_ints.hpp>
2
3  class CalculatorNode : public rclcpp::Node {
4  public:
5      CalculatorNode() : Node("calculator") {
6          service_ = this-
7          >create_service<example_interfaces::srv::AddTwoInts>(
8              "add_two_ints",
9              [this](
10                 const
11                 std::shared_ptr<example_interfaces::srv::AddTwoInts::Request> request,
12                 std::shared_ptr<example_interfaces::srv::AddTwoInts::Response> response
13             ) {
14                 response->sum = request->a + request->b;
15                 RCLCPP_INFO(this->get_logger(), "Request: %ld + %ld = %ld",

```

```

14         request->a, request->b, response->sum);
15     }
16     );
17 }
18
19 private:
20     rclcpp::Service<example_interfaces::srv::AddTwoInts>::SharedPtr
    service_;
21 };

```

ROS 1 风格 (使用 `roscpp`):

```

1  #include <ros/ros.h>
2  #include <sensor_msgs/Image.h>
3
4  class VisionNode {
5  public:
6      VisionNode() {
7          ros::NodeHandle nh;
8
9          // boost::bind 方式 (传统)
10         // image_sub_ = nh.subscribe("camera/image", 1,
    &VisionNode::imageCallback, this);
11
12         // Lambda 方式 (更现代)
13         image_sub_ = nh.subscribe<sensor_msgs::Image>(
14             "camera/image", 1,
15             [this](const sensor_msgs::ImageConstPtr& msg) {
16                 processImage(msg);
17             }
18         );
19     }
20
21 private:
22     void processImage(const sensor_msgs::ImageConstPtr& msg) {
23         // 图像处理
24     }
25
26     ros::Subscriber image_sub_;
27 };

```

1.5.22.8. STL 算法中的 Lambda

Lambda 在 STL 算法中广泛使用。这里是一些常见模式:

```

1  std::vector<Target> targets = getTargets();
2
3  // 排序
4  std::sort(targets.begin(), targets.end(),
5      [](const Target& a, const Target& b) {
6          return a.priority > b.priority; // 按优先级降序
7      });
8
9  // 查找
10 auto it = std::find_if(targets.begin(), targets.end(),
11     [](const Target& t) {
12         return t.confidence > 0.9 && t.isEnemy;
13     });
14
15 // 计数
16 int highConfCount = std::count_if(targets.begin(), targets.end(),

```



```

17     [](const Target& t) {
18         return t.confidence > 0.8;
19     });
20
21 // 过滤（移除不满足条件的）
22 targets.erase(
23     std::remove_if(targets.begin(), targets.end(),
24         [](const Target& t) {
25             return t.confidence < 0.5;
26         }),
27     targets.end()
28 );
29
30 // 变换
31 std::vector<double> distances;
32 std::transform(targets.begin(),
33     targets.end(), std::back_inserter(distances),
34     [](const Target& t) {
35         return t.distance;
36     });
37
38 // 累加
39 double totalPriority = std::accumulate(targets.begin(), targets.end(), 0.0,
40     [](double sum, const Target& t) {
41         return sum + t.priority;
42     });
43
44 // 全部满足条件?
45 bool allValid = std::all_of(targets.begin(), targets.end(),
46     [](const Target& t) {
47         return t.isValid();
48     });
49
50 // 任一满足条件?
51 bool anyEnemy = std::any_of(targets.begin(), targets.end(),
52     [](const Target& t) {
53         return t.isEnemy;
54     });

```

使用捕获访问外部变量：

```

1 double maxDistance = 100.0;
2 double minConfidence = 0.7;
3
4 auto validTargets = targets;
5 validTargets.erase(
6     std::remove_if(validTargets.begin(), validTargets.end(),
7         [maxDistance, minConfidence](const Target& t) {
8             return t.distance > maxDistance || t.confidence < minConfidence;
9         }),
10    validTargets.end()
11 );

```

1.5.22.9. 异步操作与 Lambda

Lambda 与异步操作配合使用非常方便：

```

1 #include <future>
2
3 // std::async 与 lambda
4 auto future = std::async(std::launch::async, []() {

```

```

5     // 耗时操作
6     std::this_thread::sleep_for(std::chrono::seconds(2));
7     return 42;
8 });
9
10 // 做其他事情...
11
12 int result = future.get(); // 等待并获取结果
13
14 // 并行处理多个任务
15 std::vector<std::future<DetectionResult>> futures;
16
17 for (const auto& frame : frames) {
18     futures.push_back(std::async(std::launch::async,
19                                [&detector, frame]() {
20                                    return detector.process(frame);
21                                }
22                                ));
23 }
24
25 // 收集结果
26 std::vector<DetectionResult> results;
27 for (auto& f : futures) {
28     results.push_back(f.get());
29 }

```

线程创建:

```

1 std::atomic<bool> running{true};
2 std::queue<Task> taskQueue;
3 std::mutex queueMutex;
4 std::condition_variable cv;
5
6 // 工作线程
7 std::thread worker([&]() {
8     while (running) {
9         Task task;
10        {
11            std::unique_lock<std::mutex> lock(queueMutex);
12            cv.wait(lock, [&]() {
13                return !taskQueue.empty() || !running;
14            });
15
16            if (!running && taskQueue.empty()) break;
17
18            task = std::move(taskQueue.front());
19            taskQueue.pop();
20        }
21        task.execute();
22    }
23 });

```

1.5.22.10. 高级用法

立即调用的 Lambda (IIFE) 用于复杂的初始化:

```

1 // 复杂的 const 变量初始化
2 const std::vector<int> primes = []() {
3     std::vector<int> result;
4     // 计算质数...
5     for (int n = 2; n < 100; n++) {

```

```

6         bool isPrime = true;
7         for (int i = 2; i * i <= n; i++) {
8             if (n % i == 0) {
9                 isPrime = false;
10                break;
11            }
12        }
13        if (isPrime) result.push_back(n);
14    }
15    return result;
16 }(); // 注意末尾的 () —— 立即调用
17
18 // 条件初始化
19 const auto config = [&]() {
20     if (useDefault) {
21         return Config::getDefault();
22     } else {
23         return Config::loadFromFile(filename);
24     }
25 }();

```

递归 Lambda 需要显式类型声明:

```

1 // 使用 std::function 实现递归
2 std::function<int(int)> factorial = [&factorial](int n) -> int {
3     return n <= 1 ? 1 : n * factorial(n - 1);
4 };
5
6 std::cout << factorial(5) << std::endl; // 120
7
8 // C++14 的另一种方式: 泛型 lambda 自引用
9 auto factorial2 = [](auto self, int n) -> int {
10     return n <= 1 ? 1 : n * self(self, n - 1);
11 };
12 std::cout << factorial2(factorial2, 5) << std::endl; // 120

```

Lambda 作为比较器用于容器:

```

1 // 自定义 set 排序
2 auto cmp = [](const Target& a, const Target& b) {
3     return a.priority > b.priority;
4 };
5
6 std::set<Target, decltype(cmp)> prioritySet(cmp);
7
8 // 自定义 priority_queue
9 auto pqCmp = [](const Task& a, const Task& b) {
10     return a.deadline > b.deadline; // 最早截止时间优先
11 };
12
13 std::priority_queue<Task, std::vector<Task>, decltype(pqCmp)>
    taskQueue(pqCmp);

```

1.5.22.11. 最佳实践

保持 Lambda 简短。如果 lambda 体超过几行, 考虑提取为命名函数:

```

1 // 不好: lambda 过长
2 std::sort(targets.begin(), targets.end(),
3     [](const Target& a, const Target& b) {
4         // 20 行复杂的比较逻辑...

```

```

5     });
6
7 // 好：提取为命名函数
8 bool compareTargets(const Target& a, const Target& b) {
9     // 20 行复杂的比较逻辑...
10 }
11
12 std::sort(targets.begin(), targets.end(), compareTargets);

```

注意捕获的生命周期。避免捕获引用后 lambda 存活时间超过被捕获变量：

```

1 // 危险
2 std::function<void()> createTask() {
3     std::string data = "important";
4     return [&data]() { // data 在函数返回后销毁！
5         std::cout << data << std::endl;
6     };
7 }
8
9 // 安全
10 std::function<void()> createTask() {
11     std::string data = "important";
12     return [data]() { // 按值捕获，复制数据
13         std::cout << data << std::endl;
14     };
15 }

```

明确捕获意图。避免使用 [=] 或 [&] 捕获所有变量，显式列出需要的变量更清晰：

```

1 // 不推荐：不清楚捕获了什么
2 auto lambda = [=]() { /* ... */ };
3
4 // 推荐：明确捕获的变量
5 auto lambda = [config, &logger]() { /* ... */ };

```

在类成员函数中，优先捕获具体成员而非 this（如果可能）：

```

1 class Processor {
2 public:
3     void process() {
4         // 不太好：捕获整个 this
5         auto task = [this]() {
6             return data_ * factor_;
7         };
8
9         // 更好：只捕获需要的成员
10        auto task2 = [data = data_, factor = factor_]() {
11            return data * factor;
12        };
13    }
14
15 private:
16     int data_;
17     int factor_;
18 };

```

Lambda 表达式是现代 C++ 中不可或缺的工具。它使代码更加简洁、更具表现力，特别适合回调、算法定制和异步编程场景。在 RoboMaster 开发中，无论是 ROS 的消息回调、STL 算法的使用，还是多线程编程，lambda 都会是你最常用的语法特性之一。熟练掌握 lambda 的各种用法，将显著提升你的 C++ 编程效率。

1.5.23. 多线程基础

在 RoboMaster 机器人系统中，同时发生着许多事情：相机以 60 帧每秒的速度捕获图像，IMU 以更高的频率更新姿态数据，控制算法需要实时计算电机输出，通信模块需要收发裁判系统和队友的信息。如果所有这些任务都在单一执行流中顺序进行，系统将无法满足实时性要求——当图像处理占用 CPU 时，控制循环就会停滞。

多线程编程允许程序同时执行多个任务。每个线程是一个独立的执行流，拥有自己的程序计数器和栈，但共享进程的内存空间。C++11 标准库提供了跨平台的多线程支持，包括线程创建、同步原语和原子操作。掌握多线程编程，是开发高性能机器人系统的关键技能。

1.5.23.1. 创建线程

`std::thread` 是 C++ 标准库提供的线程类，定义在 `<thread>` 头文件中。创建线程只需将可调用对象（函数、lambda、函数对象）传递给 `std::thread` 的构造函数：

```
1 #include <thread>
2 #include <iostream>
3
4 void printMessage(const std::string& msg) {
5     std::cout << "线程消息: " << msg << std::endl;
6 }
7
8 int main() {
9     // 创建线程，执行 printMessage 函数
10    std::thread t(printMessage, "Hello from thread!");
11
12    std::cout << "主线程继续执行" << std::endl;
13
14    // 等待线程完成
15    t.join();
16
17    std::cout << "线程已结束" << std::endl;
18    return 0;
19 }
```

`std::thread` 构造函数的第一个参数是要执行的函数，后续参数会被传递给该函数。线程创建后立即开始执行，与主线程并发运行。

每个 `std::thread` 对象在销毁前必须被 `join()` 或 `detach()`：

`join()` 阻塞当前线程，等待目标线程执行完成。这是最常用的方式，确保线程的工作完成后再继续。

`detach()` 将线程分离，让它在后台独立运行。分离后的线程无法再被 `join`，它会在执行完成后自动清理资源。

```
1 std::thread t1(someFunction);
2 t1.join();    // 等待 t1 完成
3
4 std::thread t2(backgroundTask);
5 t2.detach();  // 让 t2 在后台运行，不再管理它
```

如果 `std::thread` 对象在析构时既没有 `join` 也没有 `detach`，程序会调用 `std::terminate()` 终止。这是为了防止“悬空线程”——线程仍在运行但已无法访问。

使用 lambda 表达式创建线程更加灵活：

```
1 int main() {
```

```

2     int result = 0;
3
4     std::thread t([&result]() {
5         // 执行一些计算
6         result = 42;
7         std::cout << "计算完成" << std::endl;
8     });
9
10    t.join();
11    std::cout << "结果: " << result << std::endl; // 42
12
13    return 0;
14 }

```

可以创建多个线程并发执行:

```

1 void worker(int id) {
2     std::cout << "Worker " << id << " 开始工作" << std::endl;
3     std::this_thread::sleep_for(std::chrono::milliseconds(100));
4     std::cout << "Worker " << id << " 完成工作" << std::endl;
5 }
6
7 int main() {
8     std::vector<std::thread> threads;
9
10    // 创建 4 个工作线程
11    for (int i = 0; i < 4; i++) {
12        threads.emplace_back(worker, i);
13    }
14
15    // 等待所有线程完成
16    for (auto& t : threads) {
17        t.join();
18    }
19
20    std::cout << "所有工作完成" << std::endl;
21    return 0;
22 }

```

`std::this_thread` 命名空间提供了一些作用于当前线程的函数:

```

1 // 获取当前线程的 ID
2 std::thread::id myId = std::this_thread::get_id();
3
4 // 让当前线程休眠
5 std::this_thread::sleep_for(std::chrono::milliseconds(100));
6 std::this_thread::sleep_until(someTimePoint);
7
8 // 提示调度器可以切换到其他线程
9 std::this_thread::yield();

```

1.5.23.2. 数据竞争与互斥量

当多个线程同时访问共享数据, 且至少一个线程在修改数据时, 就会发生数据竞争 (data race)。数据竞争导致未定义行为, 程序可能产生错误结果、崩溃或表现出其他不可预测的行为。

```

1 int counter = 0;
2
3 void increment() {
4     for (int i = 0; i < 100000; i++) {

```

```

5         counter++; // 数据竞争!
6     }
7 }
8
9 int main() {
10     std::thread t1(increment);
11     std::thread t2(increment);
12
13     t1.join();
14     t2.join();
15
16     // 预期 200000, 但实际结果不确定
17     std::cout << "Counter: " << counter << std::endl;
18
19     return 0;
20 }

```

`counter++` 看似是原子操作，实际上它包含三步：读取 `counter` 的值，加 1，写回 `counter`。两个线程可能同时读取相同的值，各自加 1 后写回，导致一次递增被“丢失”。

互斥量（mutex）是解决数据竞争的基本工具。它提供了互斥访问——同一时刻只有一个线程可以“持有”互斥量，其他线程必须等待。

```

1 #include <mutex>
2
3 int counter = 0;
4 std::mutex mtx; // 互斥量
5
6 void increment() {
7     for (int i = 0; i < 100000; i++) {
8         mtx.lock(); // 获取锁
9         counter++; // 临界区：受保护的代码
10        mtx.unlock(); // 释放锁
11    }
12 }
13
14 int main() {
15     std::thread t1(increment);
16     std::thread t2(increment);
17
18     t1.join();
19     t2.join();
20
21     std::cout << "Counter: " << counter << std::endl; // 200000
22
23     return 0;
24 }

```

现在结果总是正确的 200000。但手动调用 `lock()` 和 `unlock()` 有风险：如果在临界区内抛出异常或提前返回，`unlock()` 可能不会执行，导致死锁。

1.5.23.3. lock_guard 与 unique_lock

`std::lock_guard` 是一个 RAII 风格的锁管理器。它在构造时自动获取锁，在析构时自动释放锁，确保锁总是被正确释放：

```

1 void increment() {
2     for (int i = 0; i < 100000; i++) {
3         std::lock_guard<std::mutex> lock(mtx); // 构造时加锁
4         counter++;

```

```

5     } // 析构时自动解锁，即使发生异常
6 }

```

C++17 引入了类模板参数推导，可以省略模板参数：

```

1 std::lock_guard lock(mtx); // C++17

```

`std::unique_lock` 提供了更灵活的锁管理。它支持延迟加锁、尝试加锁、定时加锁，以及在生命周期内多次加锁解锁：

```

1 std::mutex mtx;
2
3 void flexibleLocking() {
4     // 延迟加锁
5     std::unique_lock<std::mutex> lock(mtx, std::defer_lock);
6     // 此时还没有加锁
7
8     // 手动加锁
9     lock.lock();
10    // 临界区...
11    lock.unlock();
12
13    // 尝试加锁（非阻塞）
14    if (lock.try_lock()) {
15        // 成功获取锁
16        // ...
17    }
18
19    // 定时尝试加锁
20    if (lock.try_lock_for(std::chrono::milliseconds(100))) {
21        // 在 100ms 内成功获取锁
22        // ...
23    }
24 } // 自动解锁（如果持有锁的话）

```

`unique_lock` 可以被移动（但不能复制），这在需要传递锁的场景下很有用。它也是条件变量所需的锁类型。

当需要同时锁定多个互斥量时，使用 `std::lock` 或 `std::scoped_lock`（C++17）来避免死锁：

```

1 std::mutex mtx1, mtx2;
2
3 void safeMultiLock() {
4     // C++17: scoped_lock 同时锁定多个互斥量，避免死锁
5     std::scoped_lock lock(mtx1, mtx2);
6     // 两个互斥量都被锁定
7     // ...
8 } // 自动解锁
9
10 // C++11/14 方式
11 void safeMultiLockOld() {
12     std::unique_lock<std::mutex> lock1(mtx1, std::defer_lock);
13     std::unique_lock<std::mutex> lock2(mtx2, std::defer_lock);
14     std::lock(lock1, lock2); // 原子地锁定两个互斥量
15     // ...
16 }

```


1.5.23.4. 条件变量

互斥量保护数据免受并发访问，但有时线程需要等待某个条件成立后才能继续。例如，消费者线程需要等待队列中有数据，生产者线程需要等待队列有空间。

条件变量（`std::condition_variable`）允许线程等待某个条件，并在条件可能变化时被其他线程唤醒。它必须与互斥量和 `unique_lock` 配合使用：

```
1  #include <condition_variable>
2  #include <queue>
3
4  std::queue<int> dataQueue;
5  std::mutex mtx;
6  std::condition_variable cv;
7
8  void producer() {
9      for (int i = 0; i < 10; i++) {
10         {
11             std::lock_guard<std::mutex> lock(mtx);
12             dataQueue.push(i);
13             std::cout << "生产: " << i << std::endl;
14         }
15         cv.notify_one(); // 通知一个等待的消费者
16         std::this_thread::sleep_for(std::chrono::milliseconds(100));
17     }
18 }
19
20 void consumer() {
21     while (true) {
22         std::unique_lock<std::mutex> lock(mtx);
23
24         // 等待直到队列非空
25         cv.wait(lock, []() { return !dataQueue.empty(); });
26
27         int data = dataQueue.front();
28         dataQueue.pop();
29         lock.unlock(); // 尽早释放锁
30
31         std::cout << "消费: " << data << std::endl;
32
33         if (data == 9) break; // 结束条件
34     }
35 }
36
37 int main() {
38     std::thread prod(producer);
39     std::thread cons(consumer);
40
41     prod.join();
42     cons.join();
43
44     return 0;
45 }
```

`cv.wait(lock, predicate)` 的工作方式：

1. 检查谓词（`predicate`），如果为真则立即返回
2. 如果谓词为假，释放锁并阻塞当前线程
3. 当被 `notify_one()` 或 `notify_all()` 唤醒后，重新获取锁
4. 再次检查谓词，如果仍为假则继续等待
5. 谓词为真时返回，此时持有锁

使用谓词很重要，因为条件变量可能会“虚假唤醒”（spurious wakeup）——在没有通知的情况下被唤醒。谓词确保只有在条件真正满足时才继续执行。

`notify_one()` 唤醒一个等待的线程，`notify_all()` 唤醒所有等待的线程。选择哪个取决于场景：如果只有一个线程能处理条件变化，用 `notify_one()`；如果多个线程都需要响应，用 `notify_all()`。

条件变量还支持定时等待：

```
1 std::unique_lock<std::mutex> lock(mtx);
2
3 // 等待最多 100ms
4 if (cv.wait_for(lock, std::chrono::milliseconds(100), predicate)) {
5     // 条件在时限内满足
6 } else {
7     // 超时
8 }
9
10 // 等待直到特定时间点
11 if (cv.wait_until(lock, deadline, predicate)) {
12     // 条件在截止时间前满足
13 } else {
14     // 超时
15 }
```

1.5.23.5. 原子操作

对于简单的数值操作，使用互斥量可能过于重量级。`std::atomic` 提供了无锁的原子操作，性能更高：

```
1 #include <atomic>
2
3 std::atomic<int> counter(0);
4
5 void increment() {
6     for (int i = 0; i < 100000; i++) {
7         counter++; // 原子操作，无需互斥量
8     }
9 }
10
11 int main() {
12     std::thread t1(increment);
13     std::thread t2(increment);
14
15     t1.join();
16     t2.join();
17
18     std::cout << "Counter: " << counter << std::endl; // 200000
19
20     return 0;
21 }
```

`std::atomic` 支持多种操作：

```
1 std::atomic<int> value(0);
2
3 value.store(10); // 原子存储
4 int v = value.load(); // 原子加载
5 int old = value.exchange(20); // 原子交换，返回旧值
6
```

```

7 // 原子加减
8 value.fetch_add(5);           // 加 5, 返回旧值
9 value.fetch_sub(3);           // 减 3, 返回旧值
10 value += 10;                  // 等价于 fetch_add(10)
11
12 // 比较并交换 (CAS)
13 int expected = 20;
14 bool success = value.compare_exchange_strong(expected, 30);
15 // 如果 value == expected, 则 value = 30, 返回 true
16 // 否则 expected = value, 返回 false

```

`std::atomic<bool>` 常用作标志:

```

1 std::atomic<bool> running(true);
2
3 void workerThread() {
4     while (running) { // 原子读取
5         // 执行工作...
6     }
7 }
8
9 void stopWorker() {
10     running = false; // 原子写入
11 }

```

1.5.23.6. 线程安全的数据结构

结合互斥量和条件变量，可以构建线程安全的数据结构。以下是一个常用的线程安全队列：

```

1 template <typename T>
2 class ThreadSafeQueue {
3 public:
4     void push(T value) {
5         std::lock_guard<std::mutex> lock(mtx);
6         queue.push(std::move(value));
7         cv.notify_one();
8     }
9
10    // 阻塞式取出
11    T pop() {
12        std::unique_lock<std::mutex> lock(mtx);
13        cv.wait(lock, [this]() { return !queue.empty(); });
14        T value = std::move(queue.front());
15        queue.pop();
16        return value;
17    }
18
19    // 非阻塞式尝试取出
20    bool tryPop(T& value) {
21        std::lock_guard<std::mutex> lock(mtx);
22        if (queue.empty()) return false;
23        value = std::move(queue.front());
24        queue.pop();
25        return true;
26    }
27
28    // 定时等待取出
29    bool waitAndPop(T& value, std::chrono::milliseconds timeout) {
30        std::unique_lock<std::mutex> lock(mtx);
31        if (!cv.wait_for(lock, timeout, [this]() { return !
queue.empty(); }))) {

```

```

32         return false;
33     }
34     value = std::move(queue.front());
35     queue.pop();
36     return true;
37 }
38
39 bool empty() const {
40     std::lock_guard<std::mutex> lock(mtx);
41     return queue.empty();
42 }
43
44 size_t size() const {
45     std::lock_guard<std::mutex> lock(mtx);
46     return queue.size();
47 }
48
49 private:
50     mutable std::mutex mtx;
51     std::queue<T> queue;
52     std::condition_variable cv;
53 };

```

1.5.23.7. RoboMaster 多线程应用

在 RoboMaster 机器人系统中，多线程是必不可少的。让我们看几个典型的应用场景。
图像采集与处理分离：

```

1  class VisionSystem {
2  public:
3      VisionSystem() : running(false) {}
4
5      void start() {
6          running = true;
7          captureThread = std::thread(&VisionSystem::captureLoop, this);
8          processThread = std::thread(&VisionSystem::processLoop, this);
9      }
10
11     void stop() {
12         running = false;
13         frameQueue.push(Frame{}); // 发送空帧唤醒处理线程
14
15         if (captureThread.joinable()) captureThread.join();
16         if (processThread.joinable()) processThread.join();
17     }
18
19     std::vector<Target> getLatestTargets() {
20         std::lock_guard<std::mutex> lock(resultMutex);
21         return latestTargets;
22     }
23
24 private:
25     void captureLoop() {
26         while (running) {
27             Frame frame = camera.capture();
28             frameQueue.push(std::move(frame));
29         }
30     }
31
32     void processLoop() {
33         while (running) {

```

```

34         Frame frame = frameQueue.pop();
35         if (!frame.isValid()) break; // 收到停止信号
36
37         std::vector<Target> targets = detector.detect(frame);
38
39         {
40             std::lock_guard<std::mutex> lock(resultMutex);
41             latestTargets = std::move(targets);
42         }
43     }
44 }
45
46 Camera camera;
47 Detector detector;
48 ThreadSafeQueue<Frame> frameQueue;
49
50 std::vector<Target> latestTargets;
51 std::mutex resultMutex;
52
53 std::atomic<bool> running;
54 std::thread captureThread;
55 std::thread processThread;
56 };

```

控制循环与通信分离:

```

1  class RobotController {
2  public:
3      void start() {
4          running = true;
5          controlThread = std::thread(&RobotController::controlLoop, this);
6          commThread = std::thread(&RobotController::commLoop, this);
7      }
8
9      void stop() {
10         running = false;
11         if (controlThread.joinable()) controlThread.join();
12         if (commThread.joinable()) commThread.join();
13     }
14
15     void setTargetVelocity(double vx, double vy, double omega) {
16         std::lock_guard<std::mutex> lock(cmdMutex);
17         targetVx = vx;
18         targetVy = vy;
19         targetOmega = omega;
20     }
21
22 private:
23     void controlLoop() {
24         auto lastTime = std::chrono::steady_clock::now();
25
26         while (running) {
27             auto now = std::chrono::steady_clock::now();
28             double dt = std::chrono::duration<double>(now -
lastTime).count();
29             lastTime = now;
30
31             // 读取目标速度
32             double vx, vy, omega;
33             {
34                 std::lock_guard<std::mutex> lock(cmdMutex);
35                 vx = targetVx;

```

```

36         vy = targetVy;
37         omega = targetOmega;
38     }
39
40     // 读取传感器
41     IMUData imu = imuSensor.read();
42
43     // 计算控制输出
44     auto motorOutputs = controller.compute(vx, vy, omega, imu, dt);
45
46     // 发送到电机
47     for (size_t i = 0; i < 4; i++) {
48         motors[i].setOutput(motorOutputs[i]);
49     }
50
51     // 控制频率 1000Hz
52     std::this_thread::sleep_until(now +
std::chrono::milliseconds(1));
53     }
54 }
55
56 void commLoop() {
57     while (running) {
58         // 接收裁判系统数据
59         if (referee.hasData()) {
60             RefereeData data = referee.receive();
61             processRefereeData(data);
62         }
63
64         // 发送状态
65         sendStatus();
66
67         std::this_thread::sleep_for(std::chrono::milliseconds(10));
68     }
69 }
70
71 std::atomic<bool> running;
72 std::thread controlThread;
73 std::thread commThread;
74
75 std::mutex cmdMutex;
76 double targetVx = 0, targetVy = 0, targetOmega = 0;
77
78 IMUSensor imuSensor;
79 Motor motors[4];
80 ChassisController controller;
81 RefereeSystem referee;
82 };

```

传感器数据融合:

```

1 class SensorFusion {
2 public:
3     void start() {
4         running = true;
5         imuThread = std::thread(&SensorFusion::imuLoop, this);
6         lidarThread = std::thread(&SensorFusion::lidarLoop, this);
7         fusionThread = std::thread(&SensorFusion::fusionLoop, this);
8     }
9
10    Pose getEstimatedPose() {
11        std::lock_guard<std::mutex> lock(poseMutex);

```

```

12         return estimatedPose;
13     }
14
15 private:
16     void imuLoop() {
17         while (running) {
18             IMUData data = imu.read();
19             {
20                 std::lock_guard<std::mutex> lock(imuMutex);
21                 latestIMU = data;
22                 imuUpdated = true;
23             }
24             imuCv.notify_one();
25             std::this_thread::sleep_for(std::chrono::microseconds(500)); //
2000Hz
26         }
27     }
28
29     void lidarLoop() {
30         while (running) {
31             LidarScan scan = lidar.scan();
32             {
33                 std::lock_guard<std::mutex> lock(lidarMutex);
34                 latestScan = std::move(scan);
35                 lidarUpdated = true;
36             }
37             lidarCv.notify_one();
38             // Lidar 自身频率, 通常 10-40Hz
39         }
40     }
41
42     void fusionLoop() {
43         while (running) {
44             // 等待 IMU 数据 (高频)
45             {
46                 std::unique_lock<std::mutex> lock(imuMutex);
47                 imuCv.wait(lock, [this]() { return imuUpdated || !
running; });
48                 if (!running) break;
49
50                 // 使用 IMU 进行预测
51                 ekf.predict(latestIMU);
52                 imuUpdated = false;
53             }
54
55             // 检查是否有 Lidar 数据 (低频)
56             {
57                 std::lock_guard<std::mutex> lock(lidarMutex);
58                 if (lidarUpdated) {
59                     ekf.updateWithLidar(latestScan);
60                     lidarUpdated = false;
61                 }
62             }
63
64             // 更新估计位姿
65             {
66                 std::lock_guard<std::mutex> lock(poseMutex);
67                 estimatedPose = ekf.getState();
68             }
69         }
70     }
71
72     std::atomic<bool> running;

```

```

73     std::thread imuThread, lidarThread, fusionThread;
74
75     IMUSensor imu;
76     LidarSensor lidar;
77     ExtendedKalmanFilter ekf;
78
79     std::mutex imuMutex, lidarMutex, poseMutex;
80     std::condition_variable imuCv, lidarCv;
81
82     IMUData latestIMU;
83     LidarScan latestScan;
84     Pose estimatedPose;
85
86     bool imuUpdated = false;
87     bool lidarUpdated = false;
88 };

```

1.5.23.8. 常见陷阱与最佳实践

多线程编程容易出错，以下是一些常见陷阱和避免方法。

死锁：两个线程相互等待对方持有的锁。避免方法包括：始终以相同顺序获取多个锁，使用 `std::scoped_lock` 同时获取，避免在持有锁时调用未知代码。

```

1 // 死锁示例
2 void thread1() {
3     std::lock_guard<std::mutex> lock1(mtx1);
4     std::this_thread::sleep_for(std::chrono::milliseconds(1));
5     std::lock_guard<std::mutex> lock2(mtx2); // 等待 mtx2
6 }
7
8 void thread2() {
9     std::lock_guard<std::mutex> lock2(mtx2);
10    std::this_thread::sleep_for(std::chrono::milliseconds(1));
11    std::lock_guard<std::mutex> lock1(mtx1); // 等待 mtx1, 死锁!
12 }
13
14 // 解决方案
15 void thread1Fixed() {
16     std::scoped_lock lock(mtx1, mtx2); // 同时获取，避免死锁
17 }

```

忘记保护共享数据：每次访问共享数据都必须加锁，不仅是写入，读取也需要。

```

1 // 错误：读取时没有加锁
2 void reader() {
3     int value = sharedData; // 数据竞争!
4 }
5
6 // 正确
7 void reader() {
8     std::lock_guard<std::mutex> lock(mtx);
9     int value = sharedData;
10 }

```

锁的粒度：锁的范围应该尽可能小。持有锁的时间越长，其他线程等待的时间就越长。

```

1 // 不好：锁的范围太大
2 void process() {
3     std::lock_guard<std::mutex> lock(mtx);

```



```

4     expensiveOperation(); // 不访问共享数据也持有锁
5     sharedData = result;
6 }
7
8 // 好：只在必要时持有锁
9 void process() {
10     auto result = expensiveOperation(); // 不持有锁
11     std::lock_guard<std::mutex> lock(mtx);
12     sharedData = result; // 只在访问共享数据时持有锁
13 }

```

线程生命周期管理：确保线程在使用的资源之前不会结束，也不会再资源销毁后仍在运行。

```

1 // 危险：局部变量可能在线程结束前被销毁
2 void dangerous() {
3     int localData = 42;
4     std::thread t([&localData]() {
5         std::this_thread::sleep_for(std::chrono::seconds(1));
6         std::cout << localData << std::endl; // localData 可能已销毁！
7     });
8     t.detach(); // 分离后无法控制线程生命周期
9 }
10
11 // 安全：使用值捕获或确保数据生命周期
12 void safe() {
13     int localData = 42;
14     std::thread t([localData]() { // 值捕获
15         std::this_thread::sleep_for(std::chrono::seconds(1));
16         std::cout << localData << std::endl;
17     });
18     t.join(); // 或者等待线程完成
19 }

```

使用 RAII 管理线程：

```

1 class JoinThreads {
2 public:
3     explicit JoinThreads(std::vector<std::thread>& threads)
4         : threads(threads) {}
5
6     ~JoinThreads() {
7         for (auto& t : threads) {
8             if (t.joinable()) t.join();
9         }
10    }
11
12 private:
13     std::vector<std::thread>& threads;
14 };
15
16 void process() {
17     std::vector<std::thread> threads;
18     JoinThreads joiner(threads); // 确保所有线程被 join
19
20     for (int i = 0; i < 4; i++) {
21         threads.emplace_back(worker, i);
22     }
23
24     // 即使这里抛出异常，joiner 的析构函数也会 join 所有线程
25 }

```

多线程编程是复杂但强大的技术。在 RoboMaster 开发中，合理使用多线程可以充分利用多核处理器的能力，实现传感器采集、图像处理、控制算法、通信等任务的并行执行，从而满足机器人系统的实时性要求。但同时也要谨慎处理共享数据，避免数据竞争和死锁等问题。随着经验的积累，你会逐渐掌握编写正确、高效的多线程代码的技巧。

1.5.24. 文件操作

机器人系统需要与外部世界交换数据：读取配置文件来调整参数，记录日志以便调试和分析，保存标定数据和运行状态。C++ 标准库提供了 `fstream` 类族来处理文件输入输出，它们使用与 `cin / cout` 相同的流操作符，学习成本很低。掌握文件操作，可以让你的机器人程序更加灵活和可维护——参数可以在不重新编译的情况下修改，问题可以通过日志追溯分析。

1.5.24.1. `fstream` 基础

C++ 文件流定义在 `<fstream>` 头文件中，包含三个主要类：

- `std::ifstream`：输入文件流，用于读取文件
- `std::ofstream`：输出文件流，用于写入文件
- `std::fstream`：双向文件流，可读可写

基本的文件写入：

```
1 #include <fstream>
2 #include <iostream>
3
4 int main() {
5     // 创建并打开文件（如果文件已存在则清空）
6     std::ofstream outFile("output.txt");
7
8     if (!outFile) {
9         std::cerr << "无法打开文件" << std::endl;
10        return 1;
11    }
12
13    // 使用 << 写入，和 cout 用法相同
14    outFile << "Hello, File!" << std::endl;
15    outFile << "数值: " << 42 << std::endl;
16    outFile << "浮点数: " << 3.14159 << std::endl;
17
18    // 文件在 outFile 离开作用域时自动关闭
19    return 0;
20 }
```

基本的文件读取：

```
1 #include <fstream>
2 #include <iostream>
3 #include <string>
4
5 int main() {
6     std::ifstream inFile("output.txt");
7
8     if (!inFile) {
9         std::cerr << "无法打开文件" << std::endl;
10        return 1;
11    }
12
13    // 逐行读取
14    std::string line;
```

```

15     while (std::getline(inFile, line)) {
16         std::cout << "读取: " << line << std::endl;
17     }
18
19     return 0;
20 }

```

文件打开模式可以通过第二个参数指定：

```

1 // 常用打开模式
2 std::ios::in      // 读取 (ifstream 默认)
3 std::ios::out     // 写入 (ofstream 默认)
4 std::ios::app     // 追加模式，写入到文件末尾
5 std::ios::trunc   // 截断模式，清空现有内容 (out 的默认行为)
6 std::ios::binary  // 二进制模式
7 std::ios::ate     // 打开后定位到文件末尾
8
9 // 组合使用
10 std::ofstream logFile("log.txt", std::ios::app); // 追加写入
11 std::fstream dataFile("data.bin", std::ios::in | std::ios::out |
    std::ios::binary);

```

检查文件状态：

```

1 std::ifstream file("data.txt");
2
3 // 检查是否成功打开
4 if (!file.is_open()) {
5     std::cerr << "文件打开失败" << std::endl;
6 }
7
8 // 流状态检查
9 if (file.good()) // 一切正常
10 if (file.eof()) // 到达文件末尾
11 if (file.fail()) // 操作失败 (格式错误等)
12 if (file.bad()) // 严重错误 (如硬件故障)
13
14 // 清除错误状态
15 file.clear();
16
17 // 显式关闭 (通常不需要，析构时自动关闭)
18 file.close();

```

1.5.24.2. 文本文件读写

读取格式化数据：

```

1 // 假设文件内容:
2 // Alice 25 3.8
3 // Bob 22 3.5
4 // Charlie 24 3.9
5
6 struct Student {
7     std::string name;
8     int age;
9     double gpa;
10 };
11
12 std::vector<Student> readStudents(const std::string& filename) {
13     std::vector<Student> students;
14     std::ifstream file(filename);

```

```

15
16     if (!file) {
17         throw std::runtime_error("无法打开文件: " + filename);
18     }
19
20     Student s;
21     while (file >> s.name >> s.age >> s.gpa) {
22         students.push_back(s);
23     }
24
25     return students;
26 }

```

处理包含空格的字符串需要使用 `getline`：

```

1 // 文件内容 (逗号分隔):
2 // Alice Smith,25,3.8
3 // Bob Johnson,22,3.5
4
5 std::vector<Student> readCSV(const std::string& filename) {
6     std::vector<Student> students;
7     std::ifstream file(filename);
8     std::string line;
9
10    while (std::getline(file, line)) {
11        std::istringstream iss(line);
12        Student s;
13
14        std::getline(iss, s.name, ','); // 读到逗号为止
15        iss >> s.age;
16        iss.ignore(1); // 跳过逗号
17        iss >> s.gpa;
18
19        students.push_back(s);
20    }
21
22    return students;
23 }

```

写入格式化数据：

```

1 #include <iomanip>
2
3 void writeStudents(const std::string& filename,
4                   const std::vector<Student>& students) {
5     std::ofstream file(filename);
6
7     if (!file) {
8         throw std::runtime_error("无法创建文件: " + filename);
9     }
10
11    // 设置格式
12    file << std::fixed << std::setprecision(2);
13
14    // 写入表头
15    file << std::left << std::setw(20) << "Name"
16          << std::setw(10) << "Age"
17          << std::setw(10) << "GPA" << std::endl;
18
19    file << std::string(40, '-') << std::endl;
20
21    // 写入数据

```

```

22     for (const auto& s : students) {
23         file << std::left << std::setw(20) << s.name
24             << std::setw(10) << s.age
25             << std::setw(10) << s.gpa << std::endl;
26     }
27 }

```

1.5.24.3. 二进制文件读写

二进制模式直接读写内存中的字节，效率更高，但文件不可人工阅读：

```

1  struct SensorData {
2      double timestamp;
3      float acceleration[3];
4      float gyroscope[3];
5      uint32_t status;
6  };
7
8  // 写入二进制数据
9  void saveSensorData(const std::string& filename,
10                     const std::vector<SensorData>& data) {
11      std::ofstream file(filename, std::ios::binary);
12
13      if (!file) {
14          throw std::runtime_error("无法创建文件");
15      }
16
17      // 写入数据数量
18      size_t count = data.size();
19      file.write(reinterpret_cast<const char*>(&count), sizeof(count));
20
21      // 写入数据
22      file.write(reinterpret_cast<const char*>(data.data()),
23                data.size() * sizeof(SensorData));
24  }
25
26  // 读取二进制数据
27  std::vector<SensorData> loadSensorData(const std::string& filename) {
28      std::ifstream file(filename, std::ios::binary);
29
30      if (!file) {
31          throw std::runtime_error("无法打开文件");
32      }
33
34      // 读取数量
35      size_t count;
36      file.read(reinterpret_cast<char*>(&count), sizeof(count));
37
38      // 读取数据
39      std::vector<SensorData> data(count);
40      file.read(reinterpret_cast<char*>(data.data()),
41                count * sizeof(SensorData));
42
43      return data;
44  }

```

文件定位操作：

```

1  std::fstream file("data.bin", std::ios::in | std::ios::out |
2  std::ios::binary);

```

```

3 // 获取当前位置
4 std::streampos pos = file.tellg(); // 读取位置
5 std::streampos wpos = file.tellp(); // 写入位置
6
7 // 定位
8 file.seekg(0); // 移动到开头
9 file.seekg(100); // 移动到第 100 字节
10 file.seekg(-10, std::ios::end); // 从末尾倒数第 10 字节
11 file.seekg(50, std::ios::cur); // 从当前位置前进 50 字节
12
13 // 读取特定记录
14 size_t recordIndex = 5;
15 file.seekg(sizeof(size_t) + recordIndex * sizeof(SensorData));
16 SensorData record;
17 file.read(reinterpret_cast<char*>(&record), sizeof(record));

```

1.5.24.4. 配置文件读写

在 RoboMaster 开发中，配置文件用于存储可调参数，方便在不重新编译的情况下修改机器人行为。

简单的键值对配置文件：

```

1 // config.txt 内容:
2 // # PID 参数
3 // kp = 1.5
4 // ki = 0.1
5 // kd = 0.05
6 //
7 // # 速度限制
8 // max_speed = 5000
9 // min_speed = -5000
10
11 class ConfigReader {
12 public:
13     bool load(const std::string& filename) {
14         std::ifstream file(filename);
15         if (!file) return false;
16
17         std::string line;
18         while (std::getline(file, line)) {
19             // 跳过空行和注释
20             if (line.empty() || line[0] == '#') continue;
21
22             // 解析键值对
23             size_t eqPos = line.find('=');
24             if (eqPos == std::string::npos) continue;
25
26             std::string key = trim(line.substr(0, eqPos));
27             std::string value = trim(line.substr(eqPos + 1));
28
29             config_[key] = value;
30         }
31
32         return true;
33     }
34
35     std::string getString(const std::string& key,
36                          const std::string& defaultValue = "") const {
37         auto it = config_.find(key);
38         return (it != config_.end()) ? it->second : defaultValue;
39     }

```

```

40
41     int getInt(const std::string& key, int defaultValue = 0) const {
42         auto it = config_.find(key);
43         if (it == config_.end()) return defaultValue;
44         try {
45             return std::stoi(it->second);
46         } catch (...) {
47             return defaultValue;
48         }
49     }
50
51     double getDouble(const std::string& key, double defaultValue = 0.0)
52     const {
53         auto it = config_.find(key);
54         if (it == config_.end()) return defaultValue;
55         try {
56             return std::stod(it->second);
57         } catch (...) {
58             return defaultValue;
59         }
60     }
61
62     bool getBool(const std::string& key, bool defaultValue = false) const {
63         auto it = config_.find(key);
64         if (it == config_.end()) return defaultValue;
65         std::string val = it->second;
66         std::transform(val.begin(), val.end(), val.begin(), ::tolower);
67         return (val == "true" || val == "1" || val == "yes");
68     }
69 private:
70     static std::string trim(const std::string& s) {
71         size_t start = s.find_first_not_of(" \t\r\n");
72         if (start == std::string::npos) return "";
73         size_t end = s.find_last_not_of(" \t\r\n");
74         return s.substr(start, end - start + 1);
75     }
76
77     std::map<std::string, std::string> config_;
78 };
79
80 // 使用
81 int main() {
82     ConfigReader config;
83     if (!config.load("robot_config.txt")) {
84         std::cerr << "配置文件加载失败" << std::endl;
85         return 1;
86     }
87
88     double kp = config.getDouble("kp", 1.0);
89     double ki = config.getDouble("ki", 0.0);
90     double kd = config.getDouble("kd", 0.0);
91     int maxSpeed = config.getInt("max_speed", 3000);
92
93     std::cout << "PID: " << kp << ", " << ki << ", " << kd << std::endl;
94     std::cout << "Max Speed: " << maxSpeed << std::endl;
95
96     return 0;
97 }

```

保存配置文件：

```

1 class ConfigWriter {
2 public:
3     void set(const std::string& key, const std::string& value) {
4         config_[key] = value;
5     }
6
7     void set(const std::string& key, int value) {
8         config_[key] = std::to_string(value);
9     }
10
11    void set(const std::string& key, double value) {
12        std::ostringstream oss;
13        oss << std::fixed << std::setprecision(6) << value;
14        config_[key] = oss.str();
15    }
16
17    void set(const std::string& key, bool value) {
18        config_[key] = value ? "true" : "false";
19    }
20
21    bool save(const std::string& filename) {
22        std::ofstream file(filename);
23        if (!file) return false;
24
25        file << "# Auto-generated configuration file" << std::endl;
26        file << "# Generated at: " << getCurrentTime() << std::endl;
27        file << std::endl;
28
29        for (const auto& [key, value] : config_) {
30            file << key << " = " << value << std::endl;
31        }
32
33        return true;
34    }
35
36 private:
37    std::string getCurrentTime() {
38        auto now = std::chrono::system_clock::now();
39        auto time = std::chrono::system_clock::to_time_t(now);
40        std::string str = std::ctime(&time);
41        str.pop_back(); // 移除换行符
42        return str;
43    }
44
45    std::map<std::string, std::string> config_;
46 };

```

1.5.24.5. INI 格式配置文件

INI 格式支持分节 (section)，更适合复杂的配置：

```

1 ; robot_config.ini
2 [PID_Speed]
3 kp = 1.5
4 ki = 0.1
5 kd = 0.05
6
7 [PID_Angle]
8 kp = 2.0
9 ki = 0.0
10 kd = 0.2
11

```



```

12 [Limits]
13 max_speed = 5000
14 max_current = 10000
15
16 [Features]
17 auto_aim = true
18 debug_mode = false

```

INI 解析器实现:

```

1  class INIReader {
2  public:
3      bool load(const std::string& filename) {
4          std::ifstream file(filename);
5          if (!file) return false;
6
7          std::string line, currentSection;
8
9          while (std::getline(file, line)) {
10             line = trim(line);
11
12             // 跳过空行和注释
13             if (line.empty() || line[0] == ';' || line[0] == '#') continue;
14
15             // 检测节
16             if (line[0] == '[' && line.back() == ']') {
17                 currentSection = line.substr(1, line.size() - 2);
18                 continue;
19             }
20
21             // 解析键值对
22             size_t eqPos = line.find('=');
23             if (eqPos == std::string::npos) continue;
24
25             std::string key = trim(line.substr(0, eqPos));
26             std::string value = trim(line.substr(eqPos + 1));
27
28             // 存储为 "section.key" 格式
29             std::string fullKey = currentSection.empty() ?
30                 key : (currentSection + "." + key);
31             data_[fullKey] = value;
32         }
33
34         return true;
35     }
36
37     double getDouble(const std::string& section,
38                     const std::string& key,
39                     double defaultValue = 0.0) const {
40         std::string fullKey = section + "." + key;
41         auto it = data_.find(fullKey);
42         if (it == data_.end()) return defaultValue;
43         try {
44             return std::stod(it->second);
45         } catch (...) {
46             return defaultValue;
47         }
48     }
49
50     int getInt(const std::string& section,
51               const std::string& key,
52               int defaultValue = 0) const {

```

```

53         std::string fullKey = section + "." + key;
54         auto it = data_.find(fullKey);
55         if (it == data_.end()) return defaultValue;
56         try {
57             return std::stoi(it->second);
58         } catch (...) {
59             return defaultValue;
60         }
61     }
62
63     bool getBool(const std::string& section,
64                 const std::string& key,
65                 bool defaultValue = false) const {
66         std::string fullKey = section + "." + key;
67         auto it = data_.find(fullKey);
68         if (it == data_.end()) return defaultValue;
69         std::string val = it->second;
70         std::transform(val.begin(), val.end(), val.begin(), ::tolower);
71         return (val == "true" || val == "1" || val == "yes");
72     }
73
74 private:
75     static std::string trim(const std::string& s) {
76         size_t start = s.find_first_not_of(" \t\r\n");
77         if (start == std::string::npos) return "";
78         size_t end = s.find_last_not_of(" \t\r\n");
79         return s.substr(start, end - start + 1);
80     }
81
82     std::map<std::string, std::string> data_;
83 };
84
85 // 使用
86 int main() {
87     INIReader config;
88     if (!config.load("robot_config.ini")) {
89         std::cerr << "配置加载失败" << std::endl;
90         return 1;
91     }
92
93     // 读取速度环 PID
94     double speedKp = config.getDouble("PID_Speed", "kp", 1.0);
95     double speedKi = config.getDouble("PID_Speed", "ki", 0.0);
96     double speedKd = config.getDouble("PID_Speed", "kd", 0.0);
97
98     // 读取角度环 PID
99     double angleKp = config.getDouble("PID_Angle", "kp", 1.0);
100
101     // 读取限制
102     int maxSpeed = config.getInt("Limits", "max_speed", 3000);
103
104     // 读取功能开关
105     bool autoAim = config.getBool("Features", "auto_aim", false);
106
107     return 0;
108 }

```

1.5.24.6. 日志记录

良好的日志系统对于调试和问题追踪至关重要。以下是一个简单但实用的日志类：

```
1 #include <fstream>
```

```

2  #include <iostream>
3  #include <chrono>
4  #include <iomanip>
5  #include <mutex>
6  #include <sstream>
7
8  enum class LogLevel {
9      DEBUG,
10     INFO,
11     WARNING,
12     ERROR,
13     FATAL
14 };
15
16 class Logger {
17 public:
18     static Logger& getInstance() {
19         static Logger instance;
20         return instance;
21     }
22
23     void setLogFile(const std::string& filename) {
24         std::lock_guard<std::mutex> lock(mutex_);
25         if (file_.is_open()) {
26             file_.close();
27         }
28         file_.open(filename, std::ios::app);
29         logToFile_ = file_.is_open();
30     }
31
32     void setLogLevel(LogLevel level) {
33         minLevel_ = level;
34     }
35
36     void setConsoleOutput(bool enabled) {
37         logToConsole_ = enabled;
38     }
39
40     void log(LogLevel level, const std::string& message,
41             const char* file = "", int line = 0) {
42         if (level < minLevel_) return;
43
44         std::lock_guard<std::mutex> lock(mutex_);
45
46         std::string logLine = formatMessage(level, message, file, line);
47
48         if (logToConsole_) {
49             std::ostream& out = (level >= LogLevel::ERROR) ? std::cerr :
std::cout;
50             out << getColorCode(level) << logLine << "\033[0m" <<
std::endl;
51         }
52
53         if (logToFile_ && file_.is_open()) {
54             file_ << logLine << std::endl;
55             file_.flush(); // 确保立即写入
56         }
57     }
58
59     // 便捷方法
60     void debug(const std::string& msg, const char* file = "", int line =
0) {
61         log(LogLevel::DEBUG, msg, file, line);

```

```

62     }
63
64     void info(const std::string& msg, const char* file = "", int line =
65 0) {
66         log(LogLevel::INFO, msg, file, line);
67     }
68
69     void warning(const std::string& msg, const char* file = "", int line =
70 0) {
71         log(LogLevel::WARNING, msg, file, line);
72     }
73
74     void error(const std::string& msg, const char* file = "", int line =
75 0) {
76         log(LogLevel::ERROR, msg, file, line);
77     }
78
79     void fatal(const std::string& msg, const char* file = "", int line =
80 0) {
81         log(LogLevel::FATAL, msg, file, line);
82     }
83
84 private:
85     Logger() = default;
86     ~Logger() {
87         if (file_.is_open()) {
88             file_.close();
89         }
90     }
91
92     Logger(const Logger&) = delete;
93     Logger& operator=(const Logger&) = delete;
94
95     std::string formatMessage(LogLevel level, const std::string& message,
96                                 const char* file, int line) {
97         std::ostringstream oss;
98
99         // 时间戳
100         auto now = std::chrono::system_clock::now();
101         auto time = std::chrono::system_clock::to_time_t(now);
102         auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
103             now.time_since_epoch()) % 1000;
104
105         oss << std::put_time(std::localtime(&time), "%Y-%m-%d %H:%M:%S");
106         oss << '.' << std::setfill('0') << std::setw(3) << ms.count();
107
108         // 日志级别
109         oss << " [" << getLevelString(level) << "];"
110
111         // 文件和行号 (如果提供)
112         if (file && file[0] != '\0') {
113             oss << " [" << file << ":" << line << "];"
114         }
115
116         // 消息
117         oss << " " << message;
118
119         return oss.str();
120     }
121
122     const char* getLevelString(LogLevel level) {
123         switch (level) {
124             case LogLevel::DEBUG:    return "DEBUG";

```

```

121         case LogLevel::INFO:    return "INFO ";
122         case LogLevel::WARNING: return "WARN ";
123         case LogLevel::ERROR:   return "ERROR";
124         case LogLevel::FATAL:   return "FATAL";
125         default:                return "?????";
126     }
127 }
128
129 const char* getColorCode(LogLevel level) {
130     switch (level) {
131         case LogLevel::DEBUG:    return "\033[36m"; // 青色
132         case LogLevel::INFO:    return "\033[32m"; // 绿色
133         case LogLevel::WARNING: return "\033[33m"; // 黄色
134         case LogLevel::ERROR:   return "\033[31m"; // 红色
135         case LogLevel::FATAL:   return "\033[35m"; // 紫色
136         default:                return "\033[0m";
137     }
138 }
139
140 std::ofstream file_;
141 std::mutex mutex_;
142 LogLevel minLevel_ = LogLevel::DEBUG;
143 bool logToFile_ = false;
144 bool logToConsole_ = true;
145 };
146
147 // 便捷宏, 自动添加文件名和行号
148 #define LOG_DEBUG(msg)    Logger::getInstance().debug(msg, __FILE__,
149                          __LINE__)
150 #define LOG_INFO(msg)     Logger::getInstance().info(msg, __FILE__, __LINE__)
151 #define LOG_WARNING(msg)  Logger::getInstance().warning(msg, __FILE__,
152                          __LINE__)
153 #define LOG_ERROR(msg)    Logger::getInstance().error(msg, __FILE__,
154                          __LINE__)
155 #define LOG_FATAL(msg)    Logger::getInstance().fatal(msg, __FILE__,
156                          __LINE__)

```

使用示例:

```

1 int main() {
2     // 配置日志
3     Logger::getInstance().setLogFile("robot.log");
4     Logger::getInstance().setLogLevel(LogLevel::DEBUG);
5     Logger::getInstance().setConsoleOutput(true);
6
7     LOG_INFO("机器人系统启动");
8
9     try {
10        // 加载配置
11        LOG_DEBUG("正在加载配置文件...");
12        ConfigReader config;
13        if (!config.load("config.txt")) {
14            LOG_ERROR("配置文件加载失败");
15            return 1;
16        }
17        LOG_INFO("配置加载成功");
18
19        // 初始化硬件
20        LOG_DEBUG("初始化电机控制器...");
21        // ...
22
23        LOG_INFO("系统初始化完成, 进入主循环");

```

```

24
25     while (running) {
26         // 主循环
27         if (someError) {
28             LOG_WARNING("检测到异常状态: " + errorDescription);
29         }
30     }
31
32 } catch (const std::exception& e) {
33     LOG_FATAL(std::string("未捕获异常: ") + e.what());
34     return 1;
35 }
36
37 LOG_INFO("机器人系统关闭");
38 return 0;
39 }

```

1.5.24.7. 带格式化的日志

支持类似 `printf` 风格的格式化日志:

```

1  class FormattedLogger : public Logger {
2  public:
3      template<typename... Args>
4      void logf(LogLevel level, const char* format, Args... args) {
5          char buffer[1024];
6          snprintf(buffer, sizeof(buffer), format, args...);
7          log(level, std::string(buffer));
8      }
9  };
10
11 // 或使用 C++20 的 std::format (如果可用)
12 #ifdef __cpp_lib_format
13 #include <format>
14
15 template<typename... Args>
16 void logFormatted(LogLevel level, std::format_string<Args...> fmt, Args&&...
17 args) {
18     Logger::getInstance().log(level, std::format(fmt,
19 std::forward<Args>(args)...));
20 }
21 #endif
22
23 // 使用字符串流的方式
24 class LogStream {
25 public:
26     LogStream(LogLevel level, const char* file, int line)
27         : level_(level), file_(file), line_(line) {}
28
29     ~LogStream() {
30         Logger::getInstance().log(level_, stream_.str(), file_, line_);
31     }
32
33     template<typename T>
34     LogStream& operator<<(const T& value) {
35         stream_ << value;
36         return *this;
37     }
38
39 private:
40     LogLevel level_;
41     const char* file_;

```

```

40     int line_;
41     std::ostringstream stream_;
42 };
43
44 #define LOG(level) LogStream(level, __FILE__, __LINE__)
45
46 // 使用
47 LOG(LogLevel::INFO) << "电机速度: " << motorSpeed << " RPM, 温度: " << temp
    << "°C";

```

1.5.24.8. 日志文件轮转

长时间运行的系统需要日志轮转，避免日志文件过大：

```

1  class RotatingLogger {
2  public:
3      RotatingLogger(const std::string& baseFilename,
4                     size_t maxFileSize = 10 * 1024 * 1024, // 10MB
5                     int maxFiles = 5)
6          : baseFilename_(baseFilename),
7            maxFileSize_(maxFileSize),
8            maxFiles_(maxFiles) {
9      openNewFile();
10 }
11
12 void log(const std::string& message) {
13     std::lock_guard<std::mutex> lock(mutex_);
14
15     if (currentSize_ >= maxFileSize_) {
16         rotateFiles();
17     }
18
19     if (file_.is_open()) {
20         file_ << message << std::endl;
21         currentSize_ += message.size() + 1;
22     }
23 }
24
25 private:
26     void openNewFile() {
27         if (file_.is_open()) {
28             file_.close();
29         }
30         file_.open(baseFilename_, std::ios::app);
31
32         // 获取当前文件大小
33         file_.seekp(0, std::ios::end);
34         currentSize_ = file_.tellp();
35     }
36
37     void rotateFiles() {
38         file_.close();
39
40         // 删除最旧的文件
41         std::string oldestFile = baseFilename_ + "." +
std::to_string(maxFiles_);
42         std::remove(oldestFile.c_str());
43
44         // 重命名现有文件
45         for (int i = maxFiles_ - 1; i >= 1; i--) {
46             std::string oldName = baseFilename_ + "." + std::to_string(i);

```

```

47         std::string newName = baseFilename_ + "." + std::to_string(i
+ 1);
48         std::rename(oldName.c_str(), newName.c_str());
49     }
50
51     // 当前文件变为 .1
52     std::rename(baseFilename_.c_str(), (baseFilename_ + ".1").c_str());
53
54     // 打开新文件
55     file_.open(baseFilename_);
56     currentSize_ = 0;
57 }
58
59 std::string baseFilename_;
60 size_t maxFileSize_;
61 int maxFiles_;
62 std::ofstream file_;
63 size_t currentSize_ = 0;
64 std::mutex mutex_;
65 };

```

1.5.24.9. 数据记录与回放

在 RoboMaster 开发中，记录传感器数据用于离线分析和算法调试非常重要：

```

1  class DataRecorder {
2  public:
3      DataRecorder(const std::string& filename)
4          : file_(filename, std::ios::binary) {
5          if (!file_) {
6              throw std::runtime_error("无法创建数据文件");
7          }
8
9          // 写入文件头
10         FileHeader header;
11         header.magic = 0x524F424F; // "ROBO"
12         header.version = 1;
13         header.startTime =
std::chrono::system_clock::now().time_since_epoch().count();
14
15         file_.write(reinterpret_cast<const char*>(&header), sizeof(header));
16     }
17
18     template<typename T>
19     void record(uint32_t type, const T& data) {
20         std::lock_guard<std::mutex> lock(mutex_);
21
22         RecordHeader recordHeader;
23         recordHeader.timestamp = getTimestamp();
24         recordHeader.type = type;
25         recordHeader.size = sizeof(T);
26
27         file_.write(reinterpret_cast<const char*>(&recordHeader),
sizeof(recordHeader));
28         file_.write(reinterpret_cast<const char*>(&data), sizeof(T));
29     }
30
31     void flush() {
32         std::lock_guard<std::mutex> lock(mutex_);
33         file_.flush();
34     }
35

```



```

36 private:
37     struct FileHeader {
38         uint32_t magic;
39         uint32_t version;
40         int64_t startTime;
41     };
42
43     struct RecordHeader {
44         int64_t timestamp;
45         uint32_t type;
46         uint32_t size;
47     };
48
49     int64_t getTimestamp() {
50         return std::chrono::steady_clock::now().time_since_epoch().count();
51     }
52
53     std::ofstream file_;
54     std::mutex mutex_;
55 };
56
57 // 数据类型定义
58 enum class DataType : uint32_t {
59     IMU = 1,
60     MOTOR = 2,
61     TARGET = 3,
62     COMMAND = 4
63 };
64
65 // 使用
66 int main() {
67     DataRecorder recorder("match_2024_01_15.dat");
68
69     while (running) {
70         // 记录 IMU 数据
71         IMUData imu = readIMU();
72         recorder.record(static_cast<uint32_t>(DataType::IMU), imu);
73
74         // 记录电机数据
75         MotorData motor = readMotor();
76         recorder.record(static_cast<uint32_t>(DataType::MOTOR), motor);
77
78         // 记录目标检测结果
79         if (hasTarget) {
80             recorder.record(static_cast<uint32_t>(DataType::TARGET),
81 target);
82         }
83
84         return 0;
85     }

```

数据回放器:

```

1 class DataPlayer {
2 public:
3     DataPlayer(const std::string& filename)
4         : file_(filename, std::ios::binary) {
5         if (!file_) {
6             throw std::runtime_error("无法打开数据文件");
7         }
8     }

```

```

9         // 读取文件头
10        file_.read(reinterpret_cast<char*>(&header_), sizeof(header_));
11
12        if (header_.magic != 0x524F424F) {
13            throw std::runtime_error("无效的数据文件格式");
14        }
15    }
16
17    bool readNext(uint32_t& type, std::vector<char>& data, int64_t&
timestamp) {
18        RecordHeader recordHeader;
19        if (!file_.read(reinterpret_cast<char*>(&recordHeader),
sizeof(recordHeader))) {
20            return false;
21        }
22
23        type = recordHeader.type;
24        timestamp = recordHeader.timestamp;
25        data.resize(recordHeader.size);
26
27        return file_.read(data.data(), recordHeader.size).good();
28    }
29
30    private:
31        struct FileHeader {
32            uint32_t magic;
33            uint32_t version;
34            int64_t startTime;
35        };
36
37        struct RecordHeader {
38            int64_t timestamp;
39            uint32_t type;
40            uint32_t size;
41        };
42
43        std::ifstream file_;
44        FileHeader header_;
45    };
46
47    // 回放数据
48    void replayData(const std::string& filename) {
49        DataPlayer player(filename);
50
51        uint32_t type;
52        std::vector<char> data;
53        int64_t timestamp;
54
55        while (player.readNext(type, data, timestamp)) {
56            switch (static_cast<DataType>(type)) {
57                case DataType::IMU: {
58                    IMUData* imu = reinterpret_cast<IMUData*>(data.data());
59                    std::cout << "IMU: " << imu->gyro[0] << ", "
60                        << imu->gyro[1] << ", " << imu->gyro[2]
61                        << std::endl;
62                    break;
63                }
64                case DataType::MOTOR: {
65                    MotorData* motor =
reinterpret_cast<MotorData*>(data.data());
66                    std::cout << "Motor: " << motor->speed << " RPM"
<< std::endl;
67                    break;

```

```

67         }
68         // ... 其他类型
69     }
70 }
71 }

```

文件操作是程序与持久化数据交互的桥梁。在 RoboMaster 开发中，合理使用配置文件可以让参数调整更加灵活，良好的日志系统可以帮助快速定位问题，数据记录功能则为算法分析和改进提供了基础。掌握这些技能，将使你的机器人程序更加健壮和易于维护。

1.5.25. Eigen 矩阵库

机器人学的核心是数学，而数学的核心是线性代数。从坐标变换到姿态估计，从卡尔曼滤波到运动学求解，矩阵运算无处不在。Eigen 是 C++ 中最流行的线性代数库，它提供了高效的向量、矩阵运算以及丰富的几何变换功能。在 RoboMaster 开发中，无论是自瞄算法的坐标变换、底盘运动学解算，还是导航定位的状态估计，Eigen 都是不可或缺的工具。

Eigen 是一个纯头文件库，无需编译链接，只需包含头文件即可使用。它利用 C++ 模板和表达式模板技术，在编译时优化矩阵运算，性能可与手写的优化代码媲美。

1.5.25.1. 安装与配置

在 Ubuntu 系统上安装 Eigen：

```
1 sudo apt install libeigen3-dev
```

在 CMake 项目中使用 Eigen：

```

1 cmake_minimum_required(VERSION 3.10)
2 project(my_project)
3
4 find_package(Eigen3 REQUIRED)
5
6 add_executable(my_program main.cpp)
7 target_link_libraries(my_program Eigen3::Eigen)

```

使用时包含主头文件：

```

1 #include <Eigen/Dense>    // 稠密矩阵运算
2 #include <Eigen/Geometry> // 几何变换（四元数、旋转等）
3
4 // 或者包含全部功能
5 #include <Eigen/Eigen>

```

1.5.25.2. 向量与矩阵基础

Eigen 使用模板类表示向量和矩阵。最常用的是固定大小的类型：

```

1 #include <Eigen/Dense>
2 #include <iostream>
3
4 int main() {
5     // 向量（列向量）
6     Eigen::Vector2d v2;    // 2 维 double 向量
7     Eigen::Vector3d v3;    // 3 维 double 向量
8     Eigen::Vector4d v4;    // 4 维 double 向量
9     Eigen::Vector3f v3f;    // 3 维 float 向量

```

```

10     Eigen::Vector3i v3i;        // 3 维 int 向量
11
12     // 矩阵
13     Eigen::Matrix2d m2;         // 2x2 double 矩阵
14     Eigen::Matrix3d m3;         // 3x3 double 矩阵
15     Eigen::Matrix4d m4;         // 4x4 double 矩阵
16     Eigen::Matrix3f m3f;        // 3x3 float 矩阵
17
18     // 通用矩阵类型
19     Eigen::Matrix<double, 3, 4> m34; // 3x4 double 矩阵
20     Eigen::Matrix<float, 6, 6> m66;  // 6x6 float 矩阵
21
22     // 动态大小矩阵
23     Eigen::MatrixXd mDynamic;       // 动态大小 double 矩阵
24     Eigen::VectorXd vDynamic;       // 动态大小 double 向量
25
26     return 0;
27 }

```

类型命名规则：Matrix 或 Vector + 维度 (2/3/4/X) + 数据类型 (d=double, f=float, i=int)。X 表示动态大小。

初始化向量和矩阵：

```

1 // 向量初始化
2 Eigen::Vector3d v1(1.0, 2.0, 3.0);
3 Eigen::Vector3d v2 = {4.0, 5.0, 6.0}; // C++11
4
5 // 使用逗号初始化器
6 Eigen::Vector3d v3;
7 v3 << 7.0, 8.0, 9.0;
8
9 // 矩阵初始化 (按行填充)
10 Eigen::Matrix3d m;
11 m << 1, 2, 3,
12     4, 5, 6,
13     7, 8, 9;
14
15 // 特殊矩阵
16 Eigen::Matrix3d zero = Eigen::Matrix3d::Zero(); // 零矩阵
17 Eigen::Matrix3d ones = Eigen::Matrix3d::Ones(); // 全 1 矩阵
18 Eigen::Matrix3d identity = Eigen::Matrix3d::Identity(); // 单位矩阵
19 Eigen::Matrix3d random = Eigen::Matrix3d::Random(); // 随机矩阵 [-1, 1]
20 Eigen::Matrix3d constant = Eigen::Matrix3d::Constant(5.0); // 常数矩阵
21
22 // 动态矩阵需要指定大小
23 Eigen::MatrixXd dynMat(3, 4); // 3 行 4 列
24 dynMat.setZero(); // 设为零矩阵
25 dynMat.setIdentity(); // 设为单位矩阵 (对角线为 1)
26 dynMat.setRandom(); // 设为随机值

```

访问元素：

```

1 Eigen::Vector3d v(1, 2, 3);
2 Eigen::Matrix3d m = Eigen::Matrix3d::Random();
3
4 // 向量元素访问
5 double x = v(0); // 第一个元素
6 double y = v(1); // 第二个元素
7 double z = v(2); // 第三个元素
8
9 // 向量专用访问器

```

```

10 double x2 = v.x();    // 等价于 v(0)
11 double y2 = v.y();    // 等价于 v(1)
12 double z2 = v.z();    // 等价于 v(2)
13
14 // 矩阵元素访问
15 double elem = m(1, 2); // 第 2 行, 第 3 列 (0-indexed)
16
17 // 修改元素
18 v(0) = 10.0;
19 m(1, 2) = 5.0;
20
21 // 获取行和列
22 Eigen::Vector3d col0 = m.col(0); // 第一列
23 Eigen::RowVector3d row1 = m.row(1); // 第二行
24
25 // 矩阵块操作
26 Eigen::Matrix2d block = m.block<2, 2>(0, 0); // 左上角 2x2 块
27 Eigen::Vector2d top2 = v.head<2>();          // 前 2 个元素
28 Eigen::Vector2d bottom2 = v.tail<2>();        // 后 2 个元素

```

1.5.25.3. 基本运算

向量运算:

```

1 Eigen::Vector3d v1(1, 2, 3);
2 Eigen::Vector3d v2(4, 5, 6);
3
4 // 加减法
5 Eigen::Vector3d sum = v1 + v2;
6 Eigen::Vector3d diff = v1 - v2;
7
8 // 标量乘除
9 Eigen::Vector3d scaled = v1 * 2.0;
10 Eigen::Vector3d divided = v1 / 2.0;
11
12 // 点积
13 double dot = v1.dot(v2); // 1*4 + 2*5 + 3*6 = 32
14
15 // 叉积 (仅限 3D 向量)
16 Eigen::Vector3d cross = v1.cross(v2);
17
18 // 范数
19 double norm = v1.norm(); // L2 范数 (长度)
20 double squaredNorm = v1.squaredNorm(); // 范数的平方 (避免开方, 更快)
21
22 // 归一化
23 Eigen::Vector3d normalized = v1.normalized(); // 返回单位向量, 不修改 v1
24 v1.normalize(); // 原地归一化
25
26 // 元素级运算
27 Eigen::Vector3d elemMul = v1.cwiseProduct(v2); // 逐元素乘法
28 Eigen::Vector3d elemDiv = v1.cwiseQuotient(v2); // 逐元素除法
29 Eigen::Vector3d elemAbs = v1.cwiseAbs(); // 逐元素取绝对值
30
31 // 统计
32 double maxVal = v1.maxCoeff();
33 double minVal = v1.minCoeff();
34 double sum_all = v1.sum();
35 double mean = v1.mean();

```

矩阵运算:

```

1 Eigen::Matrix3d A = Eigen::Matrix3d::Random();
2 Eigen::Matrix3d B = Eigen::Matrix3d::Random();
3 Eigen::Vector3d v = Eigen::Vector3d::Random();
4
5 // 加减法
6 Eigen::Matrix3d sum = A + B;
7 Eigen::Matrix3d diff = A - B;
8
9 // 标量乘法
10 Eigen::Matrix3d scaled = A * 2.0;
11
12 // 矩阵乘法
13 Eigen::Matrix3d product = A * B;
14
15 // 矩阵-向量乘法
16 Eigen::Vector3d result = A * v;
17
18 // 转置
19 Eigen::Matrix3d At = A.transpose();
20 A.transposeInPlace(); // 原地转置
21
22 // 逆矩阵
23 Eigen::Matrix3d Ainv = A.inverse();
24
25 // 行列式
26 double det = A.determinant();
27
28 // 迹（对角线元素之和）
29 double tr = A.trace();
30
31 // 对角线
32 Eigen::Vector3d diag = A.diagonal();
33 Eigen::Matrix3d diagMat = v.asDiagonal(); // 向量转对角矩阵

```

求解线性方程组：

```

1 // 求解 Ax = b
2 Eigen::Matrix3d A;
3 A << 1, 2, 3,
4      4, 5, 6,
5      7, 8, 10;
6
7 Eigen::Vector3d b(3, 6, 9);
8
9 // 方法 1: 直接求逆（小矩阵可以，大矩阵不推荐）
10 Eigen::Vector3d x1 = A.inverse() * b;
11
12 // 方法 2: LU 分解（通用，较快）
13 Eigen::Vector3d x2 = A.lu().solve(b);
14
15 // 方法 3: QR 分解（更稳定）
16 Eigen::Vector3d x3 = A.colPivHouseholderQr().solve(b);
17
18 // 方法 4: LDLT 分解（对称正定矩阵最快）
19 Eigen::Matrix3d S = A.transpose() * A; // 对称正定
20 Eigen::Vector3d x4 = S.ldlt().solve(A.transpose() * b);
21
22 // 检验解的精度
23 double error = (A * x2 - b).norm();
24 std::cout << "误差: " << error << std::endl;

```

特征值分解：

```

1 Eigen::Matrix3d A;
2 A << 1, 2, 1,
3     2, 4, 2,
4     1, 2, 3;
5
6 // 自伴随矩阵（对称矩阵）的特征分解
7 Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d> solver(A);
8
9 if (solver.info() == Eigen::Success) {
10     Eigen::Vector3d eigenvalues = solver.eigenvalues();
11     Eigen::Matrix3d eigenvectors = solver.eigenvectors();
12
13     std::cout << "特征值:\n" << eigenvalues << std::endl;
14     std::cout << "特征向量:\n" << eigenvectors << std::endl;
15 }

```

1.5.25.4. 姿态表示

机器人的姿态（朝向）可以用多种方式表示：旋转矩阵、欧拉角、轴角、四元数。每种表示有其优缺点，Eigen 的 Geometry 模块提供了完整支持。

旋转矩阵是最直观表示，3x3 正交矩阵，行列式为 1：

```

1 #include <Eigen/Geometry>
2
3 // 绕 X 轴旋转
4 double angle = M_PI / 4; // 45 度
5 Eigen::Matrix3d Rx;
6 Rx << 1, 0, 0,
7     0, cos(angle), -sin(angle),
8     0, sin(angle), cos(angle);
9
10 // 使用 AngleAxis 更方便
11 Eigen::AngleAxisd rotX(angle, Eigen::Vector3d::UnitX()); // 绕 X 轴
12 Eigen::AngleAxisd rotY(angle, Eigen::Vector3d::UnitY()); // 绕 Y 轴
13 Eigen::AngleAxisd rotZ(angle, Eigen::Vector3d::UnitZ()); // 绕 Z 轴
14
15 // 转换为旋转矩阵
16 Eigen::Matrix3d R = rotZ.toRotationMatrix();
17
18 // 组合旋转（注意顺序：先应用右边的）
19 Eigen::Matrix3d Rzyx = rotZ.toRotationMatrix() *
20     rotY.toRotationMatrix() *
21     rotX.toRotationMatrix();
22
23 // 应用旋转
24 Eigen::Vector3d v(1, 0, 0);
25 Eigen::Vector3d v_rotated = R * v;

```

欧拉角用三个角度表示旋转，直观但有万向锁问题：

```

1 // 欧拉角（ZYX 顺序，常用于航空：yaw-pitch-roll）
2 double yaw = 0.5; // 偏航角
3 double pitch = 0.2; // 俯仰角
4 double roll = 0.1; // 滚转角
5
6 // 欧拉角转旋转矩阵
7 Eigen::Matrix3d R;
8 R = Eigen::AngleAxisd(yaw, Eigen::Vector3d::UnitZ()) *
9     Eigen::AngleAxisd(pitch, Eigen::Vector3d::UnitY()) *
10     Eigen::AngleAxisd(roll, Eigen::Vector3d::UnitX());

```

```

11
12 // 旋转矩阵转欧拉角
13 Eigen::Vector3d euler = R.eulerAngles(2, 1, 0); // ZYX 顺序
14 // euler(0) = yaw, euler(1) = pitch, euler(2) = roll

```

四元数是最推荐的姿态表示，避免万向锁，插值平滑，计算高效：

```

1 // 创建四元数
2 Eigen::Quaterniond q1 = Eigen::Quaterniond::Identity(); // 单位四元数（无旋转）
3
4 // 从轴角创建
5 Eigen::Quaterniond q2(Eigen::AngleAxisd(M_PI /
6 4, Eigen::Vector3d::UnitZ()));
7
8 // 从旋转矩阵创建
9 Eigen::Matrix3d R = Eigen::Matrix3d::Identity();
10 Eigen::Quaterniond q3(R);
11
12 // 直接指定分量 (w, x, y, z)
13 Eigen::Quaterniond q4(1.0, 0.0, 0.0, 0.0); // w, x, y, z
14
15 // 四元数运算
16 Eigen::Quaterniond q_product = q1 * q2; // 组合旋转
17 Eigen::Quaterniond q_inv = q2.inverse(); // 逆（共轭）
18 Eigen::Quaterniond q_conj = q2.conjugate(); // 共轭
19
20 // 归一化
21 q2.normalize();
22 Eigen::Quaterniond q_norm = q2.normalized();
23
24 // 应用旋转
25 Eigen::Vector3d v(1, 0, 0);
26 Eigen::Vector3d v_rotated = q2 * v;
27
28 // 四元数转旋转矩阵
29 Eigen::Matrix3d R2 = q2.toRotationMatrix();
30
31 // 四元数插值（球面线性插值）
32 double t = 0.5; // 插值参数 [0, 1]
33 Eigen::Quaterniond q_interp = q1.slerp(t, q2);
34
35 // 访问分量
36 double w = q2.w();
37 double x = q2.x();
38 double y = q2.y();
39 double z = q2.z();
40 Eigen::Vector3d xyz = q2.vec(); // 向量部分 (x, y, z)

```

轴角表示：

```

1 // 轴角：旋转轴 + 旋转角度
2 Eigen::Vector3d axis(0, 0, 1); // Z 轴
3 double angle = M_PI / 3; // 60 度
4
5 Eigen::AngleAxisd aa(angle, axis);
6
7 // 转换
8 Eigen::Matrix3d R = aa.toRotationMatrix();
9 Eigen::Quaterniond q(aa);
10
11 // 从旋转矩阵恢复
12 Eigen::AngleAxisd aa2(R);

```



```

13 std::cout << "轴: " << aa2.axis().transpose() << std::endl;
14 std::cout << "角: " << aa2.angle() << std::endl;

```

1.5.25.5. 坐标变换

齐次变换矩阵结合了旋转和平移:

```

1 // 创建变换
2 Eigen::Isometry3d T = Eigen::Isometry3d::Identity();
3
4 // 设置旋转
5 T.rotate(Eigen::AngleAxisd(M_PI / 4, Eigen::Vector3d::UnitZ()));
6 // 或
7 T.linear() = Eigen::Matrix3d::Identity(); // 直接设置旋转矩阵部分
8
9 // 设置平移
10 T.pretranslate(Eigen::Vector3d(1, 2, 3));
11 // 或
12 T.translation() = Eigen::Vector3d(1, 2, 3);
13
14 // 获取旋转和平移
15 Eigen::Matrix3d R = T.rotation();
16 Eigen::Vector3d t = T.translation();
17
18 // 变换点
19 Eigen::Vector3d p(1, 0, 0);
20 Eigen::Vector3d p_transformed = T * p;
21
22 // 组合变换
23 Eigen::Isometry3d T1 = Eigen::Isometry3d::Identity();
24 Eigen::Isometry3d T2 = Eigen::Isometry3d::Identity();
25 T1.translate(Eigen::Vector3d(1, 0, 0));
26 T2.rotate(Eigen::AngleAxisd(M_PI / 2, Eigen::Vector3d::UnitZ()));
27
28 Eigen::Isometry3d T_combined = T1 * T2; // 先 T2 后 T1
29
30 // 逆变换
31 Eigen::Isometry3d T_inv = T.inverse();
32
33 // 转换为 4x4 矩阵
34 Eigen::Matrix4d mat = T.matrix();

```

坐标系之间的变换:

```

1 // 相机坐标系到世界坐标系的变换
2 Eigen::Isometry3d T_world_camera = Eigen::Isometry3d::Identity();
3 T_world_camera.rotate(Eigen::Quaterniond(0.707, 0, 0.707, 0)); // 旋转
4 T_world_camera.pretranslate(Eigen::Vector3d(0, 0, 1.5)); // 相机高度
5
6 // 目标在相机坐标系中的位置
7 Eigen::Vector3d p_camera(2, 0.5, 5);
8
9 // 转换到世界坐标系
10 Eigen::Vector3d p_world = T_world_camera * p_camera;
11
12 // 从世界坐标系转换到相机坐标系
13 Eigen::Isometry3d T_camera_world = T_world_camera.inverse();
14 Eigen::Vector3d p_camera_back = T_camera_world * p_world;

```

1.5.25.6. RoboMaster 应用实例

自瞄系统坐标变换:

```
1 class CoordinateTransformer {
2 public:
3     CoordinateTransformer() {
4         // 相机到云台的变换 (固定安装参数)
5         T_gimbal_camera_ = Eigen::Isometry3d::Identity();
6         T_gimbal_camera_.translate(Eigen::Vector3d(0.05, 0, -0.03)); // 相机
偏移
7
8         // 云台到底盘的变换 (会随云台角度变化)
9         T_chassis_gimbal_ = Eigen::Isometry3d::Identity();
10    }
11
12    // 更新云台姿态
13    void updateGimbalPose(double yaw, double pitch) {
14        T_chassis_gimbal_ = Eigen::Isometry3d::Identity();
15        T_chassis_gimbal_.rotate(
16            Eigen::AngleAxisd(yaw, Eigen::Vector3d::UnitZ()) *
17            Eigen::AngleAxisd(pitch, Eigen::Vector3d::UnitY())
18        );
19    }
20
21    // 将相机坐标系中的目标转换到底盘坐标系
22    Eigen::Vector3d cameraToChassisFrame(const Eigen::Vector3d& p_camera) {
23        Eigen::Isometry3d T_chassis_camera = T_chassis_gimbal_ *
T_gimbal_camera_;
24        return T_chassis_camera * p_camera;
25    }
26
27    // 计算瞄准角度
28    Eigen::Vector2d calculateAimAngles(const Eigen::Vector3d& target_camera)
{
29        // 转换到云台坐标系
30        Eigen::Vector3d target_gimbal = T_gimbal_camera_ * target_camera;
31
32        // 计算需要的 yaw 和 pitch 角度
33        double yaw = atan2(target_gimbal.y(), target_gimbal.x());
34        double pitch = atan2(-target_gimbal.z(),
sqrt(target_gimbal.x() * target_gimbal.x() +
35            target_gimbal.y() * target_gimbal.y()));
36
37        return Eigen::Vector2d(yaw, pitch);
38    }
39 }
40
41 private:
42     Eigen::Isometry3d T_gimbal_camera_;
43     Eigen::Isometry3d T_chassis_gimbal_;
44 };
```

麦克纳姆轮底盘运动学:

```
1 class MecanumKinematics {
2 public:
3     MecanumKinematics(double wheelRadius, double lx, double ly)
4         : r_(wheelRadius), lx_(lx), ly_(ly) {
5         // 正运动学矩阵: 轮速 -> 底盘速度
6         //  $[v_x, v_y, \omega]^T = J * [w_1, w_2, w_3, w_4]^T$ 
7         double k = r_ / 4.0;
8         J_forward_ << k, k, k, k,
9            -k, k, k, -k,
```

```

10         -k/(lx_+ly_), k/(lx_+ly_), -k/(lx_+ly_), k/(lx_+ly_);
11
12     // 逆运动学矩阵: 底盘速度 -> 轮速
13     // [w1, w2, w3, w4]^T = J_inv * [vx, vy, omega]^T
14     double l = lx_ + ly_;
15     J_inverse_ << 1, -1, -l,
16                  1, 1, l,
17                  1, 1, -l,
18                  1, -1, l;
19     J_inverse_ /= r_;
20 }
21
22 // 底盘速度 -> 轮速
23 Eigen::Vector4d inverseKinematics(double vx, double vy, double omega) {
24     Eigen::Vector3d chassisVel(vx, vy, omega);
25     return J_inverse_ * chassisVel;
26 }
27
28 // 轮速 -> 底盘速度
29 Eigen::Vector3d forwardKinematics(const Eigen::Vector4d& wheelSpeeds) {
30     return J_forward_ * wheelSpeeds;
31 }
32
33 private:
34     double r_; // 轮子半径
35     double lx_; // 轮子到中心的 x 距离
36     double ly_; // 轮子到中心的 y 距离
37
38     Eigen::Matrix<double, 3, 4> J_forward_;
39     Eigen::Matrix<double, 4, 3> J_inverse_;
40 };
41
42 // 使用
43 int main() {
44     MecanumKinematics kinematics(0.076, 0.2, 0.2); // 轮径 76mm
45
46     // 期望底盘速度
47     double vx = 1.0; // 前进 1 m/s
48     double vy = 0.5; // 左移 0.5 m/s
49     double omega = 0.3; // 逆时针旋转 0.3 rad/s
50
51     // 计算轮速
52     Eigen::Vector4d wheelSpeeds = kinematics.inverseKinematics(vx, vy,
53 omega);
54     std::cout << "轮速 (rad/s): " << wheelSpeeds.transpose() << std::endl;
55     return 0;
56 }

```

扩展卡尔曼滤波器:

```

1 class EKF {
2 public:
3     EKF() {
4         // 状态: [x, y, theta, vx, vy, omega]
5         state_ = Eigen::VectorXd::Zero(6);
6
7         // 协方差矩阵
8         P_ = Eigen::MatrixXd::Identity(6, 6) * 0.1;
9
10        // 过程噪声
11        Q_ = Eigen::MatrixXd::Identity(6, 6) * 0.01;

```

```

12         Q_(3, 3) = Q_(4, 4) = Q_(5, 5) = 0.1;    // 速度噪声更大
13
14         // 观测噪声（位置观测）
15         R_ = Eigen::MatrixXd::Identity(3, 3) * 0.05;
16     }
17
18     // 预测步骤
19     void predict(double dt) {
20         // 状态转移
21         double theta = state_(2);
22         double vx = state_(3);
23         double vy = state_(4);
24         double omega = state_(5);
25
26         // 预测状态
27         Eigen::VectorXd state_pred(6);
28         state_pred(0) = state_(0) + (vx * cos(theta) - vy * sin(theta))
* dt;
29         state_pred(1) = state_(1) + (vx * sin(theta) + vy * cos(theta))
* dt;
30         state_pred(2) = state_(2) + omega * dt;
31         state_pred(3) = state_(3);
32         state_pred(4) = state_(4);
33         state_pred(5) = state_(5);
34
35         // 雅可比矩阵
36         Eigen::MatrixXd F = Eigen::MatrixXd::Identity(6, 6);
37         F(0, 2) = (-vx * sin(theta) - vy * cos(theta)) * dt;
38         F(0, 3) = cos(theta) * dt;
39         F(0, 4) = -sin(theta) * dt;
40         F(1, 2) = (vx * cos(theta) - vy * sin(theta)) * dt;
41         F(1, 3) = sin(theta) * dt;
42         F(1, 4) = cos(theta) * dt;
43         F(2, 5) = dt;
44
45         // 预测协方差
46         P_ = F * P_ * F.transpose() + Q_;
47         state_ = state_pred;
48     }
49
50     // 更新步骤（位置观测）
51     void updatePosition(const Eigen::Vector3d& z) {
52         // 观测矩阵
53         Eigen::MatrixXd H = Eigen::MatrixXd::Zero(3, 6);
54         H(0, 0) = 1;
55         H(1, 1) = 1;
56         H(2, 2) = 1;
57
58         // 卡尔曼增益
59         Eigen::MatrixXd S = H * P_ * H.transpose() + R_;
60         Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
61
62         // 更新状态
63         Eigen::Vector3d y = z - H * state_;
64
65         // 角度归一化
66         while (y(2) > M_PI) y(2) -= 2 * M_PI;
67         while (y(2) < -M_PI) y(2) += 2 * M_PI;
68
69         state_ = state_ + K * y;
70
71         // 更新协方差
72         Eigen::MatrixXd I = Eigen::MatrixXd::Identity(6, 6);

```

```

73     P_ = (I - K * H) * P_;
74 }
75
76 Eigen::VectorXd getState() const { return state_; }
77 Eigen::Vector3d getPosition() const { return state_.head<3>(); }
78 Eigen::Vector3d getVelocity() const { return state_.tail<3>(); }
79
80 private:
81     Eigen::VectorXd state_;
82     Eigen::MatrixXd P_;
83     Eigen::MatrixXd Q_;
84     Eigen::MatrixXd R_;
85 };

```

弹道解算：

```

1  class BallisticSolver {
2  public:
3      BallisticSolver(double bulletSpeed, double gravity = 9.8)
4          : v0_(bulletSpeed), g_(gravity) {}
5
6      // 计算击中目标需要的发射仰角
7      // target: 目标相对于枪口的位置 (x: 前方距离, z: 高度差)
8      double solve(double x, double z) {
9          // 使用迭代法求解
10         double pitch = atan2(z, x); // 初始猜测
11
12         for (int i = 0; i < 20; i++) {
13             double t = x / (v0_ * cos(pitch)); // 飞行时间
14             double z_hit = v0_ * sin(pitch) * t - 0.5 * g_ * t * t; // 落点
高度
15             double error = z - z_hit;
16
17             if (abs(error) < 0.001) break; // 收敛
18
19             // 调整仰角
20             pitch += error * 0.1 / x;
21         }
22
23         return pitch;
24     }
25
26     // 考虑空气阻力的弹道解算（使用数值积分）
27     Eigen::Vector2d solveWithDrag(const Eigen::Vector3d& target, double
dragCoef) {
28         double distance = target.head<2>().norm();
29         double yaw = atan2(target.y(), target.x());
30         double z = target.z();
31
32         // 二分搜索发射仰角
33         double low = -M_PI / 4, high = M_PI / 4;
34         double pitch = 0;
35
36         for (int iter = 0; iter < 50; iter++) {
37             pitch = (low + high) / 2;
38             double z_hit = simulateTrajectory(distance, pitch, dragCoef);
39
40             if (abs(z_hit - z) < 0.001) break;
41
42             if (z_hit < z) {
43                 low = pitch;
44             } else {

```

```

45         high = pitch;
46     }
47 }
48
49     return Eigen::Vector2d(yaw, pitch);
50 }
51
52 private:
53     double simulateTrajectory(double targetDist, double pitch, double drag)
54     {
55         Eigen::Vector3d pos = Eigen::Vector3d::Zero();
56         Eigen::Vector3d vel(v0_ * cos(pitch), 0, v0_ * sin(pitch));
57         double dt = 0.0001;
58
59         while (pos.x() < targetDist && pos.z() > -10) {
60             // 空气阻力
61             double v = vel.norm();
62             Eigen::Vector3d dragForce = -drag * v * vel;
63
64             // 重力
65             Eigen::Vector3d gravity(0, 0, -g_);
66
67             // 更新速度和位置
68             Eigen::Vector3d acc = dragForce + gravity;
69             vel += acc * dt;
70             pos += vel * dt;
71         }
72
73         return pos.z();
74     }
75
76     double v0_; // 初速度
77     double g_; // 重力加速度
78 };

```

1.5.25.7. 性能优化技巧

Eigen 提供了多种优化手段：

```

1 // 1. 使用固定大小矩阵（编译时优化）
2 Eigen::Matrix3d m3; // 比 MatrixXd 快
3
4 // 2. 避免不必要的临时对象
5 // 不好
6 Eigen::Matrix3d result = A * B * C * D; // 可能产生临时对象
7
8 // 好
9 Eigen::Matrix3d result;
10 result.noalias() = A * B * C * D; // 避免别名检查
11
12 // 3. 使用 .eval() 强制求值
13 auto temp = (A * B).eval(); // 立即计算，避免惰性求值的问题
14
15 // 4. 内存对齐（SIMD 优化）
16 // Eigen 默认对固定大小矩阵进行 16 字节对齐
17 // 在类中使用时需要特殊宏
18 class MyClass {
19 public:
20     EIGEN_MAKE_ALIGNED_OPERATOR_NEW // 确保 new 时正确对齐
21
22     Eigen::Vector4d vec; // 需要 32 字节对齐
23     Eigen::Matrix4d mat;

```

```

24 };
25
26 // 5. 使用 Map 包装现有数组（零拷贝）
27 double data[9] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
28 Eigen::Map<Eigen::Matrix3d> mapped(data);
29 // mapped 直接操作 data, 无拷贝
30
31 // 6. 块操作避免拷贝
32 Eigen::Matrix4d big;
33 big.topLeftCorner<3, 3>() = Eigen::Matrix3d::Identity(); // 直接写入
34
35 // 7. 并行计算（OpenMP）
36 #ifdef EIGEN_HAS_OPENMP
37 Eigen::setNbThreads(4); // 设置线程数
38 #endif

```

1.5.25.8. 常见错误

避免这些常见陷阱:

```

1 // 1. 混淆固定大小和动态大小
2 Eigen::Vector3d v3;
3 Eigen::VectorXd vx(3);
4 // v3 = vx; // 可能出问题, 建议明确
5
6 // 2. 矩阵乘法维度不匹配
7 Eigen::Matrix<double, 3, 2> A;
8 Eigen::Matrix<double, 3, 2> B;
9 // auto C = A * B; // 错误! 3x2 不能乘 3x2
10
11 // 3. 自赋值问题
12 Eigen::MatrixXd A(3, 3);
13 // A = A * A; // 可能出问题
14 A = (A * A).eval(); // 正确
15
16 // 4. 归一化零向量
17 Eigen::Vector3d v = Eigen::Vector3d::Zero();
18 // v.normalize(); // 未定义行为
19 if (v.norm() > 1e-10) v.normalize(); // 安全
20
21 // 5. 忘记初始化
22 Eigen::Matrix3d m; // 未初始化, 包含垃圾值!
23 Eigen::Matrix3d m2 = Eigen::Matrix3d::Zero(); // 正确

```

Eigen 是 RoboMaster 开发中最常用的数学库之一。从简单的向量运算到复杂的状态估计，Eigen 提供了高效、易用的接口。熟练掌握 Eigen，可以让你更专注于算法本身，而不是底层的数学实现细节。建议多阅读 Eigen 的官方文档，其中包含了更多高级功能和优化技巧。

1.5.26. Ceres 非线性优化库

在机器人系统中，许多问题可以归结为优化问题：相机标定需要找到最小化重投影误差的内参和畸变系数，状态估计需要找到最符合传感器观测的位姿，弹道模型需要拟合最接近实际轨迹的参数。这些问题的共同特点是：目标函数是非线性的，需要找到使目标函数最小化的参数。

Ceres Solver 是 Google 开发的开源 C++ 库，专门用于求解非线性最小二乘问题。它在机器人、计算机视觉和机器学习领域广泛使用，是 SLAM 后端优化、相机标定、传感器融合等应用的标准工具。Ceres 提供了强大的自动求导功能，让你专注于定义问题本身，而无需手动计算复杂的雅可比矩阵。

1.5.26.1. 安装与配置

在 Ubuntu 系统上安装 Ceres:

```
1 # 安装依赖
2 sudo apt install libgoogle-glog-dev libgflags-dev
3 sudo apt install libatlas-base-dev libeigen3-dev libsuitesparse-dev
4
5 # 安装 Ceres
6 sudo apt install libceres-dev
```

如果需要最新版本, 可以从源码编译:

```
1 git clone https://ceres-solver.googleusercontent.com/ceres-solver
2 cd ceres-solver
3 mkdir build && cd build
4 cmake ..
5 make -j4
6 sudo make install
```

在 CMake 项目中使用 Ceres:

```
1 cmake_minimum_required(VERSION 3.10)
2 project(my_project)
3
4 find_package(Ceres REQUIRED)
5
6 add_executable(my_program main.cpp)
7 target_link_libraries(my_program Ceres::ceres)
```

1.5.26.2. 非线性最小二乘问题

Ceres 求解的是如下形式的优化问题:

$$\min_x \sum_{i=1}^n \rho_i(\|f_i(x)\|^2)$$

其中 x 是待优化的参数向量, $f_{i(x)}$ 是残差函数 (residual function), ρ_i 是损失函数 (loss function, 用于处理异常值)。最简单的情况下, ρ_i 是恒等函数, 问题就是最小化残差平方和。

让我们从一个简单的例子开始——拟合曲线 $y = e^{mx+c}$ 到一组带噪声的数据点:

```
1 #include <ceres/ceres.h>
2 #include <iostream>
3 #include <vector>
4
5 // 定义残差函数 (代价函数)
6 struct ExponentialResidual {
7     ExponentialResidual(double x, double y) : x_(x), y_(y) {}
8
9     // 模板函数, 支持自动求导
10    template <typename T>
11    bool operator()(const T* const m, const T* const c, T* residual) const {
12        // residual = y - exp(m * x + c)
13        residual[0] = y_ - exp(m[0] * x_ + c[0]);
14        return true;
15    }
16
17 private:
18     const double x_;
19     const double y_;
```



```

20 };
21
22 int main() {
23     // 真实参数
24     const double m_true = 0.3;
25     const double c_true = 0.1;
26
27     // 生成带噪声的数据
28     std::vector<double> x_data, y_data;
29     for (int i = 0; i < 100; i++) {
30         double x = i * 0.1;
31         double y = exp(m_true * x + c_true) + 0.1 * ((rand() % 100) / 100.0
32 - 0.5);
33         x_data.push_back(x);
34         y_data.push_back(y);
35     }
36
37     // 初始参数猜测
38     double m = 0.0;
39     double c = 0.0;
40
41     // 构建问题
42     ceres::Problem problem;
43
44     for (size_t i = 0; i < x_data.size(); i++) {
45         // 添加残差块
46         problem.AddResidualBlock(
47             // 使用自动求导, 残差维度=1, m维度=1, c维度=1
48             new ceres::AutoDiffCostFunction<ExponentialResidual, 1, 1, 1>(
49                 new ExponentialResidual(x_data[i], y_data[i])
50             ),
51             nullptr, // 不使用损失函数
52             &m, &c    // 参数块
53         );
54     }
55
56     // 配置求解器
57     ceres::Solver::Options options;
58     options.linear_solver_type = ceres::DENSE_QR;
59     options.minimizer_progress_to_stdout = true;
60
61     // 求解
62     ceres::Solver::Summary summary;
63     ceres::Solve(options, &problem, &summary);
64
65     // 输出结果
66     std::cout << summary.BriefReport() << std::endl;
67     std::cout << "真实值: m = " << m_true << ", c = " << c_true << std::endl;
68     std::cout << "估计值: m = " << m << ", c = " << c << std::endl;
69
70     return 0;
71 }

```

这个例子展示了 Ceres 的基本使用流程：定义残差函数、构建问题、添加残差块、配置求解器、求解。

1.5.26.3. 核心概念

理解 Ceres 的几个核心概念对于正确使用它至关重要。

残差块（Residual Block）是优化问题的基本单元。每个残差块包含一个代价函数和一组参数块。代价函数计算残差向量，Ceres 会最小化所有残差的平方和。

代价函数 (Cost Function) 定义了如何从参数计算残差。Ceres 支持三种方式定义代价函数:

```
1 // 方式 1: 自动求导 (推荐)
2 // Ceres 自动计算雅可比矩阵
3 struct AutoDiffCost {
4     template <typename T>
5     bool operator()(const T* const x, T* residual) const {
6         residual[0] = x[0] * x[0] - 10.0;
7         return true;
8     }
9 };
10
11 ceres::CostFunction* cost1 =
12     new ceres::AutoDiffCostFunction<AutoDiffCost, 1, 1>(new AutoDiffCost);
13
14 // 方式 2: 数值求导
15 // 使用有限差分近似雅可比
16 struct NumericCost {
17     bool operator()(const double* const x, double* residual) const {
18         residual[0] = x[0] * x[0] - 10.0;
19         return true;
20     }
21 };
22
23 ceres::CostFunction* cost2 =
24     new ceres::NumericDiffCostFunction<NumericCost, ceres::CENTRAL, 1, 1>(
25         new NumericCost);
26
27 // 方式 3: 解析求导
28 // 手动提供雅可比矩阵 (最快但最繁琐)
29 class AnalyticCost : public ceres::SizedCostFunction<1, 1> {
30 public:
31     bool Evaluate(double const* const* parameters,
32                 double* residuals,
33                 double* jacobians) const override {
34         double x = parameters[0][0];
35         residuals[0] = x * x - 10.0;
36
37         if (jacobians != nullptr && jacobians[0] != nullptr) {
38             jacobians[0][0] = 2.0 * x; // d(x^2 - 10) / dx = 2x
39         }
40         return true;
41     }
42 };
```

自动求导是最常用的方式, 它利用 C++ 模板和运算符重载, 在编译时生成求导代码, 精度高且效率接近手写。

参数块 (Parameter Block) 是优化变量的容器。一个参数块可以包含一个或多个标量:

```
1 double pose[6];           // 6 自由度位姿作为一个参数块
2 double point[3];          // 3D 点作为一个参数块
3 double intrinsics[4];     // 相机内参作为一个参数块
4
5 problem.AddResidualBlock(cost_function, nullptr, pose, point);
```

损失函数 (Loss Function) 用于降低异常值的影响。当数据中存在错误匹配或噪声异常值时, 平方损失会被严重影响。鲁棒损失函数可以减轻这种影响:

```
1 // 常用损失函数
```

```

2 ceres::LossFunction* huber = new ceres::HuberLoss(1.0);
3 ceres::LossFunction* cauchy = new ceres::CauchyLoss(0.5);
4 ceres::LossFunction* tukey = new ceres::TukeyLoss(1.0);
5
6 problem.AddResidualBlock(cost_function, huber, &x); // 使用 Huber 损失

```

1.5.26.4. 参数化与流形

某些参数有特殊的约束。例如，四元数必须是单位四元数，旋转矩阵必须是正交矩阵。Ceres 通过流形（Manifold，旧版本称为 LocalParameterization）来处理这些约束：

```

1 // 四元数参数化（保持单位长度）
2 double quaternion[4] = {1, 0, 0, 0}; // w, x, y, z
3
4 problem.AddParameterBlock(quaternion, 4);
5 problem.SetManifold(quaternion, new ceres::EigenQuaternionManifold());
6
7 // 或者使用 Sophus 库的 SE3 参数化
8 // problem.SetManifold(pose, new ceres::ProductManifold<
9 //     ceres::EigenQuaternionManifold,
10 //     ceres::EuclideanManifold<3>>());

```

固定参数：

```

1 // 固定某些参数不优化
2 problem.SetParameterBlockConstant(intrinsics);
3
4 // 固定参数块的部分维度
5 std::vector<int> constant_indices = {0, 1}; // 固定前两个维度
6 problem.SetManifold(pose,
7     new ceres::SubsetManifold(6, constant_indices));

```

参数边界：

```

1 // 设置参数上下界
2 problem.SetParameterLowerBound(&x, 0, 0.0); // x >= 0
3 problem.SetParameterUpperBound(&x, 0, 100.0); // x <= 100

```

1.5.26.5. 求解器配置

Ceres 提供了丰富的求解器选项：

```

1 ceres::Solver::Options options;
2
3 // 线性求解器类型
4 options.linear_solver_type = ceres::DENSE_QR; // 小规模稠密问题
5 options.linear_solver_type = ceres::DENSE_SCHUR; // BA 问题
6 options.linear_solver_type = ceres::SPARSE_SCHUR; // 大规模 BA
7 options.linear_solver_type = ceres::SPARSE_NORMAL_CHOLESKY; // 大规模稀疏
8
9 // 优化算法
10 options.trust_region_strategy_type = ceres::LEVENBERG_MARQUARDT; // LM 算法
11 options.trust_region_strategy_type = ceres::DOGLEGG; // Dogleg 算法
12
13 // 收敛条件
14 options.max_num_iterations = 100;
15 options.function_tolerance = 1e-6;
16 options.gradient_tolerance = 1e-10;
17 options.parameter_tolerance = 1e-8;

```

```

18
19 // 多线程
20 options.num_threads = 4;
21
22 // 输出
23 options.minimizer_progress_to_stdout = true;
24
25 // 求解
26 ceres::Solver::Summary summary;
27 ceres::Solve(options, &problem, &summary);
28
29 // 检查结果
30 if (summary.termination_type == ceres::CONVERGENCE) {
31     std::cout << "优化收敛" << std::endl;
32 }
33
34 std::cout << summary.FullReport() << std::endl;

```

1.5.26.6. RoboMaster 应用：相机标定

相机标定是确定相机内参（焦距、主点、畸变系数）的过程。使用棋盘格标定板，通过最小化重投影误差来估计参数：

```

1  #include <ceres/ceres.h>
2  #include <ceres/rotation.h>
3  #include <Eigen/Dense>
4
5  // 重投影误差
6  struct ReprojectionError {
7      ReprojectionError(double observed_x, double observed_y,
8                      double point_x, double point_y, double point_z)
9          : observed_x_(observed_x), observed_y_(observed_y),
10            point_x_(point_x), point_y_(point_y), point_z_(point_z) {}
11
12     template <typename T>
13     bool operator()(const T* const camera_intrinsics, // fx, fy, cx, cy
14                   const T* const distortion,          // k1, k2, p1, p2
15                   const T* const camera_pose,         // angle-axis (3) +
16                   translation (3)
17                   T* residuals) const {
18         // 3D 点
19         T point[3] = {T(point_x_), T(point_y_), T(point_z_)};
20
21         // 旋转
22         T rotated_point[3];
23         ceres::AngleAxisRotatePoint(camera_pose, point, rotated_point);
24
25         // 平移
26         T transformed_point[3];
27         transformed_point[0] = rotated_point[0] + camera_pose[3];
28         transformed_point[1] = rotated_point[1] + camera_pose[4];
29         transformed_point[2] = rotated_point[2] + camera_pose[5];
30
31         // 归一化相机坐标
32         T xn = transformed_point[0] / transformed_point[2];
33         T yn = transformed_point[1] / transformed_point[2];
34
35         // 畸变
36         T r2 = xn * xn + yn * yn;
37         T radial = T(1.0) + distortion[0] * r2 + distortion[1] * r2 * r2;
38         T xd = xn * radial + T(2.0) * distortion[2] * xn * yn +

```

```

38         distortion[3] * (r2 + T(2.0) * xn * xn);
39     T yd = yn * radial + distortion[2] * (r2 + T(2.0) * yn * yn) +
40         T(2.0) * distortion[3] * xn * yn;
41
42     // 像素坐标
43     T predicted_x = camera_intrinsics[0] * xd + camera_intrinsics[2];
44     T predicted_y = camera_intrinsics[1] * yd + camera_intrinsics[3];
45
46     // 残差
47     residuals[0] = predicted_x - T(observed_x_);
48     residuals[1] = predicted_y - T(observed_y_);
49
50     return true;
51 }
52
53 static ceres::CostFunction* Create(double observed_x, double
observed_y,
54                                   double point_x, double point_y,
double point_z) {
55     return new ceres::AutoDiffCostFunction<ReprojectionError, 2, 4,
4, 6>(
56         new ReprojectionError(observed_x, observed_y, point_x, point_y,
point_z));
57 }
58
59 private:
60     double observed_x_, observed_y_;
61     double point_x_, point_y_, point_z_;
62 };
63
64 class CameraCalibrator {
65 public:
66     void addObservation(int image_id,
67                        const Eigen::Vector2d& pixel,
68                        const Eigen::Vector3d& point_3d) {
69         observations_.push_back({image_id, pixel, point_3d});
70     }
71
72     bool calibrate() {
73         // 初始化内参猜测
74         intrinsics_[0] = 500; // fx
75         intrinsics_[1] = 500; // fy
76         intrinsics_[2] = 320; // cx
77         intrinsics_[3] = 240; // cy
78
79         // 初始化畸变系数
80         std::fill(distortion_, distortion_ + 4, 0.0);
81
82         // 初始化每张图像的位姿
83         int num_images = 0;
84         for (const auto& obs : observations_) {
85             num_images = std::max(num_images, obs.image_id + 1);
86         }
87         poses_.resize(num_images * 6, 0.0);
88
89         // 构建问题
90         ceres::Problem problem;
91
92         for (const auto& obs : observations_) {
93             ceres::CostFunction* cost = ReprojectionError::Create(
94                 obs.pixel.x(), obs.pixel.y(),
95                 obs.point_3d.x(), obs.point_3d.y(), obs.point_3d.z());
96

```

```

97         problem.AddResidualBlock(
98             cost,
99             new ceres::HuberLoss(1.0),
100             intrinsics_,
101             distortion_,
102             poses_.data() + obs.image_id * 6);
103     }
104
105     // 求解
106     ceres::Solver::Options options;
107     options.linear_solver_type = ceres::DENSE_SCHUR;
108     options.max_num_iterations = 100;
109     options.minimizer_progress_to_stdout = true;
110
111     ceres::Solver::Summary summary;
112     ceres::Solve(options, &problem, &summary);
113
114     std::cout << summary.BriefReport() << std::endl;
115     return summary.termination_type == ceres::CONVERGENCE;
116 }
117
118 Eigen::Matrix3d getCameraMatrix() const {
119     Eigen::Matrix3d K;
120     K << intrinsics_[0], 0, intrinsics_[2],
121         0, intrinsics_[1], intrinsics_[3],
122         0, 0, 1;
123     return K;
124 }
125
126 Eigen::Vector4d getDistortion() const {
127     return Eigen::Vector4d(distortion_[0], distortion_[1],
128                             distortion_[2], distortion_[3]);
129 }
130
131 private:
132     struct Observation {
133         int image_id;
134         Eigen::Vector2d pixel;
135         Eigen::Vector3d point_3d;
136     };
137
138     std::vector<Observation> observations_;
139     double intrinsics_[4];
140     double distortion_[4];
141     std::vector<double> poses_;
142 };

```

1.5.26.7. RoboMaster 应用：PnP 问题

PnP（Perspective-n-Point）问题是已知相机内参和 3D-2D 对应点，求解相机位姿：

```

1  struct PnPError {
2      PnPError(const Eigen::Vector2d& observed,
3              const Eigen::Vector3d& point_3d,
4              const Eigen::Matrix3d& K)
5          : observed_(observed), point_3d_(point_3d), K_(K) {}
6
7      template <typename T>
8      bool operator()(const T* const pose, T* residuals) const {
9          // pose: [qw, qx, qy, qz, tx, ty, tz]
10         Eigen::Quaternion<T> q(pose[0], pose[1], pose[2], pose[3]);
11         Eigen::Matrix<T, 3, 1> t(pose[4], pose[5], pose[6]);

```

```

12
13 // 3D 点转换
14 Eigen::Matrix<T, 3, 1> point;
15 point << T(point_3d.x()), T(point_3d.y()), T(point_3d.z());
16
17 Eigen::Matrix<T, 3, 1> p_cam = q * point + t;
18
19 // 投影
20 T fx = T(K_(0, 0));
21 T fy = T(K_(1, 1));
22 T cx = T(K_(0, 2));
23 T cy = T(K_(1, 2));
24
25 T u = fx * p_cam(0) / p_cam(2) + cx;
26 T v = fy * p_cam(1) / p_cam(2) + cy;
27
28 // 残差
29 residuals[0] = u - T(observed_.x());
30 residuals[1] = v - T(observed_.y());
31
32 return true;
33 }
34
35 private:
36 Eigen::Vector2d observed_;
37 Eigen::Vector3d point_3d_;
38 Eigen::Matrix3d K_;
39 };
40
41 class PnP Solver {
42 public:
43 PnP Solver(const Eigen::Matrix3d& K) : K_(K) {}
44
45 bool solve(const std::vector<Eigen::Vector3d>& points_3d,
46           const std::vector<Eigen::Vector2d>& points_2d,
47           Eigen::Quaterniond& q_out,
48           Eigen::Vector3d& t_out) {
49 // 初始位姿 (可以用 EPnP 等方法初始化)
50 double pose[7] = {1, 0, 0, 0, 0, 0, 1}; // qw, qx, qy, qz, tx,
ty, tz
51
52 ceres::Problem problem;
53
54 for (size_t i = 0; i < points_3d.size(); i++) {
55 ceres::CostFunction* cost =
56 new ceres::AutoDiffCostFunction<PnPError, 2, 7>(
57 new PnPError(points_2d[i], points_3d[i], K_));
58
59 problem.AddResidualBlock(cost, new ceres::HuberLoss(1.0), pose);
60 }
61
62 // 四元数参数化
63 problem.SetManifold(pose, new ceres::ProductManifold<
64 ceres::QuaternionManifold,
65 ceres::EuclideanManifold<3>>());
66
67 ceres::Solver::Options options;
68 options.linear_solver_type = ceres::DENSE_QR;
69 options.max_num_iterations = 50;
70
71 ceres::Solver::Summary summary;
72 ceres::Solve(options, &problem, &summary);
73

```

```

74         if (summary.termination_type == ceres::CONVERGENCE) {
75             q_out = Eigen::Quaterniond(pose[0], pose[1], pose[2], pose[3]);
76             t_out = Eigen::Vector3d(pose[4], pose[5], pose[6]);
77             return true;
78         }
79         return false;
80     }
81
82 private:
83     Eigen::Matrix3d K_;
84 };

```

1.5.26.8. RoboMaster 应用：弹道模型参数辨识

通过实际发射数据拟合弹道模型参数：

```

1  struct BallisticResidual {
2      BallisticResidual(double launch_angle, double measured_distance,
3                      double measured_drop, double initial_speed)
4          : launch_angle_(launch_angle),
5            measured_distance_(measured_distance),
6            measured_drop_(measured_drop), v0_(initial_speed) {}
7
8      template <typename T>
9      bool operator()(const T* const params, T* residuals) const {
10         // params: [drag_coefficient, lift_coefficient]
11         T drag = params[0];
12         T lift = params[1];
13
14         // 数值积分模拟弹道
15         T x = T(0), z = T(0);
16         T vx = T(v0_) * cos(T(launch_angle_));
17         T vz = T(v0_) * sin(T(launch_angle_));
18
19         T dt = T(0.0001);
20         T g = T(9.8);
21
22         for (int i = 0; i < 100000 && x < T(measured_distance_); i++) {
23             T v = sqrt(vx * vx + vz * vz);
24             T ax = -drag * v * vx;
25             T az = -g - drag * v * vz + lift * vx;
26
27             vx += ax * dt;
28             vz += az * dt;
29             x += vx * dt;
30             z += vz * dt;
31         }
32
33         // 残差：实际落点与模拟落点的差异
34         residuals[0] = z - T(measured_drop_);
35
36         return true;
37     }
38 private:
39     double launch_angle_;
40     double measured_distance_;
41     double measured_drop_;
42     double v0_;
43 };
44
45 class BallisticCalibrator {

```



```

46 public:
47     void addSample(double launch_angle, double distance,
48                   double drop, double speed) {
49         samples_.push_back({launch_angle, distance, drop, speed});
50     }
51
52     bool calibrate(double& drag_out, double& lift_out) {
53         double params[2] = {0.001, 0.0}; // 初始猜测
54
55         ceres::Problem problem;
56
57         for (const auto& s : samples_) {
58             problem.AddResidualBlock(
59                 new ceres::AutoDiffCostFunction<BallisticResidual, 1, 2>(
60                     new BallisticResidual(s.angle, s.distance, s.drop,
s.speed)),
61                 nullptr, params);
62         }
63
64         // 参数边界
65         problem.SetParameterLowerBound(params, 0, 0.0); // drag >= 0
66         problem.SetParameterUpperBound(params, 0, 0.1); // drag <= 0.1
67         problem.SetParameterLowerBound(params, 1, -0.01); // lift >= -0.01
68         problem.SetParameterUpperBound(params, 1, 0.01); // lift <= 0.01
69
70         ceres::Solver::Options options;
71         options.linear_solver_type = ceres::DENSE_QR;
72         options.max_num_iterations = 200;
73
74         ceres::Solver::Summary summary;
75         ceres::Solve(options, &problem, &summary);
76
77         drag_out = params[0];
78         lift_out = params[1];
79
80         return summary.termination_type == ceres::CONVERGENCE;
81     }
82
83 private:
84     struct Sample {
85         double angle, distance, drop, speed;
86     };
87     std::vector<Sample> samples_;
88 };

```

1.5.26.9. RoboMaster 应用：IMU 内参标定

IMU 存在零偏 (bias)、刻度因子 (scale) 和轴间不正交等误差：

```

1 struct IMUCalibrationResidual {
2     IMUCalibrationResidual(const Eigen::Vector3d& measured_acc,
3                           const Eigen::Vector3d& true_gravity)
4         : measured_(measured_acc), gravity_(true_gravity) {}
5
6     template <typename T>
7     bool operator()(const T* const bias, // 3 个零偏
8                    const T* const scale, // 3 个刻度因子
9                    const T* const misalign, // 3 个轴间不正交角
10                    T* residuals) const {
11         // 构建校正矩阵 (下三角)
12         Eigen::Matrix<T, 3, 3> M;
13         M << scale[0], T(0), T(0),

```

```

14         misalign[0], scale[1], T(0),
15         misalign[1], misalign[2], scale[2];
16
17     // 校正后的测量值
18     Eigen::Matrix<T, 3, 1> measured;
19     measured << T(measured_.x()), T(measured_.y()), T(measured_.z());
20
21     Eigen::Matrix<T, 3, 1> b;
22     b << bias[0], bias[1], bias[2];
23
24     Eigen::Matrix<T, 3, 1> corrected = M * (measured - b);
25
26     // 静止时应该只测量到重力
27     Eigen::Matrix<T, 3, 1> g;
28     g << T(gravity_.x()), T(gravity_.y()), T(gravity_.z());
29
30     Eigen::Matrix<T, 3, 1> error = corrected - g;
31
32     residuals[0] = error(0);
33     residuals[1] = error(1);
34     residuals[2] = error(2);
35
36     return true;
37 }
38
39 private:
40     Eigen::Vector3d measured_;
41     Eigen::Vector3d gravity_;
42 };
43
44 class IMUCalibrator {
45 public:
46     // 添加静止状态下不同姿态的测量
47     void addStaticMeasurement(const Eigen::Vector3d& acc,
48                             const Eigen::Vector3d& expected_gravity) {
49         measurements_.push_back({acc, expected_gravity});
50     }
51
52     bool calibrate() {
53         // 初始参数
54         std::fill(bias_, bias_ + 3, 0.0);
55         scale_[0] = scale_[1] = scale_[2] = 1.0;
56         std::fill(misalign_, misalign_ + 3, 0.0);
57
58         ceres::Problem problem;
59
60         for (const auto& m : measurements_) {
61             problem.AddResidualBlock(
62                 new ceres::AutoDiffCostFunction<IMUCalibrationResidual, 3,
63                 3, 3, 3>(
64                     new IMUCalibrationResidual(m.first, m.second)),
65                 nullptr, bias_, scale_, misalign_);
66         }
67
68         // 刻度因子应该接近 1
69         for (int i = 0; i < 3; i++) {
70             problem.SetParameterLowerBound(scale_, i, 0.9);
71             problem.SetParameterUpperBound(scale_, i, 1.1);
72         }
73
74         ceres::Solver::Options options;
75         options.linear_solver_type = ceres::DENSE_QR;
76         options.max_num_iterations = 100;

```

```

76
77     ceres::Solver::Summary summary;
78     ceres::Solve(options, &problem, &summary);
79
80     std::cout << "IMU 标定结果:" << std::endl;
81     std::cout << "零偏: " << bias_[0] << ", " << bias_[1] << ", " <<
bias_[2] << std::endl;
82     std::cout << "刻度: " << scale_[0] << ", " << scale_[1] << ", " <<
scale_[2] << std::endl;
83
84     return summary.termination_type == ceres::CONVERGENCE;
85 }
86
87 Eigen::Vector3d correct(const Eigen::Vector3d& raw) const {
88     Eigen::Matrix3d M;
89     M << scale_[0], 0, 0,
90         misalign_[0], scale_[1], 0,
91         misalign_[1], misalign_[2], scale_[2];
92
93     Eigen::Vector3d b(bias_[0], bias_[1], bias_[2]);
94     return M * (raw - b);
95 }
96
97 private:
98     std::vector<std::pair<Eigen::Vector3d, Eigen::Vector3d>> measurements_;
99     double bias_[3];
100    double scale_[3];
101    double misalign_[3];
102 };

```

1.5.26.10. RoboMaster 应用：手眼标定

手眼标定确定相机与机械臂末端（或云台）之间的固定变换：

```

1 // AX = XB 问题
2 // A: 相机运动, B: 云台运动, X: 相机到云台的变换
3 struct HandEyeResidual {
4     HandEyeResidual(const Eigen::Isometry3d& A, const Eigen::Isometry3d& B)
5         : A_(A), B_(B) {}
6
7     template <typename T>
8     bool operator()(const T* const x_quat, // 4: 四元数
9                     const T* const x_trans, // 3: 平移
10                     T* residuals) const {
11         // X 的旋转和平移
12         Eigen::Quaternion<T> q_x(x_quat[0], x_quat[1], x_quat[2],
x_quat[3]);
13         Eigen::Matrix<T, 3, 1> t_x(x_trans[0], x_trans[1], x_trans[2]);
14
15         // A 的旋转和平移
16         Eigen::Quaternion<T> q_a = A_.rotation().cast<T>();
17         Eigen::Matrix<T, 3, 1> t_a = A_.translation().cast<T>();
18
19         // B 的旋转和平移
20         Eigen::Quaternion<T> q_b = B_.rotation().cast<T>();
21         Eigen::Matrix<T, 3, 1> t_b = B_.translation().cast<T>();
22
23         // AX 的旋转和平移
24         Eigen::Quaternion<T> q_ax = q_a * q_x;
25         Eigen::Matrix<T, 3, 1> t_ax = q_a * t_x + t_a;
26
27         // XB 的旋转和平移

```

```

28     Eigen::Quaternion<T> q_xb = q_x * q_b;
29     Eigen::Matrix<T, 3, 1> t_xb = q_x * t_b + t_x;
30
31     // 旋转误差 (四元数差)
32     Eigen::Quaternion<T> q_err = q_ax * q_xb.inverse();
33     residuals[0] = T(2.0) * q_err.x();
34     residuals[1] = T(2.0) * q_err.y();
35     residuals[2] = T(2.0) * q_err.z();
36
37     // 平移误差
38     Eigen::Matrix<T, 3, 1> t_err = t_ax - t_xb;
39     residuals[3] = t_err(0);
40     residuals[4] = t_err(1);
41     residuals[5] = t_err(2);
42
43     return true;
44 }
45
46 private:
47     Eigen::Isometry3d A_, B_;
48 };
49
50 class HandEyeCalibrator {
51 public:
52     void addMotionPair(const Eigen::Isometry3d& camera_motion,
53                       const Eigen::Isometry3d& gimbal_motion) {
54         motion_pairs_.push_back({camera_motion, gimbal_motion});
55     }
56
57     bool calibrate(Eigen::Isometry3d& X_out) {
58         // 初始化
59         double x_quat[4] = {1, 0, 0, 0};
60         double x_trans[3] = {0, 0, 0};
61
62         ceres::Problem problem;
63
64         for (const auto& [A, B] : motion_pairs_) {
65             problem.AddResidualBlock(
66                 new ceres::AutoDiffCostFunction<HandEyeResidual, 6, 4, 3>(
67                     new HandEyeResidual(A, B)),
68                 nullptr, x_quat, x_trans);
69         }
70
71         problem.SetManifold(x_quat, new ceres::QuaternionManifold());
72
73         ceres::Solver::Options options;
74         options.linear_solver_type = ceres::DENSE_QR;
75         options.max_num_iterations = 100;
76
77         ceres::Solver::Summary summary;
78         ceres::Solve(options, &problem, &summary);
79
80         if (summary.termination_type == ceres::CONVERGENCE) {
81             Eigen::Quaterniond q(x_quat[0], x_quat[1], x_quat[2],
x_quat[3]);
82             X_out = Eigen::Isometry3d::Identity();
83             X_out.rotate(q);
84             X_out.pretranslate(Eigen::Vector3d(x_trans[0], x_trans[1],
x_trans[2]));
85             return true;
86         }
87         return false;
88     }

```

```

89
90 private:
91     std::vector<std::pair<Eigen::Isometry3d, Eigen::Isometry3d>>
        motion_pairs_;
92 };

```

1.5.26.11. 调试与常见问题

检查雅可比矩阵：

```

1 // 使用数值方法验证自动求导的正确性
2 ceres::Problem problem;
3 // ... 添加残差块 ...
4
5 ceres::NumericDiffOptions numeric_diff_options;
6 std::vector<const ceres::Manifold*> manifolds;
7
8 // 获取参数块
9 std::vector<double*> parameter_blocks;
10 problem.GetParameterBlocks(&parameter_blocks);
11
12 // 检查每个残差块的雅可比
13 // 如果自动求导实现有误，这里会报错

```

处理优化不收敛：

```

1 // 1. 检查初值是否合理
2 // 2. 检查残差函数是否正确
3 // 3. 尝试不同的求解器
4 options.linear_solver_type = ceres::DENSE_QR; // 或 DENSE_SCHUR
5
6 // 4. 调整收敛条件
7 options.function_tolerance = 1e-4; // 放宽
8
9 // 5. 增加迭代次数
10 options.max_num_iterations = 500;
11
12 // 6. 检查是否有数值问题
13 options.check_gradients = true; // 调试时启用
14 options.gradient_check_relative_precision = 1e-4;
15
16 // 7. 使用损失函数处理异常值
17 problem.AddResidualBlock(cost, new ceres::HuberLoss(1.0), params);

```

性能优化：

```

1 // 1. 优先使用自动求导，它已经很快
2 // 2. 对于大规模问题，使用稀疏求解器
3 options.linear_solver_type = ceres::SPARSE_NORMAL_CHOLESKY;
4
5 // 3. 启用多线程
6 options.num_threads = std::thread::hardware_concurrency();
7
8 // 4. 对于 BA 问题，使用 Schur 消元
9 options.linear_solver_type = ceres::SPARSE_SCHUR;
10
11 // 5. 如果某些参数不需要优化，将其设为常量
12 problem.SetParameterBlockConstant(fixed_params);
13
14 // 6. 利用问题结构
15 ceres::Problem::Options problem_options;

```

```
16 problem_options.cost_function_ownership = ceres::DO_NOT_TAKE_OWNERSHIP;  
17 problem_options.loss_function_ownership = ceres::DO_NOT_TAKE_OWNERSHIP;
```

Ceres 是机器人开发中处理优化问题的利器。它强大的自动求导功能让你可以快速实现复杂的代价函数，而不必担心求导错误。在 RoboMaster 开发中，从相机标定到状态估计，从参数辨识到 SLAM 后端，Ceres 都能发挥重要作用。熟练掌握 Ceres，将大大提升你解决实际工程问题的能力。

1.5.27. OpenCV 基础

1.5.28. 串口通信

至此，我们已经完成了 C++ 核心特性的学习。从基础的变量和函数，到面向对象编程，再到模板、智能指针和移动语义，这些知识构成了编写现代 C++ 代码的基础。在实际的 RoboMaster 项目中，你将综合运用这些技术来构建高效、可靠的机器人控制系统。

1.6. CMake 与构建系统

1.6.1. 为什么需要构建系统

学习 C++ 的第一天，你可能写下了这样一行命令：`g++ main.cpp -o main`。编译器读取源文件，生成可执行文件，程序运行，输出“Hello, World!”。一切都很简单直接。但随着项目规模的增长，这种简单会迅速消失。当源文件从一个变成十个、一百个，当项目开始依赖外部库，当团队成员使用不同的操作系统，手动编译就变成了一场噩梦。构建系统正是为了解决这些问题而诞生的工具，它是每个 C++ 开发者必须掌握的技能。

1.6.1.1. 从一个文件到一百个文件

让我们从最简单的情况开始。一个只有 `main.cpp` 的项目，编译命令是：

```
1 g++ main.cpp -o main
```

当你把代码拆分成两个文件——`main.cpp` 和 `utils.cpp`——命令变成：

```
1 g++ main.cpp utils.cpp -o main
```

这还能接受。但 RoboMaster 的视觉系统可能有几十个源文件：检测器、跟踪器、预测器、通信模块、工具函数……命令会变成这样：

```
1 g++ main.cpp detector.cpp tracker.cpp predictor.cpp protocol.cpp serial.cpp \  
2 config.cpp logger.cpp math_utils.cpp image_utils.cpp coordinate.cpp \  
3 kalman_filter.cpp armor.cpp rune.cpp ... -o rm_vision \  
4 -I/usr/include/opencv4 -I/usr/include/eigen3 \  
5 -lopencv_core -lopencv_imgproc -lopencv_highgui -lceres -lglog \  
6 -std=c++17 -O2 -Wall
```

这行命令已经长得难以阅读，而且每次编译都要输入一遍。你可能会想到把它保存成一个 shell 脚本，这确实是一个进步，但问题远不止于此。

更大的问题是编译效率。假设你的项目有 100 个源文件，完整编译需要 5 分钟。现在你只修改了 `tracker.cpp` 中的一行代码，难道需要重新编译全部 100 个文件吗？显然不需要——只有 `tracker.cpp` 需要重新编译，然后重新链接即可。但上面那行命令无法做到这一点，它会把所有文件从头编译一遍。在大型项目中，这种浪费是不可接受的。

要实现“只编译修改过的文件”，你需要分开编译和链接两个步骤。先把每个源文件编译成目标文件（.o），再把目标文件链接成可执行文件：

```
1 g++ -c main.cpp -o main.o
2 g++ -c detector.cpp -o detector.o
3 g++ -c tracker.cpp -o tracker.o
4 # ... 对每个源文件重复 ...
5 g++ main.o detector.o tracker.o ... -o rm_vision -lopencv_core ...
```

这样，当你修改了 `tracker.cpp`，只需要重新编译 `tracker.o`，然后重新链接。但现在你需要判断哪些文件被修改了、哪些目标文件需要更新。如果 `tracker.cpp` 包含了 `config.h`，而你修改了 `config.h`，那 `tracker.o` 也需要重新编译。如果 `detector.cpp` 和 `predictor.cpp` 也包含了 `config.h`，它们都需要重新编译。

这种依赖关系会变得非常复杂。一个头文件可能被几十个源文件包含，修改它会触发大量重新编译。手动追踪这些依赖关系是不现实的——这正是构建系统要解决的核心问题之一。

1.6.1.2. 依赖分析与增量编译

构建系统的首要职责是依赖分析。它会分析源文件之间的依赖关系：哪个源文件包含了哪些头文件，哪个目标文件依赖于哪些源文件，最终的可执行文件依赖于哪些目标文件和库。这些依赖关系形成一个有向无环图（DAG），构建系统根据这个图来决定构建顺序和需要更新的目标。

当你修改一个文件并请求构建时，构建系统会检查依赖图中哪些目标受到影响。它比较源文件和目标文件的修改时间：如果源文件比目标文件新，说明目标文件需要更新。这个过程会沿着依赖图传播——如果 `tracker.o` 需要更新，而可执行文件依赖于 `tracker.o`，那么可执行文件也需要重新链接。

这种“只重新构建必要部分”的策略叫做增量编译（incremental build）。在大型项目中，增量编译可以把构建时间从几分钟缩短到几秒钟，极大地提高了开发效率。每次修改代码后，你几乎可以立即看到结果，这对于快速迭代至关重要。

依赖分析还有另一个好处：并行编译。当构建系统知道了完整的依赖图，它就知道哪些目标之间没有依赖关系，可以同时编译。在多核 CPU 上，并行编译可以成倍地缩短构建时间。一个单线程需要 10 分钟的构建，在 8 核机器上可能只需要不到 2 分钟。

1.6.1.3. 跨平台的挑战

如果你的代码只在自己的 Ubuntu 笔记本上运行，单一平台的构建脚本或许够用。但现实往往更复杂：团队中有人用 macOS，有人用 Windows；代码可能需要部署到 Jetson 等嵌入式平台；CI 服务器可能运行着不同版本的 Linux。每个平台都有自己的编译器、路径约定、库命名规则。

在 Linux 上，C++ 编译器通常是 `g++`，库文件以 `.so` 结尾，头文件可能在 `/usr/include` 或 `/usr/local/include`。在 macOS 上，默认编译器是 `clang++`，动态库以 `.dylib` 结尾，路径规则也不同。在 Windows 上，编译器可能是 MSVC 的 `cl.exe`，库文件是 `.dll` 和 `.lib`，路径使用反斜杠，还有很多 Windows 特有的编译选项。

手动维护每个平台的构建脚本是一项繁重的工作，而且容易出错。你需要在不同平台上测试，确保脚本正确处理了所有差异。任何改动都可能在某个平台上引入问题。

好的构建系统提供跨平台抽象。你用统一的方式描述项目结构、依赖关系和编译选项，构建系统负责翻译成每个平台上的具体命令。你写一份构建配置，它在所有支持的平台上都能工作。这不仅节省了维护成本，还减少了“在我机器上能编译”这类问题。

1.6.1.4. Make：元老级构建工具

Make 是最早的构建工具之一，诞生于 1976 年。在接近半个世纪后的今天，它仍然被广泛使用，足见其设计的生命力。Make 使用名为 `Makefile` 的文件来描述构建规则。

一个简单的 Makefile 看起来像这样：

```
1 CC = g++
2 CFLAGS = -std=c++17 -Wall -O2
3
4 main: main.o utils.o
5     $(CC) main.o utils.o -o main
6
7 main.o: main.cpp utils.h
8     $(CC) $(CFLAGS) -c main.cpp -o main.o
9
10 utils.o: utils.cpp utils.h
11     $(CC) $(CFLAGS) -c utils.cpp -o utils.o
12
13 clean:
14     rm -f *.o main
```

每条规则包含三部分：目标（target）、依赖（dependencies）和命令（commands）。`main.o: main.cpp utils.h` 表示 `main.o` 依赖于 `main.cpp` 和 `utils.h`，如果这两个文件中的任何一个比 `main.o` 新，就执行下面的命令重新编译。

Make 解决了依赖分析和增量编译的问题，在当时是巨大的进步。但随着项目规模增长，Makefile 的局限性逐渐显现。

首先，Makefile 的语法相当晦涩。制表符和空格的区别、变量展开的规则、隐式规则的行为——这些细节让很多人望而却步。一个语法错误可能导致难以理解的构建失败。

其次，跨平台支持有限。Makefile 本质上是在调用 shell 命令，而不同平台的 shell 和工具链差异很大。虽然可以用条件语句处理这些差异，但 Makefile 会变得越来越复杂。

第三，发现和链接外部库需要手动处理。你需要知道每个库的头文件路径、库文件路径、链接选项，并正确地写入 Makefile。当依赖的库更新或安装位置变化时，Makefile 也需要相应修改。

最后，大型项目的 Makefile 难以维护。当项目有数百个源文件、多个子目录、复杂的依赖关系时，Makefile 会变得庞大且难以理解。任何修改都要小心翼翼，担心破坏其他部分。

1.6.1.5. CMake：现代 C++ 的事实标准

CMake 诞生于 2000 年，最初是为了解决一个跨平台医学图像处理软件的构建问题。它的名字是“Cross-platform Make”的缩写，顾名思义，它的目标是提供跨平台的构建解决方案。

CMake 不直接构建项目，而是生成原生构建系统的配置文件。在 Linux 上，它默认生成 Makefile；在 Windows 上，可以生成 Visual Studio 项目文件或 Ninja 构建文件；在 macOS 上，可以生成 Xcode 项目或 Makefile。这种“元构建系统”的设计让 CMake 可以利用每个平台上最成熟的原生工具，同时为用户提供统一的接口。

CMake 使用名为 `CMakeLists.txt` 的文件来描述项目。与 Makefile 相比，CMake 的语法更加直观：

```
1 cmake_minimum_required(VERSION 3.16)
2 project(rm_vision)
3
4 set(CMAKE_CXX_STANDARD 17)
5
```



```

6 find_package(OpenCV REQUIRED)
7 find_package(Eigen3 REQUIRED)
8
9 add_executable(rm_vision
10     src/main.cpp
11     src/detector.cpp
12     src/tracker.cpp
13 )
14
15 target_include_directories(rm_vision PRIVATE include)
16 target_link_libraries(rm_vision OpenCV::OpenCV Eigen3::Eigen)

```

这段代码清晰地表达了：项目名称、C++ 标准版本、需要的外部库、源文件列表、头文件路径、链接依赖。`find_package` 会自动查找已安装的库，无需手动指定路径。`target_link_libraries` 不仅链接库文件，还会自动添加库的头文件路径和编译选项。

如今，CMake 已经成为 C++ 项目的事实标准。几乎所有主流的 C++ 库都提供 CMake 支持：OpenCV、Eigen、Ceres、PCL、gtest……ROS 2 的构建系统 ament 也是基于 CMake 的。学会 CMake，你就能构建几乎任何 C++ 项目，也能让你的项目被其他人轻松使用。

在接下来的章节中，我们将深入学习 CMake 的语法和用法。从最基本的项目配置开始，逐步涵盖多文件组织、外部库链接、安装导出，直到 ROS 2 包的构建。掌握这些知识后，你将能够为任何规模的 C++ 项目设计清晰、可维护的构建系统。

1.6.2. 编译原理回顾

在前面的“程序的执行：从源码到运行”章节中，我们已经介绍过编译的基本流程。现在我们从构建系统的角度重新审视这个过程，深入理解编译的每个阶段产生什么、依赖什么，以及构建系统如何利用这些知识来优化构建过程。理解这些原理，你才能真正明白 CMake 配置背后的含义，也能在遇到编译错误时快速定位问题。

1.6.2.1. 四个阶段的再认识

C++ 程序的构建分为四个阶段：预处理、编译、汇编和链接。虽然我们通常用一条命令完成整个过程，但理解每个阶段的输入输出对于理解构建系统至关重要。

预处理阶段处理所有以 `#` 开头的指令。`#include` 将头文件的内容逐字插入到源文件中，`#define` 进行文本替换，`#ifdef` 等条件编译指令决定哪些代码被保留。预处理的输入是源文件（.cpp）和它包含的所有头文件（.h/.hpp），输出是一个展开后的纯 C++ 代码文件。这个输出通常不保存为文件，而是直接传递给编译器，但你可以用 `g++ -E` 查看预处理结果。

```

1 # 查看预处理结果
2 g++ -E main.cpp -o main.i
3
4 # main.i 可能有数万行，包含了所有展开的头文件
5 wc -l main.i
6 # 输出可能是 50000 行甚至更多

```

编译阶段将预处理后的 C++ 代码转换为汇编代码。这是最复杂的阶段，编译器要进行词法分析、语法分析、语义检查、优化等工作。编译器在这个阶段检查语法错误和类型错误，这就是为什么我们把这类错误称为“编译错误”。编译的输出是汇编代码文件（.s）。

```

1 # 查看编译生成的汇编代码
2 g++ -S main.cpp -o main.s

```

汇编阶段将汇编代码转换为机器码，生成目标文件（object file, .o）。汇编器的工作相对简单，基本上是汇编指令到机器指令的一一对应。目标文件包含了机器码，但还不能直接执行——函数调用和全局变量引用的地址还没有确定。

```
1 # 生成目标文件
2 g++ -c main.cpp -o main.o
```

链接阶段将多个目标文件和库文件合并成最终的可执行文件或库。链接器的核心工作是符号解析（找到每个符号引用的定义）和重定位（计算并填入最终地址）。链接的输出是可执行文件或库文件。

```
1 # 链接生成可执行文件
2 g++ main.o utils.o -o main
```

对于构建系统来说，最重要的是理解这四个阶段的依赖关系。预处理依赖于源文件和所有直接或间接包含的头文件；编译依赖于预处理的结果；汇编依赖于编译的结果；链接依赖于所有目标文件和库文件。构建系统通过追踪这些依赖，判断哪些步骤需要重新执行。

1.6.2.2. 编译单元

编译单元（Translation Unit）是 C++ 编译的基本单位。简单来说，一个编译单元就是一个源文件（.cpp）经过预处理后的结果——包括源文件本身的代码，加上所有被 `#include` 进来的头文件内容。

每个编译单元独立编译，生成一个目标文件。编译器在编译一个编译单元时，对其他编译单元一无所知。它只能看到当前编译单元中的代码，无法访问其他源文件中定义的函数或变量。这种隔离是有意为之的——它使得编译可以并行进行，也使得增量编译成为可能。

```
1 // math_utils.cpp - 编译单元 1
2 #include "math_utils.h"
3
4 double Add(double a, double b) {
5     return a + b;
6 }
7
8 // main.cpp - 编译单元 2
9 #include "math_utils.h"
10
11 int main() {
12     double result = Add(1.0, 2.0); // 调用在另一个编译单元中定义的函数
13     return 0;
14 }
```

当编译 `main.cpp` 时，编译器看到 `Add(1.0, 2.0)` 这个调用。它从 `math_utils.h` 中知道 `Add` 函数的声明（参数类型和返回类型），可以检查调用是否正确，但它不知道 `Add` 函数的实现在哪里。编译器生成的目标文件中，对 `Add` 的调用被标记为“未解析的外部符号”，等待链接器来填入实际地址。

理解编译单元的独立性有几个重要意义。首先，它解释了为什么修改一个 .cpp 文件只需要重新编译这一个文件——其他编译单元不受影响。其次，它解释了为什么头文件中通常只放声明而不放定义——如果把函数定义放在头文件中，每个包含这个头文件的编译单元都会有一份定义，链接时就会出现“多重定义”错误。最后，它解释了为什么链接器是必要的——编译器无法跨编译单元解析符号引用。

1.6.2.3. 头文件与源文件的分工

C++ 中头文件 (.h/.hpp) 和源文件 (.cpp) 有着明确的分工, 理解这种分工对于组织项目结构非常重要。

头文件的职责是提供接口声明。它告诉其他编译单元“有什么可以用”, 但不提供实现细节。头文件通常包含: 类的定义(成员函数的声明, 不是实现)、函数声明、类型定义(typedef、using、enum)、常量声明(extern const)、模板的定义(模板比较特殊, 稍后讨论)、宏定义和内联函数。

```
1 // robot.h - 头文件
2 #pragma once
3
4 #include <string>
5
6 // 类定义 (包含成员声明)
7 class Robot {
8 public:
9     Robot(const std::string& name);
10    void Move(double x, double y);
11    std::string GetName() const;
12
13 private:
14    std::string name_;
15    double x_, y_;
16 };
17
18 // 函数声明
19 void InitializeSystem();
20
21 // 常量声明
22 extern const double kMaxSpeed;
23
24 // 内联函数 (可以在头文件中定义)
25 inline double Square(double x) {
26     return x * x;
27 }
```

源文件的职责是提供实现。它定义了头文件中声明的函数和变量, 包含了具体的逻辑代码。源文件通常包含: 成员函数的实现、普通函数的实现、全局变量的定义、静态成员变量的定义。

```
1 // robot.cpp - 源文件
2 #include "robot.h"
3 #include <iostream>
4
5 // 常量定义
6 const double kMaxSpeed = 10.0;
7
8 // 构造函数实现
9 Robot::Robot(const std::string& name)
10     : name_(name), x_(0), y_(0) {}
11
12 // 成员函数实现
13 void Robot::Move(double x, double y) {
14     x_ += x;
15     y_ += y;
16     std::cout << name_ << " moved to (" << x_ << ", " << y_ << ")" <<
17     std::endl;
18 }
19
20 std::string Robot::GetName() const {
```

```

20     return name_;
21 }
22
23 // 普通函数实现
24 void InitializeSystem() {
25     std::cout << "System initialized" << std::endl;
26 }

```

这种分离有几个重要的好处。首先，它实现了接口与实现的分离。使用者只需要看头文件就知道如何使用，不需要关心实现细节。其次，它支持增量编译。修改实现（.cpp）只需要重新编译这一个文件；只有修改接口（.h）才需要重新编译所有使用者。第三，它减少了编译时间。如果把实现放在头文件中，每个包含它的编译单元都要编译这些实现代码。

然而，有些情况下代码必须放在头文件中。模板是最典型的例子——模板在实例化时需要看到完整的定义，而实例化发生在每个使用模板的编译单元中，因此模板的定义通常放在头文件里。内联函数也类似，因为编译器需要在调用点展开内联函数，必须能看到函数体。

```

1 // 模板必须在头文件中定义
2 template <typename T>
3 class Stack {
4 public:
5     void Push(const T& value) {
6         data_.push_back(value);
7     }
8
9     T Pop() {
10         T value = data_.back();
11         data_.pop_back();
12         return value;
13     }
14
15 private:
16     std::vector<T> data_;
17 };

```

1.6.2.4. 目标文件与符号表

目标文件是编译的直接产物，理解它的结构有助于理解链接过程和常见的链接错误。

目标文件包含几个主要部分。代码段（.text）存储编译后的机器指令。数据段（.data）存储已初始化的全局变量和静态变量。BSS 段（.bss）存储未初始化的全局变量和静态变量（只记录大小，不占用文件空间）。符号表记录了这个编译单元定义和引用的所有符号。重定位表记录了需要在链接时修正的位置。

符号（symbol）是链接器的核心概念。函数名、全局变量名、静态变量名都是符号。符号分为两类：定义的符号（这个编译单元提供了实现）和引用的符号（这个编译单元使用了，但实现在别处）。

```

1 # 查看目标文件的符号表
2 nm main.o
3
4 # 输出示例：
5 #                U _Z3Adddd          # U 表示未定义（引用）
6 # 0000000000000000 T main              # T 表示在代码段定义
7 #                U printf             # 引用了 printf

```

符号表中的常见标记：

- **T**：在代码段中定义（函数）
- **D**：在数据段中定义（已初始化的全局变量）
- **B**：在 BSS 段中定义（未初始化的全局变量）
- **U**：未定义（引用了但未定义，需要链接器解析）
- **W**：弱符号（可以被其他定义覆盖）

你可能注意到符号名看起来很奇怪，如 `_Z3Adddd` 而不是简单的 `Add`。这是因为 C++ 支持函数重载，同名但参数不同的函数需要不同的符号名来区分。编译器通过名称修饰（name mangling）将函数名、参数类型等信息编码到符号名中。不同编译器的修饰规则可能不同，这就是为什么混用不同编译器编译的目标文件可能出问题。

```
1 # 查看解码后的符号名
2 nm -C main.o
3
4 # 输出:
5 #                U Add(double, double)
6 # 0000000000000000 T main
```

1.6.2.5. 链接：符号解析与重定位

链接器的工作是将多个目标文件和库文件合并成一个可执行文件（或库）。这个过程包含两个核心任务：符号解析和重定位。

符号解析（symbol resolution）是将符号引用与符号定义匹配的过程。链接器扫描所有输入的目标文件，收集它们定义的符号和引用的符号。对于每个引用的符号，链接器需要找到唯一的定义。如果找不到定义，就会报“undefined reference”错误；如果找到多个定义，就会报“multiple definition”错误。

```
1 # 未定义符号错误
2 g++ main.o -o main
3 # error: undefined reference to `Add(double, double)'
4
5 # 需要链接包含 Add 定义的目标文件
6 g++ main.o math_utils.o -o main
```

“undefined reference”是最常见的链接错误之一。它意味着你使用了某个函数或变量，但链接器在所有提供的目标文件和库中都找不到它的定义。常见原因包括：忘记链接某个库、忘记编译某个源文件、函数声明与定义不匹配（特别是参数类型）、C 和 C++ 混合编程时忘记 `extern "C"`。

“multiple definition”错误意味着同一个符号在多个地方被定义。常见原因是把函数定义放在了头文件中（而这个头文件被多个源文件包含），或者同一个全局变量在多个源文件中定义。

重定位（relocation）是计算并填入符号最终地址的过程。在编译时，编译器不知道函数和变量的最终地址，只能在调用处留下占位符，并在重定位表中记录这些位置。链接器确定了每个符号的最终地址后，遍历重定位表，把占位符替换为实际地址。

链接顺序有时候很重要。链接器通常从左到右处理输入文件，当处理一个库时，只会取出能解析当前未定义符号的目标文件。如果 A 依赖 B，B 依赖 C，链接顺序应该是 **A B C**。顺序错误可能导致符号找不到。虽然现代链接器越来越智能，但在某些情况下顺序仍然重要。

```
1 # 链接顺序可能影响结果
2 g++ main.o -lmath -lbase # 如果 math 依赖 base 中的符号，这个顺序是对的
3 g++ main.o -lbase -lmath # 可能会出问题
```

1.6.2.6. 静态库与共享库

当你的项目使用外部代码时，有两种链接方式：静态链接和动态链接。理解它们的区别对于配置构建系统很重要。

静态库（static library）是目标文件的打包集合。在 Linux 上扩展名是 `.a`（archive），在 Windows 上是 `.lib`。静态库本质上就是把多个 `.o` 文件打包到一起，方便分发和链接。链接静态库时，链接器会从中提取需要的目标文件，把它们的代码复制到最终的可执行文件中。

```
1 # 创建静态库
2 ar rcs libmath.a add.o subtract.o multiply.o divide.o
3
4 # 链接静态库
5 g++ main.o -L. -lmath -o main
6 # 或者直接指定库文件
7 g++ main.o libmath.a -o main
```

静态链接的优点是生成的可执行文件是自包含的，不依赖外部库文件，可以直接复制到其他机器上运行。缺点是可执行文件体积较大（包含了库代码的副本），如果多个程序使用同一个库，每个程序都有一份副本，浪费磁盘空间。更重要的是，当库需要更新（如修复安全漏洞）时，所有静态链接的程序都需要重新编译。

共享库（shared library，也叫动态库）在 Linux 上扩展名是 `.so`（shared object），在 Windows 上是 `.dll`。共享库的代码不会被复制到可执行文件中，而是在程序运行时由操作系统加载。多个程序可以共享同一个库文件，节省磁盘和内存空间。

```
1 # 创建共享库
2 g++ -shared -fPIC -o libmath.so add.cpp subtract.cpp multiply.cpp divide.cpp
3
4 # 链接共享库
5 g++ main.cpp -L. -lmath -o main
6
7 # 运行时需要能找到共享库
8 export LD_LIBRARY_PATH=./:$LD_LIBRARY_PATH
9 ./main
```

`-fPIC` 选项生成位置无关代码（Position Independent Code），这是共享库必需的，因为共享库会被加载到不同进程的不同地址。

共享库的优点是节省空间、便于更新（更新库文件后所有使用它的程序都会受益）。缺点是程序运行时依赖库文件的存在，如果库文件缺失或版本不兼容，程序无法运行。运行时加载库也会有一些性能开销（虽然通常很小）。

```
1 # 查看可执行文件依赖的共享库
2 ldd ./main
3
4 # 输出示例：
5 # linux-vdso.so.1
6 # libmath.so => ./libmath.so
7 # libstdc++.so.6 => /usr/lib/x86_64-linux-gnu/libstdc++.so.6
8 # libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
```

在 CMake 中，使用 `add_library` 创建库时可以指定类型：

```
1 # 静态库
2 add_library(mylib STATIC src1.cpp src2.cpp)
3
```



```

4 # 共享库
5 add_library(mylib SHARED src1.cpp src2.cpp)
6
7 # 让 CMake 根据 BUILD_SHARED_LIBS 变量决定
8 add_library(mylib src1.cpp src2.cpp)

```

1.6.2.7. 头文件修改的连锁反应

现在我们可以解释一个重要的问题：为什么修改头文件会导致多个源文件重新编译？

回顾编译单元的概念：每个 .cpp 文件经过预处理后形成一个编译单元，预处理会把所有 `#include` 的头文件内容插入进来。因此，一个头文件实际上是多个编译单元的一部分。

假设 `config.h` 被 `detector.cpp`、`tracker.cpp` 和 `predictor.cpp` 包含。从依赖关系看：

- `detector.o` 依赖于 `detector.cpp` 和 `config.h`
- `tracker.o` 依赖于 `tracker.cpp` 和 `config.h`
- `predictor.o` 依赖于 `predictor.cpp` 和 `config.h`

当你修改 `config.h` 时，这三个目标文件都变得“过时”了，都需要重新编译。这是正确的行为——头文件中可能定义了类的布局、常量的值、宏的内容，这些都可能影响编译结果。

这种连锁反应解释了为什么应该谨慎设计头文件的包含关系。如果一个被广泛包含的头文件发生变化，可能触发大量重新编译。有几个策略可以减轻这个问题：

前向声明（forward declaration）可以减少头文件依赖。如果你只需要使用指针或引用，不需要知道类的完整定义，可以用前向声明代替包含头文件。

```

1 // 不好：包含了完整的头文件
2 #include "robot.h"
3
4 class Controller {
5     Robot* robot_; // 只用了指针
6 };
7
8 // 好：使用前向声明
9 class Robot; // 前向声明
10
11 class Controller {
12     Robot* robot_; // 只用了指针，不需要完整定义
13 };

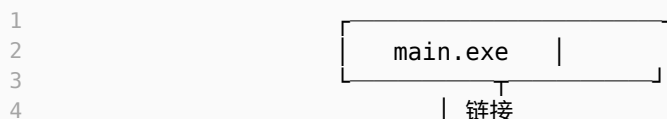
```

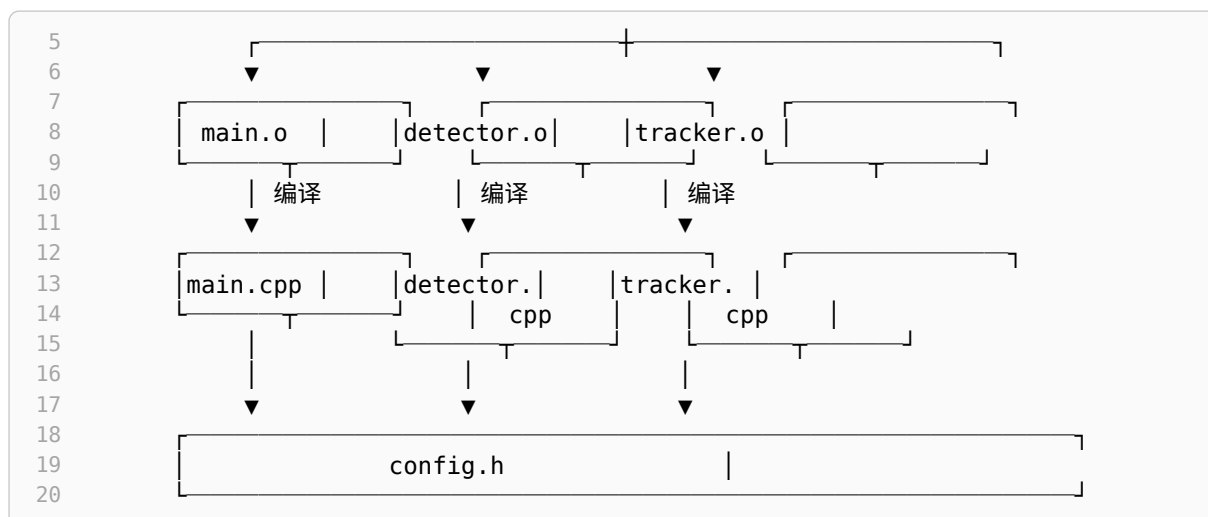
Pimpl（Pointer to Implementation）模式将实现细节隐藏在源文件中，头文件只暴露一个指向实现的指针。这样修改实现不会影响头文件，减少了重新编译的范围。

分层的头文件结构也很重要。将稳定的、基础的定义放在底层头文件中，将易变的、高层的定义放在上层头文件中。修改上层头文件不会影响底层代码。

1.6.2.8. 依赖图与构建优化

构建系统通过分析依赖关系构建一个有向无环图（DAG），图中的节点是文件（源文件、头文件、目标文件、库、可执行文件），边表示依赖关系。





当你修改一个文件时，构建系统从这个文件出发，沿着依赖边向上遍历，标记所有受影响的目标为“需要重建”。然后按照拓扑顺序（依赖在前，被依赖在后）重建这些目标。

这个依赖图还揭示了并行构建的机会。如果两个目标之间没有依赖关系（在图中没有路径相连），它们可以同时构建。在上图中，`main.o`、`detector.o` 和 `tracker.o` 的编译是独立的，可以并行进行。现代构建系统（如 Ninja）特别擅长发现和利用这种并行性。

```
1 # Make 并行编译
2 make -j8 # 最多 8 个并行任务
3
4 # Ninja 默认就会利用所有 CPU 核心
5 ninja
6
7 # CMake 构建时指定并行度
8 cmake --build . --parallel 8
```

构建系统还会缓存编译命令的哈希值。如果源文件没变，但编译选项变了（比如从 Debug 改为 Release），目标文件也需要重建。有些高级构建系统（如 ccache）还会缓存编译结果，当相同的源文件和编译选项再次出现时，直接复用之前的编译结果，进一步加速构建。

```
1 # 使用 ccache 加速编译
2 sudo apt install ccache
3 export CXX="ccache g++"
4 cmake ..
5 make
```

理解了这些编译原理，你就能更好地理解 CMake 的配置选项，知道为什么某些修改会触发大量重新编译，以及如何组织项目结构来优化构建时间。这些知识在大型项目中尤为重要——当编译时间从几分钟变成几十分钟时，优化构建过程就成了提高开发效率的关键。

1.6.3. CMake 基础语法

理解了构建系统的必要性和编译原理之后，我们正式开始学习 CMake。CMake 有自己的一套语法，虽然看起来不像传统的编程语言，但学习曲线并不陡峭。本节将介绍 CMake 的核心语法元素，让你能够编写基本的构建配置。我们从最简单的例子开始，逐步引入更多概念。

1.6.3.1. CMakeLists.txt: 项目的构建描述

CMake 的配置文件名为 `CMakeLists.txt`，注意大小写——在 Linux 等大小写敏感的系统上，`cmakelists.txt` 或 `CMakELISTS.TXT` 是无法识别的。每个需要构建的目录下都可以有一个 `CMakeLists.txt` 文件，CMake 会按照目录结构递归处理它们。

`CMakeLists.txt` 是一个文本文件，包含一系列 CMake 命令。命令的基本格式是：

```
1 command_name(arg1 arg2 arg3 ...)
```

命令名不区分大小写，`add_executable`、`ADD_EXECUTABLE` 和 `Add_Executable` 都是合法的，但惯例是使用小写。参数之间用空格或换行分隔，不需要逗号。如果参数包含空格，需要用双引号括起来。

```
1 # 这是注释，以 # 开头
2
3 # 命令名小写是惯例
4 add_executable(my_program main.cpp)
5
6 # 多个参数可以换行
7 add_executable(my_program
8     main.cpp
9     utils.cpp
10    helper.cpp
11 )
12
13 # 包含空格的参数需要引号
14 message("Hello, World!")
15 set(MY_PATH "/path/with spaces/file.txt")
```

1.6.3.2. 最小的 CMakeLists.txt

让我们从最简单的例子开始。假设你有一个只有 `main.cpp` 的项目，最小的 `CMakeLists.txt` 只需要三行：

```
1 cmake_minimum_required(VERSION 3.16)
2 project(hello)
3 add_executable(hello main.cpp)
```

这三行分别做了什么？

`cmake_minimum_required` 指定了项目需要的 CMake 最低版本。这是每个 `CMakeLists.txt` 必须有的第一条命令。它确保使用旧版本 CMake 的用户在构建项目前得到明确的错误提示，而不是遇到莫名其妙的问题。

`project` 声明项目名称。这个名称会被用于生成的项目文件（如 Visual Studio 解决方案），也可以在配置中通过变量 `PROJECT_NAME` 引用。

`add_executable` 定义一个可执行文件目标，第一个参数是目标名称，后面是组成这个目标的源文件列表。

有了这三行，你就可以构建项目了：

```
1 # 创建并进入构建目录
2 mkdir build && cd build
3
4 # 配置项目（生成构建文件）
5 cmake ..
```

```

6
7 # 构建（编译和链接）
8 cmake --build .
9
10 # 运行程序
11 ./hello

```

1.6.3.3. cmake_minimum_required: 版本要求

`cmake_minimum_required` 不仅指定最低版本，还会影响 CMake 的行为。CMake 随着版本演进引入了许多新特性，同时也改变了一些默认行为。通过指定版本，你告诉 CMake 使用那个版本的策略（policy）。

```
1 cmake_minimum_required(VERSION 3.16)
```

选择版本时需要权衡。版本太低会错过新特性和最佳实践；版本太高会让使用旧系统的用户无法构建你的项目。一般建议：

- 如果是新项目，使用较新但不是最新的版本，如 3.16 或 3.20
- 如果需要广泛兼容，可以用 3.10 或 3.12
- ROS 2 Humble 要求至少 3.16

你也可以指定版本范围：

```

1 # 最低 3.16，最高按 3.25 的行为
2 cmake_minimum_required(VERSION 3.16...3.25)

```

1.6.3.4. project: 项目声明

`project` 命令声明项目的基本信息：

```
1 project(rm_vision)
```

最简单的形式只需要项目名称。但 `project` 还支持更多参数：

```

1 project(rm_vision
2     VERSION 2.0.0
3     DESCRIPTION "RoboMaster 视觉系统"
4     LANGUAGES CXX
5 )

```

`VERSION` 指定项目版本，会设置 `PROJECT_VERSION`、`PROJECT_VERSION_MAJOR`、`PROJECT_VERSION_MINOR`、`PROJECT_VERSION_PATCH` 等变量。

`DESCRIPTION` 提供项目描述，存储在 `PROJECT_DESCRIPTION` 变量中。

`LANGUAGES` 指定项目使用的编程语言。常见值有 `C`、`CXX` (C++)、`Fortran`、`CUDA` 等。如果不指定，默认是 `C` 和 `CXX`。只指定需要的语言可以略微加快配置速度。

```

1 # 纯 C++ 项目
2 project(my_project LANGUAGES CXX)
3
4 # C 和 C++ 混合项目
5 project(my_project LANGUAGES C CXX)
6
7 # 包含 CUDA 的项目

```

```
8 project(my_project LANGUAGES CXX CUDA)
```

`project` 命令还会设置一些重要的变量：

- `PROJECT_NAME`：项目名称
- `PROJECT_SOURCE_DIR`：项目源代码目录（包含 `project` 命令的 `CMakeLists.txt` 所在目录）
- `PROJECT_BINARY_DIR`：项目构建目录

如果是顶层项目（不是被其他项目包含），这些变量也会被复制到不带 `PROJECT_` 前缀的版本，如 `CMAKE_PROJECT_NAME`。

1.6.3.5. `add_executable`：定义可执行文件

`add_executable` 是最常用的命令之一，它定义一个可执行文件目标：

```
1 add_executable(rm_vision
2     src/main.cpp
3     src/detector.cpp
4     src/tracker.cpp
5     src/predictor.cpp
6 )
```

第一个参数是目标名称，后面是源文件列表。目标名称会成为生成的可执行文件名（在 Windows 上会自动加 `.exe` 后缀）。

源文件路径可以是相对路径（相对于当前 `CMakeLists.txt` 所在目录）或绝对路径。推荐使用相对路径，因为绝对路径会破坏项目的可移植性。

```
1 # 相对路径（推荐）
2 add_executable(my_app src/main.cpp)
3
4 # 绝对路径（不推荐，但有时需要）
5 add_executable(my_app ${CMAKE_CURRENT_SOURCE_DIR}/src/main.cpp)
```

注意，`add_executable` 中只需要列出 `.cpp` 源文件，不需要列出头文件。编译器会通过源文件中的 `#include` 自动找到头文件。不过，如果你使用 IDE（如 Visual Studio、CLion），在 `add_executable` 中列出头文件可以让它们出现在项目视图中，便于浏览：

```
1 add_executable(rm_vision
2     src/main.cpp
3     src/detector.cpp
4     include/detector.h # 可选，便于 IDE 显示
5     include/config.h
6 )
```

1.6.3.6. `add_library`：定义库

`add_library` 用于创建库目标。库可以被其他目标链接使用，是组织代码的重要方式。

```
1 # 创建静态库
2 add_library(rm_core STATIC
3     src/math_utils.cpp
4     src/config.cpp
5     src/logger.cpp
6 )
7
```

```

8 # 创建共享库
9 add_library(rm_core SHARED
10     src/math_utils.cpp
11     src/config.cpp
12     src/logger.cpp
13 )

```

STATIC 创建静态库（Linux 上是 **.a** 文件），**SHARED** 创建共享库（Linux 上是 **.so** 文件）。

如果不指定库类型，CMake 会根据 **BUILD_SHARED_LIBS** 变量决定：

```

1 # 类型由 BUILD_SHARED_LIBS 决定
2 add_library(rm_core
3     src/math_utils.cpp
4     src/config.cpp
5 )

```

用户可以在配置时选择：

```

1 # 构建共享库
2 cmake -DBUILD_SHARED_LIBS=ON ..
3
4 # 构建静态库（默认）
5 cmake -DBUILD_SHARED_LIBS=OFF ..

```

还有一种特殊的库类型：**INTERFACE** 库。它不包含实际的代码，只用于传递编译选项、包含路径等。Header-only 库（如 Eigen）通常使用这种方式：

```

1 # 接口库，只有头文件
2 add_library(my_header_lib INTERFACE)
3 target_include_directories(my_header_lib INTERFACE include)

```

另一种是 **OBJECT** 库，它编译源文件生成目标文件，但不打包成库文件。这在某些高级场景中有用：

```

1 # 对象库
2 add_library(my_objects OBJECT src1.cpp src2.cpp)
3
4 # 在其他目标中使用
5 add_executable(app1 main1.cpp $<TARGET_OBJECTS:my_objects>)
6 add_executable(app2 main2.cpp $<TARGET_OBJECTS:my_objects>)

```

创建了库之后，可以用 **target_link_libraries** 将其链接到其他目标：

```

1 add_library(rm_core STATIC src/core.cpp)
2 add_executable(rm_vision src/main.cpp)
3
4 # 链接库到可执行文件
5 target_link_libraries(rm_vision rm_core)

```

1.6.3.7. 变量：set 与 \${VAR}

变量是 CMake 中存储和传递信息的基本方式。使用 **set** 命令定义变量，使用 **\${VAR}** 语法引用变量的值：

```

1 # 定义变量

```

```

2 set(MY_VARIABLE "hello")
3
4 # 使用变量
5 message(STATUS "MY_VARIABLE = ${MY_VARIABLE}")
6
7 # 变量可以用在任何需要字符串的地方
8 set(SRC_DIR "src")
9 add_executable(app ${SRC_DIR}/main.cpp)

```

变量名区分大小写，`MY_VAR`、`my_var` 和 `My_Var` 是三个不同的变量。惯例是使用大写字母和下划线。

变量可以包含列表（多个值），值之间用分号分隔：

```

1 # 定义列表变量
2 set(SOURCES main.cpp utils.cpp helper.cpp)
3
4 # 上面等价于
5 set(SOURCES "main.cpp;utils.cpp;helper.cpp")
6
7 # 使用列表
8 add_executable(app ${SOURCES})

```

当你在 `set` 中提供多个值时，它们会自动合并成分号分隔的列表。这就是为什么 `add_executable` 的参数可以是多个文件名或一个包含多个文件的变量。

如果变量未定义，`${VAR}` 会展开为空字符串。可以用 `if(DEFINED VAR)` 检查变量是否定义：

```

1 if(DEFINED MY_VAR)
2     message(STATUS "MY_VAR is defined: ${MY_VAR}")
3 else()
4     message(STATUS "MY_VAR is not defined")
5 endif()

```

CMake 预定义了许多有用的变量：

```

1 # 源代码目录（顶层 CMakeLists.txt 所在目录）
2 message(STATUS "Source dir: ${CMAKE_SOURCE_DIR}")
3
4 # 构建目录
5 message(STATUS "Binary dir: ${CMAKE_BINARY_DIR}")
6
7 # 当前处理的 CMakeLists.txt 所在目录
8 message(STATUS "Current source dir: ${CMAKE_CURRENT_SOURCE_DIR}")
9
10 # 当前处理的 CMakeLists.txt 对应的构建目录
11 message(STATUS "Current binary dir: ${CMAKE_CURRENT_BINARY_DIR}")
12
13 # C++ 编译器路径
14 message(STATUS "CXX compiler: ${CMAKE_CXX_COMPILER}")
15
16 # 构建类型（Debug/Release/...）
17 message(STATUS "Build type: ${CMAKE_BUILD_TYPE}")

```

1.6.3.8. 缓存变量

普通变量只在当前作用域内有效。缓存变量（cache variable）则保存在构建目录的 `CMakeCache.txt` 文件中，在多次配置之间持久存在，并且可以被用户在命令行修改。

使用 `set` 的 `CACHE` 选项定义缓存变量：

```
1 # 定义缓存变量
2 set(ENABLE_DEBUG ON CACHE BOOL "Enable debug mode")
3 set(MAX_THREADS 4 CACHE STRING "Maximum number of threads")
4 set(INSTALL_PREFIX "/usr/local" CACHE PATH "Installation prefix")
```

缓存变量的类型有：

- `BOOL`：布尔值，ON/OFF
- `STRING`：字符串
- `PATH`：目录路径
- `FILEPATH`：文件路径
- `INTERNAL`：内部使用，不在 GUI 中显示

最后一个参数是描述字符串，会在 `ccmake` 或 `cmake-gui` 中显示。

用户可以在配置时通过 `-D` 选项设置缓存变量：

```
1 cmake -DENABLE_DEBUG=OFF -DMAX_THREADS=8 ..
```

设置后，值会保存在 `CMakeCache.txt` 中，下次配置时不需要再次指定。

`option` 命令是定义布尔缓存变量的快捷方式：

```
1 # 这两行等价
2 option(ENABLE_TESTS "Build unit tests" ON)
3 set(ENABLE_TESTS ON CACHE BOOL "Build unit tests")
```

`option` 更简洁，适合开关类的配置。

1.6.3.9. 列表操作

CMake 的列表是分号分隔的字符串。`list` 命令提供了丰富的列表操作功能：

```
1 # 创建列表
2 set(MY_LIST a b c) # MY_LIST = "a;b;c"
3
4 # 追加元素
5 list(APPEND MY_LIST d e) # MY_LIST = "a;b;c;d;e"
6
7 # 获取长度
8 list(LENGTH MY_LIST len) # len = 5
9
10 # 获取元素（索引从 0 开始）
11 list(GET MY_LIST 0 first) # first = "a"
12 list(GET MY_LIST -1 last) # last = "e"（负数从末尾数）
13
14 # 查找元素
15 list(FIND MY_LIST "c" index) # index = 2, 找不到则为 -1
16
17 # 插入元素
18 list(INSERT MY_LIST 1 x) # MY_LIST = "a;x;b;c;d;e"
19
20 # 移除元素
21 list(REMOVE_ITEM MY_LIST x) # 按值移除
22 list(REMOVE_AT MY_LIST 0) # 按索引移除
23
24 # 排序
25 list(SORT MY_LIST)
26
```

```

27 # 去重
28 list(REMOVE_DUPLICATES MY_LIST)
29
30 # 反转
31 list(REVERSE MY_LIST)
32
33 # 连接成字符串
34 list(JOIN MY_LIST " " result) # result = "a, b, c, d, e"

```

列表操作在组织源文件时特别有用：

```

1 # 分别定义各模块的源文件
2 set(DETECTOR_SOURCES
3     src/detector/armor_detector.cpp
4     src/detector/rune_detector.cpp
5 )
6
7 set(TRACKER_SOURCES
8     src/tracker/armor_tracker.cpp
9     src/tracker/kalman_filter.cpp
10 )
11
12 set(COMMON_SOURCES
13     src/common/config.cpp
14     src/common/logger.cpp
15 )
16
17 # 合并所有源文件
18 set(ALL_SOURCES)
19 list(APPEND ALL_SOURCES ${DETECTOR_SOURCES})
20 list(APPEND ALL_SOURCES ${TRACKER_SOURCES})
21 list(APPEND ALL_SOURCES ${COMMON_SOURCES})
22
23 # 使用合并后的列表
24 add_library(rm_vision ${ALL_SOURCES})

```

这种方式比直接在 `add_library` 中列出所有文件更清晰，特别是当源文件很多、分布在多个目录时。

关于自动收集源文件的 `file(GLOB ...)`，这里需要提醒一下：

```

1 # 自动收集所有 .cpp 文件
2 file(GLOB SOURCES "src/*.cpp")
3 file(GLOB_RECURSE SOURCES "src/*.cpp") # 递归搜索子目录
4
5 add_executable(app ${SOURCES})

```

虽然 `file(GLOB)` 看起来很方便，但 CMake 官方不推荐用它来收集源文件。原因是：当你添加或删除源文件时，CMake 不会自动重新配置，因为 `CMakeLists.txt` 本身没有变化。这可能导致新文件没有被编译，或者删除的文件还在使用旧的目标文件。手动列出源文件虽然繁琐，但更加可靠。

```

1 # 推荐：手动列出源文件
2 add_executable(app
3     src/main.cpp
4     src/utils.cpp
5     src/helper.cpp
6 )

```

如果确实想用 `file(GLOB)`，可以配合 `CONFIGURE_DEPENDS` 选项 (CMake 3.12+)，让 CMake 在每次构建时检查文件变化：

```
1 file(GLOB SOURCES CONFIGURE_DEPENDS "src/*.cpp")
```

但这会略微增加构建时间，而且不是所有生成器都支持。

1.6.3.10. 条件语句

CMake 的条件语句使用 `if / elseif / else / endif` 结构。注意每个分支都需要对应的结束标记。

```
1 if(ENABLE_DEBUG)
2     message(STATUS "Debug mode enabled")
3     add_compile_definitions(DEBUG_MODE)
4 elseif(ENABLE_RELEASE)
5     message(STATUS "Release mode enabled")
6     add_compile_definitions(RELEASE_MODE)
7 else()
8     message(STATUS "Default mode")
9 endif()
```

条件表达式的求值规则值得仔细了解。对于布尔值，以下情况被视为真：`ON`、`YES`、`TRUE`、`Y` 或非零数字。以下情况被视为假：`OFF`、`NO`、`FALSE`、`N`、`IGNORE`、`NOTFOUND`、空字符串或以 `-NOTFOUND` 结尾的字符串。

变量在条件中可以直接使用名称，不需要 `${}`：

```
1 set(ENABLE_FEATURE ON)
2
3 # 这两种写法都可以，但推荐第一种
4 if(ENABLE_FEATURE)
5     message("Feature enabled")
6 endif()
7
8 if(${ENABLE_FEATURE}) # 也可以，但不推荐
9     message("Feature enabled")
10 endif()
```

直接使用变量名时，CMake 会自动解引用。但如果变量未定义，行为会不同：`if(VAR)` 在 `VAR` 未定义时为假，而 `if(${VAR})` 会展开为 `if()` 并产生错误。

CMake 提供了丰富的条件操作符：

```
1 # 逻辑操作符
2 if(A AND B)           # 与
3 if(A OR B)            # 或
4 if(NOT A)             # 非
5
6 # 比较操作符 (数值)
7 if(A EQUAL B)         # 等于
8 if(A LESS B)          # 小于
9 if(A GREATER B)       # 大于
10 if(A LESS_EQUAL B)   # 小于等于 (CMake 3.7+)
11 if(A GREATER_EQUAL B) # 大于等于 (CMake 3.7+)
12
13 # 比较操作符 (字符串)
14 if(A STREQUAL B)      # 字符串相等
15 if(A STRLESS B)       # 字符串小于 (字典序)
```



```

16 if(A STRGREATER B)    # 字符串大于
17
18 # 版本比较
19 if(A VERSION_EQUAL B)
20 if(A VERSION_LESS B)
21 if(A VERSION_GREATER B)
22
23 # 正则表达式匹配
24 if(A MATCHES "regex")
25
26 # 存在性检查
27 if(DEFINED VAR)        # 变量是否定义
28 if(EXISTS path)        # 文件或目录是否存在
29 if(IS_DIRECTORY path) # 是否是目录
30 if(IS_ABSOLUTE path)   # 是否是绝对路径
31
32 # 目标检查
33 if(TARGET target_name) # 目标是否存在

```

一个实际的例子，根据操作系统设置不同的编译选项：

```

1 if(WIN32)
2     message(STATUS "Building on Windows")
3     add_compile_definitions(PLATFORM_WINDOWS)
4 elseif(APPLE)
5     message(STATUS "Building on macOS")
6     add_compile_definitions(PLATFORM_MACOS)
7 elseif(UNIX)
8     message(STATUS "Building on Linux/Unix")
9     add_compile_definitions(PLATFORM_LINUX)
10 endif()

```

另一个常见用法是根据构建类型设置选项：

```

1 if(CMAKE_BUILD_TYPE STREQUAL "Debug")
2     message(STATUS "Debug build")
3     set(ENABLE_LOGGING ON)
4 elseif(CMAKE_BUILD_TYPE STREQUAL "Release")
5     message(STATUS "Release build")
6     set(ENABLE_LOGGING OFF)
7 endif()

```

条件语句在处理可选依赖时也很有用：

```

1 find_package(OpenCV QUIET) # QUIET 表示找不到时不报错
2
3 if(OpenCV_FOUND)
4     message(STATUS "OpenCV found: ${OpenCV_VERSION}")
5     target_link_libraries(my_app OpenCV::OpenCV)
6     target_compile_definitions(my_app PRIVATE HAS_OPENCV)
7 else()
8     message(WARNING "OpenCV not found, some features will be disabled")
9 endif()

```

1.6.3.11. 循环语句

CMake 提供了 `foreach` 和 `while` 两种循环结构。`foreach` 更常用，适合遍历列表；`while` 适合条件循环。

`foreach` 的基本形式是遍历列表中的每个元素：

```

1 set(MODULES detector tracker predictor)
2
3 foreach(module ${MODULES})
4     message(STATUS "Processing module: ${module}")
5     add_subdirectory(${module})
6 endforeach()

```

遍历范围：

```

1 # 从 0 到 9
2 foreach(i RANGE 9)
3     message(STATUS "i = ${i}")
4 endforeach()
5
6 # 从 1 到 10
7 foreach(i RANGE 1 10)
8     message(STATUS "i = ${i}")
9 endforeach()
10
11 # 从 0 到 100, 步长 10
12 foreach(i RANGE 0 100 10)
13     message(STATUS "i = ${i}")
14 endforeach()

```

遍历多个列表（同时迭代）：

```

1 set(NAMES alice bob charlie)
2 set(AGES 25 30 35)
3
4 foreach(name age IN ZIP_LISTS NAMES AGES)
5     message(STATUS "${name} is ${age} years old")
6 endforeach()

```

遍历列表并获取索引（需要一些技巧）：

```

1 set(ITEMS a b c d e)
2 list(LENGTH ITEMS count)
3 math(EXPR last "${count} - 1")
4
5 foreach(i RANGE ${last})
6     list(GET ITEMS ${i} item)
7     message(STATUS "Item ${i}: ${item}")
8 endforeach()

```

while 循环用于条件循环：

```

1 set(counter 0)
2
3 while(counter LESS 5)
4     message(STATUS "Counter: ${counter}")
5     math(EXPR counter "${counter} + 1")
6 endwhile()

```

循环中可以使用 **break()** 和 **continue()**：

```

1 foreach(i RANGE 10)
2     if(i EQUAL 3)
3         continue() # 跳过 3
4     endif()
5     if(i EQUAL 7)

```

```

6         break()      # 到 7 时退出循环
7     endif()
8     message(STATUS "i = ${i}")
9 endforeach()
10 # 输出: 0, 1, 2, 4, 5, 6

```

一个实际的例子，为每个源文件创建对应的测试：

```

1 set(TEST_SOURCES
2     test_detector.cpp
3     test_tracker.cpp
4     test_predictor.cpp
5 )
6
7 foreach(test_src ${TEST_SOURCES})
8     # 从文件名提取测试名（去掉 .cpp 后缀）
9     get_filename_component(test_name ${test_src} NAME_WE)
10
11     # 创建测试可执行文件
12     add_executable(${test_name} ${test_src})
13     target_link_libraries(${test_name} rm_core GTest::gtest_main)
14
15     # 注册测试
16     add_test(NAME ${test_name} COMMAND ${test_name})
17 endforeach()

```

1.6.3.12. 函数与宏

当你发现自己在重复编写类似的 CMake 代码时，可以将其封装成函数或宏。两者语法类似，但作用域行为不同。

函数使用 `function` / `endfunction` 定义：

```

1 function(print_variables)
2     message(STATUS "CMAKE_SOURCE_DIR: ${CMAKE_SOURCE_DIR}")
3     message(STATUS "CMAKE_BINARY_DIR: ${CMAKE_BINARY_DIR}")
4     message(STATUS "PROJECT_NAME: ${PROJECT_NAME}")
5 endfunction()
6
7 # 调用函数
8 print_variables()

```

带参数的函数：

```

1 function(add_my_library lib_name)
2     # ARGV 包含除命名参数外的所有参数
3     add_library(${lib_name} ${ARGV})
4     target_include_directories(${lib_name} PUBLIC include)
5     target_compile_features(${lib_name} PUBLIC cxx_std_17)
6 endfunction()
7
8 # 使用
9 add_my_library(utils src/utils.cpp src/helper.cpp)

```

函数参数通过以下变量访问：

- `ARGC`：参数总数
- `ARGV`：所有参数的列表
- `ARGN`：除命名参数外的剩余参数

- `ARGV0`、`ARGV1`、…：按位置访问参数

函数有自己的作用域，在函数内部定义或修改的变量不会影响外部。如果需要向外部返回值，使用 `PARENT_SCOPE`：

```
1 function(get_git_version result_var)
2     execute_process(
3         COMMAND git describe --tags --always
4         OUTPUT_VARIABLE git_version
5         OUTPUT_STRIP_TRAILING_WHITESPACE
6     )
7     set(${result_var} ${git_version} PARENT_SCOPE)
8 endfunction()
9
10 # 使用
11 get_git_version(VERSION)
12 message(STATUS "Git version: ${VERSION}")
```

宏使用 `macro` / `endmacro` 定义，语法与函数几乎相同：

```
1 macro(print_info msg)
2     message(STATUS "[INFO] ${msg}")
3 endmacro()
4
5 print_info("Hello from macro")
```

函数和宏的关键区别在于作用域：

- 函数有自己的作用域，变量修改不影响调用者
- 宏没有自己的作用域，直接在调用者的作用域执行（类似文本替换）

```
1 set(MY_VAR "original")
2
3 function(modify_in_function)
4     set(MY_VAR "modified in function")
5 endfunction()
6
7 macro(modify_in_macro)
8     set(MY_VAR "modified in macro")
9 endmacro()
10
11 modify_in_function()
12 message(STATUS "After function: ${MY_VAR}") # original (未改变)
13
14 modify_in_macro()
15 message(STATUS "After macro: ${MY_VAR}")    # modified in macro (改变了)
```

一般推荐使用函数，因为作用域隔离可以避免意外的副作用。宏主要用于需要直接影响调用者作用域的场景。

CMake 还提供了 `cmake_parse_arguments` 用于解析复杂的函数参数，支持选项、单值参数和多值参数：

```
1 function(add_rm_module)
2     # 解析参数
3     cmake_parse_arguments(
4         ARG                                # 前缀
5         "SHARED;STATIC"                   # 选项（布尔）
6         "NAME;OUTPUT_DIR"                 # 单值参数
7         "SOURCES;DEPENDS"                 # 多值参数
8         ${ARGN})
```

```

9      )
10
11     # 使用解析后的参数
12     if(ARG_SHARED)
13         add_library(${ARG_NAME} SHARED ${ARG_SOURCES})
14     else()
15         add_library(${ARG_NAME} STATIC ${ARG_SOURCES})
16     endif()
17
18     if(ARG_DEPENDS)
19         target_link_libraries(${ARG_NAME} ${ARG_DEPENDS})
20     endif()
21
22     if(ARG_OUTPUT_DIR)
23         set_target_properties(${ARG_NAME} PROPERTIES
24             LIBRARY_OUTPUT_DIRECTORY ${ARG_OUTPUT_DIR}
25         )
26     endif()
27 endfunction()
28
29 # 使用
30 add_rm_module(
31     NAME detector
32     SOURCES src/detector.cpp src/armor.cpp
33     DEPENDS OpenCV::OpenCV Eigen3::Eigen
34     SHARED
35 )

```

1.6.3.13. 注释

CMake 使用 `#` 作为单行注释的开始，从 `#` 到行尾的内容都会被忽略：

```

1 # 这是一个完整的注释行
2
3 set(MY_VAR "value") # 这是行尾注释
4
5 # 多行注释需要每行都加 #
6 # 这是第一行
7 # 这是第二行
8 # 这是第三行

```

CMake 3.0 引入了括号注释，适合注释大段内容：

```

1 #[[
2 这是一个块注释。
3 可以跨越多行。
4 不需要每行都加 #。
5
6 适合临时禁用一段代码：
7 add_executable(old_app old_main.cpp)
8 ]]
9
10 # 括号注释也可以嵌套使用 #[=[ ]=]
11 #[=[
12 外层注释
13 #[[内层注释]]
14 外层继续
15 ]=]

```

良好的注释习惯可以让 CMakeLists.txt 更易维护：

```

1  #=====
2  # RoboMaster Vision System
3  # CMake 构建配置
4  #=====
5
6  cmake_minimum_required(VERSION 3.16)
7  project(rm_vision VERSION 2.0.0)
8
9  #-----
10 # 编译选项
11 #-----
12 option(ENABLE_CUDA "Enable CUDA acceleration" OFF)
13 option(BUILD_TESTS "Build unit tests" ON)
14
15 #-----
16 # 依赖查找
17 #-----
18 find_package(OpenCV REQUIRED)
19 find_package(Eigen3 REQUIRED)
20
21 # Ceres 是可选依赖，用于高级优化功能
22 find_package(Ceres QUIET)
23
24 #-----
25 # 目标定义
26 #-----
27 add_library(rm_core
28     src/detector.cpp
29     src/tracker.cpp
30 )

```

1.6.3.14. 常用变量

CMake 预定义了大量变量，了解常用的变量可以让你更高效地编写配置。

路径相关变量是最常用的一组：

```

1  # 顶层 CMakeLists.txt 所在的源代码目录
2  message(STATUS "CMAKE_SOURCE_DIR: ${CMAKE_SOURCE_DIR}")
3
4  # 顶层构建目录
5  message(STATUS "CMAKE_BINARY_DIR: ${CMAKE_BINARY_DIR}")
6
7  # 当前正在处理的 CMakeLists.txt 所在目录
8  message(STATUS "CMAKE_CURRENT_SOURCE_DIR: ${CMAKE_CURRENT_SOURCE_DIR}")
9
10 # 当前 CMakeLists.txt 对应的构建目录
11 message(STATUS "CMAKE_CURRENT_BINARY_DIR: ${CMAKE_CURRENT_BINARY_DIR}")
12
13 # 项目的源代码目录 (project() 命令所在目录)
14 message(STATUS "PROJECT_SOURCE_DIR: ${PROJECT_SOURCE_DIR}")
15
16 # 项目的构建目录
17 message(STATUS "PROJECT_BINARY_DIR: ${PROJECT_BINARY_DIR}")

```

当项目有多层目录结构时，`CMAKE_SOURCE_DIR` 始终指向顶层，而 `CMAKE_CURRENT_SOURCE_DIR` 会随着处理的 `CMakeLists.txt` 变化。如果你的项目被另一个项目作为子项目包含，`CMAKE_SOURCE_DIR` 会指向外层项目，而 `PROJECT_SOURCE_DIR` 指向你自己的项目。

C++ 标准和编译器相关变量：

```

1 # 设置 C++ 标准版本
2 set(CMAKE_CXX_STANDARD 17)
3
4 # 要求使用指定的标准, 而不是降级
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # 不使用编译器扩展 (更好的可移植性)
8 set(CMAKE_CXX_EXTENSIONS OFF)
9
10 # 添加编译选项
11 set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra")
12
13 # 只在 Debug 模式添加的选项
14 set(CMAKE_CXX_FLAGS_DEBUG "${CMAKE_CXX_FLAGS_DEBUG} -g -O0")
15
16 # 只在 Release 模式添加的选项
17 set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O3 -DNDEBUG")
18
19 # 编译器标识
20 message(STATUS "Compiler: ${CMAKE_CXX_COMPILER_ID}") # GNU, Clang, MSVC 等
21 message(STATUS "Compiler version: ${CMAKE_CXX_COMPILER_VERSION}")

```

注意：现代 CMake 推荐使用 `target_compile_features` 和 `target_compile_options` 代替全局设置：

```

1 # 现代方式：针对特定目标设置
2 add_executable(my_app main.cpp)
3 target_compile_features(my_app PRIVATE cxx_std_17)
4 target_compile_options(my_app PRIVATE -Wall -Wextra)

```

构建类型变量：

```

1 # CMAKE_BUILD_TYPE 控制构建类型
2 # 常见值: Debug, Release, RelWithDebInfo, MinSizeRel
3
4 # 设置默认构建类型
5 if(NOT CMAKE_BUILD_TYPE)
6     set(CMAKE_BUILD_TYPE "Release" CACHE STRING "Build type" FORCE)
7 endif()
8
9 message(STATUS "Build type: ${CMAKE_BUILD_TYPE}")
10
11 # 根据构建类型设置选项
12 if(CMAKE_BUILD_TYPE STREQUAL "Debug")
13     set(ENABLE_ASSERTIONS ON)
14 else()
15     set(ENABLE_ASSERTIONS OFF)
16 endif()

```

构建类型主要影响编译优化级别和调试信息：

- **Debug**：无优化（-O0），包含调试信息（-g），适合开发调试
- **Release**：高优化（-O3），无调试信息，适合发布
- **RelWithDebInfo**：优化（-O2）+ 调试信息，适合需要调试的生产环境
- **MinSizeRel**：优化体积（-Os），适合嵌入式等空间受限场景

输出目录变量：

```

1 # 可执行文件输出目录
2 set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

```

```

3
4 # 静态库输出目录
5 set(CMAKE_ARCHIVE_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
6
7 # 共享库输出目录
8 set(CMAKE_LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/lib)
9
10 # 这样所有生成的文件会被整理到 build/bin 和 build/lib 目录

```

安装相关变量：

```

1 # 安装前缀，默认是 /usr/local (Linux)
2 message(STATUS "Install prefix: ${CMAKE_INSTALL_PREFIX}")
3
4 # 可以在配置时修改
5 # cmake -DCMAKE_INSTALL_PREFIX=/opt/rm_vision ..
6
7 # 标准安装目录（相对于 CMAKE_INSTALL_PREFIX）
8 # CMAKE_INSTALL_BINDIR      -> bin
9 # CMAKE_INSTALL_LIBDIR      -> lib 或 lib64
10 # CMAKE_INSTALL_INCLUDEDIR -> include

```

平台检测变量：

```

1 # 操作系统检测
2 if(WIN32)
3     # Windows (包括 64 位)
4 endif()
5
6 if(APPLE)
7     # macOS 或 iOS
8 endif()
9
10 if(UNIX AND NOT APPLE)
11     # Linux 或其他 Unix
12 endif()
13
14 # 更精确的检测
15 message(STATUS "System: ${CMAKE_SYSTEM_NAME}")      # Linux, Windows, Darwin
16 message(STATUS "Processor: ${CMAKE_SYSTEM_PROCESSOR}") # x86_64, arm 等
17
18 # 32 位 vs 64 位
19 if(CMAKE_SIZEOF_VOID_P EQUAL 8)
20     message(STATUS "64-bit system")
21 else()
22     message(STATUS "32-bit system")
23 endif()

```

find_package 相关变量：

```

1 find_package(OpenCV REQUIRED)
2
3 # find_package 成功后会设置以下变量（以 OpenCV 为例）
4 # OpenCV_FOUND      - 是否找到
5 # OpenCV_VERSION    - 版本号
6 # OpenCV_INCLUDE_DIRS - 头文件目录（旧式）
7 # OpenCV_LIBRARIES   - 库文件（旧式）
8 # OpenCV::OpenCV     - 导入目标（现代方式）
9
10 message(STATUS "OpenCV version: ${OpenCV_VERSION}")
11 message(STATUS "OpenCV libraries: ${OpenCV_LIBRARIES}")

```


了解这些变量可以帮助你编写更灵活、更可移植的 CMake 配置。当你遇到问题时，用 `message` 打印这些变量的值，往往能帮助你快速定位问题所在。

1.6.4. 构建流程实践

了解了 CMake 的基础语法之后，让我们完整地走一遍从配置到编译的流程。虽然前面已经简单演示过，但这里我们将深入讨论每个步骤的细节，包括不同的构建方式、生成器选择、构建类型、并行编译等实用技巧。掌握这些内容可以让你的日常开发更加高效。

1.6.4.1. 外部构建

CMake 支持两种构建方式：源内构建（in-source build）和源外构建（out-of-source build）。源内构建是在源代码目录中直接运行 CMake，生成的文件与源代码混在一起；源外构建是在单独的目录中运行 CMake，保持源代码目录的整洁。

```
1 # 源内构建（不推荐）
2 cd my_project
3 cmake .
4 make
5
6 # 源外构建（推荐）
7 cd my_project
8 mkdir build
9 cd build
10 cmake ..
11 make
```

源外构建有几个明显的优势。首先，源代码目录保持干净，生成的文件（目标文件、可执行文件、CMake 缓存等）都在构建目录中，不会污染源代码。这使得版本控制更简单——只需要在 `.gitignore` 中添加 `build/` 即可忽略所有构建产物。

其次，你可以同时拥有多个构建目录。比如一个用于 Debug 构建，一个用于 Release 构建；或者一个用于本地开发，一个用于交叉编译到 ARM 平台。每个构建目录都是独立的，互不干扰。

```
1 # 多个构建目录
2 mkdir build-debug build-release
3
4 cd build-debug
5 cmake -DCMAKE_BUILD_TYPE=Debug ..
6
7 cd ../build-release
8 cmake -DCMAKE_BUILD_TYPE=Release ..
```

最后，清理构建变得非常简单——直接删除构建目录即可，不需要担心误删源文件。

由于源外构建的诸多优势，现代 CMake 项目几乎都采用这种方式。有些项目甚至会在 `CMakeLists.txt` 中禁止源内构建：

```
1 if(CMAKE_SOURCE_DIR STREQUAL CMAKE_BINARY_DIR)
2     message(FATAL_ERROR "In-source builds are not allowed. Please use a
3         separate build directory.")
3 endif()
```

1.6.4.2. 配置与构建的两个阶段

使用 CMake 构建项目分为两个阶段：配置（configure）和构建（build）。理解这两个阶段的区别很重要。

配置阶段是运行 `cmake` 命令的过程。CMake 读取 `CMakeLists.txt`，执行其中的命令，检查系统环境，查找依赖库，最终生成原生构建系统的配置文件（如 `Makefile`）。配置阶段的输出保存在构建目录中，包括 `CMakeCache.txt`（缓存）和 `CMakeFiles/` 目录（中间文件）。

```
1 # 配置阶段
2 cd build
3 cmake ..
4
5 # 输出类似:
6 # -- The CXX compiler identification is GNU 11.4.0
7 # -- Detecting CXX compiler ABI info
8 # -- Detecting CXX compiler ABI info - done
9 # -- Check for working CXX compiler: /usr/bin/c++ - skipped
10 # -- Detecting CXX compile features
11 # -- Detecting CXX compile features - done
12 # -- Found OpenCV: /usr/local (found version "4.5.4")
13 # -- Configuring done
14 # -- Generating done
15 # -- Build files have been written to: /path/to/build
```

构建阶段是实际编译代码的过程。你可以使用生成的原生构建工具（如 `make`），也可以使用 CMake 的统一接口 `cmake --build`：

```
1 # 构建阶段 - 方式一：使用原生工具
2 make
3
4 # 构建阶段 - 方式二：使用 CMake 统一接口（推荐）
5 cmake --build .
```

`cmake --build` 的优势是跨平台通用。无论底层使用的是 `Make`、`Ninja` 还是 `Visual Studio`，命令都是相同的。它还支持一些通用选项：

```
1 # 并行构建
2 cmake --build . --parallel 8
3 cmake --build . -j 8 # 简写形式
4
5 # 只构建特定目标
6 cmake --build . --target my_app
7
8 # 清理构建产物
9 cmake --build . --target clean
10
11 # 显示详细构建命令
12 cmake --build . --verbose
```

配置只需要在首次构建或修改 `CMakeLists.txt` 后执行。如果只是修改了源代码，直接运行构建命令即可——构建系统会自动检测哪些文件需要重新编译。

```
1 # 首次构建：配置 + 构建
2 cmake ..
3 cmake --build .
4
5 # 修改源代码后：只需构建
6 cmake --build .
```

```
7
8 # 修改 CMakeLists.txt 后：需要重新配置
9 cmake .. # 或者构建系统会自动触发重新配置
10 cmake --build .
```

1.6.4.3. 生成器

CMake 是一个元构建系统，它不直接编译代码，而是生成原生构建系统的配置文件。这些原生构建系统被称为“生成器”（generator）。不同平台有不同的默认生成器：

- Linux: Unix Makefiles（默认）
- macOS: Unix Makefiles 或 Xcode
- Windows: Visual Studio 或 NMake Makefiles

你可以用 `-G` 选项指定生成器：

```
1 # 查看可用的生成器
2 cmake --help
3
4 # 使用 Ninja 生成器（推荐）
5 cmake -G Ninja ..
6
7 # 使用 Unix Makefiles
8 cmake -G "Unix Makefiles" ..
9
10 # 使用 Visual Studio (Windows)
11 cmake -G "Visual Studio 17 2022" ..
```

Ninja 是一个专注于速度的构建系统，由 Google 开发。它比 Make 更快，特别是在增量构建和并行构建方面。如果你的系统上安装了 Ninja（`sudo apt install ninja-build`），强烈推荐使用它：

```
1 # 安装 Ninja
2 sudo apt install ninja-build
3
4 # 使用 Ninja 构建
5 cmake -G Ninja ..
6 ninja # 或 cmake --build .
```

Ninja 的速度优势来自几个方面：它的依赖分析更高效，启动开销更小，默认就会利用所有 CPU 核心进行并行构建。在大型项目中，Ninja 可以比 Make 快数倍。

不同生成器生成的文件不同：

```
1 # Unix Makefiles 生成 Makefile
2 ls build/
3 # CMakeCache.txt  CMakeFiles/  Makefile  cmake_install.cmake
4
5 # Ninja 生成 build.ninja
6 ls build/
7 # CMakeCache.txt  CMakeFiles/  build.ninja  cmake_install.cmake
```

生成器的选择会影响后续的构建命令，但如果你使用 `cmake --build`，就不需要关心这个差异。

1.6.4.4. 构建类型

构建类型 (build type) 决定了编译器的优化级别和调试信息。CMake 预定义了四种构建类型：

- **Debug**：不优化，包含调试信息，适合开发调试
- **Release**：高度优化，不包含调试信息，适合发布
- **RelWithDebInfo**：优化 + 调试信息，适合需要调试的生产环境
- **MinSizeRel**：优化代码体积，适合嵌入式等空间受限场景

使用 `-DCMAKE_BUILD_TYPE` 指定构建类型：

```
1 # Debug 构建
2 cmake -DCMAKE_BUILD_TYPE=Debug ..
3
4 # Release 构建
5 cmake -DCMAKE_BUILD_TYPE=Release ..
```

不同构建类型对应不同的编译选项。对于 GCC/Clang：

```
1 Debug:      -g -O0
2 Release:    -O3 -DNDEBUG
3 RelWithDebInfo: -O2 -g -DNDEBUG
4 MinSizeRel: -Os -DNDEBUG
```

`-g` 生成调试信息，使得可以用 GDB 调试程序、查看变量值、设置断点等。`-O0` 表示不优化，代码执行顺序与源代码一致，便于调试。`-O2` 和 `-O3` 是不同级别的优化，`-O3` 最激进。`-DNDEBUG` 定义 `NDEBUG` 宏，会禁用 `assert()` 断言。

选择构建类型时需要权衡：

```
1 # 日常开发调试用 Debug
2 cmake -DCMAKE_BUILD_TYPE=Debug ..
3
4 # 性能测试和发布用 Release
5 cmake -DCMAKE_BUILD_TYPE=Release ..
6
7 # 需要调试但也需要接近真实性能时用 RelWithDebInfo
8 cmake -DCMAKE_BUILD_TYPE=RelWithDebInfo ..
```

对于 RoboMaster 开发，一个常见的实践是：在开发和调试阶段使用 Debug 构建，方便定位问题；在比赛或性能测试时使用 Release 构建，获得最佳性能。优化后的代码可能比未优化的快几倍甚至十几倍，这对实时性要求高的机器人系统至关重要。

在 `CMakeLists.txt` 中，可以设置默认构建类型：

```
1 # 如果用户没有指定构建类型，默认使用 Release
2 if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
3     message(STATUS "Setting build type to 'Release' as none was specified.")
4     set(CMAKE_BUILD_TYPE Release CACHE STRING "Choose the type of build."
        FORCE)
5     set_property(CACHE CMAKE_BUILD_TYPE PROPERTY STRINGS
6         "Debug" "Release" "MinSizeRel" "RelWithDebInfo")
7 endif()
```

注意：构建类型对于单配置生成器（如 Makefile、Ninja）在配置时指定，而对于多配置生成器（如 Visual Studio、Xcode）在构建时指定：

```
1 # 单配置生成器：配置时指定
2 cmake -DCMAKE_BUILD_TYPE=Release ..
3 cmake --build .
4
5 # 多配置生成器：构建时指定
6 cmake ..
7 cmake --build . --config Release
```

1.6.4.5. 并行编译

现代计算机都有多个 CPU 核心，利用并行编译可以显著缩短构建时间。

对于 Make，使用 `-j` 选项指定并行任务数：

```
1 # 使用 8 个并行任务
2 make -j8
3
4 # 使用所有可用核心
5 make -j$(nproc)
6
7 # nproc 命令返回 CPU 核心数
```

对于 Ninja，默认就会使用所有可用核心，无需额外指定。如果需要限制，可以用 `-j`：

```
1 # Ninja 默认并行
2 ninja
3
4 # 限制并行任务数
5 ninja -j4
```

使用 `cmake --build` 时：

```
1 # 指定并行任务数
2 cmake --build . --parallel 8
3 cmake --build . -j 8
4
5 # 使用所有核心
6 cmake --build . --parallel
7 cmake --build . -j
```

并行编译的最佳任务数通常等于或略大于 CPU 核心数。但如果内存有限（比如在小型设备或虚拟机上），过多的并行任务可能导致内存不足，反而降低效率甚至导致构建失败。这时可以减少并行任务数：

```
1 # 内存受限时，使用较少的并行任务
2 cmake --build . -j 2
```

你也可以在 `CMakeLists.txt` 中设置默认的并行数，但这不太常见，因为不同机器的配置不同：

```
1 # 设置 Make 的并行任务数（不推荐硬编码）
2 set(CMAKE_BUILD_PARALLEL_LEVEL 8)
```

1.6.4.6. 清理构建

有时候你需要从头开始重新构建，比如修改了编译器选项、遇到了奇怪的构建错误、或者想确保没有残留的旧文件。

最简单的方法是删除整个构建目录：

```
1 # 完全清理：删除构建目录
2 rm -rf build
3 mkdir build
4 cd build
5 cmake ..
6 cmake --build .
```

这是最彻底的方式，保证所有东西都是全新生成的。由于我们使用源外构建，这个操作完全安全，不会影响源代码。

如果只想清理编译产物而保留 CMake 配置，可以使用 `clean` 目标：

```
1 # 清理编译产物
2 make clean
3 # 或
4 cmake --build . --target clean
5 # 或 (Ninja)
6 ninja clean
```

这会删除目标文件、可执行文件等编译产物，但保留 CMake 生成的文件（如 Makefile、CMakeCache.txt）。下次构建时不需要重新配置。

有些情况下，你可能需要清理 CMake 缓存但保留编译产物。可以手动删除缓存文件：

```
1 # 删除 CMake 缓存，强制重新配置
2 rm CMakeCache.txt
3 rm -rf CMakeFiles/
4 cmake ..
```

一个实用的技巧是创建一个清理脚本：

```
1 #!/bin/bash
2 # clean.sh
3
4 cd "$(dirname "$0")"
5
6 if [ -d "build" ]; then
7     rm -rf build
8     echo "Build directory removed."
9 else
10    echo "Build directory does not exist."
11 fi
```

1.6.4.7. CMake 缓存

CMake 缓存是保存在构建目录中的 `CMakeCache.txt` 文件。它存储了配置过程中的所有缓存变量，包括用户设置的选项、检测到的系统信息、找到的库路径等。

```
1 # 查看缓存内容
2 cat CMakeCache.txt
3
4 # 或者用 cmake 命令查看
5 cmake -L ..      # 列出非高级缓存变量
6 cmake -LA ..     # 列出所有缓存变量
7 cmake -LAH ..    # 列出所有缓存变量及其帮助信息
```

缓存的一个重要特性是持久性。一旦某个变量被设置到缓存中，后续的配置运行会使用缓存的值，而不会重新检测或使用默认值。这既是优点（不需要每次都指定选项），也可能是陷阱（修改了选项但没有生效）。

```
1 # 首次配置，设置选项
2 cmake -DENABLE_CUDA=ON ..
3
4 # 再次配置，即使不指定，ENABLE_CUDA 仍然是 ON
5 cmake ..
6
7 # 要修改缓存的值，需要显式指定
8 cmake -DENABLE_CUDA=OFF ..
```

如果发现选项修改没有生效，检查缓存是一个好习惯。你可以用 `-U` 选项删除特定的缓存变量：

```
1 # 删除特定缓存变量
2 cmake -UENABLE_CUDA ..
3
4 # 删除匹配模式的缓存变量
5 cmake -UENABLE_* ..
```

或者直接删除 `CMakeCache.txt` 强制重新配置。

缓存变量的类型（`BOOL`、`STRING`、`PATH` 等）会影响 GUI 工具的显示方式。标记为 `INTERNAL` 的缓存变量不会在 GUI 中显示，用于存储 CMake 内部使用的信息。

1.6.4.8. 交互式配置：ccmake 和 cmake-gui

对于有很多配置选项的项目，在命令行上一个一个指定 `-D` 选项很不方便。CMake 提供了两个交互式配置工具：`ccmake`（终端界面）和 `cmake-gui`（图形界面）。

`ccmake` 是一个基于 ncurses 的终端界面工具：

```
1 # 安装 ccmake
2 sudo apt install cmake-curses-gui
3
4 # 运行 ccmake
5 cd build
6 cmake ..
```

在 `ccmake` 中：

- 使用方向键上下移动，选择要修改的变量
- 按 `Enter` 编辑变量值
- 按 `t` 切换显示高级变量
- 按 `c` 配置（configure）
- 按 `g` 生成（generate）并退出
- 按 `q` 退出

`cmake-gui` 是一个图形界面工具，更加直观：

```
1 # 安装 cmake-gui
2 sudo apt install cmake-gui
3
4 # 运行 cmake-gui
5 cmake-gui
```

在 cmake-gui 中：

1. 设置源代码目录和构建目录
2. 点击 “Configure” 按钮，选择生成器
3. 修改需要的选项（变量会以红色高亮显示新变化的项）
4. 再次点击 “Configure” 直到没有红色项
5. 点击 “Generate” 生成构建文件

这两个工具特别适合：

- 初次配置一个陌生的项目，浏览有哪些可用选项
- 需要修改多个选项时，比逐个 `-D` 更方便
- 想要了解某个选项的含义（工具会显示描述信息）

一个典型的工作流程是：首次使用交互式工具浏览和设置选项，之后在命令行进行日常构建，需要修改配置时再回到交互式工具。

1.6.4.9. 完整的构建示例

让我们用一个完整的例子串联上述内容。假设你克隆了一个 RoboMaster 视觉项目：

```
1 # 克隆项目
2 git clone https://github.com/example/rm_vision.git
3 cd rm_vision
4
5 # 创建构建目录
6 mkdir build && cd build
7
8 # 首次配置：使用 Ninja, Release 模式，启用 CUDA
9 cmake -G Ninja \
10     -DCMAKE_BUILD_TYPE=Release \
11     -DENABLE_CUDA=ON \
12     -DCMAKE_INSTALL_PREFIX=/opt/rm_vision \
13     ..
14
15 # 如果配置失败，查看详细信息
16 cmake -G Ninja -DCMAKE_BUILD_TYPE=Release .. --debug-find
17
18 # 构建（并行）
19 cmake --build . -j
20
21 # 运行测试
22 ctest --output-on-failure
23
24 # 安装
25 sudo cmake --install .
```

日常开发流程：

```
1 # 修改代码后，只需构建
2 cmake --build . -j
3
4 # 如果修改了 CMakeLists.txt，会自动重新配置
5 cmake --build . -j
6
7 # 切换到 Debug 模式调试问题
8 cd ..
9 mkdir build-debug && cd build-debug
10 cmake -G Ninja -DCMAKE_BUILD_TYPE=Debug ..
11 cmake --build . -j
12
```



```
13 # 使用 GDB 调试
14 gdb ./rm_vision
```

遇到问题时的排查：

```
1 # 查看详细的编译命令
2 cmake --build . --verbose
3
4 # 查看缓存变量
5 cmake -LA ..
6
7 # 完全重新开始
8 cd ..
9 rm -rf build
10 mkdir build && cd build
11 cmake -G Ninja -DCMAKE_BUILD_TYPE=Release ..
12 cmake --build . -j
```

掌握了这些构建流程，你能够高效地管理 CMake 项目的构建过程。无论是个人开发还是团队协作，统一的构建流程都是代码质量的重要保障。

1.6.5. 目标与属性

如果说 CMake 的语法是它的词汇和语法，那么“目标”（Target）就是它的核心思想。现代 CMake（通常指 CMake 3.0 以后的最佳实践）围绕目标组织一切：编译选项、包含路径、链接依赖都附加在目标上，而不是设置全局变量。这种以目标为中心的方式让构建配置更加清晰、模块化，也让依赖管理变得优雅。理解目标和属性是掌握现代 CMake 的关键。

1.6.5.1. 什么是目标

在 CMake 中，目标是构建系统要生成的东西。最常见的目标类型是可执行文件和库：

```
1 # 创建可执行文件目标
2 add_executable(rm_vision src/main.cpp)
3
4 # 创建库目标
5 add_library(rm_core src/core.cpp)
```

执行这些命令后，`rm_vision` 和 `rm_core` 就成为了 CMake 中的目标。你可以把目标想象成一个对象，它有自己的属性（编译选项、包含路径、链接的库等），这些属性决定了如何构建这个目标。

目标的类型包括：

- **可执行文件**：`add_executable` 创建，最终生成可运行的程序
- **静态库**：`add_library(name STATIC ...)` 创建，生成 `.a` 文件
- **共享库**：`add_library(name SHARED ...)` 创建，生成 `.so` 文件
- **接口库**：`add_library(name INTERFACE)` 创建，不生成实际文件，只用于传递属性
- **对象库**：`add_library(name OBJECT ...)` 创建，生成目标文件但不打包
- **导入目标**：表示外部已存在的库，由 `find_package` 创建

还有一种特殊的“目标”是自定义目标，用 `add_custom_target` 创建，它不对应任何文件，而是执行自定义命令：

```
1 # 自定义目标，用于生成文档
2 add_custom_target(docs
```

```

3     COMMAND doxygen ${CMAKE_SOURCE_DIR}/Doxyfile
4     WORKING_DIRECTORY ${CMAKE_SOURCE_DIR}
5     COMMENT "Generating documentation..."
6 )

```

目标之间可以建立依赖关系。当目标 A 链接目标 B 时，CMake 会确保 B 在 A 之前构建，并且 B 的某些属性会传递给 A。这种依赖关系形成了一个有向无环图，CMake 根据这个图来决定构建顺序和属性传递。

1.6.5.2. 目标属性

每个目标都有一组属性 (properties)，这些属性控制目标的构建方式。你可以把属性想象成目标的配置项。常见的属性包括：

- `INCLUDE_DIRECTORIES`：头文件搜索路径
- `COMPILE_DEFINITIONS`：预处理器宏定义
- `COMPILE_OPTIONS`：编译器选项
- `COMPILE_FEATURES`：需要的 C++ 特性
- `LINK_LIBRARIES`：链接的库
- `CXX_STANDARD`：C++ 标准版本
- `OUTPUT_NAME`：输出文件名
- `POSITION_INDEPENDENT_CODE`：是否生成位置无关代码

可以用 `set_target_properties` 直接设置属性：

```

1 add_executable(my_app main.cpp)
2
3 set_target_properties(my_app PROPERTIES
4     CXX_STANDARD 17
5     CXX_STANDARD_REQUIRED ON
6     OUTPUT_NAME "my_application"
7     RUNTIME_OUTPUT_DIRECTORY "${CMAKE_BINARY_DIR}/bin"
8 )

```

也可以用 `get_target_property` 获取属性值：

```

1 get_target_property(std_version my_app CXX_STANDARD)
2 message(STATUS "C++ standard: ${std_version}")

```

但在日常使用中，我们更多地使用 `target_*` 系列命令来设置属性，因为它们更加直观，而且支持属性的传递性（稍后详细讨论）。

1.6.5.3. `target_include_directories`：头文件路径

`target_include_directories` 为目标添加头文件搜索路径，相当于编译器的 `-I` 选项：

```

1 add_library(rm_core src/core.cpp)
2
3 target_include_directories(rm_core PUBLIC
4     ${CMAKE_CURRENT_SOURCE_DIR}/include
5 )

```

这告诉编译器在编译 `rm_core` 时，到 `include` 目录下搜索头文件。

可以指定多个路径：

```

1 target_include_directories(rm_core PUBLIC
2     ${CMAKE_CURRENT_SOURCE_DIR}/include
3     ${CMAKE_CURRENT_SOURCE_DIR}/third_party/json/include
4 )

```

路径可以是绝对路径或相对路径。使用 `CMAKE_CURRENT_SOURCE_DIR` 构造的路径可以确保在任何构建目录下都能正确找到头文件。

对于需要区分构建时和安装后路径的情况，可以使用生成器表达式：

```

1 target_include_directories(rm_core PUBLIC
2     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
3     $<INSTALL_INTERFACE:include>
4 )

```

这表示：在构建时使用源码目录下的 `include`，安装后使用安装目录下的 `include`。这在编写可供他人使用的库时非常重要。

1.6.5.4. target_compile_definitions: 预处理宏

`target_compile_definitions` 为目标添加预处理器宏定义，相当于编译器的 `-D` 选项：

```

1 add_executable(rm_vision src/main.cpp)
2
3 # 定义宏
4 target_compile_definitions(rm_vision PRIVATE
5     DEBUG_MODE
6     MAX_THREADS=8
7     PROJECT_VERSION="${PROJECT_VERSION}"
8 )

```

上面的代码相当于在编译时添加 `-DDEBUG_MODE -DMAX_THREADS=8 -DPROJECT_VERSION="1.0.0"`。

在代码中可以使用这些宏：

```

1 #ifdef DEBUG_MODE
2     std::cout << "Debug mode enabled" << std::endl;
3 #endif
4
5 for (int i = 0; i < MAX_THREADS; ++i) {
6     // ...
7 }
8
9 std::cout << "Version: " << PROJECT_VERSION << std::endl;

```

条件编译是宏定义的常见用途：

```

1 option(ENABLE_CUDA "Enable CUDA support" OFF)
2
3 add_library(rm_core src/core.cpp)
4
5 if(ENABLE_CUDA)
6     target_compile_definitions(rm_core PUBLIC WITH_CUDA)
7 endif()

```

代码中：

```

1 void process(const Image& image) {

```

```

2  #ifdef WITH_CUDA
3      cuda_process(image); // 使用 CUDA 加速
4  #else
5      cpu_process(image); // CPU 回退
6  #endif
7  }

```

注意不要在宏名中使用 `-D` 前缀，CMake 会自动添加。也不要不要在值周围加引号，除非你确实需要引号成为值的一部分：

```

1  # 正确
2  target_compile_definitions(app PRIVATE MY_MACRO=42)
3
4  # 错误：会定义为 -D-DMY_MACRO=42
5  target_compile_definitions(app PRIVATE -DMY_MACRO=42)
6
7  # 定义字符串值时需要转义引号
8  target_compile_definitions(app PRIVATE MY_STRING="hello")

```

1.6.5.5. target_compile_options: 编译选项

`target_compile_options` 为目标添加编译器选项：

```

1  add_executable(rm_vision src/main.cpp)
2
3  target_compile_options(rm_vision PRIVATE
4      -Wall           # 开启常见警告
5      -Wextra         # 开启额外警告
6      -Wpedantic      # 严格标准检查
7      -Werror         # 警告视为错误
8  )

```

不同编译器的选项可能不同，可以使用条件判断或生成器表达式处理：

```

1  # 方式一：条件判断
2  if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU" OR CMAKE_CXX_COMPILER_ID STREQUAL
3      "Clang")
4      target_compile_options(rm_vision PRIVATE -Wall -Wextra)
5  elseif(CMAKE_CXX_COMPILER_ID STREQUAL "MSVC")
6      target_compile_options(rm_vision PRIVATE /W4)
7  endif()
8  # 方式二：生成器表达式（更简洁）
9  target_compile_options(rm_vision PRIVATE
10      $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra>
11      $<$<CXX_COMPILER_ID:MSVC>:/W4>
12  )

```

生成器表达式 `$<condition:value>` 的含义是：如果 `condition` 为真，则展开为 `value`，否则展开为空。

常见的编译选项包括：

```

1  target_compile_options(my_target PRIVATE
2      # 警告控制
3      -Wall -Wextra -Wpedantic
4
5      # 优化（通常由 CMAKE_BUILD_TYPE 控制，这里仅作演示）
6      $<$<CONFIG:Release>:-O3>

```

```

7     $<$<CONFIG:Debug>:-O0 -g>
8
9     # 特定架构优化
10    -march=native # 针对当前 CPU 优化（注意可移植性）
11
12    # 安全性
13    -fstack-protector-strong
14
15    # 调试信息
16    $<$<CONFIG:Debug>:-fsanitize=address,undefined>
17 )
18
19 # 如果使用 sanitizer, 链接时也需要添加
20 target_link_options(my_target PRIVATE
21     $<$<CONFIG:Debug>:-fsanitize=address,undefined>
22 )

```

1.6.5.6. target_compile_features: C++ 标准特性

`target_compile_features` 声明目标需要的 C++ 特性, CMake 会自动选择合适的编译器选项来启用这些特性:

```

1 add_library(rm_core src/core.cpp)
2
3 # 要求 C++17 标准
4 target_compile_features(rm_core PUBLIC cxx_std_17)

```

`cxx_std_17` 是一个元特性, 表示需要完整的 C++17 支持。类似的还有 `cxx_std_11`、`cxx_std_14`、`cxx_std_20`、`cxx_std_23` 等。

你也可以指定具体的特性:

```

1 target_compile_features(rm_core PUBLIC
2     cxx_auto_type           # auto 关键字
3     cxx_range_for           # 范围 for 循环
4     cxx_nullptr             # nullptr
5     cxx_lambdas              # lambda 表达式
6     cxx_variadic_templates  # 变参模板
7 )

```

但通常直接使用 `cxx_std_XX` 更简单, 不需要逐一列出特性。

与直接设置 `CMAKE_CXX_STANDARD` 相比, `target_compile_features` 的优势是:

1. 它是目标级别的, 不是全局的
2. 它支持 PUBLIC/PRIVATE/INTERFACE, 可以传递给依赖者
3. CMake 会检查编译器是否支持所需特性

```

1 # 旧方式: 全局设置
2 set(CMAKE_CXX_STANDARD 17)
3 set(CMAKE_CXX_STANDARD_REQUIRED ON)
4
5 # 新方式: 目标级别设置 (推荐)
6 add_library(rm_core src/core.cpp)
7 target_compile_features(rm_core PUBLIC cxx_std_17)

```

当你使用 `PUBLIC` 时, 所有链接 `rm_core` 的目标也会自动要求 C++17, 这正是我们通常想要的行为。

1.6.5.7. target_link_libraries: 链接库

`target_link_libraries` 是最重要的命令之一，它声明目标的链接依赖：

```
1 add_library(rm_core src/core.cpp)
2 add_executable(rm_vision src/main.cpp)
3
4 # rm_vision 链接 rm_core
5 target_link_libraries(rm_vision PRIVATE rm_core)
```

链接一个目标会做两件事：

1. 在链接阶段，将被链接的库链接到目标中
2. 根据可见性（PUBLIC/PRIVATE/INTERFACE），传递被链接库的属性

可以链接多个库：

```
1 target_link_libraries(rm_vision PRIVATE
2     rm_core
3     rm_detector
4     rm_tracker
5 )
```

链接外部库也使用同样的命令：

```
1 find_package(OpenCV REQUIRED)
2 find_package(Eigen3 REQUIRED)
3
4 add_executable(rm_vision src/main.cpp)
5
6 # 现代方式：使用导入目标
7 target_link_libraries(rm_vision PRIVATE
8     OpenCV::OpenCV
9     Eigen3::Eigen
10 )
11
12 # 旧方式：使用变量（不推荐）
13 target_link_libraries(rm_vision PRIVATE
14     ${OpenCV_LIBRARIES}
15 )
16 target_include_directories(rm_vision PRIVATE
17     ${OpenCV_INCLUDE_DIRS}
18 )
```

现代方式（使用导入目标如 `OpenCV::OpenCV`）更简洁，因为导入目标携带了所有必要的信息（头文件路径、编译选项、依赖库等），只需一行 `target_link_libraries` 就够了。

链接系统库：

```
1 # 链接 pthread
2 find_package(Threads REQUIRED)
3 target_link_libraries(my_app PRIVATE Threads::Threads)
4
5 # 链接数学库
6 target_link_libraries(my_app PRIVATE m)
7
8 # 链接动态加载库
9 target_link_libraries(my_app PRIVATE dl)
```

1.6.5.8. PUBLIC、PRIVATE 和 INTERFACE

理解 PUBLIC、PRIVATE 和 INTERFACE 是掌握现代 CMake 的关键。它们控制属性的可见性和传递性。

让我们用一个具体的例子来解释。假设有三个目标：

- `json_parser`：一个 JSON 解析库
- `config_loader`：使用 `json_parser` 的配置加载库
- `app`：使用 `config_loader` 的应用程序

```
1 add_library(json_parser src/json_parser.cpp)
2 add_library(config_loader src/config_loader.cpp)
3 add_executable(app src/main.cpp)
4
5 target_link_libraries(config_loader ??? json_parser)
6 target_link_libraries(app PRIVATE config_loader)
```

问题是：`config_loader` 链接 `json_parser` 时，应该用 PRIVATE、PUBLIC 还是 INTERFACE？

PRIVATE 表示：这个依赖只在当前目标内部使用，不暴露给依赖当前目标的其他目标。

如果 `config_loader` 的头文件中没有使用 `json_parser` 的任何类型，只在 `.cpp` 文件中使用：

```
1 // config_loader.h - 头文件中不暴露 json_parser
2 #pragma once
3 #include <string>
4 #include <map>
5
6 class ConfigLoader {
7 public:
8     std::map<std::string, std::string> Load(const std::string& path);
9 };
10
11 // config_loader.cpp - 实现中使用 json_parser
12 #include "config_loader.h"
13 #include <json_parser/json.h> // 只在实现中使用
14
15 std::map<std::string, std::string> ConfigLoader::Load(const std::string&
16 path) {
17     JsonDocument doc = JsonParser::Parse(path);
18     // ...
19 }
```

这种情况下应该用 **PRIVATE**：

```
1 target_link_libraries(config_loader PRIVATE json_parser)
```

`app` 链接 `config_loader` 时，不会自动链接 `json_parser`，因为它被隐藏了。

PUBLIC 表示：这个依赖既在当前目标内部使用，也暴露给依赖当前目标的其他目标。

如果 `config_loader` 的头文件中使用了 `json_parser` 的类型：

```
1 // config_loader.h - 头文件中暴露了 json_parser 的类型
2 #pragma once
3 #include <json_parser/json.h> // 头文件中包含
4
5 class ConfigLoader {
```

```

6 public:
7     JsonDocument LoadRaw(const std::string& path); // 返回 json_parser 的类型
8 };

```

这种情况下应该用 `PUBLIC`：

```

1 target_link_libraries(config_loader PUBLIC json_parser)

```

`app` 链接 `config_loader` 时，会自动链接 `json_parser`，因为 `app` 要编译包含 `config_loader.h` 的代码，而 `config_loader.h` 包含了 `json_parser/json.h`。

INTERFACE 表示：这个依赖只暴露给依赖当前目标的其他目标，当前目标本身不使用。这听起来有点奇怪，什么时候会用到？最常见的场景是 header-only 库：

```

1 # header-only 库，没有源文件需要编译
2 add_library(my_header_lib INTERFACE)
3
4 target_include_directories(my_header_lib INTERFACE
5     ${CMAKE_CURRENT_SOURCE_DIR}/include
6 )
7
8 target_compile_features(my_header_lib INTERFACE cxx_std_17)

```

因为 `my_header_lib` 是纯接口库，没有实现代码，所以所有属性都是 **INTERFACE**——它们只对使用这个库的目标有意义。

另一个场景是，当你想给依赖者添加额外的要求，但自己不需要：

```

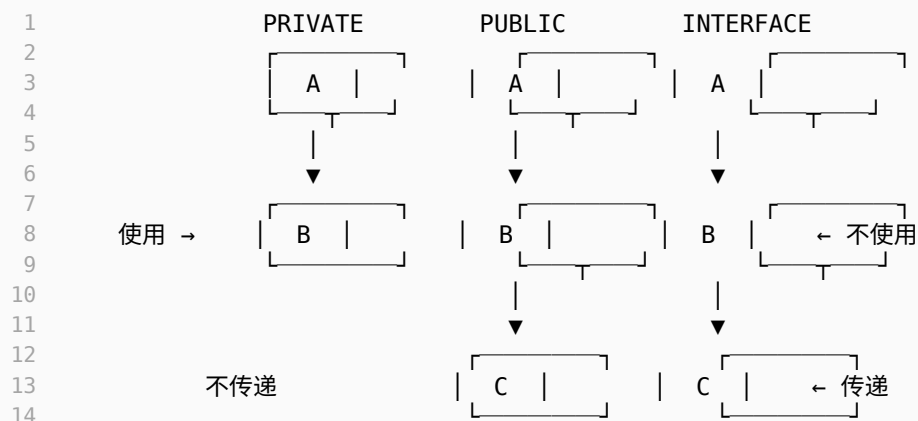
1 add_library(wrapper src/wrapper.cpp)
2
3 # wrapper 自己不需要 C++17，但使用 wrapper 的代码需要
4 target_compile_features(wrapper INTERFACE cxx_std_17)

```

让我们用表格总结：

	当前目标使用	传递给依赖者
PRIVATE	✓	✗
PUBLIC	✓	✓
INTERFACE	✗	✓

再用一个更直观的图示：



1.6.5.9. 传递性依赖

现代 CMake 的强大之处在于传递性依赖 (transitive dependencies)。当你链接一个目标时，不仅仅是链接那个库，还会自动获得它的 PUBLIC 和 INTERFACE 属性。

考虑这个例子：

```
1 # 底层库
2 add_library(math_utils src/math.cpp)
3 target_include_directories(math_utils PUBLIC include)
4 target_compile_features(math_utils PUBLIC cxx_std_17)
5
6 # 中层库，依赖 math_utils
7 add_library(geometry src/geometry.cpp)
8 target_link_libraries(geometry PUBLIC math_utils)
9 target_include_directories(geometry PUBLIC include)
10
11 # 上层库，依赖 geometry
12 add_library(renderer src/renderer.cpp)
13 target_link_libraries(renderer PUBLIC geometry)
14 target_include_directories(renderer PUBLIC include)
15
16 # 应用程序，只需要链接 renderer
17 add_executable(app src/main.cpp)
18 target_link_libraries(app PRIVATE renderer)
```

虽然 `app` 只显式链接了 `renderer`，但它自动获得了：

- `renderer`、`geometry`、`math_utils` 的头文件路径
- `renderer`、`geometry`、`math_utils` 的链接
- C++17 标准要求（来自 `math_utils`）

这就是传递性依赖的威力。你不需要手动追踪依赖链，CMake 会自动处理。

对比旧式的 CMake 写法：

```
1 # 旧方式：手动设置所有依赖（繁琐且容易出错）
2 add_executable(app src/main.cpp)
3
4 target_include_directories(app PRIVATE
5     ${CMAKE_SOURCE_DIR}/math_utils/include
6     ${CMAKE_SOURCE_DIR}/geometry/include
7     ${CMAKE_SOURCE_DIR}/renderer/include
8 )
9
10 target_link_libraries(app PRIVATE
11     math_utils
12     geometry
13     renderer
14 )
15
16 set_target_properties(app PROPERTIES CXX_STANDARD 17)
```

旧方式需要知道整个依赖链的细节，而且当依赖变化时，需要修改所有使用者。现代方式只需要声明直接依赖，其他的自动传递。

1.6.5.10. 实际项目中的应用

让我们看一个 RoboMaster 项目的完整示例：

```
1 cmake_minimum_required(VERSION 3.16)
```

```

2 project(rm_vision VERSION 2.0.0 LANGUAGES CXX)
3
4 # 查找依赖
5 find_package(OpenCV REQUIRED)
6 find_package(Eigen3 REQUIRED)
7 find_package(Ceres REQUIRED)
8
9 #=====
10 # 核心库：公共工具和类型定义
11 #=====
12 add_library(rm_common
13     src/common/config.cpp
14     src/common/logger.cpp
15     src/common/timer.cpp
16 )
17
18 target_include_directories(rm_common PUBLIC
19     ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
20     ${INSTALL_INTERFACE:include}
21 )
22
23 target_compile_features(rm_common PUBLIC cxx_std_17)
24
25 target_link_libraries(rm_common PUBLIC
26     Eigen3::Eigen # PUBLIC: 头文件暴露 Eigen 类型
27 )
28
29 #=====
30 # 检测模块
31 #=====
32 add_library(rm_detector
33     src/detector/armor_detector.cpp
34     src/detector/rune_detector.cpp
35     src/detector/nn_detector.cpp
36 )
37
38 target_include_directories(rm_detector PUBLIC
39     ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
40     ${INSTALL_INTERFACE:include}
41 )
42
43 target_link_libraries(rm_detector
44     PUBLIC rm_common # PUBLIC: 接口暴露 rm_common 的类型
45     PRIVATE OpenCV::OpenCV # PRIVATE: 只在实现中使用 OpenCV
46 )
47
48 #=====
49 # 跟踪模块
50 #=====
51 add_library(rm_tracker
52     src/tracker/armor_tracker.cpp
53     src/tracker/kalman_filter.cpp
54     src/tracker/extended_kalman.cpp
55 )
56
57 target_include_directories(rm_tracker PUBLIC
58     ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
59     ${INSTALL_INTERFACE:include}
60 )
61
62 target_link_libraries(rm_tracker
63     PUBLIC rm_common
64     PUBLIC Eigen3::Eigen # PUBLIC: 滤波器接口暴露 Eigen

```

```

65     PRIVATE Ceres::ceres      # PRIVATE: 只在优化实现中使用
66 )
67
68 #=====
69 # 预测模块
70 #=====
71 add_library(rm_predictor
72     src/predictor/motion_predictor.cpp
73     src/predictor/ballistic_solver.cpp
74 )
75
76 target_include_directories(rm_predictor PUBLIC
77     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
78     $<INSTALL_INTERFACE:include>
79 )
80
81 target_link_libraries(rm_predictor
82     PUBLIC rm_common
83     PUBLIC rm_tracker # PUBLIC: 预测器接口使用跟踪器的类型
84 )
85
86 #=====
87 # 主程序
88 #=====
89 add_executable(rm_vision src/main.cpp)
90
91 # 只需要链接直接依赖, 其他自动传递
92 target_link_libraries(rm_vision PRIVATE
93     rm_detector
94     rm_predictor
95     OpenCV::OpenCV # 主程序也需要 OpenCV 进行图像采集
96 )

```

在这个例子中:

- `rm_common` 是基础模块, 暴露 Eigen 类型, 所以 `Eigen3::Eigen` 是 PUBLIC
- `rm_detector` 内部使用 OpenCV 处理图像, 但接口不暴露 OpenCV 类型, 所以是 PRIVATE
- `rm_tracker` 的接口返回 Eigen 矩阵表示的状态, 所以 `Eigen3::Eigen` 是 PUBLIC
- `rm_predictor` 依赖 `rm_tracker`, 接口使用跟踪器的类型, 所以是 PUBLIC
- `rm_vision` 主程序是最终产物, 不会被其他目标依赖, 所以全部是 PRIVATE

遵循这些原则可以让你的 CMake 配置更加清晰、正确, 也让依赖管理变得简单。当某个库的依赖发生变化时, 只需要修改那个库的 CMakeLists.txt, 而不需要修改所有使用者。

1.6.5.11. 诊断依赖关系

当依赖关系变得复杂时, 你可能想知道某个目标实际链接了哪些库、使用了哪些头文件路径。CMake 提供了一些方法来诊断:

```

1 # 查看目标的属性
2 get_target_property(includes rm_vision INCLUDE_DIRECTORIES)
3 message(STATUS "rm_vision includes: ${includes}")
4
5 get_target_property(libs rm_vision LINK_LIBRARIES)
6 message(STATUS "rm_vision links: ${libs}")

```

更详细的方式是使用 `cmake --graphviz` 生成依赖图:

```

1 # 生成依赖图
2 cmake --graphviz=deps.dot ..
3
4 # 使用 Graphviz 渲染
5 dot -Tpng deps.dot -o deps.png

```

或者使用 `cmake --trace` 查看 CMake 的执行过程。

理解并正确使用目标和属性，是编写高质量 CMake 配置的基础。它让构建系统更加模块化、可维护，也让依赖管理变得优雅。如果你发现自己在使用全局变量（如 `CMAKE_CXX_FLAGS`、`include_directories()`）来设置编译选项，考虑改用目标级别的命令（`target_compile_options`、`target_include_directories`）——这是现代 CMake 的推荐做法。

1.6.6. 多文件项目组织

到目前为止，我们的例子大多是简单的单目录项目。但真实的项目往往有几十甚至上百个源文件，分布在多个目录中，划分为若干个模块。如何组织这些文件，如何编写对应的 CMake 配置，是每个开发者都要面对的问题。好的项目结构可以让代码更容易理解、更容易维护，也让构建配置更加清晰。本节将介绍多文件项目的组织方式和相应的 CMake 技巧。

1.6.6.1. 典型的项目目录结构

在深入 CMake 配置之前，让我们先看看 C++ 项目的常见目录结构。虽然没有强制的标准，但社区已经形成了一些广泛接受的约定。

一个典型的 C++ 项目结构如下：

```

1 my_project/
2 |— CMakeLists.txt      # 顶层 CMake 配置
3 |— README.md          # 项目说明
4 |— LICENSE            # 许可证
5 |— .gitignore         # Git 忽略规则
6 |
7 |— include/           # 公开头文件
8 |   |— my_project/
9 |       |— core.h
10 |       |— utils.h
11 |       |— config.h
12 |
13 |— src/               # 源文件和私有头文件
14 |   |— CMakeLists.txt  # 可选：子目录的 CMake 配置
15 |   |— core.cpp
16 |   |— utils.cpp
17 |   |— config.cpp
18 |   |— internal/      # 私有实现细节
19 |       |— impl.h
20 |       |— impl.cpp
21 |
22 |— apps/              # 可执行程序
23 |   |— CMakeLists.txt
24 |   |— main.cpp
25 |
26 |— tests/             # 测试代码
27 |   |— CMakeLists.txt
28 |   |— test_core.cpp
29 |   |— test_utils.cpp
30 |

```

```

31 |— examples/                # 示例代码
32 |   |— CMakeLists.txt
33 |   |— example_basic.cpp
34 |
35 |— docs/                    # 文档
36 |   |— Doxyfile
37 |
38 |— cmake/                   # CMake 模块和脚本
39 |   |— FindSomeLib.cmake
40 |   |— CompilerWarnings.cmake
41 |
42 |— third_party/             # 第三方依赖（如果不使用包管理器）
43 |   |— json/
44 |
45 |— scripts/                 # 辅助脚本
46 |   |— build.sh
47 |   |— format.sh

```

这个结构有几个关键的设计决策。首先是头文件和源文件的分离：公开头文件放在 `include/` 目录，源文件放在 `src/` 目录。其次是库代码和应用代码的分离：核心功能编译成库，可执行程序单独放在 `apps/` 目录。第三是测试代码独立：测试文件放在 `tests/` 目录，与主代码分开。

这种结构的好处是职责清晰。当你想找某个类的接口定义，去 `include/` 目录；想看实现细节，去 `src/` 目录；想运行程序，去 `apps/` 目录；想写测试，去 `tests/` 目录。新加入项目的人可以快速了解代码的组织方式。

注意 `include/` 下还有一层以项目名命名的目录（`include/my_project/`）。这样做是为了避免头文件名冲突。用户包含头文件时会写 `#include <my_project/core.h>` 而不是 `#include <core.h>`，即使他们的项目中也有一个 `core.h` 也不会冲突。

实际项目中，还有一种常见的“模块化”结构，以 RoboMaster 视觉项目 RMCV 为例：

```

1 | RMCV/
2 |   |— CMakeLists.txt      # 顶层 CMake 配置
3 |   |— main.cpp            # 主程序入口
4 |
5 |   |— aimer/              # 自动瞄准模块
6 |       |— CMakeLists.txt  # 模块配置（仅包含子目录）
7 |       |— common/        # 公共子模块
8 |           |— CMakeLists.txt
9 |           |— transformer/ # 坐标变换
10 |          |— filter/      # 滤波器
11 |          |— math/        # 数学工具
12 |       |— auto_aim/      # 自瞄核心
13 |           |— CMakeLists.txt
14 |           |— detector/   # 目标检测
15 |               |— CMakeLists.txt
16 |               |— detector_node.cpp
17 |               |— detector_rv/ # 传统视觉检测器
18 |           |— predictor/  # 运动预测
19 |               |— CMakeLists.txt
20 |               |— enemy_state/ # 状态估计
21 |               |— enemy_model/ # 运动模型
22 |           |— fire_control/ # 火控系统
23 |
24 |   |— hardware/           # 硬件抽象层
25 |       |— CMakeLists.txt
26 |       |— hardware_node.cpp
27 |       |— hik_cam/       # 海康相机驱动

```

```

28 |   └── serial/                # 串口通信
29 |
30 |   └── plugin/               # 插件系统
31 |       ├── CMakeLists.txt
32 |       ├── debug/          # 日志模块
33 |       ├── param/          # 参数管理
34 |       └── rmcv_bag/        # 数据录制
35 |
36 |   └── umt/                  # 线程通信框架 (header-only)
37 |       ├── Message.hpp
38 |       └── ObjManager.hpp
39 |
40 |   └── config/               # 配置文件
41 |       ├── camera.yaml
42 |       └── aimer.toml
43 |
44 |   └── test/                 # 测试程序
45 |       ├── test_param.cpp
46 |       ├── test_camera.cpp
47 |       └── time_sync/

```

这种结构没有单独的 `include/` 目录，而是每个模块内部组织自己的头文件。头文件和源文件放在一起，通过 `target_include_directories` 暴露给其他模块。这种方式在嵌入式和机器人项目中很常见，因为模块通常不会被外部项目复用。

1.6.6.2. `add_subdirectory`：包含子目录

当项目变大时，把所有 CMake 配置都写在一个文件里会变得难以维护。CMake 支持将配置分散到多个 CMakeLists.txt 文件中，每个子目录可以有自己的配置文件。`add_subdirectory` 命令用于包含子目录。

```

1 # 顶层 CMakeLists.txt
2 cmake_minimum_required(VERSION 3.16)
3 project(RMCV)
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # 查找依赖
8 find_package(OpenCV REQUIRED)
9 find_package(fmt REQUIRED)
10 find_package(Eigen3 REQUIRED)
11
12 # 全局 include 路径
13 include_directories(${PROJECT_SOURCE_DIR})
14 include_directories(umt) # header-only 库
15
16 # 按依赖顺序添加子目录
17 add_subdirectory(plugin) # 基础设施，无依赖
18 add_subdirectory(hardware) # 依赖 plugin
19 add_subdirectory(aimer) # 依赖 plugin, hardware
20
21 # 主程序
22 add_executable(RMCV2026 main.cpp)
23 target_link_libraries(RMCV2026
24     plugin hardware detector predictor
25     ${OpenCV_LIBS} fmt::fmt
26 )

```

当 CMake 执行 `add_subdirectory(plugin)` 时，它会进入 `plugin` 目录，读取并处理那里的 `CMakeLists.txt`，然后返回继续处理当前文件。子目录的 `CMakeLists.txt` 可以定义目标、设置变量，这些目标在整个项目中都是可见的。

`add_subdirectory` 还可以指定一个可选的二进制目录，用于存放该子目录的构建产物：

```
1 # 将 src 目录的构建产物放到 build/lib 下
2 add_subdirectory(src ${CMAKE_BINARY_DIR}/lib)
```

如果你想包含项目外部的目录，需要指定二进制目录：

```
1 # 包含外部目录
2 add_subdirectory(/path/to/external/lib external_lib_build)
```

`add_subdirectory` 的处理顺序很重要。如果目录 B 的目标依赖于目录 A 的目标，那么 A 应该在 B 之前被包含。不过，由于 CMake 会先完整解析所有配置文件，再进行构建，所以只要最终依赖关系是正确的，即使顺序不对通常也能工作。但为了清晰，推荐按照依赖顺序排列。

1.6.6.3. 子目录的 CMakeLists.txt

子目录的 `CMakeLists.txt` 文件通常比较简洁，只关注该目录的目标定义。它不需要再次调用 `cmake_minimum_required` 和 `project`，因为这些已经在顶层设置过了。

```
1 # plugin/CMakeLists.txt - 基础插件库
2
3 # 查找依赖（可以在子目录中重复声明，CMake会缓存）
4 find_package(OpenCV REQUIRED)
5 find_package(tomlplusplus CONFIG REQUIRED)
6 find_package(fmt REQUIRED)
7
8 # 收集源文件
9 aux_source_directory(./debug debug_src)
10 aux_source_directory(./param param_src)
11 aux_source_directory(./plotter plotter_src)
12
13 # 创建库
14 add_library(plugin STATIC ${debug_src} ${param_src} ${plotter_src})
15
16 # 设置头文件路径
17 target_include_directories(plugin PUBLIC
18     ${CMAKE_CURRENT_SOURCE_DIR}
19     ${OpenCV_INCLUDE_DIRS}
20 )
21
22 # 链接依赖
23 target_link_libraries(plugin PUBLIC
24     ${OpenCV_LIBS}
25     Eigen3::Eigen
26     tomlplusplus::tomlplusplus
27     fmt::fmt
28 )
29
30 # 包含子模块
31 add_subdirectory(rmcv_bag)
```

注意这里使用了 `CMAKE_CURRENT_SOURCE_DIR` 而不是 `PROJECT_SOURCE_DIR`。`PROJECT_SOURCE_DIR` 始终指向顶层项目目录（包含 `project()` 命令的那个），而 `CMAKE_CURRENT_SOURCE_DIR` 指向当前正在处理的 `CMakeLists.txt` 所在目录。在子目录中，它们是不同的。

硬件层的 CMakeLists.txt 展示了如何组织多个子模块：

```
1 # hardware/CMakeLists.txt
2
3 # 添加子模块
4 add_subdirectory(hik_cam)
5 add_subdirectory(serial)
6
7 # 创建硬件节点库（聚合库）
8 add_library(hardware STATIC
9     hardware_node.cpp
10 )
11
12 target_include_directories(hardware PUBLIC
13     ${CMAKE_CURRENT_SOURCE_DIR}
14 )
15
16 # 链接所有硬件子库
17 target_link_libraries(hardware PUBLIC
18     hardware_camera # 来自 hik_cam/
19     hardware_serial # 来自 serial/
20     ${OpenCV_LIBS}
21     fmt::fmt
22     plugin          # 依赖基础设施
23 )
```

测试的 CMakeLists.txt：

```
1 # tests/CMakeLists.txt
2
3 # 查找 GTest
4 find_package(GTest REQUIRED)
5
6 # 启用测试
7 enable_testing()
8
9 # 创建测试可执行文件
10 add_executable(test_core test_core.cpp)
11 target_link_libraries(test_core PRIVATE
12     my_project_core
13     GTest::gtest_main
14 )
15
16 add_executable(test_utils test_utils.cpp)
17 target_link_libraries(test_utils PRIVATE
18     my_project_core
19     GTest::gtest_main
20 )
21
22 # 注册测试
23 include(GoogleTest)
24 gtest_discover_tests(test_core)
25 gtest_discover_tests(test_utils)
```

你可能注意到测试目录的 CMakeLists.txt 有自己的 `find_package` 调用。这完全没问题——CMake 会缓存查找结果，多次调用不会有性能问题。这样做的好处是每个子目录都是相对自包含的，可以独立理解。

1.6.6.4. 纯转发的 CMakeLists.txt

有时候一个目录只是用来组织子目录，本身不产生任何目标。这时候 CMakeLists.txt 可以非常简洁：


```

1 # aimer/CMakeLists.txt - 纯转发
2
3 add_subdirectory(common)
4 add_subdirectory(auto_aim)

```

```

1 # aimer/auto_aim/CMakeLists.txt - 纯转发
2
3 add_subdirectory(detector)
4 add_subdirectory(predictor)
5 add_subdirectory(fire_control)

```

这种“纯转发”的 CMakeLists.txt 让目录结构更清晰，每个功能模块都在自己的子目录中定义目标。

1.6.6.5. 条件包含子目录

有时候你希望某些子目录是可选的，比如测试代码。可以使用选项来控制：

```

1 # 顶层 CMakeLists.txt
2 cmake_minimum_required(VERSION 3.16)
3 project(my_project VERSION 1.0.0 LANGUAGES CXX)
4
5 # 选项
6 option(BUILD_TESTS "Build unit tests" ON)
7 option(BUILD_EXAMPLES "Build examples" OFF)
8
9 # 核心库（始终构建）
10 add_subdirectory(src)
11 add_subdirectory(apps)
12
13 # 可选子目录
14 if(BUILD_TESTS)
15     enable_testing()
16     add_subdirectory(tests)
17 endif()
18
19 if(BUILD_EXAMPLES)
20     add_subdirectory(examples)
21 endif()

```

用户可以在配置时选择是否构建测试：

```

1 # 不构建测试
2 cmake -DBUILD_TESTS=OFF ..
3
4 # 构建测试（默认）
5 cmake ..

```

以 RMCV 为例，检测器模块有可选的 YOLO 支持：

```

1 # aimer/auto_aim/detector/CMakeLists.txt
2
3 # 检测器选项
4 option(ENABLE_YOLO_DETECTOR "Enable YOLO detector (requires OpenVINO)" OFF)
5
6 # 传统检测器（始终编译）
7 add_subdirectory(detector_rv)
8
9 # 可选：YOLO 检测器
10 if(ENABLE_YOLO_DETECTOR)

```

```

11     add_subdirectory(detector_yolo)
12     message(STATUS "YOLO detector: ENABLED")
13 else()
14     message(STATUS "YOLO detector: DISABLED")
15 endif()
16
17 # 创建 detector_node 库
18 add_library(detector_node STATIC detector_node.cpp)
19
20 target_link_libraries(detector_node PUBLIC
21     detector_traditional
22     plugin hardware aimer_common
23 )
24
25 # 条件链接和编译宏
26 if(ENABLE_YOLO_DETECTOR)
27     target_compile_definitions(detector_node PUBLIC ENABLE_YOLO_DETECTOR)
28     target_link_libraries(detector_node PUBLIC detector_yolo)
29 endif()
30
31 # 创建统一接口库
32 add_library(detector INTERFACE)
33 target_link_libraries(detector INTERFACE detector_traditional detector_node)

```

1.6.6.6. 头文件组织：公开与私有

头文件的组织是项目结构中的重要决策。一个核心问题是：哪些头文件是公开的（库的用户需要使用），哪些是私有的（只在库内部使用）。

公开头文件定义了库的接口，包括用户需要使用的类、函数、常量等。它们应该放在 `include/` 目录下，用户可以通过 `#include <my_project/xxx.h>` 来包含。公开头文件应该尽量简洁，只暴露必要的接口，隐藏实现细节。

```

1 // include/my_project/detector.h - 公开头文件
2 #pragma once
3
4 #include <vector>
5 #include <opencv2/core.hpp>
6
7 namespace my_project {
8
9 struct Armor {
10     cv::Point2f center;
11     float confidence;
12     int id;
13 };
14
15 class ArmorDetector {
16 public:
17     explicit ArmorDetector(const std::string& config_path);
18     ~ArmorDetector();
19
20     std::vector<Armor> Detect(const cv::Mat& image);
21
22 private:
23     class Impl; // Pimpl 模式，隐藏实现
24     std::unique_ptr<Impl> impl_;
25 };
26
27 } // namespace my_project

```

私有头文件是库内部使用的，用户不需要也不应该包含它们。它们可以放在 `src/` 目录下，与实现文件放在一起。私有头文件可以包含实现细节、内部数据结构、辅助函数等。

```
1 // src/internal/nn_backend.h - 私有头文件
2 #pragma once
3
4 #include <onnxruntime/core/session/onnxruntime_cxx_api.h>
5
6 namespace my_project::internal {
7
8     class NNBackend {
9     public:
10         explicit NNBackend(const std::string& model_path);
11         std::vector<float> Infer(const std::vector<float>& input);
12
13     private:
14         Ort::Env env_;
15         Ort::Session session_;
16         // 更多实现细节...
17     };
18
19 } // namespace my_project::internal
```

在 CMake 中体现这种区分：

```
1 add_library(my_project_detector
2     src/detector/armor_detector.cpp
3     src/detector/nn_backend.cpp
4 )
5
6 target_include_directories(my_project_detector
7     PUBLIC
8         # 公开头文件：用户可见
9         ${BUILD_INTERFACE:${PROJECT_SOURCE_DIR}/include}
10        ${INSTALL_INTERFACE:include}
11     PRIVATE
12         # 私有头文件：只在库内部可见
13         ${CMAKE_CURRENT_SOURCE_DIR}/src
14 )
```

这样配置后，库自己可以 `#include "internal/nn_backend.h"`（相对于 `src/` 目录），而链接这个库的用户只能 `#include <my_project/detector.h>`（相对于 `include/` 目录）。

为什么要区分公开和私有头文件？几个原因。首先是封装性：隐藏实现细节可以防止用户依赖内部结构，让你可以自由修改实现而不破坏用户代码。其次是编译隔离：私有头文件的变化只影响库本身的重新编译，不会波及用户代码。第三是清晰的接口：公开头文件就是库的“说明书”，用户只需要看这些文件就知道如何使用库。

1.6.6.7. 库与可执行文件的分离

一个好的实践是将核心功能编译成库，可执行文件只是库的一个“使用者”。这种分离有几个好处：

首先，代码更容易测试。库的每个功能都可以被测试代码独立调用和验证，而不需要通过主程序的入口。

其次，代码更容易复用。如果将来有另一个程序需要相同的功能，直接链接库即可，不需要复制代码。

第三，编译更高效。修改主程序的代码时，不需要重新编译库；修改库的内部实现时，只需要重新编译库和链接。

以 RMCV 为例，核心功能在各个库中实现：

```
1 # 主程序
2 add_executable(RMCV2026 main.cpp)
3 target_link_libraries(RMCV2026
4     plugin hardware detector predictor rmcv_bag
5     ${OpenCV_LIBS} fmt::fmt
6 )
7
8 # 测试程序 - 每个测试针对特定模块
9 add_executable(test_param test/test_param.cpp)
10 target_link_libraries(test_param ${OpenCV_LIBS} fmt::fmt plugin)
11
12 add_executable(test_serial test/test_serial.cpp)
13 target_link_libraries(test_serial ${OpenCV_LIBS} fmt::fmt plugin hardware)
14
15 add_executable(test_camera test/test_camera.cpp)
16 target_link_libraries(test_camera ${OpenCV_LIBS} fmt::fmt plugin hardware)
17
18 add_executable(test_transformer test/test_transformer.cpp)
19 target_link_libraries(test_transformer ${OpenCV_LIBS} fmt::fmt aimer_common)
20
21 add_executable(test_ballistic test/test_ballistic.cpp)
22 target_link_libraries(test_ballistic ${OpenCV_LIBS} fmt::fmt plugin hardware
23     aimer_common)
24
25 # 回放测试 - 需要完整的检测和预测模块
26 add_executable(test_playback test/test_playback.cpp)
27 target_link_libraries(test_playback
28     plugin hardware detector predictor rmcv_bag
29     ${OpenCV_LIBS} fmt::fmt
30 )
```

注意每个测试程序只链接它需要的库。`test_param` 只测试参数系统，所以只链接 `plugin`；`test_transformer` 测试坐标变换，所以链接 `aimer_common`。这样可以加快编译速度，也让依赖关系更清晰。

主程序的代码应该尽量精简，主要负责：解析命令行参数、初始化配置、创建对象、运行主循环。核心逻辑都应该在库中实现。

```
1 // main.cpp
2 #include <csignal>
3 #include "plugin/debug/debug.hpp"
4 #include "plugin/param/runtime_parameter.hpp"
5 #include "hardware/hardware_node.hpp"
6 #include "aimer/auto_aim/detector/detector_node.hpp"
7 #include "aimer/auto_aim/predictor/predictor_node.hpp"
8
9 volatile bool running = true;
10
11 void signal_handler(int) { running = false; }
12
13 int main() {
14     // 信号处理
15     std::signal(SIGINT, signal_handler);
16
17     // 初始化
18     debug::init_session();
19     runtime_param::parameter_run("aimer.toml");
```

```

20     tf::init();
21
22     // 启动各模块线程
23     hardware::start();
24     detector::start();
25     predictor::start();
26
27     // 主循环
28     while (running) {
29         std::this_thread::sleep_for(std::chrono::milliseconds(100));
30     }
31
32     // 清理
33     debug::print("info", "main", "Shutting down...");
34     return 0;
35 }

```

1.6.6.8. file(GLOB) 的使用与争议

当源文件很多时，手动在 CMakeLists.txt 中列出每个文件很繁琐。CMake 的 `file(GLOB)` 命令可以自动收集匹配特定模式的文件：

```

1 # 收集所有 .cpp 文件
2 file(GLOB SOURCES "src/*.cpp")
3
4 # 递归收集（包括子目录）
5 file(GLOB_RECURSE SOURCES "src/*.cpp")
6
7 # 使用收集到的文件列表
8 add_library(my_lib ${SOURCES})

```

这看起来很方便，但 CMake 官方文档明确不推荐用 `file(GLOB)` 来收集源文件。原因是：CMake 的配置阶段和构建阶段是分开的。`file(GLOB)` 在配置阶段执行，收集当时存在的文件。如果你之后添加或删除了源文件，但没有重新运行 CMake 配置，构建系统不会知道文件列表变化了。这可能导致新文件没有被编译，或者删除的文件还在使用旧的目标文件。

```

1 # 初始配置
2 cmake ..
3 cmake --build .
4
5 # 添加新文件 src/new_feature.cpp
6 # 如果直接构建，新文件不会被编译！
7 cmake --build . # new_feature.cpp 被忽略
8
9 # 需要重新配置
10 cmake ..
11 cmake --build . # 现在才会编译 new_feature.cpp

```

手动列出源文件虽然繁琐，但更可靠。每次添加或删除文件时，你必须修改 CMakeLists.txt，这会自动触发重新配置：

```

1 # 推荐：手动列出源文件
2 add_library(my_lib
3     src/core.cpp
4     src/utils.cpp
5     src/config.cpp
6     src/feature_a.cpp
7     src/feature_b.cpp

```

如果你确实想用 `file(GLOB)`，CMake 3.12 引入了 `CONFIGURE_DEPENDS` 选项，让 CMake 在每次构建时检查文件是否变化：

```
1 file(GLOB_RECURSE SOURCES CONFIGURE_DEPENDS "src/*.cpp")
2 add_library(my_lib ${SOURCES})
```

但这有几个问题。首先，不是所有生成器都支持这个特性。其次，每次构建都要扫描文件系统，会有一些性能开销。第三，即使支持，在某些边缘情况下仍可能出问题。

一个折中的方案是用 `aux_source_directory`，它比 `file(GLOB)` 稍微安全一些：

```
1 # plugin/CMakeLists.txt
2 aux_source_directory(./debug debug_src)
3 aux_source_directory(./param param_src)
4 aux_source_directory(./plotter plotter_src)
5
6 add_library(plugin STATIC ${debug_src} ${param_src} ${plotter_src})
```

`aux_source_directory` 只收集 CMake 认为是源文件的扩展名（.c, .cpp, .cc 等），不会意外包含头文件。但它同样有“添加新文件需要重新配置”的问题。

对于大多数项目，使用 `aux_source_directory` 是可以接受的折中方案。但如果你使用 `file(GLOB_RECURSE)` 递归收集源文件，要格外小心。

1.6.6.9. 处理第三方 SDK

有些第三方库（如海康相机 SDK）不提供 CMake 配置，需要手动处理：

```
1 # hardware/hik_cam/CMakeLists.txt
2
3 # SDK 安装路径
4 set(hikrobot "/opt/MVS")
5
6 # 收集源文件
7 aux_source_directory(. hik_camera_src)
8
9 # 创建相机库
10 add_library(hardware_camera STATIC ${hik_camera_src})
11
12 # 设置头文件路径
13 target_include_directories(hardware_camera PUBLIC ${hikrobot}/include/)
14
15 # 根据架构选择库目录
16 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64")
17     target_link_directories(hardware_camera PUBLIC ${hikrobot}/lib/64/)
18 elseif(CMAKE_SYSTEM_PROCESSOR MATCHES "aarch64")
19     target_link_directories(hardware_camera PUBLIC ${hikrobot}/lib/aarch64/)
20 else()
21     message(FATAL_ERROR "Unsupported architecture:
22     ${CMAKE_SYSTEM_PROCESSOR}!")
23 endif()
24
25 # 链接 SDK 库
26 target_link_libraries(hardware_camera PUBLIC
27     MvCameraControl # HIK SDK 库
28     ${OpenCV_LIBS})
29 )
```

这个例子展示了几个重要技巧：

1. 使用 `CMAKE_SYSTEM_PROCESSOR` 检测 CPU 架构
2. 使用 `target_link_directories` 指定库搜索路径
3. 使用 `message(FATAL_ERROR ...)` 在不支持的情况下终止配置

1.6.6.10. 条件编译可选功能

有些功能可能依赖于可选的第三方库。可以使用 `find_package` 的结果来决定是否编译：

```
1 # 顶层 CMakeLists.txt
2
3 # 相机-IMU 时间戳标定（需要 Ceres）
4 find_package(Ceres QUIET)
5 if(Ceres_FOUND)
6     add_executable(test_time_sync test/time_sync/test_time_sync.cpp)
7     target_include_directories(test_time_sync PRIVATE
8         ${PROJECT_SOURCE_DIR}/test
9         ${CERES_INCLUDE_DIRS}
10    )
11    target_link_libraries(test_time_sync
12        ${OpenCV_LIBS} fmt::fmt
13        plugin hardware detector aimer_common
14        ${CERES_LIBRARIES}
15    )
16    message(STATUS "test_time_sync will be built (Ceres found)")
17
18    # 相机外参自动标定（也需要 Ceres）
19    add_executable(test_extrinsic_calib test/extrinsic_calib/
20        test_extrinsic_calib.cpp)
21    target_link_libraries(test_extrinsic_calib
22        ${OpenCV_LIBS} fmt::fmt
23        plugin hardware aimer_common
24        ${CERES_LIBRARIES}
25    )
26 else()
27     message(STATUS "test_time_sync will NOT be built (Ceres not found)")
28
29     # 外参标定可以用网格搜索替代
30    add_executable(test_extrinsic_calib test/extrinsic_calib/
31        test_extrinsic_calib.cpp)
32    target_link_libraries(test_extrinsic_calib
33        ${OpenCV_LIBS} fmt::fmt
34        plugin hardware aimer_common
35    )
36    message(STATUS "test_extrinsic_calib will be built (without Ceres, grid
37        search only)")
38 endif()
39
40 # 模拟器测试（需要 ROS2 环境）
41 if(SIMULATOR_FOUND)
42     add_executable(test_simulator test/test_simulator.cpp)
43     target_link_libraries(test_simulator
44         plugin hardware simulator detector predictor
45         ${OpenCV_LIBS} fmt::fmt
46     )
47     message(STATUS "test_simulator will be built (ROS2 found)")
48 endif()
```

`find_package(Ceres QUIET)` 中的 `QUIET` 选项表示如果找不到包，不要打印警告信息。这对于可选依赖很有用——找到了就编译，找不到就跳过。

1.6.6.11. 模块化的目录结构

对于更大的项目，可以采用模块化的目录结构，每个功能模块是一个独立的目录，有自己完整的 include、src 和 CMakeLists.txt:

```
1 rm_vision/
2 |— CMakeLists.txt
3
4 |— common/                # 公共模块
5 |   |— CMakeLists.txt
6 |   |— include/common/
7 |       |— types.h
8 |       |— config.h
9 |       |— logger.h
10 |   |— src/
11 |       |— config.cpp
12 |       |— logger.cpp
13
14 |— detector/              # 检测模块
15 |   |— CMakeLists.txt
16 |   |— include/detector/
17 |       |— armor_detector.h
18 |       |— rune_detector.h
19 |   |— src/
20 |       |— armor_detector.cpp
21 |       |— rune_detector.cpp
22 |       |— internal/
23 |           |— nn_backend.cpp
24
25 |— tracker/              # 跟踪模块
26 |   |— CMakeLists.txt
27 |   |— include/tracker/
28 |       |— armor_tracker.h
29 |       |— kalman_filter.h
30 |   |— src/
31 |       |— armor_tracker.cpp
32 |       |— kalman_filter.cpp
33
34 |— predictor/            # 预测模块
35 |   |— CMakeLists.txt
36 |   |— include/predictor/
37 |       |— motion_predictor.h
38 |   |— src/
39 |       |— motion_predictor.cpp
40
41 |— apps/                 # 应用程序
42 |   |— CMakeLists.txt
43 |   |— rm_vision_node.cpp
44
45 |— tests/                # 测试
46 |   |— CMakeLists.txt
47 |   |— test_detector.cpp
48 |   |— test_tracker.cpp
```

顶层 CMakeLists.txt:

```
1 cmake_minimum_required(VERSION 3.16)
2 project(rm_vision VERSION 2.0.0 LANGUAGES CXX)
3
4 # 全局设置
5 set(CMAKE_CXX_STANDARD 17)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
```



```

7  set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9  # 选项
10 option(BUILD_TESTS "Build unit tests" ON)
11
12 # 依赖
13 find_package(OpenCV REQUIRED)
14 find_package(Eigen3 REQUIRED)
15
16 # 子模块（按依赖顺序）
17 add_subdirectory(common)
18 add_subdirectory(detector)
19 add_subdirectory(tracker)
20 add_subdirectory(predictor)
21
22 # 应用程序
23 add_subdirectory(apps)
24
25 # 测试
26 if(BUILD_TESTS)
27     enable_testing()
28     add_subdirectory(tests)
29 endif()

```

common 模块的 CMakeLists.txt:

```

1  # common/CMakeLists.txt
2
3  add_library(rm_common
4      src/config.cpp
5      src/logger.cpp
6  )
7
8  target_include_directories(rm_common PUBLIC
9      $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
10     $<INSTALL_INTERFACE:include>
11 )
12
13 target_compile_features(rm_common PUBLIC cxx_std_17)
14
15 # common 模块通常依赖较少
16 target_link_libraries(rm_common PUBLIC
17     Eigen3::Eigen
18 )

```

detector 模块的 CMakeLists.txt:

```

1  # detector/CMakeLists.txt
2
3  add_library(rm_detector
4      src/armor_detector.cpp
5      src/rune_detector.cpp
6      src/internal/nn_backend.cpp
7  )
8
9  target_include_directories(rm_detector
10     PUBLIC
11         $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
12         $<INSTALL_INTERFACE:include>
13     PRIVATE
14         ${CMAKE_CURRENT_SOURCE_DIR}/src
15 )

```

```

16
17 target_link_libraries(rm_detector
18     PUBLIC rm_common
19     PRIVATE ${OpenCV_LIBS}
20 )

```

apps 目录的 CMakeLists.txt:

```

1 # apps/CMakeLists.txt
2
3 add_executable(rm_vision_node rm_vision_node.cpp)
4
5 target_link_libraries(rm_vision_node PRIVATE
6     rm_detector
7     rm_tracker
8     rm_predictor
9     ${OpenCV_LIBS} # 主程序需要 OpenCV 进行图像采集
10 )

```

这种模块化结构的优点是每个模块相对独立, 可以单独理解和测试。模块之间的依赖关系也很清晰: common 是基础, detector 依赖 common, tracker 依赖 detector 和 common, predictor 依赖 tracker。

1.6.6.12. 全局编译选项

对于需要在所有目标中生效的编译选项, 可以使用全局设置:

```

1 # 顶层 CMakeLists.txt
2
3 # 编译器警告
4 add_compile_options(-Wall -Werror=return-type -Wno-unused-variable)
5
6 # 性能分析选项
7 option(ENABLE_PROFILING "Enable profiling support for VTune/perf" OFF)
8
9 if(ENABLE_PROFILING)
10     message(STATUS "Profiling mode enabled")
11     add_compile_options(-O3 -g -fno-omit-frame-pointer -march=native)
12     # 不使用 LTO, 便于符号解析
13 elseif(CMAKE_BUILD_TYPE STREQUAL Release)
14     add_compile_options(-O3 -flto -march=native)
15 elseif(CMAKE_BUILD_TYPE STREQUAL RelWithDebInfo)
16     add_compile_options(-O3 -flto -march=native -g)
17 elseif(CMAKE_BUILD_TYPE STREQUAL MinSizeRel)
18     add_compile_options(-Os -flto -march=native)
19 else()
20     add_compile_options(-O0 -g)
21 endif()
22
23 # 全局编译定义 - 所有目标都会继承
24 set(ASSET_DIR "${CMAKE_SOURCE_DIR}/asset")
25 set(CONFIG_DIR "${CMAKE_SOURCE_DIR}/config")
26 set(LOG_DIR "${CMAKE_SOURCE_DIR}/log")
27
28 add_compile_definitions(
29     ASSET_DIR="${ASSET_DIR}"
30     CONFIG_DIR="${CONFIG_DIR}"
31     LOG_DIR="${LOG_DIR}"
32 )

```

这样, 代码中可以直接使用这些宏来获取目录路径:

```

1 // 代码中使用
2 std::string config_path = std::string(CONFIG_DIR) + "/aimer.toml";

```

1.6.6.13. ccache 加速编译

对于大型项目，使用 ccache 可以显著加速重复编译：

```

1 # 顶层 CMakeLists.txt
2
3 find_program(CCACHE_PROGRAM ccache)
4 if(CCACHE_PROGRAM)
5     set_property(GLOBAL PROPERTY RULE_LAUNCH_COMPILE "${CCACHE_PROGRAM}")
6     message(STATUS "ccache found, compilation will be cached")
7 endif()

```

ccache 会缓存编译结果，当源文件没有变化时，直接使用缓存的目标文件，可以将编译时间从几分钟缩短到几秒钟。

1.6.6.14. 使用 INTERFACE 库

有时候你想创建一个“虚拟”库，它本身不产生任何编译产物，只是用来聚合其他库或传递编译选项：

```

1 # aimer/auto_aim/detector/CMakeLists.txt
2
3 # 创建统一接口库
4 add_library(detector INTERFACE)
5
6 target_include_directories(detector INTERFACE
7     ${CMAKE_CURRENT_SOURCE_DIR}
8     ${CMAKE_CURRENT_SOURCE_DIR}/common
9 )
10
11 # 链接实际的检测器实现
12 target_link_libraries(detector INTERFACE
13     detector_traditional
14     detector_node
15 )

```

用户只需要链接 `detector`，就自动获得了所有检测器相关的库。

1.6.6.15. CMake 辅助模块

随着项目增长，你可能会发现一些 CMake 代码在多处重复。可以将这些代码提取到单独的 CMake 模块中，放在 `cmake/` 目录下：

```

1 # cmake/CompilerWarnings.cmake
2
3 # 定义一个函数，为目标启用警告
4 function(enable_warnings target)
5     target_compile_options(${target} PRIVATE
6         $<$<CXX_COMPILER_ID:GNU,Clang>:
7             -Wall
8             -Wextra
9             -Wpedantic
10            -Wcast-align
11            -Wunused
12            -Woverloaded-virtual
13            -Wnon-virtual-dtor

```

```

14         >
15         ${CXX_COMPILER_ID:MSVC}:
16         /W4
17         /permissive-
18         >
19     )
20 endfunction()

```

在顶层 CMakeLists.txt 中包含这个模块：

```

1 # 添加 cmake/ 目录到模块搜索路径
2 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
3
4 # 包含自定义模块
5 include(CompilerWarnings)
6
7 # 在子目录中使用
8 # enable_warnings(my_target)

```

子目录中就可以简单地调用：

```

1 add_library(rm_detector ...)
2 enable_warnings(rm_detector)

```

这种方式可以让构建配置更加 DRY (Don't Repeat Yourself)，也便于在多个项目之间共享 CMake 代码。

1.6.6.16. 打印依赖信息

调试 CMake 配置时，打印依赖库的详细信息很有帮助：

```

1 # 顶层 CMakeLists.txt 末尾
2
3 message(STATUS "----- Dependency Debug Information -----")
4
5 # OpenCV
6 if(OpenCV_FOUND)
7     message(STATUS "OpenCV Version: ${OpenCV_VERSION}")
8     message(STATUS "OpenCV Libraries: ${OpenCV_LIBS}")
9     message(STATUS "OpenCV Include Dirs: ${OpenCV_INCLUDE_DIRS}")
10 endif()
11
12 # fmt
13 if(TARGET fmt::fmt)
14     get_target_property(FMT_INCLUDE_DIR fmt::fmt
15 INTERFACE_INCLUDE_DIRECTORIES)
16     get_target_property(FMT_LIBRARY_LOCATION fmt::fmt IMPORTED_LOCATION)
17     message(STATUS "fmt Library: ${FMT_LIBRARY_LOCATION}")
18     message(STATUS "fmt Include Dirs: ${FMT_INCLUDE_DIR}")
19 endif()
20
21 # Eigen3
22 if(Eigen3_FOUND)
23     message(STATUS "Eigen3 Include Dir: ${EIGEN3_INCLUDE_DIR}")
24 endif()
25
26 # tomlplusplus
27 if(TARGET tomlplusplus::tomlplusplus)
28     get_target_property(TOMLPP_INCLUDE_DIR tomlplusplus::tomlplusplus
29 INTERFACE_INCLUDE_DIRECTORIES)
30     message(STATUS "tomlplusplus Include Dirs: ${TOMLPP_INCLUDE_DIR}")

```

```

29 endif()
30
31 message(STATUS "-----")

```

这些信息在排查“找不到头文件”或“链接失败”等问题时非常有用。

1.6.6.17. 使用命名空间组织代码

配合目录结构，代码中也应该使用命名空间来组织：

```

1 // common/include/common/types.h
2 #pragma once
3
4 namespace rm {
5
6 struct Armor {
7     // ...
8 };
9
10 struct Pose {
11     // ...
12 };
13
14 } // namespace rm

```

```

1 // detector/include/detector/armor_detector.h
2 #pragma once
3
4 #include <common/types.h>
5
6 namespace rm::detector {
7
8 class ArmorDetector {
9 public:
10     std::vector<Armor> Detect(const cv::Mat& image);
11 };
12
13 } // namespace rm::detector

```

```

1 // tracker/include/tracker/armor_tracker.h
2 #pragma once
3
4 #include <common/types.h>
5 #include <detector/armor_detector.h>
6
7 namespace rm::tracker {
8
9 class ArmorTracker {
10 public:
11     void Update(const std::vector<Armor>& detections);
12     Pose GetPrediction() const;
13 };
14
15 } // namespace rm::tracker

```

用户代码使用时：

```

1 #include <detector/armor_detector.h>
2 #include <tracker/armor_tracker.h>
3
4 int main() {

```

```

5     rm::detector::ArmorDetector detector;
6     rm::tracker::ArmorTracker tracker;
7
8     // 或者使用 using
9     using namespace rm;
10    detector::ArmorDetector detector2;
11
12    // ...
13 }

```

命名空间可以层级化，也可以扁平化，取决于项目的复杂度和团队偏好。对于较小的项目，单层命名空间（如 `rm::ArmorDetector`）就够了；对于大型项目，模块级命名空间（如 `rm::detector::ArmorDetector`）可以避免名字冲突。

好的项目结构是代码质量的基础。它让新人能够快速理解项目，让日常开发更加高效，也让构建配置更加清晰可维护。虽然没有放之四海而皆准的“最佳”结构，但遵循社区的常见约定、保持一致性、根据项目需求做出合理的选择，就能建立起一个健康的代码库。

1.6.7. 查找与链接外部库

几乎每个 C++ 项目都会使用外部库。对于 RoboMaster 视觉项目，OpenCV 处理图像、Eigen 进行矩阵运算、Ceres 解决优化问题——这些第三方库是开发的基石。如何在 CMake 中正确地找到这些库、链接它们，是每个开发者必须掌握的技能。本节将详细介绍 CMake 的依赖管理机制，从基础的 `find_package` 到现代的 FetchContent，让你能够灵活地处理各种依赖场景。

1.6.7.1. find_package: 查找已安装的库

`find_package` 是 CMake 中查找外部库的标准方法。它会在系统中搜索指定的库，如果找到，就设置一系列变量供后续使用。

```

1 # 基本用法
2 find_package(OpenCV REQUIRED)

```

REQUIRED 表示这个库是必需的，如果找不到，CMake 会报错并终止配置。如果库是可选的，可以省略 **REQUIRED**：

```

1 # 可选依赖
2 find_package(CUDA QUIET) # QUIET 表示找不到时不输出警告
3
4 if(CUDA_FOUND)
5     message(STATUS "CUDA found, enabling GPU acceleration")
6     # 使用 CUDA...
7 else()
8     message(STATUS "CUDA not found, using CPU only")
9 endif()

```

可以指定版本要求：

```

1 # 要求至少 4.5.0 版本
2 find_package(OpenCV 4.5.0 REQUIRED)
3
4 # 要求精确版本
5 find_package(OpenCV 4.5.4 EXACT REQUIRED)

```

可以指定需要的组件：

```

1 # 只需要特定组件
2 find_package(OpenCV REQUIRED COMPONENTS core imgproc highgui)
3
4 # 有些组件是可选的
5 find_package(Boost REQUIRED COMPONENTS filesystem system)
6 find_package(Boost OPTIONAL_COMPONENTS python)

```

当 `find_package` 成功时，它会设置一系列变量。传统方式下，这些变量遵循一定的命名约定：

```

1 find_package(OpenCV REQUIRED)
2
3 # 常见的变量
4 message(STATUS "OpenCV found: ${OpenCV_FOUND}")
5 message(STATUS "OpenCV version: ${OpenCV_VERSION}")
6 message(STATUS "OpenCV include dirs: ${OpenCV_INCLUDE_DIRS}")
7 message(STATUS "OpenCV libraries: ${OpenCV_LIBRARIES}")

```

但不同的库设置的变量可能略有不同，有的用 `OpenCV_LIBS`，有的用 `OPENCV_LIBRARIES`。这种不一致性是传统方式的一个问题。

1.6.7.2. Config 模式与 Module 模式

`find_package` 有两种工作模式：Config 模式和 Module 模式。理解它们的区别有助于排查找不到库的问题。

Module 模式使用 CMake 自带的或项目提供的 `FindXXX.cmake` 文件来查找库。这些文件通常位于 CMake 的模块目录或项目的 `cmake/` 目录中。CMake 自带了许多常用库的 Find 模块，如 `FindThreads.cmake`、`FindOpenGL.cmake` 等。

```

1 # CMake 自带的 Find 模块
2 find_package(Threads REQUIRED) # 使用 FindThreads.cmake
3 find_package(OpenGL REQUIRED) # 使用 FindOpenGL.cmake

```

Module 模式的查找逻辑由 Find 模块的作者编写，通常会搜索常见的安装位置、检查环境变量、调用 `pkg-config` 等。

Config 模式使用库自己提供的 `XXXConfig.cmake` 或 `xxx-config.cmake` 文件。这些文件由库的作者编写，与库一起安装。现代的 C++ 库大多提供 Config 文件。

```

1 # 使用库提供的 Config 文件
2 find_package(OpenCV REQUIRED) # 使用 OpenCVConfig.cmake
3 find_package(Eigen3 REQUIRED) # 使用 Eigen3Config.cmake
4 find_package(Ceres REQUIRED) # 使用 CeresConfig.cmake

```

Config 模式更可靠，因为配置文件由库的作者维护，与库的版本完全匹配。它也通常提供导入目标（稍后讨论），使用起来更方便。

CMake 默认先尝试 Config 模式，如果找不到再尝试 Module 模式。可以用 `CONFIG` 或 `MODULE` 关键字强制使用某种模式：

```

1 # 强制使用 Config 模式
2 find_package(OpenCV CONFIG REQUIRED)
3
4 # 强制使用 Module 模式
5 find_package(OpenGL MODULE REQUIRED)

```

如果 `find_package` 找不到库，可以通过设置 `CMAKE_PREFIX_PATH` 告诉 CMake 去哪里找：

```
1 # 命令行指定额外的搜索路径
2 cmake -DCMAKE_PREFIX_PATH="/opt/opencv;/opt/eigen" ..
```

```
1 # 或者在 CMakeLists.txt 中
2 list(APPEND CMAKE_PREFIX_PATH "/opt/opencv")
3 find_package(OpenCV REQUIRED)
```

对于 Config 模式，也可以设置 `XXX_DIR` 变量指向 Config 文件所在目录：

```
1 cmake -DOpenCV_DIR=/opt/opencv/lib/cmake/opencv4 ..
```

1.6.7.3. 导入目标：现代方式

传统的 `find_package` 使用变量来传递库的信息，使用时需要手动设置 include 路径和链接库：

```
1 # 旧方式：使用变量
2 find_package(OpenCV REQUIRED)
3
4 add_executable(my_app main.cpp)
5 target_include_directories(my_app PRIVATE ${OpenCV_INCLUDE_DIRS})
6 target_link_libraries(my_app PRIVATE ${OpenCV_LIBRARIES})
```

现代的 `find_package` 会创建导入目标 (imported target)，这是一个“虚拟”的 CMake 目标，封装了库的所有使用要求：

```
1 # 现代方式：使用导入目标
2 find_package(OpenCV REQUIRED)
3
4 add_executable(my_app main.cpp)
5 target_link_libraries(my_app PRIVATE OpenCV::OpenCV)
```

只需要一行 `target_link_libraries`，include 路径、编译选项、依赖的其他库都会自动传递。这就是为什么现代 CMake 更加简洁。

导入目标的命名通常遵循 `PackageName::TargetName` 的格式。一个包可能提供多个导入目标：

```
1 find_package(OpenCV REQUIRED)
2
3 # OpenCV 提供的导入目标
4 target_link_libraries(my_app PRIVATE
5     OpenCV::core      # 核心功能
6     OpenCV::imgproc   # 图像处理
7     OpenCV::highgui   # GUI
8     OpenCV::calib3d    # 相机标定
9     OpenCV::dnn        # 深度学习
10 )
11
12 # 或者使用便利目标链接所有组件
13 target_link_libraries(my_app PRIVATE OpenCV::OpenCV)
```


导入目标有几个显著优势。首先是自动传递依赖：如果 A 依赖 B，链接 A 时 B 会自动链接。其次是正确的属性传播：目标携带了所有必要的编译选项、include 路径、宏定义。第三是更好的错误检查：如果使用了不存在的目标名，CMake 会报错；而使用未定义的变量只会得到空值。

可以用 `if(TARGET ...)` 检查导入目标是否存在：

```
1 find_package(OpenCV QUIET)
2
3 if(TARGET OpenCV::OpenCV)
4     message(STATUS "OpenCV imported target available")
5     target_link_libraries(my_app PRIVATE OpenCV::OpenCV)
6 else()
7     message(WARNING "OpenCV not found or no imported targets")
8 endif()
```

1.6.7.4. 常用库的查找

让我们看看 RoboMaster 开发中常用库的查找方式。

OpenCV 是计算机视觉的基础库：

```
1 find_package(OpenCV REQUIRED)
2
3 # 查看版本
4 message(STATUS "OpenCV version: ${OpenCV_VERSION}")
5
6 # 链接（现代方式）
7 target_link_libraries(my_app PRIVATE OpenCV::OpenCV)
8
9 # 或者只链接需要的模块
10 target_link_libraries(my_app PRIVATE
11     OpenCV::core
12     OpenCV::imgproc
13     OpenCV::imgcodecs
14     OpenCV::videoio
15 )
```

Eigen 是 header-only 的线性代数库：

```
1 find_package(Eigen3 REQUIRED)
2
3 # Eigen 是 header-only, 只需要 include 路径
4 target_link_libraries(my_app PRIVATE Eigen3::Eigen)
```

注意包名是 **Eigen3** 不是 **Eigen**。由于 Eigen 是 header-only 的，“链接”它实际上只是添加 include 路径和编译选项，不会有实际的库文件被链接。

Ceres Solver 是非线性优化库：

```
1 find_package(Ceres REQUIRED)
2
3 target_link_libraries(my_app PRIVATE Ceres::ceres)
```

Ceres 依赖 Eigen 和其他库，但通过导入目标，这些依赖会自动处理。

Threads 是 POSIX 线程库（pthread）：

```
1 find_package(Threads REQUIRED)
2
3 target_link_libraries(my_app PRIVATE Threads::Threads)
```

`Threads::Threads` 是跨平台的，在 Linux 上链接 `pthread`，在 Windows 上使用 Windows 线程 API。

Google Test 是单元测试框架：

```
1 find_package(GTest REQUIRED)
2
3 add_executable(my_tests test_main.cpp)
4 target_link_libraries(my_tests PRIVATE
5     GTest::gtest
6     GTest::gtest_main # 提供 main 函数
7 )
8
9 # 或者使用 gmock
10 target_link_libraries(my_tests PRIVATE
11     GTest::gmock
12     GTest::gmock_main
13 )
```

Boost 是一个大型的 C++ 库集合：

```
1 find_package(Boost REQUIRED COMPONENTS filesystem system)
2
3 target_link_libraries(my_app PRIVATE
4     Boost::filesystem
5     Boost::system
6 )
7
8 # header-only 部分不需要链接
9 find_package(Boost REQUIRED)
10 target_link_libraries(my_app PRIVATE Boost::boost) # 只有 headers
```

fmt 是现代的格式化库：

```
1 find_package(fmt REQUIRED)
2
3 target_link_libraries(my_app PRIVATE fmt::fmt)
4
5 # header-only 模式（如果这样编译的话）
6 target_link_libraries(my_app PRIVATE fmt::fmt-header-only)
```

spdlog 是高性能日志库：

```
1 find_package(spdlog REQUIRED)
2
3 target_link_libraries(my_app PRIVATE spdlog::spdlog)
```

1.6.7.5. pkg-config: 查找没有 CMake 支持的库

有些库没有提供 CMake 配置文件，但提供了 `pkg-config` 支持（`.pc` 文件）。CMake 可以通过 `PkgConfig` 模块来使用它们：

```
1 find_package(PkgConfig REQUIRED)
2
3 # 查找库
4 pkg_check_modules(LIBUSB REQUIRED libusb-1.0)
5
6 # 使用（传统方式）
7 target_include_directories(my_app PRIVATE ${LIBUSB_INCLUDE_DIRS})
8 target_link_libraries(my_app PRIVATE ${LIBUSB_LIBRARIES})
```

```

9
10 # 或者创建导入目标 (CMake 3.6+)
11 pkg_check_modules(LIBUSB REQUIRED IMPORTED_TARGET libusb-1.0)
12 target_link_libraries(my_app PRIVATE PkgConfig::LIBUSB)

```

`pkg_check_modules` 会设置以下变量：

- `<PREFIX>_FOUND`：是否找到
- `<PREFIX>_INCLUDE_DIRS`：头文件目录
- `<PREFIX>_LIBRARIES`：库文件
- `<PREFIX>_LIBRARY_DIRS`：库文件目录
- `<PREFIX>_CFLAGS`：编译标志
- `<PREFIX>_LDFLAGS`：链接标志

使用 `IMPORTED_TARGET` 时会创建 `PkgConfig::<PREFIX>` 导入目标，这是推荐的方式。

1.6.7.6. FetchContent：下载并构建依赖

当依赖没有安装在系统中时，`FetchContent` 模块可以自动下载源代码并构建。这是管理依赖的现代方式，特别适合 header-only 库和小型依赖。

```

1 include(FetchContent)
2
3 # 声明依赖
4 FetchContent_Declare(
5     json
6     GIT_REPOSITORY https://github.com/nlohmann/json.git
7     GIT_TAG v3.11.2
8 )
9
10 # 下载并添加到构建
11 FetchContent_MakeAvailable(json)
12
13 # 使用
14 target_link_libraries(my_app PRIVATE nlohmann_json::nlohmann_json)

```

`FetchContent_Declare` 声明依赖的来源，可以是 Git 仓库、压缩包 URL 等：

```

1 # 从 Git 仓库
2 FetchContent_Declare(
3     fmt
4     GIT_REPOSITORY https://github.com/fmtlib/fmt.git
5     GIT_TAG 10.1.1
6 )
7
8 # 从 URL 下载压缩包
9 FetchContent_Declare(
10     googletest
11     URL https://github.com/google/googletest/archive/release-1.12.1.tar.gz
12     URL_HASH SHA256=...
13 )
14
15 # 从本地目录（用于开发）
16 FetchContent_Declare(
17     mylib
18     SOURCE_DIR /path/to/local/mylib
19 )

```

`FetchContent_MakeAvailable` 会下载源代码（如果还没下载），然后调用 `add_subdirectory` 将其添加到构建中。这意味着依赖项目的 `CMakeLists.txt` 会被执行，其定义的目标可以直接使用。

可以一次处理多个依赖：

```
1 include(FetchContent)
2
3 FetchContent_Declare(
4     fmt
5     GIT_REPOSITORY https://github.com/fmtlib/fmt.git
6     GIT_TAG 10.1.1
7 )
8
9 FetchContent_Declare(
10    spdlog
11    GIT_REPOSITORY https://github.com/gabime/spdlog.git
12    GIT_TAG v1.12.0
13 )
14
15 FetchContent_Declare(
16    json
17    GIT_REPOSITORY https://github.com/nlohmann/json.git
18    GIT_TAG v3.11.2
19 )
20
21 # 一次性获取所有依赖
22 FetchContent_MakeAvailable(fmt spdlog json)
23
24 # 使用
25 target_link_libraries(my_app PRIVATE
26     fmt::fmt
27     spdlog::spdlog
28     nlohmann_json::nlohmann_json
29 )
```

有时候需要在 `MakeAvailable` 之前设置一些选项来控制依赖的构建：

```
1 FetchContent_Declare(
2     googletest
3     GIT_REPOSITORY https://github.com/google/googletest.git
4     GIT_TAG v1.14.0
5 )
6
7 # 在 MakeAvailable 之前设置选项
8 set(BUILD_GMOCK OFF CACHE BOOL "" FORCE)
9 set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
10
11 FetchContent_MakeAvailable(googletest)
```

`FetchContent` 的优点是简单、自动化，适合管理少量依赖。缺点是每次配置都可能触发下载（虽然会缓存），而且会增加配置时间。对于大型依赖或需要特殊编译选项的库，可能不太合适。

1.6.7.7. ExternalProject：更复杂的外部项目

`ExternalProject` 是比 `FetchContent` 更强大但也更复杂的模块。它可以下载、配置、构建、安装外部项目，支持任何构建系统（不限于 CMake）。

```
1 include(ExternalProject)
2
```

```

3 ExternalProject_Add(
4     external_opencv
5     GIT_REPOSITORY https://github.com/opencv/opencv.git
6     GIT_TAG 4.8.0
7     CMAKE_ARGS
8         -DCMAKE_INSTALL_PREFIX=${CMAKE_BINARY_DIR}/opencv_install
9         -DBUILD_EXAMPLES=OFF
10        -DBUILD_TESTS=OFF
11        -DBUILD_DOCS=OFF
12 )

```

与 FetchContent 不同，ExternalProject 在构建阶段（而非配置阶段）执行，外部项目的目标不会直接暴露给主项目。这意味着你需要手动处理链接：

```

1 ExternalProject_Add(
2     external_opencv
3     GIT_REPOSITORY https://github.com/opencv/opencv.git
4     GIT_TAG 4.8.0
5     CMAKE_ARGS
6         -DCMAKE_INSTALL_PREFIX=${CMAKE_BINARY_DIR}/opencv_install
7         -DBUILD_EXAMPLES=OFF
8     INSTALL_DIR ${CMAKE_BINARY_DIR}/opencv_install
9 )
10
11 # 获取安装目录
12 ExternalProject_Get_Property(external_opencv INSTALL_DIR)
13
14 # 创建导入目标
15 add_library(opencv_imported INTERFACE)
16 add_dependencies(opencv_imported external_opencv)
17 target_include_directories(opencv_imported INTERFACE ${INSTALL_DIR}/include/
    opencv4)
18 target_link_directories(opencv_imported INTERFACE ${INSTALL_DIR}/lib)
19 target_link_libraries(opencv_imported INTERFACE opencv_core opencv_imgproc)
20
21 # 使用
22 target_link_libraries(my_app PRIVATE opencv_imported)

```

ExternalProject 的复杂性使得它主要用于以下场景：

- 需要从源码编译大型依赖（如 OpenCV、PCL）
- 依赖使用非 CMake 构建系统
- 需要对依赖进行补丁或自定义配置
- 超级构建（superbuild）模式

对于大多数情况，推荐优先使用 `find_package`（依赖已安装）或 `FetchContent`（需要自动下载）。

1.6.7.8. 系统库 vs 本地库

在开发中，你经常需要在系统安装的库和本地编译的库之间切换。系统库通常通过包管理器安装，稳定但可能版本较旧；本地库可以使用最新版本或自定义配置。

```

1 # 优先使用系统库，找不到时用 FetchContent
2 find_package(spdlog QUIET)
3
4 if(NOT spdlog_FOUND)
5     message(STATUS "spdlog not found, fetching from GitHub")
6     include(FetchContent)

```

```

7     FetchContent_Declare(
8         spdlog
9         GIT_REPOSITORY https://github.com/gabime/spdlog.git
10        GIT_TAG v1.12.0
11    )
12    FetchContent_MakeAvailable(spdlog)
13 endif()
14
15 target_link_libraries(my_app PRIVATE spdlog::spdlog)

```

可以用选项让用户选择:

```

1 option(USE_SYSTEM_SPDLOG "Use system spdlog instead of fetching" ON)
2
3 if(USE_SYSTEM_SPDLOG)
4     find_package(spdlog REQUIRED)
5 else()
6     include(FetchContent)
7     FetchContent_Declare(
8         spdlog
9         GIT_REPOSITORY https://github.com/gabime/spdlog.git
10        GIT_TAG v1.12.0
11    )
12    FetchContent_MakeAvailable(spdlog)
13 endif()
14
15 target_link_libraries(my_app PRIVATE spdlog::spdlog)

```

对于安装在非标准位置的库, 通过 `CMAKE_PREFIX_PATH` 指定:

```

1 # 使用本地编译的 OpenCV
2 cmake -DCMAKE_PREFIX_PATH=/home/user/opencv_build/install ..

```

1.6.7.9. 编写 Find 模块

如果某个库既没有 Config 文件也没有 pkg-config 支持, 你可能需要自己编写 Find 模块。以下是一个简单的例子:

```

1 # cmake/FindMyLib.cmake
2
3 # 搜索头文件
4 find_path(MYLIB_INCLUDE_DIR
5     NAMES mylib.h
6     PATHS
7         /usr/include
8         /usr/local/include
9         $ENV{MYLIB_ROOT}/include
10 )
11
12 # 搜索库文件
13 find_library(MYLIB_LIBRARY
14     NAMES mylib
15     PATHS
16         /usr/lib
17         /usr/local/lib
18         $ENV{MYLIB_ROOT}/lib
19 )
20
21 # 标准处理
22 include(FindPackageHandleStandardArgs)

```

```

23 find_package_handle_standard_args(MyLib
24     REQUIRED_VARS MYLIB_LIBRARY MYLIB_INCLUDE_DIR
25 )
26
27 # 创建导入目标
28 if(MyLib_FOUND AND NOT TARGET MyLib::MyLib)
29     add_library(MyLib::MyLib UNKNOWN IMPORTED)
30     set_target_properties(MyLib::MyLib PROPERTIES
31         IMPORTED_LOCATION "${MYLIB_LIBRARY}"
32         INTERFACE_INCLUDE_DIRECTORIES "${MYLIB_INCLUDE_DIR}"
33     )
34 endif()
35
36 # 隐藏内部变量
37 mark_as_advanced(MYLIB_INCLUDE_DIR MYLIB_LIBRARY)

```

使用这个 Find 模块：

```

1 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
2 find_package(MyLib REQUIRED)
3 target_link_libraries(my_app PRIVATE MyLib::MyLib)

```

1.6.7.10. 完整示例

让我们看一个 RoboMaster 项目的完整依赖配置：

```

1 cmake_minimum_required(VERSION 3.16)
2 project(rm_vision VERSION 2.0.0 LANGUAGES CXX)
3
4 # 添加自定义 Find 模块路径
5 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
6
7 #=====
8 # 必需依赖
9 #=====
10
11 # OpenCV: 图像处理
12 find_package(OpenCV 4.5 REQUIRED COMPONENTS
13     core imgproc imgcodecs videoio highgui calib3d dnn
14 )
15 message(STATUS "Found OpenCV ${OpenCV_VERSION}")
16
17 # Eigen: 线性代数
18 find_package(Eigen3 3.3 REQUIRED)
19 message(STATUS "Found Eigen ${Eigen3_VERSION}")
20
21 # Threads: 多线程
22 find_package(Threads REQUIRED)
23
24 #=====
25 # 可选依赖
26 #=====
27
28 # Ceres: 非线性优化 (可选)
29 find_package(Ceres QUIET)
30 if(Ceres_FOUND)
31     message(STATUS "Found Ceres ${Ceres_VERSION}, enabling advanced
32         optimization")
33     set(HAVE_CERES ON)
34 else()
35     message(STATUS "Ceres not found, some optimization features disabled")
36

```

```

35     set(HAVE_CERES OFF)
36 endif()
37
38 # CUDA: GPU 加速 (可选)
39 find_package(CUDA QUIET)
40 if(CUDA_FOUND)
41     message(STATUS "Found CUDA ${CUDA_VERSION}, enabling GPU acceleration")
42     set(HAVE_CUDA ON)
43 else()
44     message(STATUS "CUDA not found, using CPU only")
45     set(HAVE_CUDA OFF)
46 endif()
47
48 #=====
49 # 使用 FetchContent 获取的依赖
50 #=====
51
52 include(FetchContent)
53
54 # fmt: 格式化库
55 find_package(fmt QUIET)
56 if(NOT fmt_FOUND)
57     message(STATUS "Fetching fmt...")
58     FetchContent_Declare(
59         fmt
60         GIT_REPOSITORY https://github.com/fmtlib/fmt.git
61         GIT_TAG 10.1.1
62     )
63     FetchContent_MakeAvailable(fmt)
64 endif()
65
66 # spdlog: 日志库
67 find_package(spdlog QUIET)
68 if(NOT spdlog_FOUND)
69     message(STATUS "Fetching spdlog...")
70     FetchContent_Declare(
71         spdlog
72         GIT_REPOSITORY https://github.com/gabime/spdlog.git
73         GIT_TAG v1.12.0
74     )
75     set(SPDLOG_FMT_EXTERNAL ON CACHE BOOL "" FORCE)
76     FetchContent_MakeAvailable(spdlog)
77 endif()
78
79 # nlohmann/json: JSON 解析
80 FetchContent_Declare(
81     json
82     GIT_REPOSITORY https://github.com/nlohmann/json.git
83     GIT_TAG v3.11.2
84 )
85 FetchContent_MakeAvailable(json)
86
87 #=====
88 # 测试依赖
89 #=====
90
91 option(BUILD_TESTS "Build unit tests" ON)
92
93 if(BUILD_TESTS)
94     find_package(GTest QUIET)
95     if(NOT GTest_FOUND)
96         message(STATUS "Fetching GoogleTest...")
97         FetchContent_Declare(

```



```

98         googletest
99         GIT_REPOSITORY https://github.com/google/googletest.git
100        GIT_TAG v1.14.0
101    )
102    set(BUILD_GMOCK ON CACHE BOOL "" FORCE)
103    set(INSTALL_GTEST OFF CACHE BOOL "" FORCE)
104    FetchContent_MakeAvailable(googletest)
105  endif()
106 endif()
107
108 #=====
109 # 定义目标
110 #=====
111
112 add_library(rm_core
113     src/config.cpp
114     src/logger.cpp
115 )
116
117 target_include_directories(rm_core PUBLIC
118     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
119     $<INSTALL_INTERFACE:include>
120 )
121
122 target_compile_features(rm_core PUBLIC cxx_std_17)
123
124 target_link_libraries(rm_core
125     PUBLIC
126         Eigen3::Eigen
127         fmt::fmt
128         spdlog::spdlog
129         nlohmann_json::nlohmann_json
130     PRIVATE
131         Threads::Threads
132 )
133
134 # 条件链接可选依赖
135 if(HAVE_CERES)
136     target_link_libraries(rm_core PUBLIC Ceres::ceres)
137     target_compile_definitions(rm_core PUBLIC HAVE_CERES)
138 endif()
139
140 # 添加更多目标...
141 add_subdirectory(src)
142 add_subdirectory(apps)
143
144 if(BUILD_TESTS)
145     enable_testing()
146     add_subdirectory(tests)
147 endif()

```

这个例子展示了依赖管理的最佳实践：

1. 必需依赖使用 `REQUIRED`，确保找不到时报错
2. 可选依赖使用 `QUIET`，找不到时优雅降级
3. 小型依赖优先使用系统版本，找不到时用 `FetchContent`
4. 通过编译定义（`HAVE_CERES`）让代码知道哪些功能可用
5. 清晰的日志输出，让用户了解配置情况

正确管理依赖是 CMake 配置中最重要的部分之一。掌握了 `find_package` 和 `FetchContent`，你就能够处理大多数依赖场景，让项目在不同环境中都能顺利构建。

1.6.8. 安装与导出

到目前为止，我们学习的都是如何构建项目。但如果你写了一个有用的库，想让团队的其他成员或者更广泛的社区使用，该怎么办？他们需要能够在自己的项目中 `find_package` 你的库，然后像使用 OpenCV 或 Eigen 一样使用它。这就需要正确地安装和导出你的库。本节将介绍如何让你的库成为一个“好公民”——可以被正确安装到系统中，并被其他 CMake 项目方便地使用。

1.6.8.1. 为什么需要安装

在开发阶段，你可以直接在构建目录中运行程序，链接构建目录中的库。但这种方式有几个问题：构建目录的结构可能很乱，不适合分发；其他项目无法通过 `find_package` 找到你的库；每次重新构建都可能改变文件位置。

安装 (install) 解决了这些问题。它将构建产物（可执行文件、库、头文件等）复制到一个标准的、组织良好的目录结构中。安装后的文件可以被系统中的其他程序使用，也可以打包分发给其他人。

标准的 Unix 安装目录结构通常是：

```
1 /usr/local/                # 或其他 CMAKE_INSTALL_PREFIX
2 |   bin/                   # 可执行文件
3 |   lib/                   # 库文件
4 |   |   cmake/             # CMake 配置文件
5 |   |   |   MyLib/
6 |   |   |   |   MyLibConfig.cmake
7 |   |   |   |   MyLibTargets.cmake
8 |   |   include/          # 头文件
9 |   |   |   mylib/
10 |   |   |   |   mylib.h
11 |   |   share/            # 其他资源
12 |   |   |   mylib/
13 |   |   |   |   data/
```

1.6.8.2. install 命令基础

CMake 的 `install` 命令用于指定安装规则。最基本的用法是安装目标（可执行文件和库）：

```
1 add_library(mylib src/mylib.cpp)
2 add_executable(myapp src/main.cpp)
3
4 # 安装库
5 install(TARGETS mylib
6         LIBRARY DESTINATION lib      # 共享库 (.so)
7         ARCHIVE DESTINATION lib      # 静态库 (.a)
8         RUNTIME DESTINATION bin      # 可执行文件和 DLL
9     )
10
11 # 安装可执行文件
12 install(TARGETS myapp
13         RUNTIME DESTINATION bin
14     )
```

`DESTINATION` 指定安装位置，是相对于 `CMAKE_INSTALL_PREFIX` 的路径。默认的 `CMAKE_INSTALL_PREFIX` 在 Unix 系统上是 `/usr/local`，在 Windows 上是 `C:/Program Files/${PROJECT_NAME}`。

运行安装：

```
1 # 构建
2 cmake -DCMAKE_BUILD_TYPE=Release ..
3 cmake --build .
4
5 # 安装（可能需要 sudo）
6 sudo cmake --install .
7
8 # 或者安装到自定义位置
9 cmake --install . --prefix /opt/myproject
```

也可以在配置时指定安装前缀：

```
1 cmake -DCMAKE_INSTALL_PREFIX=/opt/myproject ..
2 cmake --build .
3 cmake --install .
```

1.6.8.3. 使用 GNUInstallDirs

不同平台的安装目录约定可能不同（比如 64 位库可能放在 `lib64` 而不是 `lib`）。CMake 提供了 `GNUInstallDirs` 模块来处理这些差异：

```
1 include(GNUInstallDirs)
2
3 install(TARGETS mylib
4     LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
5     ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
6     RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
7 )
8
9 install(DIRECTORY include/
10     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
11 )
```

`GNUInstallDirs` 定义了以下变量：

- `CMAKE_INSTALL_BINDIR`：可执行文件（通常是 `bin`）
- `CMAKE_INSTALL_LIBDIR`：库文件（`lib` 或 `lib64`）
- `CMAKE_INSTALL_INCLUDEDIR`：头文件（`include`）
- `CMAKE_INSTALL_DATADIR`：数据文件（`share`）
- `CMAKE_INSTALL_DOCDIR`：文档（`share/doc/${PROJECT_NAME}`）

使用这些变量可以让你的安装更加规范和可移植。

1.6.8.4. 安装头文件

头文件的安装有几种方式。最常用的是安装整个目录：

```
1 # 安装 include 目录下的所有文件
2 install(DIRECTORY include/
3     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
4 )
```

注意 `include/` 后面的斜杠——它表示安装目录的内容而不是目录本身。如果写 `include`（没有斜杠），会安装为 `${CMAKE_INSTALL_INCLUDEDIR}/include/...`。

可以用模式过滤：

```

1 # 只安装 .h 和 .hpp 文件
2 install(DIRECTORY include/
3     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
4     FILES_MATCHING
5         PATTERN "*.h"
6         PATTERN "*.hpp"
7 )
8
9 # 排除某些文件
10 install(DIRECTORY include/
11     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
12     PATTERN "internal" EXCLUDE # 排除 internal 目录
13     PATTERN "*.in" EXCLUDE # 排除 .in 文件
14 )

```

安装单个文件：

```

1 # 安装特定文件
2 install(FILES
3     include/mylib/core.h
4     include/mylib/utils.h
5     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}/mylib
6 )

```

对于生成的头文件（如配置头文件），需要从构建目录安装：

```

1 # 从模板生成配置头文件
2 configure_file(
3     ${CMAKE_CURRENT_SOURCE_DIR}/include/mylib/config.h.in
4     ${CMAKE_CURRENT_BINARY_DIR}/include/mylib/config.h
5 )
6
7 # 安装生成的头文件
8 install(FILES
9     ${CMAKE_CURRENT_BINARY_DIR}/include/mylib/config.h
10    DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}/mylib
11 )

```

1.6.8.5. 安装其他文件

除了目标和头文件，你可能还需要安装其他文件：

```

1 # 安装数据文件
2 install(DIRECTORY data/
3     DESTINATION ${CMAKE_INSTALL_DATADIR}/${PROJECT_NAME}
4 )
5
6 # 安装配置文件
7 install(FILES config/default.yaml
8     DESTINATION ${CMAKE_INSTALL_SYSCONFDIR}/${PROJECT_NAME}
9 )
10
11 # 安装文档
12 install(FILES README.md LICENSE
13     DESTINATION ${CMAKE_INSTALL_DOCDIR}
14 )
15
16 # 安装脚本（保留执行权限）
17 install(PROGRAMS scripts/run.sh
18     DESTINATION ${CMAKE_INSTALL_BINDIR}
19 )

```

1.6.8.6. 导出目标

安装库文件和头文件只是第一步。为了让其他项目能够通过 `find_package` 使用你的库，还需要导出目标信息——包括库的位置、include 路径、编译选项、依赖等。

首先，在安装目标时添加 `EXPORT` 选项：

```
1 install(TARGETS mylib
2     EXPORT MyLibTargets          # 导出到 MyLibTargets
3     LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
4     ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
5     RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
6     INCLUDES DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
7 )
```

`EXPORT MyLibTargets` 表示将这个目标添加到名为 `MyLibTargets` 的导出集中。`INCLUDES DESTINATION` 指定安装后的 include 路径，它会被记录到导出的目标信息中。

然后，安装导出集：

```
1 install(EXPORT MyLibTargets
2     FILE MyLibTargets.cmake
3     NAMESPACE MyLib::
4     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyLib
5 )
```

这会生成一个 `MyLibTargets.cmake` 文件，包含导入目标的定义。`NAMESPACE MyLib::` 会给导出的目标添加前缀，所以其他项目使用时是 `MyLib::mylib` 而不是 `mylib`。

1.6.8.7. 生成 Config 文件

只有 `MyLibTargets.cmake` 还不够，你还需要一个 `MyLibConfig.cmake` 文件作为入口点。这个文件会被 `find_package(MyLib)` 找到并执行。

最简单的方式是手写一个：

```
1 # MyLibConfig.cmake.in
2 @PACKAGE_INIT@
3
4 include("${CMAKE_CURRENT_LIST_DIR}/MyLibTargets.cmake")
5
6 check_required_components(MyLib)
```

然后用 `configure_package_config_file` 生成：

```
1 include(CMakePackageConfigHelpers)
2
3 configure_package_config_file(
4     ${CMAKE_CURRENT_SOURCE_DIR}/cmake/MyLibConfig.cmake.in
5     ${CMAKE_CURRENT_BINARY_DIR}/MyLibConfig.cmake
6     INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyLib
7 )
8
9 install(FILES
10     ${CMAKE_CURRENT_BINARY_DIR}/MyLibConfig.cmake
11     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyLib
12 )
```

`@PACKAGE_INIT@` 是一个占位符，会被替换为一些辅助宏。`check_required_components` 用于检查用户请求的组件是否都存在。

如果你的库依赖其他库，需要在 Config 文件中查找它们：

```
1 # MyLibConfig.cmake.in
2 @PACKAGE_INIT@
3
4 # 查找依赖
5 include(CMakeFindDependencyMacro)
6 find_dependency(Eigen3)
7 find_dependency(OpenCV)
8
9 include("${CMAKE_CURRENT_LIST_DIR}/MyLibTargets.cmake")
10
11 check_required_components(MyLib)
```

`find_dependency` 和 `find_package` 类似，但它会正确传播 `REQUIRED` 和 `QUIET` 选项。

1.6.8.8. 版本文件

为了支持版本检查（如 `find_package(MyLib 1.2.0 REQUIRED)`），需要生成一个版本文件：

```
1 include(CMakePackageConfigHelpers)
2
3 write_basic_package_version_file(
4     ${CMAKE_CURRENT_BINARY_DIR}/MyLibConfigVersion.cmake
5     VERSION ${PROJECT_VERSION}
6     COMPATIBILITY SameMajorVersion
7 )
8
9 install(FILES
10     ${CMAKE_CURRENT_BINARY_DIR}/MyLibConfigVersion.cmake
11     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyLib
12 )
```

`COMPATIBILITY` 指定版本兼容性策略：

- `ExactVersion`：必须精确匹配
- `SameMajorVersion`：主版本号相同即可（如 1.2.0 兼容 1.0.0）
- `SameMinorVersion`：主版本和次版本号相同（如 1.2.3 兼容 1.2.0）
- `AnyNewerVersion`：任何更新的版本都可以

对于遵循语义化版本的库，`SameMajorVersion` 是最常用的选择。

1.6.8.9. 完整的安装配置示例

让我们看一个完整的例子，展示如何正确配置一个库的安装和导出：

```
1 cmake_minimum_required(VERSION 3.16)
2 project(RMCore VERSION 2.0.0 LANGUAGES CXX)
3
4 # 包含必要的模块
5 include(GNUInstallDirs)
6 include(CMakePackageConfigHelpers)
7
8 # 依赖
9 find_package(Eigen3 REQUIRED)
10 find_package(OpenCV REQUIRED)
11
12 #=====
```

```

13 # 定义库
14 #=====
15
16 add_library(rm_core
17     src/config.cpp
18     src/logger.cpp
19     src/math_utils.cpp
20 )
21
22 add_library(RMCore::rm_core ALIAS rm_core) # 创建别名, 便于项目内使用
23
24 target_include_directories(rm_core
25     PUBLIC
26     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
27     $<INSTALL_INTERFACE:${CMAKE_INSTALL_INCLUDEDIR}>
28 )
29
30 target_compile_features(rm_core PUBLIC cxx_std_17)
31
32 target_link_libraries(rm_core
33     PUBLIC Eigen3::Eigen
34     PRIVATE OpenCV::OpenCV
35 )
36
37 #=====
38 # 安装
39 #=====
40
41 # 安装库目标
42 install(TARGETS rm_core
43     EXPORT RMCoreTargets
44     LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
45     ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
46     RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
47     INCLUDES DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
48 )
49
50 # 安装头文件
51 install(DIRECTORY include/
52     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
53     FILES_MATCHING PATTERN "*.h" PATTERN "*.hpp"
54 )
55
56 # 安装导出目标
57 install(EXPORT RMCoreTargets
58     FILE RMCoreTargets.cmake
59     NAMESPACE RMCore::
60     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/RMCore
61 )
62
63 # 生成并安装 Config 文件
64 configure_package_config_file(
65     ${CMAKE_CURRENT_SOURCE_DIR}/cmake/RMCoreConfig.cmake.in
66     ${CMAKE_CURRENT_BINARY_DIR}/RMCoreConfig.cmake
67     INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/RMCore
68 )
69
70 # 生成并安装版本文件
71 write_basic_package_version_file(
72     ${CMAKE_CURRENT_BINARY_DIR}/RMCoreConfigVersion.cmake
73     VERSION ${PROJECT_VERSION}
74     COMPATIBILITY SameMajorVersion
75 )
76

```

```

77 install(FILESDIR
78     ${CMAKE_CURRENT_BINARY_DIR}/RMCoreConfig.cmake
79     ${CMAKE_CURRENT_BINARY_DIR}/RMCoreConfigVersion.cmake
80     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/RMCore
81 )

```

对应的 Config 模板文件:

```

1 # cmake/RMCoreConfig.cmake.in
2 @PACKAGE_INIT@
3
4 include(CMakeFindDependencyMacro)
5
6 # 查找依赖
7 find_dependency(Eigen3)
8 # 注意: OpenCV 是 PRIVATE 依赖, 不需要在这里查找
9
10 # 包含导出的目标
11 include("${CMAKE_CURRENT_LIST_DIR}/RMCoreTargets.cmake")
12
13 check_required_components(RMCore)

```

安装后的目录结构:

```

1 /usr/local/
2 |--- include/
3 |   |--- rm_core/
4 |   |   |--- config.h
5 |   |   |--- logger.h
6 |   |   |--- math_utils.h
7 |   |--- lib/
8 |   |   |--- librm_core.so
9 |   |   |--- cmake/
10 |   |       |--- RMCore/
11 |   |           |--- RMCoreConfig.cmake
12 |   |           |--- RMCoreConfigVersion.cmake
13 |   |           |--- RMCoreTargets.cmake
14 |   |           |--- RMCoreTargets-release.cmake

```

其他项目使用这个库:

```

1 find_package(RMCore 2.0 REQUIRED)
2
3 add_executable(my_app main.cpp)
4 target_link_libraries(my_app PRIVATE RMCore::rm_core)

```

1.6.8.10. 支持构建目录导出

有时候, 你希望在不安装的情况下, 其他项目也能 `find_package` 你的库 (比如在同一个工作空间的多个项目)。可以导出到构建目录:

```

1 # 导出到构建目录
2 export(EXPORT RMCoreTargets
3     FILE ${CMAKE_CURRENT_BINARY_DIR}/RMCoreTargets.cmake
4     NAMESPACE RMCore::
5 )
6
7 # 将构建目录注册到用户包注册表
8 export(PACKAGE RMCore)

```


`export(PACKAGE ...)` 会在用户的 CMake 包注册表中注册这个包，其他项目就可以找到它了。这在开发阶段很方便，但正式发布时应该使用完整的安装流程。

1.6.8.11. 安装多个组件

对于大型项目，可能希望让用户选择安装哪些组件：

```
1 # 运行时组件
2 install(TARGETS rm_vision
3     RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
4     COMPONENT Runtime
5 )
6
7 # 开发组件
8 install(TARGETS rm_core
9     EXPORT RMCoreTargets
10    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
11    ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
12    COMPONENT Development
13 )
14
15 install(DIRECTORY include/
16     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
17     COMPONENT Development
18 )
19
20 # 文档组件
21 install(DIRECTORY docs/
22     DESTINATION ${CMAKE_INSTALL_DOCDIR}
23     COMPONENT Documentation
24 )
```

用户可以选择性安装：

```
1 # 只安装运行时组件
2 cmake --install . --component Runtime
3
4 # 只安装开发组件
5 cmake --install . --component Development
```

1.6.8.12. CPack: 打包与分发

CMake 的 CPack 模块可以将项目打包成各种格式，便于分发。

```
1 # 在 CMakeLists.txt 末尾添加
2 set(CPACK_PACKAGE_NAME "RMCore")
3 set(CPACK_PACKAGE_VERSION ${PROJECT_VERSION})
4 set(CPACK_PACKAGE_DESCRIPTION_SUMMARY "RoboMaster Core Library")
5 set(CPACK_PACKAGE_VENDOR "RoboMaster Team")
6 set(CPACK_PACKAGE_CONTACT "team@example.com")
7
8 # Debian 包配置
9 set(CPACK_DEBIAN_PACKAGE_DEPENDS "libeigen3-dev, libopencv-dev")
10
11 # RPM 包配置
12 set(CPACK_RPM_PACKAGE_REQUIRES "eigen3-devel, opencv-devel")
13
14 # 包含 CPack
15 include(CPack)
```

生成包：

```

1 # 构建后生成包
2 cmake --build .
3 cpack
4
5 # 生成特定格式的包
6 cpack -G DEB # Debian 包
7 cpack -G RPM # RPM 包
8 cpack -G TGZ # tar.gz 压缩包
9 cpack -G ZIP # ZIP 压缩包

```

更细致的控制：

```

1 # 组件打包
2 set(CPACK_COMPONENTS_ALL Runtime Development Documentation)
3 set(CPACK_COMPONENT_RUNTIME_DISPLAY_NAME "Runtime Files")
4 set(CPACK_COMPONENT_DEVELOPMENT_DISPLAY_NAME "Development Files")
5 set(CPACK_COMPONENT_DOCUMENTATION_DISPLAY_NAME "Documentation")
6
7 # 组件依赖
8 set(CPACK_COMPONENT_DEVELOPMENT_DEPENDS Runtime)
9
10 # 每个组件单独打包
11 set(CPACK_COMPONENTS_GROUPING ONE_PER_GROUP)

```

1.6.8.13. 安装脚本

有时候安装过程需要执行一些脚本，比如更新库缓存：

```

1 # 安装后执行的脚本
2 install(CODE "
3     execute_process(COMMAND ldconfig)
4     message(STATUS \"Library cache updated\")
5 ")
6
7 # 或者从文件执行
8 install(SCRIPT ${CMAKE_CURRENT_SOURCE_DIR}/cmake/post_install.cmake)

```

1.6.8.14. 卸载支持

CMake 默认不提供卸载目标，但可以手动添加：

```

1 # 创建卸载目标
2 if(NOT TARGET uninstall)
3     configure_file(
4         ${CMAKE_CURRENT_SOURCE_DIR}/cmake/cmake_uninstall.cmake.in
5         ${CMAKE_CURRENT_BINARY_DIR}/cmake_uninstall.cmake
6         IMMEDIATE @ONLY
7     )
8
9     add_custom_target(uninstall
10         COMMAND ${CMAKE_COMMAND} -P ${CMAKE_CURRENT_BINARY_DIR}/
11             cmake_uninstall.cmake
12         )
13 endif()

```

卸载脚本模板：

```

1 # cmake/cmake_uninstall.cmake.in
2 if(NOT EXISTS "@CMAKE_BINARY_DIR@/install_manifest.txt")

```

```

3     message(FATAL_ERROR "Cannot find install manifest: @CMAKE_BINARY_DIR@/
install_manifest.txt")
4 endif()
5
6 file(READ "@CMAKE_BINARY_DIR@/install_manifest.txt" files)
7 string(REGEX REPLACE "\n" ";" files "${files}")
8 foreach(file ${files})
9     message(STATUS "Uninstalling $ENV{DESTDIR}${file}")
10    if(IS_SYMLINK "$ENV{DESTDIR}${file}" OR EXISTS "$ENV{DESTDIR}${file}")
11        exec_program(
12            "@CMAKE_COMMAND@" ARGS "-E remove \"$ENV{DESTDIR}${file}\""
13            OUTPUT_VARIABLE rm_out
14            RETURN_VALUE rm_retval
15        )
16        if(NOT "${rm_retval}" STREQUAL 0)
17            message(FATAL_ERROR "Problem when removing
$ENV{DESTDIR}${file}")
18        endif()
19    else()
20        message(STATUS "File $ENV{DESTDIR}${file} does not exist.")
21    endif()
22 endforeach()

```

使用：

```
1 sudo cmake --build . --target uninstall
```

1.6.8.15. 最佳实践总结

编写可安装的库时，遵循以下最佳实践：

1. 使用 `GNUInstallDirs`：确保安装路径符合平台约定
2. 使用生成器表达式区分构建和安装路径：

```

1 target_include_directories(mylib PUBLIC
2     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
3     $<INSTALL_INTERFACE:${CMAKE_INSTALL_INCLUDEDIR}>
4 )

```

3. 使用命名空间：导出目标时添加命名空间前缀（如 `MyLib::`）
4. 创建别名目标：便于在项目内部使用相同的目标名称

```
1 add_library(MyLib::mylib ALIAS mylib)
```

5. 处理依赖：在 Config 文件中使用 `find_dependency` 查找 PUBLIC 依赖
6. 提供版本文件：支持版本检查
7. 考虑组件：大型项目应该分组件安装
8. 测试安装：在 CI 中测试完整的安装和使用流程

正确的安装和导出配置让你的库成为 CMake 生态的一等公民。用户可以像使用任何标准库一样使用你的库，无需了解内部结构，只需要简单的 `find_package` 和 `target_link_libraries`。这种良好的封装是高质量库的标志。

1.6.9. ROS 2 包的构建

ROS 2 (Robot Operating System 2) 是机器人软件开发的标准框架, RoboMaster 的许多视觉和控制系统都基于 ROS 2 构建。ROS 2 使用 `ament_cmake` 作为 C++ 包的构建系统, 它是对 CMake 的封装和扩展, 提供了处理 ROS 2 特有需求 (如消息类型、服务、节点发现等) 的便利功能。理解 `ament_cmake` 是开发 ROS 2 应用的基础。

1.6.9.1. ROS 2 的包结构

ROS 2 的包 (package) 是代码组织的基本单位。一个典型的 C++ 包结构如下:

```
1 my_package/  
2 |— CMakeLists.txt          # 构建配置  
3 |— package.xml            # 包元信息  
4 |— include/  
5 |   |— my_package/  
6 |       |— my_node.hpp  
7 |— src/                  # 源文件  
8 |   |— my_node.cpp  
9 |   |— main.cpp  
10 |— launch/              # 启动文件  
11 |   |— my_launch.py  
12 |— config/              # 配置文件  
13 |   |— params.yaml  
14 |— msg/                 # 自定义消息 (可选)  
15 |   |— MyMessage.msg  
16 |— srv/                 # 自定义服务 (可选)  
17 |   |— MyService.srv  
18 |— test/                # 测试文件  
19 |   |— test_my_node.cpp
```

与普通的 CMake 项目相比, ROS 2 包多了 `package.xml` 文件, 它描述了包的元信息和依赖关系。ROS 2 的构建工具 (colcon) 会读取这个文件来确定构建顺序和依赖。

1.6.9.2. package.xml: 包元信息

`package.xml` 是每个 ROS 2 包必需的文件, 它使用 XML 格式描述包的信息:

```
1 <?xml version="1.0"?>  
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd"  
  schematypens="http://www.w3.org/2001/XMLSchema"?>  
3 <package format="3">  
4   <name>rm_vision</name>  
5   <version>2.0.0</version>  
6   <description>RoboMaster Vision System</description>  
7   <maintainer email="developer@example.com">Developer Name</maintainer>  
8   <license>MIT</license>  
9  
10  <!-- 构建工具依赖 -->  
11  <buildtool_depend>ament_cmake</buildtool_depend>  
12  
13  <!-- 构建时依赖 -->  
14  <build_depend>rclcpp</build_depend>  
15  <build_depend>std_msgs</build_depend>  
16  <build_depend>sensor_msgs</build_depend>  
17  <build_depend>cv_bridge</build_depend>  
18  <build_depend>OpenCV</build_depend>  
19  
20  <!-- 运行时依赖 -->  
21  <exec_depend>rclcpp</exec_depend>
```

```

22 <exec_depend>std_msgs</exec_depend>
23 <exec_depend>sensor_msgs</exec_depend>
24 <exec_depend>cv_bridge</exec_depend>
25
26 <!-- 构建和运行时都需要（简写） -->
27 <depend>geometry_msgs</depend>
28 <depend>tf2_ros</depend>
29
30 <!-- 测试依赖 -->
31 <test_depend>ament_lint_auto</test_depend>
32 <test_depend>ament_cmake_gtest</test_depend>
33
34 <export>
35   <build_type>ament_cmake</build_type>
36 </export>
37 </package>

```

依赖类型说明：

- `buildtool_depend`：构建工具依赖，如 `ament_cmake`
- `build_depend`：只在编译时需要的依赖
- `exec_depend`：只在运行时需要的依赖
- `depend`：编译和运行时都需要（等价于 `build_depend` + `exec_depend`）
- `test_depend`：只在测试时需要的依赖

`<export>` 标签中的 `<build_type>` 告诉 colcon 使用哪种构建系统。对于 C++ 包，通常是 `ament_cmake`。

1.6.9.3. ament_cmake vs ament_python

ROS 2 支持两种主要的包类型：

- `ament_cmake`：用于 C/C++ 包，基于 CMake
- `ament_python`：用于纯 Python 包，使用 setuptools

对于 RoboMaster 视觉系统这类需要高性能的应用，通常使用 `ament_cmake`。但你也可以创建混合包，用 C++ 实现核心功能，用 Python 编写高层逻辑或工具脚本。

```

1 <!-- C++ 包 -->
2 <buildtool_depend>ament_cmake</buildtool_depend>
3 <export>
4   <build_type>ament_cmake</build_type>
5 </export>
6
7 <!-- Python 包 -->
8 <buildtool_depend>ament_python</buildtool_depend>
9 <export>
10   <build_type>ament_python</build_type>
11 </export>
12
13 <!-- 混合包（C++ 为主，包含 Python 模块） -->
14 <buildtool_depend>ament_cmake</buildtool_depend>
15 <buildtool_depend>ament_cmake_python</buildtool_depend>
16 <export>
17   <build_type>ament_cmake</build_type>
18 </export>

```

1.6.9.4. 基本的 CMakeLists.txt 结构

ROS 2 包的 CMakeLists.txt 有一些固定的模式：

```

1 cmake_minimum_required(VERSION 3.8)
2 project(my_package)
3
4 # 设置编译选项
5 if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
6     add_compile_options(-Wall -Wextra -Wpedantic)
7 endif()
8
9 # 查找依赖
10 find_package(ament_cmake REQUIRED)
11 find_package(rclcpp REQUIRED)
12 find_package(std_msgs REQUIRED)
13
14 # 定义可执行文件
15 add_executable(my_node src/my_node.cpp)
16 ament_target_dependencies(my_node rclcpp std_msgs)
17
18 # 安装目标
19 install(TARGETS my_node
20         DESTINATION lib/${PROJECT_NAME})
21 )
22
23 # 必须: 调用 ament_package
24 ament_package()

```

几个关键点:

1. 必须 `find_package(ament_cmake REQUIRED)`
2. ROS 2 依赖通过 `find_package` 查找
3. 使用 `ament_target_dependencies` 简化依赖声明
4. 可执行文件安装到 `lib/${PROJECT_NAME}` 而不是 `bin`
5. 必须在最后调用 `ament_package()`

1.6.9.5. `ament_target_dependencies`: 简化依赖声明

`ament_target_dependencies` 是 `ament_cmake` 提供的宏，它比原生的 `target_link_libraries` 更方便:

```

1 # ament_cmake 方式 (推荐)
2 ament_target_dependencies(my_node
3     rclcpp
4     std_msgs
5     sensor_msgs
6     cv_bridge
7 )
8
9 # 等价的原生 CMake 方式 (更繁琐)
10 target_link_libraries(my_node
11     ${rclcpp_LIBRARIES}
12     ${std_msgs_LIBRARIES}
13     ${sensor_msgs_LIBRARIES}
14     ${cv_bridge_LIBRARIES}
15 )
16 target_include_directories(my_node PUBLIC
17     ${rclcpp_INCLUDE_DIRS}
18     ${std_msgs_INCLUDE_DIRS}
19     ${sensor_msgs_INCLUDE_DIRS}
20     ${cv_bridge_INCLUDE_DIRS}
21 )

```

`ament_target_dependencies` 会自动处理 include 路径、库链接和编译定义。对于 ROS 2 包，推荐使用这个宏。

但有时候你需要混合使用：

```
1 # ROS 2 依赖用 ament_target_dependencies
2 ament_target_dependencies(my_node
3   rclcpp
4   sensor_msgs
5 )
6
7 # 非 ROS 依赖用 target_link_libraries
8 find_package(Eigen3 REQUIRED)
9 find_package(OpenCV REQUIRED)
10 target_link_libraries(my_node
11   Eigen3::Eigen
12   OpenCV::OpenCV
13 )
```

1.6.9.6. ROS 2 常用包

RoboMaster 开发中常用的 ROS 2 包：

核心包：

```
1 # ROS 2 C++ 客户端库
2 find_package(rclcpp REQUIRED)
3
4 # 组件化节点支持
5 find_package(rclcpp_components REQUIRED)
6
7 # 生命周期节点
8 find_package(rclcpp_lifecycle REQUIRED)
```

消息包：

```
1 # 标准消息类型
2 find_package(std_msgs REQUIRED)           # String, Int32, Float64, Bool 等
3
4 # 传感器消息
5 find_package(sensor_msgs REQUIRED)        # Image, CameraInfo, Imu, LaserScan
6   等
7
8 # 几何消息
9 find_package(geometry_msgs REQUIRED)      # Pose, Transform, Twist, Point 等
10
11 # 可视化消息
12 find_package(visualization_msgs REQUIRED) # Marker, MarkerArray
```

图像处理：

```
1 # OpenCV 与 ROS 的桥接
2 find_package(cv_bridge REQUIRED)
3
4 # 图像传输（支持压缩）
5 find_package(image_transport REQUIRED)
6
7 # 相机信息
8 find_package(camera_info_manager REQUIRED)
```

坐标变换：

```

1 # TF2 核心库
2 find_package(tf2 REQUIRED)
3
4 # TF2 ROS 接口
5 find_package(tf2_ros REQUIRED)
6
7 # TF2 几何消息支持
8 find_package(tf2_geometry_msgs REQUIRED)
9
10 # TF2 Eigen 支持
11 find_package(tf2_eigen REQUIRED)

```

一个典型的视觉节点可能需要这些依赖：

```

1 find_package(ament_cmake REQUIRED)
2 find_package(rclcpp REQUIRED)
3 find_package(sensor_msgs REQUIRED)
4 find_package(geometry_msgs REQUIRED)
5 find_package(cv_bridge REQUIRED)
6 find_package(image_transport REQUIRED)
7 find_package(tf2_ros REQUIRED)
8 find_package(tf2_geometry_msgs REQUIRED)
9
10 # 非 ROS 依赖
11 find_package(OpenCV REQUIRED)
12 find_package(Eigen3 REQUIRED)

```

1.6.9.7. 创建库

如果你的包提供库给其他包使用，需要正确配置导出：

```

1 # 创建库
2 add_library(${PROJECT_NAME}_lib
3   src/detector.cpp
4   src/tracker.cpp
5 )
6
7 # 设置头文件路径
8 target_include_directories(${PROJECT_NAME}_lib PUBLIC
9   $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
10  $<INSTALL_INTERFACE:include>
11 )
12
13 # 添加依赖
14 ament_target_dependencies(${PROJECT_NAME}_lib
15   rclcpp
16   sensor_msgs
17   cv_bridge
18 )
19
20 # 导出依赖（让其他包知道需要哪些依赖）
21 ament_export_dependencies(
22   rclcpp
23   sensor_msgs
24   cv_bridge
25 )
26
27 # 导出 include 路径
28 ament_export_include_directories(include)
29
30 # 导出库

```



```

31 ament_export_libraries(${PROJECT_NAME}_lib)
32
33 # 安装头文件
34 install(DIRECTORY include/
35   DESTINATION include
36 )
37
38 # 安装库
39 install(TARGETS ${PROJECT_NAME}_lib
40   ARCHIVE DESTINATION lib
41   LIBRARY DESTINATION lib
42   RUNTIME DESTINATION bin
43 )

```

使用现代 CMake 目标导出的方式（推荐）：

```

1 # 创建库
2 add_library(${PROJECT_NAME}_lib
3   src/detector.cpp
4   src/tracker.cpp
5 )
6
7 target_include_directories(${PROJECT_NAME}_lib PUBLIC
8   $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
9   $<INSTALL_INTERFACE:include>
10 )
11
12 ament_target_dependencies(${PROJECT_NAME}_lib
13   rclcpp
14   sensor_msgs
15 )
16
17 # 安装头文件
18 install(DIRECTORY include/
19   DESTINATION include
20 )
21
22 # 安装并导出目标
23 install(TARGETS ${PROJECT_NAME}_lib
24   EXPORT ${PROJECT_NAME}Targets
25   ARCHIVE DESTINATION lib
26   LIBRARY DESTINATION lib
27   RUNTIME DESTINATION bin
28 )
29
30 # 导出目标
31 ament_export_targets(${PROJECT_NAME}Targets HAS_LIBRARY_TARGET)
32
33 # 导出依赖
34 ament_export_dependencies(rclcpp sensor_msgs)

```

1.6.9.8. 消息和服务的生成

ROS 2 允许定义自定义的消息（msg）、服务（srv）和动作（action）类型。这些定义会被编译成 C++ 和 Python 代码。

消息文件示例 `msg/AimTarget.msg`：

```

1 # 目标信息
2 uint8 id # 目标 ID
3 geometry_msgs/Point position # 3D 位置

```

```

4 float64 confidence      # 置信度
5 float64 yaw             # 偏航角
6 float64 pitch           # 俯仰角
7 builtin_interfaces/Time stamp # 时间戳

```

服务文件示例 `srv/SetMode.srv` :

```

1 # 请求
2 uint8 mode
3 ---
4 # 响应
5 bool success
6 string message

```

CMakeLists.txt 配置:

```

1 find_package(ament_cmake REQUIRED)
2 find_package(rosidl_default_generators REQUIRED)
3 find_package(geometry_msgs REQUIRED)
4 find_package(builtin_interfaces REQUIRED)
5
6 # 生成消息和服务
7 rosidl_generate_interfaces(${PROJECT_NAME}
8   "msg/AimTarget.msg"
9   "msg/ArmorArray.msg"
10  "srv/SetMode.srv"
11  DEPENDENCIES geometry_msgs builtin_interfaces
12 )
13
14 # 如果本包的可执行文件需要使用这些消息
15 add_executable(my_node src/my_node.cpp)
16 ament_target_dependencies(my_node rclcpp)
17
18 # 链接生成的消息库
19 rosidl_get_typesupport_target(cpp_typesupport_target
20   ${PROJECT_NAME} rosidl_typesupport_cpp)
21 target_link_libraries(my_node "${cpp_typesupport_target}")

```

package.xml 需要添加相应依赖:

```

1 <buildtool_depend>rosidl_default_generators</buildtool_depend>
2 <depend>geometry_msgs</depend>
3 <depend>builtin_interfaces</depend>
4
5 <member_of_group>rosidl_interface_packages</member_of_group>

```

在代码中使用自定义消息:

```

1 #include "my_package/msg/aim_target.hpp"
2
3 void callback(const my_package::msg::AimTarget::SharedPtr msg) {
4   RCLCPP_INFO(this->get_logger(), "Target %d at (%.2f, %.2f, %.2f)",
5     msg->id, msg->position.x, msg->position.y, msg->position.z);
6 }

```

1.6.9.9. 组件化节点

ROS 2 推荐使用组件化节点, 它们可以在同一进程中加载多个节点, 减少通信开销:

```

1 # 创建组件库
2 add_library(detector_component SHARED
3   src/detector_node.cpp
4 )
5
6 target_include_directories(detector_component PUBLIC
7   $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
8   $<INSTALL_INTERFACE:include>
9 )
10
11 ament_target_dependencies(detector_component
12   rclcpp
13   rclcpp_components
14   sensor_msgs
15   cv_bridge
16 )
17
18 # 注册组件
19 rclcpp_components_register_node(detector_component
20   PLUGIN "rm_vision::DetectorNode"
21   EXECUTABLE detector_node
22 )
23
24 # 安装组件库
25 install(TARGETS detector_component
26   ARCHIVE DESTINATION lib
27   LIBRARY DESTINATION lib
28   RUNTIME DESTINATION bin
29 )

```

组件节点的代码:

```

1 #include <rclcpp/rclcpp.hpp>
2 #include <sensor_msgs/msg/image.hpp>
3
4 namespace rm_vision {
5
6 class DetectorNode : public rclcpp::Node {
7 public:
8   explicit DetectorNode(const rclcpp::NodeOptions& options)
9     : Node("detector_node", options) {
10     // 初始化...
11   }
12
13 private:
14   // 成员...
15 };
16
17 } // namespace rm_vision
18
19 #include "rclcpp_components/register_node_macro.hpp"
20 RCLCPP_COMPONENTS_REGISTER_NODE(rm_vision::DetectorNode)

```

1.6.9.10. 安装 launch 文件

ROS 2 的 launch 文件通常用 Python 编写，需要安装到正确位置：

```

1 # 安装 launch 文件
2 install(DIRECTORY launch/
3   DESTINATION share/${PROJECT_NAME}/launch
4 )

```

launch 文件示例 `launch/vision_launch.py` :

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3 from launch.actions import DeclareLaunchArgument
4 from launch.substitutions import LaunchConfiguration
5 from ament_index_python.packages import get_package_share_directory
6 import os
7
8 def generate_launch_description():
9     pkg_share = get_package_share_directory('rm_vision')
10
11     config_file = os.path.join(pkg_share, 'config', 'params.yaml')
12
13     return LaunchDescription([
14         DeclareLaunchArgument(
15             'use_sim_time',
16             default_value='false',
17             description='Use simulation time'
18         ),
19
20         Node(
21             package='rm_vision',
22             executable='detector_node',
23             name='detector',
24             parameters=[config_file, {'use_sim_time':
LaunchConfiguration('use_sim_time')}],
25             output='screen'
26         ),
27
28         Node(
29             package='rm_vision',
30             executable='tracker_node',
31             name='tracker',
32             parameters=[config_file],
33             output='screen'
34         ),
35     ])
```

1.6.9.11. 安装配置文件

配置文件（YAML 参数文件、模型文件等）需要安装到 share 目录：

```
1 # 安装配置文件
2 install(DIRECTORY config/
3     DESTINATION share/${PROJECT_NAME}/config
4 )
5
6 # 安装模型文件
7 install(DIRECTORY models/
8     DESTINATION share/${PROJECT_NAME}/models
9 )
10
11 # 安装 URDF/Xacro 文件
12 install(DIRECTORY urdf/
13     DESTINATION share/${PROJECT_NAME}/urdf
14 )
15
16 # 安装 RViz 配置
17 install(DIRECTORY rviz/
18     DESTINATION share/${PROJECT_NAME}/rviz
19 )
```

配置文件示例 `config/params.yaml` :

```
1 detector_node:
2   ros__parameters:
3     camera_topic: /camera/image_raw
4     detection_threshold: 0.7
5     max_targets: 5
6     debug_mode: false
7     model_path: "" # 如果为空, 使用默认路径
8
9 tracker_node:
10  ros__parameters:
11    process_noise: 0.1
12    measurement_noise: 0.05
13    max_lost_frames: 10
```

在代码中获取包的资源路径:

```
1 #include <ament_index_cpp/get_package_share_directory.hpp>
2
3 std::string pkg_share
4   = ament_index_cpp::get_package_share_directory("rm_vision");
5 std::string model_path = pkg_share + "/models/detector.onnx";
```

1.6.9.12. colcon build: ROS 2 的构建工具

ROS 2 使用 colcon 作为构建工具, 它会处理工作空间中所有包的构建:

```
1 # 创建工作空间
2 mkdir -p ~/ros2_ws/src
3 cd ~/ros2_ws/src
4
5 # 克隆包
6 git clone https://github.com/example/rm_vision.git
7
8 # 返回工作空间根目录
9 cd ~/ros2_ws
10
11 # 构建所有包
12 colcon build
13
14 # 只构建特定包
15 colcon build --packages-select rm_vision
16
17 # 构建包及其依赖
18 colcon build --packages-up-to rm_vision
19
20 # 使用 Release 模式
21 colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release
22
23 # 并行构建 (默认已启用)
24 colcon build --parallel-workers 4
```

构建后的目录结构:

```
1 ros2_ws/
2 |— src/                # 源代码
3 |   |— rm_vision/
4 |— build/              # 构建中间文件
5 |   |— rm_vision/
6 |— install/            # 安装目录
```

```

7 |      |   rm_vision/
8 |      |   |   lib/
9 |      |   |   share/
10 |     |   |   include/
11 |     |   |   setup.bash
12 |     |   |   local_setup.bash
13 |     |   |   log/
    |   |   # 构建日志

```

使用工作空间：

```

1 # 加载工作空间环境
2 source ~/ros2_ws/install/setup.bash
3
4 # 运行节点
5 ros2 run rm_vision detector_node
6
7 # 使用 launch 文件
8 ros2 launch rm_vision vision_launch.py

```

1.6.9.13. --symlink-install：开发便利选项

`--symlink-install` 选项在安装时创建符号链接而不是复制文件，这对开发非常有用：

```
1 colcon build --symlink-install
```

好处：

1. 修改 Python 代码、launch 文件、配置文件后无需重新构建
2. 修改 C++ 头文件后，只需重新编译，无需重新安装
3. 节省磁盘空间

```

1 # 开发工作流
2 colcon build --symlink-install
3
4 # 修改 launch 文件 - 直接生效，无需重新构建
5 vim src/rm_vision/launch/vision_launch.py
6 ros2 launch rm_vision vision_launch.py # 使用新的 launch 文件
7
8 # 修改配置文件 - 直接生效
9 vim src/rm_vision/config/params.yaml
10 ros2 run rm_vision detector_node # 使用新的配置
11
12 # 修改 C++ 代码 - 需要重新编译
13 vim src/rm_vision/src/detector_node.cpp
14 colcon build --packages-select rm_vision # 重新编译

```

注意： `--symlink-install` 对可执行文件和库不起作用，它们仍然需要重新构建。

1.6.9.14. 测试

ROS 2 使用 ament 的测试框架：

```

1 if(BUILD_TESTING)
2   find_package(ament_lint_auto REQUIRED)
3   ament_lint_auto_find_test_dependencies()
4
5   # 单元测试
6   find_package(ament_cmake_gtest REQUIRED)
7   ament_add_gtest(test_detector test/test_detector.cpp)

```

```

8   target_link_libraries(test_detector ${PROJECT_NAME}_lib)
9
10  # 集成测试
11  find_package(launch_testing_ament_cmake REQUIRED)
12  add_launch_test(test/test_integration.py)
13  endif()

```

运行测试：

```

1  # 构建并运行测试
2  colcon build
3  colcon test
4
5  # 查看测试结果
6  colcon test-result --verbose
7
8  # 只测试特定包
9  colcon test --packages-select rm_vision

```

1.6.9.15. 完整示例：视觉节点的 CMakeLists.txt

让我们看一个完整的 RoboMaster 视觉包配置：

```

1  cmake_minimum_required(VERSION 3.8)
2  project(rm_vision)
3
4  # 编译选项
5  if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
6    add_compile_options(-Wall -Wextra -Wpedantic)
7  endif()
8
9  # C++ 标准
10 set(CMAKE_CXX_STANDARD 17)
11 set(CMAKE_CXX_STANDARD_REQUIRED ON)
12
13 #=====
14 # 依赖
15 #=====
16
17 # ROS 2 核心
18 find_package(ament_cmake REQUIRED)
19 find_package(rclcpp REQUIRED)
20 find_package(rclcpp_components REQUIRED)
21
22 # 消息类型
23 find_package(std_msgs REQUIRED)
24 find_package(sensor_msgs REQUIRED)
25 find_package(geometry_msgs REQUIRED)
26 find_package(visualization_msgs REQUIRED)
27
28 # 图像处理
29 find_package(cv_bridge REQUIRED)
30 find_package(image_transport REQUIRED)
31
32 # 坐标变换
33 find_package(tf2 REQUIRED)
34 find_package(tf2_ros REQUIRED)
35 find_package(tf2_geometry_msgs REQUIRED)
36
37 # 非 ROS 依赖
38 find_package(OpenCV REQUIRED)
39 find_package(Eigen3 REQUIRED)

```

```

40
41 #=====
42 # 自定义消息
43 #=====
44
45 find_package(rosidl_default_generators REQUIRED)
46
47 rosidl_generate_interfaces(${PROJECT_NAME}
48     "msg/Armor.msg"
49     "msg/ArmorArray.msg"
50     "msg/AimInfo.msg"
51     DEPENDENCIES std_msgs geometry_msgs
52 )
53
54 # 获取类型支持目标
55 rosidl_get_typesupport_target(cpp_typesupport_target
56     ${PROJECT_NAME} rosidl_typesupport_cpp)
57
58 #=====
59 # 核心库
60 #=====
61
62 add_library(${PROJECT_NAME}_core SHARED
63     src/detector/armor_detector.cpp
64     src/detector/number_classifier.cpp
65     src/tracker/armor_tracker.cpp
66     src/tracker/kalman_filter.cpp
67     src/solver/ballistic_solver.cpp
68 )
69
70 target_include_directories(${PROJECT_NAME}_core PUBLIC
71     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
72     $<INSTALL_INTERFACE:include>
73 )
74
75 target_link_libraries(${PROJECT_NAME}_core
76     Eigen3::Eigen
77     opencv_core
78     opencv_imgproc
79     opencv_dnn
80 )
81
82 #=====
83 # 节点组件
84 #=====
85
86 # 检测节点
87 add_library(detector_component SHARED
88     src/nodes/detector_node.cpp
89 )
90
91 target_include_directories(detector_component PUBLIC
92     $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
93     $<INSTALL_INTERFACE:include>
94 )
95
96 target_link_libraries(detector_component
97     ${PROJECT_NAME}_core
98     "${cpp_typesupport_target}"
99 )
100
101 ament_target_dependencies(detector_component
102     rclcpp
103     rclcpp_components

```



```

104   sensor_msgs
105   cv_bridge
106   image_transport
107 )
108
109 rclcpp_components_register_node(detector_component
110     PLUGIN "rm_vision::DetectorNode"
111     EXECUTABLE detector_node
112 )
113
114 # 跟踪节点
115 add_library(tracker_component SHARED
116     src/nodes/tracker_node.cpp
117 )
118
119 target_include_directories(tracker_component PUBLIC
120     ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
121     ${INSTALL_INTERFACE:include}
122 )
123
124 target_link_libraries(tracker_component
125     ${PROJECT_NAME}_core
126     "${cpp_typesupport_target}"
127 )
128
129 ament_target_dependencies(tracker_component
130     rclcpp
131     rclcpp_components
132     tf2_ros
133     tf2_geometry_msgs
134     visualization_msgs
135 )
136
137 rclcpp_components_register_node(tracker_component
138     PLUGIN "rm_vision::TrackerNode"
139     EXECUTABLE tracker_node
140 )
141
142 # 瞄准控制节点
143 add_library(aim_component SHARED
144     src/nodes/aim_node.cpp
145 )
146
147 target_include_directories(aim_component PUBLIC
148     ${BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include}
149     ${INSTALL_INTERFACE:include}
150 )
151
152 target_link_libraries(aim_component
153     ${PROJECT_NAME}_core
154     "${cpp_typesupport_target}"
155 )
156
157 ament_target_dependencies(aim_component
158     rclcpp
159     rclcpp_components
160     geometry_msgs
161     tf2_ros
162 )
163
164 rclcpp_components_register_node(aim_component
165     PLUGIN "rm_vision::AimNode"
166     EXECUTABLE aim_node

```

```

167 )
168
169 #=====
170 # 安装
171 #=====
172
173 # 安装头文件
174 install(DIRECTORY include/
175     DESTINATION include
176 )
177
178 # 安装库
179 install(TARGETS
180     ${PROJECT_NAME}_core
181     detector_component
182     tracker_component
183     aim_component
184     ARCHIVE DESTINATION lib
185     LIBRARY DESTINATION lib
186     RUNTIME DESTINATION bin
187 )
188
189 # 安装 launch 文件
190 install(DIRECTORY launch/
191     DESTINATION share/${PROJECT_NAME}/launch
192 )
193
194 # 安装配置文件
195 install(DIRECTORY config/
196     DESTINATION share/${PROJECT_NAME}/config
197 )
198
199 # 安装模型文件
200 install(DIRECTORY models/
201     DESTINATION share/${PROJECT_NAME}/models
202 )
203
204 #=====
205 # 测试
206 #=====
207
208 if(BUILD_TESTING)
209     find_package(ament_lint_auto REQUIRED)
210     ament_lint_auto_find_test_dependencies()
211
212     find_package(ament_cmake_gtest REQUIRED)
213
214     ament_add_gtest(test_detector test/test_detector.cpp)
215     target_link_libraries(test_detector ${PROJECT_NAME}_core)
216
217     ament_add_gtest(test_tracker test/test_tracker.cpp)
218     target_link_libraries(test_tracker ${PROJECT_NAME}_core)
219 endif()
220
221 #=====
222 # 导出
223 #=====
224
225 ament_export_include_directories(include)
226 ament_export_libraries(${PROJECT_NAME}_core)
227 ament_export_dependencies(
228     rclcpp
229     sensor_msgs

```

```

230 geometry_msgs
231 cv_bridge
232 tf2_ros
233 OpenCV
234 Eigen3
235 )
236
237 # 必须在最后调用
238 ament_package()

```

对应的 package.xml:

```

1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>rm_vision</name>
5   <version>2.0.0</version>
6   <description>RoboMaster Vision System for ROS 2</description>
7   <maintainer email="rm_team@example.com">RM Team</maintainer>
8   <license>MIT</license>
9
10  <buildtool_depend>ament_cmake</buildtool_depend>
11  <buildtool_depend>rosidl_default_generators</buildtool_depend>
12
13  <depend>rclcpp</depend>
14  <depend>rclcpp_components</depend>
15  <depend>std_msgs</depend>
16  <depend>sensor_msgs</depend>
17  <depend>geometry_msgs</depend>
18  <depend>visualization_msgs</depend>
19  <depend>cv_bridge</depend>
20  <depend>image_transport</depend>
21  <depend>tf2</depend>
22  <depend>tf2_ros</depend>
23  <depend>tf2_geometry_msgs</depend>
24
25  <build_depend>OpenCV</build_depend>
26  <build_depend>eigen</build_depend>
27
28  <test_depend>ament_lint_auto</test_depend>
29  <test_depend>ament_cmake_gtest</test_depend>
30
31  <member_of_group>rosidl_interface_packages</member_of_group>
32
33  <export>
34    <build_type>ament_cmake</build_type>
35  </export>
36 </package>

```

这个完整示例展示了一个典型 RoboMaster ROS 2 视觉包的结构, 包括:

- 自定义消息类型
- 核心算法库
- 多个组件化节点
- 正确的依赖处理
- 完整的安装配置
- 测试支持
- 库导出

掌握了 ROS 2 包的构建配置，你就可以开发功能完整、结构清晰的机器人软件，并与 ROS 2 生态系统无缝集成。

1.6.10. 常见问题与调试

学习 CMake 的过程中，你一定会遇到各种各样的错误和问题。有些错误信息晦涩难懂，有些问题看似简单却难以定位。本节汇总了使用 CMake 时最常见的问题及其解决方案，介绍调试 CMake 配置的技巧，并总结现代 CMake 的最佳实践。掌握这些内容可以帮助你更快地排查问题，写出更高质量的构建配置。

1.6.10.1. “找不到包” 问题

这是最常见的 CMake 错误之一，通常表现为：

```
1 CMake Error at CMakeLists.txt:10 (find_package):
2   Could not find a package configuration file provided by "OpenCV" with any
3   of the following names:
4
5     OpenCVConfig.cmake
6     opencv-config.cmake
7
8   Add the installation prefix of "OpenCV" to CMAKE_PREFIX_PATH or set
9   "OpenCV_DIR" to a directory containing one of the above files.
```

原因：CMake 无法在默认搜索路径中找到库的配置文件。

解决方案：

1. 确认库已正确安装：

```
1 # 检查 OpenCV 是否安装
2 pkg-config --modversion opencv4
3 # 或
4 dpkg -l | grep opencv
```

2. 设置 `CMAKE_PREFIX_PATH`：

```
1 # 方式一：命令行
2 cmake -DCMAKE_PREFIX_PATH="/opt/opencv;/opt/eigen" ..
3
4 # 方式二：环境变量
5 export CMAKE_PREFIX_PATH="/opt/opencv:$CMAKE_PREFIX_PATH"
6 cmake ..
```

3. 设置库特定的 `_DIR` 变量：

```
1 cmake -DOpenCV_DIR=/opt/opencv/lib/cmake/opencv4 ..
```

4. 对于 ROS 2，确保已 source 工作空间：

```
1 source /opt/ros/humble/setup.bash
2 source ~/ros2_ws/install/setup.bash
3 cmake ..
```

5. 使用 `--debug-find` 查看详细搜索过程：

```
1 cmake --debug-find ..
```

常见陷阱：

- 包名大小写敏感: `find_package(OpenCV)` 不同于 `find_package(opencv)`
- 有些包名与库名不同: Eigen 的包名是 `Eigen3` 而不是 `Eigen`
- 多版本共存时可能找到错误版本: 明确指定路径

1.6.10.2. “未定义的引用”问题

链接错误通常表现为:

```
1 /usr/bin/ld: main.cpp:(.text+0x15): undefined reference to
  `MyClass::DoSomething()'
2 collect2: error: ld returned 1 exit status
```

原因: 链接器找不到某个符号的定义。

常见原因和解决方案:

1. 忘记链接库:

```
1 # 错误: 没有链接库
2 add_executable(app main.cpp)
3
4 # 正确: 链接库
5 add_executable(app main.cpp)
6 target_link_libraries(app PRIVATE mylib)
```

2. 链接顺序问题 (静态库):

```
1 # 如果 A 依赖 B, B 依赖 C, 顺序应该是 A B C
2 target_link_libraries(app PRIVATE libA libB libC)
3
4 # 如果有循环依赖, 可能需要重复
5 target_link_libraries(app PRIVATE libA libB libA)
```

3. 忘记编译某个源文件:

```
1 # 错误: 漏掉了 myclass.cpp
2 add_library(mylib utils.cpp)
3
4 # 正确: 包含所有源文件
5 add_library(mylib utils.cpp myclass.cpp)
```

4. 声明和定义不匹配:

```
1 // header.h
2 void DoSomething(int x);
3
4 // source.cpp - 参数类型不匹配!
5 void DoSomething(double x) { ... }
```

5. C 和 C++ 混合编程忘记 `extern "C"`:

```
1 // 在 C++ 代码中调用 C 函数
2 extern "C" {
3     #include "c_library.h"
4 }
```

6. 模板实例化问题:

```
1 # 模板定义应该在头文件中
```

```
2 # 或者显式实例化
```

调试技巧:

```
1 # 查看目标文件的符号
2 nm -C mylib.a | grep DoSomething
3
4 # 查看可执行文件需要的符号
5 nm -u app | grep DoSomething
6
7 # 使用 c++filt 解码符号名
8 echo "_ZN7MyClass11DoSomethingEv" | c++filt
```

1.6.10.3. “头文件找不到” 问题

编译错误通常表现为:

```
1 fatal error: opencv2/core.hpp: No such file or directory
2     #include <opencv2/core.hpp>
3         ^~~~~~
4 compilation terminated.
```

原因: 编译器找不到头文件。

解决方案:

1. 检查 `target_include_directories` :

```
1 target_include_directories(myapp PRIVATE
2     ${CMAKE_CURRENT_SOURCE_DIR}/include
3     ${OpenCV_INCLUDE_DIRS}
4 )
```

2. 使用导入目标自动处理:

```
1 # 现代方式: 自动添加 include 路径
2 target_link_libraries(myapp PRIVATE OpenCV::OpenCV)
```

3. 检查头文件实际位置:

```
1 # 查找头文件
2 find /usr -name "core.hpp" 2>/dev/null
3
4 # 查看包的 include 路径
5 pkg-config --cflags opencv4
```

4. 检查 PUBLIC/PRIVATE 是否正确:

```
1 # 如果 mylib 的头文件被 myapp 包含
2 # mylib 的 include 路径应该是 PUBLIC
3 target_include_directories(mylib PUBLIC include)
```

调试技巧:

```
1 # 查看实际的编译命令
2 make VERBOSE=1
3 # 或
4 cmake --build . --verbose
```

```
5
6 # 检查 -I 选项是否包含正确路径
```

1.6.10.4. “ABI 不兼容” 问题

运行时错误或奇怪的崩溃，可能表现为：

```
1 symbol lookup error: ./app: undefined symbol:
  _ZNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEEC1Ev
2
3 # 或者
4 Segmentation fault (core dumped)
```

原因：不同编译器版本或编译选项编译的代码混合使用。

常见场景：

- 用 GCC 7 编译的库与 GCC 11 编译的程序链接
- Debug 模式编译的库与 Release 模式编译的程序链接
- 使用了不同的 C++ 标准库（libstdc++ vs libc++）

解决方案：

1. 统一编译器版本：

```
1 # 明确指定编译器
2 cmake -DCMAKE_CXX_COMPILER=/usr/bin/g++-11 ..
```

2. 重新编译所有依赖：

```
1 # 清理并重新构建
2 rm -rf build
3 mkdir build && cd build
4 cmake ..
5 cmake --build .
```

3. 检查 C++ ABI 标签：

```
1 # 查看库使用的符号
2 nm -C libmylib.so | grep "std::__cxx11"
```

4. 统一编译选项：

```
1 # 确保整个项目使用相同的设置
2 set(CMAKE_CXX_STANDARD 17)
3 set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

1.6.10.5. 查看详细调试信息

CMake 提供了多种方式来获取调试信息：

cmake --debug-find：

```
1 # 显示 find_package 的详细搜索过程
2 cmake --debug-find ..
3
4 # 输出类似：
5 # find_package considered the following locations for OpenCV's Config module:
6 #   /usr/lib/cmake/opencv4/OpenCVConfig.cmake
```

```
7 # ...
```

cmake --trace :

```
1 # 显示每条 CMake 命令的执行
2 cmake --trace ..
3
4 # 只追踪特定文件
5 cmake --trace-source=CMakeLists.txt ..
```

message 打印变量:

```
1 # 打印变量值
2 message(STATUS "OpenCV_VERSION: ${OpenCV_VERSION}")
3 message(STATUS "OpenCV_INCLUDE_DIRS: ${OpenCV_INCLUDE_DIRS}")
4 message(STATUS "OpenCV_LIBRARIES: ${OpenCV_LIBRARIES}")
5
6 # 打印列表 (更清晰)
7 foreach(lib ${OpenCV_LIBRARIES})
8     message(STATUS "  - ${lib}")
9 endforeach()
10
11 # 打印目标属性
12 get_target_property(inc mylib INCLUDE_DIRECTORIES)
13 message(STATUS "mylib includes: ${inc}")
14
15 # 调试信息 (只在特定条件下显示)
16 if(CMAKE_DEBUG)
17     message(STATUS "Debug: ${SOME_VAR}")
18 endif()
```

查看编译命令:

```
1 # Make
2 make VERBOSE=1
3
4 # Ninja
5 ninja -v
6
7 # CMake 通用方式
8 cmake --build . --verbose
9
10 # 生成 compile_commands.json
11 cmake -DCMAKE_EXPORT_COMPILE_COMMANDS=ON ..
12 cat compile_commands.json
```

查看缓存:

```
1 # 列出所有缓存变量
2 cmake -LA ..
3
4 # 列出所有变量 (包括高级) 及帮助信息
5 cmake -LAH ..
6
7 # 查看特定变量
8 grep "OpenCV" CMakeCache.txt
```

1.6.10.6. CMake 最低版本的选择

选择合适的 `cmake_minimum_required` 版本需要权衡:

版本太低的问题：

- 无法使用新特性
- 可能触发过时的行为
- 错过 bug 修复

版本太高的问题：

- 老系统的用户无法构建
- 限制了项目的兼容性

推荐的版本选择：

```
1 # 2024-2025 年的新项目推荐
2 cmake_minimum_required(VERSION 3.16)
3
4 # 需要更新特性时
5 cmake_minimum_required(VERSION 3.20)
6
7 # ROS 2 Humble 要求
8 cmake_minimum_required(VERSION 3.16)
9
10 # 广泛兼容性
11 cmake_minimum_required(VERSION 3.10)
12
13 # 使用版本范围（推荐）
14 cmake_minimum_required(VERSION 3.16...3.28)
```

各版本引入的重要特性：

- 3.0: 目标 INTERFACE 属性、生成器表达式
- 3.1: `target_compile_features`
- 3.8: C++17 支持
- 3.11: `FetchContent` 模块
- 3.12: `CONFIGURE_DEPENDS` for GLOB
- 3.14: `FetchContent_MakeAvailable`
- 3.16: 预编译头文件支持、统一构建
- 3.19: 预设 (presets) 支持
- 3.20: C++23 支持
- 3.21: 改进的消息颜色和格式

1.6.10.7. 常见的 CMake 反模式

以下是应该避免的做法：

反模式 1: 使用全局命令

```
1 # 不好：影响所有目标
2 include_directories(${SOME_INCLUDE_DIR})
3 link_libraries(somelib)
4 add_definitions(-DSOME_MACRO)
5
6 # 好：只影响特定目标
7 target_include_directories(myapp PRIVATE ${SOME_INCLUDE_DIR})
8 target_link_libraries(myapp PRIVATE somelib)
9 target_compile_definitions(myapp PRIVATE SOME_MACRO)
```

反模式 2: 修改 CMAKE_CXX_FLAGS

```

1 # 不好: 全局修改
2 set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra")
3
4 # 好: 目标级别设置
5 target_compile_options(myapp PRIVATE -Wall -Wextra)

```

反模式 3: 使用 file(GLOB) 收集源文件

```

1 # 不好: 新文件不会自动触发重新配置
2 file(GLOB SOURCES "src/*.cpp")
3 add_library(mylib ${SOURCES})
4
5 # 好: 手动列出源文件
6 add_library(mylib
7     src/file1.cpp
8     src/file2.cpp
9     src/file3.cpp
10 )

```

反模式 4: 不使用目标的 PUBLIC/PRIVATE

```

1 # 不好: 没有指定可见性
2 target_link_libraries(mylib otherlib)
3
4 # 好: 明确指定
5 target_link_libraries(mylib PRIVATE otherlib)

```

反模式 5: 源内构建

```

1 # 不好: 污染源代码目录
2 cd my_project
3 cmake .
4
5 # 好: 源外构建
6 cd my_project
7 mkdir build && cd build
8 cmake ..

```

反模式 6: 硬编码路径

```

1 # 不好: 不可移植
2 target_include_directories(myapp PRIVATE /home/user/libs/include)
3
4 # 好: 使用变量和 find_package
5 find_package(SomeLib REQUIRED)
6 target_link_libraries(myapp PRIVATE SomeLib::SomeLib)

```

反模式 7: 不使用导入目标

```

1 # 不好: 使用变量
2 target_include_directories(myapp PRIVATE ${OpenCV_INCLUDE_DIRS})
3 target_link_libraries(myapp PRIVATE ${OpenCV_LIBRARIES})
4
5 # 好: 使用导入目标
6 target_link_libraries(myapp PRIVATE OpenCV::OpenCV)

```

反模式 8: 在 find_package 后不检查结果

```

1 # 不好: 如果找不到会静默失败

```

```

2 find_package(SomeLib)
3 target_link_libraries(myapp PRIVATE ${SomeLib_LIBRARIES})
4
5 # 好: 使用 REQUIRED 或检查结果
6 find_package(SomeLib REQUIRED)
7 # 或
8 find_package(SomeLib)
9 if(NOT SomeLib_FOUND)
10     message(FATAL_ERROR "SomeLib not found")
11 endif()

```

1.6.10.8. 现代 CMake 最佳实践总结

最后, 让我们总结现代 CMake 的最佳实践:

项目设置:

```

1 # 要求合理的 CMake 版本
2 cmake_minimum_required(VERSION 3.16)
3
4 # 声明项目信息
5 project(MyProject
6     VERSION 1.0.0
7     DESCRIPTION "My awesome project"
8     LANGUAGES CXX
9 )
10
11 # 禁止源内构建
12 if(CMAKE_SOURCE_DIR STREQUAL CMAKE_BINARY_DIR)
13     message(FATAL_ERROR "In-source builds are not allowed")
14 endif()
15
16 # 设置默认构建类型
17 if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
18     set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
19 endif()
20
21 # 导出编译命令 (便于 IDE 和工具使用)
22 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

```

目标定义:

```

1 # 创建库
2 add_library(mylib
3     src/file1.cpp
4     src/file2.cpp
5 )
6
7 # 创建别名 (便于内部使用一致的命名)
8 add_library(MyProject::mylib ALIAS mylib)
9
10 # 设置目标属性
11 target_include_directories(mylib
12     PUBLIC
13         $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
14         $<INSTALL_INTERFACE:include>
15     PRIVATE
16         ${CMAKE_CURRENT_SOURCE_DIR}/src
17 )
18
19 target_compile_features(mylib PUBLIC cxx_std_17)
20

```

```

21 target_compile_options(mylib PRIVATE
22     $<$<CXX_COMPILER_ID:GNU,Clang>:-Wall -Wextra>
23     $<$<CXX_COMPILER_ID:MSVC>:/W4>
24 )

```

依赖管理:

```

1 # 使用 find_package 查找已安装的库
2 find_package(OpenCV REQUIRED)
3
4 # 使用导入目标
5 target_link_libraries(mylib
6     PUBLIC Eigen3::Eigen
7     PRIVATE OpenCV::OpenCV
8 )
9
10 # 可选依赖的处理
11 find_package(CUDA QUIET)
12 if(CUDA_FOUND)
13     target_compile_definitions(mylib PUBLIC HAS_CUDA)
14     target_link_libraries(mylib PRIVATE CUDA::cudart)
15 endif()
16
17 # 使用 FetchContent 获取缺失的依赖
18 include(FetchContent)
19 FetchContent_Declare(fmt
20     GIT_REPOSITORY https://github.com/fmtlib/fmt.git
21     GIT_TAG 10.1.1
22 )
23 FetchContent_MakeAvailable(fmt)

```

安装和导出:

```

1 include(GNUInstallDirs)
2 include(CMakePackageConfigHelpers)
3
4 # 安装目标
5 install(TARGETS mylib
6     EXPORT MyProjectTargets
7     LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
8     ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
9     RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
10    INCLUDES DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
11 )
12
13 # 安装头文件
14 install(DIRECTORY include/
15     DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
16 )
17
18 # 导出目标
19 install(EXPORT MyProjectTargets
20     FILE MyProjectTargets.cmake
21     NAMESPACE MyProject::
22     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
23 )
24
25 # 生成配置文件
26 configure_package_config_file(
27     cmake/MyProjectConfig.cmake.in
28     ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake
29     INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject

```

```

30 )
31
32 write_basic_package_version_file(
33     ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake
34     VERSION ${PROJECT_VERSION}
35     COMPATIBILITY SameMajorVersion
36 )
37
38 install(FILES
39     ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake
40     ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake
41     DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
42 )

```

测试:

```

1 # 使用 BUILD_TESTING 控制测试构建
2 include(CTest)
3
4 if(BUILD_TESTING)
5     find_package(GTest REQUIRED)
6
7     add_executable(mylib_test test/test_mylib.cpp)
8     target_link_libraries(mylib_test PRIVATE
9         mylib
10        GTest::gtest_main
11    )
12
13    include(GoogleTest)
14    gtest_discover_tests(mylib_test)
15 endif()

```

项目结构建议:

```

1 my_project/
2 |— CMakeLists.txt           # 顶层配置
3 |— cmake/                   # CMake 模块
4 |   |— MyProjectConfig.cmake.in
5 |   |— CompilerWarnings.cmake
6 |— include/my_project/      # 公开头文件
7 |— src/                     # 源文件和私有头文件
8 |— apps/                    # 可执行文件
9 |— tests/                   # 测试
10 |— docs/                    # 文档

```

掌握这些最佳实践，你就能写出清晰、可维护、跨平台的 CMake 配置。现代 CMake 的核心理念是“以目标为中心”——一切配置都围绕目标进行，通过 PUBLIC/PRIVATE/INTERFACE 控制属性的传播。这种方式让构建配置更加模块化，依赖管理更加自动化，是当前 C++ 项目构建的最佳实践。

1.7. 软件工程基础

1.7.1. 为什么需要软件工程

你写了一个程序，它能跑。在你的电脑上，用你准备的测试数据，它完美地完成了任务。你很满意，提交代码，转身去做下一件事。三个月后，队友问你这段代码是做什么的，你盯着屏幕，发现自己也看不懂了。半年后，新赛季开始，需要在这个基础上开发新功能，却发现代码像一团乱麻，牵一发而动全身，没人敢改。一年后，你毕业了，后辈们面对这份“遗产”，选择了重写。

这个场景在学生团队中反复上演。代码“能跑”只是最低标准，而“能维护”才是真正的挑战。软件工程就是关于如何写出能维护的代码、如何让团队高效协作、如何让项目可持续发展的学问。它不是象牙塔里的理论，而是无数程序员踩过无数坑后总结出的实践智慧。

1.7.1.1. 从个人项目到团队协作

一个人写代码和一群人写代码是完全不同的事情。

个人项目中，所有的上下文都在你的脑子里。你知道每个变量的含义，知道为什么要这样设计，知道哪些地方是临时凑合的。你不需要写注释，因为你自己能看懂；你不需要规范命名，因为你知道 `tmp2` 是什么意思；你不需要考虑接口设计，因为所有代码都是你写的。这种方式在小项目、短周期内是可行的，甚至是高效的。

但当项目变成团队协作时，一切都变了。你的队友不知道你脑子里的上下文。当他看到 `process()` 函数时，他不知道它处理什么、输入输出是什么、有什么前置条件、会不会抛异常。当他需要修改你的代码时，他不知道哪些地方可以安全地改，哪些地方有隐含的依赖。当他想复用你的模块时，他不知道该调用哪个函数、参数该怎么传。

团队协作要求代码是“自解释”的——通过清晰的命名、合理的结构、必要的注释，让任何一个新来的人都能理解代码在做什么。它要求代码是“可预测”的——遵循统一的规范，让人能根据约定推断代码的行为。它要求代码是“可演进”的——通过良好的模块化和接口设计，让修改一个地方不会意外地破坏另一个地方。

这些要求听起来很抽象，但它们会体现在具体的实践中：命名规范让变量名一目了然，代码格式让风格统一易读，设计模式让结构清晰可扩展，单元测试让修改有信心，文档让知识得以传承。这就是软件工程的内容。

1.7.1.2. 代码的生命周期

一段代码从诞生到退役，会经历漫长的生命周期。编写代码只是开始，之后的阅读、调试、修改、扩展、维护才是主旋律。据统计，程序员花在阅读代码上的时间是编写代码的十倍以上。这意味着，让代码易于阅读比让代码易于编写更重要。

在 RoboMaster 的语境下，这个生命周期与赛季节奏紧密相连。备赛期间写的代码，会在比赛中经受考验；比赛中发现的问题，会在赛后修复和改进；这个赛季的代码，会成为下个赛季的基础；老队员的代码，会交接给新队员维护。一段视觉识别的代码，可能会被五六届学生接力修改，持续服役好几年。

当你写代码时，想象一下半年后的自己、一年后的队友、两年后的后辈会如何看待这段代码。他们能理解它吗？能安全地修改它吗？能在它基础上扩展新功能吗？如果答案是否定的，那么你正在给未来的人挖坑——而那个人很可能就是你自己。

1.7.1.3. 技术债务

技术债务是软件工程中的一个重要概念。就像金融债务一样，技术债务是为了短期利益而做出的妥协，这些妥协会在未来产生“利息”——以更多的维护成本、更高的 bug 率、更慢的开发速度的形式体现。

技术债务有很多形式：为了赶进度而写的临时代码（“先这样，之后再改”），没有测试覆盖的功能（“能跑就行”），缺失的文档（“代码就是文档”），过时的依赖（“能用就不要升级”），绕过问题而不是解决问题的 workaround（“不知道为什么，但加上这行就好了”）。每一个这样的决定都在积累债务。

适度的技术债务有时是合理的。比赛前夜紧急修复 bug，你没有时间写完美的代码，先让它能工作是正确的选择。但关键是要意识到这是债务，要记录下来，要在之后偿还。问题在于，大多数技术债务是无意识积累的，没有人记得，直到债务多到压垮项目。

RoboMaster 团队特别容易积累技术债务。赛季紧张，时间永远不够；队员流动，知识容易丢失；成果导向，能跑的代码比好的代码更受认可。但技术债务的利息是残酷的：赛季中期，代码已经乱到没人敢改，只能不断打补丁；新赛季开始，想加新功能却发现无从下手，不得不大规模重构甚至重写；新队员接手，看不懂代码，只能猜测和试错。

软件工程的实践——代码规范、设计模式、测试、文档——本质上都是在控制技术债务。它们需要前期投入，但会在后期节省大量时间。这就像定期清洁房间：每天花十分钟整理，比每月花一整天大扫除要轻松得多。

1.7.1.4. RoboMaster 赛季的教训

在 RoboMaster 的实际开发中，缺乏软件工程意识会导致一系列典型问题。

备赛初期，大家各自为战，快速产出代码。每个人有自己的编码风格，有人用驼峰命名，有人用下划线，有人混用；有人喜欢大函数，有人喜欢小函数；有人写注释，有人不写。代码看起来像是几种语言的混合体。

赛季中期，需要整合各个模块。这时发现接口对不上，A 模块输出的数据格式和 B 模块期望的不一样；发现依赖冲突，两个模块用了同一个库的不同版本；发现功能重复，三个人写了三套坐标变换的代码。整合工作变成了痛苦的调试和修补。

比赛前夕，压力最大的时候。bug 层出不穷，但没人敢大改，因为不知道会不会引入新问题。代码里到处是注释掉的旧代码和临时的调试输出，没人记得哪些是有用的。参数硬编码在代码里，调一个参数要重新编译。

赛季结束，需要总结和交接。发现没有文档，只有一堆代码；代码里的魔法数字没人记得含义；一些关键的设计决策没有记录，只有当事人知道为什么。新队员接手后，只能靠读代码和猜测来理解系统。

这些问题不是命中注定的。好的软件工程实践可以避免大部分问题：统一的代码规范让代码风格一致，清晰的接口设计让模块整合顺畅，完善的测试让修改有信心，充分的文档让知识得以传承。代价是前期多花一些时间，但回报是本赛季的顺畅和多年的可持续发展。

1.7.1.5. 本章内容概览

本章将介绍软件工程的核心实践，帮助你从“能跑的代码”进化到“能维护的代码”。

我们从代码规范与风格开始。统一的代码风格是团队协作的基础。我们会介绍业界广泛采用的 Google C++ Style Guide 的核心要点，包括命名、格式、注释等方面的规范，以及如何使用 clang-format 等工具自动化地保持代码风格一致。

接下来是设计模式。设计模式是前人解决常见问题的经验总结。我们会探讨设计模式的起源和基本原则，然后详细介绍机器人开发中最实用的几种模式：单例模式管理全局资源、工厂模式创建对象、观察者模式实现事件通知、策略模式切换算法、状态模式管理行为状态。理解这些模式，你会发现很多设计问题都有现成的优雅解法。

然后是单元测试。测试是保障代码质量的关键手段。我们会介绍为什么要写测试、测试金字塔的概念、Google Test 框架的使用方法，以及如何写出易于测试的代码。在 RoboMaster 开发中，对算法模块进行单元测试可以大大提高代码的可靠性。

调试技巧是每个开发者必备的技能。我们会介绍 GDB 调试器的使用、日志系统的设计、内存调试工具的使用,帮助你高效地定位和解决问题。好的调试技能可以把“不知道为什么工作”变成“知道问题在哪里以及如何修复”。

性能分析与优化讲的是如何让程序跑得更快。我们会介绍性能分析的工作流、时间测量的方法、perf 和火焰图等分析工具的使用,以及常见的优化方向。在 RoboMaster 的实时系统中,性能优化往往是决定胜负的关键。

文档编写是经常被忽视但极其重要的工程产出。我们会讨论文档的价值、README 的写法、Doxygen 生成 API 文档的方法,以及如何做好赛季交接文档。好的文档是团队知识的载体,是项目可持续发展的保障。

最后,我们会回顾版本控制与协作的最佳实践,包括分支策略、提交规范、代码审查和持续集成。这部分与后续的 Git 章节相呼应,从软件工程的角度讨论团队协作的工作流。

软件工程不是一堆枯燥的规则,而是让开发变得更轻松、更可控、更有成就感的方法论。当你的代码清晰易读、你的测试给你信心、你的文档让交接顺畅、你的团队高效协作时,你会感谢当初在软件工程上的投入。让我们开始这段旅程。

1.7.2. 代码规范与风格

打开一个陌生的代码库,你最先注意到的是什么?不是算法的精妙,也不是架构的优雅,而是代码的“样子”——变量是怎么命名的、缩进用的是空格还是制表符、大括号放在行尾还是另起一行、注释写了什么。这些看似琐碎的细节,构成了代码的第一印象,也在很大程度上决定了你理解这段代码的难易程度。代码规范就是关于这些细节的约定,它让代码成为团队的共同语言。

1.7.2.1. 为什么需要统一的代码规范

每个程序员都有自己的编码习惯。有人喜欢用 `camelCase` 命名变量,有人偏爱 `snake_case`;有人习惯在运算符两边加空格,有人觉得紧凑一些更好;有人写详尽的注释,有人信奉“好代码不需要注释”。当一个人独自开发时,这些差异无关紧要。但当多人协作时,如果每个人都按照自己的风格编写代码,代码库很快就会变成一锅大杂烩。

想象一下这样的场景:你打开一个文件,前半部分变量名是 `imageWidth`,后半部分变成了 `image_height`;有的函数用四个空格缩进,有的用制表符;有的大括号跟在函数声明后面,有的另起一行。这种风格的不一致会给阅读者带来额外的认知负担——你的大脑需要不断地适应不同的风格,而不能专注于理解代码的逻辑。更糟糕的是,当你需要修改这样的代码时,你不知道应该遵循哪种风格,于是又增加了一种新的风格,混乱进一步加剧。

统一的代码规范解决的正是这个问题。当团队所有成员遵循相同的规范时,代码库呈现出一致的外观,仿佛出自同一人之手。新成员加入时,只需要学习一套规范,就能阅读和编写符合团队标准的代码。代码审查时,审查者可以专注于逻辑和设计,而不是争论风格问题。版本控制中的差异也会更加清晰——如果没有规范,一次简单的修改可能因为格式调整而产生大量无关的改动,淹没真正的变化。

选择哪种规范并不是最重要的,重要的是团队有一个规范并且严格遵守。业界已经有许多成熟的代码规范可供参考,其中 Google C++ Style Guide 是最广泛使用的 C++ 规范之一。它不仅规定了格式和命名,还包含了许多关于 C++ 语言使用的建议,是学习现代 C++ 最佳实践的好材料。接下来,我们以 Google C++ Style Guide 为基础,介绍 C++ 代码规范的核心要点。

1.7.2.2. 命名规范

命名是编程中最重要也最困难的事情之一。好的命名能让代码自解释，读者一眼就能明白变量的含义、函数的功能；糟糕的命名则让代码变成谜语，需要仔细阅读实现才能理解意图。

Google C++ Style Guide 对不同类型的标识符规定了不同的命名风格，这种差异化的命名让读者能够从名字本身判断标识符的类型。

文件名使用小写字母，单词之间用下划线连接。头文件使用 `.h` 扩展名，源文件使用 `.cpp` 扩展名。文件名应该反映其内容，通常与其中定义的主要类同名。例如，定义 `ImageProcessor` 类的文件应该命名为 `image_processor.h` 和 `image_processor.cpp`。

```
1 // 文件命名示例
2 image_processor.h      // 头文件
3 image_processor.cpp    // 源文件
4 robot_controller.h
5 camera_driver.cpp
```

类型名称使用大驼峰命名法 (PascalCase)，即每个单词首字母大写，不使用下划线。这包括类、结构体、类型别名、枚举类型和模板参数。这种命名风格让类型名称一眼就能与变量名区分开来。

```
1 // 类型命名示例
2 class ImageProcessor;
3 struct SensorData;
4 enum class RobotState;
5 using TargetList = std::vector<Target>;
6
7 template <typename DataType> // 模板参数也用大驼峰
8 class CircularBuffer;
```

变量名使用小写字母加下划线 (snake_case)。这包括局部变量、函数参数和公有成员变量。对于类的私有和保护成员变量，在名称末尾加一个下划线，以便与局部变量区分。这个小小的约定非常有用——当你看到 `image_width_` 时，立即知道它是成员变量而非局部变量。

```
1 // 变量命名示例
2 int image_width;           // 局部变量
3 double detection_threshold; // 函数参数
4
5 class Camera {
6 public:
7     int frame_count;       // 公有成员（如果有的话）
8
9 private:
10    int image_width_;       // 私有成员，末尾加下划线
11    double exposure_time_;
12    std::string device_path_;
13 };
```

函数名使用大驼峰命名法，与类型名相同。函数名应该是动词或动词短语，描述函数执行的动作。访问器 (getter) 和修改器 (setter) 可以使用与变量类似的命名，如 `image_width()` 和 `set_image_width()`。

```
1 // 函数命名示例
2 void ProcessImage();
3 bool DetectTarget();
4 int CalculateDistance();
```

```

5
6 // 访问器和修改器
7 int image_width() const { return image_width_; }
8 void set_image_width(int width) { image_width_ = width; }

```

常量和枚举值使用 `k` 前缀加大驼峰命名。这种命名让常量一眼就能被识别出来，避免与变量混淆。宏定义则使用全大写字母加下划线，但现代 C++ 中应尽量避免使用宏。

```

1 // 常量命名示例
2 const int kMaxBufferSize = 1024;
3 constexpr double kPi = 3.14159265358979;
4
5 enum class Color {
6     kRed,
7     kGreen,
8     kBlue
9 };
10
11 // 宏命名（尽量避免使用宏）
12 #define DEPRECATED_FUNCTION __attribute__((deprecated))

```

命名空间使用小写字母，通常是项目名或模块名的缩写。命名空间用于避免全局命名冲突，尤其在大型项目中非常重要。

```

1 // 命名空间示例
2 namespace rm { // RoboMaster 项目
3     namespace vision { // 视觉模块
4         class Detector { /* ... */ };
5     } // namespace vision
6 } // namespace rm
7
8 // 使用
9 rm::vision::Detector detector;

```

除了遵循格式规则，命名还有更本质的要求：名称应该准确、完整地描述其代表的含义。避免使用过于简短或含糊的名称，如 `temp`、`data`、`info`、`process()`；也避免使用过于冗长的名称，如 `the_current_image_frame_from_camera`。好的名称应该在准确和简洁之间取得平衡。在循环中使用 `i`、`j` 作为索引变量是可以接受的约定，但在其他场景下，应该使用更有意义的名称。

```

1 // 不好的命名
2 int n; // 什么的数量？
3 void Process(); // 处理什么？怎么处理？
4 std::vector<int> data; // 什么数据？
5
6 // 好的命名
7 int target_count;
8 void ProcessImage();
9 std::vector<int> detection_scores;

```

1.7.2.3. 格式规范

格式规范涉及代码的视觉呈现：缩进、空格、换行、大括号位置等。良好的格式让代码结构一目了然，就像排版精美的书籍比手写稿更易阅读。

缩进使用空格而非制表符，每级缩进 2 个或 4 个空格（Google 风格是 2 个，许多团队选择 4 个）。制表符在不同编辑器中可能显示为不同宽度，导致代码在你的屏幕上对齐完美，在别人的屏幕上却参差不齐。使用空格可以避免这个问题。

```
1 // 缩进示例 (2 空格)
2 class Robot {
3     void Move() {
4         if (is_enabled_) {
5             for (int i = 0; i < 10; ++i) {
6                 Step();
7             }
8         }
9     }
10 };
```

每行代码的长度应该限制在一定范围内，通常是 80 或 100 个字符。过长的行需要水平滚动才能看完，降低了可读性。当一行太长时，应该合理地换行。函数调用参数过多时，可以每个参数一行；长表达式可以在运算符处断开。

```
1 // 长行换行示例
2 void ProcessTarget(const Target& target,
3                   const CameraParams& camera_params,
4                   const GimbalState& gimbal_state,
5                   std::vector<Result*> results);
6
7 // 长表达式换行
8 double distance = std::sqrt(
9     (target.x - origin.x) * (target.x - origin.x) +
10    (target.y - origin.y) * (target.y - origin.y) +
11    (target.z - origin.z) * (target.z - origin.z));
```

大括号的位置是一个经典的“圣战”话题。Google C++ Style Guide 规定：函数的左大括号另起一行，其他情况（类定义、控制语句等）左大括号放在行尾。但实际上，许多团队选择统一将左大括号放在行尾（K&R 风格），或者统一另起一行（Allman 风格）。无论选择哪种，团队内部保持一致即可。

```
1 // Google 风格：函数大括号另起一行，其他在行尾
2 class Robot {
3     void Move() {
4         if (is_enabled_) {
5             // ...
6         }
7     }
8 };
9
10 // K&R 风格：所有大括号都在行尾
11 class Robot {
12     void Move() {
13         if (is_enabled_) {
14             // ...
15         }
16     }
17 };
18
19 // Allman 风格：所有大括号都另起一行
20 class Robot
21 {
22     void Move()
23     {
```

```

24     if (is_enabled_)
25     {
26         // ...
27     }
28 }
29 };

```

空格的使用应该一致且有助于可读性。二元运算符两边加空格，一元运算符不加；逗号和分号后面加空格，前面不加；控制语句的关键字与括号之间加空格，函数名与括号之间不加。

```

1 // 空格使用示例
2 int sum = a + b * c;           // 二元运算符两边加空格
3 int neg = -value;             // 一元运算符不加
4 Call(arg1, arg2, arg3);       // 逗号后加空格
5 for (int i = 0; i < n; ++i)    // for 后加空格，分号后加空格
6 if (condition) {              // if 后加空格
7     DoSomething();            // 函数名与括号之间不加
8 }

```

空行用于分隔逻辑上独立的代码块。函数之间用一个空行分隔；函数内部，不同逻辑步骤之间可以用空行分隔；但不要过度使用空行，导致代码过于稀疏。头文件的 `#include` 部分，通常按组分隔：系统头文件、第三方库头文件、项目头文件，每组之间用空行分隔。

```

1 #include <vector>
2 #include <string>
3
4 #include <opencv2/opencv.hpp>
5 #include <Eigen/Dense>
6
7 #include "robot/vision/detector.h"
8 #include "robot/common/types.h"

```

1.7.2.4. 注释规范

注释是代码与人之间的对话。好的注释解释“为什么”而不是“是什么”——代码本身已经说明了它在做什么，注释应该补充代码无法表达的信息：为什么选择这种方案、这里有什么需要注意的陷阱、这个魔法数字的来源是什么。

文件头注释出现在每个文件的开头，说明文件的用途、作者、创建日期、版权信息等。不同团队对文件头的要求不同，有的要求详尽，有的比较简略。

```

1 // Copyright 2024 RoboMaster Team. All rights reserved.
2 //
3 // Licensed under the MIT License.
4 //
5 // Author: Zhang San <zhangsan@example.com>
6 // Date: 2024-01-15
7 //
8 // This file implements the armor detector for RoboMaster robots.
9 // The detector uses color and shape features to identify enemy armor plates.

```

类注释放在类定义之前，说明类的用途、使用方法和注意事项。对于复杂的类，还应该说明其线程安全性、生命周期管理等。

```

1 // ArmorDetector detects enemy armor plates from camera images.
2 //
3 // Usage:

```

```

4 // ArmorDetector detector(config);
5 // detector.Init();
6 // auto armors = detector.Detect(image);
7 //
8 // Thread safety: This class is NOT thread-safe. Each thread should
9 // create its own instance.
10 class ArmorDetector {
11     // ...
12 };

```

函数注释放在函数声明之前，说明函数的功能、参数含义、返回值和可能抛出的异常。对于简单的函数，如果函数名已经足够清晰，可以省略注释。

```

1 // Detects armor plates in the given image.
2 //
3 // Args:
4 //   image: The input BGR image from camera.
5 //   timestamp: The timestamp when the image was captured.
6 //
7 // Returns:
8 //   A vector of detected armor plates, sorted by confidence.
9 //   Returns an empty vector if no armor is detected.
10 //
11 // Throws:
12 //   std::invalid_argument if image is empty.
13 std::vector<Armor> Detect(const cv::Mat& image, double timestamp);

```

行内注释用于解释单行或几行代码。它们应该放在代码的上方或右侧，解释代码的意图或需要注意的地方。避免写没有信息量的注释，如 `i++; // increment i`。

```

1 // Apply bilateral filter to reduce noise while preserving edges.
2 // Bilateral filter is slower than Gaussian but better for armor detection.
3 cv::bilateralFilter(image, filtered, 9, 75, 75);
4
5 int retry_count = 0;
6 const int kMaxRetries = 3; // Determined by network latency tests
7 while (retry_count < kMaxRetries) {
8     // ...
9 }

```

TODO 注释用于标记需要后续处理的地方。它们应该包含具体的任务描述，最好还有负责人和预计完成时间。TODO 不应该长期存在——如果一个 TODO 超过一个月还没有处理，要么应该立即处理，要么应该删除或转为正式的任务跟踪。

```

1 // TODO(zhangsan): Optimize this loop using SIMD. Current implementation
2 // is too slow for 60fps processing. Expected completion: 2024-02.
3 for (int i = 0; i < n; ++i) {
4     // ...
5 }

```

1.7.2.5. 头文件规范

头文件的组织对大型项目的编译效率和模块化至关重要。良好的头文件实践可以减少编译依赖、加快编译速度、避免重复包含等问题。

防止头文件重复包含有两种方式：`#pragma once` 和传统的 `#ifndef` 守卫。`#pragma once` 更简洁，被所有主流编译器支持，是现代 C++ 推荐的方式。

```

1 // 推荐: 使用 #pragma once
2 #pragma once
3
4 class MyClass {
5     // ...
6 };
7
8 // 传统方式: #ifndef 守卫
9 #ifndef PROJECT_MODULE_MY_CLASS_H_
10 #define PROJECT_MODULE_MY_CLASS_H_
11
12 class MyClass {
13     // ...
14 };
15
16 #endif // PROJECT_MODULE_MY_CLASS_H_

```

头文件中应该只包含必要的声明，实现放在源文件中。尽量使用前向声明（forward declaration）代替包含头文件，减少编译依赖。如果只需要使用指针或引用，就不需要包含完整的类定义。

```

1 // my_class.h
2 #pragma once
3
4 class OtherClass; // 前向声明, 不需要 #include "other_class.h"
5
6 class MyClass {
7 public:
8     void Process(OtherClass* obj); // 只用指针, 不需要完整定义
9
10 private:
11     OtherClass* other_; // 只用指针, 不需要完整定义
12 };
13
14 // my_class.cpp
15 #include "my_class.h"
16 #include "other_class.h" // 实现时才需要完整定义
17
18 void MyClass::Process(OtherClass* obj) {
19     obj->DoSomething(); // 调用成员函数需要完整定义
20 }

```

`#include` 的顺序有助于发现遗漏的依赖。推荐的顺序是：首先包含当前文件对应的头文件（如 `foo.cpp` 首先包含 `foo.h`），然后是系统头文件，接着是第三方库头文件，最后是项目内部头文件。每组之间用空行分隔，每组内部按字母顺序排列。

```

1 // image_processor.cpp
2 #include "vision/image_processor.h" // 对应的头文件放第一个
3
4 #include <algorithm>
5 #include <vector>
6
7 #include <opencv2/opencv.hpp>
8 #include <Eigen/Dense>
9
10 #include "common/config.h"
11 #include "vision/detector.h"

```

将对应的头文件放在第一个是一个聪明的技巧：如果这个头文件缺少某些必要的 `#include`，编译这个源文件时会立即报错，而不是等到其他文件包含它时才发现问题。

1.7.2.6. 现代 C++ 的额外建议

Google C++ Style Guide 最初制定时，C++11 还未发布。随着现代 C++ 的发展，一些额外的建议值得补充。

优先使用 `auto` 进行类型推导，特别是当类型很长或很明显时。但不要滥用——当类型对理解代码很重要时，显式写出类型更好。

```
1 // 好：类型很长或很明显
2 auto iter = container.begin();
3 auto result = std::make_unique<MyClass>();
4 auto lambda = [](int x) { return x * 2; };
5
6 // 好：显式类型让意图更清晰
7 double ratio = GetRatio(); // 而不是 auto ratio = GetRatio();
```

使用初始化列表语法（`{}`）初始化变量，它可以防止窄化转换，更加安全。但要注意 `std::vector` 等容器的特殊情况。

```
1 int value{42};
2 std::string name{"robot"};
3 std::vector<int> sizes{1, 2, 3}; // 包含三个元素
4
5 // 注意区分
6 std::vector<int> v1(5); // 5 个元素，值为 0
7 std::vector<int> v2{5}; // 1 个元素，值为 5
8 std::vector<int> v3(5, 1); // 5 个元素，值为 1
```

使用 `nullptr` 而不是 `NULL` 或 `0` 表示空指针。`nullptr` 有明确的类型，可以避免函数重载时的歧义。

```
1 void Process(int value);
2 void Process(int* ptr);
3
4 Process(NULL); // 歧义：可能调用 Process(int)
5 Process(nullptr); // 明确：调用 Process(int*)
```

使用范围 `for` 循环遍历容器，更简洁也更不容易出错。

```
1 std::vector<Target> targets = GetTargets();
2
3 // 传统方式
4 for (size_t i = 0; i < targets.size(); ++i) {
5     Process(targets[i]);
6 }
7
8 // 现代方式
9 for (const auto& target : targets) {
10     Process(target);
11 }
```

使用 `enum class` 而不是普通 `enum`，避免枚举值污染外层命名空间，也更类型安全。

```
1 // 不好：枚举值泄漏到外层
2 enum Color { Red, Green, Blue };
3 int Red = 5; // 错误：重定义
4
5 // 好：枚举值限定在枚举类内
6 enum class Color { kRed, kGreen, kBlue };
```



```

7 int Red = 5; // OK
8 Color c = Color::kRed;

```

使用智能指针管理动态内存，避免手动 `new` 和 `delete`。优先使用 `std::unique_ptr`，只在需要共享所有权时使用 `std::shared_ptr`。

```

1 // 不好：手动管理内存
2 MyClass* obj = new MyClass();
3 // ... 如果中间抛出异常，内存泄漏
4 delete obj;
5
6 // 好：使用智能指针
7 auto obj = std::make_unique<MyClass>();
8 // 离开作用域自动释放，异常安全

```

1.7.2.7. 工具辅助

手动维护代码格式既繁琐又容易遗漏。幸运的是，有成熟的工具可以自动化这项工作。

`clang-format` 是 LLVM 项目提供的代码格式化工具，支持多种预设风格 (Google、LLVM、Chromium 等)，也可以通过配置文件自定义。它可以集成到编辑器中，在保存文件时自动格式化，或者作为 CI 检查的一部分。

```

1 # 使用 Google 风格格式化文件
2 clang-format -style=Google -i my_file.cpp
3
4 # 使用配置文件格式化
5 clang-format -style=file -i my_file.cpp
6
7 # 检查是否符合格式（用于 CI）
8 clang-format -style=file --dry-run --Werror my_file.cpp

```

`clang-format` 的配置文件名为 `.clang-format`，放在项目根目录下。以下是一个基于 Google 风格的配置示例，适合 RoboMaster 项目使用：

```

1 # .clang-format
2 BasedOnStyle: Google
3
4 # 缩进设置
5 IndentWidth: 4
6 TabWidth: 4
7 UseTab: Never
8 AccessModifierOffset: -4
9
10 # 行宽限制
11 ColumnLimit: 100
12
13 # 大括号风格
14 BreakBeforeBraces: Attach
15
16 # 指针和引用的对齐
17 PointerAlignment: Left
18 ReferenceAlignment: Left
19
20 # 头文件排序
21 SortIncludes: true
22 IncludeBlocks: Regroup
23 IncludeCategories:
24   - Regex: '^<.*>'

```



```

25     Priority: 1
26   - Regex: '^".*"'
27     Priority: 2
28
29   # 其他
30   AllowShortFunctionsOnASingleLine: Empty
31   AllowShortIfStatementsOnASingleLine: Never
32   AllowShortLoopsOnASingleLine: false

```

clang-tidy 是另一个 LLVM 工具，用于静态代码分析。它不仅检查格式，还能发现潜在的 bug、性能问题和不符合最佳实践的代码。clang-tidy 可以配置启用哪些检查，并能自动修复某些问题。

```

1  # 运行 clang-tidy 检查
2  clang-tidy my_file.cpp -- -std=c++17
3
4  # 自动修复可修复的问题
5  clang-tidy -fix my_file.cpp -- -std=c++17

```

在 VS Code 中，可以安装 C/C++ 扩展和 Clang-Format 扩展，配置保存时自动格式化。在 CLion 中，clang-format 支持是内置的。将格式化工具集成到开发流程中，可以让团队成员无需刻意关注格式问题，工具会自动处理。

1.7.2.8. 代码审查

代码审查（Code Review）是指在代码合并到主分支之前，由其他团队成员检查代码的过程。它是保障代码质量、传播知识、统一风格的重要实践。

代码审查的价值远不止于发现 bug。通过审查，团队成员可以相互学习——资深成员可以指导新成员，新成员的新鲜视角有时也能发现资深成员忽视的问题。审查过程中的讨论可以统一团队对技术问题的认识，形成共同的最佳实践。知道代码会被审查，编写者也会更加用心，不会轻易提交敷衍的代码。

代码审查应该关注什么？首先是正确性——代码是否实现了预期的功能，是否有逻辑错误，边界条件是否处理正确。其次是可维护性——代码是否清晰易懂，命名是否恰当，结构是否合理，是否有足够的注释。然后是一致性——代码是否符合团队规范，是否与代码库中其他部分风格一致。最后是性能和安全——是否有明显的性能问题，是否存在安全隐患。

作为审查者，应该提供建设性的反馈，而不是简单地批评。指出问题的同时，最好给出改进建议。对于风格问题，如果有工具可以自动检查，就不需要在审查中花费过多精力。尊重被审查者的工作，认可做得好的地方。

作为被审查者，应该以开放的心态接受反馈。审查的目的是改进代码，而不是评价人。对于不同意的反馈，可以讨论，但要用事实和理由说服对方，而不是固执己见。将审查意见视为学习机会，而不是对自己的否定。

- 1 代码审查清单：
- 2 ☐ 代码是否实现了需求描述的功能？
- 3 ☐ 逻辑是否正确？边界条件是否处理？
- 4 ☐ 命名是否清晰、一致？
- 5 ☐ 代码结构是否清晰？函数是否过长？
- 6 ☐ 注释是否充分？是否解释了“为什么”？
- 7 ☐ 是否有重复代码可以提取？
- 8 ☐ 错误处理是否完善？
- 9 ☐ 是否有明显的性能问题？

- 10 ☐ 是否符合团队代码规范?
- 11 ☐ 测试是否充分?

1.7.2.9. RoboMaster 团队代码规范建议

基于以上讨论, 这里为 RoboMaster 团队提供一些具体的代码规范建议。这些建议可以作为起点, 团队可以根据自己的情况调整。

关于命名, 建议使用 Google 风格: 类型用 `PascalCase`, 变量和函数参数用 `snake_case`, 成员变量末尾加下划线, 常量用 `kPascalCase`。对于 RoboMaster 项目中的特定概念, 建议统一术语: 敌方装甲板用 `Armor`, 云台用 `Gimbal`, 底盘用 `Chassis`, 自瞄用 `AutoAim` 等。

关于格式, 建议使用 4 空格缩进、100 字符行宽、K&R 大括号风格。这与 ROS 社区的习惯相近, 也是许多团队的偏好。使用 `clang-format` 自动化格式检查, 将配置文件纳入版本控制。

关于项目结构, 建议按功能模块组织代码: `vision/` (视觉)、`control/` (控制)、`communication/` (通信)、`common/` (公共) 等。每个模块有自己的头文件目录和源文件目录。使用命名空间与目录结构对应, 如 `rm::vision`、`rm::control`。

```
1 robomaster_project/
2 |— CMakeLists.txt
3 |— .clang-format
4 |— README.md
5 |— include/
6 |   |— rm/
7 |       |— vision/
8 |           |— detector.h
9 |           |— tracker.h
10 |       |— control/
11 |           |— gimbal_controller.h
12 |           |— chassis_controller.h
13 |       |— common/
14 |           |— types.h
15 |           |— config.h
16 |— src/
17 |   |— vision/
18 |   |— control/
19 |   |— common/
20 |— test/
21 |   |— vision/
22 |   |— control/
```

关于注释, 建议所有公开的类和函数都有文档注释, 说明功能、参数和返回值。对于复杂的算法, 添加解释原理的注释或引用相关论文。对于临时的解决方案或已知的限制, 使用 `TODO` 或 `FIXME` 标记。

关于代码审查, 建议所有代码在合并前至少经过一人审查。对于核心模块的修改, 建议有两人审查。审查时使用检查清单, 确保不遗漏重要方面。将代码审查作为团队文化的一部分, 而不是可选的流程。

代码规范不是一成不变的教条, 而是团队协作的工具。最重要的是团队达成共识, 并且一致地执行。工具可以帮助自动化格式检查, 但对命名、设计、文档的关注需要每个成员的自觉。当遵循规范成为习惯, 代码质量自然会提升, 团队协作也会更加顺畅。

1.7.3. 设计模式的起源

设计模式(Design pattern)是软件开发者在长期实践中提炼出的、可复用的代码设计经验,它们经过大量项目验证,被系统化分类,并形成了完整的知识体系。设计模式是软件工程的基石脉络,如同大厦的结构一样。

设计模式使代码编制真正工程化,是软件工程的基石。在项目中合理地运用设计模式可以完美地解决很多问题,每种模式在现实中都有相应的原理来与之对应,且都描述了一个在我们周围不断重复发生的问题,以及该问题的核心解决方案,这也是设计模式能被广泛应用的原因。

1994年, Erich Gamma、Richard Helm、Ralph Johnson 与 John Vlissides 四人共同出版了划时代的著作 **Design Patterns: Elements of Reusable Object-Oriented Software** (中文译名:《设计模式——可复用面向对象软件的基础》)。该书首次系统化地阐述了软件开发中“设计模式”这一核心概念,为面向对象软件设计奠定了理论基石。

在这本书中, GoF 将设计模式系统地归纳为三大类,共提出了“**23 种经典设计模式**”,这些模式奠定了面向对象软件设计的基础框架。

然而需要指出的是,书中讨论的许多模式主要基于 **Java** 等当时主流的面向对象语言。在 C++ ——尤其是 **现代 C++** (Modern C++) 的发展背景下,由于语言本身拥有更强大的表达能力(如 RAII、模板、智能指针、`constexpr` 等特性),其中部分设计模式已不再必要,甚至被语言机制自然替代。

尽管如此,现代软件工程仍将设计模式分为三大类:

1. 创建型模式 (Creational Patterns)

关注对象的创建方式,以提升灵活性与可扩展性。

2. 结构型模式 (Structural Patterns)

关注类与对象的组织结构,使系统模块更清晰、更易复用。

3. 行为型模式 (Behavioral Patterns)

关注对象间的协作方式与职责划分,提升系统行为的稳定性与扩展性。

这些模式依然构成理解现代架构思想的重要基础,只是在 C++ 语境下,其实现方式随着语言进化而更为简洁与自然。

1.7.4. 设计模式的基本原则

设计模式的**核心目标**是实现**高内聚、低耦合**的软件架构。所谓“高内聚”,是指一个模块内部的各个元素紧密相关、职责明确;而“低耦合”则强调模块之间的依赖关系应尽可能松散,以便于系统的维护、扩展与升级。

为了达成这一目标,软件工程界总结出了以下**七大设计原则**,它们共同构成了设计模式的理论基石:

1. 开闭原则 (Open-Closed Principle, OCP)

对扩展开放,对修改关闭。

软件实体(类、模块、函数等)应当在不修改现有代码的前提下进行功能扩展。这一原则是实现系统良好扩展性与可维护性的关键。想要达到这样的效果,需要充分利用接口、抽象类等机制,将易变部分与稳定部分分离。

开闭原则是设计模式的总纲,其他原则都是实现开闭原则的具体手段。

2. 单一职责原则 (Single Responsibility Principle, SRP)

一个类应该只有一个引起它变化的原因。

类的职责应当单一明确，对外只提供一种功能。当一个类承担的职责过多时，这些职责之间可能会相互耦合，一个职责的变化可能会削弱或抑制这个类完成其他职责的能力。遵循单一职责原则可以提高类的可读性、可维护性，降低变更引起的风险。

3. 里氏替换原则 (Liskov Substitution Principle, LSP)

任何基类可以出现的地方，子类一定可以出现。

里氏替换原则是面向对象设计的基本原则之一，是继承复用的基石。它要求子类在继承父类时，除了添加新的方法完成新增功能外，尽量不要重写父类的方法。只有当派生类可以替换掉基类，且软件单位的功能不受影响时，基类才能真正被复用。

里氏替换原则是对开闭原则的补充，它通过规范继承关系来实现抽象化，从而支撑开闭原则的实现。这一原则在 C++ 中通过虚函数机制在语言层面得到了实现。

4. 依赖倒置原则 (Dependence Inversion Principle, DIP)

依赖于抽象，不要依赖于具体实现。

高层模块不应该依赖低层模块，两者都应该依赖其抽象（接口或抽象类）。抽象不应该依赖细节，细节应该依赖抽象。这一原则是开闭原则的基础，强调“面向接口编程”而非“面向实现编程”，从而降低模块间的耦合度。

5. 接口隔离原则 (Interface Segregation Principle, ISP)

使用多个专门的接口，而不使用单一的总接口。

客户端不应该被迫依赖它不使用的接口方法。一个接口应该只提供一种对外功能，不应该把所有操作都封装到一个接口中。接口隔离原则要求将臃肿的接口拆分为多个细粒度的接口，降低类之间的耦合度，提高系统的灵活性。

6. 合成复用原则 (Composite Reuse Principle, CRP)

优先使用对象组合，而不是继承来达到复用的目的。

在复用代码时，应优先选择对象组合（一个对象包含另一个对象）的方式，而不是使用类继承。继承会导致父类的任何变化都可能影响到子类的行为，增加了系统的脆弱性；而对象组合则降低了这种依赖关系，使系统更加灵活。这一原则是对里氏替换原则的补充，提示我们思考功能组合实现的正确方式。

7. 迪米特法则 (Law of Demeter, LoD)

一个对象应当对其他对象有尽可能少的了解。

也称为**最少知识原则**，要求一个实体应当尽量少地与其他实体发生相互作用，使得系统功能模块相对独立，从而降低各个对象之间的耦合，提高系统的可维护性。

例如在程序设计中，各个模块之间相互调用时，通常会提供一个统一的接口来实现功能。这样其他模块不需要了解模块内部的实现细节（黑盒原理），当一个模块内部的实现发生改变时，不会影响其他模块的使用。

1.7.5. 常用设计模式详解

1.7.6. 单元测试

“它在我的机器上能跑啊！”——这可能是程序员最常说的一句话，也是最让人无奈的一句话。代码能运行并不意味着代码是正确的。你今天写的代码明天还能正常工作吗？修改了一个函数，会不会破坏其他地方的功能？重构了一个模块，原来的行为还保持不变吗？如果没有测试，你只能祈祷和手动验证。而有了测试，你可以在几秒钟内得到确切的答案。单元测试是程序员给自己买的保险——平时花一点时间编写，关键时刻能省下无数调试的痛苦。

1.7.6.1. 为什么要写测试

测试最直接的价值是验证代码的正确性。当你写完一个函数，如何确认它真的按预期工作？你可能会写一个 `main` 函数，手动调用几次，看看输出对不对。这种方法能解决眼前的问题，但一旦代码发生变化，你又得重新手动验证。如果有十个函数需要验证，每次修改都要手动测试一遍，这个负担会越来越重，最终你会放弃验证，只能“希望”代码是对的。

自动化测试解决了这个问题。测试代码被保存下来，可以反复执行。每次修改代码后，只需运行一下测试，几秒钟就能知道是否破坏了什么。这种即时反馈让你可以放心地修改代码，而不是战战兢兢地担心引入 bug。在 RoboMaster 开发中，算法经常需要调优参数、改进逻辑，如果没有测试保障，每次修改都是在冒险。

测试的另一个重要价值是支撑重构。随着项目演进，最初的设计可能不再适用，代码需要重构以保持健康。但重构是危险的——你要在不改变外部行为的前提下改变内部结构，如何确保“外部行为不变”？答案就是测试。如果有充分的测试覆盖，重构后运行测试全部通过，你就可以确信重构没有引入问题。没有测试的代码几乎不可能安全地重构，最终只能任其腐烂，直到不得不重写。

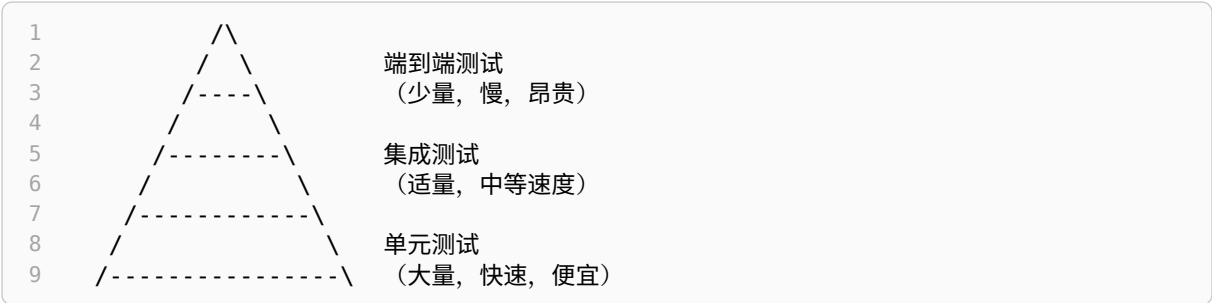
测试还能作为可执行的文档。好的测试用例清晰地展示了代码的使用方式和预期行为，比文字描述更加精确。当你不确定一个函数该怎么用时，看看它的测试用例往往能得到答案。测试用例也不会过时——如果测试与代码行为不符，测试就会失败，迫使你更新。

有些团队会说“我们没时间写测试”，但这是一个短视的观点。不写测试确实能节省一些时间，但这些时间迟早会以调试 bug 的形式还回来，而且往往是加倍偿还。一个隐蔽的 bug 可能在比赛现场暴露，导致整场比赛的失利；一个没有测试保护的修改可能引发连锁反应，花费数小时才能定位。测试不是额外的负担，而是提高开发效率的投资。

1.7.6.2. 测试金字塔

软件测试有多种类型，它们在测试范围、执行速度和维护成本上有所不同。测试金字塔是一个经典的模型，它描述了不同类型测试应有的比例。

单元测试位于金字塔的底层，数量最多。单元测试针对代码的最小可测试单元——通常是一个函数或一个类——验证其行为是否正确。单元测试应该快速(毫秒级)、独立(不依赖外部资源)、可重复（每次运行结果相同）。一个典型的项目可能有成百上千个单元测试，它们在每次代码提交时运行，提供快速反馈。



集成测试位于金字塔的中层。它测试多个模块组合在一起是否能正确协作。例如，测试视觉检测模块与跟踪模块的接口是否匹配，测试通信模块能否正确解析实际的数据包。集成测试比单元测试慢，因为涉及更多的代码和可能的外部资源，但它能发现单元测试无法发现的集成问题。

端到端测试（也叫系统测试）位于金字塔的顶层，数量最少。它测试整个系统作为一个整体是否能完成用户场景。对于 RoboMaster 机器人，端到端测试可能是在仿真环境中运行完整的自瞄流程，从图像输入到云台控制输出。端到端测试最接近真实使用场景，但也最慢、最难维护。

金字塔形状表示了推荐的测试比例：大量的单元测试、适量的集成测试、少量的端到端测试。这样的比例既能保证测试覆盖，又能保持测试的执行效率。如果你的测试金字塔是倒过来的（大量端到端测试、少量单元测试），测试运行会很慢，问题定位会很困难，测试也会很脆弱。

本节主要关注单元测试。单元测试是最基础也是最重要的测试类型，它是测试金字塔的基石。

1.7.6.3. Google Test 框架入门

Google Test (简称 gtest) 是 C++ 社区最流行的单元测试框架。它由 Google 开发并开源，提供了丰富的断言、测试组织和运行功能。许多知名项目都使用 gtest，包括 Chromium、LLVM、OpenCV 等。

安装 gtest 在 Ubuntu 上非常简单：

```
1 sudo apt install libgtest-dev
```

在 CMake 项目中使用 gtest，需要在 CMakeLists.txt 中添加相应配置：

```
1 cmake_minimum_required(VERSION 3.14)
2 project(my_project)
3
4 # 启用测试
5 enable_testing()
6
7 # 查找 GTest
8 find_package(GTest REQUIRED)
9
10 # 添加可执行文件
11 add_executable(my_tests
12     test/test_main.cpp
13     test/test_math_utils.cpp
14     test/test_detector.cpp
15 )
16
17 # 链接 GTest
18 target_link_libraries(my_tests
19     GTest::gtest
20     GTest::gtest_main
21     my_library # 被测试的库
22 )
23
24 # 注册测试
25 include(GoogleTest)
26 gtest_discover_tests(my_tests)
```

如果使用 `GTest::gtest_main`，gtest 会提供 `main` 函数，你不需要自己写。否则，需要在测试文件中添加：

```
1 #include <gtest/gtest.h>
2
3 int main(int argc, char* argv) {
4     testing::InitGoogleTest(&argc, argv);
5     return RUN_ALL_TESTS();
6 }
```

编写第一个测试非常简单。使用 `TEST` 宏定义一个测试用例，第一个参数是测试套件名，第二个参数是测试名：

```

1 #include <gtest/gtest.h>
2 #include "math_utils.h"
3
4 // 测试套件: MathUtils, 测试名: AddPositiveNumbers
5 TEST(MathUtils, AddPositiveNumbers) {
6     EXPECT_EQ(Add(1, 2), 3);
7     EXPECT_EQ(Add(10, 20), 30);
8 }
9
10 TEST(MathUtils, AddNegativeNumbers) {
11     EXPECT_EQ(Add(-1, -2), -3);
12     EXPECT_EQ(Add(-10, 20), 10);
13 }
14
15 TEST(MathUtils, AddZero) {
16     EXPECT_EQ(Add(0, 0), 0);
17     EXPECT_EQ(Add(5, 0), 5);
18     EXPECT_EQ(Add(0, 5), 5);
19 }

```

运行测试:

```

1 # 构建
2 mkdir build && cd build
3 cmake ..
4 make
5
6 # 运行所有测试
7 ./my_tests
8
9 # 运行特定测试套件
10 ./my_tests --gtest_filter=MathUtils.*
11
12 # 运行特定测试
13 ./my_tests --gtest_filter=MathUtils.AddPositiveNumbers

```

测试输出会显示每个测试的结果:

```

1 [=====] Running 3 tests from 1 test suite.
2 [-----] Global test environment set-up.
3 [-----] 3 tests from MathUtils
4 [ RUN      ] MathUtils.AddPositiveNumbers
5 [      OK  ] MathUtils.AddPositiveNumbers (0 ms)
6 [ RUN      ] MathUtils.AddNegativeNumbers
7 [      OK  ] MathUtils.AddNegativeNumbers (0 ms)
8 [ RUN      ] MathUtils.AddZero
9 [      OK  ] MathUtils.AddZero (0 ms)
10 [-----] 3 tests from MathUtils (0 ms total)
11
12 [-----] Global test environment tear-down
13 [=====] 3 tests from 1 test suite ran. (0 ms total)
14 [  PASSED  ] 3 tests.

```

1.7.6.4. 断言: EXPECT 与 ASSERT

gtest 提供了丰富的断言宏来验证条件。断言分为两类: `EXPECT_*` 和 `ASSERT_*`。两者的区别在于失败后的行为: `EXPECT_*` 失败后会继续执行后续的断言, 而 `ASSERT_*` 失败后会立即终止当前测试。

一般情况下优先使用 `EXPECT_*`, 这样一次运行可以发现多个问题。当后续断言依赖于前面的条件成立时 (比如指针非空), 使用 `ASSERT_*` 避免后续代码崩溃。

```

1 TEST(VectorTest, AccessElements) {
2     std::vector<int> v;
3     v.push_back(1);
4     v.push_back(2);
5
6     // 使用 ASSERT 检查前提条件
7     ASSERT_FALSE(v.empty()); // 如果失败, 后续代码不安全
8     ASSERT_EQ(v.size(), 2);
9
10    // 使用 EXPECT 检查具体值
11    EXPECT_EQ(v[0], 1);
12    EXPECT_EQ(v[1], 2);
13 }

```

常用的断言宏包括:

```

1 // 布尔断言
2 EXPECT_TRUE(condition);
3 EXPECT_FALSE(condition);
4
5 // 相等性断言
6 EXPECT_EQ(expected, actual); // expected == actual
7 EXPECT_NE(val1, val2);      // val1 != val2
8
9 // 比较断言
10 EXPECT_LT(val1, val2);      // val1 < val2
11 EXPECT_LE(val1, val2);      // val1 <= val2
12 EXPECT_GT(val1, val2);      // val1 > val2
13 EXPECT_GE(val1, val2);      // val1 >= val2
14
15 // 浮点数断言 (考虑精度误差)
16 EXPECT_FLOAT_EQ(expected, actual); // float 相等
17 EXPECT_DOUBLE_EQ(expected, actual); // double 相等
18 EXPECT_NEAR(val1, val2, tolerance); // 差值在容差内
19
20 // 字符串断言
21 EXPECT_STREQ(str1, str2);      // C 字符串相等
22 EXPECT_STRNE(str1, str2);      // C 字符串不等
23
24 // 异常断言
25 EXPECT_THROW(statement, exception_type); // 抛出指定异常
26 EXPECT_ANY_THROW(statement);           // 抛出任何异常
27 EXPECT_NO_THROW(statement);             // 不抛出异常
28
29 // 自定义失败消息
30 EXPECT_EQ(result, expected) << "计算结果不正确, 输入为: " << input;

```

对于浮点数比较, 永远不要使用 `EXPECT_EQ`, 因为浮点数计算存在精度误差。使用 `EXPECT_FLOAT_EQ`、`EXPECT_DOUBLE_EQ` 或 `EXPECT_NEAR`:

```

1 TEST(FloatTest, Precision) {
2     double result = 0.1 + 0.2;
3
4     // 不好: 可能因精度问题失败
5     // EXPECT_EQ(result, 0.3);
6
7     // 好: 考虑浮点精度
8     EXPECT_DOUBLE_EQ(result, 0.3);
9     EXPECT_NEAR(result, 0.3, 1e-10);
10 }

```


1.7.6.5. 测试夹具

当多个测试需要相同的初始化代码时, 可以使用测试夹具 (Test Fixture) 来避免重复。测试夹具是一个继承自 `testing::Test` 的类, 在 `SetUp()` 方法中进行初始化, 在 `TearDown()` 方法中进行清理。

```
1  #include <gtest/gtest.h>
2  #include "detector.h"
3
4  class DetectorTest : public testing::Test {
5  protected:
6      void SetUp() override {
7          // 每个测试前执行
8          config_.model_path = "test_model.onnx";
9          config_.confidence_threshold = 0.5;
10         detector_ = std::make_unique<ArmorDetector>(config_);
11         detector_->Init();
12
13         // 加载测试图像
14         test_image_ = cv::imread("test_data/armor_image.jpg");
15     }
16
17     void TearDown() override {
18         // 每个测试后执行 (可选)
19         detector_.reset();
20     }
21
22     // 测试中可以使用的成员
23     DetectorConfig config_;
24     std::unique_ptr<ArmorDetector> detector_;
25     cv::Mat test_image_;
26 };
27
28 // 使用 TEST_F 而不是 TEST
29 TEST_F(DetectorTest, DetectsArmorInTestImage) {
30     auto results = detector_->Detect(test_image_);
31     EXPECT_FALSE(results.empty());
32 }
33
34 TEST_F(DetectorTest, ReturnsEmptyForBlankImage) {
35     cv::Mat blank(480, 640, CV_8UC3, cv::Scalar(0, 0, 0));
36     auto results = detector_->Detect(blank);
37     EXPECT_TRUE(results.empty());
38 }
39
40 TEST_F(DetectorTest, ConfidenceAboveThreshold) {
41     auto results = detector_->Detect(test_image_);
42     for (const auto& armor : results) {
43         EXPECT_GE(armor.confidence, config_.confidence_threshold);
44     }
45 }
```

测试夹具的执行顺序是: 构造函数 → `SetUp()` → 测试体 → `TearDown()` → 析构函数。每个测试都会创建一个新的夹具实例, 所以测试之间是隔离的。

如果多个测试套件需要共享更昂贵的初始化 (比如加载大型模型), 可以使用 `SetUpTestSuite()` 和 `TearDownTestSuite()` 静态方法, 它们只在整个测试套件前后各执行一次:

```
1  class ExpensiveTest : public testing::Test {
2  protected:
```

```

3     static void SetUpTestSuite() {
4         // 整个测试套件只执行一次
5         shared_model_ = LoadLargeModel();
6     }
7
8     static void TearDownTestSuite() {
9         shared_model_.reset();
10    }
11
12    static std::shared_ptr<Model> shared_model_;
13 };
14
15 std::shared_ptr<Model> ExpensiveTest::shared_model_;

```

1.7.6.6. 参数化测试

当你需要用不同的输入测试同一个逻辑时，参数化测试可以避免编写大量重复的测试代码。

```

1  #include <gtest/gtest.h>
2
3  // 定义参数类型
4  struct AngleTestParam {
5      double input_degrees;
6      double expected_radians;
7  };
8
9  // 参数化测试夹具
10 class AngleConversionTest : public testing::TestWithParam<AngleTestParam>
11 {};
12
13 // 参数化测试
14 TEST_P(AngleConversionTest, DegreesToRadians) {
15     AngleTestParam param = GetParam();
16     double result = DegreesToRadians(param.input_degrees);
17     EXPECT_NEAR(result, param.expected_radians, 1e-6);
18 }
19
20 // 提供测试参数
21 INSTANTIATE_TEST_SUITE_P(
22     CommonAngles,
23     AngleConversionTest,
24     testing::Values(
25         AngleTestParam{0, 0},
26         AngleTestParam{90, M_PI / 2},
27         AngleTestParam{180, M_PI},
28         AngleTestParam{360, 2 * M_PI},
29         AngleTestParam{-90, -M_PI / 2}
30     )
31 );

```

对于简单类型，可以使用更简洁的写法：

```

1  class PrimeTest : public testing::TestWithParam<int> {};
2
3  TEST_P(PrimeTest, IsPrime) {
4      int n = GetParam();
5      EXPECT_TRUE(IsPrime(n));
6  }
7
8  INSTANTIATE_TEST_SUITE_P(
9      Primes,

```

```

10     PrimeTest,
11     testing::Values(2, 3, 5, 7, 11, 13, 17, 19)
12 );

```

`testing::Values` 之外还有其他参数生成器:

```

1 // 范围
2 testing::Range(start, end, step);
3
4 // 布尔值
5 testing::Bool(); // true, false
6
7 // 组合
8 testing::Combine(testing::Values(1, 2), testing::Values("a", "b"));
9 // 生成 (1, "a"), (1, "b"), (2, "a"), (2, "b")

```

1.7.6.7. 测试驱动开发

测试驱动开发 (Test-Driven Development, TDD) 是一种先写测试、后写实现的开发方法。它遵循“红-绿-重构”的循环:

1. **红**: 先写一个会失败的测试, 明确你要实现什么功能
2. **绿**: 写最少的代码让测试通过
3. **重构**: 在测试保护下改进代码结构

```

1 // 第一步: 写一个失败的测试
2 TEST(Calculator, Add) {
3     Calculator calc;
4     EXPECT_EQ(calc.Add(2, 3), 5);
5 }
6
7 // 此时编译失败, 因为 Calculator 类不存在
8
9 // 第二步: 写最少的代码让测试通过
10 class Calculator {
11 public:
12     int Add(int a, int b) {
13         return a + b; // 最简单的实现
14     }
15 };
16
17 // 测试通过
18
19 // 第三步: 重构 (如果需要)
20 // 当前实现已经足够简单, 无需重构
21
22 // 继续循环: 添加更多测试
23 TEST(Calculator, Subtract) {
24     Calculator calc;
25     EXPECT_EQ(calc.Subtract(5, 3), 2);
26 }
27
28 // 然后实现 Subtract...

```

TDD 的好处是:

首先, 它确保代码总是可测试的。因为测试先于实现编写, 你不会写出难以测试的代码。

其次, 它迫使你在编码前思考接口设计。写测试时你就在思考: 这个函数应该接收什么参数? 返回什么结果? 这种思考往往能产生更好的设计。

第三，它提供了即时反馈。每次实现一小块功能，马上就能验证是否正确，而不是写完一大堆代码再调试。

TDD 并不适合所有场景。对于探索性的代码、需求不明确的原型、或者 GUI 相关的代码，TDD 可能不太合适。但对于逻辑明确的算法、工具函数、数据处理模块，TDD 是非常有效的方法。

1.7.6.8. 什么样的代码容易测试

有些代码很容易测试，有些代码测试起来非常痛苦。理解什么样的代码容易测试，可以指导你写出更好的代码设计。

纯函数最容易测试。纯函数的输出只依赖于输入，没有副作用，不依赖外部状态。给定相同的输入，总是产生相同的输出。这样的函数测试起来非常简单：准备输入，调用函数，检查输出。

```
1 // 容易测试：纯函数
2 double CalculateDistance(const Point& a, const Point& b) {
3     return std::sqrt(std::pow(a.x - b.x, 2) + std::pow(a.y - b.y, 2));
4 }
5
6 TEST(Geometry, CalculateDistance) {
7     Point a{0, 0};
8     Point b{3, 4};
9     EXPECT_DOUBLE_EQ(CalculateDistance(a, b), 5.0);
10 }
```

依赖注入让代码更容易测试。如果一个类在内部创建它的依赖，测试时就很难替换这些依赖。但如果依赖是从外部传入的，测试时就可以传入模拟的依赖。

```
1 // 难以测试：内部创建依赖
2 class Tracker {
3 public:
4     void Update() {
5         Camera camera; // 内部创建，无法替换
6         auto image = camera.Capture();
7         // ...
8     }
9 };
10
11 // 容易测试：依赖注入
12 class Tracker {
13 public:
14     explicit Tracker(std::shared_ptr<ICamera> camera)
15         : camera_(camera) {}
16
17     void Update() {
18         auto image = camera_->Capture(); // 可以注入模拟相机
19         // ...
20     }
21
22 private:
23     std::shared_ptr<ICamera> camera_;
24 };
25
26 // 测试时使用模拟相机
27 class MockCamera : public ICamera {
28 public:
29     cv::Mat Capture() override {
30         return test_image_; // 返回预设的测试图像
31     }
32     cv::Mat test_image_;
33 };
```

```

34
35 TEST(Tracker, UpdateWithMockCamera) {
36     auto mock_camera = std::make_shared<MockCamera>();
37     mock_camera->test_image_ = cv::imread("test_image.jpg");
38
39     Tracker tracker(mock_camera);
40     tracker.Update();
41     // 验证结果...
42 }

```

小函数比大函数容易测试。一个做很多事情的大函数，需要准备很多前置条件，验证很多结果，测试用例会很复杂。把大函数拆成多个小函数，每个小函数单独测试，会简单得多。

避免全局状态。依赖全局变量或单例的代码难以测试，因为测试之间会相互影响。如果必须使用，至少提供重置状态的方法。

1.7.6.9. Mock 与依赖注入

当被测代码依赖外部资源（数据库、网络、硬件）时，直接使用真实依赖会让测试变慢、变脆弱、变得不可重复。Mock（模拟对象）是解决这个问题的标准方法。

Google Mock（现在是 gtest 的一部分）提供了创建 Mock 对象的便捷方式：

```

1  #include <gmock/gmock.h>
2
3  // 定义接口
4  class ISerial {
5  public:
6      virtual ~ISerial() = default;
7      virtual bool Open(const std::string& port) = 0;
8      virtual bool Write(const std::vector<uint8_t>& data) = 0;
9      virtual std::vector<uint8_t> Read(size_t size) = 0;
10 };
11
12 // 创建 Mock 类
13 class MockSerial : public ISerial {
14 public:
15     MOCK_METHOD(bool, Open, (const std::string& port), (override));
16     MOCK_METHOD(bool, Write, (const std::vector<uint8_t>& data),
17 (override));
18     MOCK_METHOD(std::vector<uint8_t>, Read, (size_t size), (override));
19 };
20 // 使用 Mock 测试
21 TEST(Communication, SendCommand) {
22     MockSerial mock_serial;
23
24     // 设置期望: Open 被调用一次, 返回 true
25     EXPECT_CALL(mock_serial, Open("/dev/ttyUSB0"))
26         .Times(1)
27         .WillOnce(testing::Return(true));
28
29     // 设置期望: Write 被调用, 参数匹配, 返回 true
30     EXPECT_CALL(mock_serial, Write(testing::_))
31         .Times(1)
32         .WillOnce(testing::Return(true));
33
34     // 被测对象使用 Mock
35     Commander commander(&mock_serial);
36     bool result = commander.SendAimCommand(1.5, 2.0);
37

```

```

38     EXPECT_TRUE(result);
39     // Mock 会自动验证期望是否被满足
40 }

```

GMock 提供了丰富的匹配器和动作：

```

1  using namespace testing;
2
3  // 匹配器
4  EXPECT_CALL(mock, Method(Eq(5)));           // 参数等于 5
5  EXPECT_CALL(mock, Method(Gt(0)));           // 参数大于 0
6  EXPECT_CALL(mock, Method(_));               // 任意参数
7  EXPECT_CALL(mock, Method(StartsWith("prefix"))); // 字符串前缀
8
9  // 调用次数
10 EXPECT_CALL(mock, Method(_).Times(3));       // 恰好 3 次
11 EXPECT_CALL(mock, Method(_).Times(AtLeast(1))); // 至少 1 次
12 EXPECT_CALL(mock, Method(_).Times(AtMost(5))); // 至多 5 次
13
14 // 返回值
15 EXPECT_CALL(mock, Method(_).WillOnce(Return(42)));
16 EXPECT_CALL(mock, Method(_).WillRepeatedly(Return(0)));
17
18 // 按顺序
19 {
20     InSequence seq;
21     EXPECT_CALL(mock, First());
22     EXPECT_CALL(mock, Second());
23     EXPECT_CALL(mock, Third());
24 }

```

依赖注入是使 Mock 成为可能的设计模式。核心思想是：类不应该自己创建依赖，而应该从外部接收依赖。这有几种实现方式：

构造函数注入是最常用的方式：

```

1  class AutoAim {
2  public:
3      AutoAim(std::shared_ptr<IDetector> detector,
4              std::shared_ptr<ITracker> tracker,
5              std::shared_ptr<IPredictor> predictor)
6          : detector_(detector), tracker_(tracker), predictor_(predictor) {}
7
8      // ...
9
10 private:
11     std::shared_ptr<IDetector> detector_;
12     std::shared_ptr<ITracker> tracker_;
13     std::shared_ptr<IPredictor> predictor_;
14 };

```

Setter 注入适合可选依赖：

```

1  class Logger {
2  public:
3      void SetOutput(std::shared_ptr<IOutput> output) {
4          output_ = output;
5      }
6
7      // ...
8  };

```

1.7.6.10. 测试覆盖率

测试覆盖率衡量测试执行了多少代码。常见的覆盖率指标包括：

- **行覆盖率**：执行了多少行代码
- **分支覆盖率**：执行了多少分支（if/else 的每个分支）
- **函数覆盖率**：调用了多少函数

使用 gcov/lcov 可以生成覆盖率报告：

```
1 # 编译时启用覆盖率
2 g++ -fprofile-arcs -ftest-coverage -o my_tests my_tests.cpp my_code.cpp
3
4 # 运行测试
5 ./my_tests
6
7 # 生成报告
8 lcov --capture --directory . --output-file coverage.info
9 genhtml coverage.info --output-directory coverage_report
10
11 # 在浏览器中查看
12 xdg-open coverage_report/index.html
```

CMake 配置：

```
1 option(ENABLE_COVERAGE "Enable coverage reporting" OFF)
2
3 if(ENABLE_COVERAGE)
4     set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -fprofile-arcs -ftest-coverage")
5     set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} -fprofile-arcs -ftest-coverage")
6 endif()
```

覆盖率是有用的指标，但不要过度追求。100% 覆盖率并不意味着代码没有 bug——测试可能覆盖了代码但没有检查正确的条件。更重要的是测试的质量而非数量。一般来说，核心逻辑应该有较高的覆盖率（80% 以上），而 UI、胶水代码等可以低一些。

1.7.6.11. RoboMaster 中的测试实践

在 RoboMaster 项目中，测试应该聚焦于核心算法和关键逻辑。以下是一些具体的建议。

算法模块是最应该测试的部分。检测算法、跟踪算法、预测算法、弹道解算——这些模块逻辑复杂、容易出错，而且相对独立，很适合单元测试。

```
1 // 弹道解算测试
2 class BallisticSolverTest : public testing::Test {
3 protected:
4     void SetUp() override {
5         solver_ = std::make_unique<BallisticSolver>(28.0); // 28 m/s 弹速
6     }
7
8     std::unique_ptr<BallisticSolver> solver_;
9 };
10
11 TEST_F(BallisticSolverTest, HorizontalTarget) {
12     // 水平目标，不需要补偿
13     double pitch = solver_->Solve(5.0, 0.0); // 5米远，高度差为0
14     EXPECT_NEAR(pitch, 0.0, 0.01); // 接近水平
15 }
16
17 TEST_F(BallisticSolverTest, ElevatedTarget) {
```

```

18     // 高处目标, 需要向上补偿
19     double pitch = solver_>Solve(5.0, 2.0); // 5米远, 高2米
20     EXPECT_GT(pitch, 0.0); // 仰角应该为正
21 }
22
23 TEST_F(BallisticSolverTest, DistanceAffectsDrop) {
24     // 距离越远, 下坠补偿越大
25     double pitch_near = solver_>Solve(3.0, 0.0);
26     double pitch_far = solver_>Solve(8.0, 0.0);
27     EXPECT_GT(pitch_far, pitch_near);
28 }

```

通信协议测试确保数据包的打包和解析正确:

```

1  class ProtocolTest : public testing::Test {
2  protected:
3      Protocol protocol_;
4  };
5
6  TEST_F(ProtocolTest, PackAimCommand) {
7      AimCommand cmd{1.5f, -0.3f, true};
8      auto packet = protocol_.Pack(cmd);
9
10     EXPECT_EQ(packet.size(), Protocol::kAimCommandSize);
11     EXPECT_EQ(packet[0], Protocol::kHeader);
12     EXPECT_EQ(packet[1], Protocol::kAimCommandId);
13 }
14
15 TEST_F(ProtocolTest, UnpackAimCommand) {
16     std::vector<uint8_t> packet = {0xA5, 0x01, /* ... */};
17     auto cmd = protocol_.Unpack<AimCommand>(packet);
18
19     ASSERT_TRUE(cmd.has_value());
20     EXPECT_NEAR(cmd->yaw, 1.5f, 0.001f);
21     EXPECT_NEAR(cmd->pitch, -0.3f, 0.001f);
22     EXPECT_TRUE(cmd->fire);
23 }
24
25 TEST_F(ProtocolTest, RejectInvalidPacket) {
26     std::vector<uint8_t> invalid = {0x00, 0x01, 0x02}; // 错误的帧头
27     auto cmd = protocol_.Unpack<AimCommand>(invalid);
28     EXPECT_FALSE(cmd.has_value());
29 }

```

坐标变换测试验证几何计算的正确性:

```

1  TEST(CoordinateTransform, CameraToWorld) {
2      Transform tf;
3      tf.SetCameraToGimbal(/* ... */);
4      tf.SetGimbalToWorld(/* ... */);
5
6      Point3D point_camera{1.0, 0.0, 5.0};
7      Point3D point_world = tf.CameraToWorld(point_camera);
8
9      // 验证变换结果
10     EXPECT_NEAR(point_world.x, expected_x, 0.001);
11     EXPECT_NEAR(point_world.y, expected_y, 0.001);
12     EXPECT_NEAR(point_world.z, expected_z, 0.001);
13 }
14
15 TEST(CoordinateTransform, RoundTrip) {
16     // 变换后再逆变换应该得到原点

```



```

17     Transform tf;
18     Point3D original{1.0, 2.0, 3.0};
19     Point3D transformed = tf.CameraToWorld(original);
20     Point3D back = tf.WorldToCamera(transformed);
21
22     EXPECT_NEAR(back.x, original.x, 0.001);
23     EXPECT_NEAR(back.y, original.y, 0.001);
24     EXPECT_NEAR(back.z, original.z, 0.001);
25 }

```

仿真环境对于端到端测试非常有价值。Gazebo、Unity 等仿真器可以模拟机器人的物理行为和传感器输入，让你在不需要真实硬件的情况下测试完整的系统。仿真测试可以自动化运行，在每次代码提交时验证系统的整体行为。虽然仿真与现实存在差距（sim-to-real gap），但它能发现大量问题，节省宝贵的实机调试时间。

测试不是写完就结束的任务，而是需要持续维护的资产。当代码变化时，测试也要相应更新。当发现 bug 时，先写一个复现 bug 的测试，再修复 bug——这样可以防止 bug 复发。将测试作为持续集成的一部分，每次代码推送都自动运行，及早发现问题。

编写测试需要时间，但这是值得的投资。测试给你信心去修改和改进代码，让你能够快速迭代而不是小心翼翼。在 RoboMaster 紧张的赛季中，这种信心尤为重要。

1.7.7. 调试技巧

1.7.8. 性能分析与优化

1.7.9. 文档编写

代码是程序员与机器的对话，而文档是程序员与人的对话——可能是与队友、与未来的维护者，也可能是与几个月后已经忘记实现细节的自己。许多程序员不喜欢写文档，觉得这是编码之外的“杂事”，浪费时间。但当你凌晨三点调试一个陌生的模块，翻遍代码却找不到任何解释时；当新队员接手项目，花了一周时间还没搞清楚系统架构时；当你试图使用去年写的库，却想不起那些参数是什么含义时——你会深刻体会到文档的价值。好的文档是项目的第二生命，它让知识得以传承，让协作成为可能。

1.7.9.1. 文档的价值

文档最直接的价值是降低沟通成本。在一个团队中，如果每个问题都需要口头解释，那么回答问题的人会被频繁打断，提问的人也要等待对方有空。而一份好的文档可以回答大多数常见问题，让团队成员能够自助式地获取信息。当有人问“这个函数怎么用”或“这个模块是做什么的”时，你可以说“看文档”，而不是每次都从头解释一遍。

文档的另一个重要价值是保存知识。人的记忆是不可靠的，今天清清楚楚的设计决策，三个月后可能就想不起来了。更何况团队成员会毕业、会换项目，如果知识只存在于人的脑子里，人走了知识也就丢了。文档将知识外化，使其独立于任何个人而存在。一份记录了设计思路、架构决策、踩过的坑的文档，是团队最宝贵的资产之一。

文档还能帮助作者理清思路。写文档的过程迫使你用清晰的语言解释自己的设计，这个过程往往会暴露出之前没有意识到的问题。如果你发现某个部分很难解释清楚，很可能是因为设计本身就不够清晰。有经验的工程师常常在写代码之前先写设计文档，通过写作来思考和验证方案。

对于 RoboMaster 团队来说，文档还有一层特殊的意义：赛季传承。每年都有老队员毕业、新队员加入，如果没有文档，新人只能从零开始摸索，前人的经验无法积累。而有了好的文档，新队员可以快速了解系统全貌，在前人的基础上继续前进，而不是每年都在重复发明轮子。

1.7.9.2. 代码即文档

在讨论如何写文档之前，我们首先要认识到：最好的文档是代码本身。如果代码写得足够清晰，很多时候不需要额外的解释。这就是“代码即文档”（Code as Documentation）的理念。

清晰的命名是代码自解释的基础。一个名为 `CalculateProjectileDropCompensation` 的函数，不需要注释也能让人明白它是计算弹道下坠补偿的。而一个名为 `Calc` 的函数，即使有注释也需要读者花费额外的精力去理解。变量名也是如此：`remainingAmmo` 比 `n` 有意义得多。好的命名就像好的路标，让读者不需要地图也能找到方向。

合理的代码结构也是一种文档。当一个函数只做一件事、一个类只有一个职责时，代码的意图就很明显。当相关的功能被组织在一起、模块之间有清晰的边界时，系统的架构就呼之欲出。相反，如果一个函数做了十件不同的事，即使每一行都有注释，也很难理解它的整体逻辑。

类型系统也是文档的一部分。在 C++ 中，使用强类型而不是原始类型可以传达更多信息。例如，`Angle` 类型比 `double` 更能说明参数的含义；`std::optional<Target>` 比返回空指针更能表达“可能没有结果”的语义；`enum class RobotState` 比一堆整数常量更能说明状态的含义和取值范围。

```
1 // 不好：类型没有传达信息
2 double Calculate(double a, double b, int mode);
3
4 // 好：类型本身就是文档
5 Velocity CalculateVelocity(Distance distance, Duration time);
6 std::optional<Target> DetectTarget(const Image& image);
```

常量和配置也应该自解释。不要在代码中使用魔法数字，而是定义有意义的常量。当读者看到 `kMaxDetectionDistance` 时，他立刻知道这是最大检测距离；而看到 `5.0` 时，他只能猜测这个数字的含义。

```
1 // 不好：魔法数字
2 if (distance < 5.0 && confidence > 0.8) {
3     Fire();
4 }
5
6 // 好：有意义的常量
7 constexpr double kMaxFiringDistance = 5.0; // meters
8 constexpr double kMinConfidenceThreshold = 0.8;
9
10 if (distance < kMaxFiringDistance && confidence > kMinConfidenceThreshold) {
11     Fire();
12 }
```

然而，代码即文档有其局限性。代码能够说明“是什么”（what）和“怎么做”（how），但很难表达“为什么”（why）。为什么选择这个算法而不是那个？为什么这个参数是 5.0 而不是 3.0？为什么要加这个看起来多余的检查？这些问题需要注释和文档来回答。

1.7.9.3. 注释的艺术

注释是代码中嵌入的文档, 用于解释代码本身无法表达的信息。好的注释能够显著提高代码的可读性, 而糟糕的注释则可能误导读者、浪费时间。掌握何时写注释、写什么注释, 是一门需要练习的艺术。

首先要明确的是, 注释不是用来解释代码“做了什么”的。如果代码需要注释来解释它在做什么, 这通常意味着代码本身不够清晰, 应该重构而不是添加注释。那些像 `i++; // 将 i 加 1` 这样的注释毫无价值, 只会增加阅读负担。

注释真正的价值在于解释“为什么”。为什么选择这种实现方式? 有什么约束或权衡? 这里有什么需要注意的陷阱? 这些是代码本身无法表达的信息, 是注释的用武之地。

```
1 // 不好: 解释"是什么"
2 // 遍历所有目标
3 for (const auto& target : targets) {
4     // 如果目标在范围内
5     if (target.distance < max_distance) {
6         // 添加到结果中
7         results.push_back(target);
8     }
9 }
10
11 // 好: 解释"为什么"
12 // 优先处理近距离目标, 因为它们对比赛结果影响更大。
13 // 远距离目标的检测置信度较低, 容易产生误判。
14 for (const auto& target : targets) {
15     if (target.distance < max_distance) {
16         results.push_back(target);
17     }
18 }
```

注释还应该解释非显而易见的代码。有时候, 为了性能、兼容性或规避某个 bug, 代码不得不写得比较晦涩。这时候, 注释可以解释这样写的原因, 避免后人“好心”地重构掉这些看起来奇怪的代码。

```
1 // 使用位运算代替除法, 在 ARM 平台上快约 10 倍。
2 // 仅当 divisor 是 2 的幂时有效。
3 int FastDivide(int value, int divisor) {
4     int shift = __builtin_ctz(divisor); // 计算尾随零的个数
5     return value >> shift;
6 }
7
8 // OpenCV 4.2 之前的版本在这里有内存泄漏, 需要手动释放。
9 // 参见: https://github.com/opencv/opencv/issues/12345
10 cv::Mat temp;
11 // ... 使用 temp ...
12 temp.release(); // 必要的手动释放
```

注释应该解释假设和约束。函数对输入有什么要求? 调用时需要满足什么前置条件? 有什么副作用? 这些信息可以帮助调用者正确使用代码。

```
1 // 计算两点之间的角度。
2 // 假设: 两点不重合 (distance > 0)。
3 // 返回值: 弧度, 范围 [-π, π]。
4 // 注意: 此函数不是线程安全的, 因为它使用了全局缓存。
5 double CalculateAngle(const Point& from, const Point& to);
```

注释要保持与代码同步。过时的注释比没有注释更糟糕，因为它会误导读者。当修改代码时，一定要检查相关注释是否需要更新。如果一段注释描述的行为与代码不符，读者会困惑：是代码错了还是注释错了？为了降低注释过时的风险，注释应该描述意图和原因，而不是复述代码的实现细节——意图通常比实现稳定。

TODO 和 FIXME 注释用于标记待办事项和已知问题。它们是对未来的承诺，提醒自己或他人这里还有工作要做。好的 TODO 注释应该包含具体的任务描述、负责人和预期时间。定期清理 TODO 是良好的习惯——如果一个 TODO 存在超过一个月还没处理，要么立即处理，要么承认它不重要并删除。

```
1 // TODO(zhangsan): 实现多目标跟踪。当前只支持单目标。
2 //                预计下周完成。
3
4 // FIXME: 在光线变化剧烈时会产生误检。
5 //                需要添加时域滤波来改善稳定性。
6
7 // HACK: 临时解决方案，等待上游库修复后移除。
8 //                跟踪 issue: https://github.com/xxx/xxx/issues/123
```

1.7.9.4. README：项目的门面

README 是项目的门面，通常是用户接触项目的第一份文档。一份好的 README 能让人在几分钟内了解项目是什么、能做什么、怎么开始使用。它应该简洁但完整，回答读者最关心的问题。

一份标准的 README 通常包含以下部分：

项目标题和简介放在最开头，用一两句话说明项目是什么、解决什么问题。如果有项目 logo 或徽章（构建状态、版本号等），可以放在这里。

```
1 # RoboMaster Vision System
2
3 基于深度学习的 RoboMaster 自瞄视觉系统，支持装甲板检测、跟踪和预测。
4
5 ![Build Status](https://img.shields.io/badge/build-passing-green)
6 ![Version](https://img.shields.io/badge/version-2.0.0-blue)
```

功能特性列出项目的主要功能，让读者快速了解项目能做什么。可以用列表形式简洁地呈现。

```
1 ## 功能特性
2
3 - 🎯 实时装甲板检测 (60+ FPS @ 1080p)
4 - 🔄 多目标跟踪与 ID 关联
5 - 📈 基于卡尔曼滤波的运动预测
6 - 🎮 支持 ROS 2 Humble 集成
7 - ⚡ CUDA 加速推理
```

快速开始是最重要的部分之一，它应该让读者能够在最短时间内运行项目。包括环境要求、安装步骤和基本使用示例。命令应该可以直接复制执行，不要让读者猜测。

```
1 ## 快速开始
2
3 ### 环境要求
4
5 - Ubuntu 22.04
```

```

6 - ROS 2 Humble
7 - CUDA 11.8+
8 - OpenCV 4.5+
9
10 ### 安装
11
12 ```bash
13 # 克隆仓库
14 git clone https://github.com/your-team/rm_vision.git
15 cd rm_vision
16
17 # 安装依赖
18 ./scripts/install_dependencies.sh
19
20 # 编译
21 colcon build --symlink-install
22 ```
23
24 ### 运行
25
26 ```bash
27 # 启动检测节点
28 ros2 launch rm_vision detector.launch.py
29
30 # 使用录制的数据测试
31 ros2 launch rm_vision detector.launch.py use_bag:=true bag_path:=/path/to/
  bag
32 ```

```

配置说明解释主要的配置选项, 让用户知道如何根据自己的需求调整系统。可以提供配置文件的示例和各选项的含义。

```

1 ## 配置
2
3 配置文件位于 `config/detector_params.yaml`:
4
5 ```yaml
6 detector:
7   model_path: "models/armor_yolov8.onnx" # 模型路径
8   confidence_threshold: 0.7               # 置信度阈值
9   nms_threshold: 0.4                     # NMS 阈值
10   target_color: "red"                   # 目标颜色
11 ```

```

项目结构帮助读者理解代码的组织方式, 特别是对于想要深入了解或贡献代码的人。

```

1 ## 项目结构
2
3 ```
4 rm_vision/
5 |— rm_vision/      # 主功能包
6 |   |— detector/   # 装甲板检测
7 |   |— tracker/    # 目标跟踪
8 |   |— predictor/  # 运动预测
9 |— rm_interfaces/  # 消息和服务定义
10 |— rm_bringup/    # 启动文件
11 |— config/        # 配置文件
12 |— models/        # 预训练模型
13 |— docs/          # 详细文档
14 ```

```

其他可选部分包括：详细文档的链接、贡献指南、许可证信息、致谢、联系方式等。对于开源项目，这些信息尤其重要。

```
1  ## 文档
2
3  详细文档请参阅 [Wiki](https://github.com/your-team/rm_vision/wiki)。
4
5  ## 贡献
6
7  欢迎贡献！请阅读 [贡献指南](CONTRIBUTING.md) 了解如何参与。
8
9  ## 许可证
10
11  本项目采用 MIT 许可证。详见 [LICENSE](LICENSE)。
12
13  ## 致谢
14
15  - 感谢 [YOLO](https://github.com/ultralytics/yolov5) 提供的目标检测框架
16  - 感谢历届队员的贡献
17
18  ## 联系我们
19
20  - 邮箱: robomaster@example.com
21  - QQ 群: 123456789
```

1.7.9.5. API 文档与 Doxygen

对于库或框架，API 文档是不可或缺的。它详细描述每个类、函数、参数的用法，是开发者使用库时的参考手册。手写 API 文档工作量大且容易过时，因此通常使用工具从代码中的注释自动生成。Doxygen 是 C++ 社区最流行的文档生成工具。

Doxygen 通过解析代码中特定格式的注释来生成文档。最常用的注释风格是 Javadoc 风格（以 `/*` 开头）和 Qt 风格（以 `/*!` 开头）。以下是一些常用的 Doxygen 命令：

```
1  /*
2   * @file armor_detector.h
3   * @brief 装甲板检测器的头文件
4   * @author Zhang San
5   * @date 2024-01-15
6   */
7
8  /*
9   * @brief 装甲板检测器类
10  *
11  * ArmorDetector 使用深度学习模型检测图像中的装甲板。
12  * 它支持红色和蓝色装甲板的检测，并能处理不同光照条件。
13  *
14  * @note 此类不是线程安全的。每个线程应创建独立的实例。
15  *
16  * 使用示例：
17  * @code
18  * ArmorDetector detector(config);
19  * detector.Init();
20  * auto armors = detector.Detect(image);
21  * for (const auto& armor : armors) {
22  *     std::cout << "检测到装甲板，置信度: " << armor.confidence << std::endl;
23  * }
24  * @endcode
25  *
26  * @see Tracker 用于目标跟踪
27  * @see Predictor 用于运动预测
```



```

28  */
29  class ArmorDetector {
30  public:
31      /*
32       * @brief 构造函数
33       * @param config 检测器配置参数
34       * @throws std::invalid_argument 如果配置无效
35       */
36      explicit ArmorDetector(const DetectorConfig& config);
37
38      /*
39       * @brief 初始化检测器
40       *
41       * 加载模型并准备推理引擎。此方法必须在 Detect() 之前调用。
42       *
43       * @return true 如果初始化成功
44       * @return false 如果初始化失败（如模型文件不存在）
45       */
46      bool Init();
47
48      /*
49       * @brief 检测图像中的装甲板
50       *
51       * @param image 输入图像，必须是 BGR 格式
52       * @param timestamp 图像时间戳，用于时间同步
53       * @return 检测到的装甲板列表，按置信度降序排列
54       *
55       * @pre Init() 已成功调用
56       * @pre image 不为空
57       *
58       * @warning 此方法会修改内部状态，不要在多线程中共享实例
59       */
60      std::vector<Armor> Detect(const cv::Mat& image, double timestamp);
61
62      /*
63       * @brief 设置目标颜色
64       * @param color 目标颜色
65       * @see TargetColor
66       */
67      void SetTargetColor(TargetColor color);
68
69      /*
70       * @brief 获取当前检测统计信息
71       * @return 包含检测帧数、平均耗时等的统计信息
72       */
73      DetectorStats GetStats() const;
74
75  private:
76      DetectorConfig config_; ///< 检测器配置
77      bool initialized_;      ///< 是否已初始化
78      // ...
79  };
80
81  /*
82   * @brief 目标颜色枚举
83   */
84  enum class TargetColor {
85      kRed,    ///< 红色方
86      kBlue   ///< 蓝色方
87  };
88
89  /*
90   * @struct Armor

```

```

91  * @brief 装甲板检测结果
92  */
93  struct Armor {
94      cv::Point2f center;    ///< 装甲板中心点（像素坐标）
95      cv::Size2f size;      ///< 装甲板尺寸（像素）
96      double confidence;    ///< 检测置信度，范围 [0, 1]
97      int id;               ///< 装甲板编号（1-5 对应英雄到哨兵）
98      TargetColor color;    ///< 装甲板颜色
99  };

```

配置和运行 Doxygen 非常简单。首先安装 Doxygen:

```
1 sudo apt install doxygen graphviz
```

然后在项目根目录生成配置文件:

```
1 doxygen -g Doxyfile
```

编辑 Doxyfile 配置主要选项:

```

1  # 项目信息
2  PROJECT_NAME           = "RoboMaster Vision"
3  PROJECT_NUMBER         = 2.0.0
4  PROJECT_BRIEF          = "自瞄视觉系统"
5
6  # 输入设置
7  INPUT                  = include src
8  RECURSIVE              = YES
9  FILE_PATTERNS          = *.h *.hpp *.cpp
10
11 # 输出设置
12 OUTPUT_DIRECTORY       = docs/api
13 GENERATE_HTML           = YES
14 GENERATE_LATEX          = NO
15
16 # 提取设置
17 EXTRACT_ALL             = NO
18 EXTRACT_PRIVATE         = NO
19 EXTRACT_STATIC          = YES
20
21 # 图表
22 HAVE_DOT               = YES
23 CALL_GRAPH              = YES
24 CALLER_GRAPH            = YES

```

运行 Doxygen 生成文档:

```
1 doxygen Doxyfile
```

生成的 HTML 文档在 docs/api/html/ 目录下, 用浏览器打开 index.html 即可查看。

1.7.9.6. 项目文档类型

除了代码中的注释和 API 文档, 一个完整的项目还需要其他类型的文档。不同文档面向不同的读者、服务于不同的目的。

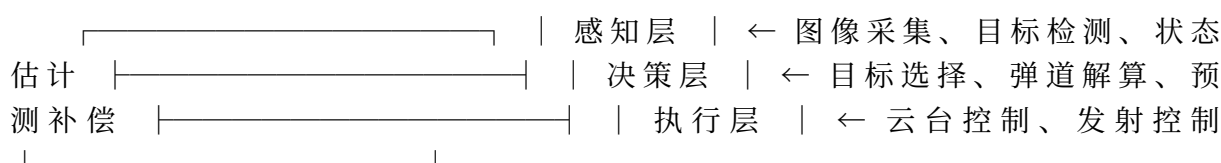
需求文档描述系统应该做什么。它定义功能需求(系统应该具备什么功能)和非功能需求(性能、可靠性、可维护性等要求)。需求文档是开发的起点, 所有后续工作都是为了满足需求。在

RoboMaster 中, 需求可能包括: 检测精度要达到多少、延迟不能超过多少毫秒、要支持哪些目标类型等。

```
1 # 自瞄系统需求文档
2
3 ## 功能需求
4
5 ### FR-001: 装甲板检测
6 - 系统应能检测红色和蓝色装甲板
7 - 支持识别装甲板编号 (1-5)
8 - 检测距离范围: 1-8 米
9
10 ### FR-002: 目标跟踪
11 - 系统应能跟踪多个目标
12 - 支持目标遮挡后重新识别
13 - 跟踪 ID 应保持稳定
14
15 ## 非功能需求
16
17 ### NFR-001: 性能
18 - 端到端延迟 < 20ms
19 - 检测帧率 >= 60 FPS
20
21 ### NFR-002: 可靠性
22 - 误检率 < 1%
23 - 连续运行 8 小时无崩溃
```

设计文档描述系统如何实现需求。它包括系统架构、模块划分、接口定义、数据流、关键算法等。设计文档是开发者的蓝图, 帮助团队成员理解系统的整体结构和设计决策。好的设计文档不仅说明“怎么做”, 还要解释“为什么这么做”。

```
1 # 自瞄系统设计文档
2
3 ## 系统架构
4
5 系统采用三层架构: 感知层、决策层、执行层。
6
```



```
1
2 ## 模块设计
3
4 ### 检测模块
5
6 采用 YOLOv8 进行装甲板检测, 原因:
7 1. 速度快, 满足实时性要求
8 2. 准确率高, 尤其对小目标
9 3. 社区支持好, 便于优化
10
11 ### 跟踪模块
12
13 采用 EKF 进行状态估计, 状态向量:
14 - 位置 (x, y, z)
15 - 速度 (vx, vy, vz)
16 - 装甲板朝向  $\theta$ 
17
```

```
18 ## 接口定义
19
20 ...
```

用户手册面向最终用户，说明如何安装、配置和使用系统。它应该假设读者不了解内部实现，用清晰的语言和步骤指导用户完成任务。对于 RoboMaster 项目，用户可能是操作手或调试人员。

变更日志（CHANGELOG）记录项目的版本历史和每个版本的变化。它帮助用户了解版本之间的差异，决定是否升级，以及升级时需要注意什么。一个常见的格式是 Keep a Changelog：

```
1 # 变更日志
2
3 本项目遵循 [语义化版本](https://semver.org/)。
4
5 ## [2.1.0] - 2024-03-15
6
7 ### 新增
8 - 支持能量机关检测
9 - 添加录像回放功能
10
11 ### 变更
12 - 升级 YOLOv8 模型，检测速度提升 20%
13 - 调整默认参数，适配新赛季规则
14
15 ### 修复
16 - 修复在强光下的误检问题 (#42)
17 - 修复内存泄漏 (#45)
18
19 ### 废弃
20 - `DetectArmor()` 已废弃，请使用 `Detect()`
21
22 ## [2.0.0] - 2024-01-10
23
24 ### 破坏性变更
25 - 重构检测接口，不兼容 1.x 版本
26 - 配置文件格式改为 YAML
27
28 ...
```

1.7.9.7. Markdown 写作技巧

Markdown 是最流行的文档格式之一，几乎所有的代码托管平台都支持它。掌握 Markdown 的写作技巧可以让你的文档更清晰、更易读。

结构化是好文档的基础。使用标题建立层次结构，让读者能够快速把握文档的脉络。但不要过度嵌套——三级标题通常就足够了，更深的层次可能意味着文档需要拆分。

```
1 # 一级标题（文档标题）
2 ## 二级标题（主要章节）
3 ### 三级标题（子章节）
```

列表用于并列的内容。有序列表用于有顺序的步骤，无序列表用于没有顺序的项目。列表项应该结构一致——要么都是完整的句子，要么都是短语。

```
1 ## 安装步骤
2
3 1. 克隆仓库
4 2. 安装依赖
```

```

5 3. 编译项目
6 4. 运行测试
7
8 ## 支持的功能
9
10 - 装甲板检测
11 - 能量机关检测
12 - 目标跟踪

```

代码块是技术文档的重要组成部分。始终指定语言以获得语法高亮，使用行内代码标记命令、函数名、文件名等。

```

1 运行 `ros2 launch` 启动节点：
2

```

```
bash ros2 launch rm_vision_detector.launch.py
```

```

1
2 函数 `Detect()` 返回检测结果。

```

表格适合展示结构化的对比信息。保持表格简洁，复杂的内容应该用其他方式呈现。

```

1 | 参数 | 类型 | 默认值 | 说明 |
2 | -----|-----|-----|-----|
3 | confidence_threshold | float | 0.7 | 置信度阈值 |
4 | nms_threshold | float | 0.4 | NMS 阈值 |
5 | target_color | string | "red" | 目标颜色 |

```

图片和图表可以大大提高文档的可理解性。系统架构图、流程图、效果截图都是有价值的。使用 Mermaid 可以在 Markdown 中直接编写图表：

```

1 ``mermaid
2 graph LR
3     A[相机] --> B[检测器]
4     B --> C[跟踪器]
5     C --> D[预测器]
6     D --> E[云台控制]
7 ...

```

链接让文档形成网络。使用相对链接引用项目内的其他文档，使用外部链接引用参考资料。但不要过度链接——每个链接都是一个潜在的离开当前上下文的机会。

```

1 详细配置说明请参阅 [配置文档](./config.md)。
2
3 算法原理基于这篇 [论文](https://arxiv.org/abs/xxx)。

```

1.7.9.8. 文档即代码

“文档即代码”（Docs as Code）是一种现代的文档管理理念：将文档视为代码一样对待，使用相同的工具和流程来管理。

首先，文档应该纳入版本控制。将文档与代码放在同一个仓库中，使用 Git 追踪变更历史。这样可以保证文档与代码的版本对应，也可以回溯历史版本。代码审查流程也应该包括文档——修改代码时，检查相关文档是否需要更新。

其次，文档应该使用纯文本格式。Markdown、reStructuredText、AsciiDoc 等格式易于编辑、易于比较差异、易于自动处理。避免使用 Word 等二进制格式，它们难以进行版本控制和协作编辑。

第三，可以利用 CI/CD 自动化文档流程。代码推送时自动构建文档、检查链接、部署到网站。这样可以保证文档始终是最新的，也可以在出现问题时及早发现。

```
1 # .github/workflows/docs.yml
2 name: Build Documentation
3
4 on:
5   push:
6     branches: [main]
7
8 jobs:
9   build:
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v3
13
14      - name: Build Doxygen
15        run: doxygen Doxyfile
16
17      - name: Deploy to GitHub Pages
18        uses: peaceiris/actions-gh-pages@v3
19        with:
20          github_token: ${ secrets.GITHUB_TOKEN }
21          publish_dir: ./docs/api/html
```

最后，鼓励团队成员贡献文档。降低贡献门槛——使用简单的格式、提供模板、及时审查 PR。将文档贡献视为与代码贡献同等重要的工作。

1.7.9.9. RoboMaster 文档实践

结合 RoboMaster 的实际情况，以下是一些具体的文档实践建议。

赛季交接文档是 RoboMaster 团队最重要的文档之一。每年赛季结束后，核心队员应该编写详细的交接文档，包括：系统整体架构和设计思路、各模块的功能和接口、调试经验和已知问题、本赛季的改进和遗留问题、对下赛季的建议。这份文档将成为新队员的入门指南，也是团队知识积累的载体。

```
1 # 2024 赛季自瞄系统交接文档
2
3 ## 系统概述
4 本赛季自瞄系统采用 YOLOv8 + EKF 架构...
5
6
7 ## 取得的成果
8
9 - 检测帧率从 30 FPS 提升到 60 FPS
10 - 命中率从 60% 提升到 80%
11 - 支持了能量机关检测
12
13 ## 遗留问题
14
15 1. 强光下仍有误检（建议添加时域滤波）
16 2. 远距离检测精度不足（建议尝试更大的模型）
17 3. 代码耦合度高（建议重构跟踪模块）
18
19 ## 下赛季建议
```

```
20
21 - 考虑使用 Transformer 架构
22 - 完善单元测试覆盖
23 - 整理代码规范
```

硬件接口文档对于软硬件协作至关重要。它应该详细描述通信协议、数据格式、引脚定义、时序要求等。当软件组和电控组需要对接时，这份文档可以避免大量的沟通成本。

```
1  # 视觉-电控通信协议
2
3  ## 物理接口
4
5  - 接口类型: UART
6  - 波特率: 115200
7  - 数据位: 8
8  - 停止位: 1
9  - 校验: 无
10
11 ## 数据帧格式
12
13 | 字节 | 内容 | 说明 |
14 |-----|-----|-----|
15 | 0 | 0xA5 | 帧头 |
16 | 1 | len | 数据长度 |
17 | 2 | seq | 帧序号 |
18 | 3 | crc8 | 头部校验 |
19 | 4-N | data | 数据内容 |
20 | N+1,N+2 | crc16 | 整帧校验 |
21
22 ## 命令定义
23
24 #### 0x01: 自瞄数据
25
26 ```c
27 struct AimData {
28     float yaw;        // 目标 yaw 角度, 单位: 度
29     float pitch;      // 目标 pitch 角度, 单位: 度
30     uint8_t fire;     // 是否开火: 0-否, 1-是
31 };
32 ```
```

调试手册记录常见问题的诊断和解决方法。比赛现场时间紧迫，一份好的调试手册可以帮助快速定位和解决问题。

```
1  # 自瞄系统调试手册
2
3  ## 问题: 检测不到目标
4
5  #### 可能原因
6
7  1. 相机未正确连接
8  2. 曝光参数不合适
9  3. 目标颜色设置错误
10
11 #### 诊断步骤
12
13  1. 检查相机是否被识别: `ls /dev/video*`
14  2. 查看原始图像: `ros2 run rqt_image_view rqt_image_view`
15  3. 检查配置文件中的 `target_color` 设置
16
17 #### 解决方案
18
```

```
19 1. 重新插拔相机, 检查 USB 连接
20 2. 使用 `scripts/auto_exposure.py` 自动调整曝光
21 3. 修改 `config/detector.yaml` 中的颜色设置
22
23 ## 问题: 云台抖动
24
25 ...
```

文档不是写完就结束的一次性工作, 而是需要持续维护的活文档。每次代码修改、问题解决、经验总结, 都应该反映到文档中。将文档更新作为开发流程的一部分, 而不是事后的补充工作。团队可以建立这样的规范: 每个 PR 如果涉及接口变更, 必须同时更新相关文档; 每次调试解决的问题, 都要记录到调试手册中; 每个赛季结束, 必须完成交接文档。

文档是软件工程中经常被忽视但极其重要的一环。它是代码与人之间的桥梁, 是知识传承的载体, 是团队协作的基础。花在文档上的时间不是浪费, 而是对未来的投资。当你写文档时, 想象一下几个月后的自己、刚加入团队的新人、或者在比赛现场焦急调试的队友——你写下的每一个字, 都可能在某个时刻帮助到他们。

1.7.10. 版本控制与协作 (可选/扩展)

版本控制是现代软件工程的基石。无论是一个人的小项目还是数百人协作的大型系统, 版本控制都是不可或缺的工具。Git 作为当今最流行的版本控制系统, 已经成为每个开发者必须掌握的技能。本节不会详细讲解 Git 的具体命令——那些内容会在后续的 Git 专题章节中介绍。这里我们从软件工程的角度, 讨论版本控制在团队协作中的作用、分支策略的选择、代码审查的实践, 以及持续集成的概念。理解这些工程实践, 你才能真正发挥版本控制的价值。

1.7.10.1. 版本控制的工程意义

在没有版本控制的年代, 开发者们用各种原始的方法管理代码变化: 手动复制文件夹并命名为 `project_v1`、`project_v2`、`project_final`、`project_final_final`; 通过邮件或 U 盘传递代码; 遇到问题时绝望地试图回忆“昨天的代码是什么样的”。这种方式在个人小项目中勉强可行, 但在团队协作中会迅速崩溃。

版本控制系统从根本上解决了这些问题。它记录了代码的完整历史——每一次修改是谁做的、什么时候做的、改了什么、为什么改。你可以随时回到过去的任何版本, 比较两个版本之间的差异, 追踪某个 bug 是何时引入的。这种时间旅行的能力给了开发者巨大的安全感: 任何修改都可以撤销, 任何错误都可以恢复。

对于团队协作, 版本控制提供了一个协调多人同时工作的机制。每个人可以在自己的分支上独立开发, 完成后合并到主线。系统会自动检测冲突, 帮助你解决不同人修改同一文件的情况。没有版本控制, 多人协作几乎是不可能的——你无法知道别人改了什么, 也无法安全地整合各自的工作。

Git 还带来了分布式的优势。每个开发者的本地仓库都包含完整的历史, 可以离线工作、本地提交, 之后再与远程仓库同步。这与早期的集中式版本控制系统 (如 SVN) 形成对比——那些系统需要持续的网络连接, 服务器宕机时所有人都无法工作。

对于 RoboMaster 开发来说, 版本控制还有特殊的意义。比赛是有时间节点的, 你可能需要回滚到某个稳定版本参加比赛, 同时继续开发新功能。不同的机器人类型可能需要不同的代码配置, 分支可以帮助管理这些变体。赛后复盘时, 你可以追溯每一个改动, 分析问题的根源。这些都是版本控制带来的工程能力。

1.7.10.2. 分支策略：组织协作的艺术

分支是 Git 最强大的特性之一，但如何使用分支却是一个需要团队约定的问题。没有统一的分支策略，团队成员各自为政，分支混乱，合并困难，最终陷入“合并地狱”。好的分支策略既要支持并行开发，又要保持代码库的整洁和稳定。

Git Flow 是最经典的分支模型，由 Vincent Driessen 在 2010 年提出。它定义了严格的分支结构：`main`（或 `master`）分支始终是生产就绪的代码；`develop` 分支是开发主线，包含最新的开发成果；每个新功能在独立的 `feature` 分支上开发；发布前创建 `release` 分支进行最后的测试和修复；生产环境的紧急修复在 `hotfix` 分支上进行。

这种模型的优点是结构清晰、角色分明。`main` 分支的稳定性有保障，可以随时发布；`develop` 分支允许激进的开发；功能分支隔离了不同功能的开发，避免相互干扰。对于有固定发布周期的产品，Git Flow 工作得很好。

但 Git Flow 也有其复杂性。多个长期分支需要频繁同步，维护成本较高。对于持续部署的项目来说，`release` 分支可能是多余的。RoboMaster 项目通常没有传统软件那样的发布周期，使用完整的 Git Flow 可能过于繁琐。

GitHub Flow 是一个更简单的替代方案。它只有一个长期分支 `main`，所有开发都在功能分支上进行。功能完成后，通过 Pull Request 合并回 `main`，合并后立即可以部署。这种模型简单、灵活，适合持续部署和快速迭代的项目。

对于 RoboMaster 团队，一个实用的策略可能介于两者之间。`main` 分支保持稳定，是比赛时使用的版本；开发在功能分支上进行；合并前通过 Pull Request 进行代码审查；在重要比赛前，可能需要创建一个 `stable` 分支进行最后的测试。具体的策略应该根据团队规模、开发节奏和项目特点来调整，没有放之四海而皆准的答案。

无论采用什么策略，有几个原则是通用的。分支应该短命——长期存在的功能分支会积累大量冲突，合并时痛苦不堪。定期从主线同步更新到功能分支，保持与主线的差距可控。合并前确保分支是可工作的，至少能通过编译和基本测试。

1.7.10.3. 提交信息：代码历史的叙事

提交信息（commit message）看似是小事，但它对代码库的可维护性有着深远的影响。好的提交信息让人一眼就能理解这次修改做了什么、为什么做；差的提交信息让人一头雾水，不得不去看代码差异才能理解变化。

我们都见过这样的提交信息：`fix bug`、`update`、`修改`、`test`、`asdf`。这些信息毫无意义——每个提交都是在“修改”什么，问题是修改了什么？几个月后当你需要追溯某个问题时，这样的提交历史毫无帮助。

Conventional Commits 是一种被广泛采用的提交信息规范。它定义了结构化的格式，让提交信息既人类可读又机器可解析。基本格式是：

```
1 <type>(<scope>): <description>
2
3 [optional body]
4
5 [optional footer]
```

`type` 表示提交的类型：`feat` 是新功能，`fix` 是 bug 修复，`docs` 是文档更新，`refactor` 是重构，`test` 是测试相关，`chore` 是构建或工具相关的杂务。`scope` 是可选

的，表示影响的范围，如模块名或组件名。`description` 是简短的描述，使用祈使语气（“add feature” 而不是 “added feature”）。

一些具体的例子可以说明什么是好的提交信息：

```
1 feat(detector): add armor number classification
2
3 Implement CNN-based number classification for detected armors.
4 The model achieves 98% accuracy on the test set.
5
6 Closes #42
```

```
1 fix(tracker): correct Kalman filter initialization
2
3 The previous initialization caused divergence when target
4 first appears at the edge of the frame. Now uses measurement
5 as initial state estimate.
```

```
1 refactor(solver): extract ballistic calculation to separate class
2
3 No functional changes. Improves testability and separation
4 of concerns.
```

这些提交信息清晰地传达了改动的类型、范围和内容。将来查看历史时，不需要看代码就能理解每次提交做了什么。

好的提交习惯还包括原子性提交——每个提交应该是一个逻辑单元，只做一件事。不要把多个不相关的修改塞进一个提交，也不要把一个功能拆成太多琐碎的提交。如果提交信息需要用“和”字连接多件事，那可能应该是多个提交。

团队应该就提交信息规范达成一致。可以使用 `commitlint` 等工具自动检查提交信息格式，确保每个人都遵守约定。这个小小的纪律投资会在代码维护时带来巨大的回报。

1.7.10.4. Pull Request 与代码审查

代码审查（Code Review）是软件工程中提高代码质量的关键实践。其核心思想很简单：在代码合并到主线之前，让其他团队成员检查和评论。这个过程可以发现 bug、改进设计、传播知识、维护代码风格一致性。

在 GitHub 和 GitLab 等平台上，代码审查通过 Pull Request（或 Merge Request）进行。开发者完成功能分支后，创建一个 Pull Request，请求将分支合并到主线。其他团队成员会收到通知，可以查看改动、添加评论、提出修改建议。只有在审查通过后，代码才能合并。

好的代码审查关注几个方面。首先是正确性：代码是否实现了预期的功能？有没有明显的 bug？边界情况是否处理了？其次是设计：代码结构是否合理？是否符合项目的架构模式？有没有更好的实现方式？第三是可读性：代码是否易于理解？命名是否清晰？是否有足够的注释？第四是测试：是否有相应的测试覆盖？测试是否充分？

作为审查者，态度和措辞很重要。审查的目的是改进代码，而不是批评人。使用“我们可以考虑…”而不是“你应该…”，提出建议而不是命令。解释为什么某个改动更好，而不仅仅是说“这样不对”。记住，每个人都是来学习和进步的。

作为被审查者，要以开放的心态接受反馈。审查者花时间看你的代码是在帮助你。不要把批评当作对个人的攻击。如果不同意某个建议，可以讨论，但要尊重团队的决定。及时响应审查意见，不要让 Pull Request 长时间挂起。

对于 RoboMaster 团队，代码审查可能需要适应比赛节奏。在紧张的开发期，可能需要简化审查流程；但在重要比赛前，应该加强审查，确保合并的代码是稳定的。可以指定特定的模块维护者负责审查该模块的代码，利用专业知识进行更有效的审查。

1.7.10.5. 持续集成与自动化

持续集成（Continuous Integration, CI）是一种软件工程实践，核心思想是频繁地将代码集成到主线，每次集成都通过自动化构建和测试来验证。这样可以尽早发现问题，避免“集成地狱”——在项目后期才发现各个组件无法一起工作。

现代的 CI 系统（如 GitHub Actions、GitLab CI、Jenkins）可以在每次代码推送时自动执行一系列任务：编译代码、运行单元测试、进行代码风格检查、生成文档、构建 Docker 镜像等。如果任何步骤失败，开发者会立即收到通知，可以及时修复问题。

CI 对代码质量的保障是全方位的。每次提交都经过编译检查，保证代码至少能构建成功。每次提交都运行测试，保证没有引入回归问题。代码风格检查确保团队遵循统一的编码规范。这些检查是自动化的、一致的、不会疲劳的，比人工检查更可靠。

持续部署（Continuous Deployment, CD）更进一步，将通过 CI 验证的代码自动部署到生产环境。对于 RoboMaster 来说，“生产环境”可能是机器人上的软件系统。虽然完全自动化的部署可能不适合比赛场景（你不希望比赛前自动更新代码），但 CI 的理念——频繁集成、自动验证——是完全适用的。

一个典型的 RoboMaster 项目 CI 配置可能包括：在 Ubuntu 环境中编译整个项目；运行所有单元测试；检查代码风格是否符合规范；对于视觉算法，可能还会在测试数据集上运行评估。这些检查在每次 Pull Request 时自动运行，只有通过所有检查的代码才能合并。

设置 CI 需要一些前期投入，但回报是巨大的。它让团队对代码库的健康状况有持续的可见性，问题在引入时就被发现而不是积累到后期。它也解放了人力——不需要手动运行测试、手动检查编译、手动验证风格。机器做它擅长的重复性工作，人专注于创造性的工作。

1.7.10.6. 冲突解决：协作的代价

当多个人同时修改同一个文件的同一部分时，Git 无法自动判断该采用谁的修改，这时就会产生冲突。冲突是分布式协作的必然产物，不是错误，而是需要人类决策的情况。

Git 会在冲突的文件中标记出冲突的位置，用特殊的标记分隔不同的版本：

```
1 <<<<<<< HEAD
2 // 你的修改
3 void process(const Image& img) {
4     detector_.detect(img);
5 }
6 =====
7 // 别人的修改
8 void process(const cv::Mat& img) {
9     detector_.process(img);
10 }
11 >>>>>> feature/new-detector
```

解决冲突需要理解两边的改动意图，然后决定最终应该保留什么。有时候选择一边即可，有时候需要手动融合两边的修改。解决后，删除冲突标记，完成合并提交。

预防胜于解决。几个实践可以减少冲突的频率和复杂度。首先，保持分支短命，频繁合并。分支存在的时间越长，积累的差异越多，冲突的可能性越大。其次，频繁从主线同步更新。定期将主线的新改动合并到你的分支，保持与主线的差距可控。第三，合理划分模块。如果团队成员

负责不同的模块，他们修改同一文件的机会就少。第四，避免大规模重构与功能开发同时进行。重构会改动很多文件，与其他人的修改冲突的概率很高。

当冲突确实发生时，保持冷静。大多数冲突是简单的，几分钟就能解决。如果遇到复杂的冲突，找相关的开发者一起解决，确保正确理解双方的意图。解决后仔细测试，确保合并后的代码是正确的。

1.7.10.7. RoboMaster 团队的 Git 实践

最后，让我们讨论一些适合 RoboMaster 团队的 Git 实践。这些建议基于机器人开发的特点和学生团队的现实情况。

仓库组织方面，可以采用单一仓库（monorepo）或多仓库策略。单一仓库把所有代码放在一起，便于跨模块修改和原子性提交，但仓库可能变得庞大。多仓库将不同模块（如视觉、导航、控制）分开，每个仓库独立演化，但跨仓库协调更复杂。RoboMaster 项目的规模通常适合单一仓库，或者核心功能一个仓库、机器人配置一个仓库的简单划分。

分支保护是必要的。`main` 分支应该设置保护规则：禁止直接推送，必须通过 Pull Request 合并；要求至少一人审查通过；要求 CI 检查通过。这些规则确保了主分支的稳定性，防止意外的破坏性提交。

比赛前的版本管理需要特别注意。比赛前一周左右，可以从 `main` 创建一个 `competition` 分支，冻结大的功能开发，只进行 bug 修复和参数调整。比赛时使用这个分支的代码。比赛后，将 `competition` 分支的有价值修改合并回 `main`。

文档和配置也应该纳入版本控制。机器人的参数配置、标定数据、启动脚本都是代码的一部分，应该和代码一起版本化。这样你可以追溯某个配置是什么时候改的，也可以轻松回滚到之前的配置。

培养团队的版本控制文化需要时间。新成员可能不熟悉 Git，需要耐心指导。可以指定经验丰富的成员作为“Git 导师”，帮助新人解决问题。定期进行代码审查，不仅是为了审查代码，也是传播好的版本控制实践。

版本控制是软件工程的基础设施。像电力和自来水一样，当它正常工作时你不会注意到它，但没有它一切都会崩溃。投入时间建立好的版本控制实践，是对团队长期生产力的投资。这些实践会随着团队一起成长，成为团队工程文化的一部分。

1.8. Git 版本控制

1.8.1. Git 基本概念

很多新手最初对版本控制的理解是“备份”：改代码前先复制一个 `xxx_final.cpp`，改坏了再回滚到 `xxx_final_final.cpp`。这种方式在单人小脚本里还能勉强用，但在团队项目里会迅速失效。你不知道哪个副本是最新的，不知道谁改了什么，也无法把两个人的修改安全合并。版本控制系统的价值不只是保存历史，而是给团队提供一套可追溯、可协作、可回滚的工程机制。

Git 是目前事实标准的版本控制工具。它最核心的思想是：每次提交（commit）都是一个完整快照，而不是“增量补丁”。这让分支、合并、回滚都变得高效且可管理。对于 RoboMaster 这种多人并行开发、赛季节奏紧张、代码需要长期传承的项目，Git 不是可选项，而是基础设施。

1.8.1.1. 版本控制到底解决什么问题

版本控制主要解决四类问题。

第一类是可追溯性。你可以明确知道某个功能是谁在什么时间引入的，某个 bug 从哪个提交开始出现，某个参数为什么被改成当前数值。当系统比赛当天突然异常时，这种追溯能力非常关键。

第二类是并行开发。视觉组、决策组、导航组可以同时修改同一个仓库，不需要互相等待。每个人在自己的分支上推进，成熟后再合并到主线。

第三类是可回滚。新功能上线后如果性能下降，可以迅速回退到稳定版本；临场修复失败也能快速恢复。

第四类是知识传承。高质量的提交历史本身就是文档。后辈可以通过提交记录理解系统如何演进，而不是只看到当前这一帧静态代码。

1.8.1.2. Git 的对象模型：不是魔法，是数据结构

理解 Git 不需要先背几十条命令，先理解它保存了什么。

Git 底层主要有三类对象：

- **blob**：文件内容。
- **tree**：目录结构，指向若干 **blob** 或子 **tree**。
- **commit**：一次快照，记录一个 **tree**、父提交、作者和提交信息。

分支本质上只是一个可移动的引用，指向某个提交。**HEAD** 是当前检出的引用。你执行一次 `git commit`，实际上是生成新提交并让当前分支指向它。

上面这套结构（**blob/tree/commit** + 分支引用 + **HEAD**）可以解释很多常见现象：为什么切分支很快（只是移动引用）；为什么 Git 擅长合并（提交图是有向无环图）；为什么 `commit message` 很重要（它是历史图上的语义锚点）。

1.8.1.3. 三个工作区：工作区、暂存区、版本库

Git 的日常操作围绕三个区域展开：

- 工作区（Working Tree）：你正在编辑的文件。
- 暂存区（Index / Staging Area）：本次准备提交的内容清单。
- 版本库（Repository）：已经提交并永久记录的历史。

这也是新手最容易混淆的地方。典型流程是：先在工作区改代码，用 `git add` 把“本次想提交”的改动放进暂存区，再用 `git commit` 把暂存区快照写入版本库。

```
1 # 1) 看当前改动
2 git status
3 # 2) 只暂存本次要提交的文件
4 git add src/detector.cpp
5 # 3) 生成提交
6 git commit -m "feat(detector): add contour pre-filter"
```

暂存区的存在让你可以把一大坨修改拆成多个语义清晰的小提交。比如你一边修 bug 一边顺手格式化文件，可以只暂存 bug 修复部分，避免把不相关改动混进同一个提交。

1.8.1.4. 为什么 RoboMaster 团队必须系统化用 Git

RoboMaster 项目有三个典型特征：

- 赛季内需求变化快，试错密度高。
- 多模块并行（视觉、决策、通信、嵌入式）。
- 人员代际交接明显。

这决定了你们不能只“会用 `git add/git commit`”，而需要明确分支策略、提交规范、合并流程和发布标签。否则仓库会在赛季中后期迅速失控：主分支不稳定、回滚困难、冲突高发、交接成本爆炸。

1.8.2. 本地仓库操作

本地操作是所有协作流程的基石。你如果在本地不能稳定管理改动，到了远程协作只会放大问题。

1.8.2.1. 创建仓库：init 与 clone

新项目从零开始时用 `git init`：

```
1 mkdir rm_vision
2 cd rm_vision
3
4 git init
5 git add .
6 git commit -m "chore: initialize repository"
```

已有远程项目时用 `git clone`：

```
1 git clone git@github.com:team/rm_vision.git
2 cd rm_vision
```

如果仓库含子模块，建议直接递归克隆：

```
1 git clone --recursive git@github.com:team/rm_vision.git
```

1.8.2.2. 每天都要看的三个命令

`git status`、`git log`、`git diff` 是最核心的可视化工具。

```
1 # 工作区/暂存区状态
2 git status
3 # 最近提交图
4 git log --oneline --graph --decorate -20
5 # 未暂存改动
6 git diff
7 # 已暂存改动
8 git diff --staged
```

- `status` 看当前工作区和暂存区状态。
- `log` 看历史结构，`--graph` 可以看到分支合并关系。
- `diff` 看改动内容，`--staged` 看已暂存内容。

养成提交前先看 `status + diff --staged` 的习惯，能显著减少误提交。

1.8.2.3. add 与 commit：提交的是“意图”而不是“时间点”

初学者常见错误是：代码刚能跑就 `git add . && git commit -m "update"`。这会把不相关改动打包成噪声提交。

更好的流程是：

1. 先把一个最小功能改完整。
2. 用 `git add <file>` 或交互式 `git add -p` 精确暂存。

3. 写有语义的提交信息。

```
1 git add -p src/tracker.cpp
2 git commit -m "fix(tracker): guard against empty candidate set"
```

推荐提交信息结构：

- **feat** : 新功能
- **fix** : 缺陷修复
- **refactor** : 重构（不改行为）
- **docs** : 文档
- **test** : 测试
- **chore** : 杂项维护

示例: **feat(protocol): add crc16 verification for serial frames**

1.8.2.4. 撤销改动：先搞清你要撤销哪一层

Git 的“撤销”命令多，是因为它们作用层不同。

只撤工作区改动（未暂存）：

```
1 git restore src/detector.cpp
```

撤销暂存（保留工作区修改）：

```
1 git restore --staged src/detector.cpp
```

回退分支指针（谨慎）：

```
1 git reset --soft HEAD~1 # 回退一个提交，改动留在暂存区
2 git reset --mixed HEAD~1 # 回退一个提交，改动留在工作区
```

对外已发布历史不要用 **reset** 改写，应该用 **revert** 生成反向提交：

```
1 git revert <commit_sha>
```

规则很简单：

- 仅本地私有分支可改写历史（**reset/rebase**）。
- 已推送且他人可能基于其开发的历史，用 **revert**。

1.8.2.5. 标签（Tag）是发布里程碑，不是装饰

比赛前后建议打标签管理可复现版本：

```
1 git tag -a v2026-pre-match -m "Pre-match stable version"
2 git push origin v2026-pre-match
```

当现场需要快速回退到已验证版本时，标签能极大缩短决策时间。

1.8.3. GitHub/GitLab 远程协作

远程协作的核心不是“把代码传上去”，而是保持本地与远程历史的一致、可理解、可合并。

1.8.3.1. 远程仓库基础：remote / fetch / pull / push

查看远程：

```
1 git remote -v
```

同步远程引用（不自动合并）：

```
1 git fetch origin
```

拉取并合并：

```
1 git pull origin main
```

推送当前分支：

```
1 git push -u origin feature/armor-detector
```

推荐把 `pull` 设为 `rebase`，减少无意义 `merge commit`：

```
1 git config --global pull.rebase true
```

1.8.3.2. SSH 认证建议

团队仓库建议统一 SSH 协议，避免 HTTPS 反复输密码。

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
2 cat ~/.ssh/id_ed25519.pub
```

把公钥添加到 GitHub/GitLab 后，用以下命令验证：

```
1 ssh -T git@github.com
```

1.8.3.3. Fork 模式与 Upstream 同步

如果你们采用 Fork 工作流，通常会有两个远程：

- `origin`：你自己的 fork。
- `upstream`：团队主仓库。

```
1 git remote add upstream git@github.com:team/rm_vision.git
2 git fetch upstream
3 git checkout main
4 git rebase upstream/main
5 git push origin main
```

这样可以保证你的功能分支始终基于最新主线。

1.8.3.4. 协作中最常见的三个坑

第一个坑：直接在 `main` 上开发。结果是主分支长期不稳定，任何人 `pull` 都可能失败。

第二个坑：长时间不同步主线。你写了一周再合并，冲突会集中出现。

第三个坑：把编译产物推上去（如 `build/`、`install/`、`log/`）。仓库体积和冲突风险都会失控。

1.8.4. 分支管理

分支不是为了“展示技巧”，而是为了隔离风险。每个功能、缺陷修复都应在独立分支完成，经过评审再合并。

1.8.4.1. 推荐分支模型（适配学生团队）

对 RoboMaster 团队来说，简单模型通常比复杂 Git Flow 更实用：

- `main`：始终可发布、可运行。
- `dev`（可选）：阶段性集成分支。
- `feature/*`：功能分支。
- `fix/*`：缺陷修复分支。
- `hotfix/*`：比赛期紧急修复分支。

分支命名建议包含模块和目标：

- `feature/vision-armor-classifier`
- `fix/tracker-timeout`
- `hotfix/serial-crc`

1.8.4.2. merge 和 rebase 怎么选

`merge` 保留真实分支拓扑，历史更完整；`rebase` 让历史更线性、阅读更干净。

常见实践：

- 本地整理自己分支时，优先 `rebase`。
- 合并到主线时，使用 PR 平台的 `Squash merge` 或普通 `merge`，保持审查记录。

把 `rebase` 理解为“重放提交”，把 `merge` 理解为“创建汇合点”，就不会混淆。

1.8.4.3. 冲突处理：不要慌，按流程来

发生冲突时的标准流程：

```
1 git fetch origin
2 git rebase origin/main
3 # 编辑冲突文件，手动选择内容
4 git add <resolved_file>
5 git rebase --continue
```

冲突文件会出现标记：

```
1 <<<<<<< HEAD
2 本分支内容
3 =====
4 目标分支内容
5 >>>>>>> origin/main
```

处理原则：

- 先保证编译通过，再追求局部优雅。
- 冲突解决后必须运行相关测试或最小回归验证。
- 复杂冲突不要单打独斗，直接拉相关作者一起看。

1.8.5. 团队 workflow (PR、Code Review)

PR (Pull Request / Merge Request) 是团队协作的核心接口。它不仅是合并代码的入口，也是技术讨论、知识扩散和质量守门点。

1.8.5.1. 推荐 PR 流程

1. 从最新主线切功能分支。
2. 小步提交，保持每个提交可解释。
3. 推送远程后创建 PR。
4. 填写清晰描述：背景、改动、验证方式、风险点。
5. 至少一名同伴审查通过后合并。

PR 描述建议模板：

```
1 ## 背景
2 修复高速旋转目标下 tracker 丢失问题。
3
4 ## 改动
5 - 调整候选目标门控条件
6 - 增加时间戳异常保护
7 - 新增 2 个单元测试
8
9 ## 验证
10 - 本地回放数据集 `data/2026-01-armor.bag`
11 - 30 分钟压力测试无崩溃
12
13 ## 风险
14 参数阈值更严格，可能降低远距离召回率
```

1.8.5.2. Code Review 的关注重点

审查者应优先看四件事：

- 行为正确性：逻辑是否符合需求。
- 边界条件：空数据、超时、异常输入是否处理。
- 可维护性：命名、模块边界、耦合度。
- 回归风险：是否影响实时性、通信协议、关键路径。

不要把审查焦点浪费在格式争论上，格式交给 `clang-format`，审查聚焦语义和风险。

1.8.5.3. CI 与合并门禁

建议把以下检查设为 PR 必过门禁：

- 编译通过 (Debug/Release 至少一套)。
- 单元测试通过。
- 基础静态检查通过 (如 `clang-tidy` 的关键规则)。

把 CI 作为硬门禁后，主分支通常会稳定很多，线上回滚次数也会下降。

1.8.5.4. 团队落地清单 (可放入 README)

如果你希望这章内容真正执行起来，可以先把下面 5 条写进团队规约：

1. `main` 开启保护，不允许直接 push。
2. 所有改动通过 PR 合并，至少 1 人 review。
3. CI 未通过禁止合并。

4. 提交信息遵循 `type(scope): message`。
5. 功能分支尽量短周期（建议 13 天内发起 PR）。

规则不用一开始就很复杂，先把这 5 条执行稳定，比讨论流程名字更重要。

1.8.6. .gitignore 与 submodule

1.8.6.1. .gitignore：让仓库只保存源码与必要资产

Git 不应跟踪编译产物、临时文件和本地环境文件。基础篇里前面已经大量出现 `build/` `install/` `log/`，这些都应该忽略。

RoboMaster C++/ROS2 项目常用的 `.gitignore` 示例：

```
1 # Build outputs
2 build/
3 install/
4 log/
5
6 # CMake
7 CMakeCache.txt
8 CMakeFiles/
9 cmake_install.cmake
10 compile_commands.json
11
12 # IDE
13 .vscode/
14 .idea/
15
16 # Python
17 __pycache__/
18 *.pyc
19
20 # Logs & runtime
21 *.log
22 core
23
24 # OS junk
25 .DS_Store
26 Thumbs.db
```

如果某文件已经被跟踪，再写到 `.gitignore` 不会自动生效，需要先取消跟踪：

```
1 git rm -r --cached build
2 git commit -m "chore(git): stop tracking build artifacts"
```

1.8.6.2. 什么时候该用 submodule

`submodule` 用于把另一个仓库作为“固定版本依赖”嵌入当前仓库，典型场景：

- 你们维护了一套通用算法库，多个项目共享。
- 需要锁定第三方仓库的特定提交，避免上游变动破坏稳定性。

添加子模块：

```
1 git submodule add git@github.com:team/rm_common.git third_party/rm_common
2 git commit -m "chore: add rm_common submodule"
```

更新子模块到新提交：

```
1 cd third_party/rm_common
2 git checkout main
3 git pull
4 cd ../../
5 git add third_party/rm_common
6 git commit -m "chore(submodule): bump rm_common"
```

克隆含子模块仓库后初始化：

```
1 git submodule update --init --recursive
```

1.8.6.3. submodule 常见误区

第一个误区是忘记更新子模块指针。你在子模块目录改完代码，但主仓库没 `git add` 子模块路径，别人拿不到更新。

第二个误区是把 submodule 当包管理器。若依赖频繁改动且强耦合，直接合并到同仓库或改用包管理方案通常更省心。

第三个误区是忽略开发体验。新成员不会处理 submodule，会导致“代码看着完整但编译不过”。必须在 README 写清初始化命令。

1.8.7. 小结：把 Git 当作工程系统而不是命令集合

学 Git 的终点不是记住更多命令，而是建立稳定的工程流程：

- 本地改动可控（小步提交、语义清晰）。
- 分支协作可控（隔离风险、及时同步）。
- 合并质量可控（PR + Review + CI）。
- 发布回滚可控（标签与稳定分支）。

把这四条流程长期执行下来，仓库会更稳：新人更容易接手，比赛前回滚也更快。

1.9. ROS/ROS2 入门

1.9.1. ROS/ROS2 架构与概念

机器人软件的本质是“多模块并发系统”。相机在持续产生日志和图像，检测器在做视觉推理，跟踪器在融合时序信息，决策模块在生成控制指令，串口节点在和下位机通信。把这些逻辑全部塞进一个进程看似简单，但很快会陷入耦合失控：难调试、难复用、难并行开发。

ROS (Robot Operating System) 提供的不是“操作系统内核”，而是一套机器人中间件框架。它把系统拆成多个节点 (node)，节点之间通过标准化通信机制交互。这样每个模块都可以独立开发、独立测试、独立替换。

1.9.1.1. ROS1 与 ROS2：该如何理解

如果你在网上搜教程，会同时看到 ROS1 和 ROS2。可以把它们理解为同一生态的两代架构：

- ROS1：成熟、资料多，但通信和实时能力先天受限。
- ROS2：基于 DDS 通信中间件，支持 QoS、多机通信、更好的工程化能力。

对于新项目，建议优先采用 ROS2。前面 CMake 章节已经覆盖 `ament_cmake` 和 `colcon build`，这套流程就是 ROS2 的工程基础。

1.9.1.2. ROS2 的核心对象

理解下面几个概念，基本就能读懂大部分 ROS2 工程：

- **node**：进程中的功能单元，如 `detector_node`、`tracker_node`。
- **topic**：发布/订阅通道，适合持续数据流，如图像、目标列表。
- **service**：请求/响应模式，适合短操作，如“重置跟踪器”。
- **action**：可反馈、可取消的长任务，适合“导航到目标点”这类过程。
- **parameter**：节点运行参数，如阈值、模型路径、串口端口号。

把它映射到 RoboMaster 会更直观：

- 相机节点发布 **Image** 话题。
- 识别节点订阅图像，发布装甲板候选。
- 决策节点订阅目标，输出云台控制命令。
- 维护节点提供服务用于在线重置状态。

1.9.1.3. 包与工作空间

ROS2 项目组织有两层：

- 包 (package)：最小构建和发布单元。
- 工作空间 (workspace)：多个包的集合。

常见工作空间结构：

```
1 ros2_ws/  
2   src/  
3     rm_interfaces/  
4     rm_detector/  
5     rm_tracker/  
6     rm_decision/  
7   build/  
8   install/  
9   log/
```

开发时你主要操作 `src/`，其余目录是构建产物。

1.9.2. 话题、服务、动作

很多新手把 ROS2 的通信方式当成“三个都差不多”，这是系统设计混乱的开端。正确做法是按语义选模型。

1.9.2.1. Topic：高频、持续、弱耦合

话题是发布/订阅模型，发布者不关心谁在订阅，订阅者也不关心发布者是谁。它最适合持续流数据：

- 相机帧
- IMU 数据
- 目标检测结果
- 控制状态广播

常用排查命令：

```
1 ros2 topic list  
2 ros2 topic info /detector/targets  
3 ros2 topic echo /detector/targets
```

```
4 ros2 topic hz /detector/targets
```

`hz` 在实战里很关键。你可以快速判断某个节点是否掉帧或阻塞。

1.9.2.2. QoS: ROS2 通信稳定性的关键旋钮

ROS2 的 QoS (Quality of Service) 决定消息传输策略。最常见的几项:

- `reliability: reliable / best_effort`
- `history: keep_last / keep_all`
- `depth`: 队列深度
- `durability`: 是否给后加入订阅者保留历史

视觉高频流常用 `best_effort + keep_last`, 控制关键数据更倾向 `reliable`。如果发布者和订阅者 QoS 不兼容, 表面现象通常是“节点都在跑但收不到数据”。

1.9.2.3. Service: 短时的一次性操作

服务是同步请求/响应, 更像函数调用。典型场景:

- 重置滤波器状态
- 切换工作模式
- 请求一次标定结果

常用命令:

```
1 ros2 service list
2 ros2 service type /tracker/reset
3 ros2 service call /tracker/reset std_srvs/srv/Trigger "{}"
```

服务不适合高频周期调用。把 60Hz 数据链路做成 service, 会把系统拖慢并放大阻塞风险。

1.9.2.4. Action: 有进度反馈、可取消的长任务

动作用于持续执行且需要反馈的任务, 例如:

- 导航到某目标点
- 云台执行扫描策略
- 自动标定流程

常用命令:

```
1 ros2 action list
2 ros2 action info /auto_aim_task
```

判断规则可以很实用:

- 只要一问一答, 用 `service`。
- 连续数据流, 用 `topic`。
- 需要进度和取消控制, 用 `action`。

1.9.3. 创建节点与发布订阅

下面按步骤搭一个最小示例, 先把发布-订阅链路跑通。

1.9.3.1. 创建工作空间与包

```
1 # 1) 准备工作空间
2 mkdir -p ~/ros2_ws/src
3 cd ~/ros2_ws/src
4
5 # 2) 创建示例包
6 ros2 pkg create rm_vision_demo --build-type ament_cmake --dependencies
  rclcpp std_msgs geometry_msgs
7
8 # 3) 回到工作空间根目录并构建
9 cd ~/ros2_ws
10 colcon build --symlink-install
11
12 # 4) 加载环境
13 source install/setup.bash
```

建议在 `~/.bashrc` 中加入 ROS2 与工作空间环境加载（注意按你实际发行版调整路径）：

```
1 source /opt/ros/<your_ros2_distro>/setup.bash
2 source ~/ros2_ws/install/setup.bash
```

1.9.3.2. 示例：目标发布节点 (publisher)

`target_publisher.cpp`：周期发布一个简化目标点。

```
1 #include <chrono>
2 #include <memory>
3
4 #include <geometry_msgs/msg/point.hpp>
5 #include <rclcpp/rclcpp.hpp>
6
7 using namespace std::chrono_literals;
8
9 class TargetPublisher final : public rclcpp::Node {
10 public:
11     TargetPublisher() : Node("target_publisher"), x_(0.0) {
12         pub_ = create_publisher<geometry_msgs::msg::Point>("/detector/target",
13             10);
14         timer_ = create_wall_timer(50ms, [this]() { Publish(); });
15     }
16 private:
17     void Publish() {
18         geometry_msgs::msg::Point msg;
19         msg.x = x_;
20         msg.y = 0.2;
21         msg.z = 5.0;
22         x_ += 0.01;
23
24         pub_->publish(msg);
25         RCLCPP_INFO_THROTTLE(get_logger(), *get_clock(), 1000,
26             "publish target x=%.3f", msg.x);
27     }
28
29     double x_;
30     rclcpp::Publisher<geometry_msgs::msg::Point>::SharedPtr pub_;
31     rclcpp::TimerBase::SharedPtr timer_;
32 };
33
34 int main(int argc, char* argv[]) {
35     rclcpp::init(argc, argv);
```

```

36   rclcpp::spin(std::make_shared<TargetPublisher>());
37   rclcpp::shutdown();
38   return 0;
39 }

```

1.9.3.3. 示例：决策订阅节点 (subscriber)

`decision_subscriber.cpp`：接收目标点并输出简单决策日志。

```

1  #include <memory>
2
3  #include <geometry_msgs/msg/point.hpp>
4  #include <rclcpp/rclcpp.hpp>
5
6  class DecisionSubscriber final : public rclcpp::Node {
7  public:
8      DecisionSubscriber() : Node("decision_subscriber") {
9          sub_ = create_subscription<geometry_msgs::msg::Point>(
10             "/detector/target", 10,
11             [this](const geometry_msgs::msg::Point::SharedPtr msg) {
12                 const bool in_range = msg->z < 8.0;
13                 RCLCPP_INFO(get_logger(),
14                     "recv target x=%.3f y=%.3f z=%.3f => %s",
15                     msg->x, msg->y, msg->z,
16                     in_range ? "fire_allowed" : "hold_fire");
17             });
18      }
19
20  private:
21      rclcpp::Subscription<geometry_msgs::msg::Point>::SharedPtr sub_;
22  };
23
24  int main(int argc, char* argv[]) {
25      rclcpp::init(argc, argv);
26      rclcpp::spin(std::make_shared<DecisionSubscriber>());
27      rclcpp::shutdown();
28      return 0;
29  }

```

在 `CMakeLists.txt` 中注册两个可执行文件并安装：

```

1  add_executable(target_publisher src/target_publisher.cpp)
2  ament_target_dependencies(target_publisher rclcpp geometry_msgs)
3
4  add_executable(decision_subscriber src/decision_subscriber.cpp)
5  ament_target_dependencies(decision_subscriber rclcpp geometry_msgs)
6
7  install(TARGETS
8      target_publisher
9      decision_subscriber
10     DESTINATION lib/${PROJECT_NAME})
11 )

```

构建与运行：

```

1  # 构建当前包
2  cd ~/ros2_ws
3  colcon build --packages-select rm_vision_demo --symlink-install
4  source install/setup.bash
5
6  # 终端 1：发布目标

```

```

7 ros2 run rm_vision_demo target_publisher
8
9 # 终端 2: 订阅并输出决策
10 ros2 run rm_vision_demo decision_subscriber

```

如果这两条节点都能稳定输出日志，说明最小链路已经打通，可以继续做 launch 和参数管理。

1.9.3.4. 最小链路验收（建议按此检查）

跑完上面的命令后，至少确认这三项：

1. `ros2 node list` 能看到 `target_publisher` 和 `decision_subscriber`。
2. `ros2 topic hz /detector/target` 频率基本稳定（与定时器设置接近）。
3. 重开一个新终端后重新 `source`，节点还能正常启动。

1.9.4. launch 文件

当节点数量超过两个后，手动开多个终端会迅速失控。`launch` 的价值在于把“如何启动系统”变成可版本化的代码。

1.9.4.1. 最小 launch 示例

`launch/vision_decision.launch.py`：

```

1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4
5 def generate_launch_description():
6     detector = Node(
7         package="rm_vision_demo",
8         executable="target_publisher",
9         name="detector_node",
10        output="screen",
11        parameters=[{"publish_rate_hz": 20.0}],
12    )
13
14    decision = Node(
15        package="rm_vision_demo",
16        executable="decision_subscriber",
17        name="decision_node",
18        output="screen",
19        remappings=[("/detector/target", "/detector/target")],
20    )
21
22    return LaunchDescription([detector, decision])

```

启动命令：

```

1 ros2 launch rm_vision_demo vision_decision.launch.py

```

1.9.4.2. 参数文件与多场景配置

比赛中常见两套配置：

- 调试配置（日志详细、阈值保守）。
- 比赛配置（性能优先、日志收敛）。

建议把参数放进 YAML:

```
1 # config/detector.yaml
2 detector_node:
3   ros__parameters:
4     threshold: 0.65
5     max_lost_frames: 8
```

然后在 launch 中加载:

```
1 parameters=["config/detector.yaml"]
```

这样调参不需要改 C++ 并重新编译。

1.9.4.3. 命名空间与重映射

多车协同或多相机场景中，命名空间非常重要。比如同一算法复用两路相机:

- /front/detector/target
- /rear/detector/target

通过 launch 设置 namespace 与 remap，可以在不改核心代码的前提下复用节点。

1.9.5. ROS 工具 (rqt、rviz2)

工具的作用很直接：先确认系统有没有连上，再看数据是否合理。

1.9.5.1. rqt：先看拓扑，再看细节

启动图形工具集:

```
1 rqt
```

最常用插件:

- rqt_graph：查看节点和话题拓扑，先确认链路是否连通。
- rqt_plot：查看数值随时间变化，适合调滤波器和控制量。
- rqt_console：集中看日志，快速定位异常节点。

如果你预期有消息但 rqt_graph 没连接，先检查话题名和 QoS 是否一致。

1.9.5.2. rviz2：把抽象数据变成可验证画面

启动:

```
1 rviz2
```

视觉与目标跟踪常用显示项:

- Image：相机图像。
- PointCloud2：点云。
- Marker/MarkerArray：检测框、轨迹、预测点。
- TF：坐标系关系。

Fixed Frame 设置错误是最常见问题之一。如果你所有显示都“看不见”，先检查 TF 树是否完整。

1.9.5.3. 一套实用排查顺序

当系统“看起来在跑但结果不对”时，按顺序排查：

1. `ros2 node list`：节点是否都在。
2. `ros2 topic list`：关键话题是否存在。
3. `ros2 topic hz/echo`：频率和内容是否合理。
4. `rqt_graph`：链路是否断开。
5. `rviz2`：坐标系和可视化数据是否一致。

不要一上来就改算法，先确认链路和数据面是正确的。

1.9.6. 实战：简单的视觉/决策节点

下面给一个可扩展的起步架构，先求能联调，再逐步替换成真实算法。

1.9.6.1. 建议的最小模块划分

- `camera_node`：采集图像并发布。
- `detector_node`：识别装甲板并发布目标。
- `tracker_node`：做时序关联、状态估计。
- `decision_node`：根据目标状态输出云台/射击控制指令。
- `serial_node`：和下位机通信。

先把接口打通，再迭代算法细节。不要反过来。

1.9.6.2. 先定义消息，再写逻辑

如果团队暂时不想维护自定义消息包，也可以用标准消息快速推进；但进入正式阶段建议抽出 `rm_interfaces` 包，统一定义：

- `ArmorTarget.msg`
- `TargetArray.msg`
- `GimbalCommand.msg`

接口稳定后，模块才能真正并行开发，避免“每次改字段全链路改代码”。

1.9.6.3. 录包与回放是算法迭代加速器

现场调试时间稀缺，建议建立数据闭环：

```
1 # 录制关键话题
2 ros2 bag record /camera/image_raw /detector/target /tracker/state
3
4 # 回放数据（可加速或减速）
5 ros2 bag play <bag_path>
```

这样你可以在实验室复现赛场问题，不依赖实时硬件环境反复试错。

1.9.6.4. 从 Demo 走向工程化

从最小示例到可比赛系统，通常按这个顺序演进：

1. 单机跑通消息链路。
2. 引入参数文件和 launch 管理。
3. 增加日志、监控、异常保护。
4. 引入单元测试和回放测试。

5. 做性能压测与瓶颈优化。

前四步做完后，ROS2 基本就能支撑你们的日常联调和回归测试。

1.9.7. 小结：先把系统边界建立清楚，再追求算法上限

ROS/ROS2 的核心价值是把复杂机器人系统拆成可协作的模块。你掌握的重点不该只是命令，而是三件事：

- 正确选择通信模型（topic/service/action）。
- 用包、参数、launch 管理系统复杂度。
- 用 rqt、rviz2、bag 建立可观测、可复现的调试闭环。

先把这三件事做扎实，再上复杂算法，会少很多返工。

2. 数学理论篇

2.1. 计算机视觉基础

2.2. 传统视觉算法

2.3. 深度学习和神经网络

2.4. 相机模型

2.5. 坐标变换

2.6. 卡尔曼滤波

Definition 2.6.1

本章使用的符号规定：

- **粗体小写字母** 表示向量，例如 $\boldsymbol{x}$
- **粗体大写字母** 表示矩阵，例如 $\boldsymbol{A}$
- **普通小写字母** 标量或向量的分量
- **普通大写字母** 表示矩阵元素

2.6.1. 什么是卡尔曼滤波？

2.6.2. 如何准确地测量体重

考虑一个存在随机测量误差的体重秤。单次测量无法得到准确结果，但通过多次测量取平均，可以获得更稳定的估计值。

对同一对象进行 n 次测量，取平均值作为估计：

$$\hat{x}_n = \frac{1}{n} \sum_{i=1}^n z_i$$

然而，直接计算平均值需要存储全部历史数据 $z_1, z_2, \dots, z_n$ ，且每次更新都需遍历所有数据，时间复杂度为 $O(n)$ 。对于资源受限的嵌入式系统，这一方法难以实现。

更理想的方案是递推 (Recursive) 形式：仅利用上一次估计 $\hat{x}_{n-1}$ 与当前测量 z_n 完成更新，无需保存历史数据。

通过代数变换，平均值公式可改写为：

$$\hat{x}_n = \hat{x}_{n-1} + \frac{1}{n}(z_n - \hat{x}_{n-1})$$

上式表明，第 n 次估计仅依赖 $\hat{x}_{n-1}$ 与 z_n ，时间和空间复杂度均为 $O(1)$ 。

$$\begin{aligned}
\hat{x}_n &= \frac{1}{n} \sum_{i=1}^n z_i = \frac{1}{n} \left(\sum_{i=1}^{n-1} z_i + z_n \right) \\
&= \frac{1}{n} \sum_{i=1}^{n-1} z_i + \frac{1}{n} z_n = \frac{n-1}{n(n-1)} \sum_{i=1}^{n-1} z_i + \frac{1}{n} z_n \\
&= \frac{n-1}{n} \cdot \frac{1}{n-1} \sum_{i=1}^{n-1} z_i + \frac{1}{n} z_n = \frac{n-1}{n} \hat{x}_{n-1} + \frac{1}{n} z_n \\
&= \hat{x}_{n-1} - \frac{1}{n} \hat{x}_{n-1} + \frac{1}{n} z_n \\
&= \hat{x}_{n-1} + \frac{1}{n} (z_n - \hat{x}_{n-1})
\end{aligned}$$

符号	含义
x	体重真值
z_n	第 n 次测量值
$\hat{x}_n$	基于前 n 次测量的估计值
$\hat{x}_{n-1}$	基于前 $n-1$ 次测量的估计值
$z_n - \hat{x}_{n-1}$	测量残差

note 2 详细推导

上述递推公式已具备状态估计滤波器的基本形式。其中增益系数 $K_n = \frac{1}{n}$ 决定了测量值与历史估计的融合权重。随着 n 增加, K_n 递减——可用数据越多, 单次新测量的影响越小。

这一结构与卡尔曼滤波的状态更新公式形式一致:

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k (z_k - h(\hat{x}_{k|k-1}))$$

其中:

- $\hat{x}_{k|k-1}$: 基于前 $k-1$ 次观测的先验估计
- z_k : 第 k 次测量值
- $h(\cdot)$: 观测函数
- K_k : 卡尔曼增益

在体重秤场景中, 状态不随时间变化 ($\hat{x}_{k|k-1} = \hat{x}_{k-1}$), 观测函数为恒等映射 ($h(x) = x$), 因此递推平均可视为卡尔曼滤波在静态系统下的特例。

将递推公式标注为:

$$\hat{x}_n = \underbrace{\hat{x}_{n-1}}_{\text{先验估计}} + \underbrace{\frac{1}{n}}_{\text{增益}} \underbrace{(z_n - \hat{x}_{n-1})}_{\text{残差}}$$

三个组成部分的含义: **先验估计** $\hat{x}_{n-1}$ 为获得新测量前的当前认知; **测量残差** $r_n = z_n - \hat{x}_{n-1}$ 为观测值与先验的偏差; **增益系数** K_n 控制残差对估计的修正幅度。这一“先验 + 增益 × 残差”的结构是递推估计的通用框架。

2.6.3. 从体重秤到赛车：静态估计的局限性

递推平均能够有效估计静态量, 是因为体重在测量过程中保持不变, 所有偏差均可归因于传感器噪声。但对于动态系统, 情况有所不同。

考虑追踪赛道上的赛车: 起始位置 0 米, 以 20 m/s 匀速行驶。GPS 每秒测量一次位置, 误差 ± 5 米。

真实轨迹为 $p_t = 20t$, GPS 测量值 $z_t = p_t + \text{噪声}$ 。

时刻 (s)	真实位置 (m)	GPS 测量 (m)	递推平均 (m)	误差 (m)
0	0	1.2	1.2	+1.2
1	20	18.3	9.8	-10.2
2	40	43.1	20.9	-19.1
3	60	56.8	29.9	-30.1
4	80	82.4	40.4	-39.6
5	100	97.6	49.9	-50.1

表 1 递推平均追踪动态目标的失效

估计误差随时间单调增大。问题在于递推平均的假设: 被估计量保持不变。该公式将测量值的所有变化都视为噪声, 无法区分噪声与真实的位置变化。

本质问题是: **递推平均只有修正, 没有预测**。正确的思路应在获得新测量前, 先根据运动规律预测当前位置, 再用测量值修正。这正是 g-h 滤波器的核心思想。

2.6.3.1. g-h 滤波器

对于运动物体, 估计应包含预测步骤:

估计 = 预测 (运动模型) + 修正 (测量)

设在第 $k - 1$ 时刻已有位置估计 $\hat{p}_{k-1}$ 与速度估计 $\hat{v}_{k-1}$ 。根据匀速运动模型, 第 k 时刻的预测为:

$$p_{\text{pred},k} = \hat{p}_{k-1} + \hat{v}_{k-1} \cdot \Delta t$$
$$v_{\text{pred},k} = \hat{v}_{k-1}$$

其中 Δt 为采样间隔。

若 $\hat{p}_0 = 0$ m, $\hat{v}_0 = 15$ m/s, $\Delta t = 1$ s, 则 $p_{\text{pred},1} = 15$ m。

当测量值 z_k 到来时, 计算残差:

$$r_k = z_k - p_{\text{pred},k}$$

位置修正

$$\hat{p}_k = p_{\text{pred},k} + g \cdot r_k$$

其中 $g \in (0, 1)$ 为位置增益。 $g = 0$ 表示完全信任预测, $g = 1$ 表示完全信任测量。

速度修正

$$\hat{v}_k = v_{\text{pred},k} + \frac{h}{\Delta t} \cdot r_k$$

其中 $h \in (0, 1)$ 为速度增益。除以 Δt 是为了将位置残差（单位：m）转换为速度修正量（单位：m/s）。

以 $g = 0.3$, $h = 0.1$, $z_1 = 18.3 \text{ m}$ 为例： $r_1 = 3.3 \text{ m}$, $\hat{p}_1 = 15.99 \text{ m}$, $\hat{v}_1 = 15.33 \text{ m/s}$ 。

g-h 滤波器算法

输入： $\hat{p}_{k-1}$, $\hat{v}_{k-1}$, z_k , Δt , g , h

预测：

$$p_{\text{pred},k} = \hat{p}_{k-1} + \hat{v}_{k-1}\Delta t, \quad v_{\text{pred},k} = \hat{v}_{k-1}$$

修正：

$$r_k = z_k - p_{\text{pred},k}$$

$$\hat{p}_k = p_{\text{pred},k} + gr_k, \quad \hat{v}_k = v_{\text{pred},k} + \frac{h}{\Delta t}r_k$$

输出： $\hat{p}_k$, $\hat{v}_k$

数值示例

参数：真实速度 20 m/s, $\Delta t = 1 \text{ s}$, $g = 0.3$, $h = 0.1$, 初始估计 $\hat{p}_0 = 0$, $\hat{v}_0 = 15$ （故意设为错误值）。

t (s)	真实位置	GPS	g-h 估计	真实速度	速度估计
0	0	—	0.0	20	15.00
1	20	18.3	16.0	20	15.33
2	40	43.1	34.9	20	16.51
3	60	56.8	54.3	20	17.68
4	80	82.4	74.8	20	18.73
5	100	97.6	96.0	20	19.60

表 2 g-h 滤波器追踪结果

位置误差逐步收敛，速度估计自动逼近真实值，初始误差通过迭代修正被消除。

递推平均与 g-h 滤波器对比

t (s)	真实位置 (m)	递推平均 (m)	g-h (m)
0	0	1.2	0.0
1	20	9.8	16.0
2	40	20.9	34.9
3	60	29.9	54.3
4	80	40.4	74.8
5	100	49.9	96.0

表 3 同一数据下两种方法的对比

	递推平均	g-h 滤波器
--	------	---------

适用场景	静态量	动态系统
预测机制	无	有（运动模型）
状态数	1	2（位置+速度）
增益	$\frac{1}{n}$ （递减）	g, h （固定）
第 5 秒误差	50.1 m	4.0 m

2.6.3.2. 从标量到矩阵：为高维系统做准备

标量公式在一维场景下足够清晰，但对于高维系统会变得繁琐：

- 一维运动： (p, v) — 2 个状态
- 二维平面： (x, y, v_x, v_y) — 4 个状态
- 三维空间 + 加速度：9 个状态

标量形式需要为每个状态单独写方程，而这些方程结构相同。矩阵形式可将其统一：无论状态维数如何，公式形式保持不变。

状态向量

定义状态向量 $\mathbf{x}_k = \begin{pmatrix} p_k \\ v_k \end{pmatrix}$ 与增益向量 $\mathbf{K} = \begin{pmatrix} g \\ h/\Delta t \end{pmatrix}$ 。两个标量修正公式可合并为：

$$\hat{\mathbf{x}}_k = \mathbf{x}_{\text{pred}} + \mathbf{K} \cdot (z_k - p_{\text{pred}})$$

增益向量的每一行对应一个状态的修正规则。

观测矩阵

GPS 仅测量位置，无法测量速度。需要从状态向量中提取可观测分量，这由观测矩阵 $\mathbf{H}$ 完成：

$$\mathbf{H} = (1 \ 0)$$

其作用为：

$$\mathbf{H}\mathbf{x}_{\text{pred}} = (1 \ 0) \begin{pmatrix} p_{\text{pred}} \\ v_{\text{pred}} \end{pmatrix} = p_{\text{pred}}$$

对于不同传感器配置， $\mathbf{H}$ 的形式不同：

- 仅测位置： $\mathbf{H} = (1 \ 0)$
- 仅测加速度： $\mathbf{H} = (0 \ 0 \ 1)$
- 同时测位置和速度： $\mathbf{H} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

完整的矩阵形式

$$\hat{\mathbf{x}}_k = \mathbf{x}_{\text{pred}} + \mathbf{K}(z_k - \mathbf{H}\mathbf{x}_{\text{pred}})$$

各项含义： $\mathbf{x}_{\text{pred}}$ 为模型预测； $\mathbf{H}\mathbf{x}_{\text{pred}}$ 为预测对应的测量值； $z_k - \mathbf{H}\mathbf{x}_{\text{pred}}$ 为测量残差； $\mathbf{K}$ 控制修正幅度。

验证

展开矩阵公式：

$$\begin{pmatrix} \hat{p}_k \\ \hat{v}_k \end{pmatrix} = \begin{pmatrix} p_{\text{pred}} \\ v_{\text{pred}} \end{pmatrix} + \begin{pmatrix} g \\ h/\Delta t \end{pmatrix} \left(z_k - (1 \ 0) \begin{pmatrix} p_{\text{pred}} \\ v_{\text{pred}} \end{pmatrix} \right)$$

$$= \begin{pmatrix} p_{\text{pred}} + g(z_k - p_{\text{pred}}) \\ v_{\text{pred}} + (h/\Delta t)(z_k - p_{\text{pred}}) \end{pmatrix}$$

与标量形式一致。

预测步骤的矩阵形式

定义状态转移矩阵：

$$\mathbf{F} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}$$

预测步骤为 $\mathbf{x}_{\text{pred}} = \mathbf{F}\hat{\mathbf{x}}_{k-1}$ ，展开后：

$$\begin{pmatrix} p_{\text{pred}} \\ v_{\text{pred}} \end{pmatrix} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \hat{p}_{k-1} \\ \hat{v}_{k-1} \end{pmatrix} = \begin{pmatrix} \hat{p}_{k-1} + \Delta t \hat{v}_{k-1} \\ \hat{v}_{k-1} \end{pmatrix}$$

2.6.3.3. 完整算法（矩阵形式）

定义：

$$\mathbf{x}_k = \begin{pmatrix} p_k \\ v_k \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}, \quad \mathbf{H} = (1 \ 0), \quad \mathbf{K} = \begin{pmatrix} g \\ h/\Delta t \end{pmatrix}$$

预测： $\mathbf{x}_{\text{pred}} = \mathbf{F}\hat{\mathbf{x}}_{k-1}$

修正： $\hat{\mathbf{x}}_k = \mathbf{x}_{\text{pred}} + \mathbf{K}(z_k - \mathbf{H}\mathbf{x}_{\text{pred}})$

2.6.3.4. g-h 滤波器的局限性

g-h 滤波器具备“预测 + 修正”的基本结构，但增益 g 、 h 需手动设定。卡尔曼滤波通过以下机制解决这一问题：

1. 引入误差协方差矩阵 $\mathbf{P}_k$ 量化估计的不确定性
2. 根据最小化估计误差的准则，动态计算最优增益 $\mathbf{K}_k$

下一章将推导这一机制的数学原理。

2.6.4. 最优增益的推导

上一章建立了 g-h 滤波器的基本框架，其核心公式为：

$$\hat{\mathbf{x}}_k = \mathbf{x}_{\text{pred}} + \mathbf{K}(z_k - \mathbf{x}_{\text{pred}})$$

该公式的结构清晰：预测值与测量值的加权融合，增益 $\mathbf{K}$ 控制两者的权重分配。然而， $\mathbf{K}$ 的取值从何而来？在 g-h 滤波器中， g 和 h 由工程师根据经验手动设定，这显然不是最优解。本章将从概率的角度重新审视状态估计问题，推导出增益的最优取值，这正是卡尔曼滤波的核心贡献。

2.6.4.1. 不确定性的量化

在讨论“最优”之前，需要明确优化的目标。回到赛车追踪的场景：预测位置为 35 米，GPS 测量值为 38 米，最终估计应该取多少？

g-h 滤波器的回答是：取决于你设定的 g 值。若 $g = 0.3$ ，则估计为 $35 + 0.3 \times (38 - 35) = 35.9$ 米。但这个答案回避了一个关键问题——凭什么选 $g = 0.3$ ？

更合理的思路是考虑预测和测量各自的可信程度。如果运动模型非常精确(比如赛车在笔直赛道上匀速行驶),预测值应当更可信;如果 GPS 精度很高而模型存在较大不确定性,测量值应当更可信。这种“可信程度”需要用数学语言精确描述,而方差正是刻画不确定性的自然工具。

设预测值 x_{pred} 的误差服从均值为零、方差为 σ_{pred}^2 的分布,测量值 z 的误差服从均值为零、方差为 σ_z^2 的分布。方差越大,对应的值越不可信。现在,优化目标可以明确表述为:选择增益 K ,使得融合后的估计 $\hat{x}$ 具有最小的误差方差。

2.6.4.2. 两个估计的最优融合

为简化推导,先考虑一个更基本的问题:有两个对同一量 x 的独立估计 x_1 和 x_2 ,误差方差分别为 σ_1^2 和 σ_2^2 ,如何将它们融合为一个更精确的估计?

自然的想法是加权平均:

$$\hat{x} = w_1 x_1 + w_2 x_2, \quad w_1 + w_2 = 1$$

设 $w_1 = 1 - K$, $w_2 = K$, 则:

$$\hat{x} = (1 - K)x_1 + Kx_2 = x_1 + K(x_2 - x_1)$$

这与滤波器的修正公式形式完全一致。接下来推导使 $\hat{x}$ 方差最小的 K 值。

设 $x_1 = x + \varepsilon_1$, $x_2 = x + \varepsilon_2$, 其中 ε_1 、 ε_2 为零均值、相互独立的误差项,方差分别为 σ_1^2 、 σ_2^2 。融合估计的误差为:

$$\hat{x} - x = (1 - K)(x + \varepsilon_1) + K(x + \varepsilon_2) - x = (1 - K)\varepsilon_1 + K\varepsilon_2$$

由于 ε_1 与 ε_2 独立,误差方差为:

$$\sigma_{\hat{x}}^2 = (1 - K)^2 \sigma_1^2 + K^2 \sigma_2^2$$

对 K 求导并令其为零:

$$\frac{d\sigma_{\hat{x}}^2}{dK} = -2(1 - K)\sigma_1^2 + 2K\sigma_2^2 = 0$$

解得:

$$K^* = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

这一结果具有直观的解释:最优增益与 x_1 的方差成正比,与 x_2 的方差成反比。换言之,越不可信的估计,其权重越低。当 $\sigma_1^2 \gg \sigma_2^2$ 时, $K^* \approx 1$, 融合结果接近 x_2 ; 当 $\sigma_1^2 \ll \sigma_2^2$ 时, $K^* \approx 0$, 融合结果接近 x_1 。

将最优增益代回方差公式,可得融合后的方差:

$$\sigma_{\hat{x}}^2 = (1 - K^*)\sigma_1^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

注意到 $\sigma_{\hat{x}}^2 < \min(\sigma_1^2, \sigma_2^2)$, 即融合后的估计总是比任一单独估计更精确。这正是信息融合的价值所在。

2.6.4.3. 卡尔曼增益的形式

回到滤波器的语境, x_1 对应预测值 x_{pred} , x_2 对应测量值 z 。将符号替换后,最优增益为:

$$K = \frac{\sigma_{\text{pred}}^2}{\sigma_{\text{pred}}^2 + \sigma_z^2}$$

这就是一维卡尔曼增益的表达式。其行为与直觉一致:

- 若预测不确定性大 (σ_{pred}^2 大), K 接近 1, 更信任测量
- 若测量噪声大 (σ_z^2 大), K 接近 0, 更信任预测
- 两者相当时, $K \approx 0.5$, 各取一半

与 g-h 滤波器的固定增益不同, 卡尔曼增益随不确定性的变化而动态调整。在滤波器运行初期, 状态估计的不确定性通常较大, 此时 K 较大, 测量值对估计的影响显著; 随着观测数据的积累, 估计逐渐收敛, σ_{pred}^2 减小, K 随之下降, 新测量的影响减弱。这种自适应特性是卡尔曼滤波的核心优势。

2.6.4.4. 不确定性的传播

卡尔曼增益的计算依赖于 σ_{pred}^2 , 而 σ_{pred}^2 又从何而来? 这需要追踪不确定性在预测和修正两个阶段的演变。

预测阶段的不确定性增长

考虑匀速运动模型 $x_k = x_{k-1} + v\Delta t$ 。即使上一时刻的位置估计是准确的, 模型本身也存在误差——赛车可能轻微加速或减速, 这种模型不确定性称为过程噪声, 记其方差为 σ_w^2 。

设第 $k-1$ 时刻的估计方差为 σ_{k-1}^2 。由于位置预测直接继承了上一时刻的估计 ($x_{\text{pred}} = \hat{x}_{k-1} + \hat{v}\Delta t$), 预测误差包含两部分: 上一时刻的估计误差和过程噪声。假设两者独立, 预测方差为:

$$\sigma_{\text{pred}}^2 = \sigma_{k-1}^2 + \sigma_w^2$$

这一公式说明, 预测阶段的不确定性总是增加——即使此前的估计很精确, 模型误差也会使不确定性累积。

修正阶段的不确定性减小

当新测量到来后, 融合公式降低了不确定性。由前面的推导, 修正后的方差为:

$$\sigma_k^2 = (1 - K)\sigma_{\text{pred}}^2 = \frac{\sigma_{\text{pred}}^2 \sigma_z^2}{\sigma_{\text{pred}}^2 + \sigma_z^2}$$

由于 $0 < K < 1$, 修正后的方差总是小于预测方差。每次测量都带来新信息, 使估计更加确定。

将增益公式代入, 修正后的方差也可写为:

$$\sigma_k^2 = (1 - K)\sigma_{\text{pred}}^2$$

这一形式在计算上更为简洁, 将在后续的多维推广中使用。

2.6.4.5. 一维卡尔曼滤波算法

综合以上推导, 一维卡尔曼滤波的完整算法如下。设系统状态为位置 x , 状态转移方程为 $x_k = x_{k-1} + u_k + w_k$, 其中 u_k 为已知控制量 (如速度乘以时间), w_k 为过程噪声。测量方程为 $z_k = x_k + v_k$, 其中 v_k 为测量噪声。

初始化

设定初始估计 $\hat{x}_0$ 和初始方差 σ_0^2 。若对初始状态一无所知, 可将 σ_0^2 设为较大值。

预测

$$x_{\text{pred}} = \hat{x}_{k-1} + u_k$$

$$\sigma_{\text{pred}}^2 = \sigma_{k-1}^2 + \sigma_w^2$$

修正

$$K_k = \frac{\sigma_{\text{pred}}^2}{\sigma_{\text{pred}}^2 + \sigma_z^2}$$

$$\hat{x}_k = x_{\text{pred}} + K_k(z_k - x_{\text{pred}})$$

$$\sigma_k^2 = (1 - K_k)\sigma_{\text{pred}}^2$$

算法在预测与修正之间交替进行，每一步都输出状态估计 $\hat{x}_k$ 及其不确定性 σ_k^2 。与 g-h 滤波器相比，核心区别在于增益 K_k 不再是预设常数，而是根据当前不确定性动态计算。

2.6.4.6. 数值示例：重新追踪赛车

沿用上一章的赛车场景，但改用卡尔曼滤波进行估计。参数设定如下：

- 过程噪声标准差： $\sigma_w = 1 \text{ m}$ （模型不完全准确）
- 测量噪声标准差： $\sigma_z = 5 \text{ m}$ （GPS 精度有限）
- 初始估计： $\hat{x}_0 = 0 \text{ m}$
- 初始标准差： $\sigma_0 = 10 \text{ m}$ （对初始位置不确定）
- 控制量： $u_k = 20 \text{ m}$ （真实速度 20 m/s , $\Delta t = 1 \text{ s}$ ）

t (s)	真实	预测	σ_{pred}	测量	K	估计
0	0	—	—	—	—	0.0
1	20	20.0	10.0	18.3	0.80	18.6
2	40	38.6	4.5	43.1	0.45	40.6
3	60	60.6	2.1	56.8	0.15	60.0
4	80	80.0	1.5	82.4	0.08	80.2
5	100	100.2	1.3	97.6	0.06	100.0

表 4 一维卡尔曼滤波追踪赛车

第 1 步计算过程

Note 2.6.4.6.1

预测： $x_{\text{pred}} = 0 + 20 = 20 \text{ m}$

预测方差： $\sigma_{\text{pred}}^2 = 10^2 + 1^2 = 101$, $\sigma_{\text{pred}} \approx 10.0 \text{ m}$

卡尔曼增益： $K = \frac{101}{101+25} = 0.80$

修正： $\hat{x}_1 = 20 + 0.80 \times (18.3 - 20) = 18.6 \text{ m}$

修正方差： $\sigma_1^2 = (1 - 0.80) \times 101 = 20.2$, $\sigma_1 \approx 4.5 \text{ m}$

note 3 第 1 步计算过程

从表中可以观察到几个重要现象。首先，卡尔曼增益 K 从初始的 0.80 逐步下降到 0.06。这是因为随着观测数据的积累，估计的不确定性 σ_{pred} 不断减小，滤波器对新测量的依赖程度降低。其次，预测标准差从 10 m 迅速收敛到约 1.3 m，说明滤波器在几步之内就建立了对状态的可靠估计。最后，第 5 秒的估计误差仅为 0.0 m，远优于 g-h 滤波器的 4.0 m 和递推平均的 50.1 m。

这一改进的根源在于：卡尔曼滤波根据实际的不确定性水平自动调整增益，而非使用固定权重。当估计不确定时（初始阶段），大量采纳测量信息；当估计已经可靠时，对异常测量保持适度怀疑。

2.6.4.7. 从 g-h 滤波器到卡尔曼滤波

至此，可以清晰地看出 g-h 滤波器与卡尔曼滤波的关系。两者共享相同的修正公式：

$$\hat{x}_k = x_{\text{pred}} + K(z_k - x_{\text{pred}})$$

差异在于增益的来源：

	g-h 滤波器	卡尔曼滤波
增益	手动设定常数 g	动态计算 $K_k = \frac{\sigma_{\text{pred}}^2}{\sigma_{\text{pred}}^2 + \sigma_z^2}$
依据	工程经验	最小化估计误差方差
自适应	无	有（增益随不确定性变化）
额外输出	无	估计的不确定性 σ_k^2

卡尔曼滤波可视为 g-h 滤波器的最优化版本：在相同的“预测-修正”框架下，通过概率推理自动确定最优增益。此外，卡尔曼滤波还输出估计的不确定性，这在实际应用中极为重要——它告诉我们估计结果有多可信。

2.6.4.8. 本章小结

本章从优化的角度重新审视了状态估计问题，得到以下核心结论：

状态估计的目标是最小化估计误差的方差。当预测和测量各自携带不确定性时，最优融合策略是根据方差进行加权：方差越大的估计，权重越低。由此导出的卡尔曼增益 $K = \frac{\sigma_{\text{pred}}^2}{\sigma_{\text{pred}}^2 + \sigma_z^2}$ 具有清晰的物理意义——它平衡了模型预测与传感器测量之间的信任程度。

不确定性在预测阶段增长（因为模型不完美），在修正阶段减小（因为测量提供新信息）。卡尔曼滤波的递推结构正是在这两个过程之间交替，持续追踪状态估计及其不确定性。

目前的推导限于一维情形。下一章将把这些概念推广到多维系统，届时方差将被协方差矩阵取代，增益将成为矩阵，但核心思想——基于不确定性的最优融合——保持不变。

2.6.5. 多维卡尔曼滤波

上一章在一维情形下完成了卡尔曼滤波的推导，核心结论是：最优增益由预测方差与测量方差的比值决定，融合后的估计具有最小的误差方差。然而，实际系统往往涉及多个相互关联的状态量。以赛车追踪为例，完整描述其运动状态至少需要位置和速度两个变量；若在二维平面上追踪，则需要 (x, y, v_x, v_y) 四个状态。本章将一维结论推广到多维情形，建立完整的卡尔曼滤波方程组。

2.6.5.1. 为什么需要协方差矩阵

在一维情形下，估计的不确定性用单个方差 σ^2 描述。推广到多维时，一个自然的想法是为每个状态分量分别记录方差。例如，对于状态向量 $\mathbf{x} = \begin{pmatrix} p \\ v \end{pmatrix}$ ，分别记录位置方差 σ_p^2 和速度方差 σ_v^2 。

这种做法忽略了一个重要现象：状态分量之间往往存在相关性。考虑以下场景：若当前位置被高估（真实位置比估计值低），根据 $p = p_0 + vt$ 的关系，很可能速度也被高估了。换言之，位置误差和速度误差并非独立，而是倾向于同向偏离。

协方差正是刻画这种相关性的工具。对于两个随机变量 X 和 Y ，协方差定义为：

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

若 X 和 Y 倾向于同向偏离各自的均值, 协方差为正; 若倾向于反向偏离, 协方差为负; 若两者独立, 协方差为零。方差是协方差的特例: $\text{Var}(X) = \text{Cov}(X, X)$ 。

对于 n 维状态向量 $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$, 所有方差和协方差可组织成一个 $n \times n$ 的对称矩阵:

$$\mathbf{P} = \begin{pmatrix} \text{Var}(x_1) & \text{Cov}(x_1, x_2) & \cdots & \text{Cov}(x_1, x_n) \\ \text{Cov}(x_2, x_1) & \text{Var}(x_2) & \cdots & \text{Cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(x_n, x_1) & \text{Cov}(x_n, x_2) & \cdots & \text{Var}(x_n) \end{pmatrix}$$

这就是协方差矩阵。对角线元素是各状态的方差, 非对角线元素是状态之间的协方差。以位置-速度系统为例:

$$\mathbf{P} = \begin{pmatrix} \sigma_p^2 & \sigma_{pv} \\ \sigma_{pv} & \sigma_v^2 \end{pmatrix}$$

其中 $\sigma_{pv} = \text{Cov}(p, v)$ 描述位置误差与速度误差的相关程度。若 $\sigma_{pv} > 0$, 表示位置被高估时速度也倾向于被高估。

协方差矩阵在卡尔曼滤波中扮演核心角色: 它不仅记录各状态的不确定性大小, 还记录状态之间的误差关联。后者对于提高估计精度至关重要——当测量修正了某个状态时, 与之相关的其他状态也应当相应调整。

2.6.5.2. 多维系统的数学描述

在建立滤波方程之前, 需要明确多维系统的数学模型。

状态转移方程

系统状态从第 $k-1$ 时刻演变到第 k 时刻的规律由状态转移方程描述:

$$\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k$$

其中 $\mathbf{F}$ 是状态转移矩阵, 描述状态的自然演化; $\mathbf{B}$ 是控制矩阵, $\mathbf{u}_k$ 是控制输入; $\mathbf{w}_k$ 是过程噪声, 代表模型的不确定性。

以匀速运动模型为例, 状态为 $\mathbf{x} = \begin{pmatrix} p \\ v \end{pmatrix}$, 状态转移方程为:

$$\begin{pmatrix} p_k \\ v_k \end{pmatrix} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p_{k-1} \\ v_{k-1} \end{pmatrix} + \begin{pmatrix} w_p \\ w_v \end{pmatrix}$$

状态转移矩阵 $\mathbf{F} = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix}$ 的含义是: 新位置 = 旧位置 + 速度 $\times$ 时间, 新速度 = 旧速度 (匀速假设)。

过程噪声 $\mathbf{w}_k$ 通常假设服从零均值高斯分布, 其协方差矩阵记为 $\mathbf{Q}$:

$$\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

$\mathbf{Q}$ 的大小反映模型的可靠程度。若系统严格遵循模型 (如匀速运动), $\mathbf{Q}$ 较小; 若模型仅为粗略近似, $\mathbf{Q}$ 较大。

观测方程

传感器测量与系统状态的关系由观测方程描述:

$$\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$$

其中 $\mathbf{H}$ 是观测矩阵, 将状态空间映射到测量空间; $\mathbf{v}_k$ 是测量噪声, 协方差矩阵记为 $\mathbf{R}$ 。继续位置-速度的例子。若传感器仅能测量位置, 观测方程为:

$$z_k = (1 \ 0) \begin{pmatrix} p_k \\ v_k \end{pmatrix} + v_k$$

观测矩阵 $\mathbf{H} = (1 \ 0)$ 表示从状态向量中提取位置分量。测量噪声方差 $\mathbf{R} = \sigma_z^2$ 反映传感器精度。

2.6.5.3. 预测阶段

卡尔曼滤波的每一步分为预测和修正两个阶段。预测阶段根据系统模型推算下一时刻的状态及其不确定性。

状态预测

状态预测直接应用状态转移方程（忽略噪声项，因为噪声均值为零）：

$$\mathbf{x}_{k|k-1} = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k$$

下标 $k|k-1$ 表示“基于前 $k-1$ 时刻信息对第 k 时刻的预测”。

协方差预测

不确定性的传播稍显复杂。设第 $k-1$ 时刻的估计协方差为 $\mathbf{P}_{k-1}$ ，需要推导预测协方差 $\mathbf{P}_{k|k-1}$ 。

状态预测误差为：

$$\mathbf{x}_k - \mathbf{x}_{k|k-1} = \mathbf{F}(\mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1}) + \mathbf{w}_k$$

等号右边第一项是上一时刻估计误差经 $\mathbf{F}$ 变换的结果，第二项是过程噪声。由于估计误差与过程噪声独立，预测协方差为：

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1}\mathbf{F}^\top + \mathbf{Q}$$

这一公式的含义是：预测不确定性来自两部分——上一时刻的估计不确定性（经状态转移放大或缩小）和模型本身的不确定性（过程噪声）。即使 $\mathbf{P}_{k-1}$ 很小， $\mathbf{Q}$ 的存在也会使 $\mathbf{P}_{k|k-1}$ 增大，这与一维情形中“预测阶段方差增加”的结论一致。

协方差变换公式

Note 2.6.5.3.1

若随机向量 $\mathbf{x}$ 的协方差矩阵为 $\mathbf{P}$ ，则线性变换 $\mathbf{y} = \mathbf{A}\mathbf{x}$ 的协方差矩阵为 $\mathbf{A}\mathbf{P}\mathbf{A}^\top$ 。

证明：设 $\mathbf{x}$ 的均值为 $\boldsymbol{\mu}$ ，则

$$\text{Cov}(\mathbf{y}) = E[(\mathbf{A}\mathbf{x} - \mathbf{A}\boldsymbol{\mu})(\mathbf{A}\mathbf{x} - \mathbf{A}\boldsymbol{\mu})^\top] = \mathbf{A}E[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]\mathbf{A}^\top = \mathbf{A}\mathbf{P}\mathbf{A}^\top$$

note 4 协方差变换公式

2.6.5.4. 修正阶段

当新测量 z_k 到来时，修正阶段将预测与测量融合，得到更精确的估计。

卡尔曼增益

多维卡尔曼增益的推导遵循与一维相同的原则：最小化修正后的估计误差协方差。推导过程涉及矩阵求导，此处直接给出结果：

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^\top(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R})^{-1}$$

这一公式的结构与一维情形对应。分子 $\mathbf{P}_{k|k-1}\mathbf{H}^\top$ 反映预测不确定性在测量空间的投影；分母 $\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^\top + \mathbf{R}$ 是预测测量的不确定性（来自状态预测）与测量本身的不确定性（来自传感

器)之和。当预测不确定性大时, K 较大, 更信任测量; 当测量不确定性大时, K 较小, 更信任预测。

状态修正

修正公式与 g-h 滤波器形式相同:

$$\hat{x}_k = x_{k|k-1} + K_k(z_k - Hx_{k|k-1})$$

括号内的 $z_k - Hx_{k|k-1}$ 称为测量残差或新息 (Innovation), 表示实际测量与预测测量之间的偏差。卡尔曼增益决定了这一偏差如何分配到各个状态分量上。

协方差修正

修正后的协方差为:

$$P_k = (I - K_k H) P_{k|k-1}$$

其中 I 为单位矩阵。这一公式表明, 测量总是减小不确定性—— $I - K_k H$ 的特征值均小于 1, 因此 P_k 在各方向上都不大于 $P_{k|k-1}$ 。

2.6.5.5. 完整算法

综合预测和修正阶段, 多维卡尔曼滤波的完整算法如下。

系统模型

状态转移: $x_k = Fx_{k-1} + Bu_k + w_k$, $w_k \sim \mathcal{N}(0, Q)$

观测方程: $z_k = Hx_k + v_k$, $v_k \sim \mathcal{N}(0, R)$

初始化

初始状态估计 $\hat{x}_0$, 初始协方差 P_0

预测

$$x_{k|k-1} = F\hat{x}_{k-1} + Bu_k$$

$$P_{k|k-1} = FP_{k-1}F^\top + Q$$

修正

$$K_k = P_{k|k-1}H^\top(HP_{k|k-1}H^\top + R)^{-1}$$

$$\hat{x}_k = x_{k|k-1} + K_k(z_k - Hx_{k|k-1})$$

$$P_k = (I - K_k H) P_{k|k-1}$$

算法在预测与修正之间交替进行, 每一步输出状态估计 $\hat{x}_k$ 及其协方差 P_k 。协方差矩阵不仅用于计算下一步的卡尔曼增益, 还直接提供了估计的置信区间。

2.6.5.6. 示例: 一维位置-速度系统

为了具体展示多维卡尔曼滤波的运算过程, 回到赛车追踪问题。此前我们假设速度已知 (作为控制输入), 现在将速度也作为待估计的状态, 考察滤波器能否从位置测量中同时恢复位置和速度。

系统设定

状态向量 $x = \begin{pmatrix} p \\ v \end{pmatrix}$, 真实初始状态为 $\begin{pmatrix} 0 \\ 20 \end{pmatrix}$ (位置 0 m, 速度 20 m/s)。

状态转移矩阵（匀速模型， $\Delta t = 1$ s）：

$$\mathbf{F} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

观测矩阵（仅测量位置）：

$$\mathbf{H} = (1 \ 0)$$

过程噪声协方差（速度可能有小幅波动）：

$$\mathbf{Q} = \begin{pmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{pmatrix}$$

测量噪声方差：

$$\mathbf{R} = (25) \quad (\sigma_z = 5 \text{ m})$$

初始估计（故意设定错误的速度）：

$$\hat{\mathbf{x}}_0 = \begin{pmatrix} 0 \\ 15 \end{pmatrix}, \quad \mathbf{P}_0 = \begin{pmatrix} 100 & 0 \\ 0 & 100 \end{pmatrix}$$

第 1 步迭代

预测：

$$\mathbf{x}_{1|0} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 15 \end{pmatrix} = \begin{pmatrix} 15 \\ 15 \end{pmatrix}$$

$$\mathbf{P}_{1|0} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 100 & 0 \\ 0 & 100 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} + \begin{pmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{pmatrix} = \begin{pmatrix} 200.25 & 100.5 \\ 100.5 & 101 \end{pmatrix}$$

卡尔曼增益：

$$\mathbf{H}\mathbf{P}_{1|0}\mathbf{H}^\top + \mathbf{R} = 200.25 + 25 = 225.25$$

$$\mathbf{K}_1 = \begin{pmatrix} 200.25 \\ 100.5 \end{pmatrix} \times \frac{1}{225.25} = \begin{pmatrix} 0.889 \\ 0.446 \end{pmatrix}$$

测量 $z_1 = 18.3$ m，修正：

$$\hat{\mathbf{x}}_1 = \begin{pmatrix} 15 \\ 15 \end{pmatrix} + \begin{pmatrix} 0.889 \\ 0.446 \end{pmatrix} \times (18.3 - 15) = \begin{pmatrix} 17.9 \\ 16.5 \end{pmatrix}$$

$$\mathbf{P}_1 = \left(\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - \begin{pmatrix} 0.889 \\ 0.446 \end{pmatrix} (1 \ 0) \right) \begin{pmatrix} 200.25 & 100.5 \\ 100.5 & 101 \end{pmatrix} = \begin{pmatrix} 22.2 & 11.2 \\ 11.2 & 56.2 \end{pmatrix}$$

完整追踪结果

t	真实位置	真实速度	测量	位置估计	速度估计	σ_p
0	0	20	—	0.0	15.0	10.0
1	20	20	18.3	17.9	16.5	4.7
2	40	20	43.1	39.8	18.7	3.5
3	60	20	56.8	59.2	19.4	3.0
4	80	20	82.4	80.0	19.8	2.7
5	100	20	97.6	99.7	19.9	2.5

表 5 多维卡尔曼滤波追踪结果

从结果中可以看到，尽管传感器只能测量位置，滤波器却成功地同时估计了位置和速度。速度估计从初始的 15 m/s 逐步收敛到接近真实值 20 m/s，这正是协方差矩阵发挥作用的结果——位置测量残差通过 σ_{pv} 的关联传递到速度估计上。

位置估计的标准差从 10 m 下降到 2.5 m，表明不确定性持续减小。若继续迭代，标准差将收敛到一个稳态值，由过程噪声 Q 和测量噪声 R 的相对大小决定。

2.6.5.7. 矩阵维度总结

多维卡尔曼滤波涉及多个矩阵，初学者容易混淆其维度。设状态向量维度为 n ，测量向量维度为 m ，控制向量维度为 l ，各矩阵的维度如下：

符号	维度	含义
x	$n \times 1$	状态向量
z	$m \times 1$	测量向量
u	$l \times 1$	控制向量
F	$n \times n$	状态转移矩阵
B	$n \times l$	控制矩阵
H	$m \times n$	观测矩阵
Q	$n \times n$	过程噪声协方差
R	$m \times m$	测量噪声协方差
P	$n \times n$	状态估计协方差
K	$n \times m$	卡尔曼增益

卡尔曼增益 K 的维度 $n \times m$ 值得注意：它将 m 维的测量残差映射到 n 维的状态修正。即使测量维度低于状态维度（如本例中 $m = 1, n = 2$ ），滤波器仍能通过协方差的关联信息更新所有状态。

2.6.5.8. 本章小结

本章将一维卡尔曼滤波推广到多维情形。核心概念的对应关系如下：

	一维	多维
状态	x	x （向量）
不确定性	σ^2 （方差）	P （协方差矩阵）
增益	$K = \frac{\sigma_{\text{pred}}^2}{\sigma_{\text{pred}}^2 + \sigma_z^2}$	$K = P_{k k-1} H^\top (H P_{k k-1} H^\top + R)^{-1}$
预测方差	$\sigma_{\text{pred}}^2 = \sigma_{k-1}^2 + \sigma_w^2$	$P_{k k-1} = F P_{k-1} F^\top + Q$
修正方差	$\sigma_k^2 = (1 - K) \sigma_{\text{pred}}^2$	$P_k = (I - K H) P_{k k-1}$

多维形式的本质与一维相同：预测阶段不确定性增加，修正阶段不确定性减小，卡尔曼增益平衡预测与测量的信任程度。协方差矩阵的引入使滤波器能够利用状态之间的关联信息，即使某些状态不能直接测量（如速度），也可以通过与可测状态（如位置）的相关性间接估计。

至此，线性卡尔曼滤波的理论框架已经完整。然而，上述推导依赖一个关键假设：状态转移和观测方程都是线性的。当系统存在非线性时——例如雷达以极坐标测量目标位置——标准卡尔曼滤波不再适用。下一章将介绍扩展卡尔曼滤波（EKF），它通过局部线性化处理非线性系统。

2.6.6. 扩展卡尔曼滤波

前两章建立的卡尔曼滤波理论依赖于线性假设：状态转移方程 $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1}$ 和观测方程 $z_k = \mathbf{H}\mathbf{x}_k$ 都是状态的线性函数。这一假设在许多场景下是合理的——匀速运动、弹簧振子、电路系统等都可以用线性方程描述。然而，现实中大量系统本质上是非线性的。本章将分析线性假设的局限性，并介绍扩展卡尔曼滤波（Extended Kalman Filter, EKF）如何通过局部线性化处理非线性系统。

2.6.6.1. 非线性系统的例子

考虑一个典型的雷达追踪问题。雷达站位于原点，追踪一架在二维平面上飞行的飞机。飞机的状态用直角坐标描述：

$$\mathbf{x} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}$$

其中 (x, y) 是位置， (v_x, v_y) 是速度。假设飞机做匀速直线运动，状态转移方程为：

$$\begin{pmatrix} x_k \\ y_k \\ v_{x,k} \\ v_{y,k} \end{pmatrix} = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_{k-1} \\ y_{k-1} \\ v_{x,k-1} \\ v_{y,k-1} \end{pmatrix}$$

这是标准的线性方程，没有问题。困难出在观测方程上。

雷达并不直接测量飞机的直角坐标 (x, y) ，而是测量极坐标：距离 r 和方位角 θ 。测量值与状态的关系为：

$$r = \sqrt{x^2 + y^2}$$

$$\theta = \arctan\left(\frac{y}{x}\right)$$

或写成向量形式：

$$\mathbf{z} = \begin{pmatrix} r \\ \theta \end{pmatrix} = \begin{pmatrix} \sqrt{x^2 + y^2} \\ \arctan(y/x) \end{pmatrix}$$

这是一个非线性观测方程——测量值不是状态的线性组合，无法写成 $\mathbf{z} = \mathbf{H}\mathbf{x}$ 的形式。平方根和反正切函数破坏了线性结构。

非线性带来的问题是什么？回顾卡尔曼滤波的核心假设：状态估计的不确定性用高斯分布描述，而高斯分布经过线性变换后仍然是高斯分布。这一性质保证了预测和修正阶段的协方差计算是精确的。

但高斯分布经过非线性变换后通常不再是高斯分布。考虑一个简单的例子：若 $x \sim \mathcal{N}(0, 1)$ ，则 $y = x^2$ 的分布是卡方分布，不是高斯分布。这意味着，当观测方程非线性时，直接套用卡尔曼滤波公式会产生系统性误差。

2.6.6.2. 线性化的思想

扩展卡尔曼滤波的核心思想是：既然无法处理非线性函数，就用线性函数来近似它。具体方法是在当前估计点对非线性函数进行泰勒展开，保留一阶项。

设非线性函数为 $\mathbf{h}(\mathbf{x})$ ，在点 $\mathbf{x}_0$ 处展开：

$$\mathbf{h}(\mathbf{x}) \approx \mathbf{h}(\mathbf{x}_0) + \frac{\partial \mathbf{h}}{\partial \mathbf{x}}|_{\mathbf{x}_0} (\mathbf{x} - \mathbf{x}_0)$$

一阶导数 $\frac{\partial \mathbf{h}}{\partial \mathbf{x}}$ 是一个矩阵，称为雅可比矩阵 (Jacobian)，记为 $\mathbf{J}$ 。对于 m 维输出、 n 维输入的函数，雅可比矩阵的维度是 $m \times n$ ，第 (i, j) 元素为 $\frac{\partial h_i}{\partial x_j}$ 。

以雷达观测为例，观测函数 $\mathbf{h}(\mathbf{x}) = \begin{pmatrix} \sqrt{x^2+y^2} \\ \arctan(y/x) \end{pmatrix}$ 对状态 $\mathbf{x} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}$ 的雅可比矩阵为：

$$\mathbf{H}_k = \frac{\partial \mathbf{h}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial r}{\partial x} & \frac{\partial r}{\partial y} & \frac{\partial r}{\partial v_x} & \frac{\partial r}{\partial v_y} \\ \frac{\partial \theta}{\partial x} & \frac{\partial \theta}{\partial y} & \frac{\partial \theta}{\partial v_x} & \frac{\partial \theta}{\partial v_y} \end{pmatrix}$$

由于 r 和 θ 都不依赖于速度 (v_x, v_y) ，后两列为零。计算非零偏导数：

$$\frac{\partial r}{\partial x} = \frac{x}{\sqrt{x^2+y^2}} = \frac{x}{r}, \quad \frac{\partial r}{\partial y} = \frac{y}{r}$$

$$\frac{\partial \theta}{\partial x} = \frac{-y/x^2}{1+(y/x)^2} = \frac{-y}{x^2+y^2} = \frac{-y}{r^2}, \quad \frac{\partial \theta}{\partial y} = \frac{1/x}{1+(y/x)^2} = \frac{x}{r^2}$$

因此，雅可比矩阵为：

$$\mathbf{H}_k = \begin{pmatrix} x/r & y/r & 0 & 0 \\ -y/r^2 & x/r^2 & 0 & 0 \end{pmatrix}$$

这个矩阵在当前状态估计 $(\hat{x}, \hat{y})$ 处计算，因此是时变的——每一步都需要重新计算。这是 EKF 与标准卡尔曼滤波的关键区别：标准卡尔曼滤波中 $\mathbf{H}$ 是常数，EKF 中 $\mathbf{H}_k$ 随估计值变化。

2.6.6.3. 扩展卡尔曼滤波算法

将线性化思想应用到卡尔曼滤波的各个阶段，得到 EKF 算法。设非线性状态转移方程为 $\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k$ ，非线性观测方程为 $\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k$ 。

预测阶段

状态预测使用非线性函数：

$$\mathbf{x}_{k|k-1} = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$$

协方差预测需要状态转移函数的雅可比矩阵 $\mathbf{F}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}|_{\hat{\mathbf{x}}_{k-1}}$ ：

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}$$

注意状态预测直接用非线性函数计算（保持精度），而协方差传播用线性化后的雅可比矩阵（便于计算）。

修正阶段

卡尔曼增益使用观测函数的雅可比矩阵 $\mathbf{H}_k = \frac{\partial \mathbf{h}}{\partial \mathbf{x}}|_{\mathbf{x}_{k|k-1}}$ ：

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R})^{-1}$$

状态修正使用非线性观测函数计算预测测量：

$$\hat{\mathbf{x}}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{h}(\mathbf{x}_{k|k-1}))$$

协方差修正：

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}$$

整个算法的流程与标准卡尔曼滤波相同，核心区别有两点：第一，状态预测和测量预测使用原始的非线性函数；第二，协方差传播和增益计算使用在当前估计点计算的雅可比矩阵。

EKF 算法

系统模型

状态转移： $\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k$

观测方程： $\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k$

预测

$$\mathbf{x}_{k|k-1} = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$$

$$\mathbf{F}_k = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}}$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}$$

修正

$$\mathbf{H}_k = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{x}_{k|k-1}}$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R})^{-1}$$

$$\hat{\mathbf{x}}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{h}(\mathbf{x}_{k|k-1}))$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}$$

2.6.6.4. 示例：雷达目标追踪

现在用 EKF 解决前面提出的雷达追踪问题。

系统设定

状态向量 $\mathbf{x} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}$ ，采样间隔 $\Delta t = 1$ s。

状态转移函数（匀速运动，线性）：

$$\mathbf{f}(\mathbf{x}) = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \mathbf{x}$$

由于状态转移是线性的，雅可比矩阵 $\mathbf{F}$ 就是系数矩阵本身，且为常数。

观测函数（非线性）：

$$\mathbf{h}(\mathbf{x}) = \begin{pmatrix} \sqrt{x^2 + y^2} \\ \arctan(y/x) \end{pmatrix}$$

观测雅可比矩阵（在预测状态 (x, y) 处计算）：

$$\mathbf{H}_k = \begin{pmatrix} x/r & y/r & 0 & 0 \\ -y/r^2 & x/r^2 & 0 & 0 \end{pmatrix}, \quad r = \sqrt{x^2 + y^2}$$

过程噪声协方差:

$$\mathbf{Q} = \begin{pmatrix} 0.25 & 0 & 0.5 & 0 \\ 0 & 0.25 & 0 & 0.5 \\ 0.5 & 0 & 1 & 0 \\ 0 & 0.5 & 0 & 1 \end{pmatrix}$$

测量噪声协方差 (距离标准差 5 m, 角度标准差 0.01 rad):

$$\mathbf{R} = \begin{pmatrix} 25 & 0 \\ 0 & 0.0001 \end{pmatrix}$$

真实轨迹与测量

设飞机从 (100, 200) m 出发, 以 (10, 5) m/s 的速度飞行。真实轨迹和带噪声的雷达测量如下:

t	真实 x	真实 y	真实 r	真实 θ	测量 r	测量 θ
0	100	200	223.6	1.107	—	—
1	110	205	232.7	1.078	229.3	1.082
2	120	210	241.9	1.052	245.1	1.048
3	130	215	251.2	1.027	248.7	1.031
4	140	220	260.6	1.004	263.2	1.001
5	150	225	270.0	0.983	267.8	0.979

表 6 雷达追踪: 真实轨迹与测量值 (角度单位: rad)

第 1 步迭代详解

初始估计 (故意设定误差):

$$\hat{\mathbf{x}}_0 = \begin{pmatrix} 95 \\ 190 \\ 8 \\ 6 \end{pmatrix}, \quad \mathbf{P}_0 = \begin{pmatrix} 100 & 0 & 0 & 0 \\ 0 & 100 & 0 & 0 \\ 0 & 0 & 25 & 0 \\ 0 & 0 & 0 & 25 \end{pmatrix}$$

预测:

$$\mathbf{x}_{1|0} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 95 \\ 190 \\ 8 \\ 6 \end{pmatrix} = \begin{pmatrix} 103 \\ 196 \\ 8 \\ 6 \end{pmatrix}$$

预测状态对应的极坐标:

$$r_{\text{pred}} = \sqrt{103^2 + 196^2} = 221.4 \text{ m}$$

$$\theta_{\text{pred}} = \arctan(196/103) = 1.087 \text{ rad}$$

计算观测雅可比矩阵 (在预测状态处):

$$\mathbf{H}_1 = \begin{pmatrix} 103/221.4 & 196/221.4 & 0 & 0 \\ -196/221.4^2 & 103/221.4^2 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0.465 & 0.885 & 0 & 0 \\ -0.00400 & 0.00210 & 0 & 0 \end{pmatrix}$$

协方差预测 ($\mathbf{F}$ 为常数矩阵):

$$P_{1|0} = F P_0 F^T + Q$$

由于矩阵较大，此处省略具体数值，直接给出结果： $P_{1|0}$ 的对角元素约为 (225, 225, 26, 26)。
卡尔曼增益：

$$S_1 = H_1 P_{1|0} H_1^T + R$$

$$K_1 = P_{1|0} H_1^T S_1^{-1}$$

测量残差：

$$z_1 - h(x_{1|0}) = \begin{pmatrix} 229.3 \\ 1.082 \end{pmatrix} - \begin{pmatrix} 221.4 \\ 1.087 \end{pmatrix} = \begin{pmatrix} 7.9 \\ -0.005 \end{pmatrix}$$

状态修正：

$$\hat{x}_1 = \begin{pmatrix} 103 \\ 196 \\ 8 \\ 6 \end{pmatrix} + K_1 \begin{pmatrix} 7.9 \\ -0.005 \end{pmatrix}$$

完整追踪结果

t	真实 x	估计 x	真实 y	估计 y	真实 v_x	估计 v_x
0	100	95.0	200	190.0	10	8.0
1	110	107.2	205	202.1	10	9.1
2	120	118.5	210	209.4	10	9.8
3	130	129.1	215	214.8	10	10.0
4	140	139.8	220	219.9	10	10.1
5	150	150.1	225	224.8	10	10.0

表 7 EKF 追踪结果（部分状态）

尽管初始估计存在明显误差（位置误差约 12 m，速度误差 2-3 m/s），EKF 在几步之内就收敛到接近真实值。这说明滤波器能够有效处理非线性观测方程。

2.6.6.5. 线性化误差与 EKF 的局限性

EKF 通过一阶泰勒展开将非线性函数近似为线性，这种近似在函数“足够接近线性”时效果良好，但在强非线性情形下会产生显著误差。

考虑一个极端例子：观测函数 $h(x) = x^2$ 在 $x_0 = 0$ 处的线性化结果是 $h(x) \approx 0$ （因为导数为零），完全丢失了原函数的信息。虽然实际系统很少如此极端，但当非线性较强或估计误差较大时，线性化误差会累积，导致滤波器性能下降甚至发散。

EKF 的另一个实践困难是雅可比矩阵的计算。对于简单系统（如本章的雷达追踪），解析求导尚可接受；但对于复杂系统，手工推导雅可比矩阵既繁琐又容易出错。虽然可以用数值差分近似，但这会引入额外的计算误差。

此外，EKF 假设线性化后的系统仍然满足高斯分布，这在理论上并不成立。非线性变换会使高斯分布变形，而 EKF 强行用高斯分布近似，相当于丢弃了分布的高阶矩信息。

这些局限促使研究者寻找更好的非线性滤波方法。下一章将介绍无迹卡尔曼滤波(UKF)，它采用完全不同的思路：与其线性化函数，不如用采样点直接“探测”非线性变换的效果。

2.6.6.6. 本章小结

扩展卡尔曼滤波通过局部线性化将标准卡尔曼滤波推广到非线性系统。其核心思想是在当前估计点对非线性函数进行一阶泰勒展开，用雅可比矩阵替代原来的常数矩阵。

与标准卡尔曼滤波相比，EKF 的主要变化是：

	标准 KF	EKF
状态转移	F （常数矩阵）	$f(\cdot)$ （非线性函数）+ F_k （雅可比）
观测方程	H （常数矩阵）	$h(\cdot)$ （非线性函数）+ H_k （雅可比）
协方差传播	精确	近似（一阶）
计算复杂度	较低	较高（需计算雅可比）

EKF 的适用条件是非线性程度适中且估计误差较小，使得线性化近似足够准确。当这些条件不满足时，需要考虑更高级的方法，如无迹卡尔曼滤波。

2.6.7. 无迹卡尔曼滤波

上一章介绍的扩展卡尔曼滤波通过线性化处理非线性系统，其本质是用简单函数近似复杂函数。这种方法在弱非线性情形下表现良好，但当非线性较强时，一阶近似的误差会显著影响滤波性能。本章介绍一种完全不同的思路——无迹卡尔曼滤波（Unscented Kalman Filter, UKF），它不再尝试近似非线性函数本身，而是通过精心选取的采样点直接捕捉概率分布经过非线性变换后的统计特性。

2.6.7.1. 从函数近似到分布近似

EKF 的困难根源在于：非线性函数会扭曲概率分布的形状。若输入是高斯分布，经过非线性变换后，输出通常不再是高斯分布。EKF 的策略是线性化函数，使得输出仍为高斯分布，但这是以牺牲精度为代价的。

UKF 的创始人 Jeffrey Uhlmann 提出了一个关键洞察：近似概率分布比近似非线性函数更容易。具体而言，与其用线性函数近似 $h(x)$ ，不如直接用一组精心选取的点来代表输入分布，将这些点通过真实的非线性函数变换，再从变换后的点恢复输出分布的统计特性。

这一思想可以用一个简单的例子说明。设 $x \sim \mathcal{N}(0, 1)$ ，考虑非线性变换 $y = x^2$ 。EKF 在 $x = 0$ 处线性化，得到 $y \approx 0$ （因为导数为零），完全丢失了信息。而 UKF 的做法是选取几个代表点，比如 $x \in \{-1, 0, 1\}$ ，计算它们的变换值 $y \in \{1, 0, 1\}$ ，再从这些点估计 y 的均值和方差。即使不知道 y 的精确分布，也能得到比 EKF 更准确的统计量估计。

这就是无迹变换（Unscented Transform）的基本思想：用确定性采样点代替随机采样，以较少的计算量捕捉非线性变换的效果。

2.6.7.2. 无迹变换

无迹变换是 UKF 的核心组件。给定一个 n 维随机变量 x ，其均值为 μ ，协方差为 P ，目标是计算经过非线性函数 $y = f(x)$ 变换后 y 的均值和协方差。

Sigma 点的选取

无迹变换选取 $2n + 1$ 个确定性采样点，称为 Sigma 点：

$$\begin{aligned}\chi_0 &= \mu \\ \chi_i &= \mu + \left(\sqrt{(n + \lambda)P}\right)_i, \quad i = 1, \cdots, n\end{aligned}$$

$$\chi_{i+n} = \boldsymbol{\mu} - \left(\sqrt{(n+\lambda)\mathbf{P}} \right)_i, \quad i = 1, \dots, n$$

其中 $\left(\sqrt{(n+\lambda)\mathbf{P}} \right)_i$ 表示矩阵平方根的第 i 列, $\lambda = \alpha^2(n + \kappa) - n$ 是缩放参数。 α 控制 Sigma 点的分散程度, 通常取较小的正数 (如 10^{-3}); κ 通常取 0 或 $3 - n$ 。

矩阵平方根 $\sqrt{\mathbf{P}}$ 定义为满足 $\sqrt{\mathbf{P}}(\sqrt{\mathbf{P}})^\top = \mathbf{P}$ 的矩阵, 通常用 Cholesky 分解计算。

Sigma 点的几何含义是: χ_0 位于分布中心, 其余 $2n$ 个点沿协方差矩阵的主轴对称分布, 距离中心的远近由 λ 控制。这组点能够精确匹配原分布的均值和协方差。

权重分配

每个 Sigma 点被赋予权重, 用于计算变换后的统计量。均值权重和协方差权重分别为:

$$W_0^m = \frac{\lambda}{n + \lambda}$$

$$W_0^c = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta)$$

$$W_i^m = W_i^c = \frac{1}{2(n + \lambda)}, \quad i = 1, \dots, 2n$$

参数 β 用于融入分布的先验知识, 对于高斯分布, $\beta = 2$ 是最优选择。

变换与恢复

将所有 Sigma 点通过非线性函数变换:

$$\mathcal{Y}_i = \mathbf{f}(\chi_i), \quad i = 0, 1, \dots, 2n$$

变换后分布的均值和协方差估计为:

$$\boldsymbol{\mu}_y = \sum_{i=0}^{2n} W_i^m \mathcal{Y}_i$$

$$\mathbf{P}_y = \sum_{i=0}^{2n} W_i^c (\mathcal{Y}_i - \boldsymbol{\mu}_y)(\mathcal{Y}_i - \boldsymbol{\mu}_y)^\top$$

这一过程完全绕过了函数的线性化, 直接从采样点的变换结果恢复统计量。由于 Sigma 点的选取能够精确匹配原分布的前两阶矩, 对于高斯输入, 无迹变换能够准确捕捉输出的均值和协方差, 精度达到二阶 (泰勒展开的二阶项), 优于 EKF 的一阶精度。

2.6.7.3. 无迹卡尔曼滤波算法

将无迹变换应用于卡尔曼滤波的预测和修正阶段, 即得到 UKF 算法。

预测阶段

首先, 根据上一时刻的状态估计 $\hat{\mathbf{x}}_{k-1}$ 和协方差 $\mathbf{P}_{k-1}$ 生成 Sigma 点:

$$\chi_{k-1}^{(i)}, \quad i = 0, 1, \dots, 2n$$

将每个 Sigma 点通过状态转移函数:

$$\chi_{k|k-1}^{(i)} = \mathbf{f}(\chi_{k-1}^{(i)}, \mathbf{u}_k)$$

从变换后的 Sigma 点计算预测均值和协方差:

$$\mathbf{x}_{k|k-1} = \sum_{i=0}^{2n} W_i^m \chi_{k|k-1}^{(i)}$$

$$\mathbf{P}_{k|k-1} = \sum_{i=0}^{2n} W_i^c \left(\chi_{k|k-1}^{(i)} - \mathbf{x}_{k|k-1} \right) \left(\chi_{k|k-1}^{(i)} - \mathbf{x}_{k|k-1} \right)^\top + \mathbf{Q}$$

过程噪声协方差 $\mathbf{Q}$ 在最后加入，与 EKF 的处理方式相同。

修正阶段

将预测 Sigma 点通过观测函数：

$$\mathcal{Z}_k^{(i)} = \mathbf{h} \left(\chi_{k|k-1}^{(i)} \right)$$

计算预测测量的均值：

$$\mathbf{z}_{k|k-1} = \sum_{i=0}^{2n} W_i^m \mathcal{Z}_k^{(i)}$$

计算测量预测的协方差和状态-测量的互协方差：

$$\mathbf{P}_{zz} = \sum_{i=0}^{2n} W_i^c \left(\mathcal{Z}_k^{(i)} - \mathbf{z}_{k|k-1} \right) \left(\mathcal{Z}_k^{(i)} - \mathbf{z}_{k|k-1} \right)^\top + \mathbf{R}$$

$$\mathbf{P}_{xz} = \sum_{i=0}^{2n} W_i^c \left(\chi_{k|k-1}^{(i)} - \mathbf{x}_{k|k-1} \right) \left(\mathcal{Z}_k^{(i)} - \mathbf{z}_{k|k-1} \right)^\top$$

卡尔曼增益：

$$\mathbf{K}_k = \mathbf{P}_{xz} \mathbf{P}_{zz}^{-1}$$

状态和协方差修正：

$$\hat{\mathbf{x}}_k = \mathbf{x}_{k|k-1} + \mathbf{K}_k \left(\mathbf{z}_k - \mathbf{z}_{k|k-1} \right)$$

$$\mathbf{P}_k = \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{P}_{zz} \mathbf{K}_k^\top$$

UKF 算法

生成 Sigma 点

$$\chi_0 = \hat{\mathbf{x}}_{k-1}, \quad \chi_i = \hat{\mathbf{x}}_{k-1} \pm \left(\sqrt{(n+\lambda) \mathbf{P}_{k-1}} \right)_i$$

预测

$$\chi_{k|k-1}^{(i)} = \mathbf{f} \left(\chi_{k-1}^{(i)} \right), \quad i = 0, \dots, 2n$$

$$\mathbf{x}_{k|k-1} = \sum W_i^m \chi_{k|k-1}^{(i)}$$

$$\mathbf{P}_{k|k-1} = \sum W_i^c \left(\chi_{k|k-1}^{(i)} - \mathbf{x}_{k|k-1} \right) \left(\cdot \right)^\top + \mathbf{Q}$$

修正

$$\mathcal{Z}_k^{(i)} = \mathbf{h} \left(\chi_{k|k-1}^{(i)} \right)$$

$$\mathbf{z}_{k|k-1} = \sum W_i^m \mathcal{Z}_k^{(i)}$$

$$\mathbf{P}_{zz} = \sum W_i^c \left(\mathcal{Z}_k^{(i)} - \mathbf{z}_{k|k-1} \right) \left(\cdot \right)^\top + \mathbf{R}$$

$$\begin{aligned}
P_{xz} &= \sum W_i^c \left(\chi_{k|k-1}^{(i)} - x_{k|k-1} \right) \left(z_k^{(i)} - z_{k|k-1} \right)^\top \\
K_k &= P_{xz} P_{zz}^{-1} \\
\hat{x}_k &= x_{k|k-1} + K_k (z_k - z_{k|k-1}) \\
P_k &= P_{k|k-1} - K_k P_{zz} K_k^\top
\end{aligned}$$

2.6.7.4. 示例：雷达追踪问题的 UKF 实现

沿用上一章的雷达追踪场景，用 UKF 重新求解并与 EKF 对比。

系统设定

状态向量 $x = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}$ ($n = 4$)，因此需要 $2n + 1 = 9$ 个 Sigma 点。

参数选取： $\alpha = 0.001$ ， $\kappa = 0$ ， $\beta = 2$ ，则 $\lambda = \alpha^2(n + \kappa) - n \approx -4$ 。

其他参数 (Q 、 R 、初始条件) 与上一章相同。

第 1 步迭代

初始估计 $\hat{x}_0 = \begin{pmatrix} 95 \\ 190 \\ 8 \\ 6 \end{pmatrix}$ ，初始协方差 $P_0 = \text{diag}(100, 100, 25, 25)$ 。

生成 Sigma 点。首先计算 $(n + \lambda)P_0 \approx 0.000004 \times P_0$ (由于 $\lambda \approx -4$ ， $n + \lambda \approx 0.000004$)。实际计算中 α 通常取较大值如 0.1 以避免数值问题，此处为说明原理采用典型参数。设 $\alpha = 0.1$ ，则 $\lambda = 0.01 \times 4 - 4 = -3.96$ ， $n + \lambda = 0.04$ 。

矩阵平方根 $\sqrt{0.04 \times P_0} = 0.2 \times \text{diag}(10, 10, 5, 5) = \text{diag}(2, 2, 1, 1)$ 。

9 个 Sigma 点为：

$$\begin{aligned}
\chi_0 &= \begin{pmatrix} 95 \\ 190 \\ 8 \\ 6 \end{pmatrix} \\
\chi_1 &= \begin{pmatrix} 97 \\ 190 \\ 8 \\ 6 \end{pmatrix}, \quad \chi_5 = \begin{pmatrix} 93 \\ 190 \\ 8 \\ 6 \end{pmatrix} \\
\chi_2 &= \begin{pmatrix} 95 \\ 192 \\ 8 \\ 6 \end{pmatrix}, \quad \chi_6 = \begin{pmatrix} 95 \\ 188 \\ 8 \\ 6 \end{pmatrix} \\
\chi_3 &= \begin{pmatrix} 95 \\ 190 \\ 9 \\ 6 \end{pmatrix}, \quad \chi_7 = \begin{pmatrix} 95 \\ 190 \\ 7 \\ 6 \end{pmatrix}
\end{aligned}$$

$$\chi_4 = \begin{pmatrix} 95 \\ 190 \\ 8 \\ 7 \end{pmatrix}, \quad \chi_8 = \begin{pmatrix} 95 \\ 190 \\ 8 \\ 5 \end{pmatrix}$$

将每个 Sigma 点通过状态转移函数（匀速运动）：

$$\chi_{1|0}^{(i)} = \mathbf{F}\chi_0^{(i)}$$

例如 $\chi_{1|0}^{(0)} = \begin{pmatrix} 95+8 \\ 190+6 \\ 8 \\ 6 \end{pmatrix} = \begin{pmatrix} 103 \\ 196 \\ 8 \\ 6 \end{pmatrix}。$

计算预测均值（各 Sigma 点变换结果的加权平均）：

$$\mathbf{x}_{1|0} = \sum_{i=0}^8 W_i^m \chi_{1|0}^{(i)}$$

由于状态转移是线性的，预测均值与 EKF 相同： $\mathbf{x}_{1|0} = \begin{pmatrix} 103 \\ 196 \\ 8 \\ 6 \end{pmatrix}。$

将预测 Sigma 点通过观测函数：

$$\mathcal{Z}^{(i)} = \begin{pmatrix} \sqrt{x_i^2 + y_i^2} \\ \arctan(y_i/x_i) \end{pmatrix}$$

例如 $\mathcal{Z}^{(0)} = \begin{pmatrix} \sqrt{103^2 + 196^2} \\ \arctan(196/103) \end{pmatrix} = \begin{pmatrix} 221.4 \\ 1.087 \end{pmatrix}。$

从变换后的点计算预测测量均值 $z_{1|0}$ 、协方差 $\mathbf{P}_{zz}$ 和互协方差 $\mathbf{P}_{xz}$ ，进而计算卡尔曼增益和状态修正。

追踪结果对比

t	真实 x	EKF x	UKF x	EKF v_x	UKF v_x
0	100	95.0	95.0	8.0	8.0
1	110	107.2	107.4	9.1	9.2
2	120	118.5	118.7	9.8	9.9
3	130	129.1	129.2	10.0	10.0
4	140	139.8	139.9	10.1	10.0
5	150	150.1	150.0	10.0	10.0

表 8 EKF 与 UKF 追踪结果对比

在这个弱非线性场景下，EKF 与 UKF 的结果非常接近。这是因为雷达观测函数虽然是非线性的，但在估计点附近的非线性程度并不强，一阶线性化已经足够准确。

两种方法的差异在强非线性场景下会更加明显。例如，当目标靠近雷达站（ r 较小）时，角度测量的非线性显著增强，此时 UKF 的优势会更加突出。

2.6.7.5. EKF 与 UKF 的对比

两种方法代表了处理非线性的两种不同哲学：EKF 近似函数，UKF 近似分布。下表总结了它们的主要区别。

	EKF	UKF
核心思想	线性化非线性函数	采样传播概率分布
近似对象	函数（泰勒展开）	分布（Sigma 点）
精度	一阶	二阶
雅可比矩阵	需要解析求导	不需要
计算量	较低	较高（ $2n + 1$ 次函数求值）
实现难度	需推导雅可比	仅需实现系统函数
数值稳定性	一般	较好
适用场景	弱非线性、雅可比易求	强非线性、雅可比难求

在实际选择时, 有几条经验法则。如果系统的非线性程度较弱, 且雅可比矩阵容易求解, EKF 是更简单高效的选择。如果非线性较强, 或者雅可比矩阵难以解析求导 (例如系统模型来自仿真器或神经网络), UKF 是更稳健的选择。当状态维度很高 ($n > 100$) 时, UKF 的 $2n + 1$ 个 Sigma 点会带来显著的计算负担, 此时可能需要考虑其他方法, 如粒子滤波。

2.6.7.6. 参数选择指南

UKF 的性能受 α 、 β 、 κ 三个参数影响。以下是常用的选择策略。

参数 α 控制 Sigma 点的分散程度。较小的 α (如 10^{-3}) 使 Sigma 点紧密围绕均值, 适合近似局部非线性; 较大的 α (如 1) 使 Sigma 点分散更广, 能够捕捉更大范围的非线性效应, 但可能引入高阶误差。实践中 $\alpha = 0.001$ 到 0.01 是常见选择。

参数 β 融入分布的先验信息。对于高斯分布, $\beta = 2$ 是理论最优值, 能够精确捕捉四阶矩。若分布明显偏离高斯, 可以调整 β , 但通常影响不大。

参数 κ 是次要的缩放参数。常见选择是 $\kappa = 0$ (简化计算) 或 $\kappa = 3 - n$ (保证正定性)。当 $n + \lambda$ 接近零或为负时, 可能导致数值问题, 此时需要调整 κ 。

对于大多数应用, 默认参数 $\alpha = 0.001$, $\beta = 2$, $\kappa = 0$ 是合理的起点, 可根据实际效果微调。

2.6.7.7. 本章小结

无迹卡尔曼滤波提供了一种不同于 EKF 的非线性处理思路。它的核心是无迹变换: 通过精心选取的 Sigma 点代表概率分布, 将这些点通过真实的非线性函数变换, 再从变换后的点恢复统计特性。这种方法避免了雅可比矩阵的计算, 且在理论上具有更高的近似精度。

UKF 的主要优势包括: 不需要解析求导, 实现更简单; 二阶精度优于 EKF 的一阶精度; 对强非线性系统更稳健。主要代价是计算量较大, 需要 $2n + 1$ 次函数求值。

至此, 我们已经完成了从递推平均到 UKF 的完整旅程。下表回顾了各方法的演进脉络:

方法	解决的问题	核心思想
递推平均	静态量估计	历史数据等权平均
g-h 滤波器	动态系统追踪	预测 + 修正 (固定增益)
卡尔曼滤波	最优增益选取	基于方差最小化动态计算增益
多维 KF	多状态关联估计	协方差矩阵描述状态间关联

EKF	非线性系统	局部线性化（雅可比矩阵）
UKF	强非线性系统	采样传播（Sigma 点）

每一步演进都是为了解决前一方法的局限：递推平均无法追踪动态系统，g-h 滤波器的增益需要手动设定，标准卡尔曼滤波仅适用于线性系统，EKF 在强非线性时精度下降。理解这一演进逻辑，比记住公式本身更为重要——它揭示了工程方法论的核心：从具体问题出发，逐步放松假设，构建更普适的解决方案。

3. 实战技术篇

3.1. 通信协议设定

3.2. 相机标定与手眼标定

3.3. 时间戳对齐

3.4. 弹道解算

4. RoboMaster 应用篇

4.1. 工业相机

4.2. 装甲板识别

4.3. 装甲板的跟踪

4.4. 能量机关识别

4.5. 自瞄算法设计

4.6. 串口模块

5. 进阶篇

5.1. 雷达站视觉方案

5.2. 哨兵决策视觉

5.3. 性能优化技巧

5.4. 实战经验与坑点总结

6. 项目分析

6.1. rm_vision

6.2. rm.cv.fans

6.3. 同济自瞄

Index of Tables

表 1 递推平均追踪动态目标的失效	545
表 2 g-h 滤波器追踪结果	546
表 3 同一数据下两种方法的对比	546
表 4 一维卡尔曼滤波追踪赛车	551
表 5 多维卡尔曼滤波追踪结果	556
表 6 雷达追踪：真实轨迹与测量值（角度单位：rad）	561
表 7 EKF 追踪结果（部分状态）	562
表 8 EKF 与 UKF 追踪结果对比	567